

Git & GitHub: From Zero to Mastery

A Complete Beginner-to-Expert Guide to Version Control,
Collaboration, Automation, Security, and Modern Software
Development

Written by Sharif Issah Tingane

SHARIF TECHNOLOGIES

Knowledge Is Power

1st Edition

Copyright and Notices

Copyright © 2026 SHARIF TECHNOLOGIES. All rights reserved except for the permissions expressly granted under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International Public License.

This book's original text is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0):

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Under this license, readers may share and adapt the licensed book material for noncommercial purposes, provided that the applicable attribution and ShareAlike requirements are followed. Commercial use requires separate permission from the rights holder.

Code examples contained in this book are separately licensed under the MIT License unless a particular example identifies another applicable licence or third-party source.

The licence applies only to material for which the stated rights holder has authority to grant the licence. Third-party material, trademarks, screenshots, quotations, logos, and other material identified as belonging to their respective owners are not automatically covered by this licence and may be subject to separate rights or permissions.

This licensing notice is a plain-language summary and does not replace the terms of the applicable Creative Commons or MIT licence.

AI assistance

This book was written with the help of an AI assistant, Claude, made by Anthropic and used through Claude Code, working with the author, Sharif Issah Tingane, who set the goals and rules and is responsible for the book. The assistant drafted the text, exercises, solutions and code, ran and recorded the commands, did the research and checking, built the editions and drew the draft cover. The Preface and Appendix N give the details; the repository's history credits the assistant as contributor and co-author.

What the licence does and does not mean

The book is copyrighted. It is not "copyright-free", not in the public domain and not owned by everyone. Making a book free to read or to download does not remove copyright; a licence is a set of permissions that the rights holder grants. Keep these questions apart, as Chapter 65 teaches: who owns the work; whether you may read it, download it, redistribute it, modify it, use it commercially, or reuse its code. For this book the answers are: the rights holder is named above; you may read and download it; you may redistribute and adapt the text for noncommercial purposes under the conditions of the licence; commercial use needs permission; and the code examples may be reused under the MIT License.

Title	Git & GitHub: From Zero to Mastery
Author	Sharif Issah Tingane

Published by	Sharif Issah Tingane, under the SHARIF TECHNOLOGIES imprint
Edition	1st Edition, electronic book
ISBN	Not yet assigned
First published	2026
Technical information	Checked in September 2026; Appendix N says what was checked and what remains open
Website	www.shariftechnologies.online

Disclaimer

This book is educational. Commands change the state of your computer and your repositories: understand a command before you run it, use a practice folder, and make backups. Operations involving secrets, history rewriting and access control need particular care. Behaviour can differ between operating systems, versions, configurations and future releases of Git and GitHub. The book was tested on the versions named in Appendix N; the platform's own current documentation prevails. **Nothing in this book is legal advice**, and the chapter on licences explains how licences are described, not what they mean for your situation.

Trademarks and non-affiliation

This book is an independent educational work by Sharif Issah Tingane, issued under the SHARIF TECHNOLOGIES name. It is not affiliated with, endorsed by, sponsored by, or published by Git, GitHub, Microsoft, or any other third party mentioned. Git, GitHub, Microsoft, Linux, Windows, macOS, Docker, GitLab, Bitbucket and other names may be trademarks of their respective owners, and this book claims no rights in them. The name SHARIF TECHNOLOGIES is not licensed by the licences above.

Third-party material

Where the book quotes short passages from official documentation, it names the source; those passages remain under their own terms and are not covered by this book's licence. Names, prices and addresses in examples are made up.

The PDF editions are set in the DejaVu fonts. Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is a trademark of Bitstream, Inc. DejaVu changes are in the public domain.

Contents

Copyright and Notices	2
Preface	9
How to Use This Book	11
Part I — Computers and Digital Projects	14
Chapter 1 — What a Computer Is [Core].....	15
Chapter 2 — Files, Folders and Paths [Core]	26
Chapter 3 — Editors and Project Folders [Core].....	37
Chapter 4 — Text, Encodings and Newlines [Core].....	47
Chapter 5 — The Internet, the Web and Servers [Core]	57
Chapter 6 — Accounts, Passwords and Trust [Core].....	67
Chapter 7 — The Terminal Without Fear [Core].....	76
Chapter 8 — Markdown in Five Pages [Core]	93
Part II — Understanding Version Control	106
Chapter 9 — The Problem of Changing Files [Core]	107
Chapter 10 — A Short History of Version Control [Core].....	115
Chapter 11 — Centralised vs Distributed [Core].....	121
Chapter 12 — Git, GitHub, GitLab, Bitbucket [Core]	128
Part III — Learning Git.....	135
Chapter 13 — Installing Git [Core].....	136
Chapter 14 — Configuration Scopes and Environments [Core]	145
Chapter 15 — Git's Core Model [Core].....	157
Chapter 16 — Your First Repository [Core].....	168
Chapter 17 — Seeing Changes and History [Core]	180
Chapter 18 — Tracking, Ignoring, Renaming, Deleting [Core]	193
Chapter 19 — Writing Good Commits [Core]	208
Chapter 20 — Branching [Core]	219
Chapter 21 — Merging [Core]	232
Chapter 22 — Merge Conflicts [Core]	243
Chapter 23 — Remotes [Core].....	253
Chapter 24 — Stash and Small Tools [Core].....	265
Part IV — Professional Git	275
Chapter 25 — Undoing Things Safely [Core]	276
Chapter 26 — Recovery with the Reflog [Core].....	289
Chapter 27 — Rebase [Deep].....	297
Chapter 28 — Cherry-pick, Bisect and Patches [Deep]	311
Chapter 29 — Tags and Versioning [Core]	323
Chapter 30 — Inside Git: The Object Model [Deep]	331
Chapter 31 — Big and Complex Repositories [Deep]	343
Chapter 32 — Customising Git [Deep]	356

Chapter 33 — Git Security [Core].....	367
Chapter 34 — Git Workflows [Core]	376
Chapter 35 — Reading Official Documentation [Core].....	384
Part V — Introduction to GitHub	391
Chapter 36 — What GitHub Is (and Is Not) [Core].....	392
Chapter 37 — Account and Profile [Core]	399
Chapter 38 — Authentication in Depth [Core]	405
Chapter 39 — Your First GitHub Repository [Core]	414
Chapter 40 — Touring the GitHub Interface [Core]	423
Chapter 41 — Notifications, Stars and Watching [Core]	429
Part VI — Working with GitHub	435
Chapter 42 — Professional Repositories and READMEs [Core].....	436
Chapter 43 — Repository Settings, Security Tab and Wiki Tour [Core].....	443
Chapter 44 — Issues [Core].....	449
Chapter 45 — Pull Requests [Core].....	456
Chapter 46 — Code Review [Core]	465
Chapter 47 — GitHub Projects [Deep].....	471
Chapter 48 — GitHub Discussions [Deep].....	477
Chapter 49 — Collaboration and Access [Core]	482
Chapter 50 — Branch Protection and Rulesets [Deep]	489
Chapter 51 — Insights [Deep]	495
Part VII — Automation and CI/CD	501
Chapter 52 — CI/CD Concepts [Core]	502
Chapter 53 — YAML for Beginners [Core]	508
Chapter 54 — GitHub Actions Fundamentals [Core].....	514
Chapter 55 — Expressions, Contexts and Conditionals [Deep].....	521
Chapter 56 — Practical Workflows [Core].....	528
Chapter 57 — Debugging and Re-running Workflows [Core].....	536
Chapter 58 — Reusable Workflows, Environments and Deployments [Deep].....	542
Chapter 59 — Self-Hosted Runners, Concurrency and Limits [Deep].....	549
Chapter 60 — Securing Workflows [Core].....	555
Chapter 61 — Actions vs External CI [Deep].....	562
Part VIII — Security on GitHub.....	567
Chapter 62 — GitHub Security Features [Core].....	568
Chapter 63 — Repository Security Practice [Core]	575
Part IX — Open Source and Professional Development	581
Chapter 64 — How Open Source Works [Core].....	582
Chapter 65 — Copyright and Licences [Core].....	588
Chapter 66 — Releases [Core].....	595
Chapter 67 — Creating an Open-Source Project [Deep]	602
Part X — Advanced GitHub	609
Chapter 68 — Organizations and Enterprise Concepts [Deep]	610

Chapter 69 — GitHub Pages [Deep]	616
Chapter 70 — Codespaces and Dev Containers [Deep].....	622
Chapter 71 — GitHub Packages and Containers [Deep]	629
Chapter 72 — Deployment Pipelines [Deep]	635
Chapter 73 — GitHub CLI [Deep].....	641
Chapter 74 — APIs, Webhooks and Apps [Deep].....	648
Chapter 75 — Other Platform Features [Deep]	656
Part XI — Master Projects	663
Chapter 76 — Professional Scenarios 1-12 [Core]	664
Chapter 77 — Capstone Project [Core]	676
Part XII — Troubleshooting and Recovery	690
Chapter 78 — Troubleshooting Handbook [Core]	691
Chapter 79 — Recovery Playbooks [Core].....	705
Part XIII — Mastery	714
Chapter 80 — Advanced Challenges [Deep].....	715
Chapter 81 — Final Mastery Assessment [Core].....	721
Appendix A — Git Command Reference	726
Appendix B — GitHub and CLI Reference	730
Appendix C — Common Git Errors	733
Appendix D — GitHub Actions YAML Reference	735
Appendix E — Git Terminology Map.....	738
Appendix F — GitHub Terminology Map.....	747
Appendix G — Security Checklist	752
Appendix H — Professional Repository Checklist	754
Appendix I — Open-Source Project Checklist	756
Appendix J — Deployment Checklist.....	757
Appendix K — Git Recovery Decision Tree	758
Appendix L — Learning Roadmap After the Book	760
Appendix M — Git Bash / PowerShell / Command Prompt Equivalence Guide	762
Appendix N — Sources and Verification Log	764
Solutions to the Exercises	770
Glossary.....	836
Index of commands	879
Index of terms.....	873
Author and Publisher	881

List of figures8

List of figures

Figure 1. 1.2 Hardware: the parts you can touch (What a Computer Is [Core]).....	16
Figure 2. 1.3.1 The operating system (What a Computer Is [Core])	19
Figure 3. 2.2 Folders (Files, Folders and Paths [Core]).....	28
Figure 4. 2.3.2 Relative paths: directions from where you are (Files, Folders and Paths [Core]).....	30
Figure 5. 5.3 Client and server (The Internet, the Web and Servers [Core])	60
Figure 6. 6.4 Authentication and authorization (Accounts, Passwords and Trust [Core])	70
Figure 7. 6.7 Keys: proving identity without sending a secret (Accounts, Passwords and Trust [Core])	71
Figure 8. 11.2 Design one: centralised (Centralised vs Distributed [Core])	122
Figure 9. 11.3 Design two: distributed (Centralised vs Distributed [Core])	123
Figure 10. 15.2 Three places (Git's Core Model [Core])	157
Figure 11. 15.4 The commit (Git's Core Model [Core])	162
Figure 12. 15.5 Pointers: HEAD and branches (Git's Core Model [Core])	163
Figure 13. 15.7 The whole picture (Git's Core Model [Core])	164
Figure 14. 20.1 The problem branches solve (Branching [Core])	219

Preface

Most books on Git and GitHub tell you what to type. This one tries to give you a way to know that what you type works, and to know how much to trust what it says.

Who this book is for

For someone who has never used version control, and perhaps has never used a terminal. It starts with what a computer is, what a file is and what a path is, and ends with recovery playbooks, security, automation and a final assessment. If you already use Git, the chapters are still useful as a check of your understanding, and the **Deep** chapters go further.

How this book was checked

The book follows a small set of rules, written down before the chapters:

1. **Commands are run, not remembered.** Every command shown was executed in a sandbox, in Bash and in zsh, and the output in the book is the recorded output. A machine check compares each recorded line in the text with the recording, and the recordings are re-run in continuous integration on newer versions of Git.
2. **Versions and dates are recorded.** Git behaviour is tied to the version tested; GitHub behaviour to the date and the documentation it was read from. Nothing is generalised beyond that.
3. **Official sources first.** Git behaviour was compared with Git's own documentation and source; GitHub behaviour with GitHub's documentation. Summaries by third parties are not substitutes, and when only one was available the text says so.
4. **Every claim has an evidence class:** officially verified, locally tested, both, time-sensitive, or needing re-verification. A locally tested claim is never called officially verified.
5. **Surprises are investigated before they are taught.** Where Git's behaviour and its manual disagreed, the book followed the behaviour and said so.
6. **Interface is taught as an idea first.** Screens change; principles last.
7. **Nothing is invented.** Where a fact was not supplied or could not be verified (an ISBN, a publication date, a contact address), the book leaves a visible placeholder. The author's biography states only what public evidence supports.

What this book did not do. Nothing was run on a live GitHub account by hand: statements about GitHub come from its documentation, and every chapter says so. Windows and macOS environments were not available for testing. Appendix N lists exactly what was checked, with what, and what is still open.

How this book was made, and the part AI played

This book was written with an AI assistant. The assistant is Claude, made by Anthropic and used through Claude Code, working in sessions with the author, Sharif Issah Tingane. The author set the goals, the scope and the rules that the book follows (run the commands, label the evidence, invent nothing), chose the licences and the imprint name, approved the plan, and is responsible for the book.

The assistant did the work of **every area** of the book: it drafted the chapters, the exercises, the solutions, the glossary and the appendices; wrote the code samples and the scripts; ran the commands in a sandbox and recorded the output printed here; searched and read the official sources and kept the research ledger; read the whole book for errors; built the PDF and EPUB editions and the automatic checks; drew the draft cover and the diagrams; and drafted the author information and the licensing pages. It also made most of the commits in the project's repository, which therefore lists Claude as a contributor and co-author.

What that means for you: the recorded output is real output from a sandbox, not from a live GitHub account, Windows or macOS, and the book says so where it matters. An AI assistant can be wrong, which is one reason the book labels how each claim was checked and lists what could not be checked (Appendix N). The full statement, area by area, is in the file `AI-ASSISTANCE.md` of the companion repository.

A word on trust

Use the book the way it asks you to use everything else: run the commands, read the output, look at the date and compare with the current documentation. If you find that something differs, trust the source and let the book's ledger tell you where its claim came from.

Sharif Issah Tingane

How to Use This Book

The book has 81 chapters in 13 parts, 14 appendices, a glossary and an index. Chapters are tagged **[Core]** (58 of them) or **[Deep]** (23 of them).

Core and Deep

- **[Core]** chapters are needed to progress. If you skip one, later chapters may not make sense.
- **[Deep]** chapters matter for mastery but not for a first pass. You can skip a Deep chapter (or a section tagged Deep inside a Core chapter) and return to it later. No Core chapter depends on a Deep chapter.

Three ways to read

1. **First-Read Path:** the shortest coherent route from no knowledge to using Git and GitHub for real work (below).
2. **Second pass:** the remaining Core chapters.
3. **Mastery pass:** all Deep chapters, then the challenges and the assessment.

The First-Read Path

Read these 38 chapters in full, and the sections named in the next table.

Order	Chapter	Tag
1	Chapter 1. What a Computer Is	Core
2	Chapter 2. Files, Folders and Paths	Core
3	Chapter 3. Editors and Project Folders	Core
4	Chapter 4. Text, Encodings and Newlines	Core
5	Chapter 5. The Internet, the Web and Servers	Core
6	Chapter 6. Accounts, Passwords and Trust	Core
7	Chapter 7. The Terminal Without Fear	Core
8	Chapter 8. Markdown in Five Pages	Core
9	Chapter 9. The Problem of Changing Files	Core
10	Chapter 11. Centralised vs Distributed	Core
11	Chapter 12. Git, GitHub, GitLab, Bitbucket	Core
12	Chapter 13. Installing Git	Core
13	Chapter 14. Configuration Scopes and Environments	Core
14	Chapter 15. Git's Core Model	Core
15	Chapter 16. Your First Repository	Core

Order	Chapter	Tag
16	Chapter 17. Seeing Changes and History	Core
17	Chapter 18. Tracking, Ignoring, Renaming, Deleting	Core
18	Chapter 19. Writing Good Commits	Core
19	Chapter 20. Branching	Core
20	Chapter 21. Merging	Core
21	Chapter 22. Merge Conflicts	Core
22	Chapter 23. Remotes	Core
23	Chapter 25. Undoing Things Safely	Core
24	Chapter 26. Recovery with the Reflog	Core
25	Chapter 36. What GitHub Is (and Is Not)	Core
26	Chapter 37. Account and Profile	Core
27	Chapter 38. Authentication in Depth	Core
28	Chapter 39. Your First GitHub Repository	Core
29	Chapter 40. Touring the GitHub Interface	Core
30	Chapter 42. Professional Repositories and READMEs	Core
31	Chapter 44. Issues	Core
32	Chapter 45. Pull Requests	Core
33	Chapter 46. Code Review	Core
34	Chapter 52. CI/CD Concepts	Core
35	Chapter 53. YAML for Beginners	Core
36	Chapter 54. GitHub Actions Fundamentals	Core
37	Chapter 56. Practical Workflows	Core
38	Chapter 57. Debugging and Re-running Workflows	Core

Read the first sections of these Core chapters on a first pass, and return later for the rest:

Chapter	Read first
Chapter 24. Stash and Small Tools	the opening sections and the summary
Chapter 29. Tags and Versioning	the opening sections and the summary
Chapter 33. Git Security	the opening sections and the summary
Chapter 34. Git Workflows	the opening sections and the summary
Chapter 35. Reading Official Documentation	the opening sections and the summary
Chapter 41. Notifications, Stars and Watching	the opening sections and the summary

Chapter	Read first
Chapter 43. Repository Settings, Security Tab and Wiki Tour	the opening sections and the summary
Chapter 49. Collaboration and Access	the opening sections and the summary
Chapter 60. Securing Workflows	the opening sections and the summary
Chapter 62. GitHub Security Features	the opening sections and the summary
Chapter 63. Repository Security Practice	the opening sections and the summary
Chapter 64. How Open Source Works	the opening sections and the summary
Chapter 65. Copyright and Licences	the opening sections and the summary
Chapter 66. Releases	the opening sections and the summary

Conventions

- **Commands** appear in code blocks. A line that starts with \$ is a command you type (without the \$); the lines below it are the recorded output. Names, dates and hashes in the output differ on your computer.
- **Bash and zsh.** The book uses one path: Bash or zsh syntax. On Windows, use **Git Bash**, which is installed with Git for Windows. Git is not Git Bash: Git is the tool, Git Bash is a shell. Appendix M compares the shells.
- **Callouts.** New term introduces a word; Caution marks something that can lose work; Security note marks a security lesson; Checked against... says which source a claim was compared with.
- **Practice.** Each chapter ends with a checkpoint, and has exercises in exercises/ and solutions in solutions/. Try before you read the solutions.
- **Projects.** Twelve projects are spread through the book; the last is the capstone.
- **A practice folder.** Do everything in a folder you can throw away.

The companion repository

The recorded command sessions, the research ledger, the verification scripts, the exercises and the files the chapters ask you to download are published in the book's companion repository: <https://github.com/Sharifwa123/Books> (folder git-and-github-mastery). The text of the book says when it names a file from there.

What you need

A computer (Windows, macOS or Linux), a text editor, a terminal, an internet connection for the later parts, and a GitHub account from Part V onwards. Chapter 13 installs Git. Some tools of later chapters are installed when needed.

If you are short of time

Read the First-Read Path, do its projects, and run the assessment (Chapter 81). Then read the rest as you need it.

Part I — Computers and Digital Projects

Chapter 1 — What a Computer Is [Core]

In this chapter

- what a computer actually does, in one sentence you can remember
- the physical parts (hardware) and why memory and storage are different things
- the invisible layer (software), and what an operating system is for
- what "installing" a program means, and how to do it safely
- why all of this matters before we meet Git

Before you start. Nothing. This book assumes that you have never used a terminal, never written code and never heard the word "repository". If you already know some of this chapter, read it quickly and use the self-test at the end to check yourself.

You will not be asked to memorise anything in this chapter. You will be asked to understand a few ideas, because every later chapter stands on them.

1.1 A computer follows instructions

A **computer** is a machine that follows instructions. That sentence is the most important one in this chapter, so let us unpack it.

New term: computer. A machine that accepts information (input), follows a list of instructions to change it (processing), keeps the results (storage), and shows them to you (output).

Everything a computer does fits that pattern.

Stage	What happens	Everyday example
Input	Something goes in	You type a letter, tap a screen, or speak
Processing	Instructions change the information	The computer turns key presses into letters on a page
Storage	The result is kept	You save the page as a file
Output	The result comes out	The letters appear on the screen, or a printer prints them

Your phone is a computer. So is a laptop, a cash machine, a modern car's dashboard and the box that runs the traffic lights. They look different, but each follows instructions.

Why this matters for Git. Git is a set of instructions (a program) that runs on a computer. It takes the files you have made as input, keeps a careful record of how they change, and shows you that record as output. If you understand what a computer does, you will always know where Git fits.

1.2 Hardware: the parts you can touch

New term: hardware. The physical parts of a computer: the parts you could, in principle, drop on your foot.

Different computers have different hardware, but a few parts appear almost everywhere. To make them easy to remember, imagine a small kitchen.

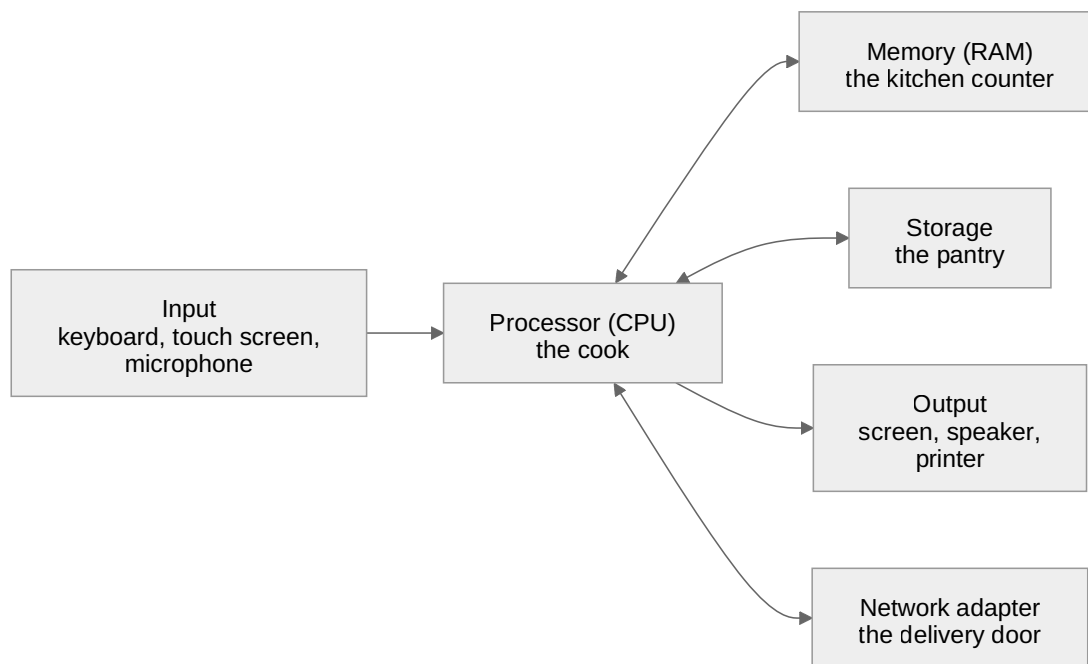


Figure 1. 1.2 Hardware: the parts you can touch

Diagram description: a processor sits in the middle. Input devices send information to it. It exchanges information with memory (a fast, temporary workspace) and with storage (a slower, permanent place to keep things). It sends results to output devices such as a screen, and it talks to other computers through a network adapter.

Part	What it does	In the kitchen
Processor (also called the CPU , "central processing unit")	Carries out instructions, one tiny step at a time, extremely fast	The cook, following a recipe
Memory (also called RAM , "random-access memory")	A fast workspace for whatever the computer is using right now	The counter where ingredients are laid out while cooking
Storage (a solid-state drive, hard disk, or the flash storage in a phone)	Keeps information for a long time, even when the power is off	The pantry and the fridge
Input devices	Send information in	The order slip
Output devices	Show or print results	The plate that leaves the kitchen

Part	What it does	In the kitchen
Network adapter	Connects to other computers (Wi-Fi or a cable)	The delivery door

New term: processor (CPU). The part of a computer that carries out instructions.

New term: memory (RAM). Fast, temporary working space. It is emptied when the computer is switched off.

New term: storage. Slower but permanent space that keeps information when the power is off.

1.2.1 The difference that will save your work: memory versus storage

Beginners often use the words "memory" and "storage" as if they meant the same thing. They do not, and confusing them causes real loss of work.

- **Memory** is the counter. It is quick, but when the power goes, it is wiped clean.
- **Storage** is the pantry. It is slower, but it keeps things.

When you type a letter in a program and have not yet saved it, the letter lives only in memory. If the battery dies, the letter is gone. When you save, the program copies the letter from memory into storage, and now it survives.

Try it. Think of the last time you lost something you were typing. Was it saved? Was it in memory or in storage when the computer switched off?

Expected result: you can describe the loss in terms of "it was only in memory".

Why this matters for Git. Saving a file to storage is the first step of keeping your work. Git goes one step further: it keeps a history of your saved work, so that even a mistake you saved can be undone. You will see exactly how in Part III. For now, remember the idea: saved is safer than unsaved, and recorded in history is safer still.

1.2.2 Bits and bytes, briefly

Computers store everything as numbers, and they write every number using only two symbols, 0 and 1. One of these symbols is a **bit**. A group of eight bits is a **byte**. A byte can hold one letter of ordinary English text.

New term: byte. A unit of storage. One byte is eight bits. Sizes of files are counted in bytes, and in larger units built from bytes: a kilobyte is about a thousand bytes, a megabyte about a million, and a gigabyte about a billion.

You do not need to calculate with bytes. You need to know that files have sizes, that the sizes are counted in bytes, and that a photo is much bigger than a paragraph of text. In Chapter 4 you will look at real bytes and see how letters are stored.

Deep. Two competing conventions exist for the larger units. Some tools count a kilobyte as 1,000 bytes; others count 1,024. When the difference matters, tools usually say which they use. It rarely matters for this book.

1.3 Software: the instructions

New term: software. Instructions that tell hardware what to do. Software has no weight or shape; it is information.

The hardware alone can do nothing useful, just as a kitchen with no recipes and no cook does nothing. Software supplies the instructions. There are two kinds you must know about.

1. **Applications** (also called **programs**, **apps** or **software packages**): software that does a job for you. A web browser (the program you use to look at web pages), a photo editor, a game and Git are all applications.
2. **The operating system**: the software that manages the computer itself and lets the applications run.

New term: application (program). Software that does a specific job for a person: browsing, writing, calculating, or tracking file history.

1.3.1 The operating system

New term: operating system (OS). The main software of a computer. It starts when the computer starts, manages the hardware, keeps track of files, and gives every application a safe place to run.

An operating system does three jobs that you will see in action again and again:

1. **It manages the hardware for everyone.** Two applications cannot both use the screen or the storage at the same instant without a referee. The operating system is the referee.
2. **It keeps the files.** It decides where information is stored and lets you find it by name.
3. **It is the translator.** An application does not talk directly to a particular brand of storage or screen. It asks the operating system, and the operating system talks to the hardware.

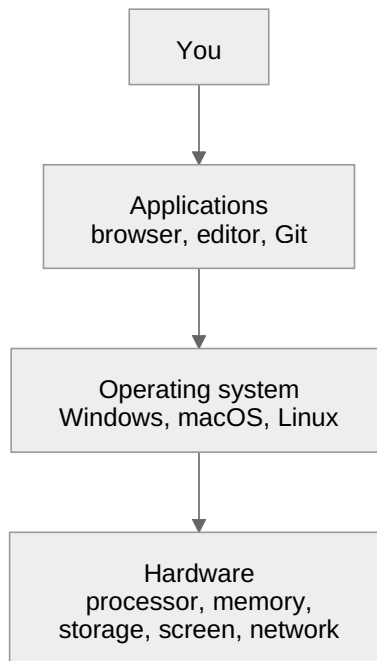


Figure 2. 1.3.1 The operating system

Diagram description: three layers from top to bottom. You use applications. Applications rely on the operating system. The operating system controls the hardware. Requests flow downward; results flow back up.

The translator idea explains something that will matter later. Because applications sit on top of an operating system, the same application may have to be prepared separately for each operating system. Git exists for several operating systems, and, importantly, the way you **start** and **use** it differs slightly between them. When it does, this book will tell you.

1.3.2 Windows, macOS and Linux

You will meet three families of operating system in this book. Here is what to know about each at the level you need.

Family	Who makes it	Where you usually meet it	What to know
Windows	Microsoft	Many desktop and laptop computers	Uses drive letters such as C: and, in its traditional command tools, the backslash \ in file locations
macOS	Apple	Apple's laptop and desktop computers	Has a single tree of folders; behaves much like Linux in the terminal
Linux	An open community; many variants	Servers, developer machines, many other devices	A family of systems, each variant being called a distribution

Verification pending [R111]. Product-level statements (who makes what, and how macOS and Linux relate) are given here from general knowledge. They will be re-checked against the vendors' and the Linux community's own documentation before publication. Nothing later in this book depends on a detail of them.

You do not need to choose between them now. The book teaches on all three, with one primary environment: a program called a terminal running a shell named **Bash** (Chapter 7 explains what that means). On Windows the book uses **Git Bash**, so that all learners see the same commands.

1.4 Applications and installing

Most applications are not in your computer when you buy it. You add them. Adding an application is called **installing** it.

New term: installer. A file or program whose job is to put an application in the right places on your computer and to register it with the operating system.

The usual steps are:

1. **Obtain** the installer from a source you trust.
2. **Run** the installer.
3. The installer may ask for **administrator permission**, because installing changes parts of the system that ordinary use should not change.
4. **Check** that it worked by opening the application.

New term: administrator. A user account, or a temporary level of permission, that is allowed to change the computer's system settings and install software for everyone who uses it.

1.4.1 Package managers

Some systems also offer a **package manager**, a program that finds, installs and updates applications for you from a curated collection, so you do not have to browse websites.

New term: package manager. A program that installs, updates and removes applications from a trusted collection.

For example, on Ubuntu and other Debian-family Linux systems, the package manager is called `apt`. The command that installs the shell called `zsh` (which the book's test environment uses) looks like this. You do not need to run it.

```
sudo apt-get install -y zsh
```

Other systems have their own equivalents, and Chapter 13 gives the steps for installing Git on each system and states how each set of steps was checked. The idea is the same everywhere: ask a trusted collection for the program by name.

1.4.2 Installing safely

A program you install can do anything you can do. That makes installation the most common way for harm to arrive. Build the following habits now, because Git will ask you to build the same ones about code later.

Security note. Install software only from the maker's own website, from your operating system's official store or package manager, or from a source your teacher or employer names. Be suspicious of a download link in an email, a "free" copy of a paid program, or a pop-up that says your computer is infected. If you are asked for administrator permission and you did not just start an installation, refuse.

Three more habits:

- **Prefer the official source.** Search for the maker's name, then check that the address you land on belongs to them.
- **Read what you are agreeing to.** At minimum, look at what is being installed and where.
- **Keep software updated.** Updates often repair security problems. Many applications update themselves; some do not.

When not to install. If you only need a program once, consider whether a web version would do. Every installed program is something to keep updated and something that could be attacked.

1.5 Where Git fits

Now you have every piece you need for a first, simple picture of Git.

- Git is an **application**, that is, software.
- It runs on your **operating system**, on your **hardware**.
- It reads the **files** you have saved in **storage**, and it stores its record of their history in storage too.
- It runs only when you ask it to (you will do so through the terminal in Chapter 7).

Nothing about Git is magical. It is a program that follows instructions, and its results are stored in ordinary files that you will learn to look at.

Checkpoint

What You Learned

- A computer follows instructions: input, processing, storage, output.
- Hardware is what you can touch. The processor carries out instructions, memory is fast and temporary, storage is slower and permanent.
- Saved work is in storage; unsaved work exists only in memory and disappears when the power goes.

- Software is the instructions. Applications do jobs for you; the operating system manages the computer and its files.
- Windows, macOS and Linux are three families of operating system; a program may work slightly differently on each.
- An installer puts an application on your computer; a package manager installs from a trusted collection; installing can require administrator permission.
- Install only from sources you trust.

New Vocabulary

- **Computer:** a machine that follows instructions to accept, change, keep and show information.
- **Hardware:** the physical parts of a computer.
- **Processor (CPU):** the part that carries out instructions.
- **Memory (RAM):** fast, temporary working space.
- **Storage:** slower, permanent space that survives power-off.
- **Byte:** a unit of storage; eight bits.
- **Software:** instructions that tell hardware what to do.
- **Application (program):** software that does a specific job for a person.
- **Operating system (OS):** the main software that manages hardware and files and lets applications run.
- **Installer:** a file or program that puts an application on your computer.
- **Administrator:** an account or permission level that can change system settings and install software.
- **Package manager:** a program that installs and updates applications from a trusted collection.

Commands Learned

None yet. (The one command shown, `sudo apt-get install -y zsh`, was an illustration of a package manager and you were not asked to run it.)

Common Mistakes

1. **Treating "memory" and "storage" as the same thing.** Unsaved work is in memory and can be lost.
2. **Installing from the first search result or from an email link.** Go to the maker's own site or your system's official source.
3. **Clicking "Allow" out of habit when a box asks for administrator permission.** Only allow it when you just asked for something that needs it.
4. **Assuming every operating system behaves identically.** The differences are small but real, and later chapters point them out.

Practice

Do the exercises in the Exercises section at the end of this chapter. Attempt them before opening the separate solutions file.

Self-Test

1. In one sentence, what does a computer do?
2. You are typing a report and the power fails. Which part of the computer held your unsaved words, and why are they gone?
3. Name the three families of operating system introduced here, and say what a distribution is.
4. What does an operating system do that an application does not?
5. A message says "Your computer is infected! Click to install the cleaner." What are two reasons to refuse?
6. Where does Git fit in the picture from section 1.5?

Before Moving On

You are ready for Chapter 2 if you can:

- [] explain the difference between memory and storage using the kitchen picture
- [] say what an operating system is for, without using the words "operating system" in your answer
- [] find out which operating system your own computer uses (look for a screen named "About" or "System information" in your settings; names vary by system)
- [] describe two safe habits for installing software

How this chapter was checked

Claim	Evidence class	Ledger
General account of computer parts, memory versus storage, software and operating systems	Not applicable (general computing concepts; no product-specific claim)	R110
Who makes Windows, macOS and Linux distributions; how they relate	Needs re-verification against vendor and community documentation	R111
apt is the package manager on Debian-family systems, and <code>sudo apt-get install -y zsh</code> is a valid install command	Locally tested: the command was run in the Ubuntu 24.04 test environment on 2026-09-29	R112

Where this leads

Chapter 2 explains how a computer organises its storage into **files and folders** and how to describe where a file is (its **path**). Chapter 7 shows you how to give commands to the computer by typing. Chapter 13 installs Git, using the ideas about installing that you met in section 1.4.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. None of these exercises can damage your computer.

Level 1 — Guided

1.1 Find your operating system.

1. Open your computer's settings.
2. Look for a page named something like "About", "System" or "System information". (The name and place differ between systems; if you cannot find it, search the settings for the word "About".)
3. Write down: the name of the operating system, and its version.

Expected result: one line such as "Windows, version ...", "macOS, version ..." or "Linux, distribution ..., version ...".

1.2 Name the parts. Copy the table below and fill in the last column with an example from your device.

Part	Job	An example on my device
Input device		
Output device		
Storage		

Level 2 — Partially guided

2.1 Memory or storage? For each situation, say whether the information is in memory or storage, and give a one-sentence reason.

- (a) A photo you took yesterday and have not deleted.
- (b) A sentence you have typed in a document but not saved.
- (c) A game you are playing right now.

2.2 The layers. Draw the three-layer picture (you, applications, operating system, hardware) from section 1.3.1 from memory. Then add one real example to each layer from your own device.

Level 3 — Independent

3.1 Explain to a friend who has never used a computer why "Save" matters. Use the kitchen picture, but do not use the words "memory" or "storage" in your explanation.

Level 4 — Professional scenario

4.1 A colleague asks you to install a free "PDF converter" that they found through an advertisement. It asks for administrator permission. Write down the steps you would take before installing anything, and the two questions you would ask your colleague.

Level 5 — Troubleshooting

5.1 A learner says: "I typed my essay for two hours. The power cut out. I switched the computer back on and the essay is gone, but all my saved photos are fine. My computer must be broken."

- (a) Is the computer broken? Explain what probably happened, using the words memory and storage.

- (b) Give two habits that would prevent the loss next time.

Chapter 2 — Files, Folders and Paths [Core]

In this chapter

- what a file is, and how a name and an extension describe it
- how folders organise files into a tree
- how to say exactly where a file is (a path), in two different ways
- hidden files, and why a file called `.git` will matter soon
- copying, moving and deleting, and which of these can be undone
- permissions, at the level you need to read an error message

Before you start. Chapter 1: you know that storage keeps information when the power is off, and that the operating system looks after files.

2.1 What a file is

New term: file. A named collection of information kept in storage.

A photo is a file. A letter you saved is a file. A program is a file. Even the list of names of your other files is kept in a special kind of file. Whenever you save something, you create or change a file.

Every file has three things you need to think about:

1. **A name:** what you call it.
2. **Contents:** the information inside it (letters, colours, sound, instructions).
3. **A place:** which folder it is in (section 2.3).

Files also carry information about themselves, called **metadata**: how big they are, when they were last changed, who owns them, and what the computer is allowed to do with them. You will meet metadata when you ask the computer to list files in detail.

New term: metadata. Information about a file (its size, dates, owner and permissions), kept separately from what is inside the file.

2.1.1 Names and extensions

File names are chosen by people. Many end with a dot and a few letters: `menu.html`, `logo.svg`, `notes.txt`. That ending is the **extension**.

New term: extension. The letters after the last dot in a file name (for example `.txt`). It is a label that suggests what kind of information is inside.

The important word is *suggests*. The extension does not change what is in the file. If you rename `holiday.jpg` to `holiday.txt`, the photo's information is unchanged; you have only changed the label, and programs that trust the label may now show nonsense.

Extension	Usually means	Chapter where it matters
.txt	plain text (letters and symbols with no formatting)	3 and 4
.md	text written in Markdown	8
.html, .css	the pages and appearance of a website	3
.svg	a picture stored as text	3
.png, .jpg	a picture stored as a compact code (binary)	4
.docx	a word-processor document (a packaged, non-text format)	3
.zip	several files compressed into one	3

UI-VERSION NOTE. Some systems hide extensions by default, so `menu.html` may appear as just `menu`. Look in your file manager's view or folder options for a setting named "file name extensions" (wording varies) and turn it on. You will make far fewer mistakes when you can see the full name.

Naming habits that will help you for the rest of the book.

- Use only lowercase letters, digits and hyphens in names you create for projects: `sunrise-bakery`, `menu.html`.
- Avoid spaces. A space is legal, but in Chapter 7 you will see that spaces force extra care when you type commands.
- Never use the words `final`, `new` or `latest` in a name. Chapter 9 explains why.
- Keep the extension. It tells people, and tools, what to expect.

2.2 Folders

New term: folder (directory). A container that holds files and other folders. "Directory" is the older word, still used by many tools and in this book's command chapters; it means the same thing as folder.

Folders inside folders make a **tree**. Every file lives in exactly one folder. Here is the folder tree of a small website that you will use throughout this book, a fictional business called Sunrise Bakery.

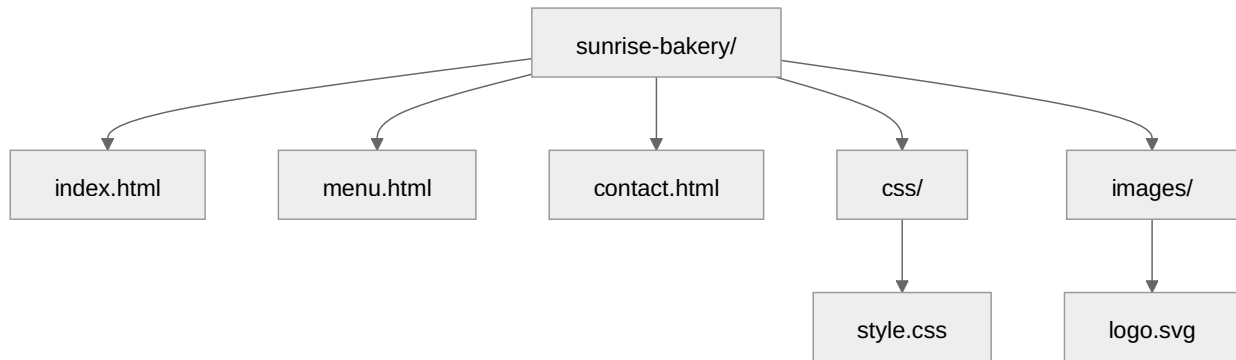


Figure 3. 2.2 Folders

Diagram description: a folder called sunrise-bakery at the top contains three page files, index.html, menu.html and contact.html, and two sub-folders. The css folder contains style.css. The images folder contains logo.svg.

New term: root. The top folder of a tree, from which every other folder can be reached. On macOS and Linux it is written /.

The top of a tree is called the root. The folder that contains another folder is its **parent**; the folder inside is its **child**. On your own computer there is one special folder that belongs to you, called your **home folder**: it is where your documents, downloads and settings normally live.

New term: home folder. The folder that belongs to your user account on a computer, where your own files are normally kept.

2.2.1 Drives

Windows shows storage devices as **drives**, each with a letter: C: is usually the main one. macOS and Linux instead put everything into one tree that begins at a single root written /, and attach other storage devices as folders somewhere inside that tree.

Verification pending [R113]. The statements about drive letters and separators in this section are given from general knowledge and will be re-checked against Microsoft's and Apple's documentation before publication. Nothing later depends on their details, because Git Bash (Chapter 7) presents the same tree-with-slashes view on every system.

2.3 Paths: saying where a file is

To open a file, the computer needs to know exactly where it is. The description of that location is a **path**.

New term: path. A written description of where a file or folder is in the folder tree, listing the folders you pass through, separated by slashes.

A path is like a postal address, or a set of directions. There are two ways to give directions, and mixing them up is one of the commonest beginner errors.

2.3.1 Absolute paths: the full address

New term: absolute path. A path that starts at the root and spells out every folder on the way, so it means the same thing wherever you are standing.

```
/home/learner/sharif-git-lab/sunrise-bakery/menu.html
```

Read it left to right: start at the root /, go into home, then learner, then sharif-git-lab, then sunrise-bakery, and there is the file menu.html. On Windows an absolute path begins with a drive letter, for example C:\Users\learner\sharif-git-lab\sunrise-bakery\menu.html.

2.3.2 Relative paths: directions from where you are

New term: relative path. A path that starts from the current directory instead of the root.

A relative path starts from where you currently are, which the computer calls the **current directory** (also the **working directory**).

New term: current directory (working directory). The folder the computer treats as "here" when you give it a relative path.

Two short forms are worth learning now, because you will type them constantly:

Written	Means
.	"this folder"
..	"the parent folder", one step up

Suppose you are in sunrise-bakery. Then css/style.css is the relative path to the stylesheet. If you are inside the css folder, the same file is just style.css, and ../menu.html is the menu page one step up.

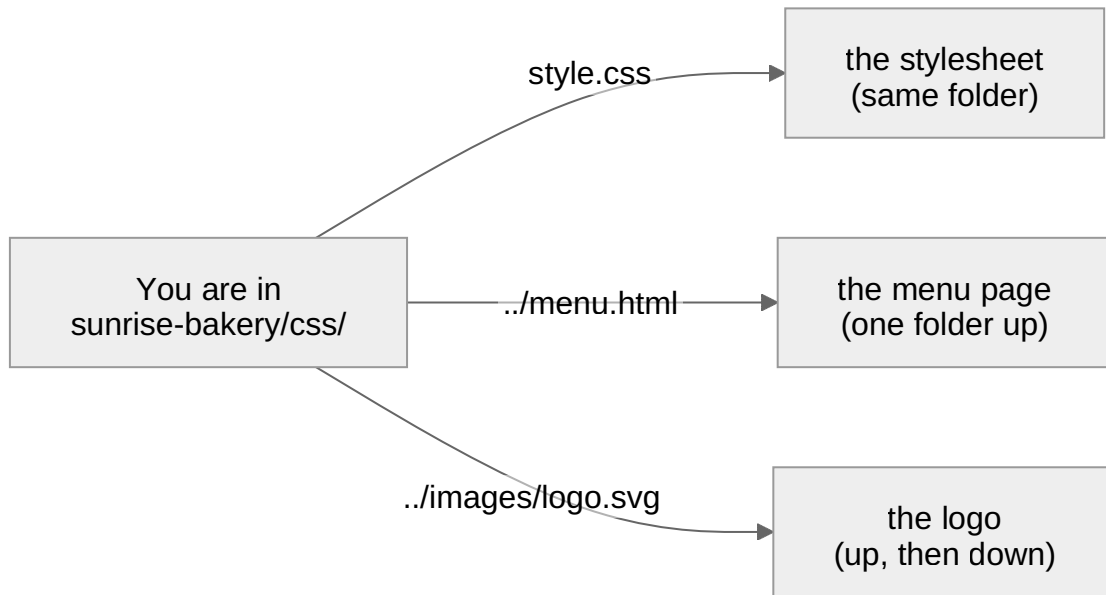


Figure 4. 2.3.2 Relative paths: directions from where you are

Diagram description: standing in the `css` folder, `style.css` is in the same folder; `../menu.html` goes up one level to find the menu page; `../images/logo.svg` goes up one level and then down into images.

2.3.3 Slashes

Paths use separators between folder names. macOS, Linux and Git Bash use the **forward slash** `/`. Windows' own traditional tools use the **backslash** `\`. Everything in this book uses the forward slash, because the tools you will use (Bash, Git Bash and Git) all write paths that way.

2.3.4 Home shorthand

Many systems let you write `~` (the tilde) to mean your home folder. So `~/sharif-git-lab` means "the folder `sharif-git-lab` inside my home folder". You will use `~` in Chapter 7.

Why paths matter for Git. Git records changes to files by their path inside your project. It stores relative paths such as `css/style.css`, so that the project means the same thing on every computer that has a copy. When a Git message mentions `css/style.css`, you now know how to read it.

2.4 Hidden files

New term: hidden file. A file that a file manager or a plain listing does not show unless you ask for it.

Systems hide files that the user normally does not need to touch, such as settings. On Linux and macOS, a file or folder whose name **starts with a dot** is hidden by ordinary listings, and so it is in the Git Bash environment that Chapter 7 introduces. Windows uses a different mechanism, a "hidden" property, but Git Bash still shows dot-names as hidden.

You will see the effect yourself in Chapter 7. For now, keep this in mind: **Git keeps its records in a hidden folder called `.git` inside your project.** The project looks the same as before; the history is in the hidden folder. Files that must not be tracked by Git are listed in a hidden text file called `.gitignore` (Chapter 18).

Verification pending [R115]. The dot-name rule was demonstrated in this book's test environment (Chapter 7 transcripts); the Windows "hidden property" statement is from general knowledge and awaits documentation-level verification.

2.5 Case sensitivity

On some systems, `Report.txt` and `report.txt` are two different files. On others, they would be treated as the same name, so you could not have both in one folder.

In the book's Linux test environment, the two names were kept apart:

```
$ touch Report.txt report.txt
$ ls
Report.txt  report.txt
```

Example output; the same result on the tested Bash and zsh.

Many Windows and macOS setups do not distinguish the two names by default, although they remember the capitals you typed. That difference can make a project that works on one computer fail on another, and it is a well-known source of surprise for Git users. **The safe habit: treat capitals as meaningful, and never create two names that differ only by capitalisation.**

Verification pending [R114]. The Linux behaviour above was tested. The Windows and macOS defaults, and how particular disks can be configured differently, are unverified.

2.6 Copying, moving, renaming, deleting

You can do four things to a file. Only some can be undone.

Action	What it does	Can you undo it?
Copy	Makes a second, identical file	Yes: delete the copy
Move	Puts the file in another folder	Yes: move it back
Renam e	Changes the name (a move within the same folder)	Yes: rename it back
Delete	Removes the file	Sometimes , and only if you notice in time

Deleting deserves care. Many graphical systems have a recycle bin or trash: a deleted file is parked there and can be restored. The command line has **no** trash: a file deleted from the terminal is gone. In Chapter 7 you will practise deleting only files you have just created, in a safe practice folder.

CAUTION. Deleting is the one everyday action with no guaranteed undo. Even the recycle bin empties, and some kinds of storage (a memory stick, a network drive) bypass it. Chapter 25 will show how Git turns "delete" into something you can undo, for files that have been committed to history.

2.7 Permissions, gently

Files carry **permissions**: rules about who may do what with them.

New term: permission. A rule that says which user may read, change or run a particular file or folder.

Three kinds of action are controlled:

- **Read:** look at the contents.
- **Write:** change or delete the contents.
- **Execute:** run the file as a program (or, for a folder, enter it).

On Linux and macOS (and in Git Bash) these are shown as a string of ten characters when you list a file in detail. Here is a real example from the test environment. First the file has private permissions, then normal ones:

```
$ chmod 600 plan.txt
$ ls -l plan.txt
-rw----- 1 learner learner 0 <date> plan.txt
$ chmod 644 plan.txt
$ ls -l plan.txt
-rw-r--r-- 1 learner learner 0 <date> plan.txt
```

Example output. The *<date>* stands for the date and time your computer prints, and *learner* for the owner's name.

You do not need to decode the string now. What you need: the letters *r*, *w* and *x* mean read, write and execute, and the dashes mean "not allowed". A **"Permission denied" message means the operating system refused an action because of these rules**. Chapter 7 shows how to read that message, and Chapter 38 shows a case where a file must have restrictive permissions for security: a private key.

Permissions also explain why **administrator** permission (Chapter 1) exists: some files belong to the system, and only an administrator may change them.

2.8 Putting it together: a project folder

A **project folder** is simply a folder that holds everything that belongs to one piece of work. All the files of the Sunrise Bakery website live in one folder called `sunrise-bakery`. Git will treat that folder as the unit it tracks, so it pays to keep one project per folder, with a clear name and a tidy inside.

Try it. Using your graphical file manager, create a folder in your home folder called `sharif-git-lab`. Inside it, create a folder called `sunrise-bakery`. Then create inside `sunrise-bakery` two folders called `css` and `images`.

Expected result: a three-level tree: `sharif-git-lab` contains `sunrise-bakery`, which contains `css` and `images`. Chapter 7 will build the same structure again with commands, and you can compare.

When not to use a folder tree. A tree fits most projects. If files need to belong to several groups at once (a photo that is in both "holiday" and "family"), a tree is awkward, and applications that use tags or albums are better. Git uses a tree because source projects are naturally trees.

Checkpoint

What You Learned

- A file is a named collection of information in storage, with metadata about itself.
- The extension is a label that suggests the kind of contents; it does not change the contents.
- Folders hold files and other folders, forming a tree with a root; you have a home folder.
- A path says where a file is. An absolute path starts at the root; a relative path starts at the current directory. `.` means this folder and `..` means the parent folder.
- Dot-names are hidden on Linux, macOS and Git Bash; Git keeps its records in a hidden `.git` folder.
- Capitalisation can matter; never rely on two names that differ only in capitals.
- Copy, move and rename are reversible; delete may not be.
- Permissions control read, write and execute; "Permission denied" means a rule was applied.

New Vocabulary

- **File:** a named collection of information kept in storage.
- **Metadata:** information about a file, such as size, dates, owner and permissions.
- **Extension:** the letters after the last dot in a file name; a label for the kind of contents.
- **Folder (directory):** a container for files and other folders.
- **Home folder:** the folder that belongs to your user account.
- **Root:** the top folder of a tree, written `/` on macOS, Linux and Git Bash.
- **Path:** a written description of where a file or folder is in the tree.
- **Absolute path:** a path that starts at the root.

- **Relative path:** a path that starts at the current directory.
- **Current directory (working directory):** the folder that counts as "here" for relative paths.
- **Hidden file:** a file not shown by ordinary listings unless requested.
- **Permission:** a rule about who may read, change or run a file.

Commands Learned

None yet. (Section 2.5 and section 2.7 quote real command output so that you can recognise it later; you were not asked to type commands.)

Common Mistakes

1. **Believing that renaming the extension converts the file.** It only changes the label.
2. **Mixing up absolute and relative paths.** If a relative path "does not work", first check where you are.
3. **Using spaces and capitals freely in project names.** Prefer lowercase with hyphens.
4. **Assuming the recycle bin exists everywhere.** The command line, memory sticks and network drives may bypass it.
5. **Creating two names that differ only in capitals.** They may collide on another computer.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between a file's name and its contents?
2. You are inside `sunrise-bakery/css`. Write the relative path to `logo.svg` in `sunrise-bakery/images`.
3. Write the absolute path of `menu.html` for the tree in section 2.2 if `sunrise-bakery` is in `/home/learner/sharif-git-lab`.
4. What does `..` mean? What does `.` mean?
5. Why might a file be invisible in a folder that clearly contains it?
6. What does "Permission denied" tell you?

Before Moving On

You are ready for Chapter 3 if you can:

- [] draw the Sunrise Bakery folder tree from memory
- [] turn on the display of file extensions on your computer
- [] explain the difference between absolute and relative paths with one example each
- [] say why deleting is the only action in section 2.6 with no guaranteed undo

How this chapter was checked

Claim	Evidence class	Ledger
Drive letters and separators on Windows versus macOS/Linux	Needs re-verification	R11 3
File names differing by case are distinct on the tested Linux system; Windows/macOS defaults	Linux: locally tested (verification/ch02-files, bash and zsh); Windows/macOS: unverified	R11 4
Dot-names are hidden from plain <code>ls</code> , shown by <code>ls -a</code> ; Windows hidden attribute	Unix-like: locally tested; Windows: unverified	R11 5
<code>ls -l</code> mode strings after <code>chmod 600</code> and <code>chmod 644</code>	Locally tested (bash 5.2 and zsh 5.9, GNU coreutils)	R11 6

Where this leads

Chapter 3 looks at the contents of files that are made of text, and at the programs (editors) used to change them. Chapter 4 looks inside text files at the level of bytes. Chapter 7 gives you a way to create, list, copy and delete files by typing, using every path idea from this chapter.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises use only your file manager; no commands are needed.

Level 1 — Guided

1.1 Show extensions. Find the setting in your file manager that shows file name extensions (search the help for "file name extensions"; wording varies). Turn it on. Expected result: file names such as `report.docx` or `photo.jpg` are shown in full.

1.2 Build the tree. In your home folder create `sharif-git-lab`, and inside it `sunrise-bakery`, and inside that `css` and `images`. Expected result: four folders in a three-level tree.

Level 2 — Partially guided

2.1 Read the address. The absolute path `/home/learner/sharif-git-lab/sunrise-bakery/css/style.css` is given. Name each folder from the root down, then name the file. Hint: split at every slash.

2.2 Relative directions. You are inside `sunrise-bakery/css`. Write relative paths to: (a) `style.css`; (b) `menu.html` in `sunrise-bakery`; (c) `logo.svg` in `sunrise-bakery/images`.

Level 3 — Independent

3.1 Design a folder tree for a school project that has a report, three photos and a spreadsheet of results. Write the name of every folder and file, following the naming habits in section 2.1.1. Explain in two sentences why you grouped things the way you did.

Level 4 — Professional scenario

4.1 A colleague sends you a folder called `Final Report NEW (2)` containing files named `Chapter One FINAL.docx`, `chapter one final v2.docx` and `IMG 0042.JPG`. Propose a tidy structure and new names. Say which names you changed and why. Which changes would help you when a program later has to be told the path to a file?

Level 5 — Troubleshooting

5.1 A learner writes: "I renamed `logo.svg` to `logo.png` so it would be a PNG picture, but now my website shows a broken image." Explain what happened using the word extension, and say how to get a real PNG.

5.2 Two people share a project. On one person's computer, `Menu.html` and `menu.html` both exist. On the other person's computer only one of them appears. Give a possible explanation and a rule that avoids the problem.

Chapter 3 — Editors and Project Folders [Core]

In this chapter

- why some files are "plain text" and others are not, and why Git cares
- the difference between a word processor and a text editor
- what to look for in a code editor
- what a "software project" is, using a small website as the running example
- how to get the Sunrise Bakery starter files ready for the rest of the book

Before you start. Chapter 2 (files, folders, paths, extensions). You should be able to create a folder and see file extensions.

3.1 Two kinds of file contents

Chapter 2 said that a file is a named collection of information. The kind of information inside decides which programs can open it and, later, how well Git can help you with it.

For this book, you need only two kinds.

New term: plain text. Information stored as ordinary characters (letters, digits, punctuation, spaces and line breaks) with nothing else added: no fonts, no colours, no pictures, no page layout.

New term: binary file. Any file whose contents are not plain text: photographs, music, compressed archives, word-processor documents and programs are all binary files.

A **text file** is a file made of plain text. When you open one in a suitable program you see exactly the characters that are in it, and nothing hidden.

3.1.1 What is inside a word-processor document?

A document from a word-processor program looks like text on a page, but the file is more than the words. It also holds fonts, spacing, page size, styles, and sometimes a history of tracked changes. To keep all of that, the file is packaged.

The test below, run in the book's test environment, creates a one-sentence document and compares it with a plain text file that holds the same sentence:

```
plain text file: 27 bytes; starts with: b'Fresh bread '
word-processor file starts with bytes: b'PK' (the ZIP signature 'PK')
is it a ZIP archive? True
contains parts (first four, sorted): [Content_Types].xml, _rels/.rels, customXml/_rels/
item1.xml.rels, customXml/item1.xml
```

```
has word/document.xml: True
visible text is about 26 bytes; the file holds far more than the text
file is larger than its text: True
```

Example output from *verification/ch03-editors/docx_vs_txt.py* (produced with a widely used Python library for creating *.docx* files; you do not need to run it).

Read the result. The plain text file is 27 bytes: the 26 characters of the sentence plus a line break. The word-processor file begins with the two bytes PK, which mark a **ZIP archive**, a container that packs several files into one. Inside the container are separate parts, one of which (*word/document.xml*) holds the words, wrapped in markup. The sentence is a tiny part of a much larger package.

New term: ZIP archive. A single file that contains other files in compressed form. Many document formats, including modern word-processor formats, use a ZIP archive as their outer package.

Verification pending [R117]. The test above shows what one library produces. The published standards that define the word-processor format have not yet been consulted, so the statement "modern word-processor files are packaged as ZIP archives of XML parts" is tested for one tool but not yet checked against the standard.

Why this matters for Git. Git is superb at tracking plain text, and it can show you exactly which lines changed. For a packaged binary file it can still keep every version, but by default it cannot show a readable difference: it can only say "this file changed". This is why the projects in this book use text files, and why documents that you want to track closely (for example this book's own chapters) are written in a text format called Markdown (Chapter 8) instead of a word-processor format.

When not to use plain text. If you need pictures, complicated layout, or colleagues who expect a word-processor file, use one, and use Git for what it does best, keeping every version. Choose plain text where you can, and where readable differences matter.

3.2 Editors

New term: text editor. A program for creating and changing plain text files.

New term: word processor. A program for writing formatted documents, with fonts, page layout and pictures, that normally saves in a packaged (binary) format.

The two look similar and are used differently.

	Text editor	Word processor
Saves	Only the characters you type	Characters plus formatting and page information
Result	Plain text file	Packaged binary file
Good for	Code, notes, configuration, Markdown	Letters, reports, anything printed
Git can show line-by-line differences	Yes	No

3.2.1 Code editors

A **code editor** is a text editor with extra help for writing code and other structured text.

New term: code editor. A text editor with features for structured text such as software: coloured text, line numbers, tools for searching many files, and often a built-in terminal (a window for typing commands, introduced in Chapter 7).

The features that will help you in this book:

- **Line numbers.** Git messages refer to lines. Seeing numbers makes them easy to follow.
- **Syntax highlighting.** The editor colours words according to their role, so mistakes stand out.
- **Show invisible characters.** You can display spaces, tabs and line endings. Chapter 4 explains why you will want this.
- **Choice of encoding and line endings.** You can say how characters are stored in the file (Chapter 4 explains both terms).
- **Open a whole folder.** You see all the files of a project in a side panel.
- **Search across files.** You can find every use of a word in the project.

New term: syntax highlighting. Colouring different parts of text (for example, tags, words and comments) so that its structure is easy to see.

3.2.2 Choosing one

This book does not depend on any particular editor, and it does not recommend a brand. Many good ones exist for every operating system, and free ones are available. Pick one that:

1. is available for your operating system and is maintained by its makers (check when the last update was released);
2. can open a folder and show line numbers;
3. lets you choose **UTF-8** encoding (Chapter 4) and see line endings;
4. you feel comfortable using.

Security note. An editor is a program you install. Apply the rules from Chapter 1: download it from its makers' own website or your system's official source. Extensions that some editors let you add are more programs; add only those you need, from the official catalogue.

A trap to avoid. Do not write code or configuration files in a word processor. Word processors may replace straight quotation marks with curly ones ("like this"), add invisible formatting, and change dashes, and any of these can make a program refuse to run. If a file that should be plain text will not work, check first that it was not edited in a word processor.

3.3 Saving and opening

When you open a file, the editor copies it from storage into memory (Chapter 1). When you save, it copies your changes from memory back to storage. Until you save, your changes exist only in memory and would be lost in a power cut.

Watch for these signals:

- Most editors mark an unsaved file, often with a dot or asterisk beside its name in the tab.
- **Save** overwrites the file with your changes. **Save As** creates a new file (choose the name and folder carefully) and leaves the original untouched.
- Some editors **auto-save**. This is convenient, but it means every keystroke may go to storage. That is one more reason to keep history with Git, which records deliberate versions.

UI-VERSION NOTE. Menu names and keyboard shortcuts differ between editors and operating systems. If you cannot find "Save As" or the encoding setting, search the editor's help for those words.

3.4 What a software project is

New term: software project. A collection of files, kept together in one folder, that make up one piece of work: a website, an application, a set of documents, or a tool.

You have used the word "project" loosely before. Here it has a precise meaning, because Git will track one project folder at a time.

The files of a project fall into a few groups. Learning to see the groups now will make Chapter 18 (deciding which files Git should track) easy.

Group	Meaning	Example in the Sunrise Bakery site
Source files	Written by people; the real work	index.html, menu.html, contact.html, css/style.css, images/logo.svg

Group	Meaning	Example in the Sunrise Bakery site
Documentation	Explains the project	README.md
Assets	Pictures, sounds, fonts	images/logo.svg
Configuration	Settings that tune the project	(none yet)
Generated files	Made automatically from source files by a tool	(none yet; a later chapter adds some)
Dependencies	Other people's software your project uses	(none)

New term: source code (source files). The files, written by people, from which a project is made. For a website, these include the pages and stylesheets.

Generated files and dependencies can be huge and can be rebuilt, so they usually should not be tracked. You will see how to tell Git to ignore them.

3.5 The Sunrise Bakery project

The book's running example is a small website for a fictional business, **Sunrise Bakery**. It has three pages, a stylesheet and a logo, and it is the project you will track, branch, share and automate for the rest of the book.

You do not need to write code. You need to understand what kind of files these are, so that when Git talks about them, you know what you are looking at.

File	Kind	What it does
index.html, menu.html, contact.html	HTML	The pages: their words and structure
css/style.css	CSS	The appearance: colours, spacing
images/logo.svg	SVG	A small picture, stored as text

New term: HTML. A text format for web pages. It marks up text with tags such as `<h1>` (a main heading) so that a browser (the program for viewing web pages) knows how to display it.

New term: CSS. A text format that describes how web pages should look: colours, fonts, spacing.

Here is the start of `index.html`. Read the idea: everything is text, and the tags in angle brackets describe the words.

```

<header>
  
  <h1>Sunrise Bakery</h1>
  <nav>
    <a href="index.html">Home</a>
    <a href="menu.html">Menu</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>
<main>
  <h2>Fresh bread every morning</h2>
  <p>We bake bread, rolls and small cakes from six o'clock, every day except Sunday.</p>
</main>

```

Notice the paths: `images/logo.svg` and `menu.html` are **relative paths** (Chapter 2). The page finds its logo by directions from where the page itself lives. If you moved `index.html` out of its folder without moving the `images` folder, the logo would disappear. That is an everyday example of why the whole folder, kept together, is the project.

The starter files have been checked mechanically: every page has balanced tags, every link between pages and every reference to the stylesheet and logo points at a file that exists, and the logo is well-formed.

Verification pending [R133]. The starter files were checked by a script of the book's own (`verification/companion-starter/check_starter.py`). They have not yet been opened in a range of browsers and screen readers.

3.5.1 Getting the starter files

The starter files come with the book, in the folder `companion/sunrise-bakery-starter`. Because you have not installed Git yet, the simplest way to obtain them is as a single ZIP archive.

Where to get them. The companion repository, <https://github.com/Sharifwa123/Books> (folder `git-and-github-mastery/companion`), holds the files. On that page, look for the option that downloads the whole repository as one ZIP archive; its label and place change from time to time, so look for the idea. If your edition came with a copy of the files, you may use that copy instead.

Once you have the ZIP archive:

1. Put it inside your practice folder, `sharif-git-lab` (Chapter 2).
2. **Extract** it (also called "unzip"): most file managers offer "Extract" or "Extract all" when you click the archive. The exact name of the command varies.
3. Rename the extracted folder `sunrise-bakery`, if it has a different name.

Try it. Open `sunrise-bakery/index.html` by double-clicking it. Your web browser should open it as a local page: a heading, a short paragraph and three menu links, styled with a warm yellow bar.

Now open `index.html` in your **text editor** instead (usually a right-click and "Open with"). Change the words "Fresh bread every morning" to "Fresh bread and rolls every morning". Save the file. Switch to the browser window and reload the page.

Expected result: the browser shows your new words. This is the whole loop of software work: edit text, save, look at the result.

Deep. The page opened straight from your storage; no server was involved. Chapter 5 explains what a web server adds.

3.6 Why a project is a folder, and why that matters next

You have now seen the idea that runs through the rest of the book: **a project is a folder of mostly text files, kept together, that change over time.**

Change is the problem. Imagine that you edit `menu.html` today, your friend edits it tomorrow, and next week you realise that last Tuesday's version had a price you need. Without a history, you have a copy of a copy of a copy called `menu_final_v2_REAL.html`. Part II explains the problem, and Part III solves it with Git.

Checkpoint

What You Learned

- Plain text files contain only characters; word-processor documents are packaged binary files whose text is a small part of the whole.
- Git can show readable line-by-line differences for text files but only "changed" for binary files.
- A text editor saves plain text; a code editor adds line numbers, colouring, invisible-character display and folder tools.
- Do not edit code or configuration in a word processor.
- Opening copies a file from storage into memory; saving copies it back.
- A software project is a folder of files: source files, documentation, assets, configuration, generated files and dependencies.
- The Sunrise Bakery project is a three-page website made of HTML, CSS and SVG text files, with a README.

New Vocabulary

- **Plain text:** information stored as ordinary characters with no formatting.

- **Binary file:** a file that is not plain text, such as a photo or word-processor document.
- **ZIP archive:** one file that holds other files in compressed form.
- **Text editor:** a program for plain text files.
- **Word processor:** a program for formatted documents.
- **Code editor:** a text editor with features for writing structured text and software.
- **Syntax highlighting:** colouring parts of text by their role.
- **Software project:** the folder of files that makes up one piece of work.
- **Source code (source files):** the human-written files from which a project is made.
- **HTML:** a text format that marks up web pages.
- **CSS:** a text format that describes how web pages look.

Commands Learned

None. (Opening, editing and saving files in a graphical editor need no commands.)

Common Mistakes

1. **Editing code in a word processor.** Curly quotes and hidden formatting can break the file.
2. **Forgetting to save, then expecting the browser to show the change.** Save, then reload.
3. **Moving one file out of its project folder.** Relative paths such as `images/logo.svg` will no longer find their targets.
4. **Choosing "Save As" and losing track of which file is which.** Prefer "Save" for the project files and a history tool for versions.
5. **Renaming a file's extension to change its type** (Chapter 2).

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Give two differences between a text editor and a word processor.
2. Why does Git show readable differences for `menu.html` but only "changed" for a `.docx` file?
3. What is a software project, in one sentence?
4. Sort these into source files, documentation, assets and generated files: `README.md`, `style.css`, `logo.svg`, a folder of files produced automatically by a build tool.
5. The logo disappears when you move `index.html` to another folder. Why?

Before Moving On

You are ready for Chapter 4 if you can:

- [] open a folder in a text editor and see line numbers
- [] edit `index.html`, save it, and see the change in the browser
- [] say why a word-processor file is not plain text
- [] name the files in the Sunrise Bakery project and what each does

How this chapter was checked

Claim	Evidence class	Le dg er
A word-processor document generated by a common library is a ZIP archive containing word/document.xml, far larger than its text	Locally tested (verification/ch03-editors/docx_vs_txt.py, python-docx 1.2.0); standard not yet consulted	R1 17
The starter files have balanced tags, resolving links and a well-formed logo	Locally tested (verification/companion-starter/check_starter.py, run in CI)	R1 33
Editor features and cautions (curly quotes, encoding choice)	General guidance; no product-specific claim is made	no ne

Where this leads

Chapter 4 looks inside text files: characters, encodings and line endings, the invisible details that sometimes make the same file behave differently on two computers. Chapter 7 introduces the terminal, and Chapter 8 introduces Markdown, the text format in which READMEs and this book are written.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book.

Level 1 — Guided

1.1 Prepare your editor. Install or open a code editor (see section 3.2.2). Open your sharif-git-lab folder in it, so its files appear in a side panel. Turn on line numbers. Expected result: you can see numbered lines when a file is open.

1.2 Edit and reload. Follow the Try it box in section 3.5.1: open index.html in a browser, change the heading text in your editor, save, and reload the page. Expected result: the change appears in the browser.

Level 2 — Partially guided

2.1 Kinds of file. For each file in the Sunrise Bakery project, write down: text or binary, and source file, documentation or asset. Hint: the logo is an SVG file, which is a picture stored as text.

2.2 Break it on purpose. Copy sunrise-bakery to a folder called sunrise-bakery-test. In the copy, move images out of the project folder. Open index.html from the copy. Describe what you see and explain it using the idea of a relative path. Then delete the test copy. Hint: only delete the copy you created.

Level 3 — Independent

3.1 Write a short note (four sentences) explaining to a classmate why the .docx version of a report is a poor choice if you want to see line-by-line changes between versions.

Level 4 — Professional scenario

4.1 A small charity keeps its volunteer schedule in a word-processor document that six people edit. They ask whether they should keep it in Git. Recommend a format and a way of working, and name one thing that would still be better in a word processor.

Level 5 — Troubleshooting

5.1 A learner edits `style.css` in a word-processor program, saves it, and the website loses all its styling. What is the most likely cause, and how would you confirm and fix it?

5.2 You change `menu.html`, save, and reload the page. Nothing changes. List three things to check, in order.

Chapter 4 — Text, Encodings and Newlines [Core]

In this chapter

- how a computer stores letters as numbers, and what "encoding" means
- what UTF-8 is, and why it is the safe default
- why text sometimes turns into strange symbols, and how to fix it
- the invisible characters that end lines, and why two computers can disagree about them
- other invisible trouble: tabs, spaces and trailing spaces

Before you start. Chapter 3: you know what a plain text file is, and that Git can show readable differences for text but not for packaged binary files.

Look, don't type. Some boxes in this chapter show commands, on lines that begin with \$, followed by what the computer printed. You are not asked to type them yet. The terminal is introduced in Chapter 7, where you can repeat every one of these. (The programs that read such commands are called shells; this book tests two of them, Bash and zsh.) For now, read the results: they are real output recorded in the book's test environment, and they show you what is hidden inside ordinary files.

4.1 From letters to numbers to bytes

A computer can only store numbers (Chapter 1 said that everything is stored as bits and bytes). So to store the letter A, people had to agree on a number for it. Two ideas follow.

1. Every character (a letter, digit, punctuation mark, emoji, or a space) is given a number, in a shared list.
2. To save the character in a file, its number is turned into one or more bytes, following a rule.

An analogy: think of a book of secret codes. Page one says "A is 65, B is 66...". To send a message you look up each letter's number and write the numbers down. The person who reads it needs the same book. If they use a different book, the message comes out as nonsense.

New term: character. A single unit of text: a letter, digit, punctuation mark, symbol, emoji, or space.

New term: encoding. The rule that turns characters into bytes when they are saved, and back into characters when they are read.

The important lesson is already visible: **to read a text file correctly, you need to know how it was encoded.**

4.2 ASCII, Unicode and UTF-8

Three names will keep appearing, so it is worth meeting them once, calmly.

Name	What it is	Size of the idea
ASCII	An old, small list of 128 characters: English letters, digits and common punctuation, plus some control codes	Enough for plain English
Unicode	A huge list that gives every character in the world's writing systems (and many symbols and emoji) its own number	Covers nearly every language
UTF-8	A way of writing Unicode numbers as bytes	Uses 1 byte for the basic English characters and 2 to 4 bytes for others

New term: Unicode. A universal list that assigns a number, called a code point, to each character of the world's writing systems, symbols and emoji.

New term: UTF-8. The most widely used encoding of Unicode. It stores the basic English characters in one byte each and other characters in two to four bytes.

UTF-8 has one property that made it popular: text made only of basic English characters is stored **exactly as ASCII** would store it, byte for byte. So old English-only files are already valid UTF-8, and the new system can still store every other language.

4.2.1 See it: one letter, two bytes

Here is a real recording. A one-word file, `café`, is created and its bytes are displayed in hexadecimal (a compact way of writing byte values that uses the digits 0–9 and letters a–f). Each pair of characters is one byte.

```
$ printf 'caf\303\251\n' > cafe.txt
$ cat cafe.txt
café
$ od -An -tx1 cafe.txt
 63 61 66 c3 a9 0a
$ wc -c cafe.txt
6 cafe.txt
```

Example output; identical in the tested Bash and zsh.

Read the bytes:

- 63 61 66 are the three characters `c`, `a`, `f`, one byte each, exactly as ASCII stores them.
- c3 a9 is the single character `é`. In UTF-8 it takes **two** bytes.
- 0a is the invisible **line break** at the end of the line (section 4.4).

So the file has four visible characters but **six bytes**. A counter of bytes and a counter of characters give different answers. Ordinary English text is the special case in which they are equal.

For an emoji, one character takes four bytes:

```
$ printf '\360\237\215\236\n' | od -An -tx1
f0 9f 8d 9e 0a
```

Example output. The four bytes `f0 9f 8d 9e` are one character, a loaf of bread; the fifth byte is the line break.

Verification pending [R118]. These demonstrations were run on real tools, so the byte values shown are facts about UTF-8 as implemented in the test environment. The statements that describe history and popularity (that ASCII has 128 characters, that UTF-8 is the most widely used encoding) are from general knowledge and will be checked against the Unicode Consortium's own publications before release.

When to use UTF-8: almost always, for any new file. Choose it in your editor (Chapter 3) and keep it.

Alternatives and older encodings. Other encodings exist, mostly older ones designed for a single language or region. You will meet them only in old files. If a file is not UTF-8, the safest action is to convert it deliberately with a tool, not to edit it in place.

4.3 When the wrong book is used: garbled text

Because the reader must know the encoding, mistakes appear as **garbled text**: `cafÃ©` instead of `café`. The technical name is *mojibake*, a Japanese word meaning "character change".

New term: mojibake. Text that appears as strange symbols because it was read using a different encoding from the one used to write it.

Here is a controlled reproduction of exactly that problem. The same word is saved in two encodings and then read with the wrong one:

```
the word:                café
stored as UTF-8 bytes:    63 61 66 c3 a9    (5 bytes for 4 characters)
the same bytes read as Latin-1: cafÃ©
stored as Latin-1 bytes:  63 61 66 e9      (4 bytes)
Latin-1 bytes read as UTF-8:    UnicodeDecodeError (unexpected end of data)
with errors replaced:      caf 
emoji U+1F35E as UTF-8:    f0 9f 8d 9e    (4 bytes for 1 character)
```

Example output from `verification/ch04-text/mojibake.py`. You do not need to run it. "Latin-1" is one of the older encodings.

Two different failures appear:

1. The UTF-8 bytes `c3 a9` were read as if each byte were a separate character, giving `Ã©`.

2. The single byte `e9` is not a valid piece of UTF-8 text at all. Some programs refuse to continue; others print a replacement mark, `?`, and carry on, so **the original character is lost**.

What to do when you see garbled text.

1. Do not retype and re-save. That can make the damage permanent.
2. Look in your editor for the encoding it thinks the file uses (often shown in the status bar).
3. If you know the correct encoding, reopen the file with it (most editors have "Reopen with encoding").
4. Convert once, to UTF-8, and save.

How this affects Git. Git stores bytes. If two people save the same text in different encodings, Git sees different bytes and reports every affected line as changed. Agreeing on UTF-8 for the whole project prevents that. (Chapter 14 shows the Git settings involved.)

4.4 Line endings

When you press the Enter key, the editor inserts an invisible character that marks the end of a line: a **line break** (also called a **newline**).

New term: line ending (newline). The invisible character or characters that mark the end of a line of text.

The complication is that two conventions exist, and the reason is historical. On a mechanical typewriter, ending a line took two separate movements: return the carriage to the left (carriage return, abbreviated CR), and roll the paper up one line (line feed, abbreviated LF). Early computer systems copied those two ideas, and they made different choices:

Convention	Characters	Written	Traditionally used by
LF	one: line feed	<code>\n</code>	Linux, macOS and other Unix-like systems
CRLF	two: carriage return then line feed	<code>\r\n</code>	Windows

Verification pending [R119]. The two conventions and their bytes are demonstrated below. Which convention each system uses by default today, and how modern programs on each system cope, is from general knowledge and awaits documentation-level verification.

4.4.1 See it: two extra bytes per line

Two files hold the same two lines, one and two. One uses LF, the other CRLF.

```
$ printf 'one\ntwo\n' > unix.txt
$ printf 'one\r\ntwo\r\n' > windows.txt
$ od -c unix.txt
0000000  o  n  e  \n  t  w  o  \n
0000010
$ od -c windows.txt
```

```
0000000  o  n  e  \r  \n  t  w  o  \r  \n
0000012
$ wc -l unix.txt
2 unix.txt
$ wc -c unix.txt
8 unix.txt
$ wc -c windows.txt
10 windows.txt
```

Example output; identical in the tested Bash and zsh. (The `od -c` command prints each byte, showing invisible characters as `\n` for line feed and `\r` for carriage return.)

Read it. The LF file is 8 bytes; the CRLF file is 10, because each of the two lines has one extra invisible character. To a person looking at an editor, the two files look identical. To Git, which compares bytes, **they are different files**.

4.4.2 Why line endings cause real trouble

Trouble 1: noisy differences. If you open a file on Windows and your editor quietly converts every LF to CRLF, then Git will see every line as changed, even though you altered nothing visible. Someone reviewing your work sees hundreds of "changes" and cannot find the real one.

Here is a real recording of that effect. A two-line file was saved with LF endings and recorded by Git, then the same visible text was saved again with CRLF endings. The command summarises what Git thinks has changed:

```
f.txt | 4 ++--
1 file changed, 2 insertions(+), 2 deletions(-)
```

Example output from Git (which you will install in Chapter 13). Two lines were "removed" and two "added", although a person would say that nothing changed. Recorded by *verification/ch04-text/git_line_endings.sh*.

Trouble 2: programs that stop working. Some programs are strict. In the test below, a small script is written twice. One has LF line endings, the other has CRLF. Both are meant to print a greeting:

```
$ ./lf.sh
hello from LF
$ ./crlf.sh
bash: ./crlf.sh: cannot execute: required file not found
$ grep -c "$(printf '\r')" crlf.sh
2
$ grep -c "$(printf '\r')" lf.sh
0
```

Example output in Bash; the same experiment in zsh gives *zsh: ./crlf.sh: bad interpreter: /bin/sh^M: no such file or directory*. (The last two commands count the lines containing a carriage return.)

The error message does not mention line endings. The reason is that the first line of the script names the program to run it with, `/bin/sh`, and the invisible carriage return at the end of that line becomes part of the name. The computer looks for a program called `"/bin/sh` followed by CR", finds nothing, and complains that the file is missing. The messages are different in the two shells, and the true cause is invisible unless you know to look.

Error messages that mention `^M` are a strong clue. `^M` is a common way of writing the carriage-return character.

The lesson. Line endings are not a matter of taste when files travel between computers. Decide on one convention per project and let the tools enforce it. You will do that with Git settings in Chapter 14 and with a file called `.gitattributes` in Chapter 32.

4.5 Other invisible characters

Three more invisible things can make otherwise identical text differ.

Invisible thing	What it is	Trouble it causes
Tab	A single character that jumps to the next column	Looks like several spaces, but is different from spaces; mixing them makes files look misaligned in other editors
Trailing spaces	Spaces at the end of a line, after the last visible character	Invisible, but change the bytes; Git reports them as changes
Missing final line break	The last line has no line break at the end	Some tools warn, and Git marks it in its output with the note <code>\ No newline at end of file</code>

Here they are, in bytes:

```
$ printf 'a\tb\n' | od -c
00000000  a  \t  b  \n
00000004
$ printf 'a b\n' | od -c
00000000  a      b  \n
00000004
$ printf 'end   \n' | od -c
00000000  e    n    d               \n
00000007
```

Example output. The first line holds a tab (`\t`), the second an ordinary space, and the third has three trailing spaces after `end`.

A tab and a space look alike on screen, but they are different bytes. For that reason, code editors can **show whitespace**: dots for spaces, arrows for tabs. Turn it on while you learn.

4.6 Text or binary? A quick way to decide

A file is text if its bytes are meant to be read as characters in some encoding. It is binary if they are not: a photograph, a music file, a ZIP archive, a program. You met the ZIP signature PK in Chapter 3.

Signs of a binary file: your editor shows unreadable symbols, or offers to "open as binary"; the file is large compared with its visible content; or Git reports "Binary files differ". Handle binary files carefully in a project: do not edit them in a text editor and re-save them.

4.7 A checklist for text files in a project

Adopt these habits now; later chapters will show the Git settings that support them.

1. Save every file as **UTF-8**.
2. Agree one **line ending** convention for the project and check what your editor is set to.
3. **Show whitespace** while editing.
4. End every file with a line break.
5. Avoid **trailing spaces**.
6. Choose **tabs or spaces** for indentation and use one consistently within a file.

Try it. Open `sunrise-bakery/index.html` in your code editor. Find these three things: (1) the encoding indicator (often at the bottom right of the window); (2) the line-ending indicator (it may say LF or CRLF); (3) the setting that shows whitespace. Turn on "show whitespace" and look at the indentation of the `<header>` block.

Expected result: the editor states the encoding (ideally UTF-8) and line ending, and dots or arrows appear where there are spaces or tabs. Write down what you found.

UI-VERSION NOTE. Where these indicators and settings are located differs by editor and version. If you cannot find them, search the editor's help for "encoding", "line endings" and "render whitespace".

Checkpoint

What You Learned

- Computers store characters as numbers; an encoding is the rule that turns characters into bytes.
- Unicode lists characters; UTF-8 is the widely used way to store them; English characters take one byte, others two to four.
- Reading a file with the wrong encoding produces garbled text (mojibake) or errors.
- A line ending is an invisible character; LF and CRLF are the two conventions, and they differ by one byte per line.
- Files that look the same can differ in bytes, and Git compares bytes.

- Tabs, trailing spaces and a missing final line break are also invisible differences.
- A simple checklist keeps text files consistent.

New Vocabulary

- **Character**: a single unit of text such as a letter, digit, symbol or space.
- **Encoding**: the rule that turns characters into bytes and back.
- **Unicode**: a universal list that gives every character a number.
- **UTF-8**: the most widely used encoding of Unicode; one byte for basic English characters, two to four for others.
- **Mojibake**: garbled text caused by reading with the wrong encoding.
- **Line ending (newline)**: the invisible character or characters that end a line.

Commands Learned

None to type yet. You read the output of `printf`, `od`, `wc` and `grep` (Chapter 7 introduces the terminal in which they run).

Common Mistakes

1. **Assuming that same-looking files are identical.** Their bytes may differ.
2. **Fixing garbled text by retyping and saving.** Reopen with the correct encoding instead.
3. **Ignoring line endings when sharing files between Windows and other systems.** Agree a convention early.
4. **Mixing tabs and spaces in one file.**
5. **Editing a binary file in a text editor.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is an encoding, in one sentence?
2. The file `cafe.txt` shows four characters and is six bytes long. Where do the two extra bytes come from?
3. What do LF and CRLF stand for, and how do the two files in section 4.4.1 differ in size?
4. A script fails with an error mentioning `^M`. What is the most likely cause?
5. Give two reasons why two files that look identical on screen might be reported by Git as different.

Before Moving On

You are ready for Chapter 5 if you can:

- [] explain what UTF-8 is without using the word "encoding" more than once
- [] find the encoding and line-ending indicators in your editor

- [] describe why a CRLF file and an LF file are different files
- [] turn on "show whitespace" in your editor

How this chapter was checked

Claim	Evidence class	Le dg er
Byte values of café (5 bytes UTF-8), the four-byte emoji, and the read-as-wrong-encoding outcomes	Locally tested: <code>verification/ch04-text/session-bytes.txt</code> (Bash and zsh, interactive), <code>mojibake.py</code> (re-run in CI); Unicode Standard not yet consulted	R1 18
LF versus CRLF bytes, sizes, the failing CRLF script and its two shell-specific messages	Locally tested: <code>session-bytes.txt</code> , <code>session-crlf-trouble.txt</code> (Bash 5.2 and zsh 5.9; also passes on the CI runner); which systems use which by default: unverified	R1 19
Tabs, spaces and trailing spaces appear as distinct bytes	Locally tested (<code>session-bytes.txt</code>)	R1 18

Where this leads

Chapter 7 finally gives you the terminal, in which you can repeat every command shown here. Chapter 8 introduces Markdown, a plain text format. Chapter 14 shows the Git settings that manage line endings, and Chapter 32 explains `.gitattributes`.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises use only your editor; the terminal comes in Chapter 7, where you can repeat the recorded commands.

Level 1 — Guided

1.1 Find the indicators. Open any text file in your editor. Write down the encoding and the line-ending style it reports (for example "UTF-8" and "LF"). Expected result: two words or abbreviations.

1.2 Show whitespace. Turn on "show whitespace" and look at a file with indented lines. Expected result: dots or arrows mark the spaces or tabs.

Level 2 — Partially guided

2.1 Count bytes. Using section 4.2.1 as a model, work out how many bytes the UTF-8 file containing exactly the text naïve (with a line break at the end) occupies. Hint: only the `i` needs two bytes.

2.2 Read the bytes. A file's bytes are `48 69 0d 0a`. Write the text it contains and name its line-ending style. Hint: 48 is H, 69 is i.

Level 3 — Independent

3.1 Explain in your own words, in four sentences, why a file that looks identical on two computers might still be reported as changed by Git.

Level 4 — Professional scenario

4.1 A team of four works on a website. Two use Windows and two use Linux. Code reviews are full of "changes" nobody made. Propose, in a short list, the conventions the team should agree (encoding, line endings, whitespace, final line break), and say how each would be checked.

Level 5 — Troubleshooting

5.1 A visitor reports that the bakery page shows CafÃ© Special where the menu says Café Special. Explain the cause, the evidence you would look for in the file, and the fix.

5.2 A small script works on a Linux server but, after being edited on a Windows laptop and copied back, fails with a message mentioning ^M. Explain why, and give two ways to prevent a repeat.

Chapter 5 — The Internet, the Web and Servers [Core]

In this chapter

- what a network and the internet are, and how the web differs from the internet
- how to read a web address (URL) piece by piece
- what "client" and "server" mean, shown with a real request and reply
- what "local" and "remote" mean, and why Git needs both ideas
- what the "S" in HTTPS does and does not promise

Before you start. Chapter 1: you know what hardware, software, applications and an operating system are.

Look, don't type. As in Chapter 4, some boxes show commands on lines that begin with \$, followed by real recorded output. You are not asked to type them; Chapter 7 introduces the terminal.

5.1 Networks, the internet, and the web

New term: network. Two or more computers connected so that they can exchange information.

Your phone and your home router form a small network. Your school's computers form a bigger one. The **internet** is what you get when networks all over the world are connected to one another.

New term: internet. The worldwide system of connected networks, which lets any connected computer exchange information with any other.

Picture the road system of a country: village lanes, town streets and motorways, all joined so that a lorry can drive from anywhere to anywhere. No single company owns all the roads, and the roads carry many kinds of traffic. The internet is the roads.

5.1.1 The web is one thing that travels on the internet

Beginners often say "internet" when they mean "the web". They are different.

New term: web (World Wide Web). A system of linked pages and applications that you view with a browser, using the internet to fetch them.

Email, video calls, online games and file downloads also travel over the internet, but they are not the web. **Git and GitHub also use the internet.** Git can copy a project to and from a computer in another city, and GitHub is a service, reached through the web, built around such copies. You will see how in Part V.

	Internet	Web
What it is	The connected networks	A service that runs on those networks
Analogy	The roads	One kind of delivery that uses the roads
Needs a browser	No	Yes, normally
Other examples on the same roads	Email, video calls, Git	(this is one of them)

New term: browser. An application for viewing web pages and web applications.

5.2 Reading a web address

Every page on the web has an address called a **URL** (uniform resource locator).

New term: URL. The written address of a resource on the internet, made of several parts such as the scheme, the host and the path.

Here is a made-up address, split into its parts by a program, so that you can see the pieces:

```
URL:      https://example.org:8443/menu/cakes?sort=price&page=2#top
scheme:   https
host:     example.org
port:     8443
path:     /menu/cakes
query:    sort=price&page=2  -> {'sort': ['price'], 'page': ['2']}
fragment: top
```

Example output from *verification/ch05-internet/url_parts.py*, which uses Python's standard library to split the address. You do not need to run it.

Part	In the example	Meaning
Scheme	https	How to talk to the server (here: secure web traffic)
Host	example.org	Which computer to talk to
Port	8443	Which "door" on that computer (usually omitted: the scheme implies a default)
Path	/menu/cakes	Which resource on that computer
Query	sort=price&page=2	Extra details you send, as name=value pairs

Part	In the example	Meaning
Fragmen t	top	A spot inside the page; it stays in your browser

The **path** here looks like a folder path (Chapter 2), and often it maps onto one. That is a habit, not a rule.

Domain names. The host part is usually a **domain name**, a human-friendly name such as `example.org`.

New term: domain name. A human-friendly name, such as `example.org`, that identifies a host on the internet.

The names `example.org` and `example.com` are reserved for use in documentation, so this book uses them and they will never belong to a real business. (This is a well-known convention; the standards that define it have not yet been consulted, see the ledger entry below.)

Verification pending [R122]. That `example.org` is reserved for examples comes from general knowledge and awaits confirmation from the standards documents.

5.2.1 How a name finds a computer

Computers on the internet find each other by numbers called **IP addresses**. People prefer names, so the internet has a service, the **domain name system (DNS)**, that works like a phone book: you give it a name and it gives back the number.

New term: IP address. A number that identifies a computer on a network, so that other computers can send information to it.

New term: DNS (domain name system). The internet's "phone book": a service that turns domain names into IP addresses.

You will rarely need to think about DNS, but you will meet it when you connect your own domain name to a website (Chapter 69).

5.3 Client and server

Almost everything on the internet has two sides.

New term: client. The computer or program that asks for something.

New term: server. A computer (or program) whose job is to answer requests. It waits until a client asks.

Use a restaurant as the picture. You (the client) look at the menu and place an order; the kitchen (the server) prepares it and sends it out. Your browser is a client: it requests a page. A web server responds with the page.

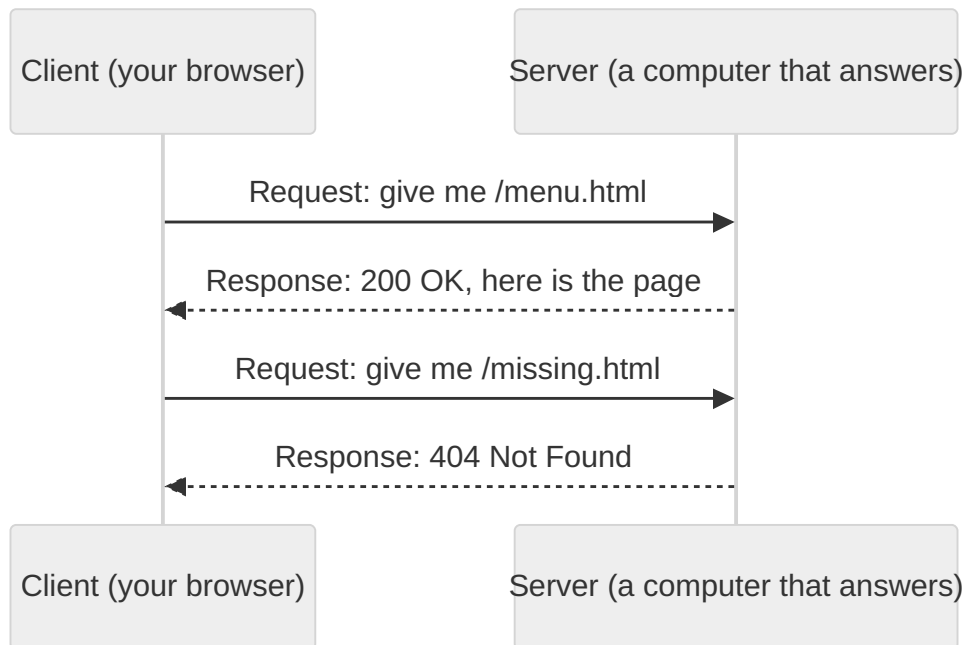


Figure 5. 5.3 Client and server

Diagram description: two participants. The client asks the server for `/menu.html`; the server replies "200 OK" with the page. The client then asks for `/missing.html`; the server replies "404 Not Found".

A "server" is not a special kind of machine. It is a role: any computer running a program that answers requests is acting as a server. To prove it, the recording below runs both sides on one computer. A small web server (a program included with the language Python) serves a folder that holds one page, and a client program called `curl` requests pages from it.

```

$ curl -i http://127.0.0.1:8765/index.html
HTTP/1.0 200 OK
Content-type: text/html
Content-Length: 24

<h1>Sunrise Bakery</h1>
$ curl -i http://127.0.0.1:8765/missing.html
HTTP/1.0 404 File not found
  
```

Example output, recorded by `verification/ch05-internet/local_server_demo.sh`. The address `127.0.0.1` means "this same computer" (section 5.4). Header spelling and the protocol version can differ between versions of the tools.

Read it as a conversation:

1. The client asked for `/index.html`. The first line of the reply is the **status line**: `200 OK`. `200` is the code for "success".
2. Then come a few **headers** (details about the reply: the type of content and its length: 24 bytes), a blank line, and the **body**: the page itself.

3. The client asked for a page that does not exist. The status line says 404: "not found".

New term: HTTP. The set of rules (protocol) that web clients and servers follow to exchange requests and responses.

You have now seen the two codes you will meet most: **200** (success) and **404** (not found). Many other status codes exist. When GitHub or a tool reports one, you can look it up.

Verification pending [R121]. The demonstration ran on a real server and client and shows the behaviour of that pair. The statement that this is HTTP, and the meaning of status codes, come from general knowledge and are yet to be checked against the official standard.

5.4 Local and remote

New term: local. On the computer you are using now.

New term: remote. On a different computer, reached over a network.

127.0.0.1 and the name localhost both mean "this very computer". A file in your sharif-git-lab folder is local. A page on example.org is remote.

Two consequences matter for everything that follows:

- **Local is fast, private and works with no connection.** Remote needs the network and depends on someone else's computer being on.
- **A copy is not the original.** If a project exists on your laptop and on a server, they are two separate copies that can differ.

This is the heart of Git and GitHub. Git keeps the complete history of your project **locally**, on your own computer, so most of your work needs no internet at all. GitHub is a **remote** place where a copy of that history is stored so that other people (and other computers of yours) can reach it. Git's commands for moving history between local and remote (Chapter 23) will make sense because you already know both words.

"The cloud." People say "in the cloud" to mean "on someone else's servers, reached over the internet". There is nothing airy about it: it is computers in buildings.

5.5 Website or web application?

A **website** is a collection of pages that mostly show information. The Sunrise Bakery site is one. Its pages are **static**: every visitor gets the same files, exactly as saved.

A **web application** does work as you use it: it remembers who you are, lets you change things, and builds each page for you from stored data. Online banking, email and GitHub itself are web applications.

	Website (static)	Web application
Pages	The same files for everyone	Built for you as you use it
Needs a program running on the server	No, only files	Yes
Example	The bakery's menu	GitHub

GitHub Pages (Chapter 69) can host static websites like the bakery's. Web applications need more.

5.6 What the "S" in HTTPS means

You will see addresses that begin `https://`. The **S** stands for **secure**.

New term: HTTPS. HTTP with encryption. The traffic between your browser and the server is scrambled so that others on the network cannot read it, and the server has to prove its identity with a certificate.

What HTTPS promises:

1. **Privacy in transit.** People between you and the server cannot read or change what you send.
2. **Identity.** You are talking to the server that owns that name, not to an impostor sitting in the middle.

What HTTPS does **not** promise:

- It does not mean the website is honest or safe. A criminal can also have a genuine certificate for a lookalike name.
- It does not protect you after the information arrives: whoever runs the server can still see what you send.

Security note. A padlock or "secure" label in the address bar means the connection is encrypted. It does not mean the site is trustworthy. Check the name in the address, and be wary of names that are almost, but not quite, the name you expect.

Verification pending [R123]. This description is a general account of HTTPS. It is not yet checked against the standard or a browser vendor's documentation.

Why it matters for Git. When Git talks to GitHub over the web, it can use HTTPS. In Chapter 38 you will choose between that and another method, SSH (introduced in Chapter 6).

5.7 Publishing and privacy

Anything you put on a server that other people can reach can be copied by them. Before you publish, ask three questions:

1. **Who can reach it?** Public means anyone.
 2. **What is in it?** Passwords, keys and personal information must never be in a project that others can read. Chapter 33 returns to this at length, because Git remembers everything that is committed, even after you delete it.
 3. **Could I undo it?** Once a copy has left your computer, treat it as permanently out.
-

5.8 Optional [Deep]: run both sides yourself

Deep. If you have Python installed, you can repeat the recording of section 5.3 after Chapter 7. In a terminal, change into a folder that contains an `index.html`, and run `python3 -m http.server 8765 --bind 127.0.0.1`. That starts a web server on your own computer. Open `http://127.0.0.1:8765/index.html` in your browser: you are now both client and server. Stop the server with `Ctrl+C`. Nothing here is needed later in the book.

Checkpoint

What You Learned

- A network connects computers; the internet connects networks; the web is one service that uses the internet.
- A URL has parts: scheme, host, port, path, query and fragment.
- A domain name is turned into an IP address by DNS.
- A client asks and a server answers; "server" names a role, not a special machine.
- HTTP responses carry a status code (200 success, 404 not found), headers and a body.
- Local means on your computer; remote means on another. Git keeps history locally and can copy it to remote places such as GitHub.
- HTTPS encrypts traffic and verifies the server's identity; it does not certify honesty.
- Assume anything published may be copied.

New Vocabulary

- **Network:** computers connected so that they can exchange information.
- **Internet:** the worldwide system of connected networks.
- **Web (World Wide Web):** linked pages and applications viewed with a browser.
- **Browser:** an application for viewing web pages.
- **URL:** the written address of a resource, made of scheme, host, path and other parts.
- **Domain name:** a human-friendly name that identifies a host, such as `example.org`.
- **IP address:** a number that identifies a computer on a network.
- **DNS (domain name system):** the service that turns domain names into IP addresses.
- **Client:** the computer or program that asks.

- **Server:** the computer or program that answers.
- **HTTP:** the rules web clients and servers use to exchange requests and responses.
- **HTTPS:** HTTP with encryption and server identity checks.
- **Local:** on the computer you are using.
- **Remote:** on a different computer, reached over a network.

Commands Learned

None to type yet. You read the output of `curl -i` requests to a local server.

Common Mistakes

1. **Saying "internet" when you mean "web".** Email, Git and games use the internet too.
2. **Believing a padlock means a site is safe.** It means the connection is encrypted.
3. **Forgetting that a copy on a server is not the same as the file on your laptop.** They can differ.
4. **Treating "server" as a special machine.** Any computer can play the role.
5. **Publishing something private "for a moment".** It may be copied at once.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between the internet and the web?
2. Split `https://example.org/menu/cakes?sort=price#top` into scheme, host, path, query and fragment.
3. Who is the client and who is the server when you open a page in a browser?
4. What does status code 404 mean?
5. Give one thing HTTPS promises and one thing it does not.
6. Which is local: a file in your `sharif-git-lab`, or a page on `example.org`?

Before Moving On

You are ready for Chapter 6 if you can:

- [] read a URL and name its parts
- [] describe the client/server conversation in your own words
- [] explain "local" and "remote" using an example from your own work
- [] say what HTTPS does not promise

How this chapter was checked

Claim	Evidence class	Le dg er
A URL splits into scheme, host, port, path, query and fragment as shown	Locally tested (<code>url_parts.py</code> , re-run in CI); RFC 3986 not consulted	R1 20
A client/server exchange returns 200 with headers and body for an existing page and 404 for a missing one	Locally tested (<code>local_server_demo.sh</code> , re-run in CI); HTTP standard not consulted	R1 21
<code>example.org</code> is reserved for documentation	Needs re-verification	R1 22
What HTTPS does and does not promise	Needs re-verification	R1 23

Where this leads

Chapter 6 explains how a server knows who you are and what you may do, an essential idea before Git connects to GitHub. Chapter 23 uses "local" and "remote" precisely, and Part V introduces GitHub as a remote service.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book.

Level 1 — Guided

1.1 Read an address. In your browser, open any public web page. Copy its address from the address bar into a document and label the scheme, host and path. Expected result: at least three labelled parts.

1.2 Watch the status. Open a page whose address you deliberately mistype (for example add `xyz` to the end of a real page's path). Expected result: a "not found" or "404"-style page. Say which side (client or server) produced that message.

Level 2 — Partially guided

2.1 Client or server? For each item say whether it is acting as client or server: (a) your browser opening a page; (b) the computer that sends the page; (c) a phone app asking for the weather; (d) a friend's laptop running the Python demo of section 5.8 while you connect to it.

2.2 Local or remote? Sort into local and remote: a photo on your phone; a document in an online office suite; a page on `example.org`; `http://localhost:8765/`.

Level 3 — Independent

3.1 Explain to a friend the difference between the internet and the web, using a picture of your own (not roads) as the analogy.

Level 4 — Professional scenario

4.1 A colleague says: "The site has a padlock, so we can trust what it asks for." Write a reply of 4–6 sentences that explains what the padlock does and does not tell you, and lists two things to check instead.

Level 5 — Troubleshooting

5.1 A learner types `https://sunrise-bakery.example.org/menu.html` and sees an error saying the server cannot be found, before any page appears. Which of the ideas in this chapter (domain names, DNS, server, client) would you check first, and in what order?

5.2 A page loads but its pictures are missing, and the browser reports 404 for `images/logo.svg`. What does the code tell you, and what would you check in the project folder? (Chapter 2 will help.)

Chapter 6 — Accounts, Passwords and Trust [Core]

In this chapter

- what an account is, and how a service knows who you are
- why passwords fail, and what to do instead
- the difference between proving who you are (authentication) and being allowed to act (authorization)
- what a token and a key pair are, in plain language
- what SSH is, at the level you need before Git uses it
- how to recognise a phishing attempt and recover from a lost account

Before you start. Chapter 5: you know that a server answers requests from clients, and that HTTPS encrypts the connection but does not prove that a site is honest.

This chapter has no commands. It teaches the ideas that GitHub will ask you to use in Chapter 38, and that Chapter 33 will need when it talks about leaked secrets. Read it slowly: security ideas are much easier to learn now than after something has gone wrong.

6.1 Accounts

Chapter 5 showed that a server answers requests from anyone who asks. That is fine for a public menu. It is not fine for your bank, your email or your projects. Those services need to know **who is asking**, and what that person is allowed to do.

New term: account. A record kept by a service that represents you: your identity, your settings and your permissions.

New term: username. The name that identifies your account on a service. It is usually public.

Your **username** and your **display name** are different things. The username is a unique label (sharif-example); the display name is the friendly name shown next to your posts. Choose your username carefully for anything professional, because on some services it appears in every web address that belongs to you, and changing it later can break links. (Chapter 37 returns to this for GitHub.)

An account is the first of two ideas you will use constantly. The second is what the service does when you try to use it: it asks you to prove that you are the owner.

6.2 Passwords, and why they fail

New term: password. A secret string of characters that you present to prove that you own an account.

A password is something you know. It is simple, and simple is why it is so often misused. Passwords fail in five common ways:

Failure	What happens
Guessing	Short or common passwords are found quickly by trying likely words
Reuse	The same password on many services: one leak opens all of them
Phishing	A fake site or message tricks you into typing your password to a criminal
Leaks	A service is broken into and its stored passwords are stolen
Sharing	You give your password to someone, who gives it to someone else

The habits that defeat most of these are simple:

1. **Use a different password for every service.** Then a leak on one does not spill onto the others.
2. **Make each password long and unpredictable.** Length matters more than clever symbols. A few unrelated words strung together (a passphrase) can be long and still memorable.
3. **Let a tool remember them.** No person can memorise dozens of long, unique passwords.

New term: password manager. An application that stores all of your passwords in an encrypted store, protected by one strong password, and fills them in for you.

A password manager turns the impossible task of remembering many secrets into the manageable task of remembering one. Choose one from a maker you can verify, and protect its master password very carefully.

Security note. Never store passwords in a plain text file, a spreadsheet, a note on your phone, or in a project that you keep with Git. In Chapter 33 you will see why files committed to Git can never be considered truly deleted.

6.3 More than one proof: two-factor authentication

A password can be stolen. So services let you add a **second proof**, taken from a different family. There are three families of proof:

Family	Meaning	Example
Something you know	A secret in your head	A password or PIN
Something you have	An object that only you hold	A phone that shows a code, or a physical security key
Something you are	A body feature	A fingerprint or face scan

New term: two-factor authentication (2FA). Proving who you are with two proofs from different families, for example a password and a code from your phone.

With 2FA, a thief who steals your password still cannot get in, because they do not hold your phone. The extra step is a small nuisance that removes a very large risk.

Second factors are not all equally strong. General security guidance ranks a physical security key or an app-generated code above a code sent by text message, because text messages can be intercepted or redirected in ways that a key in your pocket cannot. Use the strongest second factor that the service offers and that you can manage.

Verification pending [R124]. The ranking of second factors and the recommendations in this chapter are general security guidance. They will be checked against the published guidance of a standards body and the services' own documentation before release. No statement about a particular service (for example, whether GitHub requires 2FA) is made here.

6.3.1 Recovery: plan for losing the second factor

Every 2FA system needs a way back in if you lose the phone. Services normally provide **recovery codes**, one-time codes to be stored safely before you need them. Print them or keep them in your password manager, and keep them away from your phone. Add a backup email address or a second key where the service allows.

Set up recovery **on the day you switch 2FA on**. Recovering an account without preparation can be slow or impossible, because a service that can let you in without proof could let an attacker in too.

6.4 Authentication and authorization

Two words sound alike, and beginners mix them up. Learn them as a pair.

New term: authentication. Proving who you are.

New term: authorization. Deciding what you are allowed to do once your identity is known.

The picture that helps is a hotel.

- At reception you show your passport and booking: **authentication**. The hotel now knows you are the guest in room 12.
- The key card you receive opens room 12, the gym and the front door, but **not** room 13 or the manager's office: **authorization**.

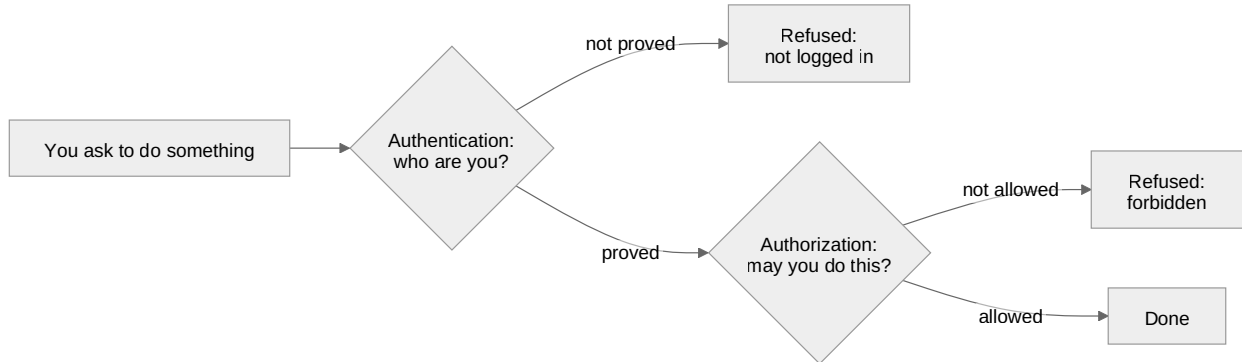


Figure 6. 6.4 Authentication and authorization

Diagram description: a request first meets an identity check. If your identity is not proved, the request is refused as "not logged in". If it is proved, a second check asks whether you may do this particular thing. If not, the request is refused as "forbidden". If yes, it is carried out.

The two failures feel different and have different cures. "I cannot log in" is an authentication problem: check the password, the second factor and the account name. "I am logged in but it says I do not have permission" is an authorization problem: nothing is wrong with your identity; you need someone to grant you access. In [Chapter 38](#) and [Chapter 49](#) you will meet both problems on GitHub.

6.5 Least privilege

New term: least privilege. Giving each person or program only the access it needs to do its job, and no more.

You would not give a new employee the keys to every room in the building. In the same way, do not give a tool or person more access than they need, and remove access when it is no longer needed. Least privilege limits the damage when something goes wrong: a stolen key that can only open one door is far less dangerous than a stolen master key. You will use this principle throughout the book, especially for tokens and automation.

6.6 Tokens: a limited stand-in for a password

Sometimes a program, not a person, has to act on your behalf: a script, or a tool such as Git. You do not want to hand a program your password, which opens everything and never expires.

New term: token. A secret string that stands in for a password, usually with limited permissions and, often, an expiry date.

A token is like a temporary visitor pass instead of your house key. It can be limited to certain rooms and cancelled without changing the locks. When we come to GitHub (Chapter 38), you will meet tokens that are made to be used by tools.

A token is **still a secret**. Whoever holds it can do what it allows. Treat it like a password: never post it, never commit it into a project, and cancel it at once if you suspect it has leaked.

6.7 Keys: proving identity without sending a secret

A password has a weakness: to prove that you know it, you send it to the other side. If someone is listening, or if the other side is dishonest, the secret is out.

There is a cleverer arrangement, used for the login method called **SSH** and for signing your work. It uses a **key pair**: two linked keys, made together.

New term: key pair. Two mathematically linked keys, a public key and a private key, created together. Something locked with one can be unlocked only with the other.

- The **public key** can be shared with anyone. Think of it as an open padlock. You hand copies to everyone.
- The **private key** stays on your computer and is never shared. Think of it as the only key that opens that padlock.

To prove who you are, you do not send the private key. The server, which holds your open padlock (public key), can set a puzzle that only the private key can solve. You solve it on your own computer and return the answer. The server is satisfied, and the private key never left your machine.

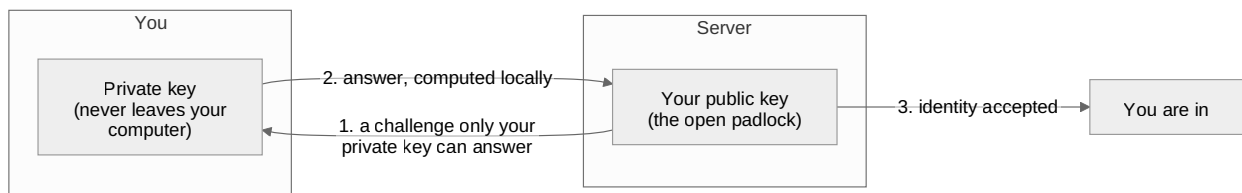


Figure 7. 6.7 Keys: proving identity without sending a secret

Diagram description: the server holds your public key. It sends a challenge. Your private key, which stays on your computer, computes the answer and sends only the answer back. The server checks the answer against your public key and accepts you.

New term: SSH (Secure Shell). A method of connecting securely to another computer over a network. It can use a key pair to prove who you are, so no password is sent.

CAUTION. Sharing the **public** key is safe and normal. Sharing the **private** key is like handing out your master key. If a private key is ever posted, committed to a project or emailed, treat it as stolen: cancel it and make a new pair. Nothing in this book will ever ask you to reveal a private key.

You will generate your first key pair in Chapter 38, with the exact steps and checks, and you will learn why a private key file must have restrictive permissions (Chapter 2).

6.8 Recognising phishing

New term: phishing. Tricking someone into giving away a secret or doing something harmful by pretending to be a trusted person or service.

Phishing does not break the computer; it breaks the person. Typical signs:

1. **Urgency or fear.** "Your account will be closed in one hour."
2. **A link that is almost right.** sunrise-bakery-login.example.net is not sunrise-bakery.example.org. Look at the host part of the address (Chapter 5).
3. **A request for a secret.** No real service will ask for your password, code or private key by email or message.
4. **A surprise.** An attachment or invoice that you were not expecting.

Here is an invented, fictional message. Read it as a detective.

Subject: URGENT security notice Dear customer, unusual activity was detected. Log in within one hour at <https://sunrise-bakery-login.example.net/verify> or your account will be deleted. Reply with your password to confirm.

At least four warning signs are present: urgency; a threat; a link on an unexpected domain; and a request for a password by reply. **Do not click.** Open the service by typing its real address yourself, or use a bookmark, and look there for any genuine notice.

If you have entered a password on a suspicious page: change it at once on the real site, change it anywhere else you used it (another reason not to reuse passwords), check the account's recent activity, and turn on 2FA if it is not already on.

6.9 What this means for Git and GitHub

Every idea in this chapter returns:

Idea	Where it returns
Account, username	Chapter 37
Password, 2FA, recovery	Chapter 38

Idea	Where it returns
Authentication versus authorization	Chapter 49 and Chapter 50
Token	Chapter 38 and Chapter 60
Key pair and SSH	Chapter 38
Least privilege	Chapter 63
Leaked secrets	Chapter 33

You do not need to remember these now. You need to remember the shape of the ideas, so that when GitHub asks you to make a token or a key, it does not feel like magic.

Checkpoint

What You Learned

- An account represents you on a service; a username identifies it.
- Passwords fail through guessing, reuse, phishing, leaks and sharing. Use unique, long passwords kept in a password manager.
- Two-factor authentication adds a second proof from a different family; plan recovery when you switch it on.
- Authentication proves who you are; authorization decides what you may do. They fail differently.
- Least privilege means giving only the access that is needed.
- A token is a limited stand-in for a password and is still a secret.
- A key pair has a shareable public key and a private key that never leaves your computer; SSH can use one to prove your identity without sending a secret.
- Phishing tricks people; look for urgency, odd domains and requests for secrets.

New Vocabulary

- **Account:** a record a service keeps to represent you.
- **Username:** the name that identifies your account.
- **Password:** a secret string that proves you own an account.
- **Password manager:** an application that stores your passwords securely.
- **Two-factor authentication (2FA):** proving who you are with two proofs from different families.
- **Authentication:** proving who you are.
- **Authorization:** deciding what you are allowed to do.
- **Least privilege:** giving only the access that is needed.
- **Token:** a limited stand-in for a password.
- **Key pair:** a linked public key and private key.
- **SSH (Secure Shell):** a secure way to connect to another computer, which can use a key pair.
- **Phishing:** tricking someone into giving away a secret by pretending to be trusted.

Commands Learned

None.

Common Mistakes

1. **Reusing one password everywhere.**
2. **Switching on 2FA without saving recovery codes.**
3. **Mixing up authentication and authorization:** "cannot log in" and "not allowed" are different problems.
4. **Sharing a private key, or a token, "just this once".**
5. **Clicking a link in an urgent message** instead of going to the site yourself.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name two ways a password can be exposed even if you chose it well.
2. In the hotel picture, which step is authentication and which is authorization?
3. What is the difference between a public key and a private key? Which may be shared?
4. Why is a token safer than giving a program your password, and why is it still a secret?
5. List three warning signs in the phishing message of section 6.8.
6. Why should recovery codes be saved before they are needed?

Before Moving On

You are ready for Chapter 7 if you can:

- [] explain authentication and authorization with your own example
- [] say what a password manager is for
- [] describe, in two sentences, how a key pair proves your identity without sending a secret
- [] name the danger of committing a token or private key into a project

How this chapter was checked

Claim	Evidence class	Le dg er
Definitions and recommendations in this chapter (account, password hygiene, 2FA families and ranking, authentication vs authorization, least privilege, tokens, key pairs, SSH, phishing)	Needs re-verification against standards-body guidance (general security concepts; no product-specific claim is made)	R1 24

Where this leads

Chapter 7 gives you the terminal. Chapter 38 sets up real credentials for GitHub using the ideas of this chapter. Chapter 33 shows why secrets must never be committed, and what to do if one is.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Never type a real password, code or key into any exercise document.

Level 1 — Guided

1.1 Audit one account. Choose one important account of your own (for example email). Without writing down any secret, record: (a) whether two-factor authentication is on; (b) what second factor is used; (c) whether you have saved recovery codes. Expected result: three short answers.

1.2 Read a password rule. A service says "Passwords must be at least 8 characters with a capital letter and a number". Explain, in two sentences, why a 20-character passphrase of four unrelated words might still be a stronger choice.

Level 2 — Partially guided

2.1 Authentication or authorization? For each, say which idea it is and give the cure. (a) "Incorrect password." (b) "You do not have permission to delete this file." (c) "Enter the code sent to your phone." (d) "This repository is private."

2.2 The hotel. Draw your own analogy for authentication and authorization using a school, a library or a gym.

Level 3 — Independent

3.1 Explain to a friend how a key pair proves your identity without sending a secret. Use the padlock picture, but do not use the words "private" or "public".

Level 4 — Professional scenario

4.1 A three-person team will share one tool account that has access to all of the company's projects. Two of them are contractors. Write a short plan that applies least privilege and does not share passwords. Say what each person should have and what you would do when a contractor leaves.

Level 5 — Troubleshooting

5.1 A colleague says: "I can log in, but the page says I am not allowed to change the settings." Decide whether this is an authentication or authorization problem, and say who can fix it.

5.2 A learner lost their phone and cannot get the 2FA codes for their account. They did not save recovery codes. List the steps you would suggest, in order, and one thing to do differently next time.

Chapter 7 — The Terminal Without Fear [Core]

In this chapter

- what a terminal, a shell, a prompt and a command are
- the environments you will meet: Command Prompt, PowerShell, Git Bash, Bash and zsh, and why this book uses one primary path
- your first fifteen commands, run in a safe practice folder
- how to read error messages, and why they are your friends
- spaces, quotes, wildcards, variables and the PATH, the ideas behind the famous error "command not found"

Before you start. Chapter 2 (paths, hidden files, permissions) and Chapter 4 (bytes, line endings). You will use the practice folder `sharif-git-lab` from Chapter 2.

Nothing in this chapter can harm your computer if you follow one rule, which is repeated where it matters: **only create and delete things inside `sharif-git-lab`.**

7.1 The idea

Until now you have given your computer instructions by pointing and clicking. There is a second way: typing sentences.

New term: terminal. A window in which you type commands and read the replies as text. (Also called a terminal emulator or console.)

New term: shell. The program that runs inside the terminal, reads what you type, and asks the operating system to carry it out.

New term: command. One instruction typed to the shell, ended by pressing the Enter key.

New term: prompt. The short piece of text the shell prints when it is waiting for you to type. It often ends with `$`, `%` or `>`.

An analogy: the terminal is the telephone handset; the shell is the person who answers and understands what you say; the command is your request. The shell answers in plain text.

Why bother? Four reasons, all of which matter for Git:

1. **Precision.** A typed command says exactly what you want, with no hunting through menus.
2. **Repeatability.** You can save a list of commands and run it again, or share it.

3. **Speed for repeated work.** One line can act on a hundred files.
4. **Git's home.** Git began as a set of commands. Most of its power, and all of its documentation, use the terminal. Graphical Git tools are wrappers around the same commands, and they hide what is really happening.

Nothing happens until you press Enter. You can type, edit and change your mind. And you can almost always stop a running command by holding the Control key and pressing C (written Ctrl+C).

7.2 Terminal, shell, and the environments you will meet

The words "terminal" and "shell" are used loosely, and different systems supply different shells. The differences are the reason for this section.

Environment	What it is	Where you meet it
Command Prompt (cmd)	The traditional Windows command shell	Windows
PowerShell	A newer, more powerful Windows shell with its own command names	Windows (and available elsewhere)
Git Bash	A terminal that provides a Bash shell and a set of Unix-style tools on Windows; it is installed together with Git for Windows	Windows, after installing Git
Bash	A very widely used shell for Linux and, historically, macOS	Linux, macOS, Git Bash
zsh	Another shell in the same family as Bash, with many small differences	macOS (see the note below), and Linux

New term: Bash. A widely used shell, the one this book uses for its examples.

New term: Git Bash. A program for Windows that provides a Bash shell and Unix-style commands, installed with Git for Windows.

Git is not Git Bash. Git is the version-control program, the subject of this book. Git Bash is a separate shell that is bundled with the Windows installer so that Windows users can type the same commands as everyone else. You can run Git from Command Prompt, PowerShell, Git Bash, zsh or Bash. They are different things that happen to be installed together.

Partly checked [R126]. The Git for Windows project's own README confirms that the Git for Windows package provides `git-bash.exe` (a Bash window) and `git-cmd.exe` (a Command Prompt window) as separate programs. That the installer bundles them for ordinary users, and how Git Bash writes drive paths such as `/c/Users/name`, is from general knowledge and has not been checked.

7.2.1 The book's primary path

To spare you from learning several dialects at once, this book uses **one primary environment**:

Your computer	Use this	Notes
Windows	Git Bash	Installed with Git for Windows (see below)
macOS	The Terminal application	Its default shell is believed to be <code>zsh</code> ; the commands in this chapter work the same in Bash and <code>zsh</code> , and differences are marked
Linux	Your distribution's terminal	Usually Bash

Verification pending [R128]. That macOS's Terminal starts `zsh` by default is from general knowledge and awaits confirmation from Apple's documentation. If your Terminal shows a Bash prompt instead, everything in this chapter still works.

Windows: getting Git Bash now. Git Bash arrives with Git for Windows, which Chapter 13 installs. You can install Git for Windows now to get Git Bash and follow this chapter exactly; installing Git early is harmless. If you would rather wait, you can follow this chapter in PowerShell or Command Prompt using the equivalence table in section 7.11, and then redo the tour in Git Bash after Chapter 13. Git Bash is the environment the rest of the book assumes.

7.2.2 What the two tested shells do differently

Every example in this chapter was run in **Bash 5.2** and in **`zsh` 5.9**, in real interactive sessions, and the recordings are checked automatically. For the basics they behave the same. The few places where they differ are called out in the text, always with real output. Where Windows shells materially differ, the equivalent commands are given in section 7.11, marked as not run.

7.3 The anatomy of a command

A command is a sentence: a verb and, usually, some details.

```
ls -l notes.txt
```

Part	In the example	Meaning
Program (the command name)	<code>ls</code>	What to do: list files

Part	In the example	Meaning
Option (also called a flag)	-l	How to do it: use the long, detailed format
Argument	notes.txt	What to do it to

Parts are separated by **spaces**, so spaces matter. Names are **case-sensitive** in Bash and zsh: `ls` works, `LS` does not.

The \$ in this book. In the examples, a line that starts with \$ shows a command that you type, without the \$. The \$ stands for the prompt. Lines below it, with no \$, are what the computer printed. Your own prompt will look different: it may show your user name, the computer's name or the current folder.

New term: option (flag). A word, usually starting with a dash, that changes how a command behaves.

New term: argument. The thing that a command acts on, such as a file name.

7.4 Open a terminal and start in the practice folder

UI-VERSION NOTE. How to open a terminal depends on your system and its version. On macOS and Linux, search your applications for "Terminal". On Windows, after Git for Windows is installed, search for "Git Bash". If you cannot find it, search your system's help for "open a terminal".

When a terminal opens, you are normally in your **home folder** (Chapter 2). The recordings in this chapter start there and use the fictional user name `learner`, so your paths will differ: `/home/learner` will be your own home folder, and on Windows, Git Bash is expected to show Windows drives in a Unix style, such as `/c/Users/yourname` (this drive style was not checked; see the note in section 7.2).

7.5 Your first tour

Work through this tour by typing each command yourself, one at a time, and comparing your result with the recording. **Type each command exactly, then press Enter.**

7.5.1 Where am I? (`pwd`)

New term: `pwd`. Short for "print working directory": prints the current directory, that is, where you are in the folder tree.

```
$ pwd
/home/learner
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

/home/learner is an **absolute path** (Chapter 2) and it is the home folder of this practice user.

7.5.2 Make a folder and step into it (mkdir, cd)

New term: mkdir. Short for "make directory": creates a new folder.

New term: cd. Short for "change directory": moves you into another folder, making it the current directory.

```
$ mkdir sharif-git-lab
$ cd sharif-git-lab
$ pwd
/home/learner/sharif-git-lab
$ ls
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

mkdir sharif-git-lab printed nothing: **most commands print nothing when they succeed**. Silence usually means success. cd sharif-git-lab moved us in, pwd confirmed it, and ls (list) printed nothing because the folder is empty.

7.5.3 Build the bakery tree (mkdir -p, ls)

The option -p means "create the parent folders too, and do not complain if they exist". One command builds the whole tree from Chapter 3:

```
$ mkdir -p sunrise-bakery/css sunrise-bakery/images
$ ls
sunrise-bakery
$ ls sunrise-bakery
css  images
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

Here the second ls names a folder, sunrise-bakery, as its argument, so it lists what is inside it: css and images. **In your own terminal ls prints its names in columns, as here; in this book's tables and lists the same names are sometimes shown one per line.**

7.5.4 Make a file (echo, >, >>, cat)

```
$ cd sunrise-bakery
$ pwd
/home/learner/sharif-git-lab/sunrise-bakery
$ echo "Sunrise Bakery" > README.md
$ cat README.md
Sunrise Bakery
```



```
$ echo "Fresh bread every morning." >> README.md
$ cat README.md
Sunrise Bakery
Fresh bread every morning.
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

- `echo` prints the words you give it.
- **>** **redirects** that output into a file instead of the screen, replacing whatever the file held. The file `README.md` did not exist, so it was created.
- `cat` prints a file's contents to the screen ("concatenate", but you will use it to show files).
- `>>` redirects too, but **appends** to the end instead of replacing.

CAUTION. A single `>` overwrites the file without asking and without a trash to recover from. Use `>>` when you mean "add to the end". Before using `>` on an existing file, be sure you do not need what is in it.

7.5.5 Copy and rename (`cp`, `mv`)

```
$ cp README.md README-copy.md
$ ls
README-copy.md README.md css images
$ mv README-copy.md notes.txt
$ ls
README.md css images notes.txt
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

`cp` source destination makes a copy; `mv` source destination moves, and **renaming is moving to a new name in the same place**, which is why there is no separate "rename" command. After `mv`, `README-copy.md` is gone and `notes.txt` exists, containing the same text.

7.5.6 Hidden files (`touch`, `ls -a`)

New term: touch. Creates an empty file, or updates a file's "last changed" time if it already exists.

```
$ touch .hidden-note
$ ls
README.md css images notes.txt
$ ls -a
. .. .hidden-note README.md css images notes.txt
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

`touch .hidden-note` created a file whose name starts with a dot. Plain `ls` does not list it. `ls -a` ("all") shows it, together with two special entries: `.` (this folder) and `..` (the parent), which you met in Chapter 2. This is exactly how Git's hidden `.git` folder will behave.

7.5.7 The long listing (`ls -l`)

```
$ ls -l notes.txt
-rw-r--r-- 1 learner learner 42 <date> notes.txt
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

Read it from left to right: the ten characters `-rw-r--r--` are the **permissions** (Chapter 2): a leading `-` means an ordinary file, then read/write for the owner, and read-only for others. `1` is a count of links, `learner learner` are the owner and group, **42 is the size in bytes**, `<date>` stands for the date and time, and last comes the name. The size is 42 because the file holds `Sunrise Bakery` plus a line break (15 bytes) and `Fresh bread every morning.` plus a line break (27 bytes). This is Chapter 4 in action.

Verification pending [R116]. The permission string above was produced by real tools on Linux. The exact form of `ls -l` differs a little between systems.

7.5.8 Moving around (`cd ..`, `cd -`, `cd ~`)

```
$ cd ..
$ pwd
/home/learner/sharif-git-lab
$ cd -
/home/learner/sharif-git-lab/sunrise-bakery
$ cd ~
$ pwd
/home/learner
$ cd sharif-git-lab/sunrise-bakery
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

- `cd ..` goes up one level to the parent folder.
- `cd -` jumps back to the folder you were in before, and prints it.
- `cd ~` goes to your home folder from anywhere. `~` is shorthand for your home folder.
- `cd sharif-git-lab/sunrise-bakery` used a **relative path** (from the home folder) to get back.

In zsh, `cd -` prints the folder with `~` abbreviating the home folder (`~/sharif-git-lab/sunrise-bakery`). It is the same folder, written differently.

7.5.9 Deleting, carefully (`rm`, `rmdir`)

CAUTION. There is no trash bin in the terminal. `rm` deletes immediately and permanently. Before you press Enter on any `rm`, read the command aloud and check where you are (`pwd`) and what the name matches (`ls`). Never run a command that you copied from a website or a chat if you do not understand every word.

```
$ rm notes.txt
$ ls
README.md  css  images
```

```
$ rm css
rm: cannot remove 'css': Is a directory
$ rmdir css
$ rmdir images
$ ls
README.md
```

Recorded in Bash; *ch07-terminal/expected-main.bash.txt*.

- `rm notes.txt` removed the file. No message: success.
- `rm css` **refused**, because `css` is a folder, and printed the error `Is a directory`. That refusal is a safety feature: it stopped you from deleting a folder by reflex.
- `rmdir` removes an **empty** folder, and only an empty one (so it is a safe way to remove folders you have finished with). Here it is refused on a folder that still holds a file:

```
$ mkdir full
$ touch full/x.txt
$ rmdir full
rmdir: failed to remove 'full': Directory not empty
$ ls full
x.txt
```

Recorded in Bash and zsh (identical); *ch07-terminal/expected-errors.bash.txt*.

There is an option that deletes a folder and everything in it. This book will not use it until Git recovery gives you a safety net, and you should treat it as the most dangerous ordinary command that you will meet.

7.6 Reading error messages

Errors are the shell explaining, in words, what it could not do. Read them slowly, from the start. One real one appeared above (`Is a directory`). More are recorded below, from a session in which the first four commands were deliberately wrong. Bash first:

```
$ mkdir sharif-git-lab
$ cd sharif-git-lab
$ cd nowhere
bash: cd: nowhere: No such file or directory
$ ls missing-folder
ls: cannot access 'missing-folder': No such file or directory
$ cat missing-file.txt
cat: missing-file.txt: No such file or directory
$ frobnicate
bash: frobnicate: command not found
```

Recorded in Bash; *ch07-terminal/expected-errors.bash.txt*.

Now the same four mistakes in zsh:

```
$ mkdir sharif-git-lab
$ cd sharif-git-lab
$ cd nowhere
```

```
cd: no such file or directory: nowhere
$ ls missing-folder
ls: cannot access 'missing-folder': No such file or directory
$ cat missing-file.txt
cat: missing-file.txt: No such file or directory
$ frobnicate
zsh: command not found: frobnicate
```

Recorded in zsh; *ch07-terminal/expected-errors.zsh.txt*.

Message	Plain meaning	Usual cause
No such file or directory (Bash: cd: nowhere: ...; zsh: cd: no such file or directory: nowhere)	The path you gave does not exist	A typo, the wrong current folder, or a missing folder
command not found	The shell has no program of that name	A typo, or the program is not installed, or is not on the PATH (section 7.8)
Is a directory	You used a file-only command on a folder	Use the folder version, or the right option
Permission denied (not recorded here)	The operating system refused because of permissions	See Chapter 2; you may need a different account or a different location

The habit: read the first line, find the name it complains about, and check that name against what you meant to type. Most errors are typos.

Note how **the wording differs by shell** although the meaning is the same. Books, forum answers and your own screen may therefore show different words for the same mistake. When you search for help, search for the meaning.

7.7 Spaces, quotes and wildcards

Because spaces separate the parts of a command, a folder with a space in its name is awkward. This recording shows the classic mistake, and its repair:

```
$ mkdir my folder
$ ls
folder  my
$ rmdir my folder
$ mkdir "my folder"
$ ls
'my folder'
$ rmdir "my folder"
```

Recorded in Bash; *ch07-terminal/expected-errors.bash.txt*.

`mkdir my folder` made **two** folders, `my` and `folder`, because the space split the command into two arguments. Putting the name in **quotes** keeps it together, and `mkdir "my folder"` made one. This is why Chapter 2 advised names without spaces. (`rmdir my folder` removed both empty folders: the same splitting applied.)

Wildcards let one pattern match many names. `*` stands for "any characters". Here `ls *.md` matched the two files that end in `.md` (Markdown files, Chapter 8) but not `c.txt`:

```
$ touch a.md b.md c.txt
$ ls *.md
a.md  b.md
```

Recorded in Bash; *ch07-terminal/expected-errors.bash.txt*.

A wildcard that matches nothing behaves differently in the two shells:

```
$ ls *.xyz
ls: cannot access '*.xyz': No such file or directory
```

Recorded in Bash; *ch07-terminal/expected-errors.bash.txt*.

In zsh the same command says:

```
$ ls *.xyz
zsh: no matches found: *.xyz
```

Recorded in zsh; *ch07-terminal/expected-errors.zsh.txt*.

Bash passes the pattern on unchanged and `ls` complains that no such file exists; zsh refuses earlier, at the pattern.

7.8 Variables, the environment and the `PATH`

7.8.1 Variables

A **variable** is a named place that holds text. Set one with `name=value` (no spaces around the equals sign), and read it by putting a `$` before the name.

New term: variable. A named piece of text that the shell remembers, which you can use in later commands.

```
$ name="Sunrise Bakery"
$ echo "$name"
Sunrise Bakery
$ echo '$name'
$name
$ echo $name
Sunrise Bakery
```

Recorded in Bash; *ch07-terminal/expected-vars.bash.txt*.

- `echo "$name"` printed the value, because double quotes let `$name` be replaced by its value. **Single quotes do not**, so `'$name'` printed the literal text `$name`. The unquoted `echo $name` also worked here, but unquoted variables are split at spaces, so **use double quotes whenever a value may contain spaces**.

7.8.2 The environment

By default a variable belongs only to the shell that made it. A variable becomes part of the **environment**, visible to programs that the shell starts, only if you export it. The recording proves it by starting a second, separate shell (`sh -c`) and asking it to print each variable:

```
$ export SHOP=sunrise
$ sh -c 'echo "shop is: $SHOP"'
shop is: sunrise
$ UNEXPORTED=hidden
$ sh -c 'echo "unexported is: [$UNEXPORTED]"'
unexported is: []
```

Recorded in Bash; *ch07-terminal/expected-vars.bash.txt*.

`SHOP` was exported, so the child saw it; `UNEXPORTED` was not, so the child saw nothing (the empty brackets).

New term: environment variable. A variable that has been exported, so that programs started from the shell can read it.

Some environment variables exist from the start. `HOME` holds your home folder; `~` is its shorthand:

```
$ echo $HOME
/home/learner
$ echo ~
/home/learner
```

Recorded in Bash; *ch07-terminal/expected-vars.bash.txt*.

This is how programs, including Git, learn things about the computer and the user, and why **the same command can behave differently on two computers**: their environments differ. Chapter 14 shows the environment changing the behaviour of Git itself.

7.8.3 The `PATH`

When you type `ls`, how does the shell know where the program is? It searches a list of folders held in an environment variable called **`PATH`**, in order, and runs the first program with that name that it finds.

New term: `PATH` (environment variable). An environment variable that lists the folders in which the shell looks for programs when you type a command name.

The recording creates a tiny program called `hello-sunrise` in a folder called `bin`, then tries to run it by name:

```
$ mkdir bin
$ printf '#!/bin/sh\neco "Hello from the Sunrise Bakery tool"\n' > bin/hello-sunrise
$ chmod +x bin/hello-sunrise
$ hello-sunrise
bash: hello-sunrise: command not found
$ PATH="$PWD/bin:$PATH"
$ hello-sunrise
Hello from the Sunrise Bakery tool
$ command -v hello-sunrise
/home/learner/bin/hello-sunrise
```

Recorded in Bash; *ch07-terminal/expected-vars.bash.txt*.

Read what happened:

1. `hello-sunrise` alone failed with `command not found`, although the program exists. The shell searched every folder in `PATH`, and `bin` is not among them.
2. `PATH="$PWD/bin:$PATH"` put our folder at the front of the list (`$PWD` holds the current directory).
3. Now the same command works, and `command -v hello-sunrise` shows where the shell found it.

The `zsh` recording differs only in the wording of the error: `zsh: command not found: hello-sunrise`.

This is the reason behind one of the most common beginner errors. After you install a program, typing its name gives `command not found` (or, in Windows shells, a message that the name "is not recognized") when the program's folder is not on the `PATH`, or when the terminal was opened before the installation. The cure is to fix the `PATH`, or to close and reopen the terminal. Chapter 13 will show how to check.

Partly checked [R129]. Microsoft's PowerShell documentation shows the message The term 'NonsenseString' is not recognized as the name of a cmdlet, function, script file, or operable program. for a program that is not found. The wording of the Command Prompt message was not checked, and wording can change between versions.

7.9 Getting help

- Many programs print a summary if you add `--help`. This is a convention, not a rule; some programs use `-h`, some use neither.
 - Git has its own built-in help, which Chapter 13 and Chapter 35 introduce.
 - When you do not know a command's name, describe what you want in a search and read the official documentation of the tool.
-

7.10 Habits that keep you safe

1. **Read before you press Enter.** Say the command to yourself in words.
2. **Check where you are** (`pwd`) and what is there (`ls`) before deleting or overwriting.
3. **Copy commands only when you understand them.** A command pasted from the web can do anything you can do, including harm.
4. **Stay in the practice folder** while learning.
5. **Use the Up arrow to recall** the previous command, and **Tab to complete** names. Most shells support both; they save typing and prevent typos.
6. **When something is wrong, stop and read the message.**

Security note. A common attack is a web page or message that tells you to "just paste this command". Treat a pasted command like an installer from a stranger (Chapter 1): if you do not understand it, do not run it.

7.11 Windows without Git Bash: equivalents

If you are using PowerShell or Command Prompt, these are the equivalents of the commands in this chapter. **They are not recorded or checked;** the Windows environments could not be run in the book's test environment.

Partly checked [R127]. The PowerShell cmdlet names and the aliases shown in the PowerShell column (`pwd`, `ls`, `cd`, `cat`, `cp`, `mv`, `rm`, `ni`) were checked against Microsoft's PowerShell 7.5 reference (the `MicrosoftDocs/PowerShell-Docs` repository). Nothing was run on Windows, and the Command Prompt column was not checked against Microsoft's documentation. Check each entry on your own machine; the `--help-style` summary for Command Prompt is `command /?`, and PowerShell has `Get-Help`. Windows PowerShell 5.1 can differ from PowerShell 7.

Task	Git Bash / Bash / zsh	PowerShell	Command Prompt
Where am I?	<code>pwd</code>	<code>Get-Location</code> (alias <code>pwd</code>)	<code>cd</code> (with no argument)
List files	<code>ls</code>	<code>Get-ChildItem</code> (alias <code>ls</code>)	<code>dir</code>
Change folder	<code>cd folder</code>	<code>Set-Location folder</code> (alias <code>cd</code>)	<code>cd folder</code>
Make a folder	<code>mkdir folder</code>	<code>New-Item -ItemType Directory folder</code> (alias <code>ni</code>)	<code>mkdir folder</code>
Show a file	<code>cat file</code>	<code>Get-Content file</code> (alias <code>cat</code>)	<code>type file</code>
Copy	<code>cp a b</code>	<code>Copy-Item a b</code> (alias <code>cp</code>)	<code>copy a b</code>
Move / rename	<code>mv a b</code>	<code>Move-Item a b</code> (alias <code>mv</code>)	<code>move a b</code> or <code>ren a b</code>

Task	Git Bash / Bash / zsh	PowerShell	Command Prompt
Delete a file	<code>rm file</code>	<code>Remove-Item file (alias rm)</code>	<code>del file</code>
Show a variable	<code>echo \$HOME</code>	<code>\$env:USERPROFILE</code>	<code>echo %USERPROFILE%</code>

The book's later chapters use Git commands, which are identical in every shell. Only the ordinary file commands differ.

7.12 What you now have

You can now move around the computer, create, copy, move and delete things in the practice folder, read error messages, quote a name with spaces, and explain PATH. That is enough to run every Git command in Part III.

Checkpoint

What You Learned

- A terminal is a window for typing commands; the shell is the program that reads them; the prompt shows it is waiting.
- This book's primary environment is Bash (Git Bash on Windows); Git is not Git Bash.
- A command has a program, options and arguments, separated by spaces and case-sensitive.
- `pwd`, `ls`, `cd`, `mkdir`, `echo`, `cat`, `cp`, `mv`, `touch`, `rm`, `rmdir` and the redirections `>` and `>>`.
- Most commands print nothing on success; errors are readable messages, worded differently by different shells.
- Quotes keep spaces in one argument; wildcards match many names.
- Variables, the environment and PATH explain "command not found" and why behaviour can differ between computers.
- There is no trash bin: deleting is permanent.

New Vocabulary

- **Terminal**: a window for typing commands and reading replies.
- **Shell**: the program that reads your commands and has them carried out.
- **Command**: one instruction typed to the shell.
- **Prompt**: the text the shell shows when it waits for input.
- **Bash**: a widely used shell; this book's primary shell.
- **Git Bash**: a Bash environment for Windows, installed with Git for Windows.
- **Option (flag)**: a word that changes how a command behaves.
- **Argument**: what a command acts on.
- **Variable**: a named piece of text the shell remembers.
- **Environment variable**: an exported variable that started programs can read.
- **PATH (environment variable)**: the list of folders searched for programs.

Commands Learned

`pwd`, `ls`, `ls -a`, `ls -l`, `cd`, `cd ..`, `cd -`, `cd ~`, `mkdir`, `mkdir -p`, `echo`, `cat`, `cp`, `mv`, `touch`, `rm`, `rmdir`, `export`, `command -v`, and the redirections `>` and `>>`.

Common Mistakes

1. **Typing the `$` prompt symbol** as part of the command.
2. **Spaces in names** without quotes, splitting one name into two.
3. **Using `>` when `>>` was meant**, overwriting a file.
4. **Deleting from the wrong folder**. Check `pwd` first.
5. **Forgetting that names are case-sensitive**.
6. **Expecting output from a command that succeeded silently**.
7. **Opening a terminal before installing a program**, then seeing `command not found`; open a new terminal.

Practice

Do the exercises in the Exercises section at the end of this chapter. They all happen inside `sharif-git-lab`.

Self-Test

1. What is the difference between a terminal and a shell?
2. Split `mkdir -p sunrise-bakery/css` into program, option and argument.
3. What is the difference between `>` and `>>`?
4. A command prints nothing. Did it fail?
5. Why does `mkdir my folder` create two folders, and how do you create one?
6. A program you installed gives `command not found`. Give two possible causes.
7. Is Git the same thing as Git Bash?

Before Moving On

You are ready for Chapter 8 if you can:

- [] open a terminal and say what your shell is
- [] create the `sunrise-bakery` tree with two commands and list it
- [] copy, rename and delete a file, saying `pwd` first
- [] explain what `PATH` is
- [] read the first line of an error message and find the name it complains about

How this chapter was checked

Claim	Evidence class	Le dg er
Behaviour and output of every command and error shown, in Bash 5.2 and zsh 5.9, including the differences in error wording, <code>cd -</code> , and unmatched wildcards	Locally tested: interactive sessions via a pseudo-terminal, recorded in <code>verification/ch07-terminal/expected-*.txt</code> , re-run in CI on the runner's Bash and zsh; GNU manuals not yet consulted	R1 25
Exported versus unexported variables; <code>PATH</code> lookup; command <code>-v</code>	Locally tested (as above)	R1 30
<code>ls -l</code> permission string	Locally tested on Linux	R1 16
Git Bash is installed with Git for Windows and is not Git	Partly verified: the Git for Windows README (<code>git-bash.exe</code> , <code>git-cmd.exe</code>); installer bundling and drive-path style unchecked	R1 26
PowerShell and Command Prompt equivalents	PowerShell names and aliases verified against Microsoft's PowerShell 7.5 reference; Command Prompt column unchecked; nothing executed on Windows	R1 27
macOS Terminal defaults to zsh	Needs re-verification	R1 28
Wording of Windows "not recognized" messages	PowerShell wording verified against Microsoft's documentation; Command Prompt wording unchecked; not recorded	R1 29

Where this leads

Chapter 8 teaches Markdown, the plain text format for READMEs. Chapter 13 installs Git and checks it from the terminal you now know how to use. Chapter 14 returns to environment variables, showing how the environment changes what Git does.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. **Work only inside `sharif-git-lab`**, and run `pwd` before any `rm`.

Level 1 — Guided

1.1 Build the tree. Open a terminal. Use `pwd` to see where you are. Create `sharif-git-lab`, enter it, and create `sunrise-bakery/css` and `sunrise-bakery/images` with one `mkdir -p`. Use `ls` to check. Expected result: `ls sunrise-bakery` prints `css` and `images`.

1.2 A file. Inside `sunrise-bakery`, use `echo` and `>` to make `README.md` that contains `Sunrise Bakery`, then use `>>` to add a second line. Show it with `cat`. Expected result: two lines.

Level 2 — Partially guided

2.1 Copy, rename, remove. Copy `README.md` to `copy.md`, rename `copy.md` to `notes.txt`, then delete `notes.txt`. Run `ls` after each step and say what it shows. Hint: `cp`, `mv`, `rm`.

2.2 Hidden. Create `.draft`, prove that `ls` does not show it and `ls -a` does. Then explain in one sentence what this has to do with `.git`.

Level 3 — Independent

3.1 Without looking back, create a folder `experiments` inside `sharif-git-lab` containing three empty files `a.txt`, `b.txt`, `c.md`. List only the `.txt` files with one command using a wildcard. Then remove the folder's contents and the folder, using only commands from this chapter.

Level 4 — Professional scenario

4.1 A colleague sends you this line to "tidy" your project: `rm -rf ~/projects/*/build *.log`. You do not know every part. Write what you would do before running anything, using the habits in section 7.10. You do not need to understand `-rf` fully; say what you would find out first.

Level 5 — Troubleshooting

5.1 Each line is what a learner typed and the reply. Say what is wrong and how to fix it. (a) `mkdir Sunrise Bakery` and later `cd Sunrise Bakery` says: `cd: too many arguments (Bash)` or a similar message. (b) `cd sharif-git-lab/Sunrise-Bakery` says: `No such file or directory`, although the folder is called `sunrise-bakery`. (c) After installing a tool, typing its name says `command not found`, but the installer said it succeeded. (d) `echo Fresh bread > README.md` was meant to add a line, but now `README.md` has one line only.

5.2 You run `hello-sunrise` (from section 7.8.3) and it works. You close the terminal, open a new one, and it says `command not found`. Why? How would you make it permanent? (Do not try to change any system files now; describe the idea.)

Chapter 8 — Markdown in Five Pages [Core]

In this chapter

- what Markdown is and why nearly every Git project uses it
- the twelve pieces of syntax you will use every day, each shown with the real result
- the mistakes that trip up nearly everyone, and what really happens when you make them
- how to write your first `README.md`

Before you start. Chapter 3 (plain text, text editors) and Chapter 4 (line endings and invisible characters). You do not need a terminal for this chapter.

8.1 What Markdown is

Chapter 3 said that a word processor saves formatting, and that this makes a file hard for Git to compare. But people still want headings, lists, links and emphasis in their notes and documentation. Markdown is the compromise.

New term: Markdown. A way of writing formatted documents in plain text, using simple marks such as `#` for a heading and `*` for emphasis, which a program can turn into a formatted page.

A Markdown file is an ordinary text file, usually named with the extension `.md`. You write the marks; a program called a **renderer** converts them into what readers see.

New term: renderer. A program that reads a marked-up text file and produces the formatted result for display.

There are two views of one document:

View	What you see	Who uses it
Source	The plain text with its marks	The author, and Git
Rendered	Headings, bold text, lists, links	Readers

Why this matters for Git. The source is plain text, so Git can show line-by-line differences, and everyone can read it even without a renderer. A Markdown file called `README.md` in a project is treated specially by services such as GitHub: they show it, rendered, on the project's front page (Chapter 42). Issues, pull requests and documentation use it too. This book's chapters are written in Markdown.

When not to use Markdown. For a page with complex layout, printed forms, or heavy graphics, use a tool built for it. Markdown is good for text-centred documents.

How to read the examples. Each example below shows the **source** you type, and the **HTML** that a standard CommonMark renderer produced from it (see section 8.6). You do not need to understand HTML: read the tags as labels. `<h1>` is a main heading, `<p>` a paragraph, `<ul>` a bulleted list, `<li>` one item of a list, `<em>` emphasis, `<strong>` strong emphasis.

8.2 The syntax you will use every day

8.2.1 Headings

A line that starts with one to six `#` characters, then a space, is a heading. More `#` means a smaller heading.

Source

```
# Sunrise Bakery

## Menu

### Breads
```

Rendered as HTML by a CommonMark renderer

```
<h1>Sunrise Bakery</h1>
<h2>Menu</h2>
<h3>Breads</h3>
```

8.2.2 Paragraphs and line breaks

A **blank line** separates paragraphs. A single line break inside a paragraph is not a break: the lines join into one paragraph.

Source

```
Fresh bread every morning.

We open at six.
```

Rendered as HTML by a CommonMark renderer

```
<p>Fresh bread every morning.</p>
<p>We open at six.</p>
```

Source

```
Fresh bread
every morning.
```

Rendered as HTML by a CommonMark renderer

```
<p>Fresh bread
every morning.</p>
```

Notice the second example: two source lines became one paragraph. To force a line break within a paragraph, end the line with two spaces:

Source

```
Fresh bread
every morning.
```

Rendered as HTML by a CommonMark renderer

```
<p>Fresh bread<br />
every morning.</p>
```

The two trailing spaces are **invisible**, which is exactly the kind of hidden difference Chapter 4 warned about. Many authors prefer a blank line and a new paragraph instead.

8.2.3 Emphasis

One pair of asterisks makes emphasis (usually shown in italics); two make **strong emphasis** (usually bold); three make both.

Source

```
This is *italic*, this is **bold**, and this is ***both***.
```

Rendered as HTML by a CommonMark renderer

```
<p>This is <em>italic</em>, this is <strong>bold</strong>, and this is <em><strong>both</strong></em>.</p>
```

Underscores inside a word are left alone:

Source

```
Use the file_name_here.txt file.
```

Rendered as HTML by a CommonMark renderer

```
<p>Use the file_name_here.txt file.</p>
```

8.2.4 Lists

Start each item with -, * or + for a bulleted list, or with 1., 2., 3. for a numbered one. Indent an item by two spaces to nest it.

Source

- White loaf
- Whole-wheat loaf
- Rolls

Rendered as HTML by a CommonMark renderer

```
<ul>
<li>White loaf</li>
<li>Whole-wheat loaf</li>
<li>Rolls</li>
</ul>
```

Source

1. Mix
2. Knead
3. Bake

Rendered as HTML by a CommonMark renderer

```
<ol>
<li>Mix</li>
<li>Knead</li>
<li>Bake</li>
</ol>
```

Source

- Breads
 - White loaf
 - Whole-wheat loaf
- Cakes

Rendered as HTML by a CommonMark renderer

```
<ul>
<li>Breads
<ul>
<li>White loaf</li>
<li>Whole-wheat loaf</li>
</ul>
</li>
<li>Cakes</li>
</ul>
```


8.2.5 Links and images

A link is [the words you see](where it goes). An image is the same with a leading `!`, and its words are the **alternative text**, shown when the image cannot be, and read aloud to people who cannot see it.

Source

```
Visit [our menu](menu.html) today.
```

Rendered as HTML by a CommonMark renderer

```
<p>Visit <a href="menu.html">our menu</a> today.</p>
```

Source

```
![Sunrise Bakery logo](images/logo.svg)
```

Rendered as HTML by a CommonMark renderer

```
<p></p>
```

Paths in links and images are the **relative paths** of Chapter 2. A link such as `menu.html` means "the file called `menu.html` next to this one".

8.2.6 Code

Put a word in single backticks to show it as code. Fence a block of lines between three backticks, optionally naming the language, to show several lines exactly as typed.

Source

```
Run `git status` now.
```

Rendered as HTML by a CommonMark renderer

```
<p>Run <code>git status</code> now.</p>
```

Source

```
```bash
git status
```
```

Rendered as HTML by a CommonMark renderer

```
<pre><code class="language-bash">git status
</code></pre>
```

A block indented by four spaces is also treated as code:

Source

```
git status
```

Rendered as HTML by a CommonMark renderer

```
<pre><code>git status
</code></pre>
```

8.2.7 Quotes and rules

Source

```
> Fresh bread every morning.
```

Rendered as HTML by a CommonMark renderer

```
<blockquote>
<p>Fresh bread every morning.</p>
</blockquote>
```

Source

```
Above

---

Below
```

Rendered as HTML by a CommonMark renderer

```
<p>Above</p>
<hr />
<p>Below</p>
```

8.2.8 Tables

A table uses | to separate columns, and a second row of dashes under the header. Tables are **not** part of the core Markdown specification but an extension that most renderers, including GitHub's, support (Chapter [42](#)).

Source

```
| Item | Price |
|-----|-----|
| Loaf | 2.50 |
| Rolls | 3.00 |
```

Rendered as HTML by a CommonMark renderer

```
<table>
<thead>
<tr>
<th>Item</th>
<th>Price</th>
</tr>
</thead>
<tbody>
<tr>
<td>Loaf</td>
<td>2.50</td>
</tr>
<tr>
<td>Rolls</td>
<td>3.00</td>
</tr>
</tbody>
</table>
```

8.2.9 Escaping

To show a mark literally, put a backslash before it:

Source

```
Cost: 3 \* 2 = 6
```

Rendered as HTML by a CommonMark renderer

```
<p>Cost: 3 * 2 = 6</p>
```

8.2.10 HTML inside Markdown

Markdown allows raw HTML tags for the rare things it cannot express:

Source

```
Text with <b>bold</b> HTML.
```

Rendered as HTML by a CommonMark renderer

```
<p>Text with <b>bold</b> HTML.</p>
```

8.3 What goes wrong

These are the mistakes that beginners make most, with the result of each, tested on a real renderer.

1. A table without its dashed second row is not a table. It stays as plain text:

Source

```
| Item | Price |
| Loaf | 2.50 |
```

Rendered as HTML by a CommonMark renderer

```
<p>| Item | Price |
| Loaf | 2.50 |</p>
```

2. Expecting a single line break to be a line break. It is not (see the two-line example above).

3. A list right after a paragraph line. Many authors forget the blank line:

Source

```
Ingredients:
- flour
- water
```

Rendered as HTML by a CommonMark renderer

```
<p>Ingredients:</p>
<ul>
<li>flour</li>
<li>water</li>
</ul>
```

This standard renderer accepted the list even without a blank line. Other renderers, and older versions of Markdown, may not. **The safe habit: always leave a blank line before a list, a table or a code block.** It is harmless, and it works everywhere.

4. Expecting a bare web address to become a link. In plain CommonMark, it does not:

Source

```
See https://example.org for details.
```

Rendered as HTML by a CommonMark renderer

```
<p>See https://example.org for details.</p>
```

To be safe, write the link explicitly ([the site](https://example.org)), or wrap the address in angle brackets so that the renderer knows it is a link:

Source

```
See <https://example.org> for details.
```

Rendered as HTML by a CommonMark renderer

```
<p>See <a href="https://example.org">https://example.org</a> for details.</p>
```

Checked against a source [R132]. GitHub's published Markdown specification (version 0.29, 2019) lists autolinks, tables, task lists and strikethrough as extensions to CommonMark, including the recognition of a bare `www.` or `http` address as a link. What GitHub's website shows today was not observed in this book's test environment, and the demonstration above is of plain CommonMark only.

8.4 Flavours of Markdown

There is more than one Markdown. The original description was informal, so different programs made different choices, and people wrote **CommonMark**, a precise written specification, to fix that.

New term: CommonMark. A precise, written specification of the core Markdown syntax, so that different renderers produce the same result.

Services then add their own extras on top. GitHub's version is called **GitHub Flavored Markdown**; among other things, it adds tables, task lists and automatic links. The practical lesson: **the core syntax in this chapter works everywhere; extras may not.** When the book reaches GitHub, it will say which features are GitHub's own, after checking them.

8.5 Your first README

A `README.md` is the front page of a project. It says what the project is and how to use it. Here is a starter for the Sunrise Bakery project. Read it as source, then look at the HTML that came out:

Source

```
# Sunrise Bakery

A small website for a *fictional* neighbourhood bakery, used as the running example in a
textbook.

## Pages

- `index.html`: the home page
- `menu.html`: the menu
- `contact.html`: how to find us

## Opening hours

| Day | Hours |
|-----|-----|
```

```
| Monday to Saturday | 06:00 to 18:00 |
| Sunday | closed |
```

```
## Licence
```

```
See the licence file when one is added.
```

Rendered as HTML by a CommonMark renderer

```
<h1>Sunrise Bakery</h1>
<p>A small website for a <em>fictional</em> neighbourhood bakery, used as the running
example in a textbook.</p>
<h2>Pages</h2>
<ul>
<li><code>index.html</code>: the home page</li>
<li><code>menu.html</code>: the menu</li>
<li><code>contact.html</code>: how to find us</li>
</ul>
<h2>Opening hours</h2>
<table>
<thead>
<tr>
<th>Day</th>
<th>Hours</th>
</tr>
</thead>
<tbody>
<tr>
<td>Monday to Saturday</td>
<td>06:00 to 18:00</td>
</tr>
<tr>
<td>Sunday</td>
<td>closed</td>
</tr>
</tbody>
</table>
<h2>Licence</h2>
<p>See the licence file when one is added.</p>
```

Try it. In your sunrise-bakery folder, create a text file named README.md (replace the existing one from Chapter 7 if there is one) with your own version of the source above. Use your editor's Markdown preview if it has one (search its help for "Markdown preview"). Change one word, save, and watch the preview change. Make one of the mistakes from section 8.3 on purpose, and see what happens.

Expected result: headings, a list and a table appear in the preview. The mistakes appear as the examples above show.

8.6 How the examples were made

Every example in this chapter was generated by a real program, not typed from memory: **markdown-it**, a widely used CommonMark renderer written in Python, with its table extension switched on. The book's build compares the source and HTML shown here with the recorded results, and fails if they differ. Other renderers may print a slightly different HTML for the same source (for example a space before a closing tag); the structure is what matters.

Checkpoint

What You Learned

- Markdown is plain text with simple marks that a renderer turns into a formatted page; files usually end in `.md`.
- The core syntax: headings, paragraphs, emphasis, lists, links, images, code, quotes, rules, tables and escapes.
- A single line break does not start a new paragraph; a blank line does.
- A table needs its dashed second row; leave blank lines before lists, tables and code blocks.
- CommonMark is a precise specification; services such as GitHub add extras.
- A `README.md` is a project's front page.

New Vocabulary

- **Markdown**: a way of writing formatted text using plain-text marks.
- **Renderer**: a program that turns marked-up text into a formatted result.
- **CommonMark**: a precise written specification of the core Markdown syntax.

Commands Learned

None.

Common Mistakes

1. **Forgetting the dashed row in a table.**
2. **Expecting one line break to start a new paragraph.**
3. **No blank line before a list, table or code block.**
4. **Expecting a bare address to be a link** on every renderer.
5. **Trailing spaces used for line breaks** (invisible, easily lost).
6. **Writing documentation in a word processor** when the project needs readable differences.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between source and rendered views?
2. Write the Markdown for a second-level heading, a bulleted list of two items, and a link to `menu.html` with the text "our menu".
3. Two lines are typed one below the other with no blank line. How many paragraphs result?
4. Why does a table without a dashed second row not work?
5. What does CommonMark solve?

Before Moving On

You are ready for Chapter 9 if you can:

- [] write a heading, a list, a link, a code block and a table from memory
- [] explain why Markdown suits Git better than a word-processor file
- [] name two mistakes that break Markdown and how to avoid them
- [] create and preview a `README.md`

How this chapter was checked

Claim	Evidence class	Ledger
The source-to-HTML results of every example	Both: rendered with <code>markdown-it-py 4.2.0</code> (CommonMark preset, tables on); the book's build compares every example with the recorded output (<code>tools/check_md_examples.py</code> , <code>verification/ch08-markdown</code>); the same renderer reproduces all 655 examples of the CommonMark specification, version 0.31.2 (the script <code>research/commonmark-conformance.py</code> in the companion repository)	R 13 1
GitHub's extras (automatic links, task lists, tables)	Officially verified (docs only): GitHub's Markdown specification, version 0.29; GitHub's live rendering not observed	R 13 2

Where this leads

Chapter 9 begins Part II, which explains the problem that version control solves. Chapter 42 returns to README files in depth, and to what GitHub adds to Markdown.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use a text editor, and a Markdown preview if it has one.

Level 1 — Guided

1.1 A first document. Create `notes.md` containing: a first-level heading `Study notes`, a paragraph of one sentence, and a bulleted list of three items. Expected result: the preview shows a heading, a paragraph and three bullets.

1.2 A link and an image. Add a link to `menu.html` with the words "our menu", and an image reference to `images/logo.svg` with the alternative text "Sunrise Bakery logo". Expected result: a clickable link; the image if the file exists.

Level 2 — Partially guided

2.1 A table. Write a table with the columns `Item` and `Price` and three rows. Hint: the second row is dashes. Then delete the dashed row and describe what changes.

2.2 Paragraphs. Write two sentences on separate lines with no blank line, and one pair with a blank line between. How many paragraphs does the preview show for each?

Level 3 — Independent

3.1 Write a `README.md` for a project of your own choosing (a school club site, a recipe collection) with: a title, a one-paragraph description, a section headed `How to use`, a numbered list of three steps, and a code block showing one command. Preview it.

Level 4 — Professional scenario

4.1 A colleague sends the volunteer rota as a `.docx` and asks you to publish it on the project page so people can see changes over time. Convert the first six rows to a Markdown table, and write two sentences explaining why the Markdown version is better for tracking changes and one situation in which the `.docx` would still be better.

Level 5 — Troubleshooting

5.1 A learner's list renders as a single paragraph with dashes in it. The source is:

```
Ingredients:
- flour
- water
```

and the other renderer they use needs a blank line before a list. What is the fix, and why is it a good habit even when the renderer does not need it?

5.2 A table shows as plain lines of `|` characters. List two possible causes in order of likelihood.

5.3 A link in a `README` works on your computer but not for a friend who opened the project in another folder structure. What kind of path did you probably use, and how would you make it robust?

Part II — Understanding Version Control

Chapter 9 — The Problem of Changing Files [Core]

In this chapter

- the "final_v2_REAL" problem, shown with real files
- six separate difficulties that appear whenever files change over time or are shared
- what people try (copies, backups, cloud folders, tracked changes) and where each falls short
- what a better system would have to do
- the definition of version control, and the vocabulary that goes with it

Before you start. Chapter 2 (files, folders), Chapter 3 (text files) and Chapter 7 (typing commands). This chapter is a story with recorded evidence: the commands are shown so you can repeat them in your practice folder, and each has real recorded output.

9.1 A very ordinary afternoon

The Sunrise Bakery keeps its prices in a small file, `menu.txt`. Over a few weeks, four people touch it. Somebody raises the price of rolls. Somebody raises the price of bread. Someone adds a cake. Each person, wanting to be careful, saves a copy instead of overwriting the old one.

Here is what the folder looks like afterwards. The files were created with commands (you will meet the same commands again in this chapter's exercises), and `ls` lists the result:

```
$ mkdir menu-history
$ cd menu-history
$ printf 'Bread: 2.50\nRolls: 3.00\n' > menu.txt
$ printf 'Bread: 2.50\nRolls: 3.20\n' > menu_v2.txt
$ printf 'Bread: 2.80\nRolls: 3.20\n' > menu_final.txt
$ printf 'Bread: 2.80\nRolls: 3.00\n' > menu_final_v2.txt
$ printf 'Bread: 2.80\nRolls: 3.20\nCake: 4.00\n' > menu_final_REAL.txt
$ ls
menu.txt  menu_final.txt  menu_final_REAL.txt  menu_final_v2.txt  menu_v2.txt
```

Recorded in Bash; `ch09-problem/expected-names.bash.txt`.

Five files, all of them "the menu". Take a moment with this list and try to answer three questions.

1. **Which is the latest?** `menu_final.txt`? `menu_final_v2.txt`? `menu_final_REAL.txt`? The names claim to know, but they contradict themselves: "final" cannot be followed by a "v2", and "REAL" implies that the other finals were fake.
2. **What is the difference between two of them?** You would have to open both and compare them line by line.
3. **Why did someone change it?** Nothing here says.

The computer can help with the second question. The command `diff` compares two text files and prints the differences:

```
$ diff menu_final.txt menu_final_v2.txt
2c2
< Rolls: 3.20
---
> Rolls: 3.00
$ diff menu_final.txt menu_final_REAL.txt
2a3
> Cake: 4.00
```

Recorded in Bash; *ch09-problem/expected-names.bash.txt*.

Read the first result. The line 2c2 means "line 2 in the first file was changed into line 2 in the second". The < line is from the first file and the > line from the second: the rolls cost 3.20 in *menu_final.txt* but 3.00 in *menu_final_v2.txt*. The second result, 2a3, means "after line 2, a line was added": the cake.

So the mechanics of comparing are solved. What is missing is a system that keeps the versions, names them for you, remembers which came from which, and records why. That is the problem this chapter is about.

9.2 Six difficulties

Whenever information changes over time, and more so when several people change it, the same six difficulties appear. Learn to recognise them, because everything in Git answers one of them.

#	Difficulty	Example
1	Which is the current one?	Five files named "menu".
2	What changed?	You must compare by eye or with a tool, each time.
3	Why did it change?	The reason lives in someone's memory, or nowhere.
4	Can I go back?	Overwrite a file and the old contents are gone.
5	How do we work at the same time?	Two people edit, and one erases the other's work.
6	Who did what, and when?	No reliable record of authorship.

9.2.1 The collision, recorded

The fifth difficulty is the most damaging, so here it is, reproduced. A shared folder holds *menu.txt*. Alice and Bob each take a private copy. Alice adds a cake; Bob raises the price of bread. Each then copies their version back to the shared folder.

```
$ mkdir shared alice bob
$ printf 'Bread: 2.50\nRolls: 3.00\n' > shared/menu.txt
$ cp shared/menu.txt alice/menu.txt
$ cp shared/menu.txt bob/menu.txt
$ printf 'Bread: 2.50\nRolls: 3.00\nCake: 4.00\n' > alice/menu.txt
$ printf 'Bread: 2.80\nRolls: 3.00\n' > bob/menu.txt
$ cp alice/menu.txt shared/menu.txt
$ cat shared/menu.txt
```

```
Bread: 2.50
Rolls: 3.00
Cake: 4.00
```

Recorded in Bash; *ch09-problem/expected-collision.bash.txt*.

Read this carefully. After Alice copied her file back, the shared menu had the cake. Then Bob copied his file back, and the shared menu became:

```
$ cp bob/menu.txt shared/menu.txt
$ cat shared/menu.txt
Bread: 2.80
Rolls: 3.00
```

Recorded in Bash; *ch09-problem/expected-collision.bash.txt*.

The cake has vanished. Nothing reported an error. Bob's copy did not know about Alice's change, so when it was copied over the shared file it silently replaced Alice's work. Alice's version still exists in her private folder, but the shared file (the one everyone trusts) has lost her change, and nobody was warned.

Nobody did anything wrong. The system had no way to notice, because a plain file has no memory of what it used to contain, and a plain copy has no idea that someone else changed the original meanwhile. **The lesson is not "be more careful". The lesson is that carefulness does not scale.** With two people it is one mistake away; with twenty it is certain.

9.3 What people try

Before looking for a proper solution, notice the things people usually do, and why each is not enough.

9.3.1 Copies with names

What we saw in section 9.1: *menu_v2.txt*, *menu_final.txt*. This gives you some history, at the cost of confusion. It does not record why or by whom, it multiplies files, and it relies on everyone using the same naming scheme without exception. It does not help at all with the collision.

9.3.2 Backups

New term: backup. A copy of your files kept elsewhere so that they can be restored after loss or damage.

A backup protects against losing the whole thing (a stolen laptop, a failed disk). Some backup programs keep old versions too. But backups are usually made on a **schedule**, not when you finish a meaningful piece of work, so their versions are arbitrary moments ("Tuesday 3 a.m.") rather than meaningful ones ("added the cake to the menu"). Backups carry no explanations, and they do not combine two people's changes.

9.3.3 Shared and synchronised folders

New term: synchronised (cloud) folder. A folder whose contents are copied automatically between several computers, or between a computer and a service on the internet, so they stay the same.

These are very useful, and they solve difficulty 1 (there is one current copy) and part of 5 (people can see each other's changes). But note what they synchronise: the latest state, including mistakes. If you delete a file or save nonsense, the mistake is copied everywhere within moments. Some services keep old versions for a time, but they generally keep them by time, not by meaning, they do not ask you to explain a change, and when two people change the same file at once they typically have to keep two conflicting copies for a person to sort out by hand.

Verification pending [R134]. Statements in this section about what backup and synchronisation tools generally do are given in general terms, because products differ and change. No product is named. They will be checked against representative vendors' documentation before publication.

9.3.4 Tracked changes inside one file

Word processors can record edits inside a document ("track changes"). That is a real help for one document, edited by a few people, but it works file by file, needs the word-processor format that Chapter 3 showed is not plain text, and cannot describe a project of many files that change together (a page, its stylesheet and its logo).

9.3.5 The summary

Approach	Solves	Leaves unsolved
Copies with names	Some history	Confusion; no reasons; collisions
Backups	Loss of everything	Meaningful versions; explanations; combining changes
Synchronised folders	One shared copy	Copies mistakes; conflicts by hand; no reasons
Tracked changes	Edits in one document	Many-file projects; plain text

9.4 What a better system would need to do

Write the requirements down, and you have written the outline of version control.

1. **Remember every version**, automatically kept, never accidentally overwritten.
2. **Name and explain each version**: who made it, when, and why, in the author's own words.
3. **Show what changed** between any two versions, line by line.
4. **Go back** to any earlier version, and recover from mistakes.
5. **Handle many files together**: a change to a page and its stylesheet is one change.

6. **Let people work at the same time**, in parallel, and then **combine** their work, and say clearly when two changes truly collide.
7. **Work without a network**, and keep the whole history on your own computer.
8. **Be trustworthy**: it should be impossible for history to be altered silently or to be damaged without detection.

Git was designed to meet these requirements. Requirement 6 is the subject of Chapters 20 to 22; requirement 7 is why Chapter 11 introduces "distributed" systems; requirement 8 is why Chapter 30 looks inside Git.

9.5 Version control, defined

New term: version. One particular state of a file or project at a point in time.

New term: history. The recorded sequence of versions of a project, with the reasons and authors for each.

New term: version control. A system that records the history of a project's files, so that you can see what changed, who changed it and why, return to earlier versions, and let many people work together. It is also called source control or revision control.

Version control is a tool for working with change over time. It is not a backup, although it makes a good part of one; it is not a file-sharing service, although services are built on it; and it is not only for programmers. Anyone who keeps a set of files that change (a novel, a thesis, the notes of a club) benefits.

When you might not need it. A single file that never changes, or a set of large binary files that people replace wholesale (raw video, say) gain less, because line-by-line comparison is impossible (Chapter 3). Version control still keeps such files, but its best features work on text.

How the rest of the book uses this. Part III teaches Git step by step. You will record versions, read them, compare them, go back, work in parallel and combine. When you do, come back to the six difficulties in section 9.2: each Git feature answers one of them.

9.6 Try it yourself

Try it. Reproduce sections 9.1 and 9.2.1 in your practice folder. Open your terminal, go to `sharif-git-lab`, and type the commands from the two recordings, one at a time. Compare each result with the recording. (Use `cd` to enter the folder they create, and remember that `ls` may sort the file names differently on your system: upper-case letters sort before lower-case ones in the recording, and your computer may do otherwise.)

Expected result: the same five files; the same two `diff` reports; and the shared `menu.txt` ending without the cake line.

If `diff` is not available in your terminal, note that fact; it does not affect the rest of the book.

Checkpoint

What You Learned

- Copies with names, backups, synchronised folders and tracked changes each address part of the problem, but none records meaningful, explained versions of a whole project.
- Six difficulties recur: which is current, what changed, why, can I go back, how do we work together, and who did what.
- Copying a file over a shared file can silently erase someone else's work; carefulness does not scale.
- A better system must remember versions, explain them, show differences, go back, handle many files together, let people work in parallel, work offline and be trustworthy.
- Version control is a system that records a project's history and supports these needs.

New Vocabulary

- **Backup:** a copy of files kept elsewhere so they can be restored after loss.
- **Synchronised (cloud) folder:** a folder whose contents are kept the same across computers or a service.
- **Version:** one particular state of a file or project at a point in time.
- **History:** the recorded sequence of versions of a project, with reasons and authors.
- **Version control:** a system that records a project's history so that changes can be seen, reversed and combined.

Commands Learned

`diff` (compares two text files line by line). Also used again: `mkdir`, `cd`, `printf`, `cp`, `cat`, `ls`.

Common Mistakes

1. **Believing "be more careful" solves collisions.** The problem is structural.

2. **Treating a synchronised folder as version control.** It copies mistakes as quickly as improvements.
3. **Naming files final.** The name is a promise that will be broken.
4. **Assuming a backup contains meaningful versions.** It contains moments.
5. **Assuming version control is only for code.** Any text that changes benefits.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name the six difficulties of changing files.
2. In the collision of section 9.2.1, why did no error appear?
3. Give two things a synchronised folder does not record.
4. Why is a backup different from version control?
5. What does the 2a3 in a diff report mean?
6. State three requirements a version-control system must meet.

Before Moving On

You are ready for Chapter 10 if you can:

- [] explain the collision story to someone else in your own words
- [] name at least three things a backup or synchronised folder does not do
- [] define version control without using the word "control"
- [] read a simple diff report

How this chapter was checked

Claim	Evidence class	Ledger
The listing of five files, the diff outputs, and the shared-folder collision (Alice's addition silently lost)	Locally tested: interactive Bash 5.2 and zsh 5.9 recordings, re-run in CI (verification/ch09-problem)	R13 5
General descriptions of backups, synchronised folders and tracked changes	Needs re-verification: general statements about product classes, no product named	R13 4

Where this leads

Chapter 10 tells, briefly, how people tried to solve this problem before Git. Chapter 11 explains the two great designs of version-control systems. Chapter 13 installs Git, and Chapter 16 creates your first version-controlled project.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Work only inside `sharif-git-lab`.

Level 1 — Guided

1.1 Reproduce the names. In `sharif-git-lab`, type the commands of section 9.1 (from `mkdir menu-history` to `ls`). Expected result: the same five files as the recording (the order may differ on your system).

1.2 Read a diff. Run the two `diff` commands. In your own words, say what each report line means.

Level 2 — Partially guided

2.1 Reproduce the collision. Type the commands of section 9.2.1. Expected result: the final `shared/menu.txt` has no `Cake` line. Explain, in two sentences, why no error appeared.

2.2 Name the difficulty. For each situation, say which of the six difficulties it shows: (a) "Is this the latest?" (b) "Why did we change the price?" (c) "Who deleted the cake line?" (d) "I saved over the old version by mistake."

Level 3 — Independent

3.1 Design a safer working method for Alice and Bob without version-control software. Write down the rules you would impose. Then list two ways the rules could still fail. (This is what people did before version control.)

Level 4 — Professional scenario

4.1 A five-person volunteer group edits one shared price list every week using a synchronised folder. Twice this month someone's change disappeared. Explain what is probably happening, what a synchronised folder does and does not record, and recommend one change of working method.

Level 5 — Troubleshooting

5.1 In section 9.2.1 you have Alice's private copy, Bob's private copy, and the shared file (which contains only Bob's change). Recover a shared menu that holds **both** the cake and the new bread price. What did you have to do by hand, and what would you want a tool to do for you?

5.2 Someone says: "Our backup runs every night, so we have version control." Give two reasons why that is not correct.

Chapter 10 — A Short History of Version Control [Core]

In this chapter

- the stages people went through in solving the problem of Chapter 9
- why the answers changed, stage by stage
- where Git came from, and why it became so widely used
- how to read a claim about history critically

Before you start. Chapter 9: the six difficulties of changing files and the eight requirements of a version-control system. This chapter is optional on a first read (its tag is Core, but the First-Read Path skips it); return to it when you are curious about why Git is designed as it is.

Verification pending [R136, R137, R138]. History is a matter of record, and this chapter has not yet been checked against the primary sources (the projects' own documentation, the archives of the communities involved, and the published accounts of the people who made these tools). Every named system, every date, and every statement of cause in this chapter is given from general knowledge and is marked for re-verification. The structure of the story, that each stage solved a problem left by the one before, is the part to remember. Apart from the one verified note on Git's first commit (section 10.6), this chapter deliberately contains **no exact dates and no quotations**, so that nothing in it can be wrong in a way that matters until it is verified.

10.1 Why history helps

Design decisions look arbitrary until you know the problem they answered. Git's commands, its vocabulary and even its odd corners (Chapters 19, 20, 27) make far more sense if you know what came before. The history of version control is a story of one question asked again and again: **how can many people change the same files, safely, without losing anyone's work?**

10.2 Stage one: people and copies

The first version control is the one in Chapter 9: folders and file names, dated copies and careful habits. It works for one person and a small project. It fails when the project grows or a second person joins. Nothing in this stage is a system: everything depends on people following a convention.

10.3 Stage two: tools for a single computer

The first software answer kept the history of **individual files** on **one computer**. You told the tool "I am about to change this file", it locked the file so nobody else could, you edited it, and you told the tool you had finished, which stored the new version and released the lock. Old versions could be recovered.

New term: lock. A marker that stops other people from changing a file until you release it.

Locking prevents the collision of section 9.2.1 by making sure that only one person can edit a file at a time. Its cost is obvious: **people wait for each other**. If the person with the lock goes on holiday, no one else can change the file.

10.4 Stage three: a central server

The next idea moved the history to **one shared server** that everyone connected to. The server held the single official history; each person "checked out" a working copy, edited it, and sent changes back. Tools of this kind also tracked whole projects rather than single files, and many of them replaced strict locking with a smarter rule: two people may edit the same file at once, and when they send their changes, the server tries to **merge** them, complaining only if the two edits truly touch the same lines.

New term: merge. To combine two sets of changes into one.

These **centralised** systems were a large step forward. They gave a project one official history, a record of who changed what, and the ability to work in parallel. Chapter 11 examines them closely, because Git's design is best understood as a reaction to their limits.

Their limits were practical:

- Almost every action needed a connection to the server. On a train, on a slow network, or when the server was down, you could not look at history or record your work.
- A mistake on the server, or a lost server, could damage or lose the only copy of the complete history.
- Working on an experimental idea without disturbing others was awkward, because branches (parallel lines of work) were slow or costly in many such tools.

10.5 Stage four: everyone has everything

The answer to those limits was **distributed** version control: instead of one server holding the history and everyone else holding a snapshot, **every person holds the complete history on their own computer**. You can look at the past, record your own work, and work on an experiment, all without a network. Sharing then becomes a matter of exchanging history between copies. Several systems took this approach; Git is the best known.

Verification pending [R136]. The naming of particular systems in each stage, and their order, are from general knowledge and are omitted here on purpose until they can be checked against the projects' own histories.

10.6 Where Git came from

Git's beginning was practical. According to general accounts, it was written in the mid-2000s by the leader of the Linux kernel project, for the needs of the kernel's own very large community of developers, when the tool they had been using was no longer available to them. Its goals, reported in those accounts, matched the requirements of section 9.4: **speed**, a **simple design**, strong support for **thousands of parallel branches**, being **fully distributed**, and being able to handle **very large projects**, all while keeping history safe from silent damage.

Partly checked [R137]. Git's own published history was read for this book: the first commit in the git/git repository is dated 7 April 2005 and is authored by Linus Torvalds, with the message Initial revision of "git", the information manager from hell. That fixes the creator and the year. The **reason** (the earlier tool becoming unavailable) and the **list of goals** are still from general accounts and have not been checked against the Git project's documentation or the kernel community's records, so do not quote them as fact.

Two features of that origin will keep appearing in this book:

1. **Git assumes that many people, some of them strangers, contribute in parallel.** That is why branches are cheap (Chapter 20) and why Git records not just changes but who made them.
2. **Git cares about integrity.** It is built so that history cannot be altered without being noticed (Chapter 30).

10.7 Why Git became widely used

Popularity is never the result of a single cause. General accounts usually list several that reinforced one another:

Reason	Effect
Fast, and works offline	Daily work does not wait for a network
Cheap branching and merging	Trying ideas and combining work became routine
Free to use and open	No cost, no single owner to depend on
Reliable integrity	People trust their history
Hosting platforms built around Git	Sharing and collaboration became easy; a network effect followed
Widespread use itself	Tools, teaching material and job requirements assumed Git

Verification pending [R138]. The list of reasons is a synthesis of general accounts, not a measured result. The claim that Git is "widely used" should be supported by named, dated surveys before publication, and will be.

10.8 Reading historical claims critically

You will meet strong claims about tools: "X is dead", "Y is the future". A few habits will keep you accurate.

1. **Ask for a date.** Tool landscapes change; a claim without a date has no meaning.
2. **Separate the tool from the platform.** A statement about a hosting platform is not a statement about the tool, and the reverse (Chapter 12).
3. **Prefer primary sources.** The tool's own documentation and the people who wrote it are better sources than summaries.
4. **Watch for causal certainty.** "Because of X, Y happened" is almost always a simplification.

Checkpoint

What You Learned

- Version control developed in stages: dated copies, single-computer file tools with locks, central servers, and distributed systems.
- Each stage solved a limit of the one before: waiting for locks, dependence on a central server, and the cost of working in parallel.
- Git was created for a very large, distributed community and aims at speed, a simple design, cheap branching, distribution, scale and integrity.
- Popularity had several reinforcing causes.
- Historical claims should be dated and traced to primary sources.

New Vocabulary

- **Lock:** a marker that stops others from changing a file until it is released.
- **Merge:** to combine two sets of changes into one.

Commands Learned

None.

Common Mistakes

1. **Treating a tool's popularity as proof that it suits your project.**
2. **Confusing a tool with a platform built on it.**
3. **Quoting undated claims about "the best" tool.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Why does file locking prevent collisions, and what is its cost?
2. Name two limits of a central-server system that a distributed system removes.
3. What does "distributed" mean in version control?
4. Give two ways to check a claim about the history of a tool.

Before Moving On

You are ready for Chapter 11 if you can:

- [] describe the four stages in your own words
- [] say what problem each stage solved and what problem it left
- [] explain why "everyone has the whole history" helps when the network is down

How this chapter was checked

Claim	Evidence class	Ledger
The staged account of version control (copies; locking tools; central servers; distributed systems)	Needs re-verification; no system or date is stated	R13 6
Git's origin: creator and year	Officially verified (docs only): the first commit in Git's own history; the reason and the goals remain unverified	R13 7
Why Git became widely used	Needs re-verification; synthesis of general accounts	R13 8

Where this leads

Chapter 11 compares the centralised and distributed designs in detail. Chapter 12 separates Git from the platforms built around it.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book.

Level 1 — Guided

1.1 Order the stages. Put these in order and give each a one-line description: distributed systems; dated copies; central server systems; file-locking tools on one computer.

Level 2 — Partially guided

2.1 Problem and answer. Complete the table:

Stage	Problem it solved	Problem it left
Dated copies		
Locking tools		
Central server		
Distributed		

Level 3 — Independent

3.1 In a paragraph, explain why "the whole history on your own computer" is more than a convenience.

Level 4 — Professional scenario

4.1 A manager says: "Everyone uses Git, so we should too." Draft a two-paragraph reply that respects the point but asks the right questions about the team's needs (Chapter [11](#)).

Level 5 — Troubleshooting

5.1 A blog post says: "Git won because it is technically perfect." Using section 10.8, list three questions you would ask before believing or repeating the claim.

Chapter 11 — Centralised vs Distributed [Core]

In this chapter

- the two designs of version-control system, drawn and compared
- the word repository, introduced informally
- why Git's design lets you work on a train, and what it costs
- how to choose between the designs, without declaring one the winner

Before you start. Chapter 9 (the eight requirements) and Chapter 5 (client, server, local, remote). Chapter 10 is not required, though it explains how the two designs arose.

11.1 The word "repository"

Before the two designs can be described, one word is needed.

New term: repository. The place where a project's files and the complete record of their history are kept.

Think of a repository as a project folder that has a memory. In a Git project the repository is the folder itself plus a hidden `.git` folder inside it (Chapter 2 said that hidden folder was coming). Chapter 15 gives the exact definition; for now, "repository" means "the project and its history, kept together".

You will also hear the shorthand **repo**.

11.2 Design one: centralised

In a **centralised** system there is one repository, held on a server. Everyone else holds only a working copy: the files of one version, with no history.

New term: centralised version control. A design in which one central server holds the only complete history, and each user holds only a working copy of the files and a connection to the server.

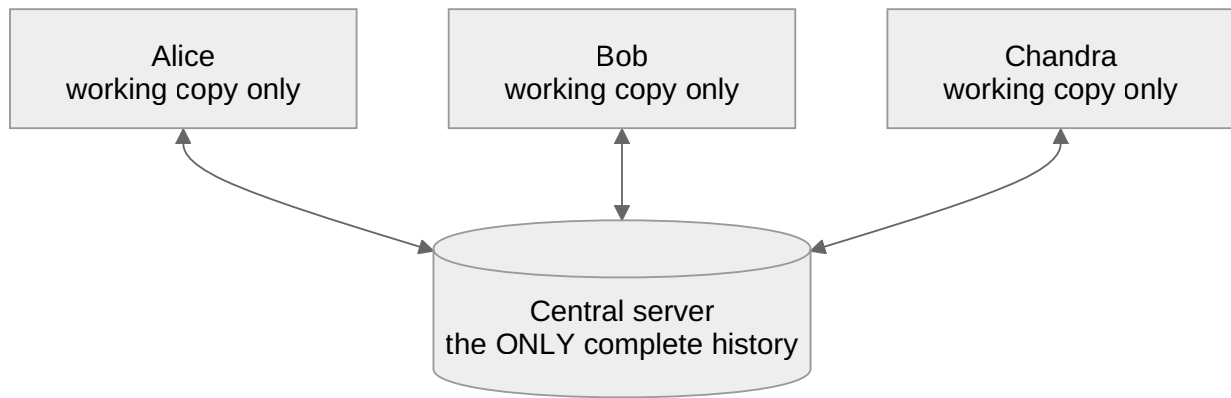


Figure 8. 11.2 Design one: centralised

Diagram description: a database symbol in the centre represents the central server that holds the complete history. Three people, Alice, Bob and Chandra, are each connected to it by a two-way arrow. Each holds only a working copy of the files.

How it feels to use: to see who changed a line last year, you ask the server. To record your work, you send it to the server. If you cannot reach the server, you cannot do either.

Strengths. There is one official history and one place to look. Administration and access control are simple to reason about, because everything passes through one door. The size of a working copy is small, because it holds one version and not all of them.

Weaknesses.

- **A single point of failure.** If the server is unreachable, nobody can record work or read history. If it is lost without a backup, the history is lost.
- **Needs the network for almost everything.**
- **Sharing half-finished work is awkward,** because the only way to share is to send it to the one official place.

New term: single point of failure. A part of a system whose failure stops the whole system, or destroys something that cannot be replaced.

11.3 Design two: distributed

In a **distributed** system, every user holds a full repository, with the complete history, on their own computer.

New term: distributed version control. A design in which every user holds a complete copy of the repository, including its whole history, so that most actions need no server and any copy can exchange history with any other.

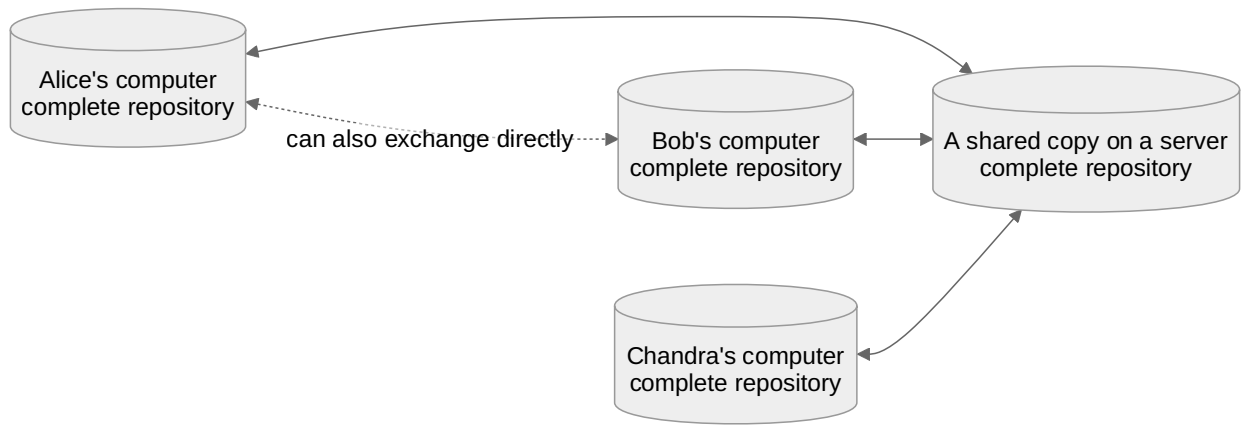


Figure 9. 11.3 Design two: distributed

Diagram description: four database symbols represent four complete repositories, Alice's, Bob's, Chandra's and a shared one on a server. Alice, Bob and Chandra each have a two-way link to the shared server copy. A dotted line shows that Alice and Bob could also exchange history directly with each other.

The word **clone** describes how you get your own full copy.

New term: clone. A complete copy of a repository, with all of its history, made on another computer.

How it feels to use: to see history, you look at your own computer. To record your work, you record it on your own computer. To share, you exchange history with another copy, and this needs a network only at that moment. If the shared server vanishes, every person still has a complete copy, and any one of them can become the new shared copy.

The "shared copy on a server" in the diagram is a convention, not a requirement of the design. Nothing in the design forces a central point; teams choose one because a well-known meeting place is convenient. That is exactly the role that GitHub plays in Part V, and Chapter 12 separates the tool from the platform.

Strengths.

- **Fast**, because most actions are local.
- **Works offline.** Record your work on a train and share it later.
- **Resilient.** Every copy is a backup of the history.
- **Cheap experiments.** You can try an idea in private, without anyone seeing half-finished work.

Weaknesses.

- **A bigger copy on every computer**, because the whole history is stored locally. Very large histories can be heavy (Chapter 31 addresses this).
- **More concepts to learn.** There are two levels of "recording" (locally, then sharing), which is the main cost of learning Git.
- **No single authority is built in.** A team must agree which copy is the official one and how changes reach it. Chapter 34 is about exactly that.

11.4 Side by side

Question	Centralised	Distributed
Where is the complete history?	On the server only	On every computer
Can I read history offline?	No	Yes
Can I record my work offline?	No	Yes
What if the server is lost?	The history may be lost	Every copy remains a full backup
How big is my copy?	One version	The whole history
Where do I try a private experiment?	Awkward	On my own computer
Is there an official copy?	Yes, by design	By agreement of the team
Main learning cost	Fewer concepts	Local vs shared history

Verification pending [R139]. These are properties of the designs. Statements about how particular products (Subversion-style systems on one side, Git and similar tools on the other) behave in practice have not yet been checked against their documentation, and are not made here.

11.5 Git versus Subversion-style systems

A common exam question, and a real decision: "Should we use Git or a centralised system such as Subversion?" Because the two are built on different designs, the honest answer is a comparison, not a verdict.

Choose a distributed tool such as Git when people work in parallel on a project, work from different places or with intermittent networks, want private experiments, or contribute from outside the organization.

A centralised tool may suit you when a team is small and always connected, a single official history with strict central control matters more than flexibility, or a project's large binary files make copying the whole history to everyone impractical, and its tooling handles them better. (These are judgements to weigh, not rules; check current documentation when you decide.)

The practical fact for most learners is that Git dominates the settings in which they will work, so learning Git is a safe investment (Chapter 10).

11.6 A common confusion: distributed does not mean "no server"

Beginners hear "distributed" and picture a project with no shared place. That is not what teams do. Nearly every team using Git chooses **one shared repository** as the meeting point. The difference from the centralised design is that this shared copy is a convention: it is a copy like any other, it holds nothing that your own copy lacks, and you can work for weeks without touching it.

Another way to say it: in a centralised system the server is the repository; in a distributed system it is a repository that the team treats as the official one.

11.7 What a later chapter will let you prove

Chapter 23 opens with an experiment that needs no account and no internet: it creates two complete repositories on your own computer and exchanges history between them. Seeing history move from one complete copy to another is the best way to understand what "distributed" means. That is why the experiment comes before you ever meet GitHub.

Checkpoint

What You Learned

- A repository is a project together with the complete record of its history.
- In a centralised design one server holds the only history and users hold working copies; it has one official source but a single point of failure and needs the network for almost everything.
- In a distributed design every user holds the complete history; most actions are local and fast, and it works offline, at the cost of a larger copy, more concepts, and a team's need to agree on the official copy.
- "Distributed" does not mean "no shared server": teams usually choose one shared repository by convention.
- The choice between designs depends on the team and the project; Git suits most parallel, varied-network work.

New Vocabulary

- **Repository:** a project together with the complete record of its history.
- **Centralised version control:** one server holds the only complete history; users hold working copies.
- **Distributed version control:** every user holds a complete copy of the repository.
- **Clone:** a complete copy of a repository, with its history, made on another computer.
- **Single point of failure:** a part whose failure stops the whole system or loses something irreplaceable.

Commands Learned

None.

Common Mistakes

1. **Thinking "distributed" means there is no shared copy.**
2. **Assuming your local copy is only a snapshot.** It holds the whole history.
3. **Treating the shared copy as special.** It is a repository like yours.
4. **Choosing a tool by popularity alone.** Weigh the team, the project and the network.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What does a repository contain besides the project's files?
2. Why is a central server a single point of failure?
3. In a distributed system, what do you do when the network is down?
4. Give one strength and one cost of the distributed design.
5. Why does a distributed team still usually choose a shared repository?

Before Moving On

You are ready for Chapter 12 if you can:

- [] draw both designs from memory
- [] explain what "the whole history on my computer" makes possible
- [] say why "distributed" does not mean "no shared repository"
- [] state one situation in which a centralised design might still be chosen

How this chapter was checked

Claim	Evidence class	Ledger
Definitions and trade-offs of the two designs	Needs re-verification (design-level statements; no product behaviour is claimed)	R139

Where this leads

Chapter 12 separates the Git tool from hosting platforms. Chapter 15 gives Git's exact vocabulary for what a repository contains. Chapter 23 proves the distributed idea with two repositories on your own computer.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book.

Level 1 — Guided

1.1 Draw them. On paper, draw the centralised and distributed designs for three people. Label what each person holds.

Level 2 — Partially guided

2.1 Which design? For each statement say "centralised", "distributed" or "either": (a) "I can read the whole history with no network." (b) "There is one official copy by design." (c) "If the server is lost, every copy is still a full backup." (d) "My computer holds only one version of the files."

Level 3 — Independent

3.1 Explain to a friend why "distributed" does not mean "there is no shared server". Use the phrase "by convention".

Level 4 — Professional scenario

4.1 A school club of eight students and one teacher will keep the website source under version control. Students often work on trains and at home with poor connections. Recommend a design and justify it with three points from section 11.4. Name one thing the group must agree on.

Level 5 — Troubleshooting

5.1 A learner says: "My repository is just the latest files; the history is on GitHub." Identify the misunderstanding and correct it.

5.2 A team's shared server is down. Half the team says work must stop; the other half says it need not. Who is right, in a distributed design, and what can and cannot continue?

Chapter 12 — Git, GitHub, GitLab, Bitbucket [Core]

In this chapter

- the single most important distinction in this book: the tool versus the platform
- what Git is, and what it is not
- what a hosting platform adds, using GitHub as the main example
- how GitLab and Bitbucket relate, and how to choose without declaring a winner
- names, trademarks and independence

Before you start. Chapter 11: you know what a repository is, and that a shared repository is a convention, not a requirement.

Independent publication. This book is an independent educational work by Sharif Issah Tingane, issued under the SHARIF TECHNOLOGIES name. It is not affiliated with, endorsed by, sponsored by, or published by Git, GitHub, GitLab, Bitbucket, Microsoft, or any company mentioned. Names of products and companies are trademarks of their respective owners and are used here only to identify them.

Verification pending [R140]. Descriptions of products and companies in this chapter are general and have not yet been checked against each product's own documentation. Products change quickly; the chapter avoids statements about specific features, prices or ownership for that reason. Check current official sources before relying on any product detail.

12.1 Two different things with similar names

Almost every beginner confuses the following two words at first, and the confusion causes real errors, so this chapter is placed before you install anything.

Git is the version-control system.

GitHub is an online platform built around Git repositories and around collaboration on software.

Read them twice. They are **two different things, made by different people, that do different jobs**, and you can use either without the other.

New term: Git. A free, open-source distributed version-control system. It is a program that runs on your own computer and keeps the history of a project in a repository.

New term: hosting platform. An online service that stores shared copies of repositories for you, and adds features for working together around them.

New term: GitHub. A widely used hosting platform for Git repositories, with features for collaboration such as tracking issues, reviewing proposed changes and automating tasks.

A picture that helps: **Git is a camera. GitHub is a photo-sharing website.** The camera takes and keeps your photographs, on your own device, whether or not you ever share one. The website stores copies of photos so that other people can see them, comment on them and gather them into albums. You can use the camera without the website. The website is useless without photographs, and the photographs are made by the camera.

	Git	GitHub
What it is	A program	An online service
Where it runs	On your computer	On the platform's servers, reached through your browser and through Git itself
What it stores	The history of your project	Shared copies of projects, plus discussion around them
Needs the internet?	No, for almost everything	Yes
Who provides it	The open-source Git project	A company
Can you use one without the other?	Yes	It exists to host Git repositories

12.2 What lives where

Beginners often look for a feature in the wrong place. Use this table to know whether something belongs to the tool or to the platform.

Concept	Belongs to
Recording a version (a commit), reading history, comparing versions	Git , on your computer
Branches, merging, tags	Git
Copying history to or from another repository (push, pull, fetch)	Git (the other repository may be on a platform)
Issues, discussions, project boards	The platform
Pull requests and reviews	The platform (Chapter 45)
Automated workflows (Actions)	The platform (Chapter 54)

Concept	Belongs to
Profiles, organizations, stars, followers	The platform
Web pages for a project	The platform

Verification pending [R140]. Which features a given platform offers, under which names and conditions, is exactly the sort of statement that must come from its current documentation. Part V and Part VI verify each one before describing it.

Why this matters. When you type a Git command, you are using the tool, and it will work with any repository that Git understands, whoever hosts it. When you click a button on a website or write a workflow file, you are using the platform's features. Knowing which is which tells you where to look for help, and what still works when the platform is down or you change platform.

12.3 Git without any platform

A team does not need a hosting platform to use Git. The tool works entirely on your computer. It can also exchange history with a second repository that sits anywhere: on a memory stick, on a colleague's laptop, on a server you run yourself, or in a folder on the same machine. This has been tested: a folder on the same computer, prepared as a shared repository, worked as the "meeting place" for pushing and fetching history in the book's test environment on two versions of Git.

Checked against the documentation [R141]. The test is recorded in the script `research/git-verification/verify-git-basics.sh` in the companion repository (bare-repository checks), and the Git 2.56.0 manual for `git init` documents the `--bare` option ("Create a bare repository"). Both the test and the manual concern the tool, not any hosting platform.

Chapter 23 will let you repeat this experiment with two repositories on your own computer and no account at all. Then, in Part V, you will connect the same repository to GitHub, and you will see that only the address changes.

12.4 Other platforms

GitHub is not the only hosting platform. Others, such as **GitLab** and **Bitbucket**, also host Git repositories and add their own collaboration features.

New term: GitLab. A hosting platform for Git repositories, with collaboration and automation features, that can be used as a service or run on a team's own servers.

New term: Bitbucket. A hosting platform for Git repositories from a different company, with its own collaboration features.

Because they all host Git repositories, **your Git skills transfer**: the commands to record, compare, branch and merge are identical. What differs is the platform's own features and vocabulary. (GitHub calls a proposal to merge changes a pull request. Some other platforms use a different name for the same idea; check the platform's documentation.)

12.4.1 Choosing a platform

There is no universal winner. When a team chooses, it weighs questions such as:

1. **Where must the code live?** Some organizations must keep code on servers they control.
2. **What features does the team need?** Issue tracking, review, automated tests and deployment differ in style and depth.
3. **Who else is here?** If you want to contribute to open-source projects, you go where they are.
4. **Cost and limits.** Plans, limits and prices change; consult the current official pages.
5. **Integrations.** The tools you already use.
6. **Privacy and legal duties.** Where data is stored, and who may see it.

Every one of these is a question about the platform. None of them changes how you use Git.

12.5 Other version-control tools

Git is not the only version-control tool, either. Other tools exist, both centralised (in the sense of Chapter 11) and distributed. This book does not teach them. If you meet a project that uses one, the concepts of Part II still apply: history, versions, differences, merging, and the distinction between centralised and distributed designs.

Git versus GitHub versus GitLab versus Bitbucket, in one sentence each:

- **Git**: the tool that keeps the history.
 - **GitHub, GitLab, Bitbucket**: three platforms that host shared copies of Git repositories and add collaboration features.
-

12.6 Avoiding the common confusions

You may hear	The accurate version
"I pushed it to Git."	"I pushed it to a repository (for example on GitHub)." Git is the tool; the repository is the destination.
"Git is owned by GitHub."	Not so: Git and GitHub are different things from different sources. (See the verification notice.)
"I need GitHub to use Git."	No: Git works locally and with any repository.
"GitHub is version control."	GitHub is a platform that hosts Git repositories; the version control is done by Git.
"I lost my repository because GitHub is down."	Your local copy is complete (Chapter 11); only sharing is affected.

Checkpoint

What You Learned

- Git is the version-control system; GitHub is an online platform built around Git repositories and collaboration.
- Git runs on your computer and works without a network; a platform stores shared copies and adds features such as issues, reviews and automation.
- Features such as pull requests and Actions belong to the platform, not to Git.
- Git can exchange history with any repository, hosted anywhere or nowhere in particular.
- GitLab and Bitbucket are other platforms for Git repositories; Git skills transfer among them.
- Choosing a platform is a decision about features, location, cost and community, not about how Git works.

New Vocabulary

- **Git**: a free, open-source distributed version-control system that runs on your computer.
- **Hosting platform**: an online service that stores shared copies of repositories and adds collaboration features.
- **GitHub**: a widely used hosting platform for Git repositories.
- **GitLab**: another hosting platform for Git repositories.
- **Bitbucket**: another hosting platform for Git repositories, from a different company.

Commands Learned

None.

Common Mistakes

1. **Saying "Git" when you mean the platform, or the reverse.**
2. **Thinking you need a platform account to use Git.**
3. **Looking for platform features (pull requests, Actions) in Git's documentation.**
4. **Believing a Git repository belongs to a platform.** The history is yours.
5. **Choosing a platform by habit, without weighing the questions in section 12.4.1.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Complete the sentence: "Git is ..., and GitHub is ...".
2. Name two things that belong to Git and two that belong to a platform.
3. Can you use Git without GitHub? Explain.
4. What changes, and what stays the same, if a team moves from one platform to another?
5. Why is "I pushed it to Git" an imprecise sentence?

Before Moving On

You are ready for Part III if you can:

- [] explain the difference between Git and GitHub to a friend, using your own analogy
- [] say where a pull request "lives" (tool or platform)
- [] name two other hosting platforms
- [] explain why your local Git copy is complete without any platform

How this chapter was checked

Claim	Evidence class	Ledger
Descriptions of Git, GitHub, GitLab and Bitbucket; what belongs to tool versus platform	Time-sensitive; unverified (no feature, price or ownership claim is made)	R1 40
A Git repository needs no hosting platform; a local shared repository works as a meeting place	Locally tested on Git 2.43.0 and the CI runner's Git (bare-repository checks). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-init); the ledger row says which.	R1 41

Where this leads

Part III begins with Chapter 13: installing and checking Git on your computer, using the terminal from Chapter 7. Chapter 23 returns to the "tool versus platform" idea with a hands-on experiment, and Part V (starting at Chapter 36) introduces GitHub properly.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book.

Level 1 — Guided

1.1 Complete the sentences. "Git is ..., and GitHub is" Then write the camera-and-photo-site analogy in your own words.

Level 2 — Partially guided

2.1 Tool or platform? Sort into "Git (tool)" or "platform": recording a version; opening an issue; comparing two versions; reviewing a proposed change; creating a branch; a profile page.

Level 3 — Independent

3.1 A friend says: "I don't want a GitHub account, so I can't learn Git." Write a two-paragraph reply.

Level 4 — Professional scenario

4.1 Your organization must keep its code on its own servers. Explain, using section 12.4.1, which of your team's Git skills stay the same and which parts of the decision are about the platform.

Level 5 — Troubleshooting

5.1 Someone says: "GitHub lost our history when it went down." What is wrong with that sentence, and what would you check?

5.2 A colleague searches Git's documentation for "pull request" and finds nothing useful. What has happened, and where should they look?

Part III — Learning Git

Chapter 13 — Installing Git [Core]

In this chapter

- how to find out whether Git is already on your computer
- how to install it on Windows, macOS and Linux, and how to choose safe answers to the installer's questions
- how to prove that the installation worked, and what to do when it did not
- the three settings every new Git user makes first
- how to ask Git for help

Before you start. Chapter 7 (a terminal, commands, PATH), Chapter 6 (why installing from a trusted source matters) and Chapter 12 (Git is not GitHub). From this chapter on, the book's commands are for you to type: open your terminal now.

Every command in this chapter that is followed by output was run for real, in Bash and in zsh, and the recordings are checked automatically. The installation steps for Windows and macOS could not be run in the book's test environment, and are marked as such.

13.1 What you are installing

Git is a program (Chapter 12). Installing it puts the program on your computer and makes the command `git` available in your terminal. There is no account to create and nothing to register.

On **Windows**, the installer that Chapter 7 described is called Git for Windows. It installs Git together with **Git Bash**, the terminal environment in which this book's examples run. On **macOS** and **Linux**, you already have a suitable terminal, and Git is installed as one more program.

13.2 First, check whether Git is already there

Open your terminal (Git Bash on Windows, Terminal on macOS, your terminal on Linux) and type:

```
$ git --version
git version <version>
$ command -v git
/usr/bin/git
```

Recorded in Bash; *ch13-install/expected-install.bash.txt*.

Recorded on Linux. The first line shows your Git's version, so your number will differ; recordings in this book mark it *<version>* when they must not depend on it. The second command, *command -v*, asks the shell where it found the program (Chapter 7); the path will look different on your computer.

- If you see a line starting `git version`, Git is installed. Note the version, and you may skip to section 13.6.

- If you see command not found (or a message that git is "not recognized"), Git is not installed, or its folder is not on your PATH (Chapter 7). Continue with the section for your system.

Partly checked [R145]. The recordings show the behaviour of Git 2.43.0 and of the CI runner's Git on Linux, and Git's manual (2.56.0) confirms that `git --version` prints the version of the Git program. The PowerShell wording of the "not recognized" message is documented by Microsoft (Chapter 7); the Command Prompt wording has not been checked.

13.3 Before you install: three rules

1. **Use the official source.** Get Git from the project's own download page, from your operating system's official package manager, or from your organization's approved source, as Chapter 1 advised. Do not use a link from a message, an advertisement, or a "free tools" website.
2. **Expect an administrator prompt.** Installing a program for all users changes system folders (Chapter 1). The prompt is legitimate if you just started the installation. Refuse an administrator prompt that appears when you did not.
3. **Read each installer screen.** Do not click "Next" without looking. The questions matter (section 13.4), and you learn what your tools do.

13.4 Windows

Checked against the installer's source (R142). The Git for Windows installer is built from a script in the `git-for-windows/build-extra` repository (fetched from its main branch; the copy's hash is in the sources manifest, `research/sources-manifest.csv` in the companion repository). It has a page for each decision in the table. **Editor:** "Choosing the default editor used by Git"; the installer's own text says Vim "is the default editor of Git for Windows only for historical reasons, and it is highly recommended to switch to a modern GUI editor instead". **First branch:** "Let Git decide" (using the default branch name of that installer build, `master` in the script) or "Override the default branch name for new repositories", with "main", "trunk" and "development" named as common choices. **PATH:** "Git from the command line and also from 3rd-party software", marked "(Recommended)". **Line endings:** three choices; "Checkout Windows-style, commit Unix-style line endings" is described as "the recommended setting on Windows" for cross-platform projects (it sets `core.autocrlf` to true), "Checkout as-is, commit Unix-style line endings" as the recommended one on Unix (input), and "Checkout as-is, commit as-is" as "not recommended for cross-platform projects". **Credentials:** a page offers "Git Credential Manager". The installer was not run here, and its pages can change, so the section describes the decisions and not the screens.

1. Open the official Git download page (search for "Git for Windows" and check that the address belongs to the Git project) and download the installer for your version of Windows.

2. Run the installer, and accept the administrator prompt.
3. The installer asks a series of questions. The important ones, and sensible answers for a learner, are:

Decision	What it means	Recommended for this book
Which text editor Git should open	Git opens an editor when you write a long commit message (Chapter 19)	An editor you already know (Chapter 3). Avoid an editor that is hard to exit if you have never used it.
Name of the first branch	New repositories start with a branch of this name (Chapter 20)	main
How Git is added to your PATH	Whether git works from every terminal, or only in Git Bash	The option that lets you run Git from the command line and from other software, so it also works in PowerShell and Command Prompt
Line endings	How Git converts line endings between Windows (CRLF) and other systems (LF) (Chapter 4)	Read the choices; for cross-platform work the installer recommends "Checkout Windows-style, commit Unix-style line endings" (LF in the repository). Chapter 14 returns to this.
The terminal to use with Git Bash	The window program that displays Git Bash	Either choice works for this book
Credential helper	Whether Git remembers logins securely (Chapter 38)	Keep the offered default

Accept the defaults for every decision you do not understand. All of them can be changed later, and none of them can harm your computer.

1. When the installer finishes, **open a new terminal**: search for Git Bash and start it. (A terminal opened before the installation does not see the new PATH; Chapter 7.)
2. Type `git --version`.

UI-VERSION NOTE. Installer pages and option names change. If a decision is not in the table, search the installer's help or the official Git for Windows documentation for its name.

13.5 macOS and Linux

macOS

Checked in part (R143). The Homebrew package collection has a formula named `git` ("Distributed revision control system", homepage `git-scm.com`) that at the time of checking built Git 2.56.0 from the project's source tarball. How macOS offers developer tools on first use of `git`, and the Git project's own download page, were **not** checked.

Git may already be present on a Mac; the check in section 13.2 will tell you. If it is not, there are two kinds of source, in order of preference: the one your operating system offers (some systems offer to install developer tools when you first type `git`), and the Git project's own list of installation methods on its download page. Follow the official page for your version, and read what it says before running anything it gives you.

Linux

Linux systems install programs with a **package manager** (Chapter 1). On Debian-family systems such as Ubuntu, the package manager is `apt`, and the installation command has this form:

```
sudo apt-get install git
```

`sudo` runs the command with administrator permission, and the system may ask for your password. The book's test environment (Ubuntu 24.04) confirmed that this form is valid: the package manager's simulation option (`apt-get install -s git`) reported that Git was already the newest version available, with the package version `1:2.43.0-1ubuntu7.3`. In that label, `2.43.0` is Git's own version; the `1:` prefix and the `-1ubuntu7.3` suffix belong to the distribution's packaging. Distribution packages often lag behind the newest Git release.

Verification pending [R144]. Other distributions use other package managers, whose commands were not run. Use your distribution's own documentation, and check the result with `git --version`.

13.6 Proving that it worked

Whichever way you installed it, check from a new terminal window:

```
$ git --version
git version <version>
$ command -v git
/usr/bin/git
```

Recorded in Bash; `ch13-install/expected-install.bash.txt`.

You need to see `git version` followed by numbers. Three problems account for nearly every failure:

What you see	Probable cause	What to do
command not found / "not recognized"	The terminal was open during the installation, or git is not on the PATH	Close the terminal, open a new one, try again. Then check the installation choices (section 13.4).
An older version than expected	An older Git earlier on the PATH	Run command <code>-v git</code> (or where git in Command Prompt) to see which one runs
A git that does something odd	A different program with the same name earlier on the PATH	Same check: which program is it?

If a command finds a program in an unexpected place, the first PATH entry that contains it wins (Chapter 7).

13.7 Your first three settings

Git records **who** made each version, so before you make your first one you must tell Git your name and email address. You also decide the name of the first branch in new repositories. This is done with `git config`, which Chapter 14 explains in depth. For now, type these three commands, replacing the example name and address with your own:

```
$ git config --global user.name "Ada Learner"
$ git config --global user.email "ada@example.org"
$ git config --global init.defaultBranch main
$ git config --global --list
user.name=Ada Learner
user.email=ada@example.org
init.defaultbranch=main
```

Recorded in Bash; *ch13-install/expected-install.bash.txt*.

Recorded in Bash and zsh (identical), with the fictional user Ada Learner. Nothing is printed after each `git config` command: silence means success (Chapter 7). The last command lists what you set. Git lower-cases the setting names in this list: *init.defaultbranch*.

- `user.name` and `user.email` are written into every commit you make. Use the name and address you are happy for others to see.
- `init.defaultBranch` is the name given to the first branch of every new repository. This book uses `main`.
- `--global` means "for my user account on this computer, in every repository". Chapter 14 explains the other scopes.

Security note. An email address written into commits is public if you later share the repository publicly. Some hosting platforms let you use a privacy-preserving address for this purpose; decide before you publish. (That statement is about platforms and is verified in Chapter 37.)

Checked against the documentation (R146). Git's `user.name` and `user.email` documentation (Git 2.56.0) says the two variables "determine what ends up in the author and committer fields of commit objects", that the environment variables `GIT_AUTHOR_NAME`, `GIT_AUTHOR_EMAIL`, `GIT_COMMITTER_NAME`, `GIT_COMMITTER_EMAIL` and `EMAIL` override them, and that the name forms "conventionally refer to some form of a personal name". `init.defaultBranch` "allows overriding the default branch name e.g. when initializing a new repository". The privacy statement is general advice and is not in Git's documentation.

13.8 Asking Git for help

Git has help built in, in three styles:

1. `git help <command>` opens the **manual page** for a command, for example `git help commit`. On systems where manual pages are installed, it shows the full reference.
2. `git <command> -h` prints a short **summary** of a command's options. It does not depend on manual pages.
3. `git help -a` lists the commands, and `git help -g` lists the conceptual guides.

```
$ git status -h 2>&1 | head -3
usage: git status [<options>] [--] [<pathspec>...]

    -v, --[no-]verbose    be verbose
```

Recorded in Bash; *ch13-install/expected-install.bash.txt*.

Recorded on Linux. The command `git status -h` prints a usage summary; here the output was shortened to its first three lines with `head -3`, and the `2>&1` joins Git's two output streams so that `head` sees them both. Later lines list the options.

Checked against the documentation (R147). The `git help` manual page (Git 2.56.0) says the `man` program is used by default, `-a` lists all available commands and `-g` the concept guides. In the book's test environment the manual pages were **not installed**, so `git help <command>` printed a notice about that instead of the manual. That is a fact about that machine, not about Git. On your computer the manual pages are usually available; if they are not, `-h` and the official online documentation (Chapter 35) still work.

13.9 Keeping Git up to date

Git is updated regularly, and updates repair problems. Update it the way you installed it: with the package manager, or by running the newer installer. Nothing in your repositories is affected, because the way Git stores history is stable. Chapter 35 shows how to check the documentation for the version you have.

Checkpoint

What You Learned

- Installing Git puts the `git` program on your computer; no account is needed.
- Check for Git first with `git --version`; command not found means it is missing or not on the `PATH`.
- Install from the official source, expect (only) a legitimate administrator prompt, and read each installer screen.
- On Windows the installer also provides Git Bash; sensible choices are a familiar editor, `main` as the first branch name, and Git available from the command line.
- Verify from a new terminal window; most failures are a stale terminal or a `PATH` problem.
- Set `user.name`, `user.email` and `init.defaultBranch` with `git config --global`.
- Git's help comes as manual pages (`git help`), short summaries (`git <command> -h`) and online documentation.

New Vocabulary

- **git config**: the command that reads and changes Git's settings.
- **Global setting**: a Git setting that applies to your user account in every repository.

Commands Learned

`git --version`, `command -v git`, `git config --global user.name`, `git config --global user.email`, `git config --global init.defaultBranch`, `git config --global --list`, `git <command> -h`, `git help`.

Common Mistakes

1. **Not opening a new terminal after installing.**
2. **Downloading Git from a source that is not official.**
3. **Clicking through installer screens** without reading them.
4. **Skipping `user.name` and `user.email`**, then finding commits recorded under an unwanted identity (Chapter 14).
5. **Confusing the version of the package manager's entry with Git's own version.**
6. **Assuming `git help` works without manual pages installed.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. How do you check whether Git is installed?
2. You installed Git while a terminal was open, and `git --version` says the command is not found. What is the likeliest reason, and what do you do?
3. Why must you set `user.name` and `user.email`?
4. What does `--global` mean?
5. Name two ways to get help for `git commit` without using the internet.

Before Moving On

You are ready for Chapter 14 if you can:

- [] show `git --version` in your own terminal
- [] say where the `git` program was found (`command -v git`)
- [] have set your name, email and default branch
- [] explain the difference between `git help commit` and `git commit -h`

How this chapter was checked

Claim	Evidence class	Ledger
<code>git --version</code> , <code>command -v git</code> , and the three <code>git config</code> commands and the listed result; behaviour of the <code>-h</code> summary	Locally tested in Bash 5.2 and zsh 5.9 on Git 2.43.0; re-run in CI on the runner's Git	R145 , R146
Installer decisions and recommended answers for Windows	Checked against the installer script's source; installer not run	R142
macOS installation methods	Homebrew formula checked; Apple's developer-tools prompt and the Git download page not checked	R143
<code>apt install</code> form on Debian-family systems; other distributions	Locally tested for Ubuntu 24.04 (simulation); other distributions unverified	R144
Manual pages not installed in the test environment	Locally observed; environment-specific	R147

Where this leads

Chapter 14 explains where Git's settings live, in which order they apply, and why the same command can behave differently on another computer. Chapter 15 then describes the parts of Git that the commands work on, and Chapter 16 uses them.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use your practice folder; nothing here can damage your computer.

Level 1 — Guided

1.1 Check. Open a new terminal and run `git --version`. Expected result: a line starting `git version` followed by numbers. If you see "command not found", follow section 13.4 or 13.5 for your system, then check again from a **new** terminal.

1.2 Locate. Run `command -v git` (in Command Prompt: `where git`). Write down where the program was found.

1.3 Identify yourself. Run the three `git config --global` commands of section 13.7 with your own name and address, then `git config --global --list`. Expected result: your three settings, with the setting name `init.defaultbranch` in lower case.

Level 2 — Partially guided

2.1 Read the summary. Run `git commit -h` and find the option that lets you supply the message on the command line. Hint: look for `-m`. Write it down with its one-line description.

2.2 Which help? For each need, say whether `git help <command>`, `git <command> -h` or the online documentation is best, and why: (a) you want the full reference for `git merge` and manual pages are installed; (b) you are on a train with no manual pages; (c) you want to compare behaviour across versions.

Level 3 — Independent

3.1 Explain to a friend, in four sentences, why they need to set `user.name` and `user.email` before their first commit, and what "global" means.

Level 4 — Professional scenario

4.1 Your organization has 20 new employees who will each install Git. Write a one-page checklist for them: where to download it from, how to verify the installation, which three settings to make, and what to do if `git` is "not found". Add one item about safe installation from Chapter 1.

Level 5 — Troubleshooting

5.1 A learner installed Git, opened the same terminal window that was already open, and typed `git --version`. The message says the command is not found. Explain why, and give the fix.

5.2 A learner runs `git --version` and sees an older version than the one they just installed. What might be happening, and which command shows which program actually runs?

Chapter 14 — Configuration Scopes and Environments [Core]

In this chapter

- what a Git setting is, and the four scopes (levels) at which it can be stored
- how to see which value Git is really using, and where it came from
- how one command can override a setting for a moment
- how the environment of your computer changes what Git does
- why the same command can behave differently on another computer, shown with a real failure
- how to test Git commands in a clean, isolated environment

Before you start. Chapter 13: you have used `git config --global` to set your name, email and default branch. Chapter 7 introduced environment variables, which return here.

Every command with output in this chapter was run for real, in Bash and in zsh, on Git 2.43.0, and re-run in CI on a newer Git. The recordings use the fictional user Ada Learner.

14.1 What a setting is

Git does not behave the same way for everybody, because it is configurable. Your name and email, the name of the first branch, which editor to open, how to treat line endings, whether to sign commits: each is a **setting**.

New term: configuration (setting). A named value that changes how Git behaves. A setting has a name of the form `section.name` (for example `user.email`) and a value.

You read and change settings with the command you met in Chapter 13:

```
git config user.email
git config user.email "ada@example.org"
```

The first form **reads** the value; the second **writes** it. But a setting can be stored in several places, and that is the subject of this chapter.

14.2 Four scopes

New term: scope. The level at which a setting is stored, which decides how widely it applies.

Git reads settings from up to four places, from the widest to the narrowest.

Scope	Applies to	Where it is stored	Written with
System	Every user of this computer	A file that belongs to the Git installation (its location depends on how Git was installed)	<code>git config --system</code> (usually needs administrator permission)
Global	You, in every repository	A file in your home folder, <code>.gitconfig</code>	<code>git config --global</code>
Local	One repository	The file <code>.git/config</code> inside that repository	<code>git config --local</code> (the default when you are inside a repository and give no scope)
Worktree	One working tree of a repository (Chapter 31)	A separate file, and only when enabled	<code>git config --worktree</code>

Checked against the documentation (R148). The `git config` manual (Git 2.56.0) lists the files: the system file `$(prefix)/etc/gitconfig`; the user files `$XDG_CONFIG_HOME/git/config` and `~/.gitconfig` (with `$HOME/.config/` used when `XDG_CONFIG_HOME` is not set), "also called 'global'", both read if both exist; the repository file `$GIT_DIR/config`; and the optional `$GIT_DIR/config.worktree`, which is "only searched when `extensions.worktreeConfig` is present". The files are read in that order, "with last value found taking precedence". "By default, options are only written to the repository specific configuration file", and `git config` "will only ever change one file at a time". Where the system file lives on Windows and macOS was not tested: the documentation gives only the build-time `$(prefix)/etc/gitconfig`.

The rule for conflicts is simple and worth memorising: **the narrower scope wins**. If `user.email` is set globally and again locally, Git uses the local one in that repository. In order of increasing priority: system, global, local, worktree.

14.2.1 See it work: local beats global

The recording below starts with a global email, creates a repository, adds a different local email, and reads the value each time. Read each result, and compare with the rule:

```
$ git config --global user.name "Ada Learner"
$ git config --global user.email "ada@example.org"
$ git init -b main scopes-demo
Initialized empty Git repository in /home/learner/scopes-demo/.git/
$ cd scopes-demo
$ git config user.email
ada@example.org
$ git config --local user.email "ada.work@example.net"
$ git config user.email
ada.work@example.net
$ git config --show-origin --show-scope user.email
local    file:.git/configada.work@example.net
```

```
$ git config --get-all user.email
ada@example.org
ada.work@example.net
```

Recorded in Bash; *ch14-config/expected-scopes.bash.txt*.

- The global value, `ada@example.org`, was used until a local value existed.
- After the local value was set, `git config user.email` printed the **local** one.
- `--show-origin --show-scope` told us where the winning value came from: scope local, file `.git/config`.
- `--get-all` showed **both** values, from all scopes, in order from widest to narrowest. Git uses the last.

14.2.2 Removing settings, and what "unset" looks like

```
$ git config --global --unset user.email
$ git config --local --unset user.email
$ git config user.email; echo "exit code: $?"
exit code: 1
```

Recorded in Bash; *ch14-config/expected-scopes.bash.txt*.

After both values were removed, `git config user.email` printed nothing at all, and its **exit code** was 1. (Chapter 7 noted that most commands print nothing on success. Here the silence means no value, and the exit code says so.) A Git that has no email will later refuse to create a commit, or guess an address from your computer's name, which is why Chapter 13 asked you to set one.

14.3 The system scope, simulated

The system scope needs administrator permission to change, so to see it safely, the next recording builds a stand-in. It writes a small file into the home folder and points Git at it with the environment variable `GIT_CONFIG_SYSTEM`. (You will not need to do this in your own work; it is a way to observe how the scopes stack.)

```
$ printf '[demo]\n\tlevel = system\n' > "$HOME/system.gitconfig"
$ export GIT_CONFIG_SYSTEM="$HOME/system.gitconfig"
$ git init -b main levels-demo
Initialized empty Git repository in /home/learner/levels-demo/.git/
$ cd levels-demo
$ git config demo.level
system
$ git config --global demo.level global
$ git config demo.level
global
$ git config --show-scope --get-all demo.level
system system
global global
$ git config --local demo.level local
$ git config demo.level
local
$ git config --show-scope --get-all demo.level
```

```
system  system
global  global
local   local
```

Recorded in Bash; *ch14-config/expected-levels.bash.txt*.

Each `git config demo.level ...` command changed a different scope, and each time the value Git used moved to the narrowest one. The `--show-scope --get-all` lines show the whole stack: system, then global, then local.

```
$ git config --show-origin --get-all demo.level
file:/home/learner/system.gitconfig      system
file:/home/learner/.gitconfig            global
file:./git/configlocal
```

Recorded in Bash; *ch14-config/expected-levels.bash.txt*.

`--show-origin` names the files: the system file, the global file `~/.gitconfig`, and the local file `.git/config`.

Deep. The fourth scope, `worktree`, exists for repositories with several working trees (Chapter 31). It must be switched on with `extensions.worktreeConfig`. The recording shows that once it is on, its value wins over the local one:

```
$ git config extensions.worktreeConfig true
$ git config --worktree demo.level worktree
$ git config demo.level
worktree
$ git config --show-scope --get-all demo.level
system  system
global  global
local   local
worktreeworktree
```

Recorded in Bash; *ch14-config/expected-levels.bash.txt*.

14.4 Overriding for one command, or for one shell

Sometimes you want a different value for a moment without changing any file.

One command: `-c`. Put `-c name=value` between `git` and the command:

```
$ git config --global user.email "ada@example.org"
$ git -c user.email="temporary@example.invalid" config user.email
temporary@example.invalid
```

Recorded in Bash; *ch14-config/expected-scopes.bash.txt*.

The temporary value was used for that command only. It changed no file.

One shell, or one command: environment variables. Git reads environment variables (Chapter 7) that change where it looks for settings:

Variable	Effect
GIT_CONFIG_GLOBAL	Use this file instead of ~/.gitconfig for the global scope. /dev/null means "no global settings".
GIT_CONFIG_SYSTEM	Use this file instead of the system file.
GIT_CONFIG_NOSYSTEM	If set to 1, ignore the system file.

The first recording line below hides the global settings for a single command; the next shows they were untouched:

```
$ GIT_CONFIG_GLOBAL=/dev/null git config user.email; echo "exit code: $?"
exit code: 1
$ git config user.email
ada@example.org
```

Recorded in Bash; *ch14-config/expected-scopes.bash.txt*.

The exit code 1 shows that with no global file, user.email had no value; afterwards, the ordinary command sees the global file again. The last recording of the scope demonstration shows that GIT_CONFIG_NOSYSTEM=1 removes the system scope from the stack even though GIT_CONFIG_SYSTEM was set:

```
$ GIT_CONFIG_NOSYSTEM=1 git config --show-scope --get-all demo.level
global  global
local   local
worktree  worktree
```

Recorded in Bash; *ch14-config/expected-levels.bash.txt*.

Precedence, summarised. For one command Git combines: system, then global, then local, then worktree, then -c on the command line, in that increasing order of priority. (A few settings can also be overridden by their own environment variables.) -c and the environment variables are the tools to use when you need an exact, reproducible environment, which is the subject of section 14.7.

Checked against the documentation (R149). The git manual (Git 2.56.0) says of GIT_CONFIG_GLOBAL and GIT_CONFIG_SYSTEM: they "take the configuration from the given files instead from global or system-level configuration files", and can be set to /dev/null to skip reading the files of that level. GIT_CONFIG_NOSYSTEM is a Boolean that skips the system-wide file, and the manual says it can be used with \$HOME and \$XDG_CONFIG_HOME "to create a predictable environment for a picky script", which is how the recordings are made.

14.5 Why the same command behaves differently on another computer

This is the reason for the whole chapter. Two people run the same Git command on the same repository and get different results. The commonest cause is **settings that one of them inherited**, from a system file, a global file or an included file, that the other does not have.

Here is a real failure, reproduced on purpose. A computer's global settings say: sign every commit, use an SSH key for signing, and use a program that does not exist for the job. (Signing commits is explained in Chapter 33; you do not need it now. Many employers set this up for their staff, and a laptop that has been configured that way can surprise its owner.)

```
$ git config --global user.name "Ada Learner"
$ git config --global user.email "ada@example.org"
$ git init -b main signing-demo
Initialized empty Git repository in /home/learner/signing-demo/.git/
$ cd signing-demo
$ echo "Sunrise Bakery" > README.md
$ git add README.md
$ git config --global commit.gpgsign true
$ git config --global gpg.format ssh
$ git config --global gpg.ssh.program false
$ git commit -m "Add README"
fatal: either user.signingkey or gpg.ssh.defaultKeyCommand needs to be configured
$ git status --short
A README.md
```

Recorded in Bash; *ch14-config/expected-signing.bash.txt*.

Look at what happened. The learner typed an ordinary `git commit -m "Add README"`. Nothing in that command mentions signing, yet Git stopped with:

```
fatal: either user.signingkey or gpg.ssh.defaultKeyCommand needs to be configured
```

The file was staged (A README.md) but not committed. The cause was not in the command, the repository, or the file; it was an **inherited setting**. The next lines show how to find it and how to get past it:

```
$ git config --show-origin --get commit.gpgsign
file:/home/learner/.gitconfig true
$ git -c commit.gpgsign=false commit -m "Add README"
[main (root-commit) c3f6cc7] Add README
1 file changed, 1 insertion(+)
create mode 100644 README.md
$ git log --oneline
c3f6cc7 (HEAD -> main) Add README
$ GIT_CONFIG_GLOBAL=/dev/null git config commit.gpgsign; echo "exit code: $?"
exit code: 1
```

Recorded in Bash; *ch14-config/expected-signing.bash.txt*.

Two things solved the puzzle:

1. `git config --show-origin --get commit.gpgsign` said that the value `true` came from the global file, `~/.gitconfig`. That is the diagnostic habit: ask Git where a value comes from.
2. `git -c commit.gpgsign=false commit ...` showed that the command works when the setting is overridden for one command.

The last line, with `GIT_CONFIG_GLOBAL=/dev/null`, showed that with no global file the setting does not exist. (You would then decide whether to change the global file, or to keep signing and configure a key properly.)

Note on the error text (R150). The error text shown is what Git 2.43.0 printed in this exact configuration. Other configurations produce other messages. The same recording is re-run in CI on Git 2.55.0, and a difference in Git's wording would show up there as a failing check.

14.5.1 Settings that commonly differ between computers

Setting	Effect when it differs
<code>commit.gpgsign</code> , <code>user.signingkey</code>	Commits are signed, or fail to be (as above)
<code>core.autocrlf</code> , <code>core.eol</code>	Line endings are converted differently (Chapter 4)
<code>pull.rebase</code> , <code>pull.ff</code>	<code>git pull</code> merges, rebases or refuses (Chapter 23)
<code>init.defaultBranch</code>	The first branch is <code>main</code> on one computer and <code>master</code> on another
<code>core.editor</code>	A different editor opens for commit messages
<code>credential.helper</code>	Passwords and tokens are remembered differently (Chapter 38)
<code>alias.*</code>	Commands mean something different because of a personal shortcut (Chapter 32)
<code>include.path</code> , <code>includeIf</code>	Extra files change any of the above (section 14.6)

14.5.2 A diagnostic routine

When Git behaves unexpectedly:

1. Say exactly what happened (the command and the message).
2. List where the relevant settings come from: `git config --show-origin --show-scope --get-all <name>`.
3. Reproduce with a clean environment (section 14.7) to see whether a setting is responsible.
4. Change the setting at the scope where it belongs, not everywhere.

14.6 Conditional includes [Deep]

A configuration file can include another, and the include can apply only inside certain folders. That is how a person keeps one email address for work projects and another for personal ones.

Deep. Here the global file includes a work file only for repositories under `~/work/`. The same command prints different values in two repositories:

```
$ git config --global user.name "Ada Learner"
$ git config --global user.email "ada@example.org"
$ mkdir -p work/project personal/project
$ printf '[user]\n\temail = ada@work.example.net\n' > ~/.gitconfig-work
$ git config --global includeIf.gitdir:~/work/.path ~/.gitconfig-work
$ cat ~/.gitconfig
[user]
    name = Ada Learner
    email = ada@example.org
[includeIf "gitdir:~/work/"]
    path = /home/learner/.gitconfig-work
$ cd work/project
$ git init -q
$ git config user.email
ada@work.example.net
```

Recorded in Bash; *ch14-config/expected-include.bash.txt*.

The `includeIf "gitdir:~/work/"` line means: for repositories whose `.git` folder is under `~/work/`, also read the given file. Inside `work/project`, `git config user.email` printed the work address, and `--show-origin` proved that it came from `.gitconfig-work`; in `personal/project`, the global address applied.

Checked against the documentation (R151). The `git config` manual (Git 2.56.0) documents `includeIf` with the conditions `gitdir`, `gitdir/i` (case-insensitive), `onbranch`, `hasconfig:remote.*.url` and `worktree`. For `gitdir`, a pattern ending in `/` gets `**` added ("it matches 'foo' and everything inside, recursively"), a pattern that does not start with `~/`, `./` or `/` gets `**/` put in front, and `../` "is not special and will match literally". Both the symlink and the real path of a directory match. Behaviour of `gitdir:` patterns for unusual paths (for example on Windows) was not tested.

14.7 Testing in a clean environment

Everything above suggests a rule for anyone who writes instructions, scripts or tests that involve Git, including this book's author: **do not test in your own environment. Test in an isolated one.** Your global settings, your system file and your environment variables all leak into your tests, so a command that "works for me" might depend on a setting that nobody else has.

A clean environment for one command needs three things:

1. **A private home folder**, so that `~/.gitconfig` is not yours.
2. `GIT_CONFIG_GLOBAL=/dev/null` and `GIT_CONFIG_NOSYSTEM=1` (or an explicit `GIT_CONFIG_SYSTEM` file you control), so that no outside settings apply.
3. **Explicit settings for everything the test needs** (name, email, default branch).

Every recording in this book is made this way. A test harness starts a fresh home for each recording, points system settings at nothing, and sets the few settings the example needs, so that the recordings show Git's behaviour and not one particular laptop's. When you write a script for your team, do the same, or someone's inherited settings will one day break it.

14.8 Settings you will meet later

Do not change these now, but recognise their names when they appear.

- `core.editor`: the editor Git opens (Chapter 19).
- `core.autocrlf` and `.gitattributes`: line-ending conversion (Chapter 32).
- `core.excludesFile`: your personal ignore list (Chapter 18).
- `pull.rebase` and `pull.ff`: what `git pull` does (Chapter 23).
- `alias.<name>`: your own shortcuts (Chapter 32).
- `commit.gpgsign`, `gpg.format`, `user.signingkey`: signing (Chapter 33).
- `credential.helper`: remembering logins (Chapter 38).

Security note. Never copy a block of Git settings from a stranger's web page or a "dotfiles" repository without reading it. A setting can run a program (for example an editor, a credential helper or an alias that starts with `!`), so a copied file can do anything you can do.

Checkpoint

What You Learned

- A Git setting is a named value; it can live in the system, global, local or worktree scope, and the narrower scope wins.
- `git config <name>` reads the effective value; `--show-origin --show-scope --get-all` shows every value and where it came from.
- Writing defaults to the local file inside a repository; `--global` and `--system` write to the wider scopes.
- `-c name=value` overrides a setting for one command; `GIT_CONFIG_GLOBAL`, `GIT_CONFIG_SYSTEM` and `GIT_CONFIG_NOSYSTEM` change which files are read.
- An inherited setting can change what a command does, so the same command can fail on another computer; the diagnostic habit is to ask where a value came from.
- Conditional includes give different settings in different folders.
- Test Git commands in an isolated environment.

New Vocabulary

- **Configuration (setting)**: a named value that changes how Git behaves.
- **Scope**: the level (system, global, local or worktree) at which a setting is stored.
- **Override**: to replace a setting's value for one command or one shell.

Commands Learned

`git config <name>`, `git config --local|--global|--system|--worktree <name> <value>`, `git config --unset`, `git config --show-origin`, `git config --show-scope`, `git config --get-all`, `git -c name=value <command>`, and the environment variables `GIT_CONFIG_GLOBAL`, `GIT_CONFIG_SYSTEM`, `GIT_CONFIG_NOSYSTEM`.

Common Mistakes

1. **Setting a value at the wrong scope**, then wondering why another repository behaves differently.
2. **Assuming your settings are everyone's**. A friend's computer has different ones.
3. **Not checking where a value came from** when behaviour is unexpected.
4. **Copying configuration from the internet** without reading it.
5. **Testing scripts in your own environment**.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name the four scopes, from widest to narrowest.
2. `user.email` is set globally and locally. Which does Git use, and how can you prove where the winning value came from?
3. What does the exit code 1 of `git config user.email` tell you?
4. How do you override a setting for a single command?
5. A colleague's commit fails with a signing error that yours does not. What is your first diagnostic step?
6. Why should tests of Git commands run in an isolated environment?

Before Moving On

You are ready for Chapter 15 if you can:

- [] list the scope and file of a setting with one command
- [] set a value locally and see it override the global value
- [] override a setting for one command with `-c`
- [] explain how an inherited setting can break a command

How this chapter was checked

Claim	Evidence class	Le dg er
Scopes, precedence, <code>--show-origin</code> , <code>--show-scope</code> , <code>--get-all</code> , <code>unset</code> behaviour and exit codes; <code>worktree</code> scope with <code>extensions.worktreeConfig</code>	Both: run in Bash 5.2 and zsh 5.9 on Git 2.43.0 and re-run in CI on a newer Git; checked against the Git 2.56.0 manual	R1 48

Claim	Evidence class	Le dg er
-c, GIT_CONFIG_GLOBAL, GIT_CONFIG_SYSTEM, GIT_CONFIG_NOSYSTEM	Same; variables checked in the git manual	R1 49
The inherited-signing failure and its message	Locally tested; the message is version- and configuration-specific	R1 50
includeIf "gitdir:"	Locally tested; conditions and matching rules checked in the Git 2.56.0 manual	R1 51

Where this leads

Chapter 15 describes the parts of Git that these settings and the following commands act on: the working tree, the index, the repository and HEAD. Chapter 32 returns to settings that change Git's behaviour permanently: aliases, hooks, attributes and credential helpers. Chapter 33 explains commit signing, the setting that broke a commit in section 14.5.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Work in a new practice folder, and do not change settings you do not understand.

Level 1 — Guided

1.1 Where does it come from? In any repository, run `git config --show-origin --show-scope user.email`. Expected result: one line with a scope, a file and your address.

1.2 Local beats global. Create a repository (`mkdir scope-lab, cd scope-lab, git init`). Set a different local email with `git config --local user.email "lab@example.org"`. Show the effective value, then all values with `--get-all`. Expected result: the effective value is the local one; `--get-all` lists both.

1.3 One command only. Use `git -c user.email="temp@example.invalid" config user.email` and then `git config user.email` without `-c`. Expected result: the first prints the temporary value; the second prints the stored value.

Level 2 — Partially guided

2.1 Remove and restore. Remove the local value from exercise 1.2 with `--unset`. Predict, then check, which value is now effective. What does the exit code of `git config user.email` tell you when no value exists at any scope?

2.2 Hide the global file. Run `GIT_CONFIG_GLOBAL=/dev/null git config user.email`. Explain what you saw. (In Command Prompt or PowerShell the syntax for a temporary environment variable differs; use Git Bash for this exercise.)

Level 3 — Independent

3.1 Set up two folders, `~/work/` and `~/personal/`, and configure Git so that repositories under work use a work email and everything else uses your personal one, with a conditional include (section 14.6). Prove it with `--show-origin`.

Level 4 — Professional scenario

4.1 A new team member's commits are signed on their laptop but the CI system rejects your unsigned commits from a shared build server. Describe how you would find out which settings differ between the machines, and where you would change the setting so that only the build server is affected.

Level 5 — Troubleshooting

5.1 A colleague's `git commit -m "Update"` fails with `fatal: either user.signingkey or gpg.ssh.defaultKeyCommand needs to be configured, although they have not touched signing`. Give three diagnostic steps in order, and the safest way to commit while you investigate.

5.2 Two people run `git pull` on the same repository. One gets a merge, the other gets an error about divergent branches. Which setting family (section 14.5.1) is the likeliest cause, and what command shows the difference?

5.3 You are writing a script that runs `git commit`. It works on your laptop but fails on the build server. Name three settings that may differ, and how to make the script independent of them.

Chapter 15 — Git's Core Model [Core]

In this chapter

- the five ideas on which every Git command rests: the working tree, the index, the repository, the commit, and the pointers HEAD and branch
- a picture of how they fit together, which you will use for the rest of the book
- the four states a file can be in, and how `git status` reports each one
- a look at what Git really stores, with real output, so that nothing is magic

Before you start. Chapter 9 (why version control exists), Chapter 11 (the repository as a project with its history), and Chapter 14 (you have a name and email set). You will type commands in this chapter; every recording was run for real.

This chapter is the most important one in Part III. Take your time, and come back to its pictures whenever a later chapter feels confusing. Almost every Git surprise is a misunderstanding of one of the ideas here.

15.1 Snapshots

Chapter 9 asked for a system that remembers every version of a project. Git's answer is simple to state: **each time you record your work, Git stores a complete snapshot of the project as it is at that moment**, together with a note saying who recorded it, when and why.

New term: snapshot. A complete picture of all the files of a project at one moment.

You can think of a photograph album. Each photograph is the whole scene at one instant; you can flip back to any of them. (Behind the scenes Git stores snapshots very efficiently, so the album does not grow as fast as this picture suggests. Chapter 30 shows how.)

Everything else in this chapter answers three questions. **Where do I edit? How do I choose what goes into the next snapshot? Where do the snapshots live?**

15.2 Three places

Git's model has three places, and the first thing to learn is to keep them apart.

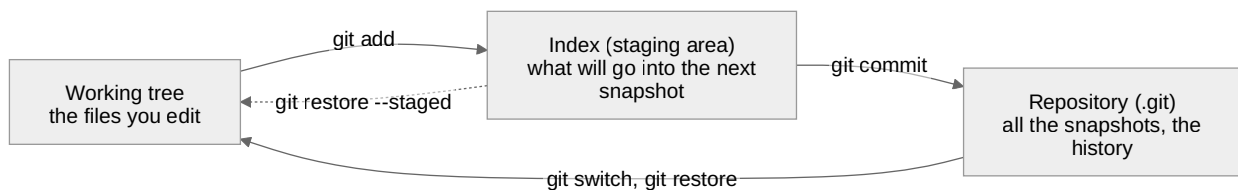


Figure 10. 15.2 Three places

Diagram description: three boxes in a row. On the left, the working tree, the files you edit. In the middle, the index, or staging area. On the right, the repository. An arrow labelled `git add` runs from the working tree to the index. An arrow labelled `git commit` runs from the index to the repository. An arrow labelled `git switch`, `git restore` runs from the repository back to the working tree. A dotted arrow labelled `git restore --staged` runs from the index back towards the working tree.

15.2.1 The working tree

New term: working tree. The folder of files you actually see and edit: your project as it is now, including changes not yet recorded.

It is an ordinary folder. Your editor opens files from it; your browser opens `index.html` from it. Git does not stop you from changing anything in it.

15.2.2 The index (staging area)

New term: index (staging area). A holding area that lists exactly what will go into the next snapshot. Its formal name is the index; most people call it the staging area, and this book uses both.

The index is what makes Git flexible. You may have edited five files, but only two belong in the change you are about to record. You **stage** those two (`git add`), leaving the others out, and the commit contains only what is staged. Think of packing a parcel: the working tree is the room full of things, the index is the box you are filling, and the commit is the sealed and labelled parcel.

New term: stage. To add a file's current contents to the index, so that they will be included in the next commit.

15.2.3 The repository

New term: local repository. The complete history of the project, kept on your computer in a hidden folder called `.git` inside the project folder.

The **repository** holds the snapshots. `.git` is created by `git init` (Chapter 16). If you delete the `.git` folder, the project folder is left, but its history is gone, so never modify `.git` by hand.

The rule of thumb: edit in the working tree, stage into the index, commit into the repository.

15.3 Watching the three places

The recording below builds a project from nothing and runs `git status` at each step. `git status` is the command that tells you which files are in which place. Read each report in turn. (The commands are explained one by one in Chapter 16; for now, follow what the reports say.)

```
$ git config --global user.name "Ada Learner"
$ git config --global user.email "ada@example.org"
$ git init -b main model-demo
Initialized empty Git repository in /home/learner/model-demo/.git/
$ cd model-demo
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Recorded in Bash; *ch15-model/expected-model.bash.txt*.

A new, empty repository. No commits yet means the repository has no history, and nothing to commit means there is nothing new in the working tree.

Now a file appears in the working tree:

```
$ echo "Sunrise Bakery" > README.md
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md

nothing added to commit but untracked files present (use "git add" to track)
```

Recorded in Bash; *ch15-model/expected-model.bash.txt*.

README.md is **untracked**: it exists in the working tree, but Git has never been told to include it in a snapshot. Git says so, and suggests the next step, `git add`.

Now stage it, and record it:

```
$ git add README.md
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md

$ git commit -m "Add README"
[main (root-commit) 9b522e5] Add README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
$ git status
On branch main
nothing to commit, working tree clean
```

Recorded in Bash; *ch15-model/expected-model.bash.txt*.

After `git add`, the report changed to Changes to be committed: new file: README.md. That is the staging area speaking: the file is in the index, ready for the next commit. After `git commit`, the working tree is clean: everything in the working tree matches the last snapshot, and nothing is staged.

Next, a change to a file that Git already knows:

```
$ echo "Fresh bread every morning." >> README.md
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
$ git add README.md
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md

$ git commit -m "Add opening line"
[main c3c1c38] Add opening line
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch15-model/expected-model.bash.txt*.

Read the first report: Changes not staged for commit ... modified: README.md. The file changed in the working tree, but the change has not been staged, so a commit now would not include it. After `git add`, the report says Changes to be committed ... modified: README.md: the change is staged. The commit then recorded it.

15.3.1 The four states of a file

That walk-through showed every state a file can be in. Learn them as a set.

State	Meaning	git status says	Where the change is
Untracked	Git has never been told about this file	Untracked files	Working tree only
Modified	A file Git tracks has changed since the last commit	Changes not staged for commit	Working tree
Staged	The change is in the index, ready to be committed	Changes to be committed	Index (and working tree)
Committed	The change is in a snapshot in the repository	nothing to commit, working tree clean	Repository

New term: tracked file. A file that Git knows about, because it is in the last commit or in the index.

Note on wording (R154). The status messages above were recorded on Git 2.43.0 and re-run in CI on Git 2.55.0, where they were identical. The wording is Git's own and can change between versions, which is why the book shows recorded output and not descriptions of it.

15.4 The commit

New term: commit. A recorded snapshot of the project, together with a note about who made it, when, and why, and a link to the commit that came before it. Also the action of creating one.

A commit is the unit of history. Each commit has:

1. **A snapshot** of all tracked files as staged.
2. **An author and a committer**, with name, email and a time.
3. **A message**, in the author's words, saying why.
4. **A parent**: the commit that came immediately before it (the first commit has none).
5. **An identifier**: a long code, called a hash, that names this commit and no other.

New term: hash (commit identifier). A long string of letters and digits, computed by Git from the commit's contents, that identifies it uniquely. It is usually shown shortened to its first seven characters.

The log of the demonstration project shows two commits:

```
$ git log --oneline
c3c1c38 (HEAD -> main) Add opening line
9b522e5 Add README
```

Recorded in Bash; *ch15-model/expected-model.bash.txt*.

You can look inside the newest one. The next recording asks Git for the type of what HEAD names (a commit) and then prints its raw contents:

```
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree 2152ed97942876843c1435fe8f6159d07b509541
parent 9b522e59f1a9c908f07b62485b3f13af75a0def2
author Ada Learner <ada@example.org> 1767604380 +0000
```

```
committer Ada Learner <ada@example.org> 1767604380 +0000
Add opening line
```

Recorded in Bash; *ch15-model/expected-model.bash.txt*.

Read the output of `git cat-file -p HEAD`:

- tree 2152ed9... is the **snapshot**: a pointer to the stored picture of the whole project.
- parent 9b522e5... is the **previous commit** (the first one from the log above).
- author and committer record who, and when. (The number is the time in seconds since the start of 1970; Git prints it in this raw form here.)
- The last line is the **message**.

Because each commit names its parent, commits form a **chain**, and the chain is the history:

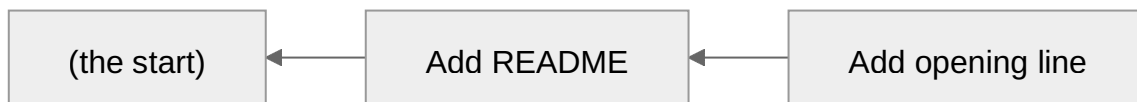


Figure 11. 15.4 The commit

Diagram description: two commits in a row. The newer one, "Add opening line", points back to its parent, "Add README", which points back to the start of history. Arrows run from newer to older.

Why the hash matters. The hash is calculated from everything in the commit, including its parent's hash. Change anything in an old commit and its hash changes, and so does every later commit's. That is how Git notices tampering: history cannot be altered without leaving a different chain (Chapter 30 explains it fully).

Checked against the documentation (R152). Git's glossary (Git 2.56.0) defines a commit as "a single point in the Git history", and, as a verb, "the action of storing a new snapshot of the project's state in the Git history, by creating a new commit representing the current state of the index and advancing HEAD to point at the new commit". A commit object "contains the information about a particular revision, such as parents, committer, author, date and the tree object which corresponds to the top directory of the stored revision". The index is "a stored version of your working tree"; the working tree is "the tree of actual checked out files", normally containing "the contents of the HEAD commit's tree, plus any local changes that you have made but not yet committed". Hash chaining is explained in Chapter 30 with its own evidence.

15.5 Pointers: HEAD and branches

A history of connected commits needs a way to say where you are and what the latest commit of a line of work is. Git uses two kinds of pointer.

New term: branch. A movable name that points to a commit, and, by extension, to that commit's whole chain of history. The default branch of a new repository is often called `main`.

New term: HEAD. A pointer that says which commit you are currently looking at and working on. Usually it points to a branch, which points to a commit.

Both are simple enough to display. The recording continues:

```
$ ls -a
.  ..  .git  README.md
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
c3c1c387782f6bea6232038b7469106e47b543e0
```

Recorded in Bash; *ch15-model/expected-model.bash.txt*.

Read those three commands:

1. `ls -a` shows the hidden `.git` folder beside `README.md` (Chapter 2).
2. `cat .git/HEAD` prints `ref: refs/heads/main`. **HEAD points to the branch `main`.**
3. `cat .git/refs/heads/main` prints a long code. That is the **hash of the latest commit**: the branch `main` is nothing more than a small file that holds one commit's hash.

That is the whole secret of branches: **a branch is a name for a commit**. When you make a new commit, Git creates it with the current commit as its parent and moves the branch name forward to it. Creating a branch (Chapter 20) costs almost nothing, because it is only a new name pointing at an existing commit. This is why Git's branches are cheap, the point made in Chapter 10.



Figure 12. 15.5 Pointers: HEAD and branches

Diagram description: HEAD points to the branch name `main`; `main` points to the newest commit, "Add opening line"; that commit points to its parent, "Add README".

15.5.1 Other pointers, met later

Pointer	What it is	Chapter
Tag	A name that is fixed to one commit, used to mark a version such as "v1.0"	Chapter 29
Remote	A name (often <code>origin</code>) for another repository you share history with	Chapter 23

Pointer	What it is	Chapter
Remote-tracking branch	Your local note of where a branch on a remote was the last time you checked, for example origin/main	Chapter 23

You need not remember them now. The point is that **every one of these is a name for a commit**, and the commits themselves are the history.

15.6 The index, recorded

One more recording shows the index directly:

```
$ git ls-files --stage
100644 cb7b0f73c5bafa598e86f818ac65939bbf51a2f8 0      README.md
```

Recorded in Bash; *ch15-model/expected-model.bash.txt*.

`git ls-files --stage` lists what is in the index: the file's **mode** (100644 means an ordinary file), the identifier of its stored **contents**, a stage number (0 is normal; it becomes non-zero only during a conflict, Chapter 22), and the file's path. After a commit, the index matches the last snapshot, which is why `git status` reports it as clean.

15.7 The whole picture

Put the pieces together, including the parts you will meet in Chapter 23:

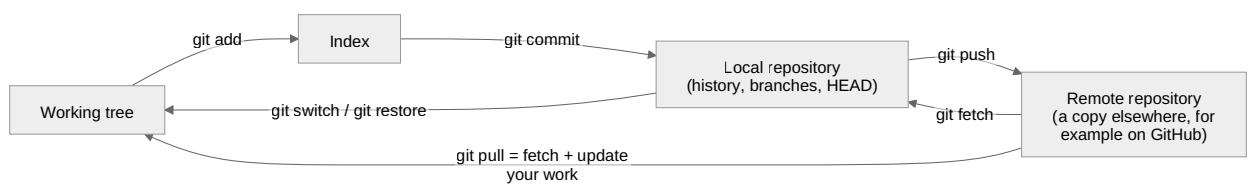


Figure 13. 15.7 The whole picture

Diagram description: the working tree feeds the index with `git add`; the index feeds the local repository with `git commit`; the local repository sends history to a remote with `git push`; the remote sends history back into the local repository with `git fetch`; `git switch` and `git restore` move files from the local repository into the working tree; `git pull` combines a fetch with updating your work.

If you can reproduce this diagram from memory and say what each arrow does, you understand Git's model. Chapters 16 to 23 teach one arrow at a time.

Checkpoint

What You Learned

- Git records **snapshots** of a project; a commit is a snapshot plus author, time, message and a link to its parent.
- Three places matter: the **working tree** (where you edit), the **index** (what will be committed) and the **repository** (the history in `.git`).
- A file is untracked, modified, staged or committed; `git status` reports which.
- Commits form a chain through their parents; a hash identifies each commit and depends on its content.
- A **branch** is a name for a commit, and **HEAD** says where you are; both are tiny pointers.
- Tags, remotes and remote-tracking branches are also names, met later.

New Vocabulary

- **Snapshot**: a complete picture of all the files of a project at one moment.
- **Working tree**: the folder of files you see and edit.
- **Index (staging area)**: the holding area that lists what the next commit will contain.
- **Stage**: to add a file's current contents to the index.
- **Local repository**: the complete history, kept in the hidden `.git` folder.
- **Tracked file**: a file that is in the last commit or in the index.
- **Commit**: a recorded snapshot plus author, time, message and parent; also the action of making one.
- **Hash (commit identifier)**: the long code that uniquely identifies a commit.
- **Branch**: a movable name that points to a commit.
- **HEAD**: the pointer saying which commit you are currently on.

Commands Learned

`git status`, `git add`, `git commit -m`, `git log --oneline`, `git ls-files --stage`, `git cat-file -t`, `git cat-file -p`. (You read the output of most of these; Chapter 16 teaches you to use them.)

Common Mistakes

1. **Believing that saving a file records it in Git.** Saving changes the working tree only.
2. **Forgetting to stage a change** and expecting it in the commit.
3. **Thinking the index is the same as the last commit.** It is what will go into the next one.
4. **Deleting `.git` "to clean up".** That deletes the history.
5. **Thinking a branch is a copy of the project.** It is a name for a commit.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name the three places and say what lives in each.
2. A file is edited and saved but not staged. Which state is it in, and what does `git status` say?
3. What are the five parts of a commit?
4. What is a branch, really? What does `.git/refs/heads/main` contain?
5. What does HEAD point to, usually?
6. Why does changing an old commit change all later hashes?

Before Moving On

You are ready for Chapter 16 if you can:

- [] draw the three places and the arrows between them from memory
- [] classify a file as untracked, modified, staged or committed from a `git status` report
- [] explain "a branch is a name for a commit"
- [] say where the history is stored and why not to edit it by hand

How this chapter was checked

Claim	Evidence class	Ledger
The status reports at each stage, the four file states, <code>git ls-files --stage</code>	Locally tested: Bash 5.2 and zsh 5.9 on Git 2.43.0; re-run in CI on the runner's Git 2.55.0 with identical output. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-status</code>); the ledger row says which.	R1 54
A commit has a tree, a parent, author, committer and message	Locally tested (<code>git cat-file -p HEAD</code>)	R1 52
HEAD contains ref: <code>refs/heads/main</code> ; the branch file holds a commit hash	Locally tested. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>gitrepository-layout</code>); the ledger row says which.	R1 53
The conceptual definitions (snapshot, working tree, index)	Compared with the Git 2.56.0 glossary (commit, commit object, index, working tree)	R1 52

Where this leads

Chapter 16 uses these ideas to create a repository and record your first versions. Chapter 17 reads history and compares versions. Chapter 20 creates and switches branches, and Chapter 30 looks inside `.git` to see how snapshots and hashes are really stored.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use a new practice repository.

Level 1 — Guided

1.1 Watch the states. In `sharif-git-lab`, run `mkdir model-lab`, `cd model-lab`, `git init`. Create `notes.md` with `echo "Hello" > notes.md`. After each of these steps run `git status` and write down the state of `notes.md`: (a) after creating it; (b) after `git add notes.md`; (c) after `git commit -m "Add notes"`; (d) after `echo "More" >> notes.md`. Expected result: untracked, staged, clean, modified.

1.2 Look inside. Run `cat .git/HEAD`, then `cat .git/refs/heads/main` (use the branch name that `HEAD` shows). Write down what each contains.

Level 2 — Partially guided

2.1 The three places. Draw the working tree, the index and the repository. Put `notes.md` at each stage of exercise 1.1 into the place(s) where its changes are.

2.2 Staging on purpose. Create two files, `a.txt` and `b.txt`. Stage only `a.txt` and run `git status`. Which state is each file in? Commit and check again.

Level 3 — Independent

3.1 Without the chapter open, explain to a friend the difference between saving a file, staging it and committing it, using an analogy of your own.

Level 4 — Professional scenario

4.1 A colleague says: "I saved my changes, so they are in Git." Write a four-sentence reply that uses the words working tree, index and repository, and explain what they need to do to record the change.

Level 5 — Troubleshooting

5.1 A learner edited `menu.html`, ran `git commit -m "Update menu"` and Git said no changes added to commit. Which state was the file in, why did the commit refuse, and what is the fix?

5.2 A learner deleted the `.git` folder of their project "to tidy up" and now `git status` says `fatal: not a git repository`. What did they delete, what still exists, and what could they do?

Chapter 16 — Your First Repository [Core]

In this chapter

- how to turn a folder into a Git repository (`git init`)
- how to read `git status`, the command you will run more than any other
- how to stage (`git add`) and record (`git commit`) your work
- **Project 1:** track a simple folder of study notes
- **Project 2 (start):** put the Sunrise Bakery site under version control
- the everyday mistakes at this stage, with their real error messages

Before you start. Chapter 15 (working tree, index, repository, the four file states), Chapter 14 (your name, email and default branch are set) and Chapter 7 (typing commands). Open your terminal and go to `~/sharif-git-lab`.

Every recording in this chapter was run for real in Bash and zsh, using the fictional user Ada Learner. Hashes and dates in your own terminal will differ from the recordings.

16.1 Turning a folder into a repository

You do not "install" version control on a project. You tell Git: start keeping the history of this folder. That is `git init`.

New term: `git init`. Creates a new, empty Git repository in the current folder, by making the hidden `.git` folder inside it.

Before you run it, look at where you are. **`git init` acts on the current folder**, so a wrong `pwd` gives you a repository in the wrong place.

```
$ mkdir -p sharif-git-lab
$ cd sharif-git-lab
$ mkdir study-notes
$ cd study-notes
$ git status
fatal: not a git repository (or any of the parent directories): .git
$ git init
Initialized empty Git repository in /home/learner/sharif-git-lab/study-notes/.git/
$ ls -a
.  ..  .git
```

Recorded in Bash; *ch16-firstrepo/expected-notes.bash.txt*.

Read the recording step by step:

1. `mkdir study-notes` and `cd study-notes` made a new practice folder and entered it.
2. `git status` said `fatal: not a git repository`. That is not a disaster; it is the correct answer. It means: this folder is not tracked by Git yet. It is also the quickest way to find out whether a folder is a repository.
3. `git init` created the repository, and reported where: `.../study-notes/.git/`.

4. `ls -a` shows the new hidden folder `.git` (Chapter 2).

The folder looks the same as before, but it now has a memory.

CAUTION. Run `git init` only in a folder that is meant to be one project. Never run it in your home folder, or in a folder that already sits inside another repository, because Git would then try to track everything below it. If you ever create one by mistake, delete only the hidden `.git` folder that you just made (and nothing else), after checking with `pwd` and `ls -a`. Deleting `.git` removes the history, not your files, so do it only when the repository is new and holds nothing you want.

16.2 `git status`: the command you will run constantly

New term: `git status`. Shows which files are untracked, modified or staged, and which branch you are on.

Run `git status` before and after nearly everything you do. It does not change your files or your history, so it is always safe to run. Chapter 15 introduced the four file states; here they are again in a real session:

```
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Recorded in Bash; *ch16-firstrepo/expected-notes.bash.txt*.

A new, empty repository. `On branch main` names the branch (Chapter 20). `No commits yet` means the history is empty. Now the first file:

```
$ printf '# Study notes\n\nToday I learned what version control is.\n' > notes.md
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    notes.md

nothing added to commit but untracked files present (use "git add" to track)
```

Recorded in Bash; *ch16-firstrepo/expected-notes.bash.txt*.

`notes.md` is **untracked**. Git lists it and suggests the next command: `git add`.

Git's suggestions in the parentheses, like (use "git add <file>..." to include in what will be committed), are worth reading: they change with the state and tell you the command that makes sense next.

16.3 Staging: git add

New term: git add. Puts the current contents of a file (or files) into the index, so that they will be part of the next commit.

```
$ git add notes.md
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   notes.md
```

Recorded in Bash; *ch16-firstrepo/expected-notes.bash.txt*.

After `git add notes.md`, the same file appears under Changes to be committed. Nothing has been recorded yet: the file is **staged**. Notice the hint (use "git rm --cached <file>..." to unstage); Chapter 25 teaches how to take a file back out of the staging area.

16.3.1 The forms of git add

Command	What it stages	When to use it
<code>git add notes.md</code>	That one file	The safest habit: you choose exactly what goes in
<code>git add notes.md menu.md</code>	Several named files	Same
<code>git add css/</code>	Everything inside a folder	A folder that belongs together
<code>git add .</code>	Every new and changed file in the current folder and below	Only after <code>git status</code> has shown you what is there

CAUTION. `git add .` stages everything under the current folder, including things you never meant to record: passwords in a settings file, large downloads, temporary files. Chapter 18 teaches how to keep those out. Until then, run `git status` first and check every file in the list.

16.4 Recording: `git commit`

New term: `git commit`. Records what is in the index as a new snapshot in the history, with a message.

```
$ git commit -m "Add first study notes"
[main (root-commit) dffcc1c] Add first study notes
1 file changed, 3 insertions(+)
create mode 100644 notes.md
$ git status
On branch main
nothing to commit, working tree clean
```

Recorded in Bash; *ch16-firstrepo/expected-notes.bash.txt*.

Decode the output of `git commit -m "Add first study notes"`:

```
[main (root-commit) dffcc1c] Add first study notes
1 file changed, 3 insertions(+)
create mode 100644 notes.md
```

- `main` is the branch that received the commit.
- `(root-commit)` appears only for the very first commit of a repository: it has no parent.
- `dffcc1c` is the short form of the new commit's hash.
- The rest of the first line is the message.
- The next line summarises the change: one file, three lines added.
- `create mode 100644 notes.md` says a new file was recorded (the number is its file mode, Chapter 15).

Then `git status` says nothing to commit, working tree clean: the working tree matches the last snapshot.

16.4.1 The message

The `-m` option supplies the message on the command line, in quotes. Write what the change does, in a short sentence: Add first study notes, not stuff or changes. Chapter 19 is about writing good messages, because you (and your team) will read them for years.

16.4.2 Reading the history

```
$ git log
commit dffcc1c3a28fc135c5cc88c0fb8149fefc625a87 (HEAD -> main)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:10:00 2026 +0000

    Add first study notes
```

Recorded in Bash; *ch16-firstrepo/expected-notes.bash.txt*.

`git log` shows the history, newest first: the full hash, the author, the date, and the message. Chapter 17 shows many more ways to read it.

16.5 Project 1: track a folder of study notes

Project 1. Goal: put a folder of your own notes under version control and record three versions of it. Time: about 20 minutes. You need: a terminal in `sharif-git-lab` and a text editor.

1. Create a folder `study-notes` and enter it. Run `git status`; expect fatal: not a git repository.
2. Run `git init`. Check with `ls -a` and `git status`.
3. Create `notes.md` with a title and one sentence (use your editor, or the `printf` command in the recording above). Run `git status`. Which state is the file in?
4. Stage it (`git add notes.md`) and check `git status` again.
5. Commit it: `git commit -m "Add first study notes"`. Check `git status` and `git log`.
6. Add a second sentence to `notes.md`, save, and run `git status`. Which state is it in now? Stage and commit it with a suitable message.
7. Add a new file, `questions.md`, with two questions you have about Git. Stage and commit it.
8. Run `git log`. You should see three commits, with your own messages.

Expected result: `git status` reports nothing to commit, working tree clean, and `git log` lists three commits, newest first.

Checkpoint questions: At step 3, where was the change (working tree, index or repository)? What would have happened to `questions.md` at step 7 if you had run `git commit` without `git add`?

16.6 Project 2 (start): the Sunrise Bakery site

Now a real project. You copied the Sunrise Bakery starter files into `~/sharif-git-lab/sunrise-bakery` in Chapter 3. Put them under version control:

```
$ cd sharif-git-lab/sunrise-bakery
$ ls
README.md  contact.html  css  images  index.html  menu.html
$ git init
Initialized empty Git repository in /home/learner/sharif-git-lab/sunrise-bakery/.git/
$ git status
On branch main

No commits yet

Untracked files:
```

Recorded in Bash; *ch16-firstrepo/expected-bakery.bash.txt*.

The listing shows the six items of the project, and `git status` shows them all as untracked (a folder such as `css/` is shown as one entry, and its files are staged individually later). Because this is the first commit and the folder holds only the files we mean to record, `git add .` is safe here.

```

$ git add .
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
    new file:   contact.html
    new file:   css/style.css
    new file:   images/logo.svg
    new file:   index.html
    new file:   menu.html

$ git commit -m "Add the Sunrise Bakery starter site"
[main (root-commit) 0ed5bc9] Add the Sunrise Bakery starter site
6 files changed, 127 insertions(+)
create mode 100644 README.md
create mode 100644 contact.html
create mode 100644 css/style.css
create mode 100644 images/logo.svg
create mode 100644 index.html
create mode 100644 menu.html

```

Recorded in Bash; *ch16-firstrepo/expected-bakery.bash.txt*.

The `git status` after staging lists every file as new file, including `css/style.css` and `images/logo.svg` inside the folders. The commit output summarises the whole change: six files, 127 lines added.

```

$ git log --stat
commit 0ed5bc9afc23af4d12117d41bb8a958c3cfccffd (HEAD -> main)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:08:00 2026 +0000

    Add the Sunrise Bakery starter site

 README.md      | 12 +++++
 contact.html   | 27 +++++
 css/style.css  | 24 +++++
 images/logo.svg | 5 +++++
 index.html     | 27 +++++
 menu.html      | 32 +++++
6 files changed, 127 insertions(+)
$ git ls-files
README.md
contact.html
css/style.css
images/logo.svg
index.html
menu.html

```

Recorded in Bash; *ch16-firstrepo/expected-bakery.bash.txt*.

`git log --stat` adds a summary of the files changed by each commit, and `git ls-files` lists all files Git tracks. Notice the paths, `css/style.css` and `images/logo.svg`: relative paths from the top of the project (Chapter 2), exactly the form that Git uses in every message.

Project 2 continues in later chapters: you will change the site, review the changes, branch, merge, and publish it.

16.7 The mistakes everyone makes first

Every message below is real, recorded on Git 2.43.0 and re-run on a newer Git. Read them now, so that they are familiar when they appear.

16.7.1 Committing before staging

```
$ git commit -m "Try to commit before staging"
On branch main

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    a.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Recorded in Bash; *ch16-firstrepo/expected-mistakes.bash.txt*.

Git refused. It did not say "error"; it repeated the state (Untracked files) and ended with nothing added to commit but untracked files present. **A commit records only what is staged**, so with nothing staged there is nothing to commit. The fix is `git add`.

16.7.2 Leaving the message empty

If you run `git commit` without `-m`, Git opens your editor so that you can write a message, and expects you to save and close the editor. If you save an **empty** message, Git aborts:

```
$ git add a.txt
$ git commit
hint: Waiting for your editor to close the file...
Aborting commit due to empty commit message.
$ git status --short
A  a.txt
```

Recorded in Bash; *ch16-firstrepo/expected-mistakes.bash.txt*.

Aborting commit due to empty commit message. The file stayed staged (A a.txt), so no work was lost. (The hint line about the editor depends on your editor; the recording used a stand-in that closes at once and writes nothing.)

Deep. If an unfamiliar editor opens and you cannot leave it, remember that it is a program like any other and that an editor's help or a search for its name plus "how to exit" will tell you. Better still, choose an editor you know for Git (Chapter 13), or always use `-m`. (The exit keys of particular terminal editors are not covered here.)

16.7.3 Nothing new to commit

```
$ git commit -m "Add a.txt"
[main (root-commit) 2cfec03] Add a.txt
1 file changed, 1 insertion(+)
create mode 100644 a.txt
$ git commit -m "Commit again with nothing new"
On branch main
nothing to commit, working tree clean
```

Recorded in Bash; *ch16-firstrepo/expected-mistakes.bash.txt*.

The second git commit said nothing to commit, working tree clean. That is not an error: it means there is no change to record.

16.7.4 A change that was never staged

```
$ printf 'second\n' >> a.txt
$ git commit -m "Commit without staging"
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   a.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Recorded in Bash; *ch16-firstrepo/expected-mistakes.bash.txt*.

The file was **modified** but not staged, so Git refused: no changes added to commit. Look at the last line: (use "git add" and/or "git commit -a"). The option `-a` means "automatically stage every change to files that Git already tracks":

```
$ git commit -am "Stage and commit tracked changes in one step"
[main 1a3f294] Stage and commit tracked changes in one step
1 file changed, 1 insertion(+)
$ git log --oneline
1a3f294 (HEAD -> main) Stage and commit tracked changes in one step
2cfec03 Add a.txt
```

Recorded in Bash; *ch16-firstrepo/expected-mistakes.bash.txt*.

`git commit -am "..."` staged the modification and committed it in one step. Use it only for files that Git already tracks; it never picks up new files, and it stages every tracked change, so check `git status` first. Chapter 19 returns to this.

16.7.5 "Please tell me who you are"

If you have not set `user.name` and `user.email` (Chapter 13), Git cannot say who made the commit, and stops with a message beginning `Author identity unknown`, followed by instructions to set them. The exact wording depends on your computer's name, so it is not recorded here; the fix is the two `git config --global` commands from Chapter 13.

16.7.6 A new repository's first branch

If `init.defaultBranch` is not set, older Git versions create a branch called `master` and print a long hint about it. Newer versions print the same kind of hint and announce a planned change of default:

```
$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/learner/demo/.git/
```

Recorded in Bash; *ch16-firstrepo/expected-nodefault.bash.txt*.

This is Git 2.43.0. On the newer Git that ran in the book's automated checks (2.55.0), the same command prints a hint that says the default branch name **will change to "main" in Git 3.0**.

```
$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
```

Recorded in Bash; *ch16-firstrepo/expected-nodefault.bash.alt.txt*.

The newer form, recorded from the automated check on Git 2.55.0.

That is why Chapter 13 asked you to set `init.defaultBranch` yourself: with it set, no hint appears and the branch name is the one you chose.

Checked against the documentation (R157). Git's `BreakingChanges` document (Git 2.56.0) lists, among the changes planned for Git 3.0: "In new repositories, the default branch name will be `main`. We have been warning that the default name will change since [the commit that added the advice about `init.defaultBranch`] (2020-12-11). The new name matches the default branch name used in new repositories by many of the big Git forges." `init.defaultBranch` "allows overriding the default branch name e.g. when initializing a new repository". The hint you saw on Git 2.55.0, will change to `"main"` in Git 3.0, states the same plan.

16.8 What you can do now

You can turn any folder into a repository, see what state its files are in, choose what goes into a snapshot and record it with a message. That is a complete, working version-control workflow. Everything else in this book adds to it: seeing differences (Chapter 17), keeping the wrong things out (Chapter 18), writing good messages (Chapter 19), working in parallel (Chapters 20 to 22), and sharing (Chapter 23).

Checkpoint

What You Learned

- `git init` makes the current folder a repository by creating `.git`; run it in the right folder only.
- `git status` reports the state of every file, is safe to run at any time (it changes neither your files nor your history), and suggests the next command.
- `git add` stages; `git commit -m` records what is staged, with a message.
- The commit output tells you the branch, (`root-commit`) for the first commit, the short hash, the message and a summary.
- A commit records only what is staged; `git commit -am` stages tracked, modified files for you.
- Common errors: nothing staged, empty message, nothing to commit, an unstaged modification, unknown identity.

New Vocabulary

- **git init**: creates a new repository in the current folder.
- **git status**: shows the state of files and the current branch.
- **git add**: stages files for the next commit.
- **git commit**: records the staged snapshot with a message.

Commands Learned

`git init`, `git status`, `git status --short`, `git add <file>`, `git add .`, `git commit -m`, `git commit -am`, `git log`, `git log --oneline`, `git log --stat`, `git ls-files`.

Common Mistakes

1. **Running `git init` in the wrong folder.**
2. **Running `git add .` without looking at `git status`.**
3. **Expecting a commit to include unstaged changes.**
4. **Saving an empty commit message.**
5. **Writing messages like "stuff".**
6. **Assuming "nothing to commit" is an error.**

Practice

Do the exercises in the Exercises section at the end of this chapter, including Project 1 (section 16.5).

Self-Test

1. What does `git init` create, and where?
2. How can you find out whether a folder is already a repository?
3. What is the difference between `git add` and `git commit`?
4. What does (root-commit) in the commit output mean?
5. Why did `git commit -m "Commit without staging"` fail in the recording, and what fixes it?
6. What is the danger of `git add .`?

Before Moving On

You are ready for Chapter 17 if you can:

- [] create a repository, stage a file and commit it from memory
- [] read a `git status` report and say which state each file is in
- [] explain "nothing to commit" and "no changes added to commit"
- [] show your project's history with `git log --oneline`

How this chapter was checked

Claim	Evidence class	Ledger
<code>git init</code> , <code>git status</code> , <code>git add</code> , <code>git commit</code> , <code>git log</code> , <code>git ls-files</code> outputs for the study-notes and bakery projects	Locally tested: Bash 5.2 and zsh 5.9 on Git 2.43.0; re-run in CI on Git 2.55.0 with identical output. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-add</code> , <code>git-commit</code> , <code>git-init</code>); the ledger row says which.	R155
The error and refusal messages of section 16.7	Locally tested; the initial-branch hint differs between 2.43.0 and 2.55.0 (both recorded)	R156, R157
The Author identity unknown message	Described, not recorded: depends on the computer's name	R156

Where this leads

Chapter 17 reads the history you have created and compares versions. Chapter 18 shows which files should never be staged and how to tell Git to ignore them. Chapter 19 turns the habit of committing into the craft of good commits.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Work in `sharif-git-lab` and run `pwd` before `git init`.

Level 1 — Guided

1.1 Project 1. Do all eight steps of section 16.5 (the study-notes repository). Expected result: `git status` is clean and `git log --oneline` lists three commits.

1.2 Read the output. Write down, in your own words, what each part of your first commit's output means: the branch, (root-commit), the short hash, the summary line.

Level 2 — Partially guided

2.1 Project 2 (start). Put `sunrise-bakery` under version control as in section 16.6. Before you run `git add .`, run `git status` and list every file it shows. Expected result: one commit called "Add the Sunrise Bakery starter site" containing six files.

2.2 Make the mistakes on purpose. In a scratch repository, reproduce three of the messages in section 16.7 (commit before staging; nothing to commit; modification not staged). For each, write down the message and the fix.

Level 3 — Independent

3.1 In `sunrise-bakery`, change the heading in `index.html` (for example to "Fresh bread and rolls every morning"), save it, and record the change as a second commit using only the commands you know. Then show the history with `git log --oneline`.

Level 4 — Professional scenario

4.1 A friend wants to start version-controlling a folder that contains source files, a folder of large photographs and a file called `passwords.txt`. Using what you know so far, say which files you would stage now and which you would leave out, and why. (Chapter 18 will give you the tool to keep them out permanently.)

Level 5 — Troubleshooting

5.1 A learner runs `git commit -m "Update"` and sees no changes added to commit (use "git add" and/or "git commit -a"). They insist that they saved the file. Explain what has happened and give two ways to fix it.

5.2 A learner ran `git init` in their home folder by mistake and now `git status` lists hundreds of untracked files. Explain the problem and how to undo it safely.

5.3 Two commits both have the message "stuff". Explain why this is a problem, and what you would do about it in future.

Chapter 17 — Seeing Changes and History [Core]

In this chapter

- how to read the history of a project (`git log`) in several formats
- how to name a commit: hashes, `HEAD`, `HEAD~1` and ranges
- how to read a diff, the report of what changed, line by line
- the four comparisons Git can make (`git diff`, `git diff --staged`, `git diff HEAD` and `commit-to-commit`), and when to use each
- `git show`, for looking at one commit

Before you start. Chapter 16 (you can create a repository, stage and commit) and Chapter 15 (working tree, index, repository). This chapter uses a small new example, the bakery menu, that stays with us through Part III.

The recordings were made in Bash and zsh on Git 2.43.0 and re-run in CI on Git 2.55.0; they were identical, including every hash.

17.1 The example: a menu with three versions

A history is only interesting if it has something in it. Here is the history that this chapter reads: a repository called `bakery-menu` that holds one file, `menu.md`, with three commits. (The same repository will be used in Chapters 20 to 24.) The commands are the ones you already know; the recording shows how it was made:

```
$ mkdir bakery-menu
$ cd bakery-menu
$ git init
Initialized empty Git repository in /home/learner/bakery-menu/.git/
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.50\n- Rolls (six): 3.00\n' > menu.md
$ git add menu.md
$ git commit -m "Add the menu"
[main (root-commit) 8a52ffe] Add the menu
1 file changed, 4 insertions(+)
create mode 100644 menu.md
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.80\n- Rolls (six): 3.00\n' > menu.md
$ git commit -am "Raise the price of the white loaf"
[main 9f10b43] Raise the price of the white loaf
1 file changed, 1 insertion(+), 1 deletion(-)
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.80\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n' > menu.md
$ git commit -am "Add coconut cake"
[main 81f772e] Add coconut cake
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch17-history/expected-build.bash.txt*.

Each commit changed the menu: the first created it, the second raised the price of the white loaf, the third added a coconut cake.

17.2 Reading the history: `git log`

New term: `git log`. Shows the history of the current branch: the commits, newest first.

```
$ git log
commit 81f772ee32b93d8fcf80dbb34ab74053b69d97a2 (HEAD -> main)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:08:00 2026 +0000

    Add coconut cake

commit 9f10b43614a19024637b4644c75e84a4aae6af83
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:07:00 2026 +0000

    Raise the price of the white loaf

commit 8a52ffecdf9a4a5f307aald31e3429b29ad0d6f9
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:06:00 2026 +0000

    Add the menu
```

Recorded in Bash; *ch17-history/expected-build.bash.txt*.

For each commit, `git log` shows:

- commit 81f772e...: the **full hash**, the commit's identifier. (HEAD -> main) marks that this is the commit HEAD and the branch main point to now (Chapter 15).
- Author: who made the change.
- Date: when. (The recordings use invented, evenly spaced dates so that they are reproducible. Yours will be the real time.)
- The **message**, indented.

Commits are listed **newest first**, and each is the parent of the one above it.

Checked against the documentation (R158). The git manual (Git 2.56.0) says that `-p/--paginate` pipes output "into less (or if set, `$PAGER`) if standard output is a terminal", and that `--no-pager` turns the pager off; `GIT_PAGER` overrides `$PAGER`. The `core.pager` documentation adds that when the `LESS` environment variable is unset, Git sets it to `FRX`. In a real terminal, Git shows long output through the pager (press `q` to quit and the space bar for the next page); the recordings switch it off, so it is not shown here.

17.2.1 Useful forms of `git log`

The full form is long. Git has options that show less, or more.

One line per commit: `--oneline`.

```
$ git log --oneline
81f772e (HEAD -> main) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch17-history/expected-build.bash.txt*.

Each line is the short hash, the branch decoration if any, and the message's first line. This is the form you will use most.

Only the last few: -2.

```
$ git log -2 --oneline
81f772e (HEAD -> main) Add coconut cake
9f10b43 Raise the price of the white loaf
```

Recorded in Bash; *ch17-history/expected-build.bash.txt*.

With a summary of files changed: --stat.

```
$ git log --stat
commit 81f772ee32b93d8fcf80dbb34ab74053b69d97a2 (HEAD -> main)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:08:00 2026 +0000

    Add coconut cake

 menu.md | 1 +
 1 file changed, 1 insertion(+)

commit 9f10b43614a19024637b4644c75e84a4aae6af83
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:07:00 2026 +0000

    Raise the price of the white loaf

 menu.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

commit 8a52ffecdf9a4a5f307aa1d31e3429b29ad0d6f9
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:06:00 2026 +0000

    Add the menu

 menu.md | 4 ++++
 1 file changed, 4 insertions(+)
```

Recorded in Bash; *ch17-history/expected-build.bash.txt*.

--stat adds a line per file changed and how many lines were added (+) and removed (-). It shows the size of each change at a glance.

With the changes themselves: -p.

```
$ git log -p -1
commit 81f772ee32b93d8fcf80dbb34ab74053b69d97a2 (HEAD -> main)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:08:00 2026 +0000

    Add coconut cake

diff --git a/menu.md b/menu.md
index 1f2a95a..97e3bea 100644
--- a/menu.md
+++ b/menu.md
@@ -2,3 +2,4 @@

- White loaf: 2.80
- Rolls (six): 3.00
+- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch17-history/expected-build.bash.txt*.

-p (for "patch") prints the actual changes after the message. Here -1 limited it to one commit. The block after diff --git is a diff, explained in section 17.4.

Your own format: --format.

```
$ git log --format='%h %an: %s'
81f772e Ada Learner: Add coconut cake
9f10b43 Ada Learner: Raise the price of the white loaf
8a52ffe Ada Learner: Add the menu
```

Recorded in Bash; *ch17-history/expected-build.bash.txt*.

%h is the short hash, %an the author's name and %s the subject (first line of the message). The Git manual lists many placeholders; you can build exactly the report you need.

Only the commits that touched a file: -- <path>.

```
$ git log --oneline -- menu.md
81f772e (HEAD -> main) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch17-history/expected-build.bash.txt*.

The -- separates options from file names. Here all three commits changed menu.md, so all appear; in a larger project this narrows the history to one file.

17.2.2 Summary

Command	Shows
git log	Full history, newest first
git log --oneline	One line per commit
git log -n or git log -3	Only the last n commits

Command	Shows
<code>git log --stat</code>	Files changed and sizes
<code>git log -p</code>	The changes themselves
<code>git log --format=...</code>	A format you design
<code>git log -- <path></code>	Only commits touching a path

17.3 Naming a commit

Most Git commands take a commit as an argument. There are several ways to name one.

Written	Means
81f772ee32b93d8fcf80dbb34ab74053b69d97a2	The full hash
81f772e	A short hash : enough of the beginning to be unambiguous (usually seven characters)
HEAD	The commit you are on now
main	The commit that the branch <code>main</code> points to
HEAD~1	The parent of HEAD (one step back)
HEAD~2	The parent's parent (two steps back)
HEAD^	The parent of HEAD (for a normal commit, the same as HEAD~1; merges, Chapter 21, have two parents and make the two forms differ)
HEAD~2..HEAD	A range : the commits reachable from HEAD but not from HEAD~2

New term: revision. Any way of naming a commit: a hash, a branch name, HEAD, HEAD~2. Git's manuals call these "revisions".

The command `git rev-parse` turns any revision into its full hash, which makes the relations visible:

```
$ git rev-parse HEAD~1
81f772ee32b93d8fcf80dbb34ab74053b69d97a2
$ git rev-parse --short HEAD^
81f772e
$ git log --oneline HEAD~2..HEAD
0dff14c (HEAD -> main) Raise the price of rolls
81f772e Add coconut cake
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

HEAD~1 was the third commit ("Add coconut cake"), and HEAD^ gave the same commit in short form. The range HEAD~2..HEAD showed the two newest commits: the new one ("Raise the price of rolls", made in the same recording) and the one before it.

Checked against the documentation (R160). The revision syntax is documented in `gitrevisions` (Git 2.56.0). Its worked example shows that `A^` is `A^1` and `A~1` (the parent), that `A^^` is `A~2`, and that `A^2` is the second parent of a merge; `<rev1>..rev2` means "commits that are reachable from `<rev2>` but exclude those that are reachable from `<rev1>`", and an omitted side "defaults to `HEAD`". The forms above are a small part of that page, which also lists forms such as `<rev>@{n}`, `<rev>^{/text}` and `:/text`.

Why a hash, and not a number? In a distributed system (Chapter 11) two people make commits at the same time on different computers. Sequence numbers would collide. A hash is calculated from a commit's content, so every commit has its own name without any central authority. (Chapter 30 explains how.)

17.4 Reading a diff

New term: diff. A report of the differences between two versions of a file or project, line by line.

Chapter 9 used the `diff` command to compare two files. Git produces a richer version. Read this real one, from the commit "Raise the price of the white loaf":

```
$ git show HEAD~1
commit 9f10b43614a19024637b4644c75e84a4aae6af83
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:07:00 2026 +0000

    Raise the price of the white loaf

diff --git a/menu.md b/menu.md
index 0b15fe9..1f2a95a 100644
--- a/menu.md
+++ b/menu.md
@@ -1,4 +1,4 @@
 # Sunrise Bakery menu

-- White loaf: 2.50
+- White loaf: 2.80
- Rolls (six): 3.00
```

Recorded in Bash; `ch17-history/expected-diff.bash.txt`.

The first block is `git show HEAD~1`; read the diff part line by line:

Part	Example	Meaning
The header	<code>diff --git a/menu.md b/menu.md</code>	Comparing the file <code>menu.md</code> in version a (before) and version b (after)

Part	Example	Meaning
Identifiers	index 0b15fe9..1f2a95a 100644	The stored contents before and after (Chapter 30), and the file mode
Old and new file	--- a/menu.md and +++ b/menu.md	The two versions being compared
The hunk header	@@ -1,4 +1,4 @@	This chunk covers lines 1 to 4 of the old file (-1,4) and lines 1 to 4 of the new one (+1,4)
Context lines	# Sunrise Bakery menu	Unchanged lines around the change, shown with a leading space
A removed line	-- White loaf: 2.50	Present before, absent after. It begins with -, and the rest is the old line
An added line	+- White loaf: 2.80	Absent before, present after

The line with the price appears twice: once with - (the old value) and once with + (the new one), so a changed line is shown as a removal and an addition next to each other. Everything else in the change is unchanged, and Git shows a few lines of it for context.

New term: hunk. One contiguous chunk of changes in a diff, introduced by a header such as @@ -1,4 +1,4 @@.

Because the file was small, one hunk covers everything. In a large file, a diff has several hunks with unchanged stretches between them.

17.5 The four comparisons

Chapter 15 gave you three places: working tree, index and repository. `git diff` can compare any two of them. Learn the commands as a set; each answers a different question.

Command	Compares	Answers the question
<code>git diff</code>	Working tree and index	What have I changed that I have not yet staged?
<code>git diff --staged</code>	Index and the last commit	What will my next commit contain?
<code>git diff HEAD</code>	Working tree and the last commit	Everything I have changed since the last commit
<code>git diff A B</code>	Commit A and commit B	What changed between two versions?

The recording below walks through them in order. First a change is made to the menu (rolls go up to 3.20) and left unstaged:

```
$ git status --short
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.80\n- Rolls (six): 3.20\n- Coconut cake (slice): 4.00\n' > menu.md
$ git status --short
M menu.md
$ git diff
diff --git a/menu.md b/menu.md
index 97e3bea..9674cda 100644
--- a/menu.md
+++ b/menu.md
@@ -1,5 +1,5 @@
# Sunrise Bakery menu

- White loaf: 2.80
-- Rolls (six): 3.00
+- Rolls (six): 3.20
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

`git status --short` shows `M menu.md`: a modification in the working tree (the `M` is in the second column: not staged). `git diff` shows the change, `-3.00` becoming `+3.20`.

Now what does the **staged** comparison say at this moment?

```
$ git diff --staged
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

Nothing: `git diff --staged` printed no output, because nothing is staged. Now stage the file, and ask again:

```
$ git add menu.md
$ git status --short
M menu.md
$ git diff
$ git diff --staged
diff --git a/menu.md b/menu.md
index 97e3bea..9674cda 100644
--- a/menu.md
+++ b/menu.md
@@ -1,5 +1,5 @@
# Sunrise Bakery menu

- White loaf: 2.80
-- Rolls (six): 3.00
+- Rolls (six): 3.20
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

After `git add`, `git status --short` shows `M menu.md` (the `M` is now in the first column: staged). Look at what changed:

- the second `git diff` now prints **nothing**: the working tree and the index are the same.

- `git diff --staged` now shows the change: the index differs from the last commit.

Whichever place holds the change, `git diff HEAD` shows it, since it compares the working tree with the last commit:

```
$ git diff HEAD
diff --git a/menu.md b/menu.md
index 97e3bea..9674cda 100644
--- a/menu.md
+++ b/menu.md
@@ -1,5 +1,5 @@
 # Sunrise Bakery menu

- White loaf: 2.80
-- Rolls (six): 3.00
+- Rolls (six): 3.20
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

Finally, record it, and compare two commits:

```
$ git commit -m "Raise the price of rolls"
[main 0dff14c] Raise the price of rolls
1 file changed, 1 insertion(+), 1 deletion(-)
$ git diff HEAD~2 HEAD
diff --git a/menu.md b/menu.md
index 1f2a95a..9674cda 100644
--- a/menu.md
+++ b/menu.md
@@ -1,4 +1,5 @@
 # Sunrise Bakery menu

- White loaf: 2.80
-- Rolls (six): 3.00
+- Rolls (six): 3.20
+- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

`git diff HEAD~2 HEAD` compares the commit two steps back with the newest: the net effect of two commits in one report (rolls' price and the coconut cake). And the size of a longer stretch:

```
$ git diff --stat HEAD~3 HEAD
menu.md | 5 +++--
1 file changed, 3 insertions(+), 2 deletions(-)
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

`--stat` gives the summary instead of the lines.

17.5.1 A habit worth forming

Before every commit, run `git status` and `git diff --staged`. `git status` says which files are staged; `git diff --staged` shows exactly what will be recorded. That is the last chance to notice a stray change, or a password, before it enters history. (You will use this habit for the rest of the book.)

17.6 One commit at a time: `git show`

New term: `git show`. Displays one commit: its message and the changes it made.

Chapters ago you saw the last commit's raw data. `git show` is the readable version:

```
$ git show
commit 81f772ee32b93d8fcf80dbb34ab74053b69d97a2 (HEAD -> main)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:08:00 2026 +0000

    Add coconut cake

diff --git a/menu.md b/menu.md
index 1f2a95a..97e3bea 100644
--- a/menu.md
+++ b/menu.md
@@ -2,3 +2,4 @@

- White loaf: 2.80
- Rolls (six): 3.00
+- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

With no argument, `git show` shows the commit HEAD. With an argument (`git show HEAD~1`, as in section 17.4) it shows that commit. With `--stat` it shows only the summary:

```
$ git show --stat HEAD~2
commit 8a52ffecdf9a4a5f307aa1d31e3429b29ad0d6f9
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:06:00 2026 +0000

    Add the menu

menu.md | 4 ++++
1 file changed, 4 insertions(+)
```

Recorded in Bash; *ch17-history/expected-diff.bash.txt*.

This is the quickest way to answer, "What did that commit actually do?"

17.7 Putting the tools together

You want to know...	Use
What is the history of the project?	<code>git log --oneline</code>
What did commit X change?	<code>git show X</code>
What have I changed since the last commit?	<code>git diff HEAD</code> (or <code>git status first</code>)
What is about to be committed?	<code>git diff --staged</code>
What changed between two versions?	<code>git diff A B</code>
Who touched this file, and when?	<code>git log -- <file></code>
How big was each change?	<code>git log --stat</code> or <code>git diff --stat</code>

Chapter 24 adds `git blame`, which shows for every line who last changed it, and searches through history.

Checkpoint

What You Learned

- `git log` shows history, newest first; `--oneline`, `-n`, `--stat`, `-p`, `--format` and `-- <path>` change what it shows.
- A commit can be named by hash, short hash, branch, HEAD, HEAD~n, HEAD^ and ranges.
- A diff shows removed lines with -, added lines with +, context lines with a space, in hunks with @@ headers.
- `git diff` (working tree vs index), `git diff --staged` (index vs last commit), `git diff HEAD` (working tree vs last commit) and `git diff A B` (two commits).
- `git show` displays one commit.
- Before every commit, check `git status` and `git diff --staged`.

New Vocabulary

- **git log**: shows the history of commits.
- **Revision**: any way of naming a commit (hash, branch, HEAD~2).
- **Diff**: a report of line-by-line differences between two versions.
- **Hunk**: one contiguous chunk of changes in a diff.
- **git show**: displays one commit's message and changes.

Commands Learned

`git log`, `git log --oneline`, `git log -n`, `git log --stat`, `git log -p`, `git log --format`, `git log -- <path>`, `git show`, `git show --stat`, `git diff`, `git diff --staged`, `git diff HEAD`, `git diff <a> <b>`, `git diff --stat`, `git rev-parse`.

Common Mistakes

1. **Running `git diff` and seeing nothing** after `git add`, and concluding that nothing changed. Use `git diff --staged`.
2. **Confusing - and + lines.** - is the old version, + is the new one.
3. **Reading the log oldest-first.** It is newest first.
4. **Assuming `HEAD~1` and `HEAD^` are always the same.** They differ for merge commits.
5. **Skipping `git diff --staged` before committing.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. In a diff, what do a line starting with -, a line starting with + and a line starting with a space mean?
2. You edited and staged a file. Which command shows the staged change, and why does `git diff` show nothing?
3. What does `HEAD~2` mean? What does `HEAD~2..HEAD` mean?
4. Which command shows exactly what your next commit will contain?
5. How would you list only the commits that changed `menu.html`?
6. Why does Git name commits with hashes rather than numbers?

Before Moving On

You are ready for Chapter 18 if you can:

- [] read a diff and say what changed
- [] use the four `git diff` forms and say which two places each compares
- [] show a project's history in three different formats
- [] name a commit three different ways

How this chapter was checked

Claim	Evidence class	Ledger
All <code>git log</code> forms and their output	Locally tested: Bash 5.2, zsh 5.9, Git 2.43.0; identical on the CI runner's Git 2.55.0 (including hashes)	R158
The four <code>git diff</code> comparisons and their output at each stage	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-diff</code>); the ledger row says which.	R159
<code>HEAD~n</code> , <code>HEAD^</code> , ranges, <code>git rev-parse</code>	Locally tested (as above); forms checked against <code>gitrevisions</code> (Git 2.56.0)	R160
The behaviour of the pager	Checked against the <code>git</code> and <code>core.pager</code> documentation (not shown in the recordings)	R158

Where this leads

Chapter 18 shows how to keep files out of the snapshots, and how to rename and delete files in a way Git understands. Chapter 19 shows how to write commits worth reading. Chapter 25 uses `git diff` and the revisions of this chapter to undo changes.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use the bakery-menu repository built in section 17.1 (type the commands yourself), and your sunrise-bakery project.

Level 1 — Guided

1.1 Rebuild the menu history. Type the commands of section 17.1 to create bakery-menu with three commits. Expected result: `git log --oneline` shows three commits.

1.2 Four log formats. Run `git log`, `git log --oneline`, `git log --stat` and `git log -p -1`. In one line each, say what each adds.

Level 2 — Partially guided

2.1 The four comparisons. Change the price of the white loaf in `menu.md`, save, and run `git diff`, `git diff --staged` and `git diff HEAD`. Then stage the file and run all three again. Fill in a table with what each printed, and explain it with the picture of Chapter 15.

2.2 Name it three ways. Show the commit before the newest using `HEAD~1`, using `HEAD^` and using its short hash from `git log --oneline`. Prove that all three are the same commit with `git rev-parse`.

Level 3 — Independent

3.1 In sunrise-bakery, make two commits that each change a different file. Then use `git log`, `git show` and `git diff` (with two revisions) to produce a report of the difference between the first commit and the last, and write down the command you used.

Level 4 — Professional scenario

4.1 Before a release, your manager asks: "What exactly changed on the website since Monday's version?" Describe the commands you would run to answer, in order, and how you would choose the two revisions. (Chapter 29 will give you a neater way to name "Monday's version".)

Level 5 — Troubleshooting

5.1 A learner edits a file, runs `git add`, then `git diff`, sees nothing and decides "Git lost my change". What is happening, and which command shows the change?

5.2 A colleague's `git log` output stops at the first screen and seems to hang. What is happening, and how do they get out?

5.3 A learner reads a diff with a line `-- White loaf: 2.50` and another `+ - White loaf: 2.80` and is confused by the two dashes. Explain each character.

Chapter 18 — Tracking, Ignoring, Renaming, Deleting [Core]

In this chapter

- which files belong in a repository, and which never should
- the `.gitignore` file: how to write it, test it and fix it
- personal ignore rules that apply to every repository (the global ignore file)
- why ignoring a file does not stop Git tracking it, and what to do about that
- renaming and deleting files in a way that Git understands
- why secrets never belong in a repository

Before you start. Chapter 16 (staging, committing), Chapter 17 (reading `git status` and diffs) and Chapter 4 (why files can differ in invisible ways). The Sunrise Bakery project is used again; recordings begin from its first commit.

Every recording was run in Bash and zsh on Git 2.43.0 and re-run in CI on Git 2.55.0, with identical output.

18.1 Not everything belongs in a repository

Chapter 16 warned that `git add .` stages everything. A repository is a project's history, and history should hold the project's **source**: the things people write and need. It should not hold things that are:

Kind of file	Why it should not be committed	Examples
Secrets	Anything committed stays in history, even after deletion (section 18.8).	Passwords, API keys, private keys, <code>.env</code> files
Generated files	They can be rebuilt from the source, and they change on every build, filling history with noise.	Compiled output, minified files, <code>build/</code> or <code>dist/</code> folders
Dependencies	They belong to other people, are large, and can be downloaded again from a list.	<code>node_modules/</code> , downloaded libraries
Logs and temporary files	They are private to one run or one computer.	<code>*.log</code> , <code>*.tmp</code> , editor swap files
Personal or machine-specific files	They mean nothing on another computer.	<code>.DS_Store</code> , <code>Thumbs.db</code> , editor settings
Very large binary files	They bloat every copy of the history forever.	Raw video, big archives (see Chapter 31)

The tool for keeping them out is a plain text file called `.gitignore`.

18.2 The four kinds of file, and a fifth

You met four states in Chapter 15: untracked, modified, staged, committed. Ignoring adds a fifth.

New term: ignored file. A file that Git has been told to leave alone: it is not shown as untracked and is not staged by `git add ..`

An ignored file still exists in your working tree. Git just pretends not to see it.

18.3 .gitignore

New term: .gitignore. A text file, kept in the repository, that lists patterns of files Git should ignore.

The recording starts with a clean bakery project. A learner adds three files that should not be committed: a build log, a temporary note and a settings file that holds a secret. (The "secret" is a made-up value. Nothing in this book ever asks you to use a real one.)

```
$ cd sharif-git-lab/sunrise-bakery
$ git status
On branch main
nothing to commit, working tree clean
$ printf 'error log\n' > build.log
$ printf 'draft notes\n' > notes.tmp
$ printf 'PAYMENT_API_KEY=FAKE_TOKEN_DO_NOT_USE_0000\n' > .env
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .env
    build.log
    notes.tmp

nothing added to commit but untracked files present (use "git add" to track)
$ git status --short
?? .env
?? build.log
?? notes.tmp
```

Recorded in Bash; *ch18-tracking/expected-tracking.bash.txt*.

Git lists all three as untracked. `git add .` would have staged every one of them. Now the ignore file:

```
$ printf '*.log\n*.tmp\n.env\n' > .gitignore
$ git status --short
?? .gitignore
```

Recorded in Bash; *ch18-tracking/expected-tracking.bash.txt*.

The command wrote a `.gitignore` containing three lines: `*.log`, `*.tmp` and `.env`. Afterwards `git status --short` shows only `.gitignore` itself as new: the three files have disappeared from Git's view. Git can even tell you why each was ignored:

```
$ git check-ignore -v build.log notes.tmp .env
.gitignore:1:*.log      build.log
.gitignore:2:*.tmp     notes.tmp
.gitignore:3:.env      .env
```

Recorded in Bash; *ch18-tracking/expected-tracking.bash.txt*.

`git check-ignore -v` prints, for each file, the rule that ignores it: the file `.gitignore`, the line number and the pattern. This command is your first tool when an ignore rule does not behave as you expect.

To see the ignored files themselves, add `--ignored`:

```
$ git status --short --ignored
?? .gitignore
!! .env
!! build.log
!! notes.tmp
```

Recorded in Bash; *ch18-tracking/expected-tracking.bash.txt*.

The `!!` marks mean ignored. Finally, commit the ignore file itself, so that everyone who clones the project gets the same rules:

```
$ git add .gitignore
$ git commit -m "Ignore logs, temporary files and the local environment file"
[main 8de9399] Ignore logs, temporary files and the local environment file
1 file changed, 3 insertions(+)
create mode 100644 .gitignore
$ git ls-files
.gitignore
README.md
contact.html
css/style.css
images/logo.svg
index.html
menu.html
```

Recorded in Bash; *ch18-tracking/expected-tracking.bash.txt*.

Notice that the listing of tracked files contains `.gitignore` but none of the three ignored files.

A rule worth remembering. `.gitignore` is a normal, tracked file. Commit it, so the whole team shares the rules.

Checked against the documentation (R161). The pattern rules recorded here agree with gitignore (Git 2.56.0): a blank line matches nothing; a line starting with # is a comment; ! at the start negates a pattern, but "it is not possible to re-include a file if a parent directory of that file is excluded"; a slash at the end matches only directories; * matches anything except a slash and ? any one character except a slash; a pattern with a slash at the beginning or middle is relative to the .gitignore file's directory, and otherwise may match at any level below. Also documented there: */ matches in all directories, /** matches everything inside, and a/**/b matches zero or more directories. The order of precedence (command line, then .gitignore files, then .git/info/exclude, then core.excludesFile) and the default for core.excludesFile, \$XDG_CONFIG_HOME/git/ignore (or \$HOME/.config/git/ignore), are in the same page.

18.4 Writing patterns

Each line of .gitignore is a **pattern**. Blank lines are skipped, and a line starting with # is a comment. The patterns used in this chapter are these:

Pattern	Matches	Notes
*.log	Any file whose name ends in .log, in any folder	* stands for any characters
build/	A folder called build, and everything in it	A trailing / means directory
/todo.txt	todo.txt in the top folder only	A leading / anchors the pattern to the top of the repository
docs/ *.tmp	.tmp files directly inside docs	A pattern with a / in the middle is relative to the top
! important .log	An exception : do not ignore this file, even though an earlier rule did	! negates; it must come after the rule it overrides

A recording tests all of them at once. Eight files are created and the five patterns are written:

```
$ mkdir patterns-demo
$ cd patterns-demo
$ git init
Initialized empty Git repository in /home/learner/patterns-demo/.git/
$ mkdir build docs logs
$ printf 'x\n' > app.log
$ printf 'x\n' > important.log
$ printf 'x\n' > todo.txt
$ printf 'x\n' > docs/todo.txt
$ printf 'x\n' > build/output.js
$ printf 'x\n' > docs/draft.tmp
$ printf 'x\n' > docs/notes.md
$ printf 'x\n' > logs/today.log
$ printf '*.log\n!important.log\nbuild/\n/todo.txt\ndocs/*.tmp\n' > .gitignore
```

Recorded in Bash; *ch18-tracking/expected-patterns.bash.txt*.

Which files does Git now treat as new, and which are ignored?

```
$ git status --short
?? .gitignore
?? docs/
?? important.log
$ git status --short --ignored
?? .gitignore
?? docs/
?? important.log
!! app.log
!! build/
!! docs/draft.tmp
!! logs/
!! todo.txt
```

Recorded in Bash; *ch18-tracking/expected-patterns.bash.txt*.

Read the two reports together.

- **New (not ignored):** .gitignore; docs/ (it still holds docs/todo.txt and docs/notes.md, which no pattern matches); and important.log, saved by the !important.log exception.
- **Ignored (!!):** app.log (rule *.log); build/ (the whole folder, rule build/); docs/draft.tmp (rule docs/*.tmp); logs/ (its only file, today.log, matches *.log); and todo.txt in the top folder (rule /todo.txt).
- **Not ignored although named todo.txt:** docs/todo.txt: the leading / anchored the rule to the top folder only.

Now the explanation by rule:

```
$ git check-ignore -v app.log important.log todo.txt docs/todo.txt build/output.js docs/
draft.tmp logs/today.log
.gitignore:1:*.log      app.log
.gitignore:2:!important.log    important.log
.gitignore:4:/todo.txt  todo.txt
.gitignore:3:build/     build/output.js
.gitignore:5:docs/*.tmp docs/draft.tmp
.gitignore:1:*.log      logs/today.log
```

Recorded in Bash; *ch18-tracking/expected-patterns.bash.txt*.

Read each line as file, line number and pattern, then the path it applied to. Notice that important.log was reported with the exception rule !important.log, that is, it matched an exception and is therefore **not** ignored; and docs/todo.txt does not appear because no rule matched it. (Git also stops looking inside an ignored folder: build/output.js was ignored because the folder build/ was.)

18.4.1 Comments and organization

A .gitignore can carry comments to say why, which is kind to the next person:

```
# Secrets and local settings: never commit these
.env
```

```
# Build output: rebuilt from the source
build/
dist/

# Editor and operating-system files
.DS_Store
*.swp
```

18.4.2 Starting points for real projects

Projects of the same kind ignore similar things. These are starting points, not complete lists; adapt them to your own project.

A static website like the bakery's

```
.env
*.log
*.tmp
.DS_Store
Thumbs.db
```

A project that downloads its libraries with a package tool (for example a JavaScript project)

```
node_modules/
dist/
build/
.env
*.log
```

A Python project

```
__pycache__/
*.pyc
.venv/
.env
```

Checked against GitHub's template collection (R162). These lists are short illustrations. Compared with the Node.gitignore and Python.gitignore templates in GitHub's github/gitignore repository (the collection used by GitHub's template chooser): node_modules/, dist, .env and *.log appear in the Node template (which has build/Release rather than a plain build/); __pycache__/, .venv, .env, build/ and dist/ appear in the Python template, and it ignores *.py[codz], a wider pattern than *.pyc. The templates are much longer than the lists here. Before relying on a list for a real project, start from a current, maintained template and adjust it to your own project.

18.4.3 Exceptions and ignored folders

An exception only works if Git still looks inside the folder. Suppose a project has a folder of large originals, assets/raw/, which should be ignored, except for one file, logo-final.psd, that must be kept. The obvious rules do not work; the recording shows both the failure and the fix:

```
$ mkdir -p assets/raw
$ printf 'x\n' > assets/raw/logo-final.psd
$ printf 'x\n' > assets/raw/big.psd
$ printf 'assets/raw/\n!assets/raw/logo-final.psd\n' > .gitignore
$ git status --short --ignored
?? .gitignore
!! assets/
$ printf 'assets/raw/*\n!assets/raw/logo-final.psd\n' > .gitignore
$ git status --short --ignored
?? .gitignore
?? assets/
!! assets/raw/big.psd
```

Recorded in Bash; *ch18-tracking/expected-exception.bash.txt*.

With the rule `assets/raw/` (the folder), Git ignored the **whole folder** (`!! assets/`) and never looked at the exception, so `logo-final.psd` was ignored too. After the rules were changed to ignore the folder's contents (`assets/raw/*`) and then to re-include the one file, the file appears as new (`?? assets/`) and only `big.psd` stays ignored (`!! assets/raw/big.psd`).

The rule: you cannot re-include a file if a parent folder is ignored. Ignore the folder's contents instead of the folder.

18.5 Personal rules: the global ignore file

Some files you want ignored in every repository on your computer, because they come from your tools, not from the project: an operating-system file, an editor's swap files. Putting those in each project's `.gitignore` would clutter it with other people's habits. Git lets you keep a personal list.

```
$ mkdir demo
$ cd demo
$ git init
Initialized empty Git repository in /home/learner/demo/.git/
$ git config --global core.excludesFile ~/.gitignore_global
$ printf '.DS_Store\n*.swp\n' > ~/.gitignore_global
$ git config --global --get core.excludesFile
/home/learner/.gitignore_global
$ printf 'x\n' > .DS_Store
$ printf 'y\n' > notes.txt
$ git status --short
?? notes.txt
$ git check-ignore -v .DS_Store
/home/learner/.gitignore_global:1:.DS_Store      .DS_Store
```

Recorded in Bash; *ch18-tracking/expected-global-ignore.bash.txt*.

The setting `core.excludesFile` (Chapter 14) points at a file of your choice; the recording writes two patterns into it, `.DS_Store` and `*.swp`, and then creates two files in a fresh repository. `git status --short` shows only `notes.txt`. `git check-ignore -v` shows the rule came from the **global** file: `/home/learner/.gitignore_global`, line 1.

Where each kind of rule belongs.

Rule	Put it in	Why
Something every contributor must ignore (node_modules/, .env)	The project's .gitignore	Shared with the team
Something only your tools produce	Your global ignore file	Private to you

18.6 Ignoring does not untrack

This is the most common surprise. .gitignore only affects files that Git is **not yet tracking**. If a file is already in the history, adding it to .gitignore changes nothing:

```
$ printf 'temporary\n' > scratch.txt
$ git add scratch.txt
$ git commit -m "Add scratch file"
[main 0c6c05c] Add scratch file
1 file changed, 1 insertion(+)
create mode 100644 scratch.txt
$ printf 'scratch.txt\n' > .gitignore
$ git status --short
?? .gitignore
```

Recorded in Bash; *ch18-tracking/expected-states.bash.txt*.

scratch.txt was committed. The learner then wrote scratch.txt into .gitignore. `git status --short` reports only the new .gitignore; it says **nothing** about scratch.txt, which Git is still tracking and will keep tracking. Ignoring alone did not help.

To stop tracking a file **without deleting it** from your working tree, use `git rm --cached`:

```
$ git rm --cached scratch.txt
rm 'scratch.txt'
$ git status --short
D scratch.txt
?? .gitignore
$ git commit -am "Stop tracking the scratch file"
[main 936eed8] Stop tracking the scratch file
1 file changed, 1 deletion(-)
delete mode 100644 scratch.txt
$ git status --short
?? .gitignore
```

Recorded in Bash; *ch18-tracking/expected-states.bash.txt*.

- `git rm --cached scratch.txt` removed the file from the index only. `git status --short` shows `D scratch.txt`: staged for deletion from the repository, while the file itself stays in the folder.
- The commit recorded that the repository no longer contains it. From now on the .gitignore rule applies, and the file is left alone.

CAUTION. Stopping tracking removes the file from future versions. It remains in the older commits, so anyone can still read it in history. If a file contains a secret, ignoring or untracking it is **not enough**; see section 18.8 and Chapter 33.

18.7 Renaming and deleting

18.7.1 Renaming: `git mv`

New term: `git mv`. Renames or moves a file and stages that change in one step.

```
$ git mv menu.html carte.html
$ git status --short
R  menu.html -> carte.html
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    menu.html -> carte.html
```

Recorded in Bash; *ch18-tracking/expected-states.bash.txt*.

`git mv menu.html carte.html` renamed the file and staged the rename. `git status` reports `renamed: menu.html -> carte.html`. Git does not actually store "renames": it stores snapshots. It works out that a rename happened because a file disappeared and another with (nearly) the same contents appeared. The commit output says so in words: `rename menu.html => carte.html (100%)`, where 100% means the contents are identical.

You can also rename the file in your file manager or with `mv`, and then tell Git with `git add` for both the old and the new name; Git will still recognise the rename. `git mv` is simply the tidier way.

Renaming has a side effect for a website. Other pages that link to `menu.html` now point at a file that no longer exists. Git tracked the rename but cannot fix your links: check them.

18.7.2 Deleting: `git rm`

New term: `git rm`. Deletes a file from the working tree and stages the deletion.

```
$ git rm contact.html
rm 'contact.html'
$ git status --short
R menu.html -> carte.html
D contact.html
```

Recorded in Bash; *ch18-tracking/expected-states.bash.txt*.

`git rm contact.html` deleted the file and staged the deletion (D in the first column). Now the commit:

```
$ git commit -m "Rename the menu page and remove the contact page"
[main a496e78] Rename the menu page and remove the contact page
2 files changed, 27 deletions(-)
 rename menu.html => carte.html (100%)
 delete mode 100644 contact.html
$ git ls-files
README.md
carte.html
css/style.css
images/logo.svg
index.html
$ git log --oneline --stat -1
a496e78 (HEAD -> main) Rename the menu page and remove the contact page
 menu.html => carte.html | 0
 contact.html          | 27 -----
2 files changed, 27 deletions(-)
```

Recorded in Bash; *ch18-tracking/expected-states.bash.txt*.

The commit recorded two things at once: a rename and a deletion. `git log --oneline --stat -1` shows them: the renamed file with 0 lines changed, and `contact.html` with 27 lines removed. Because Git keeps history, **the deleted file is not lost**: it still exists in the earlier commit, and Chapter 25 shows how to bring it back.

18.7.3 Deleting without `git rm`

If you delete a file in your file manager, or with `rm`, Git notices and tells you, but does not stage the deletion for you:

```
$ rm contact.html
$ git status --short
D contact.html
$ git status
On branch main
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    contact.html

no changes added to commit (use "git add" and/or "git commit -a")
```

Recorded in Bash; *ch18-tracking/expected-manual-delete.bash.txt*.

The state is deleted in the working tree, not staged (`D`, the letter in the second column). You can stage the deletion with `git add` (it works for deleted files too), or undo the deletion:

```
$ git add contact.html
$ git status --short
D  contact.html
$ git restore --staged contact.html
$ git restore contact.html
$ git status --short
$ ls
README.md  contact.html  css  images  index.html  menu.html
```

Recorded in Bash; *ch18-tracking/expected-manual-delete.bash.txt*.

- `git add contact.html` staged the deletion (`D contact.html`).
- `git restore --staged contact.html` took it out of the index again, and `git restore contact.html` brought the file back from the last commit. The final `ls` shows `contact.html` is back.

This is the first look at **restoring**; Chapter 25 teaches it properly. Note that it worked because the file had been committed: **Git can only restore what it has recorded.**

18.8 Secrets never belong in a repository

Section 18.1 listed secrets first for a reason. Suppose the settings file `.env` held a real password, and a learner ran `git add .` and committed. The password is now part of the history. Deleting the file later removes it from the newest version only. It remains in the commit where it was added, and in every copy of the repository that has that commit: every clone, and every backup. Anyone with the repository can read it.

Here is the proof, recorded with a made-up token. A settings file is committed, then deleted in the next commit:

```
$ mkdir secret-demo
$ cd secret-demo
$ git init
Initialized empty Git repository in /home/learner/secret-demo/.git/
$ printf 'PAYMENT_API_KEY=FAKE_TOKEN_DO_NOT_USE_0000\n' > .env
$ git add .env
$ git commit -m "Add settings"
[main (root-commit) c512a2f] Add settings
 1 file changed, 1 insertion(+)
 create mode 100644 .env
$ git rm .env
rm '.env'
$ git commit -m "Remove the settings file"
[main 9ee72a4] Remove the settings file
 1 file changed, 1 deletion(-)
 delete mode 100644 .env
$ git ls-files
$ git log --oneline
9ee72a4 (HEAD -> main) Remove the settings file
```

```
c512a2f Add settings
$ git show HEAD~1:.env
PAYMENT_API_KEY=FAKE_TOKEN_DO_NOT_USE_0000
```

Recorded in Bash; *ch18-tracking/expected-secret-history.bash.txt*.

The file is gone from the newest version (`git ls-files` printed nothing), yet `git show HEAD~1:.env` printed it exactly as it was in the earlier commit. The syntax `<commit>:<path>` means the file at that path, as of that commit. The "deleted" secret is one command away for anyone with the repository. (`git log -p -- .env` would show it too, in the diff of the commit that added the file.)

The habit that prevents the problem is the one shown in section 18.3:

1. **Create `.gitignore` before the first commit**, listing `.env` and similar files.
2. Run `git status` before you stage anything, and **read the whole list**.
3. Keep a template such as `.env.example`, with fake values, in the repository, so that others know what settings to create.
4. If a secret is committed, treat the secret as **stolen**: replace it at once (change the password, revoke the key), and only then think about cleaning history (Chapter 33 and Chapter 79).

Security note. In this book, every example secret contains the word `FAKE`, and the repository's automated checks refuse to accept anything that looks like a real credential and does not carry that marker.

18.9 When `.gitignore` seems not to work

Work through these questions, in order. The first tool is `git check-ignore -v <file>`.

Symptom	Likely cause	Fix
The file is still shown as modified or staged	The file is already tracked (section 18.6)	<code>git rm --cached <file></code> and commit
<code>git check-ignore -v</code> prints nothing for the file	No pattern matches	Check the pattern and where the rule lives (leading <code>/</code> , trailing <code>/</code>)
A file inside an ignored folder is not ignored as expected	A later <code>!</code> exception un-ignored it	Read the rules in order; the last matching rule wins
The rule is in the wrong folder	A <code>.gitignore</code> applies to its own folder and below	Move the rule to the right file
The rule is in a local-only place	You put it in your global file, so teammates do not have it	Put shared rules in the project's <code>.gitignore</code>

Checkpoint

What You Learned

- A repository should hold source, not secrets, generated files, dependencies, logs, machine-specific or very large files.
- `.gitignore` lists patterns of files to ignore; commit it so that everyone shares it.
- Patterns: `*`, a trailing `/` for directories, a leading `/` for the top folder, `!` for exceptions.
- `git check-ignore -v` explains why a file is ignored; `git status --ignored` lists ignored files.
- Personal rules go in a global ignore file (`core.excludesFile`).
- Ignoring does not untrack a file that is already tracked: use `git rm --cached`.
- `git mv` renames and `git rm` deletes, both staged; Git recognises renames by content.
- Files deleted outside Git show as deleted; `git restore` can bring back committed files.
- A secret in history is exposed even after the file is deleted.

New Vocabulary

- **Ignored file:** a file Git has been told to leave alone.
- **`.gitignore`:** a text file listing patterns of files Git should ignore.
- **`git mv`:** renames or moves a file and stages the change.
- **`git rm`:** deletes a file and stages the deletion.

Commands Learned

`git check-ignore -v`, `git status --ignored`, `git rm`, `git rm --cached`, `git mv`, `git restore`, `git restore --staged`, `git config --global core.excludesFile`.

Common Mistakes

1. **Adding a secret to the repository** "just to get it working".
2. **Writing a `.gitignore` rule after the file is already tracked**, and expecting it to untrack the file.
3. **Forgetting the trailing `/`** for a directory pattern, or the leading `/` for a top-level file.
4. **Putting personal rules in the shared `.gitignore`.**
5. **Using `rm/mv` outside Git and forgetting to stage the change.**
6. **Not committing the `.gitignore`.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name four kinds of file that should not be committed, and say why.
2. Write the `.gitignore` lines that ignore all `.log` files, a folder called `build`, and a file `secret.txt` only in the top folder.
3. What does the exception pattern `!important.log` do, and why must it come after the rule it overrides?

4. You added `notes.tmp` to `.gitignore`, but Git still shows it as modified. Why, and what do you do?
5. What is the difference between `git rm file` and `git rm --cached file`?
6. A real password was committed yesterday and deleted from the newest version today. Is it safe? What should you do first?

Before Moving On

You are ready for Chapter 19 if you can:

- [] write a `.gitignore` for the bakery project and test it with `git check-ignore -v`
- [] explain why `.gitignore` alone does not untrack a file
- [] rename and delete a tracked file with Git
- [] explain why a committed secret is exposed even after deletion

How this chapter was checked

Claim	Evidence class	Ledger
The behaviour and output of ignoring, <code>check-ignore</code> , <code>status --ignored</code> , the global ignore file, and of the five patterns	Locally tested: Bash 5.2, zsh 5.9, Git 2.43.0; identical on the CI runner's Git 2.55.0; pattern rules checked in <code>gitignore</code> (Git 2.56.0)	R161
The <code>.gitignore</code> starting points for a website, JavaScript and Python projects	Compared with GitHub's <code>Node.gitignore</code> and <code>Python.gitignore</code> templates; not run	R162
Ignoring does not untrack; <code>git rm --cached</code> ; <code>git mv</code> ; <code>git rm</code> ; <code>deleted-outside-Git</code> and <code>git restore</code>	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-mv</code> , <code>git-rm</code>); the ledger row says which.	R163
A secret stays in history after the file is deleted	Locally tested with a made-up token (<code>session-secret-history</code>); Chapter 33 shows the full incident response	R164

Where this leads

Chapter 19 turns the habit of committing into good commits. Chapter 25 returns to `git restore` and shows how to undo changes safely. Chapter 33 deals in depth with secrets that have already been committed, and Chapter 32 shows the related file `.gitattributes`.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use your sunrise-bakery project (or a copy). Use only made-up secrets.

Level 1 — Guided

1.1 Ignore three files. Create `build.log`, `notes.tmp` and `.env` (the last containing `PAYMENT_API_KEY=FAKE_TOKEN_DO_NOT_USE_0000`). Write a `.gitignore` for them, then run `git status --short`, `git check-ignore -v build.log notes.tmp .env` and `git status --short --ignored`. Commit the `.gitignore`. Expected result: the three files are `!!`; the commit contains only `.gitignore`.

1.2 Rename and delete. With `git mv`, rename `menu.html` to `carte.html`; with `git rm`, delete `contact.html`; commit both in one commit. Then use `git log --oneline --stat -1` to read the result.

Level 2 — Partially guided

2.1 Test the patterns. Reproduce the pattern test of section 18.4 in a scratch repository. Change `/todo.txt` to `todo.txt` and predict, then check, what changes for `docs/todo.txt`.

2.2 Untrack a file. Commit a file `scratch.txt`, add it to `.gitignore`, and show that Git still tracks it. Then untrack it with `git rm --cached` without deleting it from disk.

2.3 A global rule. Create your own global ignore file with `.DS_Store` and `*.swp`, and prove with `git check-ignore -v` that it applies in a new repository.

Level 3 — Independent

3.1 Write a `.gitignore` for a small web project that has an `assets/raw/` folder of large originals (ignore it), a `dist/` build folder, `*.log`, `.env`, and one file `assets/raw/logo-final.psd` that must be kept even though its folder is ignored. Test it with `git check-ignore -v` and explain any surprise. (Hint: read section 18.4 about exceptions and about ignored folders.)

Level 4 — Professional scenario

4.1 A new colleague's first commit contains a folder `node_modules/` with 30,000 files. Write a short plan: how to remove it from the repository going forward, what to put in `.gitignore`, how to tell the colleague, and what you would check before their next commit. (Chapter 31 will add tools for cleaning history; for now, plan only what this chapter allows.)

Level 5 — Troubleshooting

5.1 A learner added `secrets.txt` to `.gitignore` but `git status` still lists it as modified. Give the diagnosis, the command that proves it, and the fix.

5.2 A learner committed `.env` containing a real API key, then deleted the file and committed again. They say the problem is solved. Explain why it is not, and list the first two things they must do, in order.

5.3 `git check-ignore -v app.log` prints nothing, yet the learner is sure `*.log` is in `.gitignore`. Give three possible reasons.

Chapter 19 — Writing Good Commits [Core]

In this chapter

- why a commit is a message to a future reader, and what that reader needs
- the anatomy of a good commit message, and examples of bad ones
- how to fix the last commit (`git commit --amend`) and what that does to history
- atomic commits: one logical change per commit
- empty commits, authorship and timestamps
- habits that keep a history useful

Before you start. Chapter 16 (staging and committing), Chapter 17 (reading `git log`, `git show` and `git diff`) and Chapter 18 (keeping the wrong files out). The recordings use the bakery-menu repository of Chapter 17, with its three commits.

Every recording was run in Bash and zsh on Git 2.43.0 and re-run in CI on Git 2.55.0.

19.1 Who reads a commit?

You will read your own commits far more often than you write them. Six months from now, something on the bakery's website is wrong, and you need to know when it changed and why. You will run `git log`, and the messages are all you have. A history that says `stuff, fixes and update` answers nothing. A history that says `what changed, and why`, answers the question in seconds.

So a commit has two audiences:

1. **Tools**, which store the snapshot correctly whatever you write.
2. **People**, including you later, who need to understand the change.

Only people need good commits, and they are the reason to write them.

19.2 The anatomy of a message

A commit message has up to three parts.

Add the lemon tart to the menu	<- subject: one short line
The tart sells well at the farmers' market and is now part of the regular range. Its price includes tax.	<- body: why, in sentences
Refs: menu-42	<- footer: references (optional)

Part	What to write	Habit
Subject	What the commit does, in one short line	Start with a capital letter, use the imperative ("Add", "Fix", "Raise", as if giving an order to the codebase), no full stop, keep it short (about 50 characters is a common guide)

Part	What to write	Habit
Blank line	Separates subject from body	Needed: many tools treat the first line as the subject
Body	Why the change was made, and anything a reader could not guess from the diff	Write sentences; the diff already says what
Footer	References to related work items, and similar	Optional; team rules vary

Checked against the documentation (R165). Git itself asks for very little. `git commit` (Git 2.56.0, section DISCUSSION) says: "Though not required, it's a good idea to begin the commit message with a single short (no more than 50 characters) line summarizing the change, followed by a blank line and then a more thorough description. The text up to the first blank line in a commit message is treated as the commit title, and that title is used throughout Git." The Git project's own SubmittingPatches asks contributors to "describe your changes in imperative mood, e.g. 'make xyzzy do frotz'", calls 50 characters "the soft limit", to "skip the full stop", and to convey "the why behind your change". Those are that project's conventions for its own contributors. The capital letter in the table above is another common habit (the Git project itself does not capitalise after its area: prefix), so treat every row as a convention and follow your team's.

19.2.1 Good and bad subjects

Poor	Better	Why
stuff	Add the lemon tart to the menu	Says what changed
fixes	Fix the broken link to the menu page	Says which fix
update	Raise the price of rolls to 3.20	Says what and how much
Changed some things and also added the hours and fixed the price	Two commits: Raise the price of rolls and Add the opening hours	Two changes belong in two commits (section 19.4)
WIP	Add a draft of the weekend menu	"Work in progress" says nothing; describe the work

19.2.2 Writing a body: two -m options

The `-m` option can be repeated. The first is the subject; each further `-m` becomes a paragraph, separated by a blank line:

```
$ git commit --allow-empty -m "Summarise the change" -m "The body explains why, in complete sentences, after a blank line."
[main 573a346] Summarise the change
```

Recorded in Bash; *ch19-commits/expected-messages.bash.txt*.

Then `git log -1` shows the message as stored, subject and body separated by a blank line:

```
$ git log -1
commit 573a3460805e3e95ac6f40a56908bc88249022b3 (HEAD -> main)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:17:00 2026 +0000

    Summarise the change

    The body explains why, in complete sentences, after a blank line.
```

Recorded in Bash; *ch19-commits/expected-messages.bash.txt*.

Deep. If you run `git commit` with no `-m`, Git opens your editor with an empty message and some comment lines. Write the subject on the first line, a blank line, then the body; save and close. Lines beginning with `#` are comments and are not part of the message (this is Git's normal behaviour, stated in its documentation; the editor session itself is not recorded here).

19.3 Fixing the last commit: `git commit --amend`

Everybody writes a bad message, or forgets a file. Git lets you fix the **last** commit.

New term: `git commit --amend`. Replaces the last commit with a new one, so you can change its message or add forgotten changes to it.

19.3.1 A bad message

The recording adds an empty marker commit (section 19.5), makes a change and commits it with the message stuff, then fixes the message:

```
$ cd bakery-menu
$ git commit --allow-empty -m "Mark the start of the spring menu"
[main aa2ff61] Mark the start of the spring menu
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.80\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n- Lemon tart: 3.50\n' > menu.md
$ git commit -am "stuff"
[main 72a36ee] stuff
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch19-commits/expected-messages.bash.txt*.

```
$ git commit --amend -m "Add the lemon tart to the menu"
[main 5c151e3] Add the lemon tart to the menu
Date: Mon Jan 5 09:10:00 2026 +0000
1 file changed, 1 insertion(+)
$ git log --oneline -3
```

```
5c151e3 (HEAD -> main) Add the lemon tart to the menu
aa2ff61 Mark the start of the spring menu
81f772e Add coconut cake
```

Recorded in Bash; *ch19-commits/expected-messages.bash.txt*.

Read what happened:

- The first block shows the commit stuff was recorded as 72a36ee.
- After `git commit --amend -m "Add the lemon tart to the menu"`, the log shows 5c151e3 Add the lemon tart to the menu in its place. The hash **changed** (72a36ee to 5c151e3).
- The line `Date: Mon Jan 5 09:10:00 2026 +0000` in the commit output is the original author date, kept by the amend.

19.3.2 A forgotten file

```
$ printf 'Prices include tax.\n' > notes.txt
$ git add notes.txt
$ git commit --amend --no-edit
[main f71f66b] Add the lemon tart to the menu
Date: Mon Jan 5 09:10:00 2026 +0000
2 files changed, 2 insertions(+)
create mode 100644 notes.txt
$ git show --stat --oneline HEAD
f71f66b (HEAD -> main) Add the lemon tart to the menu
menu.md | 1 +
notes.txt | 1 +
2 files changed, 2 insertions(+)
```

Recorded in Bash; *ch19-commits/expected-messages.bash.txt*.

`git add notes.txt` staged a forgotten file, and `git commit --amend --no-edit` added it to the last commit **without** changing the message (`--no-edit` means "keep the message"). The commit output shows 2 files changed, and `git show --stat` confirms that the single commit now contains both files. The hash changed again (5c151e3 to f71f66b).

19.3.3 What amend really does

A commit's hash is calculated from its content (Chapter 15). Change the message or the snapshot, and you have a **different commit**. So `--amend` does not modify the old commit; it **creates a new commit and moves the branch to it**. The old commit is still in the repository, though nothing points to it. The reflog, Git's private record of where HEAD has been, proves it:

```
$ git reflog -6
573a346 (HEAD -> main) HEAD@{0}: commit: Summarise the change
f71f66b HEAD@{1}: commit (amend): Add the lemon tart to the menu
5c151e3 HEAD@{2}: commit (amend): Add the lemon tart to the menu
72a36ee HEAD@{3}: commit: stuff
aa2ff61 HEAD@{4}: commit: Mark the start of the spring menu
81f772e HEAD@{5}: commit: Add coconut cake
$ git show --stat --oneline 72a36ee
72a36ee stuff
menu.md | 1 +
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch19-commits/expected-messages.bash.txt*.

The reflog lists each move of HEAD, newest first: two commit (amend) entries and the original commit 72a36ee stuff further down. The last command shows that the old commit is still readable by its hash. (Chapter 26 uses this to recover work.)

CAUTION. Amend only commits that **you have not shared** with anyone. Once someone else has a copy of the old commit (Chapter 23), replacing it creates two different histories with the same beginning, and sorting that out is painful. The rule for the whole book: **do not rewrite history that others already have**. Chapter 27 covers the safe exceptions.

19.4 Atomic commits

New term: atomic commit. A commit that contains **one logical change**, complete in itself: not several unrelated changes together, and not half of a change.

The name comes from atom: indivisible. A useful test: **can you describe the commit in one sentence without using "and"**? If not, it is probably two commits.

Why it matters.

- **Review is easier.** A reader can check one idea at a time.
- **Undoing is precise.** If the price change was a mistake, you can undo it without also undoing the opening hours (Chapter 25).
- **Searching is easier.** `git log` becomes a readable list of changes.
- **Bugs are easier to find.** Chapter 28 shows a tool that searches history for the commit that broke something; it works best when commits are small.

How to make them. You already have the tools: edit, then stage only some of the changes with `git add <file>`, and commit; repeat. In the recording, two unrelated changes were made at once: the price of rolls (a change to `menu.md`) and a new file `hours.md`:

```
$ cd bakery-menu
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.80\n- Rolls (six): 3.20\n- Coconut cake\n(slice): 4.00\n' > menu.md
$ printf 'Open Monday to Saturday.\n' > hours.md
$ git status --short
M menu.md
?? hours.md
$ git add menu.md
$ git commit -m "Raise the price of rolls"
[main bf46067] Raise the price of rolls
1 file changed, 1 insertion(+), 1 deletion(-)
$ git add hours.md
$ git commit -m "Add the opening hours"
[main b27846f] Add the opening hours
1 file changed, 1 insertion(+)
create mode 100644 hours.md
```

Recorded in Bash; *ch19-commits/expected-atomic.bash.txt*.

Because the changes were staged separately, they became **two** commits, each with its own message:

```
$ git log --oneline --stat -2
b27846f (HEAD -> main) Add the opening hours
hours.md | 1 +
1 file changed, 1 insertion(+)
bf46067 Raise the price of rolls
menu.md | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

Recorded in Bash; *ch19-commits/expected-atomic.bash.txt*.

Read the `--stat` summary: the newest commit added only `hours.md`; the one before only changed `menu.md`. Each can be understood, reviewed and undone on its own.

Deep. What if two unrelated changes are inside one file? `git add -p` lets you choose, chunk by chunk (each chunk of a diff is a hunk, Chapter 17), which parts to stage. Git shows each hunk and asks whether to stage it; you answer `y` for yes, `n` for no, `q` to quit and `?` for help. Its prompt is interactive, so it is not shown in a recording here. Practise it in a scratch repository before you rely on it.

When not to be strict. The "one logical change" test is a guide, not a law. A tiny fix to a typo you noticed while working, or a change that spans several files but is one idea, is fine. What matters is that a reader can tell what the commit is for.

19.5 Empty commits

New term: empty commit. A commit with a message but no change to any file.

Normally, Git refuses to make a commit when there is nothing to record (Chapter 16). The option `--allow-empty` overrides that:

```
$ git commit --allow-empty -m "Mark the start of the spring menu"
[main aa2ff61] Mark the start of the spring menu
```

Recorded in Bash; *ch19-commits/expected-messages.bash.txt*.

Empty commits are unusual, but useful as a **marker** in the history ("start of the spring menu") or as a placeholder for a note. Use them sparingly; they add entries that carry no change.

19.6 Authorship and timestamps

Every commit records two identities and two times:

- **Author:** the person who wrote the change, and when.
- **Committer:** the person who recorded it in this repository, and when.

Usually they are the same person. They differ when someone applies another person's change (for example a maintainer accepting a contribution), or when a commit is rewritten later (Chapter 27).

`git commit --author` records a different author. Here Ada commits a change written by "Grace Baker":

```
$ cd bakery-menu
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.80\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n- Lemon tart: 3.50\n' > menu.md
$ git commit -am "Add the lemon tart to the menu" --author="Grace Baker
<grace@example.net>"
[main 85d89ce] Add the lemon tart to the menu
Author: Grace Baker <grace@example.net>
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch19-commits/expected-authorship.bash.txt*.

The commit output notes Author: Grace Baker <grace@example.net>, because it differs from the committer. The format fuller shows both:

```
$ git log -1 --format=fuller
commit 85d89ce09c6ec84fee0c1defc39afcc321167630 (HEAD -> main)
Author:      Grace Baker <grace@example.net>
AuthorDate:  Mon Jan 5 09:09:00 2026 +0000
Commit:      Ada Learner <ada@example.org>
CommitDate:  Mon Jan 5 09:09:00 2026 +0000

    Add the lemon tart to the menu
```

Recorded in Bash; *ch19-commits/expected-authorship.bash.txt*.

Notice Author: and Commit: are different people, each with a date (AuthorDate and CommitDate). A tailored format shows the same in one line per commit:

```
$ git log --format='%h author=%an committer=%cn %s' -2
85d89ce author=Grace Baker committer=Ada Learner Add the lemon tart to the menu
81f772e author=Ada Learner committer=Ada Learner Add coconut cake
```

Recorded in Bash; *ch19-commits/expected-authorship.bash.txt*.

`%an` is the author's name and `%cn` the committer's.

Honest authorship matters. Use `--author` only to credit a real person whose change you are recording, with their permission. Git does not verify identities: anyone can claim any name and email. To prove who made a commit, commits can be **signed** (Chapter 33).

19.6.1 Times

The recordings show invented, evenly spaced dates so that they can be reproduced. In real use, Git writes your computer's clock time and time zone offset (for example +0000). If your clock or time zone is wrong, your history records the wrong time, and Git will not warn you.

19.7 Habits that keep history useful

1. **Look before you commit:** `git status`, then `git diff --staged` (Chapter 17).
 2. **One logical change per commit** (section 19.4).
 3. **A subject that says what, a body that says why** (section 19.2).
 4. **Commit early and often on your own computer** (small steps are easy to undo), and **tidy before sharing**. A private history may be messy; a shared one should be readable. Chapter 27 shows how to tidy safely.
 5. **Do not commit secrets, large downloads or generated files** (Chapter 18).
 6. **Do not commit broken work to a shared branch** without a clear reason; an unfinished idea belongs on your own branch (Chapter 20).
 7. **Use `git commit -a` sparingly.** It stages every change to tracked files, so you may commit changes you did not mean to (Chapter 16).
 8. **Do not amend a commit that someone else already has** (section 19.3.3).
-

Checkpoint

What You Learned

- A commit message is for future readers; write what changed (subject) and why (body).
- Conventions: a short imperative subject, a blank line, a body of sentences. These are conventions, not Git rules.
- `git commit --amend` replaces the last commit with a new one (new hash); the old one is still recoverable. Amend only unshared commits.
- Atomic commits contain one logical change; stage files selectively to make them.
- `--allow-empty` creates a marker commit with no change.
- A commit has an author and a committer, each with a time; `--author` credits another person; Git does not verify identities.

New Vocabulary

- **git commit --amend:** replaces the last commit with a new one.
- **Atomic commit:** a commit containing one logical change.
- **Empty commit:** a commit with a message but no file change.

Commands Learned

`git commit -m <subject> -m <body>`, `git commit --amend`, `git commit --amend --no-edit`, `git commit --allow-empty`, `git commit --author`, `git log -1 --format=fuller`, `git reflog`, `git show <hash>`.

Common Mistakes

1. **Vague messages** such as `stuff` and `update`.
2. **Bundling unrelated changes** in one commit.
3. **Amending a commit that has already been shared.**
4. **Forgetting to look at `git diff --staged`** before committing.
5. **Using `git commit -a` without checking** what it will stage.
6. **Assuming the author field proves who wrote the change.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What are the parts of a good commit message, and what belongs in each?
2. What does `git commit --amend --no-edit` do, and why does the hash change?
3. Give the test for an atomic commit.
4. Where does the commit you replaced with `--amend` go?
5. What is the difference between the author and the committer?
6. Why should you not amend a commit that a colleague already has?

Before Moving On

You are ready for Chapter [20](#) if you can:

- [] write a subject and body that a colleague would thank you for
- [] fix a bad last message and add a forgotten file with `--amend`
- [] split two unrelated changes into two commits
- [] explain why amending shared history is dangerous

How this chapter was checked

Claim	Evidence class	Ledger
Message conventions (imperative subject, length, body, footer)	Blank line and 50-character title checked in <code>git commit</code> ; imperative mood in the project's <code>SubmittingPatches</code> ; the rest is convention	R1 65
Amend behaviour: new hash, retained author date, forgotten file, old commit in the reflog and readable by hash	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; identical on the CI runner's Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-commit</code> , <code>git-reflog</code>); the ledger row says which.	R1 66
Author versus committer, <code>--author</code> , fuller format	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-commit</code>); the ledger row says which.	R1 67

Claim	Evidence class	Ledger
Atomic commits by staging files separately; empty commits	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-commit); the ledger row says which.	R1 68
git add -p prompt keys; editor comment lines	Described from general knowledge, not recorded	R1 65

Where this leads

Chapter 20 moves from a single line of commits to parallel lines of work. Chapter 25 uses precise commits to undo changes safely, and Chapter 26 shows how to recover a commit that seems lost, such as the one replaced by `--amend`.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use your bakery-menu repository from Chapter 17 (or rebuild it), and never amend a commit you have shared.

Level 1 — Guided

1.1 Improve a message. Change `menu.md` (add a lemon tart at 3.50), commit it with the message stuff, then fix the message with `git commit --amend -m "..."`. Show the log before and after, and note that the hash changed.

1.2 A forgotten file. Create `notes.txt`, stage it, and add it to the last commit with `git commit --amend --no-edit`. Use `git show --stat --oneline HEAD` to check that one commit now holds both files.

Level 2 — Partially guided

2.1 Two commits from one afternoon. Change the price of rolls in `menu.md` and create a new file `hours.md`. Make two commits, one for each. Show them with `git log --oneline --stat -2`.

2.2 Body text. Make a commit with a subject and a body using two `-m` options. Show it with `git log -1`. What separates subject from body?

2.3 Whose commit? Make a commit that credits another (real, consenting, or fictional) person as author. Show both identities with `git log -1 --format=fuller`.

Level 3 — Independent

3.1 Rewrite these five messages so a colleague could tell what each commit did: `fix`, `wip`, `changes`, `update page`, `more stuff` and also the `css`. Invent plausible details about the Sunrise Bakery site, and say which of them should really be split into two commits.

Level 4 — Professional scenario

4.1 Your team's history contains Merge fixes, stuff, asdf and a commit that changes 40 files across four unrelated features. You are asked to propose commit guidelines for the team. Write a one-page proposal covering the subject, body, size of commits, when to amend, and what to review before committing. Say which parts are Git rules and which are conventions.

Level 5 — Troubleshooting

5.1 A learner amended a commit that they had already given to a colleague (Chapter 23 will show how). The colleague now has a commit that does not exist in the learner's history. Explain why, and what advice you would give.

5.2 A learner ran `git commit --amend` and thinks they destroyed the original commit. Show them how to prove it still exists, and name the chapter that shows how to get it back.

5.3 A learner used `git commit -a -m "Update prices"` and later discovered it also committed an unrelated experiment in another tracked file. What happened, and how would they avoid it?

Chapter 20 — Branching [Core]

In this chapter

- why branches exist and what problem they solve
- listing, creating, switching, renaming and deleting branches
- what happens to your files when you switch, including the two ways switching can go wrong
- the older `git checkout` and how it relates to `git switch`
- detached HEAD: the state you will meet, and how to leave it
- branch names and how branches are used in teams

Before you start. Chapter 15 (a branch is a name for a commit, HEAD says where you are), Chapter 17 (`git log`) and Chapter 19 (good commits). The recordings use the bakery-menu repository with its three commits.

Every recording was run in Bash and zsh on Git 2.43.0 and re-run in CI on Git 2.55.0 (the two outputs were identical, except for one hint, recorded twice).

20.1 The problem branches solve

The bakery wants to try a new idea: adding tea to the menu. Meanwhile the current menu, the one customers see, must stay correct: someone might still need to fix a price. You want **two lines of work at once**: the stable menu, and the experiment.

Without branches you would copy the whole folder (Chapter 9) and struggle to combine the two later. Git's answer is the **branch**.

Reminder. A branch is a movable name that points to a commit (Chapter 15). Because it is only a name, creating one costs almost nothing, and you can create as many as you like.

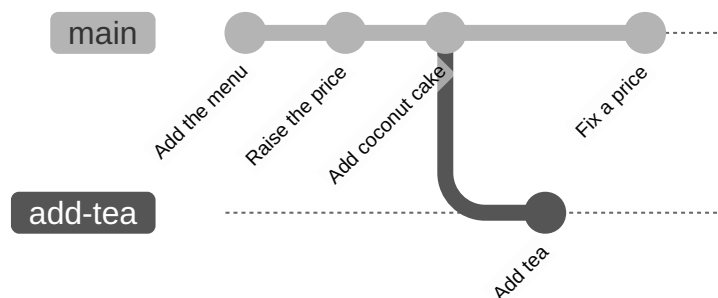


Figure 14. 20.1 The problem branches solve

Diagram description: three commits in a line on the branch `main`. From the third commit a second line, called `add-tea`, splits off and gets one commit, "Add tea". Meanwhile `main` also gets a new commit, "Fix a price". The two lines now differ, and each can go on growing independently.

Branches let people (and one person at different times) work in parallel without disturbing each other, and they make **experiments safe**: if the tea idea is bad, delete the branch and `main` never noticed.

20.2 Seeing the branches

New term: git branch. Lists, creates, renames and deletes branches.

Start in the repository:

```
$ cd bakery-menu
$ git branch
* main
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

`git branch` with no argument **lists** the branches. The asterisk `*` marks the **current** branch, the one `HEAD` points to. There is only `main` so far. To print only the current branch's name, use `git branch --show-current`.

20.3 Creating a branch

```
$ git branch add-tea
$ git branch
  add-tea
* main
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

`git branch add-tea` **created** the branch but did **not switch to it**: the asterisk is still on `main`. The new branch points to the same commit as `main`: that is the whole cost of a new branch, one new name.

20.4 Switching branches

New term: git switch. Changes the current branch: it moves `HEAD` to another branch and updates the working tree to match.

```
$ git switch add-tea
Switched to branch 'add-tea'
$ git branch
* add-tea
  main
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

`git switch add-tea` moved HEAD to the branch `add-tea`. The asterisk moved with it. No files changed yet, since the two branches point to the same commit.

Checked against the documentation (R169). In Git's own documentation, `git switch` and `git restore` were marked "THIS COMMAND IS EXPERIMENTAL. THE BEHAVIOR MAY CHANGE." up to and including Git 2.50.0. The release notes of **Git 2.51.0** say that both commands "are declared to be no longer experimental", and the manual pages of 2.51.0 and later no longer carry the warning. Older Git versions you may meet still have it, and in every recording in this book the commands behave identically on Git 2.43.0 and 2.55.0.

20.4.1 Committing on a branch

Now make a change on `add-tea`, and commit it:

```
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.80\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n- Tea: 1.50\n' > menu.md
$ git commit -am "Add tea to the menu"
[add-tea 5a16424] Add tea to the menu
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

The commit was recorded on `add-tea`: notice `[add-tea 5a16424]` in the output. The graph view shows what happened to the history:

```
$ git log --oneline --all --graph
* 5a16424 (HEAD -> add-tea) Add tea to the menu
* 81f772e (main) Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

Read the graph, newest at the top:

- `*` marks each commit; the vertical line joins each to its parent.
- `(HEAD -> add-tea)` says that HEAD is on `add-tea`, which points to the newest commit, "Add tea to the menu".
- `(main)` marks the commit that `main` still points to: "Add coconut cake". The branch `main` did **not** move when you committed on `add-tea`.

The options: `--oneline` (one line per commit), `--all` (show all branches, not only the current one) and `--graph` (draw the lines). You will use this combination throughout the rest of Part III.

20.4.2 What switching does to your files

A branch is a different **version of the whole project**, so switching changes the files in your working tree. Watch the file `menu.md` as the branch changes:

```
$ git switch main
Switched to branch 'main'
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
$ git switch add-tea
Switched to branch 'add-tea'
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
- Tea: 1.50
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

On main, menu.md has no tea. On add-tea, it has the tea line. Git rewrote the file when you switched. Nothing was lost; each version is safe in its branch. This is the moment that Chapter 15 promised: **the working tree is whatever the current branch says it should be.**

20.5 Creating and switching in one step

You nearly always want to create a branch and start working on it. `git switch -c` does both (`-c` stands for create):

```
$ git switch main
Switched to branch 'main'
$ git switch -c fix-typo
Switched to a new branch 'fix-typo'
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

`git branch -vv` shows more detail per branch: the branch name, the commit it points to and that commit's message:

```
$ git branch -vv
add-tea 5a16424 Add tea to the menu
* fix-typo 81f772e Add coconut cake
main 81f772e Add coconut cake
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

The new branch `fix-typo` points to the same commit as `main` (81f772e). The other, `add-tea`, points to its own newer commit.

20.6 Renaming and deleting

Rename a branch with `git branch -m old new`:

```
$ git branch -m fix-typo tidy-menu
$ git branch
  add-tea
  main
* tidy-menu
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

Delete one with `git branch -d`. Git protects you: it refuses to delete the branch you are standing on, and it refuses to delete a branch whose work has **not been merged**:

```
$ git branch -d tidy-menu
error: cannot delete branch 'tidy-menu' used by worktree at '/home/learner/bakery-menu'
$ git switch main
Switched to branch 'main'
$ git branch -d tidy-menu
Deleted branch tidy-menu (was 81f772e).
$ git branch -d add-tea
error: the branch 'add-tea' is not fully merged.
If you are sure you want to delete it, run 'git branch -D add-tea'
$ git branch -D add-tea
Deleted branch add-tea (was 5a16424).
```

Recorded in Bash; *ch20-branching/expected-branches.bash.txt*.

Read each step:

1. `git branch -d tidy-menu` failed: you cannot delete the branch you are on. Switch away first.
2. After `git switch main`, the same command succeeded: `tidy-menu` had no commits of its own (it pointed to the same commit as `main`), so nothing would be lost.
3. `git branch -d add-tea` **refused**: `add-tea` has a commit that is not in any other branch. Deleting it would leave that commit unreachable (a common way to lose work). The message tells you the forceful alternative.
4. `git branch -D add-tea` (capital D) **forced** the deletion. Git printed the commit it had pointed to: `5a16424`. The commit was not destroyed at once, but nothing points to it any more, so it is hard to find. (Chapter 26 shows how to get it back.)

CAUTION. `git branch -D` deletes a branch **even if it has unmerged work**. Use it only when you are certain that you no longer need that work, and note the hash it prints so that you can recover it (Chapter 26).

On the newer Git (2.55.0) the refusal in step 3 is worded slightly differently:

```
$ git branch -d add-tea
error: the branch 'add-tea' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D add-tea'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
```

Recorded in Bash; *ch20-branching/expected-branches.bash.alt.txt*.

The newer message adds a *hint:* line, and tells you how to switch the hint off. The meaning is the same.

20.7 Switching when you have uncommitted changes

Changes that you have not committed live in the working tree (and, if staged, in the index). They are not part of any branch. So what happens to them when you switch? Two cases, both recorded.

20.7.1 The changes do not clash: Git carries them along

The recording stages a new file, `hours.md`, on `main`, and switches to `add-tea`:

```
$ printf 'Open Monday to Saturday.\n' > hours.md
$ git add hours.md
$ git switch add-tea
A      hours.md
Switched to branch 'add-tea'
$ git status --short
A  hours.md
```

Recorded in Bash; *ch20-branching/expected-dirty.bash.txt*.

Git switched, and **took the staged change with it**: A `hours.md` (staged as a new file) is now on `add-tea`. The line `A hours.md` printed by `git switch` is Git telling you that the change came with you. This is convenient, and a surprise if you did not expect it: the uncommitted change now belongs to whichever branch you are on when you eventually commit.

20.7.2 The changes clash: Git refuses

Now `main` and `add-tea` have different contents of `menu.md`, and the learner changes `menu.md` (uncommitted) on `main`, then tries to switch:

```
$ git switch main
A      hours.md
Switched to branch 'main'
$ printf '# Sunrise Bakery menu\n\n- White loaf: 3.00\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n' > menu.md
$ git switch add-tea
error: Your local changes to the following files would be overwritten by checkout:
      menu.md
Please commit your changes or stash them before you switch branches.
Aborting
```



```
$ git status --short
A  hours.md
M  menu.md
```

Recorded in Bash; *ch20-branching/expected-dirty.bash.txt*.

Git **refused**, with a clear message: Your local changes to the following files would be overwritten by checkout: menu.md. Switching would have had to overwrite the uncommitted edit. It offers two ways out: **commit** the changes, or **stash** them.

20.7.3 Stashing

```
$ git stash
Saved working directory and index state WIP on main: 81f772e Add coconut cake
$ git switch add-tea
Switched to branch 'add-tea'
$ git stash pop
Auto-merging menu.md
On branch add-tea
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   hours.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   menu.md

Dropped refs/stash@{0} (7ad1cd045fd28554d6d12eecf6fb3dae681bfff32)
```

Recorded in Bash; *ch20-branching/expected-dirty.bash.txt*.

git stash set the uncommitted changes aside (Chapter 24 covers it fully), which made the working tree clean, so git switch add-tea succeeded. git stash pop then re-applied the saved changes on the new branch (Auto-merging menu.md), and reported the state of the working tree.

A safe rule: before you switch branches, run git status. If it is clean, switching is safe. If not, commit or stash first, unless you mean to carry the change.

20.8 The older command: git checkout

Before git switch existed, one command did nearly everything: git checkout. It still works, and you will see it in tutorials and scripts.

```
$ git checkout -b legacy-style
Switched to a new branch 'legacy-style'
$ git branch
* legacy-style
  main
$ git checkout main
Switched to branch 'main'
$ git branch -d legacy-style
Deleted branch legacy-style (was 81f772e).
```

Recorded in Bash; *ch20-branching/expected-checkout.bash.txt*.

`git checkout -b legacy-style` created and switched (like `git switch -c`); `git checkout main` switched back (like `git switch main`); and `git branch -d` deleted the merged branch.

Why two commands? `git checkout` grew to do **two unrelated jobs**: switching branches, and restoring files from a commit. That made it confusing. Newer Git separates them: `git switch` for branches (this chapter) and `git restore` for files (Chapter 25). This book teaches the new commands and points out the old forms where you will meet them.

Checked against the documentation (R170). The release notes of Git 2.23.0 introduce the two commands "to split 'checking out a branch to work on advancing its history' and 'checking out paths out of the index and/or a tree-ish to work on advancing the current history' out of the single `git checkout` command". That is the reason given above, in the project's own words.

If you give `git switch` a name that does not exist, it says so:

```
$ git switch nowhere
fatal: invalid reference: nowhere
```

Recorded in Bash; *ch20-branching/expected-checkout.bash.txt*.

20.9 Detached HEAD

Normally HEAD points to a **branch**, and the branch points to a commit. It is also possible to point HEAD **directly at a commit**, with no branch. That state has a name.

New term: detached HEAD. The state in which HEAD points directly to a commit instead of to a branch.

You get there when you look at an old commit:

```
$ git switch --detach HEAD~1
HEAD is now at 9f10b43 Raise the price of the white loaf
$ git status
HEAD detached at 9f10b43
nothing to commit, working tree clean
$ git switch main
Previous HEAD position was 9f10b43 Raise the price of the white loaf
Switched to branch 'main'
```

Recorded in Bash; *ch20-branching/expected-checkout.bash.txt*.

`git switch --detach HEAD~1` moved HEAD to the commit before the newest one. `git status` says HEAD detached at 9f10b43. You can look around, and even make commits, but **new commits belong to no branch**, so they are easy to lose once you switch away. `git switch main` brought us back; Git also noted Previous HEAD position was 9f10b43 ... in case you want to return.

Detached HEAD is not an error, and you do not need to fear it, but you should recognise it. Chapter 26 explains how to work in it safely and how to rescue commits made there. For now: **if `git status` says "HEAD detached", switch to a branch before you make commits.**

20.10 Naming branches

Git allows most names, but some are rejected, and some are better than others.

```
$ git branch feature/add-tea
$ git branch fix-broken-link
$ git branch "bad name"
fatal: 'bad name' is not a valid branch name
$ git branch bad..name
fatal: 'bad..name' is not a valid branch name
$ git branch
  feature/add-tea
  fix-broken-link
* main
$ git branch --show-current
main
```

Recorded in Bash; *ch20-branching/expected-naming.bash.txt*.

Newer Git versions (2.55.0 was tested) print two extra *hint:* lines after each *fatal:* line, pointing to *git help check-ref-format*. The refusal itself is the same.

Two rejections are recorded: a name with a **space** and a name containing `..` are not valid. A name with a slash, `feature/add-tea`, is valid. Slashes group branch names (the branches appear as `feature/add-tea`).

Conventions. These are habits, not Git rules, and your team may have its own:

- lowercase words joined by hyphens: `fix-broken-link`;
- a **short prefix** that says the kind of work: `feature/`, `fix/`, `docs/`, `hotfix/`;
- a name that says what the branch is for, so it still makes sense in a month;
- no spaces, and avoid characters that are special to your shell.

Checked against the documentation (R171). The rules for what Git rejects are in `git check-ref-format` (Git 2.56.0). Among them: a name cannot contain two consecutive dots `..`; ASCII control characters, space, tilde `~`, caret `^` or colon `:`; a question mark `?`, asterisk `*` or open bracket `[`; a backslash; or the sequence `@{`; it cannot begin or end with a slash, end with a dot, or be the single character `@`; and no slash-separated part may begin with a dot or end with `.lock`. The naming conventions above (lowercase, hyphens, prefixes) are habits, not Git rules, and are labelled that way.

20.11 How teams use branches

You now know the mechanics. Here is the shape of what teams do (Chapter 34 covers it in detail).

Kind	Life	Purpose
main	Long-lived	The stable, current version of the project
Feature or topic branches	Short-lived (hours to days)	One piece of work, merged back into main when finished
Release or maintenance branches	Long-lived	Keep older versions fixable
Hotfix branches	Very short	An urgent repair

Two guidelines that will save you trouble:

1. **Keep branches short-lived.** The longer a branch lives, the more main moves away from it, and the harder it is to bring back (Chapter 22).
2. **Do one thing per branch.** A branch called add-tea should contain the tea, not also a redesign of the page.

The next chapter shows how work on a branch is brought back into main: merging.

Checkpoint

What You Learned

- A branch is a name for a commit; branches make parallel work and safe experiments cheap.
- `git branch` lists (current branch starred), creates, renames (-m) and deletes (-d, -D); `git switch` changes branch; `git switch -c` creates and switches.
- `git log --oneline --all --graph` draws the history of all branches.
- Switching changes your working tree to the version on that branch.
- Uncommitted changes that do not clash come along; those that would be overwritten stop the switch (commit or stash).
- `git branch -d` refuses to delete unmerged or current branches; `-D` forces and can lose work.
- `git checkout` is the older command for switching (and restoring files).
- Detached HEAD means HEAD points at a commit, not a branch.
- Good branch names are short, lowercase, hyphenated and prefixed by kind.

New Vocabulary

- **git branch:** lists, creates, renames and deletes branches.
- **git switch:** changes the current branch.
- **Detached HEAD:** the state in which HEAD points directly at a commit.

Commands Learned

`git branch`, `git branch <name>`, `git branch -vv`, `git branch -m`, `git branch -d`, `git branch -D`, `git branch --show-current`, `git switch`, `git switch -c`, `git switch --detach`, `git checkout`, `git checkout -b`, `git log --oneline --all --graph`.

Common Mistakes

1. **Forgetting that `git branch <name>` does not switch.**
2. **Committing on the wrong branch.** Check `git branch` (or `git status`) first.
3. **Switching with uncommitted changes** and being surprised that they came along.
4. **Using `git branch -D`** without reading the warning.
5. **Making commits in a detached HEAD**, then switching away.
6. **Leaving branches alive for weeks** until they are hard to merge.

Practice

Do the exercises in the Exercises section at the end of this chapter, including Project 4 (part 1).

Self-Test

1. What is a branch, in one sentence?
2. What is the difference between `git branch add-tea` and `git switch -c add-tea`?
3. You switched from `main` to `add-tea` and `menu.md` changed. Why?
4. Why did `git branch -d add-tea` refuse, and what does `-D` do differently?
5. Your `git switch` was refused with "would be overwritten". Name two ways out.
6. What does "HEAD detached at 9f10b43" mean, and what should you do before making commits?

Before Moving On

You are ready for Chapter 21 if you can:

- [] create a branch, commit on it and see the graph
- [] switch between branches and explain why the files change
- [] delete a merged branch and explain why `-d` sometimes refuses
- [] recognise a detached HEAD and get back to a branch

How this chapter was checked

Claim	Evidence class	Ledger
Listing, creating, switching, <code>-c</code> , <code>-vv</code> , <code>rename</code> , <code>delete</code> and <code>force-delete</code> ; the graph; files change on switch	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; identical on the CI runner's Git 2.55.0 except one hint (both recorded); "experimental" note checked in the Git 2.50.0 and 2.51.0 documentation and release notes	R1 69
Switching with uncommitted changes (carried; refused; stash)	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-switch</code>); the ledger row says which.	R1 72
<code>git checkout</code> equivalents; detached HEAD	Locally tested (as above); reason for the split checked in the Git 2.23.0 release notes	R1 70

Claim	Evidence class	Le dg er
Invalid branch names; naming conventions	Recorded messages: locally tested; rules checked in <code>git check-ref-format</code> ; conventions: general practice (labelled as such)	R1 71

Where this leads

Chapter 21 brings a branch's work back into another branch. Chapter 22 handles the case in which the two branches changed the same lines. Chapter 24 covers `git stash` fully, and Chapter 26 shows how to recover a deleted branch or work done in a detached HEAD.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use `bakery-menu` (rebuild it from Chapter 17 if needed) or your `sunrise-bakery` project. Check `git status` before every switch.

Level 1 — Guided

1.1 Create, switch, commit. Create the branch `add-tea` with `git branch add-tea`, switch to it with `git switch add-tea`, change `menu.md` to add tea, and commit. Show the history with `git log --oneline --all --graph`. Expected result: `add-tea` is one commit ahead of `main`.

1.2 The files change. Switch to `main` and `cat menu.md`; switch back and `cat` again. Describe what changed and why.

Level 2 — Partially guided

2.1 One step. Delete `add-tea` after merging is not yet possible (Chapter 21). Instead, create `fix-typo` with a single command, rename it to `tidy-menu`, and delete it safely. Which command did you use to delete it, and why did it succeed?

2.2 Provoke the protection. Try `git branch -d add-tea`. Read the message. What does it protect you from?

2.3 Naming. Try to create branches called `bad name` and `feature/add-tea`. Which worked? Write down the message for the one that failed.

Level 3 — Independent

3.1 In `sunrise-bakery`, create a branch `add-hours-page`, add a new file `hours.html`, commit, switch back to `main`, and show that `hours.html` is not there. Then switch back and show that it is.

Level 4 — Professional scenario

4.1 A colleague says: "I made three commits, switched to `main`, and my work has vanished." You know branches better than that. Write the checks and commands you would use to find the work, and the two most likely explanations. (Chapter 26 gives the full recovery tools.)

Level 5 — Troubleshooting

5.1 `git switch main` prints error: Your local changes to the following files would be overwritten by checkout. List two safe ways out and one unsafe way, with the consequences.

5.2 `git status` says HEAD detached at 3f2a9c1. A learner makes two commits and then runs `git switch main`. Where are the two commits now, and what should the learner have done first?

5.3 A learner ran `git branch -D experiment`, then realised the work was needed. What did Git print when it deleted the branch, and how can that help?

Chapter 21 — Merging [Core]

In this chapter

- what merging means, and which branch you must be on
- the fast-forward merge, which is only a pointer moving
- the three-way merge, which creates a merge commit
- `--no-ff` and `--squash`, and what each does to the shape of history
- **Project 4**: work with branches on the Sunrise Bakery site

Before you start. Chapter 20 (branches, switching, the graph view) and Chapter 17 (`git log`, `git diff`). The recordings use the bakery-menu repository.

Every recording was run in Bash and zsh on Git 2.43.0 and re-run in CI on Git 2.55.0.

21.1 What merging means

Chapter 20 made lines of work. **Merging** brings them back together.

A **merge** combines the changes of one branch into another, so that the receiving branch contains the work of both. Earlier chapters used the word loosely; this chapter shows exactly what Git does.

Two things to fix in your mind before typing anything.

1. **The direction.** You merge a branch **into the branch you are on**. The command is `git merge <source>`, and it changes the **current** branch. The source branch is not changed. So to bring `add-tea` into `main`, you first `git switch main`, then run `git merge add-tea`.
 2. **The situation decides the result.** Depending on how the two branches have moved, Git does one of two very different things: it either just moves a pointer (fast-forward), or it builds a new commit (a true merge).
-

21.2 Fast-forward: when nothing has diverged

Suppose `main` has not moved since you branched, and all the new work is on `add-tea`:

```
$ cd bakery-menu
$ git switch -c add-tea
Switched to a new branch 'add-tea'
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.80\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n- Tea: 1.50\n' > menu.md
$ git commit -am "Add tea to the menu"
[add-tea aled54d] Add tea to the menu
1 file changed, 1 insertion(+)
$ git switch main
Switched to branch 'main'
```

Recorded in Bash; *ch21-merging/expected-ff.bash.txt*.

The history is a straight line: add-tea is simply ahead of main:

```
$ git log --oneline --all --graph
* aled54d (add-tea) Add tea to the menu
* 81f772e (HEAD -> main) Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch21-merging/expected-ff.bash.txt*.

Now merge add-tea into main:

```
$ git merge add-tea
Updating 81f772e..aled54d
Fast-forward
 menu.md | 1 +
 1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch21-merging/expected-ff.bash.txt*.

Read the output. Updating 81f772e..aled54d names the two commits: where main was and where it will be. Fast-forward says what Git did: **it moved the branch pointer forward** along the existing line. No new commit was created; nothing needed combining, because main had nothing that add-tea did not already contain.

```
$ git log --oneline --all --graph
* aled54d (HEAD -> main, add-tea) Add tea to the menu
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch21-merging/expected-ff.bash.txt*.

Now main and add-tea point to the same commit (HEAD -> main, add-tea). Two commands confirm that the branch is finished with, and delete it:

```
$ git branch --merged
add-tea
* main
$ git branch -d add-tea
Deleted branch add-tea (was aled54d).
```

Recorded in Bash; *ch21-merging/expected-ff.bash.txt*.

git branch --merged lists the branches whose work is already contained in the current branch; git branch -d add-tea succeeded, where in Chapter 20 it had refused, because the tea commit is now part of main.

New term: fast-forward. A merge in which the receiving branch simply moves forward to the source branch's commit, because it has not diverged. No merge commit is created.

21.3 Diverged history: the three-way merge

Real projects rarely stay in a straight line. While tea was being added on `add-tea`, someone added the opening hours on `main`. Now **both** branches have commits that the other lacks: the histories have **diverged**.

New term: diverged. Two branches have diverged when each has commits that the other does not have.

```
$ printf 'Open Monday to Saturday.\n' > hours.md
$ git add hours.md
$ git commit -m "Add opening hours"
[main 1e5eef8] Add opening hours
 1 file changed, 1 insertion(+)
 create mode 100644 hours.md
```

Recorded in Bash; *ch21-merging/expected-threeway.bash.txt*.

```
$ git log --oneline --all --graph
* 1e5eef8 (HEAD -> main) Add opening hours
| * aled54d (add-tea) Add tea to the menu
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch21-merging/expected-threeway.bash.txt*.

The graph shows the shape, with two lines splitting from `81f772e`: `main` went on to "Add opening hours" and `add-tea` went on to "Add tea to the menu". Git cannot fast-forward now, because moving `main` to `add-tea` would throw away the opening hours. It must **combine** the two.

```
$ git merge --no-edit add-tea
Merge made by the 'ort' strategy.
 menu.md | 1 +
 1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch21-merging/expected-threeway.bash.txt*.

Read the output. Merge made by the 'ort' strategy. says a merge commit was created. ("ort" is the name of Git's default algorithm for combining changes. The recordings show the name used by both versions tested here; older versions of Git used another name, and the name is not something you need to remember.) The summary shows the change that came in from `add-tea`.

The flag `--no-edit` matters here: **a merge commit needs a message**, and normally Git opens your editor with a suggested one (Merge branch 'add-tea') for you to accept or change. `--no-edit` accepts the suggestion without opening the editor. It is used in the recordings only so that they run without an editor; in your own work you may simply save and close the editor.

The new history:

```
$ git log --oneline --all --graph
* 94d0fc6 (HEAD -> main) Merge branch 'add-tea'
|\
| * a1ed54d (add-tea) Add tea to the menu
* | 1e5eef8 Add opening hours
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch21-merging/expected-threeway.bash.txt*.

The graph now **joins** two lines. The top commit, 94d0fc6 Merge branch 'add-tea', is a **merge commit**.

New term: merge commit. A commit that has two (or more) parents. It records the point at which two lines of work were combined.

The two lines on the left of the graph are its two parents: 1e5eef8 (the main side) and a1ed54d (the add-tea side). Git worked out what to combine by looking at three commits: the two branch tips and their **common ancestor**, 81f772e, the last commit they share. That is why this is called a **three-way merge**: it compares each tip with the shared starting point to see what each branch changed.

Two commands find merge commits:

```
$ git show --stat --oneline HEAD
94d0fc6 (HEAD -> main) Merge branch 'add-tea'

 menu.md | 1 +
 1 file changed, 1 insertion(+)
$ git log --oneline --merges
94d0fc6 (HEAD -> main) Merge branch 'add-tea'
```

Recorded in Bash; *ch21-merging/expected-threeway.bash.txt*.

`git log --merges` lists only merge commits. `git show --stat --oneline HEAD` shows that the merge commit's summary lists the file that came in from the other branch.

21.4 --no-ff: keep the record of a branch

Sometimes a fast-forward would be possible, but you want a merge commit anyway, so that the history shows "these commits were a unit of work". That is the option `--no-ff` ("no fast-forward"):

```
$ git switch main
Switched to branch 'main'
$ git merge --no-ff add-tea -m "Merge the hot drinks"
```

```
Merge made by the 'ort' strategy.
menu.md | 2 ++
1 file changed, 2 insertions(+)
```

Recorded in Bash; *ch21-merging/expected-noff-squash.bash.txt*.

-m "Merge the hot drinks" supplied the merge message. Here add-tea held two commits (tea and coffee). The graph:

```
$ git log --oneline --graph
* 45e4d88 (HEAD -> main) Merge the hot drinks
|\
| * 0bbfb6b (add-tea) Add coffee to the menu
| * aled54d Add tea to the menu
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch21-merging/expected-noff-squash.bash.txt*.

The merge commit 45e4d88 Merge the hot drinks groups the two commits of add-tea visibly. Without --no-ff, Git would have moved main forward and the two commits would have sat in a straight line with no sign that they were once a separate branch.

Trade-off. --no-ff preserves where work came from, at the cost of extra merge commits and a busier history. Straight-line history (fast-forward) is easier to read but forgets the branch. Teams pick one on purpose (Chapter 34).

21.5 --squash: many commits become one

New term: squash. To combine several commits into one.

A third option takes all the commits of a branch and turns their combined change into **one new commit** on the receiving branch, with no link to the original branch. Here add-juice has two commits (juice, lemonade) that create a new file, and they are squashed into main:

```
$ git switch -c add-juice
Switched to a new branch 'add-juice'
$ printf 'Juice: 2.20\n' > cold-drinks.md
$ git add cold-drinks.md
$ git commit -m "Add juice"
[add-juice 8e389ad] Add juice
1 file changed, 1 insertion(+)
create mode 100644 cold-drinks.md
$ printf 'Juice: 2.20\nLemonade: 2.00\n' > cold-drinks.md
$ git commit -am "Add lemonade"
[add-juice 663bebc] Add lemonade
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch21-merging/expected-noff-squash.bash.txt*.

Now switch back and squash:

```
$ git switch main
Switched to branch 'main'
$ git merge --squash add-juice
Updating 45e4d88..663bebc
Fast-forward
Squash commit -- not updating HEAD
 cold-drinks.md | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 cold-drinks.md
$ git status --short
A cold-drinks.md
$ git commit -m "Add cold drinks"
[main 6178945] Add cold drinks
 1 file changed, 2 insertions(+)
 create mode 100644 cold-drinks.md
```

Recorded in Bash; *ch21-merging/expected-noff-squash.bash.txt*.

Read the output carefully:

- (The Fast-forward line here is Git describing the staged change, not a fast-forward of the branch.) Squash commit -- not updating HEAD says that --squash does **not** create the commit itself. It stages the combined change and stops.
- `git status --short` shows A cold-drinks.md: the new file is staged.
- **You** then commit it: `git commit -m "Add cold drinks"` created one commit containing both changes.

```
$ git log --oneline --graph
* 6178945 (HEAD -> main) Add cold drinks
* 45e4d88 Merge the hot drinks
|\
| * 0bbfb6b (add-tea) Add coffee to the menu
| * aled54d Add tea to the menu
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch21-merging/expected-noff-squash.bash.txt*.

The graph shows one new commit at the top, 6178945 Add cold drinks, and **no line joining it to the branch** add-juice. As far as Git's history is concerned, this is an ordinary commit.

That has a consequence:

```
$ git branch -d add-juice
error: the branch 'add-juice' is not fully merged.
If you are sure you want to delete it, run 'git branch -D add-juice'
$ git branch -D add-juice
Deleted branch add-juice (was 663bebc).
```

Recorded in Bash; *ch21-merging/expected-noff-squash.bash.txt*.

`git branch -d add-juice` **refused**: Git does not see the branch's commits inside `main`, because the squash made a new commit with a different identity. The work is in `main`, but Git cannot prove it, so `-d` treats the branch as unmerged. You may then delete it with `-D` (after checking that you really did squash it).

On the newer Git (2.55.0), the refusal message adds a *hint*: line, as in Chapter 20:

```
$ git branch -d add-juice
error: the branch 'add-juice' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D add-juice'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
```

Recorded in Bash; *ch21-merging/expected-noff-squash.bash.alt.txt*.

Trade-off. `--squash` gives a clean, one-commit-per-feature history and hides messy work-in-progress commits. It also **discards** the individual commits and the record of the branch. This is one of the merge options that hosting platforms offer (Chapter 45).

21.6 Choosing

Merge style	What happens	History shape	Use when
Fast-forward (automatic when possible)	Pointer moves	Straight line	The branch has not diverged; you want linear history
Merge commit (three-way; or <code>--no-ff</code>)	New commit with two parents	Lines split and join	You want to record that the work was a unit; branches have diverged
Squash	One new commit with the combined change	Straight line, no trace of the branch	The branch's small commits are noise; you want one entry per feature

There is no universally best choice. The right one depends on what your team wants the history to say, and Chapter 34 returns to it. Whatever the style, the content of `main` afterwards is the same.

21.7 A merge checklist

1. **Commit or stash** your work (`git status` clean; stashing is covered in Chapter 24).
2. **Switch to the branch that should receive** the work: `git switch main`.
3. **Merge**: `git merge <branch>`.
4. **Read the output**. Did it fast-forward, make a merge commit, or report a conflict (Chapter 22)?
5. **Look at the result**: `git log --oneline --graph`.
6. **Delete the finished branch**: `git branch -d <branch>`.

Deep. If a merge goes wrong, you can leave it with `git merge --abort` while it is unfinished, or undo a finished merge (Chapter 25). Do not panic and do not delete anything: the work is safe in the history.

21.8 Project 4: work with branches

Project 4. Goal: use branches on the Sunrise Bakery site, and bring the work back into main in two different ways. Time: about 30 minutes. You need: the sunrise-bakery repository with its commits, and a clean `git status`.

1. Create a branch `add-cakes-page` and switch to it. Add a new file `cakes.html` (copy `menu.html` and change its heading to "Cakes"). Commit it.
2. Switch back to `main`. Check with `ls` that `cakes.html` is **not** there. Create a second branch, `update-hours`, and in it change the "six o'clock" text in `index.html` to "half past five". Commit it. Switch back to `main`.
3. Run `git log --oneline --all --graph`. You should see `main` and two branches each one commit ahead.
4. Merge `add-cakes-page` into `main`. Which kind of merge was it?
5. Merge `update-hours` into `main`. Which kind was it this time, and why is it different?
6. Look at the graph, and delete both branches with `git branch -d`.

Expected result: the first merge fast-forwards (`main` had not moved). After it, `main` has moved, so `update-hours` has diverged: the second merge creates a merge commit. The graph shows one merge. Both `-d` deletions succeed.

Checkpoint questions: Why did the second merge differ from the first? What would change if you had run `git merge --no-ff` for the first? What is the common ancestor for the second merge?

Chapter 22 asks what happens when both branches change the same line.

Checkpoint

What You Learned

- Merging combines one branch into the branch you are on; the source is not changed.
- A **fast-forward** merge only moves a pointer; no merge commit.
- Diverged branches need a **three-way merge**, which compares each tip with the common ancestor and creates a **merge commit** with two parents.
- `--no-ff` forces a merge commit; `--squash` stages the combined change for one new commit and leaves no link to the branch.
- `git branch -d` refuses after a squash, because Git cannot see the branch's commits in the target.
- Choose a merge style by what you want history to say.

New Vocabulary

- **Fast-forward:** a merge that only moves the receiving branch forward.
- **Diverged:** each of two branches has commits the other lacks.
- **Merge commit:** a commit with two or more parents.
- **Squash:** combining several commits into one.

Commands Learned

`git merge <branch>`, `git merge --no-ff`, `git merge --squash`, `git merge --no-edit`, `git branch --merged`, `git log --merges`, `git log --oneline --graph`.

Common Mistakes

1. **Merging in the wrong direction** (being on the wrong branch).
2. **Merging with uncommitted changes.**
3. **Expecting a squash merge to leave a link to the branch.**
4. **Deleting a branch with `-d` without checking that its work is in the target.**
5. **Being surprised by a merge commit** and not reading the output.

Practice

Do the exercises in the Exercises section at the end of this chapter, including Project 4 (section 21.8).

Self-Test

1. You are on `main` and run `git merge add-tea`. Which branch changes?
2. What does Fast-forward in the output mean?
3. Why can the `tea` branch not be fast-forwarded in section 21.3?
4. What is a merge commit, and how many parents does it have?
5. What does `git merge --squash` leave for you to do?
6. Why did `git branch -d` refuse after the squash?

Before Moving On

You are ready for Chapter [22](#) if you can:

- [] perform a fast-forward merge and a three-way merge and tell them apart
- [] read a graph with a merge commit
- [] explain what `--no-ff` and `--squash` do
- [] delete a merged branch safely

How this chapter was checked

Claim	Evidence class	Ledger
Fast-forward merge, <code>--merged</code> , <code>-d</code> afterwards	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; identical on CI's Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-merge</code>); the ledger row says which.	R173
Three-way merge, <code>merge commit</code> , <code>--merges</code> , the <code>ort</code> strategy line	Locally tested (as above); the strategy naming across versions not investigated. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-merge</code>); the ledger row says which.	R174
<code>--no-ff</code> and <code>--squash</code> , and the <code>-d</code> refusal after a squash	Locally tested (as above); the refusal hint differs between 2.43.0 and 2.55.0 (both recorded). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-merge</code>); the ledger row says which.	R175

Where this leads

Chapter 22 handles merges that cannot be completed automatically. Chapter 23 shows how merging and sharing fit together, and Chapter 27 shows another way to combine work. Chapter 45 shows how a hosting platform offers these merge styles behind a button.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use `bakery-menu` (rebuild it from Chapter 17 if needed) or your `sunrise-bakery` project. Check that `git` status is clean before every merge.

Level 1 — Guided

1.1 Fast-forward. Create `add-tea`, add a `tea` line to `menu.md`, `commit`, switch to `main` and run `git merge add-tea`. Expected result: the output says `Fast-forward`, and no merge commit exists.

1.2 Which branch moved? After exercise 1.1, run `git log --oneline --all --graph`. Which branch pointers moved, and which did not?

Level 2 — Partially guided

2.1 Three-way merge. Create `add-tea` with one commit. On `main`, add a new file `hours.md` and `commit`. Merge `add-tea` into `main` with `git merge --no-edit add-tea`. Find the merge commit with `git log --merges`.

2.2 Parents. Run `git show --stat HEAD` on the merge commit from 2.1. How many parents does it show, and which commits are they?

Level 3 — Independent

3.1 Force a merge commit. Repeat 1.1 on a fresh branch, but use `--no-ff`. Compare the graph with the one from 1.2.

3.2 Squash. Make a branch with two commits that create a new file. Squash-merge it into `main`, check `git status --short`, then commit. Explain why the graph shows no link to the branch.

Level 4 — Professional scenario

4.1 Choose a style. Your team wants one history entry per feature and does not care about a feature's work-in-progress commits. Which merge style fits, and what is lost? Then show the commands.

Level 5 — Troubleshooting

5.1 After a squash merge, `git branch -d feature` prints that the branch is not fully merged. Is the work lost? How can you check before you use `-D`?

5.2 A learner runs `git merge main` while on `add-tea`, expecting `main` to receive the tea line. What actually happened, and how do they get the result they wanted?

Chapter 22 — Merge Conflicts [Core]

In this chapter

- why Git sometimes cannot merge on its own
- how to read the markers Git writes into your file
- how to resolve a conflict by hand, and how to conclude the merge
- how to walk away with `git merge --abort`
- a clearer marker style, `diff3`
- **Project 6:** resolve a real conflict on the Sunrise Bakery site

Before you start. Chapter 21. The recordings use the bakery-menu repository and were run in Bash and zsh on Git 2.43.0 and re-run in CI on Git 2.55.0.

22.1 Why conflicts happen

In Chapter 21 Git combined two branches by itself. It can do that when the branches changed **different** parts of the files. When both branches changed the **same lines** in different ways, Git cannot know which change you want. It does not guess. It stops and asks you.

New term: merge conflict. A situation in which Git cannot combine two sets of changes automatically because both changed the same part of a file, or one changed what the other removed. A conflict is not an error in your work; it is Git asking a question only you can answer.

Here is the smallest example. The white loaf costs 2.80. On the branch `raise-bread` someone makes it 3.00:

```
$ git switch -c raise-bread
Switched to a new branch 'raise-bread'
$ printf '# Sunrise Bakery menu\n\n White loaf: 3.00\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n' > menu.md
$ git commit -am "Raise the white loaf to 3.00"
[raise-bread 73e14f7] Raise the white loaf to 3.00
1 file changed, 1 insertion(+), 1 deletion(-)
```

Recorded in Bash; `ch22-conflicts/expected-conflict.bash.txt`.

Meanwhile, on `main`, someone else makes the same line 2.90:

```
$ git switch main
Switched to branch 'main'
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.90\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n' > menu.md
$ git commit -am "Raise the white loaf to 2.90"
[main 4afeda6] Raise the white loaf to 2.90
1 file changed, 1 insertion(+), 1 deletion(-)
```

Recorded in Bash; *ch22-conflicts/expected-conflict.bash.txt*.

Both commits changed the same line, starting from the same ancestor. Now merge:

```
$ git merge raise-bread
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
Automatic merge failed; fix conflicts and then commit the result.
```

Recorded in Bash; *ch22-conflicts/expected-conflict.bash.txt*.

Read the three lines. Auto-merging menu.md says Git tried. CONFLICT (content): Merge conflict in menu.md names the problem and the file. Automatic merge failed; fix conflicts and then commit the result. says what you must do. The merge is now **half done**: Git stopped in the middle and is waiting.

CAUTION. A conflict does not lose anything. Both versions are safe in the history. Do not delete branches, do not run `git reset --hard` in a hurry, and do not close the terminal in fear. Read what Git says.

22.2 Ask Git what state you are in

```
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   menu.md

no changes added to commit (use "git add" and/or "git commit -a")
```

Recorded in Bash; *ch22-conflicts/expected-conflict.bash.txt*.

You have unmerged paths means the merge is unfinished. The status gives you your two ways out: fix the conflicts and commit, or `git merge --abort`. The file menu.md is listed under **Unmerged paths** as both modified: both branches changed it.

To list only the conflicted files (useful when there are many):

```
$ git diff --name-only --diff-filter=U
menu.md
```

Recorded in Bash; *ch22-conflicts/expected-conflict.bash.txt*.

22.3 Read the markers

Open the file. Git has written both versions into it:

```
$ cat menu.md
# Sunrise Bakery menu

<<<<<<< HEAD
- White loaf: 2.90
=====
- White loaf: 3.00
>>>>>>> raise-bread
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch22-conflicts/expected-conflict.bash.txt*.

New term: conflict markers. The lines Git inserts into a file at a conflict: <<<<<<<, ===== and >>>>>>>. They are not part of your content. They must all be removed before you finish.

Read it as three parts:

Marker line	Meaning
<<<<<<< HEAD	Start. What follows is your side: the branch you are on (main)
=====	The divider between the two versions
>>>>>>> raise-bread	End. What came just before is the other side, from the branch you merged in

Everything outside the markers (the heading, the rolls, the cake) was merged cleanly and needs nothing. Only the lines between <<<<<<< and >>>>>>> need your decision.

22.4 Resolve it

Resolving means editing the file until it holds exactly what you want, with **no markers left**. You have three choices: keep your side, keep theirs, or write something new. Here the two people agree to settle on 2.95, so the resolved file is:

```
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.95\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n' > menu.md
```

Recorded in Bash; *ch22-conflicts/expected-conflict.bash.txt*.

The recording overwrote the file in one command; in real work you would open it in your editor, delete the three marker lines and the version you do not want, and adjust the rest by hand. Then tell Git you are done with that file by **staging** it:

```
$ git add menu.md
$ git status
On branch main
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   menu.md
```

Recorded in Bash; *ch22-conflicts/expected-conflict.bash.txt*.

Git now says All conflicts fixed but you are still merging. Committing finishes the merge:

```
$ git commit -m "Merge raise-bread, settling on 2.95"
[main cce1095] Merge raise-bread, settling on 2.95
$ git log --oneline --graph
*   cce1095 (HEAD -> main) Merge raise-bread, settling on 2.95
|\
| * 73e14f7 (raise-bread) Raise the white loaf to 3.00
* | 4afeda6 Raise the white loaf to 2.90
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch22-conflicts/expected-conflict.bash.txt*.

The graph is the same shape as a normal three-way merge (Chapter 21): two parents, one merge commit. The merge commit is where the conflict was settled, and it records your decision.

CAUTION. Git does not check that the markers are gone. If you stage a file that still contains <<<<<<, Git will happily commit it, and your menu will contain junk lines. Before you `git add`, search the file for <<<<<<, ===== and >>>>>>, and read it once as a reader would.

22.5 Changing your mind: `git merge --abort`

If the conflict is bigger than you expected, or you are not sure what you are doing, you can undo the unfinished merge and return to how things were before you typed `git merge`:

```
$ git merge --abort
$ git status
On branch main
nothing to commit, working tree clean
```

Recorded in Bash; *ch22-conflicts/expected-abort.bash.txt*.

The working tree is clean. The file has your side's content again:

```
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.90
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch22-conflicts/expected-abort.bash.txt*.

And the history is as before, with the two branches still diverged:

```
$ git log --oneline --graph --all
* 4afeda6 (HEAD -> main) Raise the white loaf to 2.90
| * 73e14f7 (raise-bread) Raise the white loaf to 3.00
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch22-conflicts/expected-abort.bash.txt*.

Nothing was lost, and you can try the merge again later. `--abort` works only while the merge is unfinished. Once you have committed the merge, you must undo it another way (Chapter 25).

Checked against the documentation (R356). The `git merge` manual (Git 2.56.0) says that `git merge --abort` "will abort the merge process and try to reconstruct the pre-merge state", but that if there were uncommitted changes when the merge started, and especially if they were changed further after it started, it "will in some cases be unable to reconstruct the original (pre-merge) changes". It warns that running `git merge` with non-trivial uncommitted changes "is discouraged", and recommends to "always commit or stash your changes before running `git merge`". (In a quick test here, an uncommitted edit to a different file did survive `--abort`; the manual's warning is the rule to follow.) The safe habit is the same: start every merge with a clean `git status`.

22.6 A clearer marker style: `diff3`

The default markers show two versions, and you must guess what the line said before. The setting `merge.conflictStyle` can change that. With `diff3`, Git also shows the **common ancestor**:

```
$ git config merge.conflictStyle diff3
```

Recorded in Bash; *ch22-conflicts/expected-diff3.bash.txt*.

Repeat the same conflict:

```
$ git merge raise-bread
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
Automatic merge failed; fix conflicts and then commit the result.
```

Recorded in Bash; *ch22-conflicts/expected-diff3.bash.txt*.

```
$ cat menu.md
# Sunrise Bakery menu

<<<<<< HEAD
- White loaf: 2.90
|||||| 81f772e
- White loaf: 2.80
=====
- White loaf: 3.00
>>>>>> raise-bread
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch22-conflicts/expected-diff3.bash.txt*.

There is a new section between `||||||` and `=====`: the original line, `White loaf: 2.80`. Now you can see what each side changed: `main` went from 2.80 to 2.90; `raise-bread` went from 2.80 to 3.00. That often makes the decision obvious. The recording then aborts the merge:

```
$ git merge --abort
```

Recorded in Bash; *ch22-conflicts/expected-diff3.bash.txt*.

Checked against the documentation (R177). Git's `merge.conflictStyle` documentation (Git 2.56.0) says the default is `merge`, which shows the two sides; `diff3` "adds a `||||||` marker and the original text before the `=====` marker"; and a third style, `zdiff3`, "is similar to `diff3` but removes matching lines on the two sides from the conflict region when those matching lines appear near either the beginning or the end of a conflict region". The documentation also says the `merge` style "tends to produce smaller conflict regions than `diff3`". A change of the default is not announced in Git 2.56.0's `BreakingChanges` document. This book recorded `merge` and `diff3`, not `zdiff3`.

Whichever style you choose, remember: the extra section is only for reading. You still delete all the marker lines.

22.7 Reducing the pain

- **Merge often.** Small, frequent merges make small conflicts.
- **Keep branches short.** The longer two branches live apart, the more they can collide.
- **Keep commits focused** (Chapter 19). A commit that changes one thing is easy to reason about when it conflicts.

- **Talk.** Two people editing the same line is often a coordination problem, not a Git problem.
- **Start from a clean tree** so `--abort` has an easy job.

Editors and dedicated merge tools can show the two sides next to each other. They change the view, not the process: you still end with a clean file, `git add`, and `git commit`.

22.8 A conflict checklist

1. **Stay calm.** Read the output and `git status`.
 2. **Find the files:** `git diff --name-only --diff-filter=U`.
 3. **Open each file** and find the markers.
 4. **Decide** what the final content should be. Ask the other author if needed.
 5. **Edit:** remove the markers and the unwanted lines.
 6. **Check** that no `<<<<<<<, =====, >>>>>>>` remain.
 7. **Stage** each resolved file: `git add <file>`.
 8. **Commit** to finish the merge: `git commit`.
 9. **Or abort:** `git merge --abort`, at any time before step 8.
-

22.9 Project 6: resolve a merge conflict

Project 6. Goal: cause, read and resolve a real conflict on the Sunrise Bakery site, and practise leaving one. Time: about 30 minutes. You need: the sunrise-bakery repository with a few commits and a clean `git status`.

1. Create a branch `heading-warm` and change the first heading of `index.html` to "Warm bread, every morning". Commit. Switch back to `main`.
2. Create a branch `heading-fresh` and change the **same heading** to "Fresh bread, every morning". Commit. Switch back to `main`.
3. Merge `heading-warm` into `main` (a fast-forward). Then merge `heading-fresh`. Read what Git says.
4. Open `index.html`. Find the markers, and say which side is HEAD.
5. Run `git merge --abort`. Check with `git status` that everything is as before.
6. Merge again. This time write a heading that combines both ideas, remove all markers, `git add index.html`, and commit.
7. Show the graph with `git log --oneline --graph --all`.

Expected result: the first merge fast-forwards, the second stops with `CONFLICT (content)`, the abort restores the previous state, and the second attempt ends in a merge commit that holds your combined heading.

Checkpoint questions: Why did the first merge not conflict? How did you make sure that no marker was left in the file? What would have changed if you had used `git config merge.conflictStyle diff3` first?

Checkpoint

What You Learned

- A conflict arises when both branches change the same lines differently; Git stops rather than guessing.
- The merge is left unfinished, and `git status` says what to do.
- Markers show your side (HEAD), a divider, and the other side.
- You resolve by editing the file, staging it, and committing.
- `git merge --abort` returns you to before the merge.
- `merge.conflictStyle diff3` also shows the common ancestor.

New Vocabulary

- **Merge conflict:** Git cannot combine changes automatically and asks you to decide.
- **Conflict markers:** the <<<<<<, ===== and >>>>>> lines Git writes into a conflicted file.

Commands Learned

`git merge --abort`, `git diff --name-only --diff-filter=U`, `git config merge.conflictStyle diff3`.

Common Mistakes

1. **Committing with the markers still in the file.**
2. **Panicking and deleting the branch or running a destructive reset.**
3. **Keeping one side without reading the other.**
4. **Forgetting `git add` after editing**, so the file stays unmerged.
5. **Starting a merge with uncommitted work.**

Practice

Do the exercises in the Exercises section at the end of this chapter, including Project 6 (section 22.9).

Self-Test

1. Why did Git stop in section 22.1 but not in Chapter 21?
2. What does <<<<<< HEAD mark, and which branch is on that side?
3. What must be true of the file before you run `git add`?
4. How do you leave an unfinished merge?
5. What extra information does `diff3` add?

Before Moving On

You are ready for Chapter 23 if you can:

- [] cause a conflict on purpose and read its markers

- [] resolve it and finish the merge
- [] abort a conflicted merge
- [] explain what HEAD means inside the markers

How this chapter was checked

Claim	Evidence class	Ledger
Conflict output, status, markers, resolution and merge commit	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; identical on CI's Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-merge); the ledger row says which.	R1 76
git merge --abort may be unable to restore uncommitted changes present when the merge started	Officially verified (docs only): the git merge manual, Git 2.56.0	R3 56
--abort restores the pre-merge state; diff3 shows the ancestor	Locally tested (as above); the three styles and the default checked in the merge.conflictStyle documentation; zdiff3 not run	R1 77

Where this leads

Chapter 23 moves work between repositories, where conflicts also appear when histories diverge. Chapter 27 shows conflicts in a different setting, and Chapter 45 shows how a hosting platform reports them.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use bakery-menu (rebuild it from Chapter 17 if needed). Start every exercise with a clean git status.

Level 1 — Guided

1.1 Cause a conflict. Create raise-bread and change the white loaf price to 3.00; commit. On main, change the same line to 2.90; commit. Run git merge raise-bread. Expected result: CONFLICT (content): Merge conflict in menu.md.

1.2 Read the state. Run git status and git diff --name-only --diff-filter=U. Which words tell you the merge is unfinished?

Level 2 — Partially guided

2.1 Read the markers. Run cat menu.md. Which price is on the HEAD side, and which is on the raise-bread side?

2.2 Resolve. Edit the file so the loaf costs 2.95 and no marker remains, stage it, and commit. Check the graph with git log --oneline --graph.

Level 3 — Independent

3.1 Abort. Cause the conflict again on a fresh copy and leave it with `git merge --abort`. Prove with `git status`, `cat` and `git log --oneline --graph --all` that nothing changed.

3.2 Show the ancestor. Set `git config merge.conflictStyle diff3`, cause the conflict again, and find the line that shows the original price.

Level 4 — Professional scenario

4.1 Two files. Make the same kind of conflict in two files at once. Resolve one, leave the other, and use `git status` to prove that the merge is not finished. Then finish it.

Level 5 — Troubleshooting

5.1 A learner commits a merge and later notices `<<<<<< HEAD` inside `menu.md`. What went wrong, and how do they repair it?

5.2 A learner ran `git merge --abort` but Git says `There is no merge to abort`. What are two possible reasons?

Chapter 23 — Remotes [Core]

In this chapter

- what a remote is, using a shared folder on your own computer, with no account and no internet
- clone, push, fetch and pull, and the difference between them
- remote-tracking branches such as origin/main, and upstream tracking
- what happens when two people push at once
- managing remotes with `git remote`
- **Project 5:** share a repository between two "people"

Before you start. Chapter 20, Chapter 21, Chapter 22 and the idea of a remote from Chapter 5. Everything here runs on one computer; a hosting platform such as GitHub appears only in Chapter 36.

Every recording was run in Bash and zsh on Git 2.43.0 and re-run in CI on Git 2.55.0.

23.1 A remote can be a folder

Until now your repository lived in one place. To share work, Git copies commits between repositories. The other repository is called a **remote**, and it does not have to be a website: it can be a folder on the same computer, or one reached over a network. The commands are identical, which is why we can learn them without an account.

We build the smallest sharing arrangement. One **bare repository** plays the shared place. Two people, Alice and Bob, each keep their own copy.

New term: bare repository. A repository with no working files: only Git's database. It is what you share through, because nobody edits files inside it. Hosting platforms keep bare repositories.

```
$ mkdir shared
$ git init --bare shared/bakery.git
Initialized empty Git repository in /home/learner/shared/bakery.git/
```

Recorded in Bash; *ch23-remotes/expected-bare.bash.txt*.

`git init --bare` creates the shared repository. By convention its folder name ends in `.git`. Now Alice **clones** it, meaning she makes her own copy:

```
$ git clone shared/bakery.git alice
Cloning into 'alice'...
warning: You appear to have cloned an empty repository.
done.
```

Recorded in Bash; *ch23-remotes/expected-bare.bash.txt*.

The warning is expected: the shared repository is empty. Alice adds the first commit:

```
$ cd alice
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.50\n- Rolls (six): 3.00\n' > menu.md
$ git add menu.md
$ git commit -m "Add the menu"
[main (root-commit) 807a5c6] Add the menu
 1 file changed, 4 insertions(+)
 create mode 100644 menu.md
```

Recorded in Bash; *ch23-remotes/expected-bare.bash.txt*.

23.2 The remote, and its name

Ask Git which remotes exist:

```
$ git remote -v
origin /home/learner/shared/bakery.git (fetch)
origin /home/learner/shared/bakery.git (push)
```

Recorded in Bash; *ch23-remotes/expected-bare.bash.txt*.

`git clone` created a remote called `origin` and remembered its address. `origin` is only a **name**, a short label for the address, and it is the conventional name for "the place I cloned from". `(fetch)` and `(push)` are two directions and are normally the same address.

Now look at the branches with more detail:

```
$ git branch -vv
* main 807a5c6 [origin/main: gone] Add the menu
```

Recorded in Bash; *ch23-remotes/expected-bare.bash.txt*.

`[origin/main: gone]` is Git's way of saying that Alice's `main` is set to follow `origin/main`, but that branch does not exist yet, because the shared repository is empty. Publishing it fixes that:

```
$ git push -u origin main
To /home/learner/shared/bakery.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Recorded in Bash; *ch23-remotes/expected-bare.bash.txt*.

Read the output. `* [new branch] main -> main` says a new branch was created in the remote. `-u` (short for `--set-upstream`) also told Git that Alice's `main` should follow `origin/main`:

```
$ git branch -vv
* main 807a5c6 [origin/main] Add the menu
$ git status
On branch main
```

```
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Recorded in Bash; *ch23-remotes/expected-bare.bash.txt*.

New term: remote-tracking branch. Your local note of where a branch on a remote was the last time you looked, for example `origin/main`. You cannot commit on it; Git moves it only when you fetch, pull or push.

New term: upstream branch. The branch on a remote that a local branch follows. It is what makes plain `git push` and `git pull` know where to go, and what lets `git status` say `Your branch is up to date with 'origin/main'`.

Now Bob clones the same shared repository:

```
$ cd ..
$ git clone shared/bakery.git bob
Cloning into 'bob'...
done.
$ cd bob
$ git log --oneline
807a5c6 (HEAD -> main, origin/main, origin/HEAD) Add the menu
$ git remote -v
origin  /home/learner/shared/bakery.git (fetch)
origin  /home/learner/shared/bakery.git (push)
```

Recorded in Bash; *ch23-remotes/expected-bare.bash.txt*.

Bob has Alice's commit and the same remote address. Notice `HEAD -> main, origin/main, origin/HEAD`: Bob's main and his note of `origin/main` currently point to the same commit.

23.3 Fetch, and why `pull` is two steps

Alice makes a new commit and pushes it:

```
$ cd alice
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.80\n- Rolls (six): 3.00\n' > menu.md
$ git commit -am "Raise the price of the white loaf"
[main 23d585e] Raise the price of the white loaf
1 file changed, 1 insertion(+), 1 deletion(-)
$ git push
To /home/learner/shared/bakery.git
807a5c6..23d585e  main -> main
```

Recorded in Bash; *ch23-remotes/expected-fetch.bash.txt*.

807a5c6..23d585e main -> main shows the remote branch moving from the old commit to the new one. Bob has not heard about it yet:

```
$ cd ../bob
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Recorded in Bash; *ch23-remotes/expected-fetch.bash.txt*.

Bob's Git says he is up to date, but that only describes what he **last knew**. Git does not talk to the remote on its own. To learn what happened, Bob **fetches**:

```
$ git fetch
From /home/learner/shared/bakery
 807a5c6..23d585e  main      -> origin/main
```

Recorded in Bash; *ch23-remotes/expected-fetch.bash.txt*.

New term: fetch. Download new commits and update the remote-tracking branches, **without changing your own branches or files**.

Only origin/main moved. Bob's main and his files did not. Now his status can tell him something new:

```
$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)

nothing to commit, working tree clean
```

Recorded in Bash; *ch23-remotes/expected-fetch.bash.txt*.

He can look at the difference safely before deciding anything:

```
$ git log --oneline main
807a5c6 (HEAD -> main) Add the menu
$ git log --oneline origin/main
23d585e (origin/main, origin/HEAD) Raise the price of the white loaf
807a5c6 (HEAD -> main) Add the menu
$ git diff main origin/main
diff --git a/menu.md b/menu.md
index 0b15fe9..1f2a95a 100644
--- a/menu.md
+++ b/menu.md
@@ -1,4 +1,4 @@
 # Sunrise Bakery menu
```



```
-- White loaf: 2.50
+- White loaf: 2.80
- Rolls (six): 3.00
```

Recorded in Bash; *ch23-remotes/expected-fetch.bash.txt*.

Then he brings the work into his branch with **pull**:

```
$ git pull
Updating 807a5c6..23d585e
Fast-forward
 menu.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Recorded in Bash; *ch23-remotes/expected-fetch.bash.txt*.

New term: pull. Fetch, then integrate: `git pull` is `git fetch` followed by a merge (by default) of the upstream branch into your current branch.

Here the merge was a fast-forward (Chapter 21) because Bob had nothing of his own. Afterwards:

```
$ git log --oneline
23d585e (HEAD -> main, origin/main, origin/HEAD) Raise the price of the white loaf
807a5c6 Add the menu
$ git branch -vv
* main 23d585e [origin/main] Raise the price of the white loaf
```

Recorded in Bash; *ch23-remotes/expected-fetch.bash.txt*.

The rule to remember. `fetch` is safe and only updates your notes. `pull` changes your branch. If you are unsure, `fetch` first, look, then merge.

23.4 When two people push at once

Now the case that makes remotes interesting. Alice pushes a commit. Bob, who did not fetch, commits something else and tries to push:

```
$ cd alice
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.80\n- Rolls (six): 3.00\n' > menu.md
$ git commit -am "Raise the price of the white loaf"
[main 23d585e] Raise the price of the white loaf
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git push
To /home/learner/shared/bakery.git
 807a5c6..23d585e  main -> main
```

Recorded in Bash; *ch23-remotes/expected-diverge.bash.txt*.

```
$ cd ../bob
$ printf 'Open Monday to Saturday.\n' > hours.md
$ git add hours.md
$ git commit -m "Add opening hours"
[main 41cdbf3] Add opening hours
 1 file changed, 1 insertion(+)
 create mode 100644 hours.md
```

Recorded in Bash; *ch23-remotes/expected-diverge.bash.txt*.

Bob's push is **rejected**:

```
$ git push
To /home/learner/shared/bakery.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to '/home/learner/shared/bakery.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Recorded in Bash; *ch23-remotes/expected-diverge.bash.txt*.

Read it. ! [rejected] main -> main (fetch first) says the push was refused, and (fetch first) tells him why: the remote has work he does not have. Git protects the shared branch: it will not overwrite Alice's commit. The hint suggests git pull. Nothing is lost.

Bob fetches and asks for the state:

```
$ git fetch
From /home/learner/shared/bakery
   807a5c6..23d585e  main       -> origin/main
$ git status
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
   (use "git pull" if you want to integrate the remote branch with yours)

nothing to commit, working tree clean
```

Recorded in Bash; *ch23-remotes/expected-diverge.bash.txt*.

diverged is the word from Chapter 21: each side has one commit the other lacks. Now he pulls:

```
$ git pull
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
```

```
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
```

Recorded in Bash; *ch23-remotes/expected-diverge.bash.txt*.

Recent versions of Git refuse a `git pull` on diverged branches until you say how to reconcile them: by merging, rebasing or accepting only fast-forwards. You choose once, either for one command or as a setting (Chapter 14). This book chooses **merge** for now, with `--no-rebase`, and `--no-edit` only to skip the editor in the recording:

```
$ git pull --no-rebase --no-edit
Merge made by the 'ort' strategy.
 menu.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Recorded in Bash; *ch23-remotes/expected-diverge.bash.txt*.

```
$ git log --oneline --graph
* 4d21943 (HEAD -> main) Merge branch 'main' of /home/learner/shared/bakery
|\
| * 23d585e (origin/main, origin/HEAD) Raise the price of the white loaf
* | 41cdbf3 Add opening hours
|/
* 807a5c6 Add the menu
```

Recorded in Bash; *ch23-remotes/expected-diverge.bash.txt*.

The graph is the merge from Chapter 21, and the merge commit's message names the remote address. Now Bob's push is accepted, since his branch contains everything the remote has:

```
$ git push
To /home/learner/shared/bakery.git
 23d585e..4d21943  main -> main
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Recorded in Bash; *ch23-remotes/expected-diverge.bash.txt*.

CAUTION. When a push is rejected, do **not** reach for `--force`. Forcing tells the remote to discard the commits you do not have, which would delete Alice's work. Fetch, integrate, then push normally. Chapter 33 returns to why force-pushing shared branches is dangerous.

Checked against the documentation and the source (R180). The `pull.rebase` and `pull.ff` settings are documented (Git 2.56.0): `pull.rebase` set to `true` rebases "instead of merging", and the manual calls this "a possibly dangerous operation"; `pull.ff` set to `only` allows only fast-forwards. The git pull manual of Git 2.50.0 said that on divergent branches "the user needs to specify how to reconcile the divergent branches with `--rebase` or `--no-rebase`". The manual of Git 2.55.0 and 2.56.0 instead lists `git pull --ff-only` as "the default". The source code settles which is right: in `builtin/pull.c` of Git v2.56.0, when no `--ff`, `--ff-only` or `rebase` choice is set (on the command line or in configuration) and the branches have diverged, Git shows the advice and stops with `Need to specify how to reconcile divergent branches`. So a plain `git pull` still fast-forwards when it can, still refuses when the branches have diverged, and the recordings on Git 2.43.0, 2.55.0 and 2.56.0 agree with the source. The 2.55+ manual sentence is looser than the behaviour; the book follows the behaviour.

23.5 Managing remotes

You can have more than one remote. The `git remote` commands manage them:

```
$ cd alice
$ git remote
origin
$ git remote -v
origin /home/learner/shared/bakery.git (fetch)
origin /home/learner/shared/bakery.git (push)
```

Recorded in Bash; *ch23-remotes/expected-remote-cmds.bash.txt*.

Add a second remote, rename it, and push to it:

```
$ git remote add backup ../shared/bakery.git
$ git remote -v
backup ../shared/bakery.git (fetch)
backup ../shared/bakery.git (push)
origin /home/learner/shared/bakery.git (fetch)
origin /home/learner/shared/bakery.git (push)
$ git remote rename backup mirror
$ git remote
mirror
origin
$ git push mirror main
Everything up-to-date
```

Recorded in Bash; *ch23-remotes/expected-remote-cmds.bash.txt*.

Here `mirror` points at the same shared repository, so Git found nothing new to send. List the remote-tracking branches, then remove the remote:

```
$ git branch -r
mirror/main
origin/main
```

```
$ git remote remove mirror
$ git remote
origin
$ git branch -r
origin/main
```

Recorded in Bash; *ch23-remotes/expected-remote-cmds.bash.txt*.

Removing a remote also removes its remote-tracking branches, as the second `git branch -r` shows. Finally, the tracking setup is stored as ordinary configuration (Chapter 14):

```
$ git config --get remote.origin.url
/home/learner/shared/bakery.git
$ git config --get branch.main.remote
origin
$ git config --get branch.main.merge
refs/heads/main
```

Recorded in Bash; *ch23-remotes/expected-remote-cmds.bash.txt*.

`branch.main.remote` says which remote, and `branch.main.merge` says which branch on it. Together they are the "upstream" from section 23.2.

23.6 Push, fetch and pull at a glance

Command	Talks to the remote?	Changes your branch?	Changes the remote?
<code>git fetch</code>	Yes	No (only <code>origin/...</code> notes)	No
<code>git pull</code>	Yes	Yes (fetch, then merge)	No
<code>git push</code>	Yes	No	Yes
<code>git clone</code>	Yes	Creates a new repository	No
<code>git status</code>	No	No	No

The last row matters: `git status` compares with the **last fetched** state, not with what is on the remote right now.

23.7 Project 5: share a repository between two people

Project 5. Goal: act as two collaborators on one computer. Time: about 40 minutes. You need: Git configured (Chapter 14) and an empty folder to work in.

1. Create a bare repository `shared/bakery.git`. Clone it twice, as `alice` and `bob`.
2. As Alice, add `menu.md`, commit, and `git push -u origin main`.
3. As Bob, `git pull` and confirm that the menu arrived.
4. As Alice, change a price and push. As Bob, run `git fetch`, then `git status`, then `git diff main origin/main`, then `git pull`.

5. Make both people commit different files without fetching. Let Bob push first, then Alice; read Alice's rejection, fetch, pull, and push.
6. Draw the final graph with `git log --oneline --graph --all` and explain each line.

Expected result: step 5 produces one rejected push, and after the pull a merge commit and a successful push. Both clones end with the same history.

Checkpoint questions: Why did `git status` not tell Bob about Alice's change before he fetched? Which commands changed his branch, and which did not?

Checkpoint

What You Learned

- A remote is another repository; it can be a local folder. A bare repository is what you share through.
- `git clone` copies a repository and creates the remote origin.
- `git push -u origin main` publishes a branch and sets its upstream.
- `git fetch` updates `origin/...` without touching your branches; `git pull` is `fetch` plus `merge`.
- A push is rejected when the remote has commits you lack; fetch, integrate, push again. Never force it to get past this.
- `git remote` adds, renames, lists and removes remotes; tracking settings are ordinary configuration.

New Vocabulary

- **Bare repository**: a repository without working files, used as a shared hub.
- **Remote-tracking branch**: your local note of a remote branch, such as `origin/main`.
- **Upstream branch**: the remote branch a local branch follows.
- **Fetch**: download new commits without changing your branches.
- **Pull**: fetch, then merge into the current branch.

Commands Learned

`git init --bare`, `git clone`, `git remote`, `git remote -v`, `git remote add`, `git remote rename`, `git remote remove`, `git push -u origin main`, `git push`, `git fetch`, `git pull`, `git pull --no-rebase`, `git branch -vv`, `git branch -r`.

Common Mistakes

1. **Believing `git status` shows the remote's current state.**
2. **Using `pull` blindly** instead of fetching and looking first.
3. **Forcing a rejected push.**
4. **Forgetting `-u`** on the first push, then wondering why `git pull` says there is no tracking information.
5. **Confusing the remote's name (origin) with its address.**

Practice

Do the exercises in the Exercises section at the end of this chapter, including Project 5 (section 23.7).

Self-Test

1. What does `git clone` create besides a copy of the files?
2. What does `-u` do in `git push -u origin main`?
3. Which changes: `git fetch`, or `git pull`?
4. Why was Bob's push rejected in section 23.4, and what was the safe response?
5. What is `origin/main`, and when does it move?

Before Moving On

You are ready for Chapter 24 if you can:

- [] build a bare repository and clone it twice
- [] explain fetch versus pull, and prove it with `git status`
- [] resolve a rejected push without forcing
- [] list, add, rename and remove a remote

How this chapter was checked

Claim	Evidence class	Ledger
Bare repository, clone, <code>remote -v</code> , push <code>-u</code> , tracking, <code>origin/main</code>	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; identical on CI's Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-clone</code> , <code>git-push</code> , <code>git-remote</code>); the ledger row says which.	R1 78
Fetch changes only <code>origin/...</code> ; status wording after fetch; pull fast-forward	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-fetch</code> , <code>git-pull</code>); the ledger row says which.	R1 79
Rejected push, divergence, pull advice, <code>git remote</code> subcommands	Locally tested (as above), same result on Git 2.43.0, 2.55.0 and 2.56.0; <code>pull.rebase/pull.ff</code> and the <code>git pull</code> default checked in the manuals, source builtin/ <code>pull.c</code> v2.56.0 confirms the refusal, the manual's "default" wording is looser; hosting-platform behaviour not covered	R1 80

Where this leads

Chapter 24 adds small everyday tools. Chapter 36 and Chapter 39 replace the shared folder with a hosting platform: the commands stay the same, and only the address and the sign-in change. Chapter 45 shows how platforms turn "pull before you push" into a review process.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Work in an empty folder, and use a bare repository as the shared remote.

Level 1 — Guided

1.1 Build the hub. Create `shared/bakery.git` with `git init --bare`, then clone it as `alice`. Which warning does Git print, and why?

1.2 Publish. As Alice, commit a `menu.md` and run `git push -u origin main`. Run `git branch -vv` before and after. What changed?

Level 2 — Partially guided

2.1 Second clone. Clone the hub as `bob`. Run `git log --oneline` and `git remote -v` in Bob's clone.

2.2 Fetch, then pull. As Alice, change a price and push. As Bob, run `git status`, then `git fetch`, then `git status` again, then `git pull`. Explain each of the three status messages.

Level 3 — Independent

3.1 A rejected push. Make Alice and Bob each commit a different file. Push Alice's first. Push Bob's second. Record the exact rejection line.

3.2 Recover. Bring Bob's clone up to date without using `--force`, then push. Show the final graph.

Level 4 — Professional scenario

4.1 Add a backup remote. Add a second bare repository `backup.git` as a remote called `backup`. Push `main` to it, then confirm with `git branch -r` that `backup/main` exists. Rename the remote to `mirror`, then remove it.

Level 5 — Troubleshooting

5.1 A learner says: "git status says I'm up to date, but my colleague pushed an hour ago." Explain, and give the command sequence that shows the truth.

5.2 A learner runs `git pull` and gets `fatal: Need to specify how to reconcile divergent branches`. What does this mean, and what are three ways to proceed?

5.3 A learner wants to get past a rejected push with `git push --force`. Explain what would be lost, and give the safe alternative.

Chapter 24 — Stash and Small Tools [Core]

In this chapter

- `git stash`: put unfinished work aside and bring it back
- a first look at tags, which are names for particular commits
- `git clean`: remove untracked files, carefully
- `git grep`, `git blame` and `git log -S`: find things in a repository
- a **practice session** that tidies up the Sunrise Bakery site

Before you start. Chapter 21 and Chapter 17. The recordings use the bakery-menu repository and were run in Bash and zsh on Git 2.43.0, then re-run in CI on Git 2.55.0.

This chapter is a toolbox. Each tool is small, and each solves a problem you will meet within your first weeks.

24.1 Stash: put work aside

You are in the middle of an edit when someone asks you to fix something else on main. Your work is not ready to commit, and Git will not let you switch branches if it would overwrite it (Chapter 20). You need somewhere to put the half-finished work.

New term: stash. A shelf inside your repository where Git stores uncommitted changes, so that the working tree becomes clean again. You can bring the changes back later.

Here is a half-finished edit to `menu.md`, and a new file, `ideas.txt`:

```
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.80\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n- Tea: 1.50\n' > menu.md
$ printf 'draft\n' > ideas.txt
$ git status --short
M menu.md
?? ideas.txt
```

Recorded in Bash; *ch24-stash/expected-stash.bash.txt*.

M means a tracked file was modified; ?? means Git does not track `ideas.txt` yet. Now stash:

```
$ git stash
Saved working directory and index state WIP on main: 81f772e Add coconut cake
$ git status --short
?? ideas.txt
```

Recorded in Bash; *ch24-stash/expected-stash.bash.txt*.

Read it carefully. The **modified** file was saved on the shelf and reverted, so `menu.md` is clean again. But `ideas.txt` is **still there**: by default `git stash` only takes changes to tracked files. Check the shelf:

```
$ git stash list
stash@{0}: WIP on main: 81f772e Add coconut cake
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch24-stash/expected-stash.bash.txt*.

The entry `stash@{0}` is the newest one (they form a stack: the last in is the first out). The file `menu.md` has its committed content again, without the tea line. Now bring the work back:

```
$ git stash pop
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   menu.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ideas.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (9958d02c6f90ac44192aff2010753d5a93af29c9)
$ git status --short
M menu.md
?? ideas.txt
```

Recorded in Bash; *ch24-stash/expected-stash.bash.txt*.

`git stash pop` **applied** the saved changes and **dropped** the entry from the shelf (Dropped refs/stash@{0} (...)). The status shows the modified file back.

To include untracked files, add `-u`:

```
$ git stash -u
Saved working directory and index state WIP on main: 81f772e Add coconut cake
$ git status --short
$ git stash list
stash@{0}: WIP on main: 81f772e Add coconut cake
```

Recorded in Bash; *ch24-stash/expected-stash.bash.txt*.

Both files went onto the shelf, and the working tree is completely clean. Bring them back the same way:

```
$ git stash pop
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   menu.md
```

```

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ideas.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (130e22629620c583dc9031508b0eda23eecf5126)
$ git status --short
 M menu.md
?? ideas.txt

```

Recorded in Bash; *ch24-stash/expected-stash.bash.txt*.

And when the shelf is empty:

```

$ git stash drop
No stash entries found.

```

Recorded in Bash; *ch24-stash/expected-stash.bash.txt*.

Two stash habits.

- `git stash pop` applies and removes. `git stash apply` applies and **keeps** the entry, which is useful when you are not sure the apply will work.
- Stashes are not a safe archive. They are easy to forget, and dropping one removes your only reference to it (Chapter 26 shows how recovery works). If the work matters for more than an hour, commit it on a branch instead.

CAUTION. If applying a stash conflicts with newer changes, you get the same kind of conflict as in Chapter 22. Resolve it the same way. Check `git status` before you stash and after you pop.

24.2 Tags: names for commits

A branch name moves when you commit. Sometimes you want a name that **stays** on one commit: "this is version 1.0". That is a **tag**.

New term: tag. A fixed name for one commit, used to mark a point in history such as a release. Unlike a branch, a tag does not move when you commit.

Two kinds exist:

New term: lightweight tag. A tag that is only a name pointing at a commit.

New term: annotated tag. A tag that is a full Git object with a tagger, a date and a message. This is the kind normally used for releases.

```
$ git tag
$ git tag v0.1
$ git tag -a v1.0 -m "First public menu"
$ git tag
v0.1
v1.0
```

Recorded in Bash; *ch24-stash/expected-tags.bash.txt*.

The first command listed no tags. `git tag v0.1` made a lightweight tag; `git tag -a v1.0 -m "First public menu"` made an annotated one. Tags show up in the log when you ask for decorations:

```
$ git log --oneline --decorate
81f772e (HEAD -> main, tag: v1.0, tag: v0.1) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch24-stash/expected-tags.bash.txt*.

Both tags point to the same commit here. An annotated tag stores extra information, which `git show` reveals before showing the commit:

```
$ git show v1.0
tag v1.0
Tagger: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:11:00 2026 +0000

First public menu

commit 81f772ee32b93d8fcf80dbb34ab74053b69d97a2 (HEAD -> main, tag: v1.0, tag: v0.1)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:08:00 2026 +0000

    Add coconut cake

diff --git a/menu.md b/menu.md
index 1f2a95a..97e3bea 100644
--- a/menu.md
+++ b/menu.md
@@ -2,3 +2,4 @@
- White loaf: 2.80
- Rolls (six): 3.00
+- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch24-stash/expected-tags.bash.txt*.

Delete and filter tags:

```
$ git tag -d v0.1
Deleted tag 'v0.1' (was 81f772e)
$ git tag
v1.0
$ git tag -l "v1.*"
v1.0
```

Recorded in Bash; *ch24-stash/expected-tags.bash.txt*.

`git tag -d` deleted only the name; the commit is untouched. The pattern `"v1.*"` in `git tag -l` is a wildcard. Chapter 29 covers version numbering, sharing tags with a remote, and the difference between a tag and a GitHub Release.

24.3 Clean: remove untracked files

Sometimes a project fills with files Git does not track: build leftovers, scratch notes. `git clean` removes them. It is also one of the few Git commands that deletes files **for good**, because untracked files were never saved in a commit. Git therefore makes you say so.

```
$ printf 'scratch\n' > scratch.txt
$ mkdir tmp
$ printf 'x\n' > tmp/a.txt
$ git status --short
?? scratch.txt
?? tmp/
```

Recorded in Bash; *ch24-stash/expected-clean.bash.txt*.

Here are a scratch file and a folder with a file in it. Always start with a **dry run**, `-n`:

```
$ git clean -n
Would remove scratch.txt
$ git clean -n -d
Would remove scratch.txt
Would remove tmp/
```

Recorded in Bash; *ch24-stash/expected-clean.bash.txt*.

Without `-d`, untracked folders are left alone. The dry run names exactly what would go. When you are sure, `-f` (force) makes it real:

```
$ git clean -f
Removing scratch.txt
$ git status --short
?? tmp/
$ git clean -f -d
Removing tmp/
```

Recorded in Bash; *ch24-stash/expected-clean.bash.txt*.

```
$ git status --short
$ ls
menu.md
```

Recorded in Bash; *ch24-stash/expected-clean.bash.txt*.

CAUTION. `git clean -f` cannot be undone: the files are not in any commit, so nothing can bring them back. Run `git clean -n` first, read the list, and check that nothing important is on it. Files that `.gitignore` covers (Chapter 18) are skipped unless you add `-x`, which is a further reason to be careful.

24.4 Finding things

Three tools answer three different questions.

"Where does this text appear?" `git grep` searches the tracked files:

```
$ git grep "loaf"
menu.md:- White loaf: 2.80
$ git grep -n "Rolls"
menu.md:4:- Rolls (six): 3.00
$ git grep -c "e"
menu.md:3
```

Recorded in Bash; *ch24-stash/expected-search.bash.txt*.

`-n` adds line numbers and `-c` counts matching lines per file. You can also search an **old version**. Here, two commits back, the loaf cost 2.50:

```
$ git grep "loaf" HEAD~2
HEAD~2:menu.md:- White loaf: 2.50
```

Recorded in Bash; *ch24-stash/expected-search.bash.txt*.

"Who last changed each line?" `git blame` prints each line with the commit that last touched it:

```
$ git blame -s menu.md
^8a52ffe 1) # Sunrise Bakery menu
^8a52ffe 2)
9f10b436 3) - White loaf: 2.80
^8a52ffe 4) - Rolls (six): 3.00
81f772ee 5) - Coconut cake (slice): 4.00
$ git blame -s -L 3,3 menu.md
9f10b436 3) - White loaf: 2.80
```

Recorded in Bash; *ch24-stash/expected-search.bash.txt*.

Your Git may show a longer hash. With `git blame -s -L 3,3`, Git 2.43.0 and 2.55.0 print the hash as 8 characters (9f10b436) and Git 2.56.0 prints 7 (9f10b43). Hashes shortened to different lengths still name the same commit.

`-s` hides names and dates so the output is short. The hash on each line is the commit that last changed it. A leading `^` (as in `^8a52ffe`) marks a line from the very first commit. `-L 3,3` limits the output to line 3. Despite the name, blame is not about fault: it is how you find the commit that explains a line, and then `git show <hash>` tells you why.

"When did this text first appear (or disappear)?" `git log -S` finds commits that added or removed a given string:

```
$ git log -S"Coconut" --oneline
81f772e (HEAD -> main) Add coconut cake
```

Recorded in Bash; *ch24-stash/expected-search.bash.txt*.

24.5 Practice session: a tidy-up

Practice session. Goal: use the small tools on the Sunrise Bakery site. Time: about 30 minutes. You need: the sunrise-bakery repository with a few commits.

1. Edit a tracked file without committing, and create a new untracked file. Use `git stash -u`, check that the tree is clean, then `git stash pop`.
2. Tag the current commit `v0.1` (lightweight) and `v1.0` (annotated, with a message). List them and run `git show v1.0`.
3. Create a scratch file and a scratch folder. Run `git clean -n -d` first, read the list, then `git clean -f -d`.
4. Use `git grep` to find every place a price appears. Use `git blame -s` on `menu.html` (or your menu file) and pick one line; then `git show` its commit.
5. Use `git log -S"cake" --oneline` to find when the cake first appeared.

Expected result: each command behaves as in this chapter, and the final `git status` is clean.

Checkpoint questions: Which of these steps could lose work if you were careless? What did you do to make it safe?

Checkpoint

What You Learned

- `git stash` shelves changes to tracked files; `-u` includes untracked files; `pop` applies and removes.
- A tag is a fixed name for a commit; annotated tags carry a message and tagger.
- `git clean` deletes untracked files for good: run `-n` first.

- `git grep` searches content, `git blame` shows the commit for each line, `git log -S` finds when text changed.

New Vocabulary

- **Stash**: a shelf for uncommitted changes.
- **Tag**: a fixed name for one commit.
- **Lightweight tag**: a tag that is only a name.
- **Annotated tag**: a tag with a tagger, date and message.

Commands Learned

`git stash`, `git stash -u`, `git stash list`, `git stash pop`, `git stash drop`, `git tag`, `git tag -a`, `git tag -d`, `git tag -l`, `git clean -n`, `git clean -f`, `git clean -f -d`, `git grep`, `git blame -s`, `git log -S`.

Common Mistakes

1. **Assuming `git stash` also takes untracked files.**
2. **Treating the stash as permanent storage.**
3. **Running `git clean -f` without a dry run.**
4. **Thinking `git tag -d` deletes the commit.**
5. **Reading `git blame` as a way to assign fault.**

Practice

Do the exercises in the Exercises section at the end of this chapter, including the practice session (section 24.5).

Self-Test

1. Why did `ideas.txt` stay behind after the first `git stash`?
2. What is the difference between `git stash pop` and `git stash apply`?
3. Why does the book insist on `git clean -n` first?
4. What is the difference between a lightweight and an annotated tag?
5. Which command tells you when a string was first added?

Before Moving On

You are ready for Chapter 25 if you can:

- [] stash and restore work, with and without untracked files
- [] create, list, show and delete tags
- [] preview and perform a clean
- [] search, blame and pickaxe a repository

How this chapter was checked

Claim	Evidence class	Ledger
stash, stash -u, list, pop, empty-shelf message	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; identical on CI's Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-stash); the ledger row says which.	R1 81
Lightweight and annotated tags, show, -d, -l; clean -n/-f/-d	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-clean, git-tag); the ledger row says which.	R1 82
git grep, git blame -s, git log -S	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-blame, git-grep, git-log); the ledger row says which.	R1 83

Where this leads

Chapter 25 is about undoing things safely, where stash and clean appear again. Chapter 29 returns to tags for versioning, and Chapter 33 returns to why history that contains a secret is hard to clean.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use bakery-menu (rebuild it from Chapter 17 if needed).

Level 1 — Guided

1.1 Stash and pop. Edit menu.md without committing. Run `git stash`, check `git status --short` and `git stash list`, then `git stash pop`.

1.2 Tag a commit. Create a lightweight tag v0.1 and an annotated tag v1.0. Run `git log --oneline --decorate`.

Level 2 — Partially guided

2.1 Untracked files. Create ideas.txt, then run `git stash`. Is ideas.txt still there? Repeat with `git stash -u`.

2.2 Dry-run clean. Create a scratch file and a scratch folder. Run `git clean -n`, then `git clean -n -d`. Explain the difference.

Level 3 — Independent

3.1 Search. Use `git grep -n` to find the line with the rolls, then search an older commit for the price of the white loaf.

3.2 Blame. Use `git blame -s -L 3,3 menu.md` and then `git show` on the commit hash it prints. What did that commit change?

Level 4 — Professional scenario

4.1 Interrupted work. You are halfway through a menu edit on `main`. A colleague asks for a fix on another branch. Show the commands that keep your edit safe, switch, and return, without making a commit for the unfinished edit.

Level 5 — Troubleshooting

5.1 A learner stashed work two days ago and cannot remember what it was. Which commands show the entry and its content?

5.2 A learner ran `git clean -f` and lost an important untracked file. Can Git recover it? Why or why not?

5.3 A learner deleted a tag with `git tag -d v1.0` and fears the release commit is gone. Is it?

Part IV — Professional Git

Chapter 25 — Undoing Things Safely [Core]

In this chapter

- how to tell what you want to undo before you type anything
- `git restore`: discard an edit, or unstage a file
- `git reset`: move a branch back, in three strengths (`--soft`, `default`, `--hard`)
- `git revert`: undo a commit by adding a new one
- which of these can lose work, and which cannot

Before you start. Chapter 19, Chapter 17 and Chapter 20. The recordings use the bakery-menu repository and were run in Bash and zsh on Git 2.43.0, then re-run in CI on Git 2.55.0.

Everyone makes mistakes with Git. Almost all of them can be undone, but which command is right depends on where the mistake is. So the first skill is diagnosis.

25.1 First, find where the mistake is

Chapter 15 described three places where your work lives: the **working tree** (your files), the **index** (what you have staged) and the **repository** (the commits). An undo command works on one or more of them.

The mistake is...	You want to...	Use
an edit in a file, not yet staged	throw the edit away	<code>git restore <file></code>
a file staged by mistake	unstage it, keep the edit	<code>git restore --staged <file></code>
the last commit(s), not yet shared	move the branch back	<code>git reset</code>
a commit that is already shared	add a new commit that cancels it	<code>git revert</code>

Before any undo, run `git status` and `git log --oneline`. Read them. Then choose.

CAUTION. Some undo commands are safe and some destroy work. This chapter marks them. The rule: a command that discards uncommitted changes is permanent, because Git never saw those changes.

25.2 `git restore`: discard an edit

You changed the white loaf price to 9.99 by mistake:

```
$ printf '# Sunrise Bakery menu\n\n White loaf: 9.99\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n' > menu.md
$ git status --short
M menu.md
```

Recorded in Bash; *ch25-undo/expected-restore.bash.txt*.

git diff shows the change:

```
$ git diff
diff --git a/menu.md b/menu.md
index 97e3bea..d71ff95 100644
--- a/menu.md
+++ b/menu.md
@@ -1,5 +1,5 @@
 # Sunrise Bakery menu

-- White loaf: 2.80
+- White loaf: 9.99
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch25-undo/expected-restore.bash.txt*.

To bring the file back to how it was at the last commit:

```
$ git restore menu.md
$ git status --short
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch25-undo/expected-restore.bash.txt*.

New term: git restore. Restores files from the index or from a commit into your working tree, or (with `--staged`) into the index.

The file is back and `git status --short` is empty. The edit is **gone for good**: it was never committed, so Git kept no copy.

CAUTION. `git restore <file>` throws away uncommitted edits permanently. Run `git diff <file>` first if you are not sure that you want that.

25.3 git restore --staged: unstage

You staged a change (a tea line) and realised it does not belong in the next commit:

```
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.80\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n- Tea: 1.50\n' > menu.md
$ git add menu.md
$ git status --short
M  menu.md
```

Recorded in Bash; *ch25-undo/expected-restore.bash.txt*.

M (the first column) means the change is staged. To take it out of the index, without touching the file:

```
$ git restore --staged menu.md
$ git status --short
M  menu.md
```

Recorded in Bash; *ch25-undo/expected-restore.bash.txt*.

Now the status shows M (the second column): still modified, no longer staged. The edit is safe in the file. If you also want to discard the edit:

```
$ git restore menu.md
$ git status --short
```

Recorded in Bash; *ch25-undo/expected-restore.bash.txt*.

The two columns of `git status --short` are worth remembering: the **first** is the index, the **second** is the working tree.

25.4 Restoring from an older commit

`git restore` can also fetch a file from **any commit**. Here `--source=HEAD~2` takes `menu.md` from two commits back:

```
$ git restore --source=HEAD~2 menu.md
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.50
- Rolls (six): 3.00
```

Recorded in Bash; *ch25-undo/expected-restore.bash.txt*.

The file now shows the old content (2.50, only two lines). Git reports it as modified, as any change from the last commit would be:

```
$ git status --short
M  menu.md
$ git restore menu.md
$ git status --short
```

Recorded in Bash; *ch25-undo/expected-restore.bash.txt*.

The last `git restore menu.md` brings the file back to the latest commit. Nothing was committed by the experiment. This is a safe way to look at, or take back, an old version of a single file.

25.5 `git reset`: move the branch back

`git restore` deals with files. `git reset` moves **the branch** (and HEAD) to a different commit. It has three strengths. To show them, two commits are added:

```
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Add tea"
[main 60cb0bb] Add tea
 1 file changed, 1 insertion(+)
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -am "Add coffee"
[main bf6d066] Add coffee
 1 file changed, 1 insertion(+)
$ git log --oneline
bf6d066 (HEAD -> main) Add coffee
60cb0bb Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch25-undo/expected-reset.bash.txt*.

`--soft`: undo the commit, keep everything staged

```
$ git reset --soft HEAD~1
$ git status --short
M  menu.md
$ git log --oneline
60cb0bb (HEAD -> main) Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch25-undo/expected-reset.bash.txt*.

The commit "Add coffee" is gone from the log, but the change is still **staged** (M). It is as if you had run `git add` but not `git commit`. Use this to redo a commit, for instance to combine it with the next one.

```
$ git commit -m "Add coffee"
[main 1558157] Add coffee
 1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch25-undo/expected-reset.bash.txt*.

Default (--mixed): undo the commit, keep the changes unstaged

```
$ git reset HEAD~1
Unstaged changes after reset:
M      menu.md
$ git status --short
M menu.md
$ git log --oneline
60cb0bb (HEAD -> main) Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch25-undo/expected-reset.bash.txt*.

The commit is gone, and the change is still in the file (`M`), but no longer staged. Git prints `Unstaged changes after reset:` to list what it left in your files. (The wording of that message may differ in other Git versions.) This is the default: `git reset HEAD~1` with no option.

--hard: undo the commit and discard the change

```
$ git commit -am "Add coffee"
[main df83910] Add coffee
1 file changed, 1 insertion(+)
$ git reset --hard HEAD~1
HEAD is now at 60cb0bb Add tea
$ git status --short
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
- Tea: 1.50
```

Recorded in Bash; *ch25-undo/expected-reset.bash.txt*.

HEAD is now at 60cb0bb Add tea says the branch moved. The file no longer has the coffee line. `--hard` also reset your **working tree** to match.

New term: git reset. Moves the current branch to another commit. `--soft` leaves the index and working tree alone; the default also resets the index; `--hard` resets the index **and** the working tree.

Option	Branch moves	Index reset	Working tree reset	Can lose uncommitted work?
<code>--soft</code>	yes	no	no	no
(default)	yes	yes	no	no
<code>--hard</code>	yes	yes	yes	yes

After a `--hard` reset, the commit is not lost

The "Add coffee" commit was **committed**, so Git can still find it. The **reflog** is Git's private diary of where HEAD has been:

```
$ git reflog -4
60cb0bb (HEAD -> main) HEAD@{0}: reset: moving to HEAD~1
df83910 HEAD@{1}: commit: Add coffee
60cb0bb (HEAD -> main) HEAD@{2}: reset: moving to HEAD~1
1558157 HEAD@{3}: commit: Add coffee
$ git reset --hard HEAD@{1}
HEAD is now at df83910 Add coffee
```

Recorded in Bash; *ch25-undo/expected-reset.bash.txt*.

HEAD@{1} is where HEAD was one step ago: the commit df83910 Add coffee. Resetting to it brings the commit back:

```
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
- Tea: 1.50
- Coffee: 2.00
$ git log --oneline
df83910 (HEAD -> main) Add coffee
60cb0bb Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch25-undo/expected-reset.bash.txt*.

The coffee line and the commit are both back. This is the safety net for committed work. Chapter 26 teaches it fully.

But uncommitted work is different

```
$ printf -- '- Precious draft: 5.00\n' >> menu.md
$ git status --short
M menu.md
$ git reset --hard
HEAD is now at 81f772e Add coconut cake
$ git status --short
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch25-undo/expected-lost.bash.txt*.

The line "Precious draft" was never committed. `git reset --hard` discarded it, and the reflog has nothing about it:

```
$ git reflog -2
81f772e (HEAD -> main) HEAD@{0}: reset: moving to HEAD
81f772e (HEAD -> main) HEAD@{1}: commit: Add coconut cake
```

Recorded in Bash; *ch25-undo/expected-lost.bash.txt*.

Both entries describe commits and moves; neither mentions your edit. Because the edit was never even staged (never passed to `git add`), **there is no way to get the draft back with Git**.

Deep. Staging changes this. If the edit had been staged with `git add` before the reset, Git would already have stored its contents as an object, and `git fsck --lost-found` would list it as a dangling blob that you could read and save. Staging before a risky command is therefore a cheap safety net. This was tried once on Git 2.43.0 and is not part of the recorded sessions; Chapter 80 uses `git fsck --lost-found` for a lost commit.

CAUTION. `git reset --hard` is the most dangerous everyday command. Before you run it, run `git status` and `git stash` (Chapter 24) if there is anything you want to keep. Commits can be recovered from the reflog. Uncommitted changes cannot.

CAUTION. Do not reset commits that you have already **pushed** and that others may have pulled. Moving a shared branch back rewrites history that other people already have, and causes the problems described in Chapter 23. For shared commits, use `git revert`.

25.6 `git revert`: undo by adding a commit

`git revert` does not remove history. It creates a **new commit that does the opposite** of an earlier one. Someone committed a bad line:

```
$ printf -- '- Mystery pie: 0.01\n' >> menu.md
$ git commit -am "Add a mystery pie"
[main 1080a4c] Add a mystery pie
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch25-undo/expected-revert.bash.txt*.

Revert it:

```
$ git revert --no-edit HEAD
[main 3075407] Revert "Add a mystery pie"
Date: Mon Jan 5 09:10:00 2026 +0000
1 file changed, 1 deletion(-)
```

Recorded in Bash; *ch25-undo/expected-revert.bash.txt*.

--no-edit accepts Git's suggested message, Revert "Add a mystery pie". Without it, Git opens your editor. The history now has both commits:

```
$ git log --oneline
3075407 (HEAD -> main) Revert "Add a mystery pie"
1080a4c Add a mystery pie
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch25-undo/expected-revert.bash.txt*.

The original commit is still there, followed by its reversal. The file is back to normal:

```
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch25-undo/expected-revert.bash.txt*.

```
$ git show --stat --oneline HEAD
3075407 (HEAD -> main) Revert "Add a mystery pie"
menu.md | 1 -
1 file changed, 1 deletion(-)
```

Recorded in Bash; *ch25-undo/expected-revert.bash.txt*.

The revert commit removes one line, exactly what the original added.

New term: git revert. Creates a new commit that undoes the changes of an earlier commit, leaving history intact.

Because it only adds to history, `git revert` is the safe choice for anything already shared. Anyone who pulls receives the correction like any other commit.

Reverting a merge commit

A merge commit has **two parents** (Chapter 21), so "undo this commit" is ambiguous: undo the changes relative to which parent? Git refuses to guess. Here a branch `add-tea` was merged into `main` with a merge commit, and the first attempt to revert it fails:

```
$ git switch -q -c add-tea
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -qam "Add tea"
$ git switch -q main
$ printf 'Open Monday to Saturday.\n' > hours.md
$ git add hours.md
```

```
$ git commit -qm "Add opening hours"
$ git merge -q --no-ff --no-edit add-tea
$ git revert --no-edit HEAD
error: commit 0f412999532a95752723706687e3813749d8e86b is a merge but no -m option was
given.
fatal: revert failed
```

Recorded in Bash; *ch25-undo/expected-revert-merge.bash.txt*.

is a merge but no `-m` option was given. The option `-m` (`--mainline`) names the parent that counts as the **mainline**, numbered from 1. For a merge made on *main*, parent 1 is the side you were on (main), so `-m 1` means "go back to how main was before the merge":

```
$ git revert --no-edit -m 1 HEAD
[main f999f2d] Revert "Merge branch 'add-tea'"
Date: Mon Jan 5 09:16:00 2026 +0000
1 file changed, 1 deletion(-)
```

Recorded in Bash; *ch25-undo/expected-revert-merge.bash.txt*.

The result:

```
$ git log --oneline
f999f2d (HEAD -> main) Revert "Merge branch 'add-tea'"
0f41299 Merge branch 'add-tea'
1e5eef8 Add opening hours
73ad7ed (add-tea) Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch25-undo/expected-revert-merge.bash.txt*.

The tea line is gone from `menu.md`, and the history shows both the merge and its reversal.

CAUTION. Git's documentation warns that reverting a merge "declares that you will never want the tree changes brought in by the merge": later merges of the same branch will only bring in commits that are not ancestors of the reverted merge. That may or may not be what you want, and it is a good reason to think before you revert a merge.

Reverting an empty commit (a commit that changes nothing) is not covered here. The official `git revert` documentation says nothing about it, so this book does not teach it.

25.7 Reset or revert?

	git reset	git revert
What happens to history	commits are removed from the branch	a new commit is added
Safe after pushing?	no	yes
Leaves a record of the mistake?	no	yes
Best for	local commits you have not shared	shared commits

25.8 An undo checklist

1. **Look:** `git status`, `git log --oneline`.
2. **Decide where the mistake is:** working tree, index, or commits.
3. **Prefer the safe option:** `restore --staged` and `revert` lose nothing.
4. **Before a destructive one** (`restore <file>`, `reset --hard`): `git diff`, and stash anything you want.
5. **Never reset shared history.**
6. **If it went wrong:** stop, run `git reflog` (Chapter 26).

Checkpoint

What You Learned

- Diagnose first: where is the mistake (files, index or commits)?
- `git restore <file>` discards an uncommitted edit permanently; `git restore --staged <file>` unstages and keeps it.
- `git restore --source=<commit> <file>` takes a file from an older commit.
- `git reset` moves the branch: `--soft` keeps the staged change, the default keeps the file change, `--hard` discards it.
- After a reset, committed work is recoverable via the `reflog`; uncommitted work is not.
- `git revert` adds a commit that cancels an earlier one, and is the safe choice for shared history.

New Vocabulary

- **git restore:** bring file content from the index or a commit into the working tree or index.
- **git reset:** move the current branch to another commit, with three strengths.
- **git revert:** add a commit that undoes an earlier commit.

Commands Learned

`git restore`, `git restore --staged`, `git restore --source`, `git reset --soft`, `git reset`, `git reset --hard`, `git reflog`, `git revert --no-edit`.

Common Mistakes

1. **Running `git reset --hard` with uncommitted work.**
2. **Resetting a branch that has been pushed.**
3. **Using `git restore <file>` before checking `git diff`.**
4. **Forgetting which column of `git status --short` is the index.**
5. **Panicking after a reset instead of reading the reflog.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Which command unstages a file without losing the edit?
2. What is the difference between `reset --soft` and `reset --hard`?
3. Why can the "Add coffee" commit be recovered but the "Precious draft" cannot?
4. Which of `reset` and `revert` is safe after pushing, and why?
5. What does `git restore --source=HEAD~2 menu.md` do?

Before Moving On

You are ready for Chapter 26 if you can:

- [] undo an unstaged edit, and a staged one
- [] explain the three `git reset` options
- [] revert a commit
- [] say which undo commands can lose work

How this chapter was checked

Claim	Evidence class	Ledger
<code>restore</code> , <code>restore --staged</code> , <code>restore --source</code>	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-restore</code>); the ledger row says which.	R1 84
<code>reset --soft/--mixed/--hard</code> , reflog recovery, loss of uncommitted work	Locally tested (as above); wording of the Unstaged changes after <code>reset</code> message may vary. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-reset</code>); the ledger row says which.	R1 85
<code>revert</code> of an ordinary commit, and of a merge commit with <code>-m 1</code>	Locally tested (as above), and checked against the <code>git revert</code> documentation (Git 2.56.0) for <code>-m</code> and its warning	R1 86
A staged edit survives <code>reset --hard</code> as a dangling blob (<code>git fsck --lost-found</code>); an unstaged edit does not	Locally tested once on Git 2.43.0 (not a recorded session); the <code>git fsck</code> manual (Git 2.56.0) confirms that dangling objects are printed and that <code>--lost-found</code> writes them out	R3 55

Claim	Evidence class	Le dg er
revert of an empty commit	The documentation is silent; deliberately not taught	R1 86

Where this leads

Chapter 26 turns the reflog into a full recovery toolkit. Chapter 27 rewrites history in a more controlled way, and Chapter 33 shows why removing a committed secret needs more than a revert.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use bakery-menu (rebuild it from Chapter 17 if needed). Run `git status` before every command.

Level 1 — Guided

1.1 Discard an edit. Change a price in `menu.md` without committing. Look at it with `git diff`, then discard it with `git restore menu.md`. How do you know it worked?

1.2 Unstage. Edit the file, `git add` it, then unstage it with `git restore --staged`. Read `git status --short` at each step.

Level 2 — Partially guided

2.1 Take back an old version. Use `git restore --source=HEAD~2 menu.md`. What does the file contain? What does `git status --short` say? Undo the experiment.

2.2 Soft reset. Make a commit, then `git reset --soft HEAD~1`. Where is the change now? Commit it again with a better message.

Level 3 — Independent

3.1 The three resets. Make two commits. Run `git reset --soft HEAD~1`, recommit; `git reset HEAD~1`, recommit; `git reset --hard HEAD~1`. Fill in a table: what happened to the log, the index and the file each time?

3.2 Recover. After the hard reset in 3.1, use `git reflog` to find the lost commit and `git reset --hard HEAD@{1}` to bring it back.

Level 4 — Professional scenario

4.1 A bad commit on a shared branch. A colleague already pulled your commit "Add a mystery pie". Remove its effect without rewriting history. Show the log and explain why reset would be wrong.

Level 5 — Troubleshooting

5.1 A learner edited a file for an hour, then ran `git reset --hard "to tidy up"`. Can the edit be recovered with Git? Why not, and what should they have done first?

5.2 A learner ran `git restore menu.md` and the changes disappeared. They ask you to recover them. What do you say?

5.3 A learner ran `git reset --hard HEAD~3` and now wants the three commits back. What is the first command you tell them to run?

Chapter 26 — Recovery with the Reflog [Core]

In this chapter

- what the reflog is, and why it exists
- reading `git reflog`
- recovering a deleted branch
- what a detached HEAD is, and how to save work made in one
- a decision tree for "I think I lost my work"

Before you start. Chapter 25 (especially `git reset --hard`) and Chapter 20. The recordings use the bakery-menu repository and were run in Bash and zsh on Git 2.43.0, then re-run in CI on Git 2.55.0.

The most useful thing to know about losing work in Git is that you usually have not. Git rarely deletes a commit when you move away from it. It only stops pointing at it. The reflog remembers where you were.

26.1 What the reflog is

New term: reflog. A private, local log of where HEAD (and each branch) has pointed, kept by Git for you. Every commit, switch, reset, merge and rebase adds a line.

The ordinary log (Chapter 17) shows the history of commits reachable from a branch. The reflog shows the history of your movements. It is different in three ways:

- It is **local**: it is not pushed, and a fresh clone starts with a single clone: entry and none of the original repository's history.
- It is **temporary**: Git eventually removes old entries.
- It records **every move**, including moves that left commits unreachable.

```
$ git reflog
81f772e (HEAD -> main) HEAD@{0}: commit: Add coconut cake
9f10b43 HEAD@{1}: commit: Raise the price of the white loaf
8a52ffe HEAD@{2}: commit (initial): Add the menu
```

Recorded in Bash; *ch26-reflog/expected-deleted-branch.bash.txt*.

Read one line: `81f772e HEAD@{0}: commit: Add coconut cake`.

Part	Meaning
81f772e	the commit HEAD pointed to after the event
HEAD@{0}	how many moves ago (0 is now, 1 is one move ago)
commit: Add coconut cake	what happened

The list is newest first. HEAD@{1} is where you were one step earlier.

Checked against the documentation (Git 2.56.0, R189). Two settings control how long the reflog keeps entries: `gc.reflogExpire`, which "defaults to 90 days", applies to entries whose commits are still reachable from a branch; `gc.reflogExpireUnreachable`, "defaults to 30 days", applies to entries whose commits are **not** reachable from the current tip (as after a `git reset --hard`, or a deleted branch). Separately, `git gc` prunes unreachable objects only after a grace period, `gc.pruneExpire`, which is **two weeks** by default. The documentation explains why the unreachable period is shorter: such entries are "generally created as a result of using `git commit --amend` or `git rebase`" and are the commits from before the amend or rebase, which most users want to expire sooner. In practice: treat a lost commit as recoverable for **about a month**, but not for anything old, and remember that these are defaults that a system or a team can change (Chapter 14).

26.2 Recovering a deleted branch

You start an experiment on a branch and commit some work:

```
$ git switch -c experiment
Switched to a new branch 'experiment'
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Add tea"
[experiment 1493bc1] Add tea
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch26-reflog/expected-deleted-branch.bash.txt*.

Then you switch back and delete the branch, believing that the experiment is finished:

```
$ git switch main
Switched to branch 'main'
$ git branch -D experiment
Deleted branch experiment (was 1493bc1).
$ git branch
* main
```

Recorded in Bash; *ch26-reflog/expected-deleted-branch.bash.txt*.

Git deleted it with `-D` (force, Chapter 20) and told you the last commit, 1493bc1. The branch name is gone. The commit is not. To find it again, ask the reflog:

```
$ git reflog -4
81f772e (HEAD -> main) HEAD@{0}: checkout: moving from experiment to main
1493bc1 HEAD@{1}: commit: Add tea
81f772e (HEAD -> main) HEAD@{2}: checkout: moving from main to experiment
81f772e (HEAD -> main) HEAD@{3}: commit: Add coconut cake
```

Recorded in Bash; *ch26-reflog/expected-deleted-branch.bash.txt*.

The line `1493bc1 HEAD@{1}: commit: Add tea` is the commit you made on experiment. Now give it a branch name again:

```
$ git branch experiment-restored HEAD@{1}
$ git branch
  experiment-restored
* main
```

Recorded in Bash; *ch26-reflog/expected-deleted-branch.bash.txt*.

`git branch experiment-restored HEAD@{1}` creates a branch at the commit that `HEAD@{1}` names. The log shows the recovered work:

```
$ git log --oneline experiment-restored
1493bc1 (experiment-restored) Add tea
81f772e (HEAD -> main) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch26-reflog/expected-deleted-branch.bash.txt*.

The commit "Add tea" is back, on a new branch. **Notice what saved the work:** the hash printed by `git branch -D (1493bc1)` would also have worked, with `git branch experiment-restored 1493bc1`. Reading the output of destructive commands is a recovery technique.

26.3 Detached HEAD

Normally `HEAD` points to a **branch**, and the branch points to a commit. Sometimes you check out a commit directly: to look at old code, or by accident. Then `HEAD` points straight at the commit.

(Chapter 20 named this state; here it matters for recovery.)

```
$ git switch --detach HEAD~1
HEAD is now at 9f10b43 Raise the price of the white loaf
$ git status
HEAD detached at 9f10b43
nothing to commit, working tree clean
```

Recorded in Bash; *ch26-reflog/expected-detached.bash.txt*.

`HEAD detached at 9f10b43` says it. This is a legitimate place to look around. The danger is that **commits made here belong to no branch**:

```
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Add tea on a detached HEAD"
[detached HEAD fae405d] Add tea on a detached HEAD
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch26-reflog/expected-detached.bash.txt*.

The commit has been made, and Git says `[detached HEAD fae405d]`. Now you leave:

```
$ git switch main
Warning: you are leaving 1 commit behind, not connected to
any of your branches:

    fae405d Add tea on a detached HEAD

If you want to keep it by creating a new branch, this may be a good time
to do so with:

    git branch <new-branch-name> fae405d

Switched to branch 'main'
```

Recorded in Bash; *ch26-reflog/expected-detached.bash.txt*.

Read the warning. Git tells you that one commit is left behind, not connected to any of your branches, and it even prints the command to save it: `git branch <new-branch-name> fae405d`. If you close the window without reading it, the commit is no longer visible:

```
$ git log --oneline --all
81f772e (HEAD -> main) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch26-reflog/expected-detached.bash.txt*.

`git log --all` shows only the three original commits; the tea commit is gone from view. But the reflog remembers:

```
$ git reflog -3
81f772e (HEAD -> main) HEAD@{0}: checkout: moving from
fae405dddf099cd51e07afda8545230358e7aa2e to main
fae405d HEAD@{1}: commit: Add tea on a detached HEAD
9f10b43 HEAD@{2}: checkout: moving from main to HEAD~1
```

Recorded in Bash; *ch26-reflog/expected-detached.bash.txt*.

The line `fae405d HEAD@{1}: commit: Add tea on a detached HEAD` gives you the hash. Save it on a branch:

```
$ git branch rescued fae405d
$ git branch
* main
  rescued
$ git log --oneline rescued
fae405d (rescued) Add tea on a detached HEAD
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch26-reflog/expected-detached.bash.txt*.

The commit is safe on rescued.

The rule. If Git says `HEAD detached` and you want to keep any work: create a branch first (`git switch -c <name>`). If you only looked around, `git switch main` is all you need.

26.4 A bad reset

Chapter 25 showed the same technique for `git reset --hard`: `git reflog` finds the commit that `HEAD` pointed to before the reset, and `git reset --hard HEAD@{1}` (or the hash) brings it back. That example is the one to reread if you have just run a reset you regret.

26.5 What the reflog cannot save

- **Uncommitted changes.** Work that was never committed was never recorded (Chapter 25, section on `--hard`).
- **Old work**, once entries have expired and garbage collection has run (see the defaults in section 26.1).
- **Work in a different clone.** The reflog is per repository.
- **A repository you deleted.**

The habit that avoids all of these: **commit early, on a branch**. A commit you made an hour ago is almost always recoverable. An edit you never committed is not.

26.6 "I think I lost my work": a decision tree

1. **Stop.** Do not run more commands that move things around, and do not delete the folder.
 2. **git status.** Is the work sitting there, uncommitted? Then it is not lost.
 3. **git branch --all and git stash list.** Is it on another branch, or in a stash?
 4. **git log --oneline --all.** Is it in a commit you cannot see from the current branch?
 5. **git reflog.** Look for the last commit that had your work. Note the hash.
 6. **Recover it:** `git branch <name> <hash>` (safe, adds a branch), or `git reset --hard <hash>` (moves the current branch; check `git status` first).
 7. **If it was never committed** and is not in a stash, Git cannot help. Look in your editor's undo history and in backups (Chapter 3).
-

Checkpoint

What You Learned

- The reflog is a local, temporary record of where `HEAD` has been; `HEAD@{n}` names a position in it.
- A deleted branch's commits survive; find them in the reflog (or in the hash printed by `git branch -D`) and give them a new branch.
- A detached `HEAD` points at a commit rather than a branch; commits made there need a branch to be kept.
- Git warns when you leave commits behind, and prints the command to save them.
- The reflog cannot recover uncommitted work, or old entries that have expired.

New Vocabulary

- **Reflog:** Git's local log of where HEAD and branches have pointed.

Commands Learned

`git reflog`, `git reflog -n, HEAD@{n}`, `git branch <name> <commit>`, `git switch --detach`, `git log --all`.

Common Mistakes

1. **Committing on a detached HEAD** without creating a branch.
2. **Ignoring the "leaving 1 commit behind" warning.**
3. **Deleting a branch with -D without noting the hash.**
4. **Assuming the reflog is permanent, or shared.**
5. **Running more commands before looking at the reflog.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What does `HEAD@{1}` mean?
2. After `git branch -D experiment`, how can you find the deleted commit?
3. What does `HEAD detached at 9f10b43` tell you?
4. What did Git print when you left the detached HEAD with a commit, and why does it matter?
5. Name two things the reflog cannot recover.

Before Moving On

You are ready for Chapter 27 if you can:

- [] read a reflog line
- [] recover a deleted branch
- [] rescue a commit made on a detached HEAD
- [] work through the "lost work" decision tree

How this chapter was checked

Claim	Evidence class	Ledger
Reflog lines and <code>HEAD@{n}</code> ; deleted-branch recovery	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-reflog</code>); the ledger row says which.	R187

Claim	Evidence class	Ledger
Detached HEAD, the leaving-commits warning, rescue with <code>git branch</code>	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-switch</code> , <code>glossary-content</code>); the ledger row says which.	R188
Reflog expiry defaults (90 days, 30 days) and the <code>gc.pruneExpire</code> grace period (two weeks)	Checked against the Git 2.56.0 documentation (<code>config/gc</code>); not measured by test	R189

Where this leads

Chapter 27 rewrites history on purpose; the reflog is what makes that safe to try. Chapter 28 uses `git switch --detach` for bisecting, and Chapter 33 explains why a leaked secret in old history cannot be fixed by these tools alone.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use `bakery-menu` (rebuild it from Chapter 17 if needed).

Level 1 — Guided

1.1 Read the reflog. Run `git reflog`. Which entry is `HEAD@{0}`? What does the text after the second colon describe?

1.2 Make and lose a branch. Create a branch `experiment`, commit a change on it, switch to `main` and delete the branch with `git branch -D experiment`. Write down the hash Git prints.

Level 2 — Partially guided

2.1 Recover it. Use the hash from 1.2 to create `experiment-restored`. Check the log.

2.2 Recover it without the hash. Repeat 1.2, but this time ignore the printed hash. Find the commit with `git reflog` and recover it with `HEAD@{n}`.

Level 3 — Independent

3.1 Detached HEAD. Run `git switch --detach HEAD~1`, make a commit, and switch back to `main`. Read the warning in full. What does it tell you to do?

3.2 Rescue it. Use `git reflog` to find the commit from 3.1 and save it on a new branch.

Level 4 — Professional scenario

4.1 Bad reset. You ran `git reset --hard HEAD~2` and realised that you needed those commits. Write the exact commands that recover them, and say what you would check before running the last one.

Level 5 — Troubleshooting

5.1 A learner cloned a repository yesterday and cannot find a colleague's deleted branch in their reflog. Why not?

5.2 A learner edited a file for an hour and never committed. They ask the reflog for it. What do you tell them?

5.3 A learner says: "Git says HEAD detached and I have made five commits, but I want to keep them." What is the safest single command, and what should it be given as a name and target?

Chapter 27 — Rebase [Deep]

In this chapter

- what rebase does, and how it differs from merge
- `git rebase <branch>`: replay your commits on a new base
- rebase conflicts, and `--abort` and `--continue`
- `git rebase -i`: combining commits
- the golden rule: never rebase commits that others already have

Deep. This chapter is marked **[Deep]**. You can skip it on a first read and return after Chapter 34. Merge (Chapter 21) is enough to work with Git.

Before you start. Chapter 21, Chapter 22, Chapter 26 and Chapter 23. The recordings use bakery-menu and were run in Bash and zsh on Git 2.43.0, then re-run in CI on Git 2.55.0.

27.1 The problem rebase solves

Chapter 21 joined two diverged branches with a **merge commit**. That is honest, and it leaves a history with many small merges. Some teams prefer a **straight line**, where the work of a branch appears after the work that was already on `main`. **Rebase** does that.

New term: rebase. Take the commits of one branch and replay them, one by one, on top of another commit. The replayed commits are **new commits**, with new hashes.

27.2 Rebase a branch onto `main`

The same starting situation as the three-way merge: `main` got "Add opening hours", `add-tea` got "Add tea". They have diverged:

```
$ git switch -c add-tea
Switched to a new branch 'add-tea'
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Add tea"
[add-tea 73ad7ed] Add tea
1 file changed, 1 insertion(+)
$ git switch main
Switched to branch 'main'
$ printf 'Open Monday to Saturday.\n' > hours.md
$ git add hours.md
$ git commit -m "Add opening hours"
[main 1e5eef8] Add opening hours
1 file changed, 1 insertion(+)
create mode 100644 hours.md
$ git log --oneline --all --graph
* 1e5eef8 (HEAD -> main) Add opening hours
```

```
| * 73ad7ed (add-tea) Add tea
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch27-rebase/expected-rebase.bash.txt*.

Now, standing on add-tea, rebase it onto main:

```
$ git switch add-tea
Switched to branch 'add-tea'
$ git rebase main
Successfully rebased and updated refs/heads/add-tea.
```

Recorded in Bash; *ch27-rebase/expected-rebase.bash.txt*.

Read what happened. Git took the commit "Add tea", set it aside, moved to the tip of main, and applied it there again. The graph is now a straight line:

```
$ git log --oneline --all --graph
* 2cc423d (HEAD -> add-tea) Add tea
* 1e5eef8 (main) Add opening hours
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch27-rebase/expected-rebase.bash.txt*.

Look at the hash. Before: 73ad7ed Add tea. After: 2cc423d Add tea. **The commit was not moved; it was copied.** The content of the change is the same, but its parent is different, so it is a different commit, with a different hash. The old commit still exists (Chapter 26), but no branch points to it.

Now main can absorb the branch with a plain fast-forward (Chapter 21):

```
$ git switch main
Switched to branch 'main'
$ git merge add-tea
Updating 1e5eef8..2cc423d
Fast-forward
 menu.md | 1 +
 1 file changed, 1 insertion(+)
$ git log --oneline --graph
* 2cc423d (HEAD -> main, add-tea) Add tea
* 1e5eef8 Add opening hours
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch27-rebase/expected-rebase.bash.txt*.

There is no merge commit: the history reads as if the tea had been added after the opening hours.

Merge or rebase? They produce the same content. Merge keeps the true shape of the work, including the fork; rebase gives a tidy line but rewrites the branch's commits. Chapter 34 compares the styles.

27.3 When rebase hits a conflict

Both branches changed the same line, as in Chapter 22:

```
$ git switch -c raise-bread
Switched to a new branch 'raise-bread'
$ printf '# Sunrise Bakery menu\n\n- White loaf: 3.00\n- Rolls (six): 3.00\n- Coconut cake
(slice): 4.00\n' > menu.md
$ git commit -am "Raise the white loaf to 3.00"
[raise-bread 73e14f7] Raise the white loaf to 3.00
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git switch main
Switched to branch 'main'
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.90\n- Rolls (six): 3.00\n- Coconut cake
(slice): 4.00\n' > menu.md
$ git commit -am "Raise the white loaf to 2.90"
[main 4afeda6] Raise the white loaf to 2.90
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git switch raise-bread
Switched to branch 'raise-bread'
```

Recorded in Bash; *ch27-rebase/expected-rebase-conflict.bash.txt*.

Rebase applies your commits **one at a time**, and each may conflict:

```
$ git rebase main
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
error: could not apply 73e14f7... Raise the white loaf to 3.00
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
Could not apply 73e14f7... Raise the white loaf to 3.00
```

Recorded in Bash; *ch27-rebase/expected-rebase-conflict.bash.txt*.

Git stops in the middle and says which commit it could not apply, and it lists the ways out. (On newer Git versions, 2.55.0 was tested, the output has one more hint line about *advice.mergeConflict*, and the commit subject is shown with a # before it, as in *Could not apply 73e14f7... # Raise the white loaf to 3.00*. The meaning is the same.) Ask for the state:

```
$ git status
interactive rebase in progress; onto 4afeda6
Last command done (1 command done):
  pick 73e14f7 Raise the white loaf to 3.00
No commands remaining.
You are currently rebasing branch 'raise-bread' on '4afeda6'.
(fix conflicts and then run "git rebase --continue")
```

```

(use "git rebase --skip" to skip this patch)
(use "git rebase --abort" to check out the original branch)

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   menu.md

no changes added to commit (use "git add" and/or "git commit -a")
$ git diff --name-only --diff-filter=U
menu.md

```

Recorded in Bash; *ch27-rebase/expected-rebase-conflict.bash.txt*.

Two details differ from a merge conflict. The status says interactive rebase in progress (even though this rebase was not interactive; the name is Git's), and it names the commit being replayed. And, confusingly, which side is which is **reversed**: the "current" side (HEAD) is main (the base you are rebasing onto), and the side being applied is your commit.

You can walk away:

```

$ git rebase --abort
$ git status
On branch raise-bread
nothing to commit, working tree clean
$ git log --oneline --all --graph

```

Recorded in Bash; *ch27-rebase/expected-rebase-conflict.bash.txt*.

`git rebase --abort` puts the branch back as it was before you started:

```

* 4afeda6 (main) Raise the white loaf to 2.90
| * 73e14f7 (HEAD -> raise-bread) Raise the white loaf to 3.00
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu

```

Recorded in Bash; *ch27-rebase/expected-rebase-conflict.bash.txt*.

Or you can resolve it. Start again, and this time fix the file, stage it, and continue:

```

$ git rebase main
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
error: could not apply 73e14f7... Raise the white loaf to 3.00
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
Could not apply 73e14f7... Raise the white loaf to 3.00
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.95\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n' > menu.md
$ git add menu.md
$ git rebase --continue

```

Recorded in Bash; *ch27-rebase/expected-rebase-conflict.bash.txt*.

(hint: Waiting for your editor to close the file... appears because Git opens an editor for the commit message; the recording's editor closes at once. In your own terminal you would see your editor open, and you would save and close it.)

```
hint: Waiting for your editor to close the file...
[detached HEAD 608311e] Raise the white loaf to 3.00
 1 file changed, 1 insertion(+), 1 deletion(-)
Successfully rebased and updated refs/heads/raise-bread.
$ git log --oneline --all --graph
* 608311e (HEAD -> raise-bread) Raise the white loaf to 3.00
* 4afeda6 (main) Raise the white loaf to 2.90
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch27-rebase/expected-rebase-conflict.bash.txt*.

The result is a straight line again: "Raise the white loaf to 3.00" now sits on top of main, and its content is the resolution you chose (2.95).

The rebase checklist for conflicts:

1. Read `git status`.
2. Edit the file; remove the markers (Chapter 22).
3. `git add <file>`.
4. `git rebase --continue`.
5. Repeat for each commit that conflicts, or `git rebase --abort` to stop.

27.4 Interactive rebase: combining commits

`git rebase -i` (interactive) lets you edit a list of commits before they are replayed: keep, combine, reword, reorder or drop. A common use is to tidy several small commits into one before sharing them.

In a rebase, to **squash** a commit is to combine it into the one before it. (Chapter 21 used the word for `git merge --squash`; the effect is similar, the mechanism different.)

Three small commits were made:

```
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Add tea"
[main 60cb0bb] Add tea
 1 file changed, 1 insertion(+)
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -am "Add coffee"
[main bf6d066] Add coffee
 1 file changed, 1 insertion(+)
$ printf -- '- Juice: 2.20\n' >> menu.md
$ git commit -am "Add juice"
[main bde8e38] Add juice
 1 file changed, 1 insertion(+)
$ git log --oneline
```

```
bde8e38 (HEAD -> main) Add juice
bf6d066 Add coffee
60cb0bb Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch27-rebase/expected-squash.bash.txt*.

Normally, `git rebase -i HEAD~3` opens your editor with a list such as `pick <hash> Add tea`, one line per commit. You change `pick` to `squash` on the lines that should be folded into the first. The recording cannot open an editor, so it does the same edit with a program (`sed`), through the variable `GIT_SEQUENCE_EDITOR`. In your own work you edit the list by hand.

```
$ GIT_SEQUENCE_EDITOR="sed -i -e '2s/^pick/squash/' -e '3s/^pick/squash/'" git rebase -i HEAD~3
hint: Waiting for your editor to close the file...
hint: Waiting for your editor to close the file...
[detached HEAD 69d6e2b] Add tea
Date: Mon Jan 5 09:09:00 2026 +0000
1 file changed, 3 insertions(+)
Successfully rebased and updated refs/heads/main.
```

Recorded in Bash; *ch27-rebase/expected-squash.bash.txt*.

The recorded run changed the second and third lines from `pick` to `squash`. Git combined all three commits into one. It also gave you a chance to edit the combined message (the editor was closed at once here):

```
$ git log --oneline
69d6e2b (HEAD -> main) Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
$ git log -1 --format=%B
Add tea

Add coffee

Add juice
```

Recorded in Bash; *ch27-rebase/expected-squash.bash.txt*.

```
$ git show --stat --oneline HEAD
69d6e2b (HEAD -> main) Add tea
 menu.md | 3 +++
1 file changed, 3 insertions(+)
```

Recorded in Bash; *ch27-rebase/expected-squash.bash.txt*.

The three commits are now one, and the message shows all three original messages.

The other actions

The list of commands in the editor is longer than pick and squash. According to Git's documentation (checked against Git 2.56.0):

Command	What it does
pick	use the commit as it is
reword	use the commit, but stop to let you edit its message
edit	stop after applying the commit, so that you can change the files or the message, amend, and continue
squash	fold the commit into the one before it, and combine the messages
fixup	fold the commit into the one before it, and discard its message
drop	remove the commit (or delete its line)
break	stop at this point, without applying a commit

You can also **reorder** the commits by moving lines. The recordings below drive the todo list with sed (through GIT_SEQUENCE_EDITOR) instead of an editor, so that they can run unattended. Three small commits touch three different files: tea, coffee and juice:

```
$ cd bakery-menu
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -qm "Add tea"
$ printf 'Coffee: 2.00\n' > coffee.md
$ git add coffee.md
$ git commit -qm "Add coffee"
$ printf 'Juice: 2.20\n' > juice.md
$ git add juice.md
$ git commit -qm "Add juice"
$ git log --oneline -3
b68f4e3 (HEAD -> main) Add juice
28638f5 Add coffee
60cb0bb Add tea
```

Recorded in Bash; *ch27-rebase/expected-interactive-actions.bash.txt*.

Drop the second commit (coffee): change pick to drop on line 2:

```
$ GIT_SEQUENCE_EDITOR="sed -i '2s/^pick/drop/'" git rebase -i HEAD~3
hint: Waiting for your editor to close the file...
Successfully rebased and updated refs/heads/main.
$ git log --oneline -3
ecea039 (HEAD -> main) Add juice
60cb0bb Add tea
81f772e Add coconut cake
```

Recorded in Bash; *ch27-rebase/expected-interactive-actions.bash.txt*.

Coffee is gone from the history (and from the files). The other two commits were replayed with new hashes. **Reorder** the two that remain: move the first line to the end:

```
$ GIT_SEQUENCE_EDITOR="sed -i '1{h;d};\;$G'" git rebase -i HEAD~2
hint: Waiting for your editor to close the file...
Successfully rebased and updated refs/heads/main.
$ git log --oneline -3
f80220e (HEAD -> main) Add tea
41c5eb6 Add juice
81f772e Add coconut cake
```

Recorded in Bash; *ch27-rebase/expected-interactive-actions.bash.txt*.

Tea and juice have swapped places. **Reword** the oldest commit: the recording also gives Git a message-editing program through `GIT_EDITOR`, which replaces the message:

```
$ GIT_SEQUENCE_EDITOR="sed -i '1s/^pick/reword/'" GIT_EDITOR="sed -i '1s/.*/Add juice to
the drinks menu/'" git rebase -i HEAD~2
hint: Waiting for your editor to close the file...
hint: Waiting for your editor to close the file...
[detached HEAD 4cf07c0] Add juice to the drinks menu
Date: Mon Jan 5 09:13:00 2026 +0000
1 file changed, 1 insertion(+)
create mode 100644 juice.md
Successfully rebased and updated refs/heads/main.
$ git log --oneline -3
bd7866c (HEAD -> main) Add tea
4cf07c0 Add juice to the drinks menu
81f772e Add coconut cake
```

Recorded in Bash; *ch27-rebase/expected-interactive-actions.bash.txt*.

The message changed, and, again, so did every hash from that commit onward.

CAUTION. Dropping a commit removes its changes, and later commits may depend on them. Here coffee and juice touched **different files**, so the drop was clean. When the later commits build on the dropped one, the replay **conflicts**. Three commits appended to the same file, and the middle one dropped:

```
$ GIT_SEQUENCE_EDITOR="sed -i '2s/^pick/drop/'" git rebase -i HEAD~3
hint: Waiting for your editor to close the file...
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
error: could not apply bde8e38... Add juice
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
Could not apply bde8e38... Add juice
```

Recorded in Bash; *ch27-rebase/expected-drop-conflict.bash.txt*.

Your Git may word this differently. Git 2.55.0 and 2.56.0 (recorded and re-run in CI) print one more hint line, Disable this message with "git config set advice.mergeConflict false", and name the dropped commit as # Add juice in the Could not apply line. The meaning is the same.

```
$ git status --short
UU menu.md
$ git rebase --abort
$ git log --oneline -3
bde8e38 (HEAD -> main) Add juice
bf6d066 Add coffee
60cb0bb Add tea
```

Recorded in Bash; *ch27-rebase/expected-drop-conflict.bash.txt*.

git rebase --abort (section 27.3) put everything back. The message is the conflict from Chapter 22: the juice commit expected the coffee line to be there.

The real editor flow (the list opens in your editor and you edit it by hand) is what these sed commands imitate; the commands in the list are the ones above.

27.5 The golden rule

Because rebase **replaces commits with new ones**, it is safe only for commits that nobody else has.

CAUTION. Never rebase commits that you have already shared (pushed to a place where others may have pulled them). Rebasing them creates different commits with the same content. Your colleagues still have the old ones; the two versions of history then have to be reconciled, and that is painful and easy to get wrong. Rebase your **own, local, unpublished** work. For shared work, use merge or git revert (Chapter 25).

A rebased branch that was already pushed can only be pushed again by **forcing** the push, which overwrites the remote's branch. Chapter 33 returns to the dangers of forced pushes.

Two safer companions: git pull --rebase and --force-with-lease

git pull --rebase does a fetch and then rebases your local commits on top of what arrived, instead of merging. It is the way to avoid the merge commit that Chapter 23 created when two people pushed at once. Alice pushes a commit; Bob, who has a local commit of his own, pulls with --rebase and then pushes:

```
$ cd alice
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -qam "Add tea"
$ git push -q origin main
$ cd ../bob
```

```
$ printf 'Open Monday to Saturday.\n' > hours.md
$ git add hours.md
$ git commit -qm "Add opening hours"
```

Recorded in Bash; *ch27-rebase/expected-pull-rebase.bash.txt*.

```
$ git pull --rebase
From /home/learner/hub
   81f772e..965e160  main      -> origin/main
Successfully rebased and updated refs/heads/main.
$ git log --oneline --graph
* 872b896 (HEAD -> main) Add opening hours
* 965e160 (origin/main, origin/HEAD) Add tea
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
$ git push origin main
To /home/learner/hub.git
   965e160..872b896  main -> main
```

Recorded in Bash; *ch27-rebase/expected-pull-rebase.bash.txt*.

The history stays a straight line, with Bob's commit on top and a new hash (872b896), and the push is now a plain fast-forward. This only rewrote Bob's **own, unpublished** commit, so the golden rule holds. (Git's `git pull` documentation lists `--rebase` and the settings `pull.rebase` and `branch.<name>.rebase` that make it the default.)

git push --force-with-lease is a forced push with a safety check. Suppose Alice has pushed a branch `topic` and then amends her commit. A plain push is refused, because the remote branch is not an ancestor of hers:

```
$ cd alice
$ git switch -c topic
Switched to a new branch 'topic'
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -qam "Add tea"
$ git push -q -u origin topic
$ git commit -q --amend -m "Add tea to the menu"
$ git push origin topic
To /home/learner/hub.git
 ! [rejected]        topic -> topic (non-fast-forward)
error: failed to push some refs to '/home/learner/hub.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Recorded in Bash; *ch27-rebase/expected-force-with-lease.bash.txt*.

`--force-with-lease` says: "overwrite the remote branch, **but only if it is still where I last saw it**":

```
$ git push --force-with-lease origin topic
To /home/learner/hub.git
+ cece341...19c0972 topic -> topic (forced update)
```

Recorded in Bash; *ch27-rebase/expected-force-with-lease.bash.txt*.

+ cece341...19c0972 ... (forced update) shows that the remote branch was replaced. Now let Bob add a commit to topic behind Alice's back, and let Alice amend again without fetching:

```
$ cd ../bob
$ git fetch -q
$ git switch -q topic
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -qam "Add coffee"
$ git push -q origin topic
$ cd ../alice
$ git commit -q --amend -m "Add tea, again"
$ git push --force-with-lease origin topic
To /home/learner/hub.git
! [rejected]        topic -> topic (stale info)
error: failed to push some refs to '/home/learner/hub.git'
```

Recorded in Bash; *ch27-rebase/expected-force-with-lease.bash.txt*.

The lease protects Bob: Alice's push is **rejected with stale info**, because the remote branch is no longer at the commit that her remote-tracking branch remembers. A plain `--force` would have destroyed Bob's commit. Git's documentation calls it "like taking a 'lease' on the ref without explicitly locking it".

CAUTION. The protection compares the remote branch with your remote-tracking branch. If you run `git fetch` first, that note is updated and the lease will be granted, even though it means overwriting the commit that you have just fetched. Fetch, **look** at what arrived, and only then decide to force. The documentation also states that the forms of the option other than `--force-with-lease=<ref>:<expected>` are "still experimental", so treat the plain form as a helpful check and not a guarantee.

If a rebase goes wrong and you have already finished it, the reflog (Chapter 26) still holds the state from before: find the entry from before rebase (start) and reset to it.

Checkpoint

What You Learned

- Rebase replays a branch's commits on top of another commit, giving a straight history.
- The replayed commits are new commits with new hashes; the originals remain until they expire.
- A rebase can conflict once per commit; resolve, `git add`, `git rebase --continue`, or `git rebase --abort`.

- `git rebase -i` edits the list of commits; squash combines them.
- Never rebase commits that others already have.

New Vocabulary

- **Rebase:** replay commits on top of another commit, producing new commits.

Commands Learned

`git rebase <branch>`, `git rebase --abort`, `git rebase --continue`, `git rebase -i`.

Common Mistakes

1. **Rebasing shared commits.**
2. **Confusing which side is which in a rebase conflict.**
3. **Forgetting `git rebase --continue` after resolving.**
4. **Being surprised that hashes change.**
5. **Rebasing with uncommitted work in the tree.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What happens to the hash of a commit when it is rebased, and why?
2. Which branch do you stand on when you run `git rebase main` to bring add-tea up to date?
3. What are the three ways out of a rebase conflict?
4. What does squash do in an interactive rebase?
5. State the golden rule of rebase.

Before Moving On

You are ready for Chapter [28](#) if you can:

- ☐ rebase a local branch onto `main`
- ☐ abort a rebase and resolve one
- ☐ explain why rebasing shared commits is dangerous
- ☐ combine commits with a squash

How this chapter was checked

Claim	Evidence class	Ledger
Rebase of a diverged branch, new hashes, fast-forward afterwards	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-rebase); the ledger row says which.	R190
Rebase conflict, --abort, --continue	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-rebase); the ledger row says which.	R190
Interactive squash, drop, reorder and reword (todo list edited by a program), and the drop conflict	Locally tested (as above); the list of commands checked against the Git 2.56.0 documentation	R192
pull --rebase, --force-with-lease (rejection with stale info)	Locally tested (as above); semantics checked against the git pull and git push documentation (Git 2.56.0)	R191

Where this leads

Chapter 28 adds cherry-pick, bisect and patches, which also copy commits. Chapter 34 compares merge-based and rebase-based team workflows, and Chapter 33 returns to forced pushes.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use bakery-menu (rebuild it from Chapter 17 if needed). Only rebase **local** work.

Level 1 — Guided

1.1 Diverge. Create add-tea with one commit. On main, add hours.md and commit. Show the graph with `git log --oneline --all --graph`.

1.2 Rebase. Switch to add-tea and run `git rebase main`. Show the graph again. What changed?

Level 2 — Partially guided

2.1 Hashes. Write down the hash of "Add tea" before and after the rebase. Why are they different?

2.2 Finish. Switch to main and run `git merge add-tea`. What kind of merge is it?

Level 3 — Independent

3.1 A rebase conflict. Make both branches change the same line. Start the rebase, read `git status`, and abort it. Prove that the branch is unchanged.

3.2 Resolve it. Repeat, but resolve the conflict and finish with `git rebase --continue`.

Level 4 — Professional scenario

4.1 Tidy before sharing. Make three small commits on a local branch. Combine them into one with `git rebase -i HEAD~3` (change pick to squash on lines 2 and 3). Check the log and the combined message.

Level 5 — Troubleshooting

5.1 A learner rebased a branch that a colleague had already pulled. What will the colleague see, and what should the learner have done instead?

5.2 A learner finished a rebase and now realises that the result is wrong. Where can they find the state from before the rebase?

5.3 During a rebase, `git status` says both `modified: menu.md`. The learner edits the file and runs `git rebase --continue`, and Git refuses. What did they forget?

Chapter 28 — Cherry-pick, Bisect and Patches [Deep]

In this chapter

- `git cherry-pick`: copy one commit to another branch
- `git bisect`: find the commit that introduced a bug, by halving
- `git format-patch`, `git apply` and `git am`: share a change as a file

Deep. This chapter is marked **[Deep]**. You can skip it on a first read. Each tool here solves a specific problem that you may not meet for a while.

Before you start. Chapter 21, Chapter 25 and Chapter 17. The recordings use `bakery-menu` and were run in Bash and `zsh` on Git 2.43.0, then re-run in CI on Git 2.55.0.

28.1 Cherry-pick: copy one commit

Merging brings all the work of a branch. Sometimes you want **one commit** from another branch, without the rest: a bug fix made on a feature branch that `main` needs now.

New term: cherry-pick. Copy the change made by one commit onto the current branch, as a new commit.

A branch `drinks` has two commits, `tea` and `coffee`:

```
$ git switch -c drinks
Switched to a new branch 'drinks'
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Add tea"
[drinks 73ad7ed] Add tea
1 file changed, 1 insertion(+)
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -am "Add coffee"
[drinks 398a33e] Add coffee
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch28-tools/expected-cherry-pick.bash.txt*.

Back on `main`, the history looks like this:

```
$ git switch main
Switched to branch 'main'
$ git log --oneline --all --graph
* 398a33e (drinks) Add coffee
* 73ad7ed Add tea
```

```
* 81f772e (HEAD -> main) Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch28-tools/expected-cherry-pick.bash.txt*.

Cherry-pick the **first** of the two, tea. drinks~1 means "one commit before the tip of drinks" (Chapter 17):

```
$ git cherry-pick drinks~1
[main eb86246] Add tea
Date: Mon Jan 5 09:10:00 2026 +0000
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch28-tools/expected-cherry-pick.bash.txt*.

The result is a **new commit** on main: hash eb86246, where the original was 73ad7ed. As with a rebase (Chapter 27), the change is copied and the copy gets a new identity because its parent is different. The Date: line shows that Git kept the original author date.

```
$ git log --oneline --all --graph
* eb86246 (HEAD -> main) Add tea
| * 398a33e (drinks) Add coffee
| * 73ad7ed Add tea
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
$ git status --short
```

Recorded in Bash; *ch28-tools/expected-cherry-pick.bash.txt*.

The graph shows the copy on main and the original still on drinks; they are not linked in any way. Now the second one:

```
$ git cherry-pick drinks
[main 8b679d1] Add coffee
Date: Mon Jan 5 09:11:00 2026 +0000
1 file changed, 1 insertion(+)
$ git log --oneline
8b679d1 (HEAD -> main) Add coffee
eb86246 Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch28-tools/expected-cherry-pick.bash.txt*.

Two cautions.

- A copied commit can **conflict**, just as in Chapter 22, if the code around it differs. Git then stops; you resolve, git add, and git cherry-pick --continue, or you give up with git cherry-pick --abort. Here is a conflict on purpose: main and raise-bread both changed the price of the loaf, and main cherry-picks the branch's commit:


```
$ git cherry-pick raise-bread
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
error: could not apply 73e14f7... Raise the white loaf to 3.00
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git cherry-pick --continue".
hint: You can instead skip this commit with "git cherry-pick --skip".
hint: To abort and get back to the state before "git cherry-pick",
hint: run "git cherry-pick --abort".
$ git status --short
UU menu.md
```

Recorded in Bash; *ch28-tools/expected-cherry-pick-conflict.bash.txt*.

Your Git may word this differently. Git 2.55.0 and 2.56.0 (recorded and re-run in CI) print one more hint line, Disable this message with "git config set advice.mergeConflict false". The meaning is the same.

Git stops and leaves `menu.md` unmerged (UU), with the same kind of message as a conflicted merge. Abort, and everything returns to the state before:

```
$ git cherry-pick --abort
$ git status --short
```

Recorded in Bash; *ch28-tools/expected-cherry-pick-conflict.bash.txt*.

Or resolve and continue: try again, write the resolved file, git add it, and git cherry-pick --continue:

```
$ git cherry-pick raise-bread
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
error: could not apply 73e14f7... Raise the white loaf to 3.00
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git cherry-pick --continue".
hint: You can instead skip this commit with "git cherry-pick --skip".
hint: To abort and get back to the state before "git cherry-pick",
hint: run "git cherry-pick --abort".
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.95\n- Rolls (six): 3.00\n- Coconut cake\n(slice): 4.00\n' > menu.md
$ git add menu.md
$ git cherry-pick --continue --no-edit
[main df67f32] Raise the white loaf to 3.00
Date: Mon Jan 5 09:10:00 2026 +0000
1 file changed, 1 insertion(+), 1 deletion(-)
```

Recorded in Bash; *ch28-tools/expected-cherry-pick-conflict.bash.txt*.

```
$ git log --oneline
df67f32 (HEAD -> main) Raise the white loaf to 3.00
4afeda6 Raise the white loaf to 2.90
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch28-tools/expected-cherry-pick-conflict.bash.txt*.

The new commit keeps the original message and author date, and has your resolution.

- Because the copy is a different commit, merging drinks into main later will not know that the changes are already there. Usually Git handles that well, but the history contains the same change twice.

28.2 Bisect: find where a bug came in

You notice something wrong, and you know that it was fine last week. Somewhere among the commits since then, one broke it. Instead of reading them all, you can search by **halving**.

New term: bisect. A search through history that finds the commit that introduced a change, by repeatedly testing the commit in the middle of the remaining range.

Five commits are added to the menu. The third one adds a bad line, a "Mystery pie" at 0.01:

```
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Add tea"
[main 60cb0bb] Add tea
1 file changed, 1 insertion(+)
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -am "Add coffee"
[main bf6d066] Add coffee
1 file changed, 1 insertion(+)
$ printf -- '- Mystery pie: 0.01\n' >> menu.md
$ git commit -am "Add a mystery pie"
[main 7a5ee8a] Add a mystery pie
1 file changed, 1 insertion(+)
$ printf -- '- Juice: 2.20\n' >> menu.md
$ git commit -am "Add juice"
[main 59d1a45] Add juice
1 file changed, 1 insertion(+)
$ printf -- '- Scones: 1.80\n' >> menu.md
$ git commit -am "Add scones"
[main 2862c91] Add scones
1 file changed, 1 insertion(+)
$ git log --oneline
2862c91 (HEAD -> main) Add scones
59d1a45 Add juice
7a5ee8a Add a mystery pie
bf6d066 Add coffee
60cb0bb Add tea
```

```
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch28-tools/expected-bisect.bash.txt*.

(The commits are listed with the newest at the top; "Add a mystery pie" is the third one after "Add coconut cake".) You tell Git one commit that is bad (the current one) and one that is good (five commits back):

```
$ git bisect start
status: waiting for both good and bad commits
$ git bisect bad HEAD
status: waiting for good commit(s), bad commit known
$ git bisect good HEAD~5
Bisecting: 2 revisions left to test after this (roughly 1 step)
[bf6d06696d870f97a07b7fc658a78580e0976059] Add coffee
```

Recorded in Bash; *ch28-tools/expected-bisect.bash.txt*.

Git checks out the commit **in the middle**, bf6d066 Add coffee, and asks you to test it. You would run your program or read the file, and then say `git bisect good` or `git bisect bad`, and Git would pick the next middle. With n commits, this takes about $\log_2(n)$ steps: 1000 commits need only about 10 tests.

You can let Git do the testing if you can write a command that succeeds (exit status 0) when a commit is good and fails when it is bad. `git bisect run` runs it at each step. Here the test is "the file does **not** contain the word Mystery":

```
$ git bisect run sh -c '! grep -q Mystery menu.md'
running 'sh' '-c' '! grep -q Mystery menu.md'
Bisecting: 0 revisions left to test after this (roughly 1 step)
[59d1a45ac75c3ce66c8226746cdb12c9cf8f311a] Add juice
running 'sh' '-c' '! grep -q Mystery menu.md'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[7a5ee8a1ad6ad0d61d041c66d9346164b5a51b96] Add a mystery pie
running 'sh' '-c' '! grep -q Mystery menu.md'
7a5ee8a1ad6ad0d61d041c66d9346164b5a51b96 is the first bad commit
commit 7a5ee8a1ad6ad0d61d041c66d9346164b5a51b96
Author: Ada Learner <ada@example.org>
Date: Mon Jan 5 09:11:00 2026 +0000

    Add a mystery pie

menu.md | 1 +
1 file changed, 1 insertion(+)
bisect found first bad commit
```

Recorded in Bash; *ch28-tools/expected-bisect.bash.txt*.

Read the end: 7a5ee8a... is the first bad commit, followed by the commit's details. Git narrowed five commits down to one in three steps, without help.

When you are done, **always** return to where you were:

```
$ git bisect reset
Previous HEAD position was 7a5ee8a Add a mystery pie
Switched to branch 'main'
$ git status --short
```

Recorded in Bash; *ch28-tools/expected-bisect.bash.txt*.

`git bisect reset` leaves the bisect and returns you to your branch. While bisecting, your repository sits at a detached HEAD (Chapter 26); `reset` takes you back.

CAUTION. Do not commit or change files in the middle of a bisect. And write down the good and bad commits before you start, so that you can restart.

On Git 2.55.0, bisect puts quotes around the words: *waiting for both 'good' and 'bad' commits* and *is the first 'bad' commit*.

Bisecting by hand

`bisect run` needs a command that can judge a commit. When the judgement is yours (you read the file, or try the program), you answer `git bisect good` or `git bisect bad` at each step. Same five commits, same bad third commit:

```
$ git bisect start
status: waiting for both good and bad commits
$ git bisect bad
status: waiting for good commit(s), bad commit known
$ git bisect good HEAD~5
Bisecting: 2 revisions left to test after this (roughly 1 step)
[bf6d06696d870f97a07b7fc658a78580e0976059] Add coffee
```

Recorded in Bash; *ch28-tools/expected-bisect-manual.bash.txt*.

Your Git may word this differently. Git 2.55.0 and 2.56.0 (recorded and re-run in CI) put quotation marks around the words: *waiting for both 'good' and 'bad' commits*, and *print is the first 'bad' commit* at the end, followed by `commit <hash> (HEAD)`. The meaning is the same.

Git checked out the middle commit. Test it (here, count the lines that contain `Mystery`), and tell Git what you found:

```
$ grep -c Mystery menu.md
0
$ git bisect good
Bisecting: 0 revisions left to test after this (roughly 1 step)
[59d1a45ac75c3ce66c8226746cdb12c9cf8f311a] Add juice
$ grep -c Mystery menu.md
1
```

Recorded in Bash; *ch28-tools/expected-bisect-manual.bash.txt*.

The count was 0, so that commit was good, and Git moved on. The next commit has the bad line, so answer bad, and repeat until Git names the culprit:

```
$ git bisect bad
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[7a5ee8a1ad6ad0d61d041c66d9346164b5a51b96] Add a mystery pie
$ grep -c Mystery menu.md
1
$ git bisect bad
7a5ee8a1ad6ad0d61d041c66d9346164b5a51b96 is the first bad commit
commit 7a5ee8a1ad6ad0d61d041c66d9346164b5a51b96
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:11:00 2026 +0000

    Add a mystery pie

menu.md | 1 +
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch28-tools/expected-bisect-manual.bash.txt*.

`git bisect log` records what you told Git (six lines here, one per command), which is useful if you must repeat the search:

```
$ git bisect log | grep -c '^git bisect'
6
$ git bisect reset
Previous HEAD position was 7a5ee8a Add a mystery pie
Switched to branch 'main'
```

Recorded in Bash; *ch28-tools/expected-bisect-manual.bash.txt*.

The count of steps is what the messages say: "roughly N steps". The estimate is a halving estimate, and the official documentation describes the method as a binary search. Wording of the messages varies a little between Git versions (the quoted 'good' and 'bad' above).

28.3 Patches: share a change as a file

Sometimes you cannot push to a shared place: the receiver is offline, or an open-source project asks for changes by email. A **patch** is a file that describes a change.

New term: patch. A text file that describes changes to files, so that they can be applied elsewhere.

`git format-patch` turns commits into patch files. Here, the tea commit on a new branch tea:

```
$ git switch -c tea
Switched to a new branch 'tea'
$ printf -- '- Tea: 1.50\n' >> menu.md
```

```
$ git commit -am "Add tea"
[tea 73ad7ed] Add tea
1 file changed, 1 insertion(+)
$ git format-patch -1 HEAD
0001-Add-tea.patch
```

Recorded in Bash; *ch28-tools/expected-patch.bash.txt*.

Git wrote a file named 0001-Add-tea.patch. The start of the file:

```
$ head -n 8 0001-Add-tea.patch
From 73ad7ed97c9b9d17ef5c51e3e0f15a8e9e8a0638 Mon Sep 17 00:00:00 2001
From: Ada Learner <ada@example.org>
Date: Mon, 5 Jan 2026 09:10:00 +0000
Subject: [PATCH] Add tea

---
 menu.md | 1 +
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch28-tools/expected-patch.bash.txt*.

It looks like an email: who wrote it, when, and the subject ([PATCH] Add tea), followed by a summary and, further down (not shown), the change itself. It keeps the author and message. Now switch to main, which does not have the tea yet. First **check** that the patch applies cleanly:

```
$ git switch main
Switched to branch 'main'
$ git apply --check 0001-Add-tea.patch
```

Recorded in Bash; *ch28-tools/expected-patch.bash.txt*.

No output means that it would apply without problems. `git am` then applies it and records it as a commit, keeping the author and message:

```
$ git am 0001-Add-tea.patch
Applying: Add tea
$ git log --oneline
eb86246 (HEAD -> main) Add tea
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch28-tools/expected-patch.bash.txt*.

The commit has a new hash (eb86246) because its parent is different, as with cherry-pick.

There is also a plain `git apply`, which changes the files but does **not** commit. A diff that you saved as a file works too. Here a change is saved, reverted, checked, and applied:

```
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git diff > tea.patch
$ git restore menu.md
$ git apply --check tea.patch
```

```
$ git apply tea.patch
$ git status --short
M menu.md
?? tea.patch
$ git diff --stat
menu.md | 1 +
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch28-tools/expected-apply.bash.txt*.

The file is modified but nothing is staged or committed, and *tea.patch* is an untracked file. Now the patch is already applied, so a second check fails, which is what `--check` is for:

```
$ git apply --check tea.patch
error: patch failed: menu.md:3
error: menu.md: patch does not apply
```

Recorded in Bash; *ch28-tools/expected-apply.bash.txt*.

When `git am` meets a patch that does not apply, it stops in the middle, like a rebase. Here `main` got a different commit in the same place, so the *tea* patch does not fit:

```
$ cd bakery-menu
$ git switch -q -c tea
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -qam "Add tea"
$ git format-patch -q -1 HEAD
$ git switch -q main
$ printf -- '- Bagels: 1.20\n' >> menu.md
$ git commit -qam "Add bagels"
$ git am 0001-Add-tea.patch
Applying: Add tea
error: patch failed: menu.md:3
error: menu.md: patch does not apply
Patch failed at 0001 Add tea
hint: Use 'git am --show-current-patch=diff' to see the failed patch
When you have resolved this problem, run "git am --continue".
If you prefer to skip this patch, run "git am --skip" instead.
To restore the original branch and stop patching, run "git am --abort".
```

Recorded in Bash; *ch28-tools/expected-am-conflict.bash.txt*.

Your Git may word this differently. Git 2.55.0 and 2.56.0 (recorded and re-run in CI) print one more hint line, Disable this message with `"git config set advice.mergeConflict false"` and put `hint:` in front of the three instruction lines (When you have resolved this problem...). The meaning is the same.

Git lists the three ways out: `fix` and `git am --continue`, `git am --skip`, or `git am --abort`. The `abort` restores the branch:

```
$ git status --short
?? 0001-Add-tea.patch
$ git am --abort
$ git status --short
?? 0001-Add-tea.patch
$ git log --oneline -2
85fe3dd (HEAD -> main) Add bagels
81f772e Add coconut cake
```

Recorded in Bash; *ch28-tools/expected-am-conflict.bash.txt*.

Sending patches by **email** (`git send-email`) was not run in this book, and is not needed here: the patch files can be attached to any message.

28.4 Which tool for which job?

You want to...	Use
bring one commit to the current branch	<code>git cherry-pick <commit></code>
bring all of a branch	<code>git merge</code> or <code>git rebase</code> (Chapters 21 , 27)
find which commit broke something	<code>git bisect</code>
send a change without a shared repository	<code>git format-patch</code> , then <code>git am</code> on the other side

Checkpoint

What You Learned

- `git cherry-pick <commit>` copies one commit onto the current branch as a new commit.
- `git bisect` halves the range of commits until it finds the first bad one; `bisect run` automates the test; `bisect reset` returns to your branch.
- `git format-patch` writes commits as patch files; `git apply --check` tests one; `git am` applies it and keeps the author and message.

New Vocabulary

- **Cherry-pick**: copy the change of one commit onto the current branch.
- **Bisect**: find the commit that introduced a change by halving the range.
- **Patch**: a text file describing a change.

Commands Learned

`git cherry-pick`, `git bisect start`, `git bisect good`, `git bisect bad`, `git bisect run`, `git bisect reset`, `git format-patch`, `git apply --check`, `git am`.

Common Mistakes

1. **Forgetting `git bisect reset`.**
2. **Cherry-picking a commit that depends on an earlier one.**
3. **Expecting the copy to have the original's hash.**
4. **Changing files while bisecting.**
5. **Applying a patch without `--check`.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Why does a cherry-picked commit have a different hash?
2. About how many tests does bisect need for 1000 commits?
3. What must a command do for `git bisect run` to use it?
4. What does `git bisect reset` do?
5. What does `git am` keep that a plain `git apply` and commit would lose?

Before Moving On

You are ready for Chapter 29 if you can:

- [] cherry-pick a commit
- [] find a bad commit with `git bisect run`
- [] make and apply a patch file

How this chapter was checked

Claim	Evidence class	Ledger
<code>git cherry-pick</code> of one and of two commits; new hashes; author date kept	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-cherry-pick</code>); the ledger row says which.	R193
<code>git bisect run</code> and manual good/bad narrow to the first bad commit; <code>bisect log</code> ; <code>bisect reset</code>	Locally tested (as above); message wording varies by version	R195
<code>format-patch</code> , <code>apply</code> (with and without <code>--check</code>), <code>am</code> and <code>am --abort</code> ; <code>cherry-pick</code> conflict, <code>abort</code> and <code>continue</code>	Locally tested (as above); email (<code>send-email</code>) not run	R194

Where this leads

Chapter 29 names commits permanently. Chapter 34 discusses when teams cherry-pick, and Chapter 33 covers the signing of commits and tags.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use bakery-menu (rebuild it from Chapter 17 if needed).

Level 1 — Guided

1.1 Cherry-pick. Create a branch `drinks` with two commits (tea, coffee). On `main`, cherry-pick only the first. Show the graph.

1.2 Compare hashes. Write down the hash of "Add tea" on `drinks` and on `main`. Why do they differ?

Level 2 — Partially guided

2.1 The second commit. Cherry-pick the coffee commit too. Read the log. What is the relation between `main` and `drinks` now?

2.2 Make a patch. Commit a change on a branch, run `git format-patch -1 HEAD`, and read the first eight lines of the file.

Level 3 — Independent

3.1 Apply a patch. On `main`, run `git apply --check` on your patch, then `git am`. Check the log and the author.

3.2 Bisect by hand. Make five commits, one of which adds a bad line. Use `git bisect start`, `git bisect bad`, `git bisect good <commit>`, and at each step check the file and answer `git bisect good` or `git bisect bad`. Finish with `git bisect reset`.

Level 4 — Professional scenario

4.1 Automate it. Repeat 3.2 with `git bisect run sh -c '! grep -q Mystery menu.md'`. How many steps did it take? How many would 1000 commits need?

Level 5 — Troubleshooting

5.1 A learner cherry-picked a commit that used a function added in the previous commit, and the result does not work. What went wrong?

5.2 A learner ran `git bisect start yesterday` and forgot to reset. `git status` says `HEAD detached`. What should they run?

5.3 `git apply --check` prints an error. Should the learner run `git apply` anyway? Why not?

Chapter 29 — Tags and Versioning [Core]

In this chapter

- tagging an older commit, and listing tags
- version numbers, and semantic versioning as a convention
- why tags sort oddly, and how to fix the order
- `git describe`
- sharing tags with a remote, and deleting them there

Before you start. Chapter 24 (which introduced tags) and Chapter 23. The recordings use `bakery-menu` and were run in Bash and zsh on Git 2.43.0, then re-run in CI on Git 2.55.0.

Chapter 24 made a tag on the newest commit. This chapter uses tags properly: to mark releases, on any commit, and to share them.

29.1 Tag any commit

A tag does not have to be on the commit you are standing on. Give the commit's hash at the end, and you can mark history retrospectively. Here the very first commit gets `v0.1.0`, the current one gets `v1.0.0`, and a third tag marks the commit before it as a release candidate:

```
$ git tag -a v0.1.0 8a52ffe -m "First draft of the menu"
$ git tag -a v1.0.0 -m "First public menu"
$ git tag v1.0.0-rc.1 HEAD~1
```

Recorded in Bash; *ch29-tags/expected-versions.bash.txt*.

`-a` makes an **annotated** tag (Chapter 24) and `-m` gives its message. Without `-a`, as in the third command, the tag is **lightweight**.

Show the tags, and the commit that a tag names:

```
$ git show v0.1.0 --stat --oneline
tag v0.1.0

First draft of the menu
8a52ffe (tag: v0.1.0) Add the menu
 menu.md | 4 ++++
 1 file changed, 4 insertions(+)
```

Recorded in Bash; *ch29-tags/expected-versions.bash.txt*.

`git show` prints the tag's own details first (the tag name, the message), then the commit it points to. Tags marking a release are almost always **annotated**: the tagger, date and message are part of the record.

29.2 Version numbers

Names such as `v0.1.0` and `v1.0.0` follow a convention called **semantic versioning**.

New term: semantic versioning. A convention for numbering releases as MAJOR.MINOR.PATCH: the first number changes when the change is incompatible with earlier versions, the second when features are added in a compatible way, and the third for compatible bug fixes. A hyphen and a label, as in `1.0.0-rc.1`, mark a pre-release.

Checked against the specification (Semantic Versioning 2.0.0, R198). The specification says: "Given a version number MAJOR.MINOR.PATCH, increment the MAJOR version when you make incompatible API changes, the MINOR version when you add functionality in a backward compatible manner, the PATCH version when you make backward compatible bug fixes". A pre-release "MAY be denoted by appending a hyphen and a series of dot separated identifiers", and "pre-release versions have a lower precedence than the associated normal version" (`1.0.0-alpha < 1.0.0`). The specification also says of the letter v: `v1.2.3` "is not a semantic version", but prefixing one with a v is "a common way ... to indicate it is a version number", and it even uses `git tag v1.2.3 -m "Release version 1.2.3"` as its example. The tag names in this chapter are examples of the convention, not a requirement of Git: Git accepts any name (Chapter 20 listed the rules for names).

A leading v (as in `v1.0.0`) is common but not part of the number.

29.3 Listing and ordering

Plain `git tag` lists tags **alphabetically**, which is not always version order:

```
$ git tag
v0.1.0
v1.0.0
v1.0.0-rc.1
```

Recorded in Bash; *ch29-tags/expected-versions.bash.txt*.

The pre-release `v1.0.0-rc.1` appears after `v1.0.0`, but a release candidate comes before the release. `--sort=version:refname` sorts the numbers as versions ("`1.9`" before "`1.10`"), but by itself it does not fix that pre-release problem:

```
$ git tag --sort=version:refname
v0.1.0
v1.0.0
v1.0.0-rc.1
```

Recorded in Bash; *ch29-tags/expected-versions.bash.txt*.

A setting names the suffix that marks a pre-release, and then it sorts correctly:

```
$ git -c versionsort.suffix=-rc tag --sort=version:refname
v0.1.0
v1.0.0-rc.1
v1.0.0
```

Recorded in Bash; *ch29-tags/expected-versions.bash.txt*.

`-c versionsort.suffix=-rc` sets that option for one command only (Chapter 14). To make it permanent, use `git config versionsort.suffix -rc`.

29.4 git describe

`git describe` names the current commit in terms of the nearest tag:

```
$ git describe
v1.0.0
$ git describe --tags 8a52ffe
v0.1.0
```

Recorded in Bash; *ch29-tags/expected-versions.bash.txt*.

At a tagged commit it prints just the tag. The second command asks about another commit (the first one), and finds the tag `v0.1.0` on it. Both tags here were annotated, so both were found.

On a commit after the tag, `git describe` adds the number of commits since the tag and a short hash. Two commits were added after `v1.0`:

```
$ cd bakery-menu
$ git tag -a v1.0 -m "First public menu"
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -qam "Add tea"
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -qam "Add coffee"
$ git describe
v1.0-2-g398a33e
```

Recorded in Bash; *ch29-tags/expected-describe-after.bash.txt*.

`v1.0-2-g398a33e` reads: tag `v1.0`, **2** commits after it, and `g` (for "git") followed by the abbreviated hash of the current commit. `--abbrev=4` shortens the hash:

```
$ git describe --abbrev=4
v1.0-2-g398a
```

Recorded in Bash; *ch29-tags/expected-describe-after.bash.txt*.

By default `git describe` only looks at **annotated** tags (Git's documentation: "By default (without `--all` or `--tags`) `git describe` only shows annotated tags"). A lightweight tag is ignored until you say `--tags`:

```
$ git tag v1.1-light
$ git describe
v1.0-2-g398a33e
$ git describe --tags
v1.1-light
```

Recorded in Bash; *ch29-tags/expected-describe-after.bash.txt*.

--always prints a bare abbreviated hash when no tag can be used, and --dirty adds a mark if the working tree has uncommitted changes (there were none here):

```
$ git describe --always --dirty
v1.0-2-g398a33e
```

Recorded in Bash; *ch29-tags/expected-describe-after.bash.txt*.

29.5 Sharing tags

Tags are **local** until you push them (Chapter 23). Even after `git push`, tags stay behind. To see this, a bare repository `hub.git` acts as the remote, and the branch is pushed first (with `-q`, quietly). Then a tag is made, another lightweight one too, and only one is sent:

```
$ git remote add origin ../hub.git
$ git push -q origin main
$ git tag -a v1.0.0 -m "First public menu"
$ git tag v0.9-lightweight
$ git push origin v1.0.0
To ../hub.git
 * [new tag]          v1.0.0 -> v1.0.0
```

Recorded in Bash; *ch29-tags/expected-remote.bash.txt*.

`git push origin v1.0.0` sent one tag. To see what the remote has, ask it directly:

```
$ git ls-remote --tags origin
1d32143cf5c3f7fd0d2bc2c5b7494e76ae07ac2b refs/tags/v1.0.0
81f772ee32b93d8fcf80dbb34ab74053b69d97a2 refs/tags/v1.0.0^{}
```

Recorded in Bash; *ch29-tags/expected-remote.bash.txt*.

`git ls-remote --tags origin` lists the tags on the remote without downloading anything. Each line is a hash and a name. `v1.0.0` appears twice: the first line (`1d32143...`) is the **tag object** (the annotated tag itself); the second, ending in `^{}`, is the **commit** it points to. (A lightweight tag would appear once.) The lightweight tag `v0.9-lightweight` is **not** on the remote, because it was not pushed.

To push all your tags at once:

```
$ git push origin --tags
To ../hub.git
* [new tag]          v0.9-lightweight -> v0.9-lightweight
$ git ls-remote --tags origin
81f772ee32b93d8fcf80dbb34ab74053b69d97a2 refs/tags/v0.9-lightweight
1d32143cf5c3f7fd0d2bc2c5b7494e76ae07ac2b refs/tags/v1.0.0
81f772ee32b93d8fcf80dbb34ab74053b69d97a2 refs/tags/v1.0.0^{}
```

Recorded in Bash; *ch29-tags/expected-remote.bash.txt*.

Now the lightweight tag is there too, pointing straight at a commit. Be careful with `--tags`: it sends every local tag, including ones you may have made by mistake.

29.6 Deleting a tag

Deleting a tag locally does **not** delete it on the remote:

```
$ git tag -d v0.9-lightweight
Deleted tag 'v0.9-lightweight' (was 81f772e)
$ git ls-remote --tags origin
81f772ee32b93d8fcf80dbb34ab74053b69d97a2 refs/tags/v0.9-lightweight
1d32143cf5c3f7fd0d2bc2c5b7494e76ae07ac2b refs/tags/v1.0.0
81f772ee32b93d8fcf80dbb34ab74053b69d97a2 refs/tags/v1.0.0^{}
```

Recorded in Bash; *ch29-tags/expected-remote.bash.txt*.

The local tag is gone, but the remote still lists `v0.9-lightweight`. Remove it there explicitly:

```
$ git push origin --delete v0.9-lightweight
To ../hub.git
- [deleted]          v0.9-lightweight
$ git ls-remote --tags origin
1d32143cf5c3f7fd0d2bc2c5b7494e76ae07ac2b refs/tags/v1.0.0
81f772ee32b93d8fcf80dbb34ab74053b69d97a2 refs/tags/v1.0.0^{}
```

Recorded in Bash; *ch29-tags/expected-remote.bash.txt*.

- [deleted] confirms it.

CAUTION. Deleting a tag that other people may have already fetched does not remove it from their repositories, and re-using the name for a different commit causes confusion. Treat published tags as permanent: if a release is wrong, make a new tag (`v1.0.1`) rather than moving the old one.

Checked against GitHub's documentation (R196). "About releases" says releases "are based on Git tags", that a tag's date "may be different" from the release date, and that GitHub "will automatically include links to download a zip file and a tarball containing the contents of the repository at the point of the tag's creation". Chapter 66 covers them in full.

Checkpoint

What You Learned

- A tag can name any commit, given by hash: `git tag -a v0.1.0 <hash> -m "..."`.
- Semantic versioning (MAJOR.MINOR.PATCH) is a convention, not a Git rule.
- `git tag` lists alphabetically; `--sort=version:refname` sorts as versions, and `version-sort.suffix` places pre-releases correctly.
- `git describe` names a commit after the nearest tag.
- Tags are not sent by an ordinary push: use `git push origin <tag>` or `--tags`; `git ls-remote --tags` shows the remote's tags.
- Deleting a tag remotely needs `git push origin --delete <tag>`.

New Vocabulary

- **Semantic versioning:** the convention of MAJOR.MINOR.PATCH version numbers.

Commands Learned

`git tag -a <name> <commit> -m`, `git tag --sort=version:refname`, `git describe`, `git push origin <tag>`, `git push origin --tags`, `git push origin --delete <tag>`, `git ls-remote --tags`.

Common Mistakes

1. **Expecting `git push` to send tags.**
2. **Using `--tags` and pushing tags made by mistake.**
3. **Deleting a tag locally and assuming it is gone from the remote.**
4. **Moving or re-using a published tag.**
5. **Trusting alphabetical order as version order.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. How do you tag a commit that is not the current one?
2. What do the three numbers in 1.4.2 mean under semantic versioning?

3. Why did `git tag list v1.0.0-rc.1` after `v1.0.0`?
4. Why does `git ls-remote --tags` show `v1.0.0` twice?
5. How do you remove a tag from a remote?

Before Moving On

You are ready for Chapter 33 if you can:

- [] tag an older commit
- [] push a tag and see it on the remote
- [] delete a tag locally and remotely
- [] explain why published tags should not move

How this chapter was checked

Claim	Evidence class	Leader
Tagging older commits, listing, <code>--sort</code> , <code>version-sort.suffix</code> , <code>describe</code> on tagged commits	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0	R196
Pushing, listing (<code>ls-remote</code>) and deleting tags on a remote	Locally tested (as above), with a local bare repository	R197
Semantic versioning rules (MAJOR.MINOR.PATCH, pre-release precedence, the <code>v</code> prefix)	Checked against the Semantic Versioning 2.0.0 specification text	R198
<code>describe</code> after a tag (<code><tag>-<n>-g<hash></code>), <code>--abbrev</code> , <code>--tags</code> , <code>--always</code> , <code>--dirty</code> ; <code>version-sort.suffix</code>	Locally tested (as above); checked against the <code>git describe</code> and <code>version-sort.suffix</code> documentation (Git 2.56.0)	R198

Where this leads

Chapter 33 covers signing commits and tags. Chapter 66 shows how a hosting platform builds releases on top of tags, and Chapter 34 shows how teams use tags.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use `bakery-menu` (rebuild it from Chapter 17 if needed) and a bare repository as the remote (Chapter 23).

Level 1 — Guided

1.1 Tag the first commit. Find the hash of the first commit with `git log --oneline`. Tag it `v0.1.0` with `git tag -a`. Show it with `git show v0.1.0`.

1.2 Tag the latest commit. Tag `HEAD` as `v1.0.0`. List the tags.

Level 2 — Partially guided

2.1 A pre-release. Add a lightweight tag `v1.0.0-rc.1` on the commit before `HEAD`. List the tags with `git tag`. Is the order sensible? Try `--sort=version:refname` and then `-c versionsort.suffix=-rc`.

2.2 Describe. Run `git describe`. Then run `git describe --tags <first-commit-hash>`.

Level 3 — Independent

3.1 Push one tag. Create the bare repository, add it as origin, push main, then push only `v1.0.0`. Use `git ls-remote --tags origin` to show that only it arrived.

3.2 Push them all. Push the remaining tags with `--tags`. How many lines does `ls-remote` show for an annotated tag, and why?

Level 4 — Professional scenario

4.1 A wrong release. You published `v1.0.0` on the wrong commit. The team has already fetched it. What do you do? Write the commands, and say why you do not move the tag.

Level 5 — Troubleshooting

5.1 A learner deleted a tag with `git tag -d` and says the release "is still on the server". Explain and give the fix.

5.2 A learner pushed with `git push origin --tags` and now dozens of old test tags are on the shared repository. What are the options, and what is the lesson?

5.3 A learner sorts tags with `git tag` and sees `v1.10.0` before `v1.2.0`. Why, and what is the fix?

Chapter 30 — Inside Git: The Object Model [Deep]

In this chapter

- how Git stores your files: **blobs**, **trees**, **commits** and **tags**
- why a hash identifies content
- how branches and tags are just small files
- what a packfile is, and what `git gc` does
- `git fsck`, dangling objects, and `git archive`

Deep. This chapter is marked **[Deep]**. You can use Git well without it. It explains why Git behaves as it does, and it makes the reflog (Chapter 26), rebase (Chapter 27) and the security chapter (Chapter 33) much easier to understand.

Before you start. Chapter 15, Chapter 19 and Chapter 20. Every command here was run in Bash and zsh on Git 2.43.0 and re-run in CI on Git 2.55.0. The examples inspect the `.git` folder; the file layout described is what these versions do. Git's own documentation calls the internals subject to change, so do not write programs that depend on the layout.

30.1 A database of named things

Chapter 15 showed that a commit is a snapshot. Underneath, Git is a **content-addressed database**: you give it content, it gives you back a name (a hash), and you can ask for the content by that name.

New term: object. Any item in Git's database. Git has four kinds: blob, tree, commit and tag.

Ask Git to store a line of text. `git hash-object` computes the name without storing; `-w` writes it:

```
$ git init obj-demo
Initialized empty Git repository in /home/learner/obj-demo/.git/
$ cd obj-demo
$ echo 'White loaf: 2.80' | git hash-object --stdin
28adfbf9397e4db31a26821309402a09c1b6b943
$ echo 'White loaf: 2.80' | git hash-object -w --stdin
28adfbf9397e4db31a26821309402a09c1b6b943
```

Recorded in Bash; *ch30-objects/expected-blob.bash.txt*.

The name `28adfbf9397e4db31a26821309402a09c1b6b943` is a **hash** of the content. The same content always gives the same name, on any computer. With `-w`, the object was stored:

```
$ find .git/objects -type f
.git/objects/28/adfbf9397e4db31a26821309402a09c1b6b943
```

Recorded in Bash; *ch30-objects/expected-blob.bash.txt*.

The stored object is a file in `.git/objects`, in a folder named after the **first two** characters of the hash (28), with the rest as the file name. Ask what kind of object it is, how big, and what it contains:

```
$ git cat-file -t 28adfbf
blob
$ git cat-file -s 28adfbf
17
$ git cat-file -p 28adfbf
White loaf: 2.80
```

Recorded in Bash; *ch30-objects/expected-blob.bash.txt*.

New term: blob. An object that holds the contents of one file, and nothing else: no name, no folder, no date. (The word is short for "binary large object".)

The blob is 17 bytes: the sixteen visible characters plus the newline. Storing **the same content again** does not make a second copy:

```
$ echo 'White loaf: 2.80' | git hash-object -w --stdin
28adfbf9397e4db31a26821309402a09c1b6b943
$ find .git/objects -type f | wc -l
1
```

Recorded in Bash; *ch30-objects/expected-blob.bash.txt*.

The count is still one file. Two files with identical content, anywhere in your project or its history, share one blob.

The name is computed, not chosen

The hash is not random. It is the SHA-1 checksum of a short header, the word `blob`, a space, the size in bytes, and a zero byte, followed by the content. You can compute it yourself with an ordinary tool:

```
$ printf 'blob 17\0White loaf: 2.80\n' | shasum
28adfbf9397e4db31a26821309402a09c1b6b943 -
```

Recorded in Bash; *ch30-objects/expected-blob.bash.txt*.

The result is the same name that Git produced. The last line of the recording shows which hash function this repository uses:

```
$ git rev-parse --show-object-format
sha1
```

Recorded in Bash; *ch30-objects/expected-blob.bash.txt*.

Checked against the documentation (Git 2.56.0, R202). SHA-1 is the default object format in the versions tested. Git's own document on planned breaking changes, in its section on Git 3.0, says that "the default hash function for new repositories will be changed from "sha1" to "sha256"", and gives as the reason that "SHA-1 has been deprecated by NIST in 2011" (it also plans the "reftable" reference format and the default branch name main for new repositories). Those are plans for a future version, not a description of today's default. A SHA-256 repository "cannot be read by older versions of Git", and the project's transition document says that SHA-256 was picked in late 2018 as SHA-1's successor. So: treat "SHA-1" as the current default, expect it to change, and never write a script that depends on the length of a hash.

30.2 Trees and commits

A blob has no file name. The names live in **trees**.

New term: tree. An object that lists the contents of one folder: for each entry a mode, a type (blob or another tree), a hash and a name.

Build a small project with a folder in it, and commit:

```
$ git init obj-demo
Initialized empty Git repository in /home/learner/obj-demo/.git/
$ cd obj-demo
$ printf 'White loaf: 2.80\nRolls (six): 3.00\n' > menu.md
$ mkdir drinks
$ printf 'Tea: 1.50\n' > drinks/hot.md
$ git add .
$ git commit -m "Add menu and drinks"
[main (root-commit) 762af16] Add menu and drinks
 2 files changed, 3 insertions(+)
 create mode 100644 drinks/hot.md
 create mode 100644 menu.md
```

Recorded in Bash; *ch30-objects/expected-commit.bash.txt*.

Look at the commit **object** itself:

```
$ git cat-file -p HEAD
tree 3f1f9623d71be5d01728beffeab8c162cc97e3ca
author Ada Learner <ada@example.org> 1767603960 +0000
```

```
committer Ada Learner <ada@example.org> 1767603960 +0000
Add menu and drinks
```

Recorded in Bash; *ch30-objects/expected-commit.bash.txt*.

A commit has four things: the hash of a **tree** (the whole project at that moment), the **author** and **committer** (with a timestamp, in seconds since 1970, and a time zone), and the **message**. A commit that had a parent would also have a parent line naming it. This first commit has none.

Then look at the tree that the commit names:

```
$ git cat-file -p 'HEAD^{tree}'
040000 tree 8214205a3a9ec4a01ff1c38afed63b0fa8265355    drinks
100644 blob 5cf2ab8b14d0142893464d70f752b9b4d25797a3    menu.md
```

Recorded in Bash; *ch30-objects/expected-commit.bash.txt*.

Two entries: `menu.md` is a blob (mode 100644, an ordinary file), and `drinks` is another **tree** (mode 040000, a folder). The shortcut `git ls-tree HEAD` shows the same thing, and `-r` walks into folders:

```
$ git ls-tree HEAD
040000 tree 8214205a3a9ec4a01ff1c38afed63b0fa8265355    drinks
100644 blob 5cf2ab8b14d0142893464d70f752b9b4d25797a3    menu.md
$ git ls-tree -r HEAD
100644 blob 45f2c5990d976c063458edad8ee69ecb26749412    drinks/hot.md
100644 blob 5cf2ab8b14d0142893464d70f752b9b4d25797a3    menu.md
```

Recorded in Bash; *ch30-objects/expected-commit.bash.txt*.

You can read one file from any commit with `commit:path`:

```
$ git cat-file -p HEAD:drinks/hot.md
Tea: 1.50
```

Recorded in Bash; *ch30-objects/expected-commit.bash.txt*.

The **index** (Chapter 15) is a list of the same kind, the tree that the next commit will use:

```
$ git ls-files --stage
100644 45f2c5990d976c063458edad8ee69ecb26749412 0    drinks/hot.md
100644 5cf2ab8b14d0142893464d70f752b9b4d25797a3 0    menu.md
```

Recorded in Bash; *ch30-objects/expected-commit.bash.txt*.

So a commit points to a tree, a tree points to blobs and other trees, and each name is the hash of what it names. A change to any file changes its blob's hash, so the tree's hash, so the commit's hash. That is why a commit hash identifies the **whole** project and its entire history: each commit contains its parent's hash, so a change anywhere back in time would change every hash after it. (This is also why rebasing creates new commits, Chapter 27.)

30.3 Refs: names for commits

Branches, tags and HEAD are not objects. They are **refs**: small files that contain a hash or the name of another ref.

New term: ref. A name that points to an object, stored as a small file under `.git/refs` (or, in some cases, in a packed list). Branches, tags and remote-tracking branches are refs.

Make a commit, an annotated tag, and a branch:

```
$ git init obj-demo
Initialized empty Git repository in /home/learner/obj-demo/.git/
$ cd obj-demo
$ printf 'White loaf: 2.80\n' > menu.md
$ git add menu.md
$ git commit -m "Add menu"
[main (root-commit) d2c521c] Add menu
 1 file changed, 1 insertion(+)
 create mode 100644 menu.md
$ git tag -a v1.0 -m "First menu"
$ git switch -c experiment
Switched to a new branch 'experiment'
```

Recorded in Bash; *ch30-objects/expected-refs.bash.txt*.

Now read the files:

```
$ cat .git/HEAD
ref: refs/heads/experiment
$ cat .git/refs/heads/main
d2c521ce41194be46021aef2f83e2fe37d6fe9f8
$ cat .git/refs/tags/v1.0
da7d3cd54643668fae087fd3eb73361482a60fa2
```

Recorded in Bash; *ch30-objects/expected-refs.bash.txt*.

`.git/HEAD` says `ref: refs/heads/experiment`: HEAD points to a branch (Chapter 26 called the alternative a detached HEAD). `.git/refs/heads/main` is a file with **41 characters**: the commit's hash, and a newline. **That is all a branch is.** Creating a branch writes one small file; deleting one removes it. This is why branches in Git are cheap. (After `git gc`, Git may move these files into one file, `.git/packed-refs`. This was checked by hand for this chapter but not recorded; it is one more reason to ask `git rev-parse` rather than read the files.)

`git show-ref` lists them all:

```
$ git show-ref
d2c521ce41194be46021aef2f83e2fe37d6fe9f8 refs/heads/experiment
d2c521ce41194be46021aef2f83e2fe37d6fe9f8 refs/heads/main
da7d3cd54643668fae087fd3eb73361482a60fa2 refs/tags/v1.0
```

Recorded in Bash; *ch30-objects/expected-refs.bash.txt*.

Both branches point to the same commit. The tag has a **different** hash, because an annotated tag is an object of its own (the fourth kind):

```
$ git cat-file -t v1.0
tag
$ git cat-file -p v1.0
object d2c521ce41194be46021aef2f83e2fe37d6fe9f8
type commit
tag v1.0
tagger Ada Learner <ada@example.org> 1767604020 +0000

First menu
```

Recorded in Bash; *ch30-objects/expected-refs.bash.txt*.

New term: tag object. The object that an annotated tag creates: it names another object (usually a commit), a type, a tag name, the tagger and a message. A lightweight tag has no object of its own; it is a ref straight to the commit.

Three ways to ask "what does this name point to?":

```
$ git rev-parse main v1.0 'v1.0^{commit}'
d2c521ce41194be46021aef2f83e2fe37d6fe9f8
da7d3cd54643668fae087fd3eb73361482a60fa2
d2c521ce41194be46021aef2f83e2fe37d6fe9f8
```

Recorded in Bash; *ch30-objects/expected-refs.bash.txt*.

The tag `v1.0` resolves to the tag object, and `v1.0^{commit}` follows it through to the commit. This is why `git ls-remote` listed annotated tags twice in Chapter 29.

30.4 Loose objects and packfiles

Each object you create starts as a **loose object**: one file. Git also has a compact format, the **packfile**, which stores many objects in one file and stores similar versions as differences.

New term: packfile. A single file in which Git stores many objects compactly, using differences between similar objects. Git creates packfiles when it tidies (`git gc`) and when it sends or receives objects over a network.

Three commits have been made. Count the objects, then let Git tidy:

```
$ git init obj-demo
Initialized empty Git repository in /home/learner/obj-demo/.git/
$ cd obj-demo
$ printf 'White loaf: 2.80\n' > menu.md
$ git add menu.md
$ git commit -m "Add menu"
```



```
[main (root-commit) d2c521c] Add menu
 1 file changed, 1 insertion(+)
 create mode 100644 menu.md
$ printf 'White loaf: 2.90\n' > menu.md
$ git commit -am "Raise the price"
[main 4e686c1] Raise the price
 1 file changed, 1 insertion(+), 1 deletion(-)
$ printf 'White loaf: 3.00\n' > menu.md
$ git commit -am "Raise the price again"
[main 2171d29] Raise the price again
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Recorded in Bash; *ch30-objects/expected-storage.bash.txt*.

```
$ git count-objects -v | grep -E '^(count|in-pack|packs):'
count: 9
in-pack: 0
packs: 0
```

Recorded in Bash; *ch30-objects/expected-storage.bash.txt*.

count is the number of **loose** objects: nine (three commits, three trees, three blobs). in-pack and packs say that none are packed yet. Now run garbage collection:

```
$ git gc -q
$ git count-objects -v | grep -E '^(count|in-pack|packs):'
count: 0
in-pack: 9
packs: 1
```

Recorded in Bash; *ch30-objects/expected-storage.bash.txt*.

New term: garbage collection. Git's housekeeping: `git gc` packs loose objects into packfiles and removes objects that nothing refers to any more (after a safety period).

The nine loose files became nine objects inside **one** pack. The history is unchanged:

```
$ git log --oneline
2171d29 (HEAD -> main) Raise the price again
4e686c1 Raise the price
d2c521c Add menu
$ git cat-file -p HEAD~2:menu.md
White loaf: 2.80
```

Recorded in Bash; *ch30-objects/expected-storage.bash.txt*.

Git runs this kind of housekeeping on its own from time to time, so you rarely need to.

Checked against the documentation (Git 2.56.0, R201). `git gc` "will call `prune --expire 2.weeks.ago`": unreachable objects younger than **two weeks** are kept (the setting is `gc.pruneExpire`). Git also runs a light-weight `git gc --auto` "from time to time" from some commands: it packs the loose objects when there are "approximately more than" **6700** of them (`gc.auto`), and consolidates packs when there are more than **50** (`gc.autoPackLimit`). Chapter 26 gives the reflog's own expiry periods (90 and 30 days) and the practical rule: recover soon.

30.5 Checking and exporting

`git fsck` checks that the database is consistent. A healthy repository prints nothing and exits with status 0:

```
$ git init obj-demo
Initialized empty Git repository in /home/learner/obj-demo/.git/
$ cd obj-demo
$ printf 'White loaf: 2.80\n' > menu.md
$ git add menu.md
$ git commit -m "Add menu"
[main (root-commit) d2c521c] Add menu
 1 file changed, 1 insertion(+)
 create mode 100644 menu.md
$ printf 'Rolls (six): 3.00\n' >> menu.md
$ git commit -am "Add rolls"
[main ee9eec6] Add rolls
 1 file changed, 1 insertion(+)
$ git fsck 2>/dev/null; echo "exit status: $?"
exit status: 0
```

Recorded in Bash; *ch30-objects/expected-fsck.bash.txt*.

(Here `2>/dev/null` hides the progress messages, which do not matter to us. `echo "exit status: $?"` prints the exit status.)

After a `git reset --hard` (Chapter 25), the commit that was dropped is still in the database, but no branch or tag leads to it. `fsck` can find such objects. The option `--no-reflogs` ignores the reflog, so that the commit really counts as unreachable:

```
$ git reset --hard HEAD~1
HEAD is now at d2c521c Add menu
$ git fsck --no-reflogs 2>/dev/null
dangling commit ee9eec6763a660d81626c783ad78359f8920c2ff
```

Recorded in Bash; *ch30-objects/expected-fsck.bash.txt*.

New term: dangling object. An object in the database that no ref leads to. Such objects will be removed by garbage collection eventually.

dangling commit `ee9eec6...` is the "Add rolls" commit we reset away. This is how commits can be recovered even without the reflog, until `gc` removes them.

Finally, `git archive` exports a commit as a plain archive, without the `.git` folder. This is handy for sharing a snapshot:

```
$ git archive --format=tar HEAD | tar -t
menu.md
```

Recorded in Bash; *ch30-objects/expected-fsck.bash.txt*.

The archive (piped to `tar -t`, which lists its contents) holds only the files of the commit.

30.6 What this explains

You have seen...	...because
Two identical files take space once	Same content, same blob hash
Commits are cheap; branches are cheaper	A commit is a few small objects; a branch is a 41-byte file
Rebase gives new hashes	A commit's hash includes its parent's hash and its tree
<code>reset --hard</code> does not delete commits at once	The objects stay until <code>gc</code> removes unreachable ones
A secret committed once stays in history	The blob is reachable from every commit that has it; Chapter 33

Checkpoint

What You Learned

- Git is a content-addressed database of four kinds of object: blobs (file contents), trees (folders), commits and tags.
- An object's name is the SHA-1 hash of a header and its content in these versions of Git, so identical content is stored once.
- A commit names a tree, its parent(s), an author, a committer and a message.
- Branches, tags and `HEAD` are refs: small files that contain a hash or another ref's name.
- Objects begin as loose files; `git gc` packs them and eventually removes unreachable objects.
- `git fsck` checks the database and lists dangling objects; `git archive` exports a snapshot.

New Vocabulary

- **Object**: any item in Git's database (blob, tree, commit or tag).
- **Blob**: an object holding the contents of one file.
- **Tree**: an object listing the contents of a folder.
- **Ref**: a name that points to an object, stored as a small file.

- **Tag object:** the object created by an annotated tag.
- **Packfile:** one file storing many objects compactly.
- **Garbage collection:** housekeeping that packs objects and removes unreachable ones.
- **Dangling object:** an object no ref leads to.

Commands Learned

`git hash-object`, `git cat-file -t`, `git cat-file -s`, `git cat-file -p`, `git ls-tree`, `git ls-files --stage`, `git show-ref`, `git rev-parse`, `git count-objects -v`, `git gc`, `git fsck`, `git archive`.

Common Mistakes

1. **Editing files inside `.git` by hand.**
2. **Assuming a branch is a copy of the files.**
3. **Thinking `git gc` is needed in daily work.**
4. **Trusting "unreachable means deleted".**
5. **Assuming the hash algorithm will never change.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is a blob, and what does it not contain?
2. Why is the same content stored only once?
3. What is inside `.git/refs/heads/main`?
4. What is the difference between a lightweight and an annotated tag, at the object level?
5. What does `git fsck --no-reflogs` report after a `git reset --hard`, and why?

Before Moving On

You are ready for Chapter [31](#) if you can:

- [] name the four kinds of object
- [] explain how a commit leads to files
- [] read `.git/HEAD` and a branch file
- [] describe what `git gc` does

How this chapter was checked

Claim	Evidence class	Ledger
hash-object, cat-file, loose storage, hash = SHA-1 of header plus content	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0 (the hash was also recomputed with shasum). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-cat-file, git-hash-object); the ledger row says which.	R199
Commit, tree, ls-tree, index, refs, tag objects	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-ls-tree, gitrepository-layout); the ledger row says which.	R200
Packfiles, gc, fsck, dangling objects, archive	Locally tested (as above)	R201
Default hash function and the planned change in Git 3.0; gc grace period and automatic gc thresholds	Checked against the Git 2.56.0 documentation (BreakingChanges, technical/hash-function-transition, config/gc)	R201, R202

Where this leads

Chapter 31 deals with very large repositories, where these storage facts matter. Chapter 32 customises Git, Chapter 33 shows why history is hard to scrub, and Chapter 35 shows how to read the official documentation that describes these internals.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use a new, empty repository (`git init obj-demo`). Do not edit files inside `.git`.

Level 1 — Guided

1.1 A blob by hand. Run `echo 'White loaf: 2.80' | git hash-object -w --stdin`. Then run `git cat-file -t, -s` and `-p` on the hash. What are the three answers?

1.2 Where is it? Run `find .git/objects -type f`. How is the path related to the hash?

Level 2 — Partially guided

2.1 Same content twice. Store the same line again. Does the number of files change? Why?

2.2 Compute it yourself. Use `printf` and `shasum` to compute the hash of blob `17\0White loaf: 2.80\n`. Compare it with Git's.

Level 3 — Independent

3.1 Follow a commit. Commit a `menu.md` and a `drinks/hot.md`. Follow the chain by hand: `git cat-file -p HEAD`, then the tree it names, then a blob.

3.2 What is a branch? Create a branch. Print `.git/HEAD` and `.git/refs/heads/<branch>`. How many characters are in the branch file?

Level 4 — Professional scenario

4.1 Tags at object level. Create one lightweight and one annotated tag on the same commit. Use `git cat-file -t` on each. Explain the difference and how `git rev-parse <tag>^{commit}` treats them.

Level 5 — Troubleshooting

5.1 A colleague says "I deleted the branch, so the commits are gone from the disk". Explain what is really true, and how you would check.

5.2 After `git gc`, a learner counts objects and sees `count: 0`. They fear the data is lost. What happened?

5.3 A learner wants to write a script that reads `.git/refs/heads/main` to find the current commit. What is the safer command, and why?

Chapter 31 — Big and Complex Repositories [Deep]

In this chapter

- git worktree: several folders, one repository
- shallow clones: download only recent history
- sparse checkout: show only part of a big project
- submodules: a repository inside a repository
- Git LFS: large files kept outside the history
- what this chapter does not cover, and why

Deep. This chapter is marked **[Deep]**. You can skip it on a first read. Everything here solves a problem that appears when projects grow, and most small projects never meet them.

Before you start. Chapter 23 and Chapter 30. The recordings use bakery-menu and local folders as "remotes" (Chapter 23). They ran in Bash and zsh on Git 2.43.0, and git-lfs 3.4.1, and were re-run in CI on Git 2.55.0.

31.1 Worktrees: another folder, same repository

You are in the middle of a task on one branch, and an urgent fix is needed on another. Chapter 24 offered a stash; another way is a second **working folder** connected to the same repository.

New term: worktree. An additional working folder that belongs to the same repository, with its own checked-out branch. All worktrees share one history.

```
$ cd bakery-menu
$ git worktree add ../bakery-hotfix -b hotfix
Preparing worktree (new branch 'hotfix')
HEAD is now at 81f772e Add coconut cake
$ git worktree list
/home/learner/bakery-menu      81f772e [main]
/home/learner/bakery-hotfix    81f772e [hotfix]
```

Recorded in Bash; *ch31-bigrepos/expected-worktree.bash.txt*.

git worktree add ../bakery-hotfix -b hotfix created a new folder next to the current one, made a new branch hotfix, and checked it out there. git worktree list shows both. Work in the new folder like in any repository:

```
$ cd ../bakery-hotfix
$ printf -- '- Bagels: 1.20\n' >> menu.md
$ git commit -am "Add bagels"
[hotfix 0a5f828] Add bagels
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch31-bigrepos/expected-worktree.bash.txt*.

Back in the first folder, both branches are visible: the commit made in the other folder is in the shared history.

```
$ cd ../bakery-menu
$ git log --oneline --all --graph
* 0a5f828 (hotfix) Add bagels
* 81f772e (HEAD -> main) Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch31-bigrepos/expected-worktree.bash.txt*.

When the fix is finished, remove the extra folder. The branch stays:

```
$ git worktree remove ../bakery-hotfix
$ git worktree list
/home/learner/bakery-menu 81f772e [main]
$ git branch
  hotfix
* main
```

Recorded in Bash; *ch31-bigrepos/expected-worktree.bash.txt*.

Two rules: a branch can be checked out in only one worktree at a time, and the extra folders are ordinary folders that you must not delete by hand (use `git worktree remove`).

31.2 Shallow clones: recent history only

A project with a long history takes time and disk space to download. A **shallow clone** takes only the most recent commits. To demonstrate, a bare repository `hub.git` plays the remote, and it is cloned by its `file://` address (a plain path would copy everything, and ignore the depth):

```
$ git clone --bare -q bakery-menu hub.git
$ git clone --depth 1 file://$PWD/hub.git shallow
Cloning into 'shallow'...
```

Recorded in Bash; *ch31-bigrepos/expected-shallow.bash.txt*.

`--depth 1` asked for only the latest commit.

New term: shallow clone. A clone that contains only the most recent commits, up to a chosen depth, instead of the whole history.

```
$ cd shallow
$ git log --oneline
81f772e (grafted, HEAD -> main, origin/main, origin/HEAD) Add coconut cake
$ git rev-parse --is-shallow-repository
true
```

Recorded in Bash; *ch31-bigrepos/expected-shallow.bash.txt*.

The log has one commit, marked grafted: Git has cut the history off there. `--is-shallow-repository` confirms it. Missing history can be fetched later:

```
$ git fetch --unshallow
$ git log --oneline
81f772e (HEAD -> main, origin/main, origin/HEAD) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
$ git rev-parse --is-shallow-repository
false
```

Recorded in Bash; *ch31-bigrepos/expected-shallow.bash.txt*.

After `git fetch --unshallow`, all three commits are there, and the repository is no longer shallow.

Trade-offs. A shallow clone is fast and small, which is why it is used in automation (Chapter 54). But history commands (`log`, `blame`, `bisect`) see only what was downloaded, and some operations do not work in a shallow repository until you fetch more.

31.3 Sparse checkout: only some folders

Sometimes the whole history is fine, but you only want part of the files on disk. A repository with `docs` and `src` folders:

```
$ cd bakery-menu
$ mkdir docs src
$ printf 'How to bake\n' > docs/guide.md
$ printf 'print\n' > src/main.py
$ git add .
$ git commit -m "Add docs and src"
[main ca85072] Add docs and src
 2 files changed, 2 insertions(+)
 create mode 100644 docs/guide.md
 create mode 100644 src/main.py
$ ls
docs  menu.md  src
```

Recorded in Bash; *ch31-bigrepos/expected-sparse.bash.txt*.

`git sparse-checkout set docs` limits the working folder to docs (and the files at the top level):

```
$ git sparse-checkout set docs
$ ls
docs  menu.md
$ git sparse-checkout list
docs
```

Recorded in Bash; *ch31-bigrepos/expected-sparse.bash.txt*.

The `src` folder disappeared from the disk, but not from the repository: its files are still in every commit. Turn it off again to get everything back:

```
$ git sparse-checkout disable
$ ls
docs  menu.md  src
```

Recorded in Bash; *ch31-bigrepos/expected-sparse.bash.txt*.

This is the cone style of sparse checkout, the simple one, which selects whole folders.

31.4 Submodules: a repository inside a repository

Sometimes a project uses another repository, such as a shared library, and wants to pin an exact version of it. A **submodule** does that.

New term: submodule. A repository placed inside another repository at a fixed path. The outer repository records which commit of the inner one to use.

A small library repository is made first. Then the first attempt to add it as a submodule **fails**:

```
$ mkdir lib-repo
$ cd lib-repo
$ git init -q
$ printf 'shared code\n' > lib.txt
$ git add lib.txt
$ git commit -q -m "Add lib"
$ cd ../bakery-menu
$ git submodule add ../lib-repo vendor/lib
Cloning into '/home/learner/bakery-menu/vendor/lib'...
fatal: transport 'file' not allowed
fatal: clone of '/home/learner/lib-repo' into submodule path '/home/learner/bakery-menu/vendor/lib' failed
```

Recorded in Bash; *ch31-bigrepos/expected-submodule.bash.txt*.

fatal: transport 'file' not allowed: Git refuses, by default, to use a local path as the source of a submodule. This is a deliberate safety rule in recent Git versions. For a normal submodule, the address is a network address, and this rule does not matter. To do the demonstration locally we have to allow it explicitly, for this one command only:

```
$ git -c protocol.file.allow=always submodule add ../lib-repo vendor/lib
Cloning into '/home/learner/bakery-menu/vendor/lib'...
done.
```

Recorded in Bash; *ch31-bigrepos/expected-submodule.bash.txt*.

CAUTION. Do not switch this protection off permanently (`protocol.file.allow=always` in your global configuration) just to make a local demonstration convenient. The rule exists because cloning a malicious local repository can place unexpected files inside your repository's Git folder (see the documentation note below).

Checked against the documentation (Git 2.56.0, R205). Git's `protocol.allow` documentation says that, by default, "known-safe protocols (`http`, `https`, `git`, `ssh`) have a default policy of `always`", while "all other protocols (including `file`) have a default policy of `user`". The `user` policy means that a protocol "is only able to be used when `GIT_PROTOCOL_FROM_USER` is either unset or has a value of `1`", and that it is meant for protocols that you use directly but that you do not want used "by commands which execute `clone`/`fetch`/`push` commands without user input, e.g. recursive submodule initialization". The rule came with the security fix for **CVE-2022-39253** (Git 2.30.6 and later releases, including 2.38.1): the release notes say that "the value of `protocol.file.allow` is changed to be `user` by default", and that cloning from a malicious local repository could place "arbitrary files" in the new repository's `$GIT_DIR`. That is the reason for the refusal above.

Now the state:

```
$ git status --short
A .gitmodules
A vendor/lib
$ cat .gitmodules
[submodule "vendor/lib"]
    path = vendor/lib
    url = ../lib-repo
$ git submodule status
2e4ae7032781338fb40d5d24c3d4e709ac2eea22 vendor/lib (heads/main)
```

Recorded in Bash; *ch31-bigrepos/expected-submodule.bash.txt*.

`.gitmodules` is a small text file, part of the project, that lists each submodule and where it comes from. `git submodule status` shows the exact commit that is checked out in it. Commit:

```
$ git commit -m "Add the lib submodule"
[main b3a8826] Add the lib submodule
2 files changed, 4 insertions(+)
create mode 100644 .gitmodules
create mode 160000 vendor/lib
$ git ls-tree -r HEAD
```

```
100644 blob 098fc94ae95f14ae76c1b1142e20206bb58df196    .gitmodules
100644 blob 97e3bea37ef238e599134c082bd1ebca865bdb2b    menu.md
160000 commit 2e4ae7032781338fb40d5d24c3d4e709ac2eea22  vendor/lib
```

Recorded in Bash; *ch31-bigrepos/expected-submodule.bash.txt*.

Look at the last line: the submodule is recorded as `160000 commit 2e4ae70...`. A submodule is not a folder of files in the outer repository; it is **a pointer to one commit** of the other repository. That is what "pinning a version" means, and it is why a submodule does not change on its own when the library changes.

Warnings. Submodules add work: after cloning, people must fetch their contents (the option `--recurse-submodules` does it), and updating a submodule is a separate commit in the outer project. They are a common source of confusion; use them when you truly need to pin another repository. The whole life cycle, recorded (the same `-c protocol.file.allow=always` is needed here only because the "remote" is a local folder). The library was added, committed, and then the project was cloned **with** `--recurse-submodules`:

```
$ cd bakery-menu
$ git -c protocol.file.allow=always submodule add -q ../lib-repo vendor/lib
$ git commit -qm "Add the lib submodule"
$ cd ..
$ git -c protocol.file.allow=always clone -q --recurse-submodules bakery-menu bakery-copy
$ cd bakery-copy
$ git submodule status
2e4ae7032781338fb40d5d24c3d4e709ac2eea22 vendor/lib (heads/main)
$ ls vendor/lib
lib.txt
```

Recorded in Bash; *ch31-bigrepos/expected-submodule-flow.bash.txt*.

The clone has the library's files (`lib.txt`) at the pinned commit. Now the library gets a new commit, and the project updates its submodule to the library's newest commit with `git submodule update --remote`:

```
$ cd ../lib-repo
$ printf 'more shared code\n' >> lib.txt
$ git commit -qam "Update lib"
$ cd ../bakery-copy
$ git -c protocol.file.allow=always submodule update --remote vendor/lib
From /home/learner/lib-repo
 2e4ae70..41273bf  main      -> origin/main
Submodule path 'vendor/lib': checked out '41273bf3125415fc50ec5b8cbdb76d5821d39af7'
```

Recorded in Bash; *ch31-bigrepos/expected-submodule-flow.bash.txt*.

The outer project sees that the submodule pointer changed (`M vendor/lib`); the change has to be **committed**, so that the project pins the newer version:

```
$ git status --short
 M vendor/lib
$ git commit -qam "Use the newer lib"
```

Recorded in Bash; *ch31-bigrepos/expected-submodule-flow.bash.txt*.

To **remove** a submodule, unregister it and delete it from the project, then commit:

```
$ git submodule deinit -f vendor/lib
Cleared directory 'vendor/lib'
Submodule 'vendor/lib' (../lib-repo) unregistered for path 'vendor/lib'
$ git rm -q -f vendor/lib
$ git commit -qm "Remove the lib submodule"
$ git ls-files
.gitmodules
menu.md
```

Recorded in Bash; *ch31-bigrepos/expected-submodule-flow.bash.txt*.

`.gitmodules` remains in the list of files (now empty of submodule entries), and the library is gone from the project.

Checked by running it (R206). The clone, update and removal above were run in Bash and zsh (Git 2.43.0; CI on 2.55.0). Git's `git submodule` documentation was checked for the commands `update --remote` and `deinit` (Git 2.56.0). Removing a submodule leaves some history behind (the module's Git data can remain in `.git/modules`); this was not investigated.

31.5 Git LFS: large files

Git stores every version of every file forever (Chapter 30). For large binary files (images, videos, datasets) that becomes heavy. **Git LFS** (Large File Storage) is an extension that keeps the contents of such files on a separate server and stores only a small pointer in the repository.

New term: Git LFS. An extension to Git that stores large files outside the normal history, leaving small text pointers in the repository.

LFS is a separate program (`git-lfs`), which must be installed. To use it, first set it up once, then say which files to handle:

```
$ cd bakery-menu
$ git lfs install
Updated Git hooks.
Git LFS initialized.
$ git lfs track "*.png"
Tracking "*.png"
```

Recorded in Bash; *ch31-bigrepos/expected-lfs.bash.txt*.

`git lfs install` wired LFS into Git. `git lfs track "*.png"` says "treat all PNG files with LFS". The rule is stored in `.gitattributes`:

```
$ cat .gitattributes
*.png filter=lfs diff=lfs merge=lfs -text
```

Recorded in Bash; *ch31-bigrepos/expected-lfs.bash.txt*.

Add a picture (here a fake one) and commit:

```
$ printf 'not really a picture' > logo.png
$ git add .gitattributes logo.png
$ git commit -m "Add logo"
[main a21b8cd] Add logo
2 files changed, 4 insertions(+)
create mode 100644 .gitattributes
create mode 100644 logo.png
```

Recorded in Bash; *ch31-bigrepos/expected-lfs.bash.txt*.

`git lfs ls-files` lists the files that LFS manages. And what does Git store? Ask for the content of that file inside the commit:

```
$ git lfs ls-files
fela4ab1fb * logo.png
$ git cat-file -p HEAD:logo.png
version https://git-lfs.github.com/spec/v1
oid sha256:fela4ab1fb6347036bead73597d5d2958b26ac0b96f68ca2f6dcee53a50f297e
size 20
```

Recorded in Bash; *ch31-bigrepos/expected-lfs.bash.txt*.

The repository holds only a three-line **pointer**: a version, a hash (oid) and a size. The real 20 bytes are kept by LFS, outside the object database. In this recording nothing was sent to a server; a real project would have LFS storage on the remote.

CAUTION. A file committed before LFS tracking is already in the history and stays there. Setting up LFS afterwards does not shrink the repository. And the hosting platform may limit LFS storage or bandwidth, which is time-sensitive information that was not checked (Chapter 36 returns to platform limits).

Checked against the Git LFS specification (R207). The specification (`docs/spec.md` in the Git LFS project) says that pointer files "MUST contain only UTF-8 characters", that "the first key is always version", that the required keys are version (a URL), oid (a hash "prefixed by its hashing method"; "currently, only sha256 is supported") and size (in bytes), and that a pointer must be "less than 1024 bytes". The recorded pointer matches that format. To convert files that are **already in the history**, Git LFS has `git lfs migrate import`; its documentation says that such conversions "rewrite your Git history", so the cautions of Chapter 33 and Chapter 27 apply. No push to an LFS server was run, and hosting-platform limits are covered in the chapters on the platform.

31.6 What is not covered

- **Partial clones** and **subtrees** are run below. **Monorepo** versus **multirepo** and **performance tuning** are design advice, not commands; they are described in general terms and not tested.
- **Monorepo** (one big repository for many projects) versus **multirepo** (many small ones). This is a design decision, not a command. Each choice has costs: a monorepo has one history and one place to look, but grows large; many repositories stay small, but sharing code between them is harder.
- **Performance tuning** of large repositories.

Partial clone

A **partial clone** downloads the commits and trees but leaves out some file contents (blobs), fetching them only when needed. `git clone --filter=blob:none` asks the server for exactly that. The server must allow filters: Git's documentation for `uploadpack.allowFilter` says "if this option is set, `upload-pack` will support partial clone and partial fetch object filtering". Here the local "server" is set up to allow them, and the clone is made by its `file://` address:

```
$ git clone -q --filter=blob:none file://$PWD/hub.git partial
$ cd partial
$ git config remote.origin.partialclonefilter
blob:none
$ git config extensions.partialclone
$ git log --oneline
81f772e (HEAD -> main, origin/main, origin/HEAD) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
$ git cat-file -p HEAD:menu.md
# Sunrise Bakery menu

- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
```

Recorded in Bash; *ch31-bigrepos/expected-partial-clone.bash.txt*.

The clone records the filter in `remote.origin.partialclonefilter` (`blob:none`). The history is complete, and the first time a file's content is needed (here `git cat-file -p HEAD:menu.md`), Git fetches it on demand. Git's design notes call it "a performance optimization for Git that allows Git to function without having a complete copy of the repository", for "extremely large repositories". The documentation for `--filter` lists other forms, such as `blob:limit=<n>` and `object:type=`.

CAUTION. A partial clone depends on the server being reachable later: commands that need a missing file will fetch it, and fail offline.

Subtree

`git subtree` is an alternative to submodules: the other project's files are copied **into** yours, and its history is either merged or squashed. It is a separate command, from Git's `contrib` collection, and is included in many installations (check with `git subtree`). Adding the library as a squashed subtree:

```
$ cd bakery-menu
$ git subtree add -q --prefix=vendor/lib ../lib-repo main --squash
git fetch ../lib-repo main
From ../lib-repo
 * branch          main          -> FETCH_HEAD
$ ls vendor/lib
lib.txt
```

Recorded in Bash; *ch31-bigrepos/expected-subtree.bash.txt*.

The files are now ordinary files of the outer project, not a pointer:

```
$ git log --oneline
75db5cc (HEAD -> main) Merge commit '3747446675b9f9036312d68ab8f0bd7e40fc9c87' as 'vendor/lib'
3747446 Squashed 'vendor/lib/' content from commit 2e4ae70
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch31-bigrepos/expected-subtree.bash.txt*.

The log shows a squashed commit (the library's content at one commit) and a merge commit that puts it under `vendor/lib`. Git's `git-subtree` documentation describes `add`, `merge`, `pull`, `push` and `split`; only `add` was run here. Compared with a submodule, a subtree needs no extra clone step for other people, but the library's history is mixed into your own or squashed.

Checkpoint

What You Learned

- `git worktree add` gives a second working folder on the same repository; `git worktree remove` cleans up.
- A shallow clone (`--depth`) fetches only recent history; `git fetch --unshallow` completes it.
- Sparse checkout limits which folders appear on disk.
- A submodule records a pointer to one commit of another repository, described in `.gitmodules`; Git refuses local-path submodules by default.
- Git LFS keeps large files outside the history and stores small pointer files.

New Vocabulary

- **Worktree**: an extra working folder sharing one repository.
- **Shallow clone**: a clone with only recent history.

- **Submodule:** a repository placed inside another, pinned to one commit.
- **Git LFS:** an extension that stores large files outside the normal history.

Commands Learned

`git worktree add`, `git worktree list`, `git worktree remove`, `git clone --depth`, `git fetch --unshallow`, `git sparse-checkout set`, `git sparse-checkout disable`, `git submodule add`, `git submodule status`, `git lfs install`, `git lfs track`, `git lfs ls-files`.

Common Mistakes

1. **Deleting a worktree folder by hand.**
2. **Expecting `git log` in a shallow clone to show everything.**
3. **Thinking a sparse checkout removes files from the repository.**
4. **Expecting a submodule to update itself.**
5. **Adding LFS after a large file is already in the history.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What do two worktrees share, and what does each have of its own?
2. Why does `git log` show grafted in a shallow clone?
3. What does sparse checkout change: the repository or the folder on disk?
4. What does a submodule record in the outer repository?
5. What does Git store for a file tracked by LFS?

Before Moving On

You are ready for Chapter [32](#) if you can:

- [] create and remove a worktree
- [] make a shallow clone and complete it
- [] limit a working folder with sparse checkout
- [] explain what a submodule and an LFS pointer are

How this chapter was checked

Claim	Evidence class	Ledger
Worktree add, list, remove	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-clone</code> , <code>git-sparse-checkout</code> , <code>git-worktree</code>); the ledger row says which.	R203

Claim	Evidence class	Ledger
Shallow clone, unshallow; sparse checkout	Locally tested (as above), with local bare repositories. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-clone, git-sparse-checkout, git-worktree); the ledger row says which.	R203
Submodule add, the file transport refusal, .gitmodules, mode 160000	Locally tested (as above); the reason and the release that introduced the rule checked against the Git documentation and the 2.30.6 release notes	R205
Git LFS install, track, pointer file	Locally tested with git-lfs 3.4.1 (CI: runner's git-lfs), and the pointer format checked against the Git LFS specification; no LFS server	R207
Partial clone (--filter=blob:none) and git subtree add	Locally tested (as above); checked against the Git 2.56.0 documentation	R204
Monorepo versus multirepo; performance tuning	General advice, not tested	R204
Submodule clone (--recurse-submodules), update --remote, deinit and removal	Locally tested (as above)	R206

Where this leads

Chapter 32 customises Git's behaviour. Chapter 54 shows shallow clones in automation, and Chapter 36 returns to hosting-platform limits.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use bakery-menu (rebuild it from Chapter 17 if needed), and local bare repositories as remotes (Chapter 23). For Level 4, git-lfs must be installed.

Level 1 — Guided

1.1 A second folder. Run `git worktree add ../bakery-hotfix -b hotfix`. What does `git worktree list` show?

1.2 Work in it. Commit a change in the new folder, then look at `git log --oneline --all --graph` from the first folder.

Level 2 — Partially guided

2.1 Clean up. Remove the worktree with `git worktree remove`. Does the hotfix branch still exist?

2.2 A shallow clone. Make a bare copy with `git clone --bare`, then clone it with `--depth 1` using a `file://` address. How many commits does `git log` show?

Level 3 — Independent

3.1 Complete it. Run `git fetch --unshallow` and count the commits again. Check `git rev-parse --is-shallow-repository` before and after.

3.2 Sparse. Add `docs` and `src` folders and commit. Use `git sparse-checkout set docs`, then `ls`, then `git sparse-checkout disable`.

Level 4 — Professional scenario

4.1 Pointer files. Set up `git lfs track "*.png"`, add a small fake `logo.png`, and commit. Use `git cat-file -p HEAD:logo.png` to read what Git stored. What is in it, and where is the real file?

4.2 A submodule. Create a second repository `lib-repo` with one commit. Add it as a submodule of `bakery-menu` at `vendor/lib`. Read what Git says when it refuses, and explain why you may use `-c protocol.file.allow=always` for this demonstration but should not set it permanently.

Level 5 — Troubleshooting

5.1 A learner deleted the extra worktree folder with `rm -r` and now `git worktree list` still shows it. What should they do?

5.2 A learner made a shallow clone and `git blame` stops at the first commit. Why, and what is the fix?

5.3 A learner added a 300 MB video, then set up LFS. The repository is still huge. Why?

Chapter 32 — Customising Git [Deep]

In this chapter

- **aliases:** your own short names for commands
- **hooks:** scripts that Git runs at certain moments
- **attributes:** per-file rules such as line endings
- **credential helpers:** how Git remembers passwords, and the risk in that
- **rerere:** teach Git to remember how you resolved a conflict

Deep. This chapter is marked **[Deep]**. You can skip it on a first read. It changes how Git behaves for you, and each feature works best once you know the plain behaviour well.

Before you start. Chapter 14 (scopes) and Chapter 30. The recordings ran in Bash and zsh on Git 2.43.0 and were re-run in CI on Git 2.55.0.

32.1 Aliases

An **alias** is a shortcut that you define in your configuration (Chapter 14).

New term: alias. A name you define in Git's configuration as a shortcut for another command, with its options.

Four aliases: three shortenings of Git commands and one that runs a shell command (a leading !):

```
$ cd bakery-menu
$ git config --global alias.st "status --short"
$ git config --global alias.lg "log --oneline --graph --all"
$ git config --global alias.last "log -1 --stat"
$ git config --global alias.hello '!echo hello from a shell alias'
$ printf -- '- Tea: 1.50\n' >> menu.md
```

Recorded in Bash; *ch32-custom/expected-alias.bash.txt*.

Use them like built-in commands:

```
$ git st
M menu.md
$ git commit -qam "Add tea"
```

Recorded in Bash; *ch32-custom/expected-alias.bash.txt*.

```
$ git lg
* 31cb2c3 (HEAD -> main) Add tea
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch32-custom/expected-alias.bash.txt*.

```
$ git last
commit 31cb2c3316e22b96397393788023fb0b98ff2c6d (HEAD -> main)
Author: Ada Learner <ada@example.org>
Date:   Mon Jan 5 09:14:00 2026 +0000

    Add tea

menu.md | 1 +
1 file changed, 1 insertion(+)
```

Recorded in Bash; *ch32-custom/expected-alias.bash.txt*.

```
$ git hello
hello from a shell alias
```

Recorded in Bash; *ch32-custom/expected-alias.bash.txt*.

`git help st` explains an alias, and the configuration lists all of them:

```
$ git help st
'st' is aliased to 'status --short'
$ git config --get-regexp '^alias\.'
alias.st status --short
alias.lg log --oneline --graph --all
alias.last log -1 --stat
alias.hello !echo hello from a shell alias
```

Recorded in Bash; *ch32-custom/expected-alias.bash.txt*.

Aliases are stored with `--global` in your user configuration, so they follow you across repositories. Keep them few and simple. A short alias for a command you type all the time is useful; an alias that hides what Git does makes your own history harder to read, and makes your instructions to other people unclear. **In this book, every command is typed in full.**

32.2 Hooks

A **hook** is a script that Git runs automatically at a fixed moment: before a commit, after a merge, before a push.

New term: hook. A script in the `.git/hooks` folder (or in a configured folder) that Git runs at a specific moment, for example before it creates a commit. A hook that exits with a failure status stops the operation.

This hook, `pre-commit`, refuses a commit if `menu.md` contains the word `TODO`. It is a small shell script, written with `printf`, and made executable:

```
$ cd bakery-menu
$ printf '#!/bin/sh\nif grep -q TODO menu.md; then\n  echo "pre-commit: remove the TODO\n  before committing"\n  exit 1\nfi\n' > .git/hooks/pre-commit
$ chmod +x .git/hooks/pre-commit
```

Recorded in Bash; *ch32-custom/expected-hook.bash.txt*.

Now try to commit a file with a `TODO` in it:

```
$ printf 'TODO fix the price\n' >> menu.md
$ git commit -am "Add a note"; echo "exit status: $?"
pre-commit: remove the TODO before committing
exit status: 1
```

Recorded in Bash; *ch32-custom/expected-hook.bash.txt*.

The hook printed its message and exited with status 1, so **no commit was made**: the change is still waiting, and the log is unchanged:

```
$ git status --short
M menu.md
$ git log --oneline
81f772e (HEAD -> main) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch32-custom/expected-hook.bash.txt*.

If you must skip a hook, `--no-verify` does so:

```
$ git commit --no-verify -am "Add a note"
[main 08f6ccc] Add a note
1 file changed, 1 insertion(+)
$ git log --oneline
08f6ccc (HEAD -> main) Add a note
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

Recorded in Bash; *ch32-custom/expected-hook.bash.txt*.

Hooks are a real convenience (checking formatting, running tests, catching secrets before they are committed, Chapter 33) with three limits:

1. They are **not copied when someone clones** the repository. Git does not send hooks, for safety:

```
$ git clone -q . ../copy
$ test -e ../copy/.git/hooks/pre-commit && echo "hook copied" || echo "hook not copied"
hook not copied
```

Recorded in Bash; *ch32-custom/expected-hook.bash.txt*.

1. They can be skipped (`--no-verify`), so they are a help and not a security barrier. Anything that must be enforced needs a check on the server too (Chapter 54).
2. They run **code on your computer**. Never copy a hook from a source that you do not trust.

More hooks

Git's githooks documentation (Git 2.56.0) lists many more hooks than pre-commit. On the **commit** side: pre-commit, prepare-commit-msg, commit-msg and post-commit; around **merges and rebases**: pre-merge-commit, post-merge, pre-rebase, post-rewrite; around **checkouts and pushes**: post-checkout and pre-push; and on the **server** that receives a push: pre-receive, update, proc-receive and post-receive, among others. The documentation says that pre-commit and commit-msg "can be bypassed with the `--no-verify` option", and that a non-zero exit status makes the command abort.

The commit-msg hook receives the name of the file that holds the proposed message and may refuse it. This one rejects messages shorter than twelve characters:

```
$ cd bakery-menu
$ printf '#!/bin/sh\nif [ "$(wc -c < "$1")" -lt 12 ]; then\n  echo "commit-msg: message is too short"\n  exit 1\nfi\n' > .git/hooks/commit-msg
$ chmod +x .git/hooks/commit-msg
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Tea"; echo "exit status: $?"
commit-msg: message is too short
exit status: 1
```

Recorded in Bash; *ch32-custom/expected-commit-msg-hook.bash.txt*.

The short message was refused; a longer one passes:

```
$ git commit -am "Add tea to the menu"
[main aled54d] Add tea to the menu
1 file changed, 1 insertion(+)
$ git log --oneline -2
aled54d (HEAD -> main) Add tea to the menu
81f772e Add coconut cake
```

Recorded in Bash; *ch32-custom/expected-commit-msg-hook.bash.txt*.

Hooks on the server belong to whoever runs the server. A hosting platform offers its own mechanisms instead (Chapter 60 and Chapter 50).

32.3 Attributes: rules per file

A file named `.gitattributes` gives rules for files that match a pattern. It is part of the project, so it is shared with everyone. Here: text files ending in `.md` are text with LF line endings, and `.bin` files are binary:

```
$ cd bakery-menu
$ printf '*.md text eol=lf\n*.bin binary\n' > .gitattributes
$ git add .gitattributes
$ git commit -m "Add attributes"
[main 8776a78] Add attributes
1 file changed, 2 insertions(+)
create mode 100644 .gitattributes
```

Recorded in Bash; *ch32-custom/expected-attributes.bash.txt*.

Ask Git which attributes apply to a file:

```
$ git check-attr -a menu.md
menu.md: text: set
menu.md: eol: lf
```

Recorded in Bash; *ch32-custom/expected-attributes.bash.txt*.

```
$ git check-attr -a logo.bin
logo.bin: binary: set
logo.bin: diff: unset
logo.bin: merge: unset
logo.bin: text: unset
```

Recorded in Bash; *ch32-custom/expected-attributes.bash.txt*.

`binary` is a shortcut that switches off text handling: `text`, `diff` and `merge` become `unset`. Line endings were the subject of Chapter 4: Windows files often end lines with two characters (CR and LF), Linux and macOS with one (LF). With the `.gitattributes` rule above, a file with Windows endings is stored with LF:

```
$ printf 'one\r\ntwo\r\n' > notes.md
$ git add notes.md
warning: in the working copy of 'notes.md', CRLF will be replaced by LF the next time Git
touches it
$ git ls-files --eol notes.md
i/lf    w/crlf  attr/text eol=lf      notes.md
```

Recorded in Bash; *ch32-custom/expected-attributes.bash.txt*.

The warning says exactly this: the working copy has CRLF (w/crlf), the copy that Git stores in the index has LF (i/lf), and the attribute in force is text eol=lf. `git ls-files --eol` shows this for any file, which is the quickest way to debug a line-ending problem.

Checked against the documentation (Git 2.56.0, R210). The text attribute "marks the path as a text file, which enables end-of-line conversion": line endings are "normalized to LF in the index" when a file is added, and may be converted when it is copied back to the working directory. Specifying `eol` "automatically sets text if text was left unspecified". If text is unspecified, Git uses the setting `core.autocrlf` to decide: `true` "is the same as setting the text attribute to auto on all files and `core.eol` to `crlf`" (for people who want CRLF in their working folder and LF in the repository), and `input` performs no conversion on output. An attribute in `.gitattributes` is shared by everyone who clones, which is why it is usually better than a per-user `core.autocrlf`. The wording of the warning above was the same on Git 2.43.0 and 2.55.0.

32.4 Credential helpers

When Git talks to a remote that needs a password or a token (Chapter 23, Chapter 38), it asks a **credential helper** to supply it, or to remember it.

New term: credential helper. A program that Git asks to look up, store or delete a username and password (or token) for a remote address.

Git ships a simple helper called `store`, which writes credentials to a file. It is used here only to show how the mechanism works, with a made-up credential (`not-a-real-password`), and a file in the home folder:

```
$ git config --global credential.helper 'store --file ~/creds'
$ printf 'protocol=https\nhost=example.org\nusername=ada\npassword=not-a-real-password\n\n'
| git credential approve
$ cat ~/creds
https://ada:not-a-real-password@example.org
```

Recorded in Bash; *ch32-custom/expected-credential.bash.txt*.

`git credential approve` tells the helper to remember a credential. Then the file `~/creds` has this content: **the password, in plain text, in a URL**. Anyone who can read the file can read the password.

CAUTION. The `store` helper keeps passwords and tokens **unencrypted** on disk. Do not use it for real credentials. Use the helper that your operating system or your Git installer provides, which keeps secrets in the system's protected store (the names differ between systems, and this chapter did not test them). Chapter 33 covers credentials in depth.

`git credential fill` asks the helper for a credential, the way Git does internally:

```
$ printf 'protocol=https\nhost=example.org\n\n' | git credential fill
protocol=https
host=example.org
username=ada
password=not-a-real-password
```

Recorded in Bash; *ch32-custom/expected-credential.bash.txt*.

And `git credential reject` tells the helper to forget it. The file is empty afterwards:

```
$ printf 'protocol=https\nhost=example.org\nusername=ada\npassword=not-a-real-password\n\n'
| git credential reject
$ cat ~/.creds; echo "creds file is now empty"
creds file is now empty
```

Recorded in Bash; *ch32-custom/expected-credential.bash.txt*.

Checked against the documentation (Git 2.56.0, R211). Git's `gitcredentials` documentation lists the helpers that come with Git or are recommended: `cache` (credentials kept "in memory for a short period of time"), `store` ("store credentials indefinitely on disk"; the documentation calls it "discouraged" in its example), and, for particular systems, `git-credential-libsecret` (Linux), `git-credential-osxkeychain` (macOS), `git-credential-wincred` (Windows), and **Git Credential Manager** ("cross platform, included in Git for Windows"). The `store` helper's own page states that it "will store your passwords unencrypted on disk", and that its `.git-credentials` file "is stored in plaintext". How each of the others stores its secrets was not tested here.

32.5 rerere: remember conflict resolutions

Chapter 22 and Chapter 27 showed that you can resolve the same conflict more than once (for example, when you repeat a merge or rebase). `rerere` stands for "**re**use **re**corded **re**solution".

New term: `rerere`. A Git feature that records how you resolved a conflict and, when the same conflict appears again, applies your earlier resolution automatically.

Turn it on for the repository, then create the familiar conflict (both branches change the price of the loaf) and merge:

```
$ cd bakery-menu
$ git config rerere.enabled true
$ git switch -c raise-bread
Switched to a new branch 'raise-bread'
$ printf '# Sunrise Bakery menu\n\n White loaf: 3.00\n- Rolls (six): 3.00\n- Coconut cake
(slice): 4.00\n' > menu.md
$ git commit -qam "Raise the white loaf to 3.00"
$ git switch -q main
$ printf '# Sunrise Bakery menu\n\n White loaf: 2.90\n- Rolls (six): 3.00\n- Coconut cake
```

```
(slice): 4.00\n' > menu.md
$ git commit -qam "Raise the white loaf to 2.90"
$ git merge raise-bread
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
Recorded preimage for 'menu.md'
Automatic merge failed; fix conflicts and then commit the result.
```

Recorded in Bash; *ch32-custom/expected-rerere.bash.txt*.

Notice the new line: Recorded preimage for 'menu.md'. Git kept a copy of the conflicted file. Resolve it as usual, and commit:

```
$ printf '# Sunrise Bakery menu\n\n- White loaf: 2.95\n- Rolls (six): 3.00\n- Coconut cake (slice): 4.00\n' > menu.md
$ git add menu.md
$ git commit -m "Merge raise-bread, settling on 2.95"
Recorded resolution for 'menu.md'.
[main a87e5b1] Merge raise-bread, settling on 2.95
```

Recorded in Bash; *ch32-custom/expected-rerere.bash.txt*.

Recorded resolution for 'menu.md'. Git now knows how you fixed it. Undo the merge, to simulate a repeated merge, and try again:

```
$ git reset --hard HEAD~1
HEAD is now at 5dc50d3 Raise the white loaf to 2.90
$ git merge raise-bread
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
Resolved 'menu.md' using previous resolution.
Automatic merge failed; fix conflicts and then commit the result.
```

Recorded in Bash; *ch32-custom/expected-rerere.bash.txt*.

The conflict is reported again (CONFLICT (content)), but then: **Resolved 'menu.md' using previous resolution**. Look at the file:

```
$ git status --short
UU menu.md
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.95
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
$ git rerere status
```

Recorded in Bash; *ch32-custom/expected-rerere.bash.txt*.

The file already holds 2.95, without markers. The status still says UU (unmerged), because Git leaves the final decision to you: check the result, `git add menu.md` and commit.

rerere is most useful when you rebase the same branch many times, or when you keep merging in a long-lived branch.

32.6 What is not covered

- **Custom merge drivers**, and git maintenance start (which installs a schedule on your computer).
 - **Server-side hooks**: listed above, but not run.
 - **Credential helpers other than store**, which depend on your operating system.
-

Checkpoint

What You Learned

- Aliases are shortcuts defined in configuration; a leading ! runs a shell command.
- A hook is a script that runs at a fixed moment; a failing pre-commit stops the commit; hooks are not cloned and can be skipped with --no-verify.
- .gitattributes sets per-file rules such as text eol=lf and binary; git check-attr and git ls-files --eol show what applies.
- A credential helper stores and returns passwords; the store helper keeps them in plain text.
- rerere remembers a conflict resolution and applies it when the same conflict returns.

New Vocabulary

- **Alias**: a configured shortcut for a Git command.
- **Hook**: a script that Git runs at a specific moment.
- **Credential helper**: a program that stores and supplies credentials.
- **rerere**: a feature that reuses recorded conflict resolutions.

Commands Learned

git config alias.<name>, git help <alias>, git commit --no-verify, git check-attr, git ls-files --eol, git credential approve|fill|reject, git config rerere.enabled true, git rerere status.

Common Mistakes

1. **Aliases that hide what Git does.**
2. **Relying on a hook to enforce a rule.**
3. **Copying a hook from an untrusted source.**
4. **Storing real credentials with credential.helper store.**
5. **Committing a resolution that rerere applied without reading it.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What does a leading `!` mean in an alias?
2. Why is a hook not a security barrier?
3. What does `git ls-files --eol` tell you?
4. Why is `credential.helper store` unsafe for real passwords?
5. What does `rerere` record, and when does it act?

Before Moving On

You are ready for Chapter 33 if you can:

- [] define and use an alias
- [] write a simple pre-commit hook and bypass it
- [] read `git check-attr` and `git ls-files --eol`
- [] explain the risk of the `store` helper

How this chapter was checked

Claim	Evidence class	Ledger
Aliases, including a <code>!</code> alias and <code>git help <alias></code>	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0. The manual was not consulted for aliases	R208
pre-commit and commit-msg hooks block a commit; <code>--no-verify</code> ; hooks not cloned	Locally tested (as above); hook list checked against the githooks documentation (Git 2.56.0)	R209
<code>.gitattributes</code> , <code>check-attr</code> , <code>ls-files --eol</code> , CRLF warning	Locally tested (as above); text, eol and <code>core.autocrlf</code> checked against the documentation	R210
<code>credential approve/fill/reject</code> with the store helper	Locally tested (as above), made-up credential only; the list of helpers and the plain-text warning checked against the documentation	R211
<code>rerere</code> recording and reuse	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-rerere</code>); the ledger row says which.	R208
<code>merge -X theirs</code> , <code>merge -s ours</code> , <code>maintenance run --task=commit-graph</code> and <code>--task=gc</code>	Locally tested (as above); the task list checked against the git maintenance documentation	R212
Merge drivers, <code>maintenance start</code> , server-side hooks, OS credential helpers	Not run	R212

Where this leads

Chapter 33 returns to secrets and credentials. Chapter 54 shows how automation on a server covers what a local hook cannot enforce.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use bakery-menu (rebuild it from Chapter 17 if needed). Use only made-up credentials, and remove any file you create for them.

Level 1 — Guided

1.1 An alias. Define `alias.st` as `status --short` and use `git st`. Where is the alias stored?

1.2 Explain it. Run `git help st`. What does Git print?

Level 2 — Partially guided

2.1 A shell alias. Define an alias that starts with `!` and prints a message. Run it.

2.2 List them. List all your aliases with `git config --get-regexp '^alias\.'`.

Level 3 — Independent

3.1 A hook. Write a pre-commit hook that refuses a commit when `menu.md` contains `TODO`. Make it executable and test it. What exit status does it use, and what does the failed commit leave in the working folder?

3.2 Skip it and clone it. Commit with `--no-verify`. Then clone the repository and check whether the hook came with it.

Level 4 — Professional scenario

4.1 Line endings. Add `.gitattributes` with `*.md text eol=lf`. Create a file with CRLF line endings, add it, and read `git ls-files --eol`. Explain each column.

4.2 rerere. Turn on `rerere`, cause a conflict, resolve it and commit. Undo the merge with `git reset --hard HEAD~1` and merge again. What does Git say, and why is the status still `uu`?

Level 5 — Troubleshooting

5.1 A team wants to guarantee that no commit contains `TODO`. Is a pre-commit hook enough? Why not?

5.2 A colleague set `credential.helper store` and used a real token. What is the risk, and what should they do?

5.3 A learner copied a pre-commit hook from a forum post and it runs on every commit. What is the risk?

Chapter 33 — Git Security [Core]

In this chapter

- why a secret committed once is a secret leaked
- keeping secrets out of a repository in the first place
- what removing a secret from history does, and does not, achieve
- signing commits, and what a signature proves
- the risks of forced pushes, and of code you did not write
- an incident checklist

Before you start. Chapter 18 (`.gitignore` and the demonstration of a committed secret), Chapter 26, Chapter 23 and Chapter 6. The recordings ran in Bash and zsh on Git 2.43.0 and were re-run in CI on Git 2.55.0. **Every secret in this book is made up** (`not-a-real-key`).

33.1 The core lesson

Chapter 30 showed that Git keeps every version of every file, and that a commit's name depends on everything before it. That is what makes Git reliable. It is also why **a secret that was ever committed stays in the history**, even after you delete the file.

New term: secret. Any value that gives access if someone else has it: a password, a token, an API key, a private key, or a file that contains them.

Here a made-up key is committed in a file called `.env`, and then removed in the next commit:

```
$ cd bakery-menu
$ printf 'API_KEY=not-a-real-key\n' > .env
$ git add .env
$ git commit -m "Add config"
[main 50ff373] Add config
 1 file changed, 1 insertion(+)
 create mode 100644 .env
$ git rm -q .env
$ git commit -m "Remove the env file"
[main 1cadcf4] Remove the env file
 1 file changed, 1 deletion(-)
 delete mode 100644 .env
```

Recorded in Bash; *ch33-security/expected-history-rewrite.bash.txt*.

The latest commit removes the file. The history keeps it:

```
$ git log --oneline
1cadcf4 (HEAD -> main) Remove the env file
50ff373 Add config
81f772e Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
```

```
$ git show HEAD~1:.env
API_KEY=not-a-real-key
$ git log --all --oneline -S"not-a-real-key"
1cadcf4 (HEAD -> main) Remove the env file
50ff373 Add config
```

Recorded in Bash; *ch33-security/expected-history-rewrite.bash.txt*.

`git show HEAD~1:.env` prints the key from the earlier commit, and `git log -S` (Chapter 24) finds both commits that touch it. Anyone who has a copy of this repository has the secret. Deleting the file changed nothing about that.

CAUTION. If a real secret reaches a repository that anyone else can read, assume that it is **compromised**. The first step is **not** to clean the history. It is to **revoke or rotate the secret** (make it stop working, and issue a new one) at the place that issued it. Only then is cleaning worth considering.

33.2 Keeping secrets out

Prevention is much cheaper than cleanup.

Keep secrets in files that Git ignores. A `.env` file is a common place for settings. Put its name in `.gitignore` (Chapter 18):

```
$ cd bakery-menu
$ printf '.env\n' > .gitignore
$ printf 'API_KEY=not-a-real-key\n' > .env
$ git status --short
?? .gitignore
$ git check-ignore -v .env
.gitignore:1:.env      .env
```

Recorded in Bash; *ch33-security/expected-ignore.bash.txt*.

The file exists, but `git status` does not list it, and `git check-ignore -v` names the rule that hides it. Git even refuses a normal `git add`:

```
$ git add .env
The following paths are ignored by one of your .gitignore files:
.env
hint: Use -f if you really want to add them.
hint: Turn this message off by running
hint: "git config advice.addIgnoredFile false"
```

Recorded in Bash; *ch33-security/expected-ignore.bash.txt*.

(On newer Git versions, 2.55.0 was tested, the last two hint lines are replaced by one: *hint: Disable this message with "git config set advice.addIgnoredFile false"*. The refusal is the same.)

But **-f overrides the rule**, and nothing stops you:

```
$ git add -f .env
$ git status --short
A .env
?? .gitignore
```

Recorded in Bash; *ch33-security/expected-ignore.bash.txt*.

So `.gitignore` is a safety net for mistakes, not a lock. Add a few more habits:

- **Read git status and git diff --staged before every commit** (Chapter 19).
- **Keep example files without real values** (for example `.env.example`) in the repository, and the real one out of it.
- **A hook** can check for obvious secrets before a commit (Chapter 32), but it can be skipped, so it does not replace the habits above.
- **Give each token the least power that works**, and a short life. A stolen token with little power does little harm.

Checked against GitHub's documentation (R214). GitHub's secret-scanning documentation says it "scans your entire Git history on all branches of your repository for hardcoded credentials", raises an alert, and advises: "rotate the affected credential immediately"; removing a secret from the history "is time-intensive and often unnecessary if you've already revoked the credential". **Push protection** "blocks pushes that contain secrets before they reach your repository", including command-line pushes; for repositories it "is disabled by default" and "requires GitHub Secret Protection to be enabled", while push protection for users "is enabled by default" and stops you from pushing secrets to public repositories. On branches, GitHub's documentation says a protected-branch rule blocks force pushes and deletion by default. Which plans include each feature is time-sensitive and was not checked. Chapter 62 covers it.

33.3 If it already happened

The order of steps matters.

1. **Revoke or rotate the secret** at its source. Do this first, and do it even if you think nobody saw it. From this point the leaked value is useless.
2. **Find out where it went:** which branches, which remotes, which forks or copies, and which builds or logs.
3. **Only then, remove it from history** if you need to. Removal does not un-leak the secret. Its purpose is to stop it from spreading and to make the repository clean.

For step 3, Git's own older command for rewriting history (`git filter-branch`) has a long list of problems, and Git's documentation recommends a separate tool, **git-filter-repo**. It is not part of Git; it is a script that you install separately (this book used version 2.47.0, from the Python package index). In the example below, it removes every trace of `.env`:

```
$ git filter-repo --path .env --invert-paths --force
Parsed 5 commits
New history written in <n> seconds; now repacking/cleaning...
Repacking your repo and cleaning out old unneeded objects
HEAD is now at 81f772e Add coconut cake
Completely finished after <n> seconds.
```

Recorded in Bash; *ch33-security/expected-history-rewrite.bash.txt*.

It reported the commits it processed, and then it rewrote history. Look at the result:

```
$ git log --oneline
81f772e (HEAD -> main) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
$ git log --all --oneline -S"not-a-real-key"
```

Recorded in Bash; *ch33-security/expected-history-rewrite.bash.txt*.

The two commits that touched `.env` are **gone**, not just the file. The old ones, and the ones after them, all have **new hashes** (here there were none after them, and the earlier ones are unchanged because they come before the change). The key is no longer found in any commit (the search prints nothing):

```
$ git log --all --oneline -S"not-a-real-key"
```

Recorded in Bash; *ch33-security/expected-history-rewrite.bash.txt*.

Checked against the documentation (R215). The `git filter-branch` manual page (Git 2.56.0) opens with a WARNING: the command "has a plethora of pitfalls", its "safety and performance issues cannot be backward compatibly fixed", "its use is not recommended", and it asks readers to "use an alternative history filtering tool such as `git filter-repo`". The `git-filter-repo` manual (version 2.47.0) says the tool aborts "if run from a repo that is not a fresh clone", and that by default it removes the `origin` remote (its `--force` option disables that check). The tool here was run only on a repository with no remote; on a repository with a remote, the rewritten history has to be pushed, which is a forced push (section 33.5).

What a rewrite does **not** do:

- It does **not** remove copies that other people already have: clones, forks, and downloads.
- It does **not** remove the secret from a hosting platform's caches, or from pull requests or other places that the platform keeps. Whether and how the platform removes it is time-sensitive and was not verified.
- It does **not** make the secret safe again. Only step 1 does.

That is why the order is **revoke first**.

33.4 Signing commits

Anyone can write any name and email into a commit (Chapter 14). Nothing in Git stops you from committing as someone else. A **signature** adds proof that the commit was made by the holder of a particular key.

New term: signed commit. A commit that carries a cryptographic signature made with the author's private key, so that others can check it against the matching public key.

The recording uses **GPG** (a widely installed signing tool) with a key created just for the demonstration. One commit is signed with `-S`, and one is not:

```
$ cd bakery-menu
$ gpg --batch --passphrase '' --quick-gen-key "Ada Learner <ada@example.org>" default
default never 2>/dev/null; echo "key created: $"
key created: 0
$ git config user.signingkey "ada@example.org"
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -q -S -am "Add tea (signed)"
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -q -am "Add coffee (unsigned)"
```

Recorded in Bash; *ch33-security/expected-sign.bash.txt*.

Ask Git whether the signatures are good, with the format code `%G?`:

```
$ git log --format='%G? %GS: %s' -3
N : Add coffee (unsigned)
G Ada Learner <ada@example.org>: Add tea (signed)
N : Add coconut cake
```

Recorded in Bash; *ch33-security/expected-sign.bash.txt*.

G means a good signature, made by the key of Ada Learner. N means no signature. (Other letters exist for bad or unverifiable signatures. Only G and N were run here.)

Two things a signature proves, and two that it does not:

- It **proves** that the holder of the key made the commit, and that the commit was not changed afterwards.
- It does **not** prove that the person is who they say (that depends on how you came to trust the key), and it does not prove that the code is safe.

Chapter 14 showed the other side: what happens if signing is switched on but the key is unavailable. Git can also sign with an SSH key; Chapter 38 runs that (with a demonstration key). The way hosting platforms show a "verified" mark was **not** run.

Checked against GitHub's documentation (R216). GitHub marks a commit or tag with a verifiable GPG, SSH or S/MIME signature "Verified" (or "Partially verified"); the default statuses are Verified, Unverified and none for an unsigned commit; and "vigilant mode" is off by default. SSH-key signing on the Git side is in Chapter 38, with the platform side.

33.5 Force pushes

Chapter 23 showed a rejected push and warned against forcing it. Here is the reason.

A **forced push** (`git push --force`) makes the remote branch match yours, even if that discards commits that others have pushed. It is what you must do after rewriting history that is already shared (Chapter 27, section 33.3). Consequences:

- Other people's work can be **lost** from the shared branch.
- Everyone who already has the old history now has a history that **conflicts** with the new one.
- Hosting platforms can **protect** branches against forced pushes. (Whether and how was not verified for this chapter; see Chapter 62.)

Rules of thumb: never force-push to a shared branch without agreement; use a forced push only on your own branches; and prefer `git revert` (Chapter 25) for shared history.

33.6 Code you did not write

Whenever you clone a repository, install a package, or copy a script, you run or trust **someone else's code**.

- **Hooks** (Chapter 32) and scripts in a cloned repository run with your permissions. Read them first.
- **Submodules** (Chapter 31) pull code from another address. Git refuses local-path submodules by default, as the recording in that chapter showed.
- **Dependencies** (packages) can change, or be replaced by malicious versions. Pin versions, and review what you add. (Ways to do this on a hosting platform are in Chapter 62.)

Verification pending [R217]. This section states general good practice, not tested facts. **No real-world incident is named in this chapter:** none was researched and verified from a primary source, and the book does not repeat incidents from memory. If a later revision adds incident examples, each must be verified and cited.

33.7 Incident checklist: "I committed a secret"

1. **Stop.** Do not try to hide it by deleting the file.

2. **Revoke or rotate** the secret at its source.
 3. **Check for misuse:** look at the service's access logs for uses that were not yours.
 4. **Find every copy:** branches, remotes, forks, clones, builds, and pasted messages.
 5. **Decide whether to rewrite history.** If you do, agree it with everyone who shares the repository, use a maintained tool, and expect to force-push.
 6. **Tell the people who need to know,** following your organization's rules.
 7. **Add a prevention:** .gitignore, an example file, a hook or a scanner, and a note in the project's documentation.
-

Checkpoint

What You Learned

- A committed secret stays in history after the file is deleted; the first response is to revoke it.
- .gitignore keeps mistakes out, but `git add -f` overrides it; read what you stage.
- `git-filter-repo` can rewrite history to remove a file entirely; the rewrite does not remove copies that others have, and does not make the secret safe.
- A signed commit proves which key made it (%G? shows G for good, N for none); it does not prove trust or safety.
- A forced push overwrites shared history and can lose others' work.
- Hooks, submodules and dependencies are other people's code running with your permissions.

New Vocabulary

- **Secret:** a value that gives access if someone else has it.
- **Signed commit:** a commit carrying a cryptographic signature that others can verify.

Commands Learned

```
git log -S, git show <commit>:<path>, git check-ignore -v, git add -f, git filter-repo --path
<path> --invert-paths, git commit -S, git log --format=%G?
```

Common Mistakes

1. **Cleaning the history before revoking the secret.**
2. **Believing that deleting a file removes its history.**
3. **Using `git add -f` to get past a .gitignore warning.**
4. **Force-pushing a shared branch.**
5. **Running a hook or script that was not read first.**

Practice

Do the exercises in the Exercises section at the end of this chapter. Use only made-up secrets.

Self-Test

1. A file with a secret was deleted in the last commit. Is the secret safe? Why not?
2. What is the first step after committing a real secret?
3. What does `git check-ignore -v .env` show?
4. What does `%G` print for a good signature, and for an unsigned commit?
5. Name two things that rewriting history does not fix.

Before Moving On

You are ready for Chapter 34 if you can:

- [] explain why deleting a committed secret is not enough
- [] state the order: revoke, find, then clean
- [] keep a `.env` file out of a repository
- [] say what a signature proves and what it does not

How this chapter was checked

Claim	Evidence class	Ledger
A deleted secret is still in history; <code>git log -S</code> finds it	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0	R213
<code>.gitignore</code> , <code>check-ignore</code> , add refusal and <code>-f</code>	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>gitignore</code>); the ledger row says which.	R213
<code>git filter-repo</code> removes a path from all history (no remote); <code>filter-branch</code> not recommended	Locally tested with <code>git-filter-repo</code> 2.47.0 (also installed in CI); warning text and fresh-clone rule checked in the manuals	R215
GPG-signed commit shows G; unsigned shows N	Locally tested with GnuPG and a throw-away key	R216
Platform secret scanning, push protection, protected branches, "Verified" marks	Checked against <code>github/docs</code> (commit 2eaab0b); plans and prices not checked; not run on a live account	R214, R216
SSH-key signing	Run in Chapter 38 (Git side only)	R236
Supply-chain and incident guidance	General practice, not tested ; no incident named	R217

Where this leads

Chapter 34 shows how teams organise their branches. Chapter 38 covers tokens and SSH keys on the platform, and Chapter 62 returns to secret scanning, protected branches and dependency security, once their facts are verified.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. **Use only made-up secrets, in a practice repository that you never push anywhere.** For Level 3 you need `git-filter-repo` (a separate program); for Level 4 you need GnuPG.

Level 1 — Guided

1.1 Leave a trace. In a practice repository, commit a file `.env` containing `API_KEY=not-a-real-key`, then delete it and commit the deletion. Use `git show HEAD~1:.env` to read the key.

1.2 Find it. Use `git log --all --oneline -S"not-a-real-key"`. Which commits does it list, and why both?

Level 2 — Partially guided

2.1 Prevention. Put `.env` in `.gitignore`, recreate the file, and run `git status --short` and `git check-ignore -v .env`. What do they show?

2.2 The override. Try `git add .env`, then `git add -f .env`. What is the difference? Why is this a weakness of `.gitignore`?

Level 3 — Independent

3.1 Rewrite. In the practice repository from 1.1, run `git filter-repo --path .env --invert-paths --force`. What happened to the two commits and to the hashes?

3.2 What did it not fix? List three things that the rewrite did not undo, even if the repository had been shared.

Level 4 — Professional scenario

4.1 The order of steps. A colleague has just pushed a real token to a shared repository. Put these steps in the right order and say why: rewrite history; tell the team; revoke the token; check the service's logs; add a prevention.

4.2 Sign a commit. Create a throw-away GPG key, configure `user.signingkey`, make one signed and one unsigned commit, and show them with `git log --format='%G? %s'`.

Level 5 — Troubleshooting

5.1 A learner says: "I deleted the file and pushed, so the secret is gone." Explain what is wrong, and what to do first.

5.2 A learner rewrote a shared branch's history and force-pushed. A teammate now gets conflicts on every pull. Explain why, and what the teammate must not do.

5.3 A learner sees `G` in `%G?` and concludes that the code is safe. What does the signature actually show?

Chapter 34 — Git Workflows [Core]

In this chapter

- what a workflow is, and why teams need one
- working alone
- the **feature-branch workflow**, run end to end with two "people" on one computer
- the common named models, in general terms
- how to choose

Before you start. Chapter 20, Chapter 21, Chapter 23 and Chapter 29. The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on Git 2.55.0.

Git does not tell you how to use branches. Two teams with the same Git commands can work in very different ways. This chapter is about the agreements that make a team's use of Git predictable.

34.1 A workflow is an agreement

New term: workflow. The agreed way in which a person or a team uses branches, commits, merges and releases to get changes into a project.

A workflow answers questions such as:

- Where does new work start, and where does it end up?
- Who may change the main branch, and how?
- How does a change get reviewed?
- How are releases marked and fixed?

The commands are the same in every workflow (Chapters 20 to 29). What differs is the **rules**: a shared understanding that everyone can state in one minute. A workflow that nobody can state is not a workflow; it is a habit.

34.2 Working alone

When you are the only person, the simplest workflow is fine:

- Commit on main in small steps (Chapter 19).
- Use a branch for anything risky or unfinished (Chapter 20).
- Tag the versions that matter (Chapter 29).
- Push to a remote as a backup (Chapter 23).

Even alone, branches make experiments safe, and small commits make mistakes easy to undo (Chapter 25).

34.3 The feature-branch workflow, step by step

The most common team pattern is: **every change is made on its own short-lived branch, reviewed, and then merged into main**. main always holds work that has been accepted.

Here is the whole cycle, with two people on one computer. A shared bare repository, `hub.git`, plays the remote, and Alice and Bob each have a clone (Chapter 23). Alice starts a branch and publishes it:

```
$ cd alice
$ git switch -c add-tea
Switched to a new branch 'add-tea'
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -am "Add tea"
[add-tea cece341] Add tea
1 file changed, 1 insertion(+)
$ git push -u origin add-tea
To /home/learner/hub.git
* [new branch]      add-tea -> add-tea
branch 'add-tea' set up to track 'origin/add-tea'.
```

Recorded in Bash; *ch34-workflows/expected-feature-branch.bash.txt*.

She pushes the branch to the shared place so that others can see it. (On a hosting platform, this is the point at which she would open a pull request: Chapter 45.) Bob looks at her work. First he fetches, which changes nothing of his own (Chapter 23):

```
$ cd ../bob
$ git fetch
From /home/learner/hub
* [new branch]      add-tea -> origin/add-tea
$ git branch -r
origin/HEAD -> origin/main
origin/add-tea
origin/main
```

Recorded in Bash; *ch34-workflows/expected-feature-branch.bash.txt*.

Then he **reviews** the change by reading the difference between main and her branch:

```
$ git diff main origin/add-tea
diff --git a/menu.md b/menu.md
index 97e3bea..ae8d71a 100644
--- a/menu.md
+++ b/menu.md
@@ -3,3 +3,4 @@
- White loaf: 2.80
- Rolls (six): 3.00
- Coconut cake (slice): 4.00
+- Tea: 1.50
```

Recorded in Bash; *ch34-workflows/expected-feature-branch.bash.txt*.

Satisfied, Bob merges the branch. He uses `--no-ff` (Chapter 21) so that the history records that a branch existed, and publishes the result:

```
$ git merge --no-ff origin/add-tea -m "Merge add-tea"
Merge made by the 'ort' strategy.
 menu.md | 1 +
 1 file changed, 1 insertion(+)
$ git push origin main
To /home/learner/hub.git
 81f772e..9fff21e  main -> main
```

Recorded in Bash; *ch34-workflows/expected-feature-branch.bash.txt*.

Now the branch has done its job, so Bob deletes it on the remote:

```
$ git push origin --delete add-tea
To /home/learner/hub.git
 - [deleted]          add-tea
$ git branch -r
 origin/HEAD -> origin/main
 origin/main
```

Recorded in Bash; *ch34-workflows/expected-feature-branch.bash.txt*.

Alice cleans up on her side. `git fetch --prune` updates her view of the remote, and removes her notes about branches that no longer exist there:

```
$ cd ../alice
$ git fetch --prune
From /home/learner/hub
 - [deleted]          (none)      -> origin/add-tea
 81f772e..9fff21e  main      -> origin/main
$ git branch -vv
* add-tea cece341 [origin/add-tea: gone] Add tea
main      81f772e [origin/main: behind 2] Add coconut cake
```

Recorded in Bash; *ch34-workflows/expected-feature-branch.bash.txt*.

Her own `add-tea` branch shows `[origin/add-tea: gone]`: the remote branch is gone. She returns to `main`, brings it up to date, and deletes her local branch (which Git allows now, because it is merged):

```
$ git switch main
Switched to branch 'main'
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
 (use "git pull" to update your local branch)
$ git pull
Updating 81f772e..9fff21e
Fast-forward
 menu.md | 1 +
 1 file changed, 1 insertion(+)
$ git branch -d add-tea
Deleted branch add-tea (was cece341).
```

Recorded in Bash; *ch34-workflows/expected-feature-branch.bash.txt*.

The history shows the shape of the workflow:

```
$ git log --oneline --graph
* 9fff21e (HEAD -> main, origin/main, origin/HEAD) Merge add-tea
|\
| * cece341 Add tea
|/
* 81f772e Add coconut cake
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
```

Recorded in Bash; *ch34-workflows/expected-feature-branch.bash.txt*.

The cycle is: **branch, commit, push, review, merge, delete**. Everything else in this chapter is a variation on it.

What made it work:

- The branch was **short-lived** and about **one thing**.
- Somebody other than the author looked at it before it reached main.
- main only received changes through a merge.
- Nobody rewrote shared history.

What a platform adds. Hosting platforms wrap this cycle in a web page (a pull request), with comments, approvals and automatic checks (Chapter 45, Chapter 54). The Git commands underneath are the ones you just saw.

34.4 Named models

Teams have described their workflows with names. The descriptions below are **general**, and are not statements about any organization's official practice.

Model	Shape in one line
Trunk-based	Everyone commits to main (the "trunk") in very small steps, or on branches that last hours, and merges often.
Feature branch	As in section 34.3: one short branch per change, reviewed, merged to main.
Release branch	A branch is cut from main for a version being prepared, so that fixes go there while main moves on; fixes are then merged or copied back (Chapter 28 introduced cherry-pick).
A model with long-lived develop and release branches	Work is collected on a develop branch; main holds only released versions; more branch types exist for releases and urgent fixes. It has many rules and suits projects with formal release cycles.
Forking	People without direct write access copy ("fork") the project, work in their copy, and propose changes back (Chapter 49). Common in open source.

Checked in part against official sources (R219). Three of the models have an official description. **Feature branch:** GitHub's documentation calls "GitHub flow" "a lightweight, branch-based workflow" in which you create a branch, make changes, and open a pull request. **Forking:** GitHub defines a fork as "a new repository that shares code and visibility settings with the original 'upstream' repository". **Release and topic branches:** Git's own `gitworkflows` manual describes how the Git project works: the integration branches `maint` ("the next maintenance release"), `master` ("the next release"), `next` ("a testing branch for topics being tested for stability") and `seen`, plus **topic branches**, which solve the problem that "bad commits cannot be undone" when everything goes straight to an integration branch. The **trunk-based** model and the model with long-lived `develop` branches have named published descriptions whose sources were not reached, so their rows stay general descriptions; this chapter attributes no model to any person or company.

Comparing them.

	Change size	Branch lifetime	Review	Release handling	Suits
Trunk-based	very small	hours	often after the fact, or pairing	tags on main	small teams; strong automated tests
Feature branch	small	days	before merge	tags on main	most teams
Release branch	any	weeks (release branch)	before merge	one branch per version	products with several supported versions
develop and release branches	any	long	before merge	dedicated branches	formal, scheduled releases
Forking	any	varies	maintainers review	maintainers	open source; outside contributors

34.5 Choosing

There is no best workflow. Ask four questions:

1. **How many people** change the code at once? (More people, more need for rules.)
2. **How much do you trust your automated tests?** (Good tests allow smaller, faster merges.)
3. **How many versions do you support at once?** (Several supported versions need release branches.)
4. **Who may write to the repository?** (Outsiders need forks.)

Then apply the smallest workflow that answers them, write it down in a short file in the repository (for example `CONTRIBUTING.md`, Chapter 64), and change it when it stops fitting.

CAUTION. The most damaging habits are the same in every workflow: long-lived branches that drift away from `main` and then produce enormous conflicts (Chapter 22); rewriting shared history (Chapter 27, Chapter 33); and committing directly to a branch that everyone depends on without anyone looking.

Verification pending [R220]. The advice in this section, and the size of the teams for which each model suits, is general experience and not a tested or cited fact.

Checkpoint

What You Learned

- A workflow is an agreement about how a team uses branches, reviews and merges; it is not a Git feature.
- The feature-branch cycle: branch, commit, push, review, merge, delete.
- `git fetch --prune` removes remote-tracking branches whose remote branch was deleted.
- Common models: trunk-based, feature branch, release branches, a model with a `develop` branch, and forking.
- Choose the smallest workflow that fits the team, and write it down.

New Vocabulary

- **Workflow:** the agreed way a team uses branches, reviews, merges and releases.

Commands Learned

`git push -u origin <branch>`, `git diff main origin/<branch>`, `git merge --no-ff`, `git push origin --delete <branch>`, `git fetch --prune`, `git branch -vv`.

Common Mistakes

1. **Long-lived branches that drift from `main`.**
2. **Merging without anyone else looking.**
3. **Leaving merged branches lying around.**
4. **Adopting a complicated model for a small team.**
5. **Not writing the workflow down.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What do all workflows share, and what differs between them?
2. Name the six steps of the feature-branch cycle.
3. What does `git fetch --prune remove?`
4. Why did Bob merge with `--no-ff`?
5. Which questions help you choose a workflow?

Before Moving On

You are ready for Chapter 35 if you can:

- [] run the feature-branch cycle with a shared bare repository
- [] explain fetch, diff and merge as a review step
- [] describe at least three named models in one sentence each
- [] say what you would choose for a small team, and why

How this chapter was checked

Claim	Evidence class	Le dg er
Feature-branch cycle with a shared bare repository (push, fetch, diff, <code>--no-ff</code> merge, remote branch deletion, <code>fetch --prune</code> , <code>branch -d</code>)	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0	R2 18
Descriptions of named models	Feature branch, forking and topic/integration branches checked against official sources; trunk-based and develop models general description, no attribution	R2 19
Advice on choosing	General experience, not tested	R2 20

Where this leads

Chapter 35 teaches how to read the official documentation for everything in Part IV. Chapter 45 turns the review step into a pull request on a hosting platform, and Chapter 49 covers the forking model.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use a bare repository as the shared remote and two clones, `alice` and `bob`, on one computer (Chapter 23).

Level 1 — Guided

1.1 Start a feature. As Alice, create `add-tea`, commit a change, and publish the branch with `git push -u origin add-tea`.

1.2 Look at it. As Bob, run `git fetch` and `git branch -r`. What is new, and did anything in Bob's own branches change?

Level 2 — Partially guided

2.1 Review it. As Bob, run `git diff main origin/add-tea`. What would you check before you approve?

2.2 Merge it. Bob merges with `--no-ff` and pushes `main`. How does the graph show that a branch existed?

Level 3 — Independent

3.1 Clean up. Delete the branch on the remote, then, as Alice, run `git fetch --prune` and `git branch -vv`. What does `[origin/add-tea: gone]` mean?

3.2 Finish. As Alice, switch to `main`, pull, and delete the local branch with `git branch -d`. Why did `-d` succeed?

Level 4 — Professional scenario

4.1 Write a workflow. A team of four builds a bakery website, releases every week, and has decent tests. Write the rules of a workflow for them in eight lines or fewer: where work starts, how it is reviewed, how it is merged, how releases are marked.

Level 5 — Troubleshooting

5.1 A branch has lived for six weeks. Merging it produces conflicts in twelve files. Explain why, and how the team could avoid it.

5.2 A team lets everybody commit straight to `main`, and `main` is often broken. Name two changes that would help.

5.3 A new hire asks: "Do we use Git Flow?" The team has never named its workflow. What would you tell them, and what would you do?

Chapter 35 — Reading Official Documentation [Core]

In this chapter

- where Git's own documentation lives
- how to read a command synopsis
- how to read an error message
- how to check that what you read matches the version you have
- how to treat a source, including this book

Before you start. Chapter 13 and Chapter 15. The recordings ran in Bash and zsh on Git 2.43.0 and were re-run in CI on Git 2.55.0.

This chapter closes Part IV. Everything so far was taught by running commands. From now on you will meet commands and features that this book does not cover, and Git and GitHub both change over time. The skill that lasts is being able to look up the answer yourself, and to judge whether it applies to you.

35.1 Documentation comes in layers

Layer	What it is	Good for
The built-in usage text	<code>git <command> -h</code> prints a short summary of the options	a quick reminder of an option's spelling
The reference manual	one page per command (<code>git help <command></code> , or <code>man git-<command></code>)	the exact meaning of each option
Guides and tutorials	longer explanations of concepts and workflows	learning an idea from the start
Release notes	what changed in each version	finding out when something changed
The platform's documentation	for GitHub and similar services, the documents that the platform publishes	features of the service, which are separate from Git

Two of these you can read **without the internet**, and they always match the version that you have installed. That makes them the first place to look.

Checked against the documentation (R221). The `git help` manual page (Git 2.56.0) says the `man` program is used by default; `-m/--man`, `-i/--info` and `-w/--web` choose the format; the `help.format` setting sets the default (`man`, `info`, or `web/html`); and with `--web` "a web browser will be used", chosen by `help.browser` or `web.browser`. `git help -a` lists all available commands and `git help -g` lists the concept guides. Whether your system has the manual pages installed, and what Git for Windows opens, depends on your installation: try `git help commit` and read what happens. (The test computer had its manual pages removed, so the message for that case is the one in the recording.)

35.2 The version you have

Before you trust any page, know which Git you have:

```
$ git --version
git version <version>
```

Recorded in Bash; *ch35-docs/expected-synopsis.bash.txt*.

(The recording shows <version> in place of the number, because the number depends on your computer. In your terminal you will see it in full, such as the versions named in Chapter 13.) Documentation that says "since version 2.x" or "new in..." is only useful when you know your own number. Documents on the web usually carry a version too, often selectable in a menu; look for it, and use the one that matches.

The same applies to any tutorial, video or answer that you find: **check its date and the version it describes**. Advice about Git that is five years old can be wrong now, because the defaults and the messages change. This book has met that several times: the same command printed different advice text on Git 2.43.0 and on 2.55.0 (Chapter 20, Chapter 27, Chapter 33).

35.3 Reading a synopsis

Every command's usage text starts with a **synopsis**: a compact description of how to write the command. For example, the first lines for `git tag`:

```
$ git tag -h | head -n 6
usage: git tag [-a | -s | -u <key-id>] [-f] [-m <msg> | -F <file>] [-e]
      <tagname> [<commit> | <object>]
or: git tag -d <tagname>...
or: git tag [-n[<num>]] -l [--contains <commit>] [--no-contains <commit>]
      [--points-at <object>] [--column[=<options>] | --no-column]
      [--create-reflog] [--sort=<key>] [--format=<format>]
```

Recorded in Bash; *ch35-docs/expected-synopsis.bash.txt*.

The notation is a convention that Git's documentation follows:

Symbol	Meaning
<code>git tag</code>	Literal text that you type exactly
<code>[...]</code>	Optional : you may leave it out
<code><name></code>	A placeholder : replace it with your own value
<code>a b</code>	Alternatives : choose one
<code>...</code>	The item before it may be repeated
<code>or:</code>	A different way of using the same command

(The synopsis grows as Git grows. On Git 2.55.0 the first form has one more optional part, `[(-trailer <token>[(=|:)<value>])...]`, which the 2.43.0 recording above does not have. Round brackets group alternatives, so `(=|:)` means "either = or :". This is a live example of why you must read the documentation for your own version.)

Read the first form: `git tag`, optionally one of `-a`, `-s` or `-u <key-id>`, optionally `-f`, optionally a message with `-m <msg>` (or from a file with `-F <file>`), optionally `-e`; then the required `<tagname>`; then optionally a `<commit>` or `<object>`. That is what you used in Chapter 24 and Chapter 29: `git tag -a v1.0 -m "..."`, with an optional commit at the end. The second form, `git tag -d <tagname>...`, deletes one or more tags.

Checked against the documentation (R222). Git's CodingGuidelines (Git 2.56.0), section "Synopsis Syntax", states the notation: three dots mean "one or more" (`<file>...`); square brackets mean optional (`[<file>...]` is zero or more); a vertical bar separates alternatives (`[-q | --quiet]`); parentheses group (`[(<rev>|<range>)...]`); and placeholders are lowercase words in angle brackets. The table above agrees with these rules.

After the synopsis, the reference manual explains each option, gives examples, and often lists related commands ("See also"). Read the description of one option at a time, with a scratch repository open beside it, and try it. That is the method of this whole book.

35.4 Reading an error message

Git's messages often tell you the fix. A mistyped command:

```
$ git comit
git: 'comit' is not a git command. See 'git --help'.

The most similar command is
    commit
```

Recorded in Bash; `ch35-docs/expected-synopsis.bash.txt`.

Git says what it did not understand, points to help, and suggests the most similar command. Read the **whole message**, top to bottom, before you search the web. Most Git errors follow the same pattern as the ones in this book:

1. **What happened** (`fatal:`, `error:`).
2. **Why** (often on the same line).
3. **What you can do** (lines that start with `hint:`, or indented suggestions).

In this book's examples, `error:` usually marks something that Git refused, and `fatal:` marks a command that Git could not carry on with; the exact rules were not checked in the official documentation. Advice lines (`hint:`) can be switched off; the recordings in this book show several of them (Chapter 24 and others).

35.5 Reading a platform's documentation

Hosting platforms publish their own documents about their own features: accounts, repositories, pull requests, automation and security settings. Three cautions:

- **They describe the service as it is now, not as it was.** Interfaces and limits change (Chapter 36).
- **They may depend on your plan or account type.** A feature described on a page may not be available to you.
- **They are not Git's documentation.** Whether something is a Git feature or a platform feature decides where to look (Chapter 12 introduced the difference).

Checked against GitHub's documentation source (R223). GitHub's documentation is open source, in the `github/docs` repository, and each page carries a versions header saying which products it applies to (for example `fpt` for Free, Pro and Team, `ghes` for GitHub Enterprise Server and `ghes` for GitHub Enterprise Cloud). This book's GitHub statements were checked against that source at commit `2eaab0b` (29 September 2026) and are labelled with their ledger row. The service changes, so when you meet a GitHub fact, check the page for your kind of account and its date before you rely on it.

35.6 How to treat a source, including this book

When you read a claim, ask:

1. **Who says so?** An official document, an experienced author, or a stranger?
2. **How does it know?** Did the author run the command, or repeat what they remember?
3. **For which version, and on what date?**
4. **Can I test it in a scratch repository?** Almost always, yes.

This book tries to answer those questions for you. Each chapter ends with a table that says how its claims were checked: locally tested means that the commands were run, and for which Git version; not verified means that no official source was consulted. Pending markers show what still needs checking. That is a promise about method, not a promise that the book is free of errors. **When the book and the official documentation disagree, trust the documentation for your version, and tell the author.**

Checkpoint

What You Learned

- Documentation has layers: built-in usage text, the reference manual, guides, release notes, and a platform's own pages.
- The built-in text and manual match the version you have installed.
- Know your Git version (`git --version`) and check the version and date of any source.
- A synopsis uses `[ ]` for optional, `<name>` for a placeholder, `|` for alternatives and `...` for repetition.
- Read error messages fully: what happened, why, and what to do.

- Platform documentation describes the service, changes over time, and may depend on your plan.

New Vocabulary

- **Synopsis:** the compact description of how to write a command.

Commands Learned

`git --version`, `git <command> -h`, `git help <command>`.

Common Mistakes

1. **Trusting an old answer without checking its version.**
2. **Reading only the first line of an error message.**
3. **Confusing a Git feature with a platform feature.**
4. **Reading a manual page without trying the command.**
5. **Trusting any source without asking how it knows.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the quickest way to see the options of `git tag`?
2. In `git tag -d <tagname>...`, what do the angle brackets and the three dots mean?
3. Why does the version of Git matter when you read documentation?
4. What three parts do most Git error messages have?
5. Name two cautions for platform documentation.

Before Moving On

You are ready for Chapter [36](#) if you can:

- [] find and read the usage text of any Git command
- [] read a synopsis, including optional and repeated parts
- [] state your Git version and where to check it
- [] say what to check before you trust a source

How this chapter was checked

Claim	Evidence class	Ledger
<code>git --version</code> , <code>git tag -h</code> , the "most similar command" message	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0	R22 1

Claim	Evidence class	Ledger
Synopsis notation as it reads in usage text	Locally tested (usage text); definition checked in CodingGuidelines	R22 2
<code>git help options</code> and <code>help.format</code>	Checked against the <code>git help</code> manual page; not run (no manual pages installed on the test computer)	R22 1
How manual pages open on each system; platform documentation	Page versions headers checked in <code>github/docs</code> ; how manual pages open on your system depends on your installation	R22 3

Where this leads

Part V begins with Chapter 36: what a hosting platform is, and what it is not. Every platform fact in it is labelled with how it was checked.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises use only your own computer.

Level 1 — Guided

1.1 Your version. Run `git --version`. Write down the number. Where else could you check it?

1.2 A usage text. Run `git tag -h`. Copy the first form of the synopsis.

Level 2 — Partially guided

2.1 Read a synopsis. For the first form of `git tag`, list which parts are optional, which is required, and which are placeholders.

2.2 Alternatives and repeats. In `git tag -d <tagname>...`, what does `...` allow you to do? Try it with two tags in a scratch repository.

Level 3 — Independent

3.1 A different command. Run `git switch -h` and `git restore -h`. For each, write the meaning of two options in your own words, and test both in a scratch repository.

3.2 The manual. Try `git help commit`. What happens on your system: a manual page, a web page, or a message? Write down how you would read the manual on this computer.

Level 4 — Professional scenario

4.1 A dated answer. You find a forum post from several years ago that says: "Run this Git command to fix your problem." List four checks you make before you run it, and the scratch-repository test that you would do first.

Level 5 — Troubleshooting

5.1 A learner mistypes `git comit` and searches the web for an hour. What should they have read first?

5.2 A learner follows a tutorial and gets different output from the tutorial's screenshot. Give three possible reasons, related to this chapter.

5.3 A learner cannot tell whether a feature belongs to Git or to their hosting platform. How can they decide where to look?

Part V — Introduction to GitHub

Chapter 36 — What GitHub Is (and Is Not) [Core]

In this chapter

- what GitHub is, in one honest paragraph
- what it is not
- how the Git you already know connects to it
- the pieces that a GitHub repository has, beyond Git
- how to read GitHub facts in this book

Independent publication. This book is an independent educational work by Sharif Issah Tingane, issued under the SHARIF TECHNOLOGIES name. It is not affiliated with, endorsed by, sponsored by, or published by Git, GitHub, Microsoft, or any company mentioned. Names of products and companies are trademarks of their respective owners and are used here only to identify them.

How GitHub facts are checked in this book. Parts I to IV were tested by running Git on a computer. GitHub is a service on the internet that changes over time, so its statements are checked differently: against GitHub's own documentation, whose source is published as the open `github/docs` repository (checked at commit `2eaab0b`, 29 September 2026), and, for legal terms, against GitHub's site-policy repository (commit `b9578b5`, 29 May 2026). Both are recorded in the sources manifest (`research/sources-manifest.csv` in the companion repository). What the website looks like was not seen on a live account, so descriptions of screens are labelled, and anything the sources do not cover stays marked Verification pending. The parts about **Git** (commands, the remote, URLs) were run and tested.

Before you start. Chapter [12](#) and Chapter [23](#). The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on Git 2.55.0.

36.1 One honest paragraph

Git is the version-control program that you have used in Parts III and IV. It runs on your computer, needs no account and no internet, and works the same with any remote (Chapter [23](#)).

GitHub (defined in Chapter [12](#)) is an online platform that hosts Git repositories and adds services around them, such as pull requests, issue tracking and automation.

In terms of what you learned, GitHub is **a remote with a website and services attached**. A remote is a place that holds a copy of your repository, that you push to and fetch from. The shared folder `hub.git` in Chapter [23](#) and Chapter [34](#) did the first part of that job. GitHub does the same, from anywhere on the internet, and adds features for working together: discussing changes, reviewing them, tracking tasks, and running automatic checks.

Checked against GitHub's documentation (R224). GitHub's page "What is GitHub?" says GitHub "is a platform for building software" that supports the stages Plan, Create, Review, Test, Deploy and Operate, and that it "is based on the open-source software, Git" and builds on it "by hosting your Git projects, called repositories, in the cloud, as well as adding planning and collaboration tools". Which features are available on which plan is time-sensitive; look at the current plans page before relying on one.

36.2 What GitHub is not

Misunderstanding	The reality
"GitHub is Git."	Chapter 12
"You need GitHub to use Git."	No. Everything in Parts III and IV worked with no account and no network.
"Everything on GitHub is Git."	A repository's code and history are Git. Features such as issues and pull requests belong to the platform, and are stored by it (not in your <code>.git</code> folder).
"A repository on GitHub is public."	Chapter 39
"GitHub is a backup."	Chapter 23
"GitHub is neutral and permanent."	GitHub's Terms of Service (site-policy repository) say "You own Your Content", that GitHub "may refuse or remove User-Generated Content that violates applicable law or our terms and policies", and that after an account is cancelled GitHub will, barring legal requirements, "delete your full profile and the Content of your repositories within 90 days" and that this "cannot be recovered". The same terms name Microsoft as an Affiliate of GitHub. Chapter 65 returns to ownership and licences.

Checked against the Terms of Service (R225). The row above quotes GitHub's Terms of Service as published in its site-policy repository on the date given at the start of this chapter. The book quotes it to show that a hosting service is not a permanent archive; it is not legal advice, and terms change. Read the current terms before you rely on any of it.

36.3 How your Git connects to it

To Git, a GitHub repository is just another remote, with a web address. Add one to a repository. This example uses a made-up address (`example-owner/sunrise-bakery`) and **does not contact the internet**: `git remote add` only records the address.

```
$ cd bakery-menu
$ git remote add origin https://github.com/example-owner/sunrise-bakery.git
```

Recorded in Bash; *ch36-whatgh/expected-remote-url.bash.txt*.

```
$ git remote -v
origin https://github.com/example-owner/sunrise-bakery.git (fetch)
origin https://github.com/example-owner/sunrise-bakery.git (push)
```

Recorded in Bash; *ch36-whatgh/expected-remote-url.bash.txt*.

Two lines: one address to fetch from, one to push to. The address is stored in the configuration:

```
$ git config --get remote.origin.url
https://github.com/example-owner/sunrise-bakery.git
```

Recorded in Bash; *ch36-whatgh/expected-remote-url.bash.txt*.

Notice that `git branch -vv` shows no connection to a remote branch yet:

```
$ git branch -vv
* main 81f772e Add coconut cake
```

Recorded in Bash; *ch36-whatgh/expected-remote-url.bash.txt*.

That is right: nothing has been pushed, so `main` does not follow any `origin/main` (Chapter 23). The rest of the connection works exactly as it did with the shared folder: `git push -u origin main`, `git fetch`, `git pull`. The differences are the address, and who you are: because the service belongs to somebody else, it needs to know who is asking (Chapter 6, Chapter 38).

The anatomy of that address. `https://github.com/example-owner/sunrise-bakery.git` has the parts that Chapter 5 described: a scheme (`https`), a host (`github.com`), and a path: an **owner** (`example-owner`, a person or an organization) and a **repository name** (`sunrise-bakery`). The `.git` at the end is common but often optional.

Checked in part against GitHub's documentation (R226). "About remote repositories" says that you can push to two kinds of address, an HTTPS URL like `https://github.com/user/repo.git` and an SSH URL like `git@github.com:user/repo.git`, that HTTPS clone URLs "are available on all repositories, regardless of visibility" and "work even if you are behind a firewall or proxy", and that SSH needs a key pair with the public key added to your account. That `.git` at the end of the address is optional was **not** found in these pages and is not relied on. The `git remote add` commands here were run locally without contacting GitHub.

36.4 What a repository has beyond Git

A GitHub repository contains your Git repository, and around it a set of services. As a **map for the next chapters**:

Piece	What it is for	Chapter
Code and history	Your Git repository (files, commits, branches, tags)	Chapter 39
README and licence	The description shown for the project; the terms of reuse	Chapter 39 and Chapter 65
Issues	Tracking tasks, bugs and ideas	Chapter 44
Pull requests	Proposing, discussing and reviewing changes	Chapter 45
Actions	Automatic checks and tasks that run on the platform's computers	Chapter 54
Releases	Named, downloadable versions built on tags	Chapter 66
Settings and security	Access, protection, scanning and secrets	Chapter 49 and Chapter 62

Checked against GitHub's documentation (R227). Each row names a feature that has its own section in the current documentation: repositories, issues, pull requests, GitHub Actions and releases. For releases, "About releases" says that they "are based on Git tags", that a tag date "may be different" from a release date, and that GitHub "will automatically include links to download a zip file and a tarball containing the contents of the repository at the point of the tag's creation". Names and availability can change; the chapter numbers refer to where this book covers each one.

Where does it live? The first row is in your Git repository, and you can clone it. The other pieces, such as issues and pull requests, are stored by the platform; a normal `git clone` does **not** bring them with you. If you want to keep them, you need another way (Chapter [74](#)).

36.5 Why learn Git first

This book taught computers first, then Git, then GitHub, on purpose. Because you know what a remote is, a repository is, a branch is, and how to undo and to recover, the platform's features make sense as layers over things you already understand. And if the platform changes its buttons, its plans or its name, the Git knowledge remains.

Checkpoint

What You Learned

- GitHub is an online platform that hosts Git repositories and adds services; it is not Git.
- You do not need GitHub to use Git.
- To Git, a GitHub repository is a remote with a web address; `git remote add` only records it.
- A repository's code is Git; issues, pull requests and similar features are stored by the platform.
- GitHub statements in this book are checked against GitHub's own documentation and are labelled where they could not be.

New Vocabulary

No new terms. This chapter uses **GitHub** and **hosting platform** from Chapter 12.

Commands Learned

`git remote add origin <url>`, `git remote -v`, `git config --get remote.origin.url`.

Common Mistakes

1. **Believing that GitHub is Git.**
2. **Thinking a clone brings issues and pull requests with it.**
3. **Publishing a repository without deciding that it should be public.**
4. **Treating a hosted copy as a backup plan.**
5. **Trusting a GitHub fact, in any source, without checking its date.**

Practice

Do the exercises in the Exercises section at the end of this chapter. None needs a GitHub account.

Self-Test

1. In one sentence, what is GitHub, in terms of what you know about remotes?
2. Name two things in this book that worked without GitHub.
3. Which pieces of a GitHub repository come with `git clone`, and which do not?
4. What does `git remote add origin <url>` do, and what does it not do?

5. Why are the GitHub facts in this part labelled with how they were checked?

Before Moving On

You are ready for Chapter 37 if you can:

- [] explain the difference between Git and GitHub
- [] add a remote by address and read `git remote -v`
- [] name the parts of a GitHub repository address
- [] say which features belong to the platform and not to Git

How this chapter was checked

Claim	Evidence class	Ledger
<code>git remote add</code> , <code>remote -v</code> , <code>config --get</code> , no upstream before a push	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0; no network access to GitHub . Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-remote</code>); the ledger row says which.	R22 6
What GitHub offers, and the map of features	Checked against <code>github/docs</code> (commit 2eaab0b); plan availability not checked	R22 4, R22 7
Terms, ownership, removal of content	Quoted from site-policy (commit b9578b5); plans not checked	R22 5

Where this leads

Chapter 37 covers accounts and profiles, Chapter 38 covers signing in from Git, and Chapter 39 creates a first repository.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. None of these needs a GitHub account or an internet connection. Use `bakery-menu` (rebuild it from Chapter 17 if needed).

Level 1 — Guided

1.1 Two sentences. Write one sentence saying what Git is and one saying what GitHub is, without using the word "GitHub" in the first.

1.2 Add an address. Run `git remote add origin https://github.com/example-owner/sunrise-bakery.git` in a practice repository, then `git remote -v`. What do the two lines show?

Level 2 — Partially guided

2.1 Read the address. Label the scheme, the host, the owner and the repository name in `https://github.com/example-owner/sunrise-bakery.git`.

2.2 No connection yet. Run `git branch -vv` and `git config --get remote.origin.url`. Why does `main` show no remote branch?

Level 3 — Independent

3.1 Git or platform? For each item, decide whether it is part of Git or a platform feature: a commit; a pull request; a branch; an issue; a tag; a release page; a `.gitignore` file.

3.2 What a clone brings. In your own words, list what `git clone` copies and what it does not.

Level 4 — Professional scenario

4.1 Explain it to a colleague. A colleague says: "We use GitHub, so we do not need Git." Write a short, kind explanation with two examples from this book.

Level 5 — Troubleshooting

5.1 A learner says: "I cloned the project, but the issues are missing." What is the misunderstanding?

5.2 A learner says: "My code is on GitHub, so I have a backup." List two ways in which that can be untrue.

5.3 A learner finds a GitHub tutorial with different buttons from the ones on their screen. Give two possible reasons and one action.

Chapter 37 — Account and Profile [Core]

In this chapter

- what a GitHub account is, and how to choose a username
- how your Git identity and your account connect
- what a profile shows, and who can see it
- what to keep off a profile
- organizations, briefly

How to read this chapter. Everything here is about how a web service behaves. The statements are checked against GitHub's own documentation (the open `github/docs` repository at commit `2eaab0b`, 29 September 2026) and its `site-policy` repository (commit `b9578b5`), and are labelled with the ledger row that records the check. No step was run on a live account, and what the screens look like was not seen, so buttons and menus are described by their idea, not their position. The Git side (your name and email in commits) was tested in Chapter 14.

Before you start. Chapter 36 and Chapter 6 (accounts, passwords and trust in general). No commands in this chapter are new.

37.1 What an account is

An **account** on a platform is a stored identity: a **username**, one or more **email addresses**, a way to prove that you are you (a password and more, Chapter 38), and settings. Everything you do on the platform (a repository, a comment, a pull request) is attributed to it.

Checked against GitHub's documentation (R228). "Creating an account on GitHub" says that to get started you need "a personal account and a verified email address", that you sign up at `github.com/signup` (or with the supported social logins Google or Apple), and that without a verified email address "you won't be able to complete some basic GitHub tasks, such as creating a repository". It recommends two-factor authentication. "Types of GitHub accounts" names three kinds of account: user accounts, organization accounts and enterprise accounts; a personal account is the kind you get by signing up yourself, and the same page calls accounts created for automation machine users. GitHub's username policy says names "are available on a first-come, first-served basis", that requests to reclaim a name because it looks inactive are not accepted, and that name squatting is prohibited. The exact character rules for a username were not found in these pages; the sign-up form states them.

Before you sign up, decide three things:

1. **Which email address?** Use one that you will keep for years, and that you can read. Recovery (Chapter 38) depends on it.

2. **Which username?** It becomes part of every address of your repositories (Chapter 36: `github.com/<username>/<repository>`). Choose one that you will not be embarrassed by in five years, and consider whether it should be tied to your real name. Employers and clients may look at it.
3. **Personal or work?** If a workplace or school will ask you to use its own arrangements, ask before you create an account in their name, and keep personal projects separate from theirs.

37.2 Your Git identity and your account

In Chapter 14 you set `user.name` and `user.email`. Git writes them into every commit **as text**, and Git itself never checks them (Chapter 33).

The platform, however, **can connect** a commit to an account by its **email address**: if the email in a commit matches an address that an account has confirmed, the platform may show that account as the author of the commit. If the email matches no account, the commit still exists, but it will not be linked to a profile.

Checked against GitHub's documentation (R229). "Email addresses" says GitHub "uses your commit email address to associate commits with your account", and that to have commits attributed to you and shown in your contribution graph you must use "an email address that is connected to your account", or the `noreply` address provided in your email settings. For commits pushed from the command line the address comes from your Git configuration; to use the `noreply` address there, set it with `git config`. To use it for web-based edits, choose **Keep my email address private** in the settings. There is also an option to block command-line pushes that expose your personal address. "Any commits you made prior to changing your commit email address are still associated with your previous email address." The book adds one thing the documentation does not say: a commit's email is public wherever the repository is public, and stays in the history (Chapter 30).

A safe habit: decide, before you make your first public commit, which email you are willing to publish, and set it with `git config` (Chapter 14). Changing it later does not change commits that already exist.

37.3 The profile

A **profile** is your public page. Its parts, as GitHub's documentation lists them:

Part	What it is
Name, picture, short description	What visitors see first
Repositories	The public repositories that you own
Pinned repositories	A few that you choose to show at the top

Part	What it is
Profile README	A special repository, named the same as your username, whose README is displayed on the profile
Contribution graph	A calendar of activity
Organizations	Groups that you belong to (Chapter 68)

Checked against GitHub's documentation (R230). "About your profile" lists the profile README, personal information (picture, name and bio), contribution activity, pinned items and a status. A **profile README** is shown when "you've created a repository with a name that matches your GitHub username", the repository is public, it contains a README.md in its root, and that file has some content. **Pinned** items are "up to six repositories and gists, combined". The **contribution graph** ("Contributions on your profile" and its reference page) records contributions with times in UTC, shows only public-repository activity by default (you can choose to show private activity in anonymised form), and counts actions such as creating a repository or forking always, and opening an issue or pull request, reviewing, and making a commit sometimes. A commit counts only if its email is associated with your account, it was made in a standalone repository (not a fork), and it is on the default branch or gh-pages, and you are a collaborator or organization member, or have forked it, or have opened a pull request or issue there. A setting exists to "make profile private and hide activity". A graph of activity is not a measure of skill: it counts what the platform counts, on its rules, and quiet weeks are normal.

Practical advice.

- Pin the repositories that show your best and most complete work, not the newest.
- Write a short description of who you are and what you are learning, in plain language.
- Everything on a public profile can be read by anybody, including people that you do not know, and it may be copied and remembered (Chapter 33).

37.4 What to keep off a profile

- Your home address, phone number, or your daily schedule.
- Private email addresses that you do not want to receive mail on.
- Anything that identifies children or other people without their agreement.
- Names of clients or employers that you have not been allowed to share.
- Secrets of any kind (Chapter 33).

Treat a profile as a **shop window**: it should show what you want visitors to see, and nothing else.

37.5 Organizations

An **organization** is a shared account that owns repositories and has **members** with roles, so that a team, a school class, or a project does not depend on one person's account. Chapter 68 covers them. For now, note that a repository can live under a person or under an organization, and that this changes its address (`github.com/<owner>/<repository>`) and who can manage it.

Checked against GitHub's documentation (R231). "Types of GitHub accounts" says organizations are "shared accounts where a large number of people can collaborate across many projects at once"; that "you cannot sign in to an organization", because each person signs in to their own user account and actions are attributed to it; that members can have different roles; that owners and security managers manage settings; and that teams are nested sub-groups of members. Which organization features depend on the plan was not checked.

37.6 A checklist for your first account

1. Choose the email address and the username on paper first.
2. Create the account through the platform's own site (type the address yourself; do not follow a link from a message).
3. Confirm your email address when asked.
4. Turn on stronger sign-in protection now (Chapter 38).
5. Decide which email address commits will publish, and set `git config user.email` accordingly (Chapter 14).
6. Fill in only the profile fields that you are happy to make public.
7. Sign out and sign in again, to prove that you can.

CAUTION. Do not create the account from a shared or borrowed computer without signing out, and never share the password. A platform account that owns your code is worth protecting.

Checkpoint

What You Learned

- An account is a stored identity; everything you do on the platform is attributed to it.
- A username appears in every repository address, so choose it deliberately.
- The platform can link a commit to an account by email; a commit's email is public in a public repository and stays in the history.
- A profile shows what you choose to share; GitHub offers a setting to make it private, and the graph shows only public activity by default. Show only what you want visitors to see.
- An organization owns repositories for a group.

New Vocabulary

No new terms. This chapter uses **account** from Chapter 6.

Commands Learned

None. `git config user.email` from Chapter 14 is the Git side of your identity.

Common Mistakes

1. **Choosing a username in a hurry.**
2. **Publishing a private email address in public commits.**
3. **Treating the contribution graph as a score.**
4. **Putting personal details on a public profile.**
5. **Skipping stronger sign-in protection at the start.**

Practice

Do the exercises in the Exercises section at the end of this chapter. They need no live account.

Self-Test

1. Name three things to decide before creating an account.
2. How can a commit be linked to an account?
3. Why should you choose the commit email before the first public commit?
4. Name three things to keep off a public profile.
5. Why is the contribution graph not a measure of skill?

Before Moving On

You are ready for Chapter 38 if you can:

- [] explain the link between `user.email` and an account
- [] name what to decide before creating an account
- [] say what should stay off a profile
- [] describe what an organization is for

How this chapter was checked

Claim	Evidence class	Ledger
Git writes <code>user.name</code> and <code>user.email</code> into commits as text (Chapter 14)	Locally tested in Chapter 14	n/a here
Account creation, usernames, profile parts, the contribution graph, email matching, organizations	Checked against <code>github/docs</code> and <code>site-policy</code> ; screens not seen (no live account)	R228-R231

Where this leads

Chapter 38 covers signing in and protecting the account. Chapter 39 creates your first repository, and Chapter 68 returns to organizations.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. **These exercises need no live account.** They are planning exercises; check each answer against GitHub's current documentation.

Level 1 — Guided

1.1 Decide first. On paper, write the email address and the username that you would use for a new account, and one sentence saying why for each.

1.2 Your Git identity. Run `git config --get user.name` and `git config --get user.email` (Chapter 14). Which of these would appear in a public commit?

Level 2 — Partially guided

2.1 Public or private? For each item, say whether you would put it on a public profile: a photo of your face; your favourite programming language; your home town; your employer's name; your phone number.

2.2 The address. If your username were `ada-bakes` and your repository `sunrise-bakery`, what would the repository's address look like (Chapter 36)?

Level 3 — Independent

3.1 Email planning. You have a private email address and a work email address. Which would you set as `user.email` for public projects, and why? What is the risk of the other choice?

3.2 A profile plan. List the three repositories (real or imagined) that you would pin and why.

Level 4 — Professional scenario

4.1 Personal and work. A new job asks you to contribute to their repositories. Write a short plan for keeping personal projects and work projects apart (accounts, emails, and identities in Git).

Level 5 — Troubleshooting

5.1 A learner made public commits using a private email address, and now wants it removed from history. Explain why this is hard, and what to do before the first commit next time.

5.2 A learner's commits do not show their profile picture on the platform. Give one possible reason related to this chapter.

5.3 A recruiter comments on a quiet contribution graph. How should the learner think about it?

Chapter 38 — Authentication in Depth [Core]

In this chapter

- how Git proves to a platform who you are: HTTPS or SSH
- SSH key pairs: making one, and what to protect
- tokens, credential managers and two-factor sign-in, in principle
- signing commits with an SSH key
- a checklist for a safe setup

How to read this chapter. The **Git and SSH side** was run and tested: addresses, key generation, and signing. Everything that happens **on the platform's website** (adding a key to an account, creating a token, two-factor settings) is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026), and was **not** run on a live account, so menus and buttons are described by their idea.

Before you start. Chapter 37, Chapter 33, Chapter 32 (credential helpers) and Chapter 6. The recordings ran in Bash and zsh on Git 2.43.0 with OpenSSH 9.6 and were re-run in CI on Git 2.55.0.

38.1 Two questions

When Git talks to a hosted repository, two questions arise:

1. **Who are you?** (authentication)
2. **What may you do here?** (authorisation: can you read this repository, can you push to it?)

This chapter is about the first question. The second is settled by the repository's settings (Chapter 49).

Authentication (introduced in Chapter 6) is proving who you are to a system, for example with a password, a key or a one-time code. **Authorisation**, by contrast, is what the system then allows you to do.

Git offers two ways to reach a hosted repository, and the difference shows in the address.

38.2 HTTPS or SSH

The same repository can have two kinds of address (Chapter 36 introduced the first). Switching between them is one command:

```
$ cd bakery-menu
$ git remote add origin https://github.com/example-owner/sunrise-bakery.git
$ git remote -v
origin https://github.com/example-owner/sunrise-bakery.git (fetch)
origin https://github.com/example-owner/sunrise-bakery.git (push)
```

Recorded in Bash; *ch38-auth/expected-urls.bash.txt*.

```
$ git remote set-url origin git@github.com:example-owner/sunrise-bakery.git
$ git remote -v
origin git@github.com:example-owner/sunrise-bakery.git (fetch)
origin git@github.com:example-owner/sunrise-bakery.git (push)
$ git remote set-url origin https://github.com/example-owner/sunrise-bakery.git
$ git config --get remote.origin.url
https://github.com/example-owner/sunrise-bakery.git
```

Recorded in Bash; *ch38-auth/expected-urls.bash.txt*.

	HTTPS address	SSH address
Looks like	https://github.com/owner/repo.git	git@github.com:owner/repo.git
You prove yourself with	a username with a token or password, usually stored by a credential helper	a key pair
Works through	ordinary web connections	the SSH protocol (some networks block it)
Good for	getting started; restricted networks	daily work once set up

`git remote set-url` changes an address without touching anything else (Chapter 23). Both forms reach the same repository.

Checked against GitHub's documentation (R232). "About authentication to GitHub" says that you can reach repositories from the command line in two ways, HTTPS and SSH, and that "the method of authenticating is determined based on whether you choose an HTTPS or SSH remote URL". On HTTPS you "must authenticate with a personal access token": "when Git prompts you for your password, enter your personal access token", and "password-based authentication for Git has been removed in favor of more secure authentication methods". Unless you use a credential helper, Git prompts every time. HTTPS works "even if you are behind a firewall or proxy". Which method GitHub currently recommends first is not stated as one choice: the documentation offers both, and it may change.

38.3 SSH keys

An SSH **key pair** is two files that belong together, made by one command.

A **key pair** (Chapter 6 introduced the idea) is two linked cryptographic keys: a **private key**, which you keep secret, and a **public key**, which you can share freely. Something proved with the private key can be checked with the public key; the public key cannot practically be used to work out the private key.

Make a pair for the demonstration. `-t ed25519` chooses a modern key type, `-c` adds a label, `-f` names the file, and `-q` keeps the tool quiet. (`-N ""` sets **no passphrase**, only so that this recording can run without asking; see the caution below.)

```
$ mkdir -p ~/.ssh
$ chmod 700 ~/.ssh
$ ssh-keygen -q -t ed25519 -f ~/.ssh/id_demo -N "" -C "ada@example.org"
$ ls ~/.ssh
id_demo  id_demo.pub
```

Recorded in Bash; *ch38-auth/expected-ssh-key.bash.txt*.

Two files: `id_demo` (private) and `id_demo.pub` (public). Look at their permissions, the first line of each, and the type:

```
$ ls -l ~/.ssh/id_demo ~/.ssh/id_demo.pub | cut -c1-10
-rw-----
-rw-r--r--
$ head -n 1 ~/.ssh/id_demo | tr -d '-'
BEGIN OPENSSSH PRIVATE KEY
$ cut -d' ' -f1,3 ~/.ssh/id_demo.pub
ssh-ed25519 ada@example.org
$ ssh-keygen -l -f ~/.ssh/id_demo.pub | cut -d' ' -f1,4
256 (ED25519)
```

Recorded in Bash; *ch38-auth/expected-ssh-key.bash.txt*.

- The private key is readable and writable **only by you** (`-rw-----`). The tool sets that, and SSH refuses to use a private key that other users can read.
- The private file's first line announces what it is: `BEGIN OPENSSSH PRIVATE KEY` (the recording removes the dashes that surround it). Anyone who gets the whole file can pretend to be you. Because the block is so recognisable, automatic scanners look for that line, and this book's own checks reject it in the text.
- The public file is one line: the key type (`ssh-ed25519`), the key itself (not shown), and the label. That line is what you give to a service.
- The key is 256 (ED25519).

CAUTION. Protect the private key with a passphrase in real use: a passphrase encrypts the file, so that a stolen copy is useless without it. The demonstration used none, for convenience only. Never put a private key in a repository, an email or a chat (Chapter 33). If you think a private key was exposed, remove its public half from every service that trusts it, and make a new pair.

On the platform. To use the key you add the **public** line to your account's settings, and the platform then accepts connections that prove possession of the matching private key. A helper program called an agent can hold an unlocked key in memory for a while, so that you do not type the passphrase every time.

Checked against GitHub's documentation (R233). To use SSH you generate a key pair, add the private key to the SSH agent, and add the **public** key to your account. The documentation's example generates an ed25519 key, and says to use `rsa -b 4096` "if you are using a legacy system that doesn't support the Ed25519 algorithm"; hardware-backed ed25519-sk keys are covered too. To test: `ssh -T git@github.com`, check that the fingerprint matches GitHub's published one, and expect the reply "Hi USERNAME! You've successfully authenticated, but GitHub does not provide shell access." A key unused for a year is deleted automatically. How the agent is started differs by operating system; follow GitHub's current instructions for yours.

38.4 Tokens

A **token** is a long random string that works like a password for a program. Platforms use them so that you can give a tool limited access without giving away your account password.

New term: access token. A secret string that a program presents instead of a password. It can be limited to certain actions and to a certain time, and can be revoked without changing the account password.

Good habits, whichever platform you use:

- Give a token **only the permissions it needs**, for **as short a time** as workable.
- Treat it as a password: never commit it (Chapter 33), never paste it into a shared place.
- If it leaks, **revoke it first** (Chapter 33).
- Let a credential helper store it, rather than typing it into every command. Chapter 32 showed the mechanism, and warned that the simple store helper keeps it in **plain text**.

Checked against GitHub's documentation (R234). "Managing your personal access tokens" says GitHub currently supports two types, **fine-grained** personal access tokens and **classic** ones, and recommends fine-grained tokens "whenever possible". A fine-grained token is limited to one user or organization, can be limited to specific repositories, and has specific permissions. Both types "are tied to the user who generated them". GitHub recommends setting an expiration, and after it a token "is automatically revoked"; an expired or revoked token cannot be restored. A valid token pushed to a public repository or gist "will be automatically revoked". Token names, limits and pages change; check the current page before creating one.

38.5 Credential managers

A **credential manager** is a credential helper (Chapter 32) that stores passwords and tokens in a protected place provided by the operating system, or asks you to sign in through a browser and then remembers the result. Installers for Git often include one.

Checked in part against GitHub's documentation (R235). "Caching your GitHub credentials in Git" recommends two ways: the GitHub CLI (`gh auth login`, choosing HTTPS), and **Git Credential Manager**, which manages authentication "including 2FA" so that you need not create and store a token by hand, and which on macOS stores credentials in the keychain. Which helpers ship with Git for Windows, macOS and Linux, and how each stores secrets, was not tested here (Chapter 32 lists the helpers in Git's own documentation).

38.6 Two-factor sign-in, passkeys and recovery

A password alone is the weakest form of protection: it can be guessed, stolen or reused. **Two-factor authentication** (2FA) asks for a second proof besides the password: a code from an app, a hardware key, or a device that you hold. **Passkeys** are a newer way to sign in with a key held by your device instead of a password.

- **Turn on 2FA before anything else** on an account that owns your code.
- **Keep the recovery codes** somewhere safe and separate from the device. Losing both the second factor and the codes can lock you out for good.
- Use a **password manager**, and a different password for every service (Chapter 6).
- Never approve a sign-in prompt that you did not start.

Checked against GitHub's documentation (R236). "About authentication to GitHub" lists the ways to sign in to the website: username and password (or a social login with Google or Apple), two-factor authentication, which it calls recommended, and a passkey. With 2FA on, you enter a code from a time-based one-time password (TOTP) app or one sent as a text message; you can add a security key or GitHub Mobile as further methods. A **passkey** gives "a secure, passwordless login" and "satisfies both password and 2FA requirements". GitHub asks you to keep recovery codes ("one-time codes for account recovery if you can't access your two-factor authentication method") and recommends "two or more authentication methods". The same page says GitHub recommends a password manager for the password.

38.7 Signing commits with an SSH key

Chapter 33 signed a commit with GPG and said that Git can also sign with an SSH key. Here it is, using a demonstration key. The setup names the signing format, the key, and a file of **allowed signers** (who is allowed to sign, and with which public key), so that Git can check signatures too:

```
$ cd bakery-menu
$ mkdir -p ~/.ssh
$ ssh-keygen -q -t ed25519 -f ~/.ssh/id_sign -N "" -C "ada@example.org"
$ printf 'ada@example.org %s\n' "$(cut -d' ' -f1,2 ~/.ssh/id_sign.pub)" > ~/.ssh/
allowed_signers
$ git config gpg.format ssh
$ git config user.signingkey ~/.ssh/id_sign.pub
$ git config gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers
```

Recorded in Bash; *ch38-auth/expected-ssh-sign.bash.txt*.

Then one signed commit and one unsigned commit:

```
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -q -S -am "Add tea (signed with an SSH key)"
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -q -am "Add coffee (unsigned)"
```

Recorded in Bash; *ch38-auth/expected-ssh-sign.bash.txt*.

Ask Git for the result, using %G? again:

```
$ git log --format='%G? %GS: %s' -3
N : Add coffee (unsigned)
G ada@example.org: Add tea (signed with an SSH key)
N : Add coconut cake
```

Recorded in Bash; *ch38-auth/expected-ssh-sign.bash.txt*.

G means that the signature is good and that the signer is in the allowed-signers file. N means no signature. Verify the signed commit explicitly:

```
$ git verify-commit HEAD~1 2>&1 | sed 's/ with ED25519 key.*//; s/^Good "git" signature
for/Good signature for/'
Good signature for ada@example.org
```

Recorded in Bash; *ch38-auth/expected-ssh-sign.bash.txt*.

(The recording removes the key's fingerprint from that line, because it is different for every key.)

What a platform adds. Hosting platforms can show a mark next to signed commits when they can match the signature to a key on an account. That mark says that some key on the account made the signature. It is not a certificate of good code.

Checked against GitHub's documentation (R236). "About commit signature verification" says a commit or tag with a signature that is cryptographically verifiable, whether GPG, SSH or S/MIME, is marked "Verified" (or "Partially verified"); by default the statuses are Verified, Unverified and no status for an unsigned commit, and "vigilant mode" is off by default. It calls SSH signatures "the simplest to generate", and says you can even upload an existing authentication key to use as a signing key. A GPG key can expire or be revoked, which SSH does not offer. Once verified when pushed, a signature stays verified within the repository's network. As this chapter said above, the mark is about who signed; it does not say the code is good.

38.8 A checklist for a safe setup

1. Use a **unique, strong password** from a password manager, and **turn on 2FA**. Store the recovery codes safely.
2. Choose **SSH** (a key pair with a passphrase) or **HTTPS with a credential manager**; do not store tokens in plain text.
3. Give each token the **fewest permissions** and a **short life**.
4. Keep private keys and tokens **out of every repository** (Chapter 33).
5. Test the connection before you rely on it, following the platform's current guide.
6. **Revoke** anything that you no longer use, and anything that may have leaked.
7. Optionally, **sign your commits** (Chapter 33, section 38.7).

Checkpoint

What You Learned

- Authentication proves who you are; authorisation decides what you may do.
- A hosted repository has an HTTPS address and an SSH address; `git remote set-url` switches between them.
- An SSH key pair has a private key (secret, mode `-rw-----`) and a public key (shared); protect the private key with a passphrase.
- A token works like a password for programs; give it minimal permissions and a short life, and revoke it if it leaks.
- Two-factor sign-in, a password manager and stored recovery codes protect an account that owns your code.
- Git can sign commits with an SSH key and verify them with an allowed-signers file.

New Vocabulary

- **Access token**: a secret string that a program presents instead of a password.

Commands Learned

`git remote set-url, ssh-keygen -t ed25519, git config gpg.format ssh, git config user.signingkey, git config gpg.ssh.allowedSignersFile, git verify-commit.`

Common Mistakes

1. **Creating a key without a passphrase and leaving it that way.**
2. **Committing a private key or a token.**
3. **Giving a token more permissions or a longer life than needed.**
4. **Turning on 2FA and losing the recovery codes.**
5. **Reading the "signed" mark as proof that the code is safe.**

Practice

Do the exercises in the Exercises section at the end of this chapter. Use only demonstration keys, and delete them afterwards.

Self-Test

1. What is the difference between authentication and authorisation?
2. What does `git remote set-url` change?
3. Which of the two key files may be shared, and which must not be?
4. Why does SSH insist on `-rw-----` for the private key?
5. Why give a token few permissions and a short life?

Before Moving On

You are ready for Chapter 39 if you can:

- [] tell an HTTPS address from an SSH address and switch between them
- [] make a key pair and say which file is secret
- [] name three habits that protect an account and its tokens
- [] sign a commit with an SSH key and read %G?

How this chapter was checked

Claim	Evidence class	Ledger
HTTPS and SSH address forms; <code>remote set-url</code>	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0	R232
Key generation, file permissions, key type	Locally tested with OpenSSH 9.6 (CI: the runner's OpenSSH)	R233
SSH commit signing and verification with an allowed-signers file	Locally tested (as above)	R236
Tokens, credential managers, 2FA, passkeys, the platform's key settings and "Verified" marks	Checked against github/docs (commit 2eaab0b); not run on a live account	R232-R236

Where this leads

Chapter 39 creates your first repository and pushes to it, using one of the methods above. Chapter 60 returns to tokens inside automation, and Chapter 62 to the platform's own protections.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Use only **demonstration keys** in a scratch folder, and delete them afterwards. OpenSSH must be installed (ssh-keygen).

Level 1 — Guided

1.1 Two addresses. In a practice repository, add `https://github.com/example-owner/sunrise-bakery.git` as origin, then switch it to `git@github.com:example-owner/sunrise-bakery.git` with `git remote set-url`. Show both with `git remote -v`.

1.2 Make a key. Run `ssh-keygen -t ed25519 -f ./demo_key -C "demo"` and give a passphrase when asked. Which two files appeared?

Level 2 — Partially guided

2.1 Which is which? Use `ls -l` on both files. Which has restricted permissions, and why?

2.2 Read the public key. Print the public file. Which parts does the line have, and which of the two files may you share?

Level 3 — Independent

3.1 Sign with SSH. Configure `gpg.format ssh, user.signingkey` (the `.pub` file) and an allowed-signers file. Make one signed and one unsigned commit and compare them with `git log --format='%G? %s'`.

3.2 Verify. Run `git verify-commit` on the signed commit. What does it say?

Level 4 — Professional scenario

4.1 A leaked key. A colleague pasted a private key into a public chat. Write the steps, in order, that they should take, and say what you would check afterwards.

4.2 A token policy. Write five rules for a small team about creating and using access tokens.

Level 5 — Troubleshooting

5.1 A learner says: "My SSH key works, so I do not need 2FA." Explain why that is wrong.

5.2 `ssh` refuses a private key with the message that its permissions are too open. What should the learner do, and why does SSH care?

5.3 A learner committed a token by mistake and wants to "just delete the file". What is the correct first step (Chapter 33)?

Chapter 39 — Your First GitHub Repository [Core]

In this chapter

- two ways to get a project onto GitHub: start there, or start on your computer
- what the platform's "create a repository" choices mean
- connecting an existing repository, and the "unrelated histories" surprise
- cloning, and the daily cycle of push, fetch and pull
- **Project 3**: publish the Sunrise Bakery website

How to read this chapter. The **Git** commands here were run against local bare repositories that behave like a new, empty remote. What the **website** offers is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026), but was **not** run on a live account, so the chapter names the choices and not the exact position of buttons.

Before you start. Chapter [38](#) (a way to sign in), Chapter [23](#) and Chapter [36](#). The recordings ran in Bash and zsh on Git 2.43.0 and were re-run in CI on Git 2.55.0.

39.1 Two routes

There are two ways to connect a local project and a hosted repository:

Route	You begin with	Then
A. Start on the platform	an empty hosted repository	you clone it, work, and push
B. Start on your computer	a repository that already has commits	you create an empty hosted repository, add it as a remote, and push

Both end in the same place: a local repository and a hosted one that follow each other. In terms of Chapter [23](#), the hosted repository is `origin`.

39.2 Creating the hosted repository

On the platform you create a repository from a web form, choosing a name and a few options.

Checked against GitHub's documentation (R237). "Creating a new repository" and the quickstart describe the form: choose the plus icon in the upper-right corner of any page and then **New repository**; choose an owner and a name ("the repository name must not exceed 100 characters, and can only contain ASCII letters, digits, and the characters `.`, `-`, and `_`"), an optional description, and a visibility. The optional items are a README, a `.gitignore` file and a software licence. The choices, in the words of this book:

- **Owner and name.** The owner is you or an organization (Chapter 37); the name becomes part of every address (Chapter 36).
- **Visibility.** Public ("accessible to everyone on the internet") or private ("only accessible to you, people you explicitly share access with, and, for organization repositories, certain organization members"; organizations that are part of an enterprise can also have internal repositories). Decide deliberately. A public repository, and everything in its history, can be copied by anyone (Chapter 33).
- **A README.** A file that describes the project (Chapter 42).
- **A `.gitignore`.** A starter list of files to ignore (Chapter 18).
- **A licence.** The terms under which others may reuse your work (Chapter 65).

The important consequence is this: if you tick any of the "add a file" options, the platform makes the first commit itself. Your hosted repository is then **not empty**. That matters for Route B (section 39.4), and GitHub's documentation says the same: "If you're importing an existing repository to GitHub, don't choose any of these options, as you may introduce a merge conflict." It also describes an "empty repository" as one that "contains no files" and is "often made if you don't initialize the repository with a README when creating it", and shows its quick-setup page with the clone address.

39.3 Route A: clone, work, push

Start with an empty hosted repository. Here a bare repository, `new-repo.git`, stands in for it. Clone it, make the first commit, and push:

```
$ git clone new-repo.git fresh
Cloning into 'fresh'...
warning: You appear to have cloned an empty repository.
done.
```

Recorded in Bash; *ch39-firstrepo/expected-first-clone.bash.txt*.

Git warns that the repository is empty, which is expected (Chapter 23 showed the same). Add a README, commit and publish:

```
$ cd fresh
$ printf '# Sunrise Bakery\n\nFresh bread every morning.\n' > README.md
$ git add README.md
$ git commit -m "Add a README"
[main (root-commit) 3034e23] Add a README
1 file changed, 3 insertions(+)
create mode 100644 README.md
$ git push -u origin main
```

```
To /home/learner/new-repo.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Recorded in Bash; *ch39-firstrepo/expected-first-clone.bash.txt*.

`git push -u origin main` created the branch on the remote and made your main follow it. Check the state:

```
$ git status -sb
## main...origin/main
$ git log --oneline
3034e23 (HEAD -> main, origin/main) Add a README
```

Recorded in Bash; *ch39-firstrepo/expected-first-clone.bash.txt*.

`## main...origin/main` says that main follows origin/main and that neither is ahead or behind. Your project is now on the "remote". If the remote were a hosting platform, this is the moment when the files would appear on its web page.

What if the hosted repository already has a README? Then you would clone it (a normal `git clone <address>`), and the README would come with you. Route A works either way.

39.4 Route B: connect an existing repository

You have a project with history, and want to publish it. Create an **empty** hosted repository (no README, no `.gitignore`, no licence: nothing that makes a first commit). The platform's own instructions usually offer three lines to copy. In Git terms they are: name your branch `main`, add the remote, push.

```
$ cd bakery-menu
$ git branch -M main
$ git remote add origin ../new-repo.git
$ git push -u origin main
To ../new-repo.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Recorded in Bash; *ch39-firstrepo/expected-connect-existing.bash.txt*.

`git branch -M main` renames the current branch to `main` if it has another name (here it already was `main`, so nothing changed). Then the address is added and the branch is pushed with `-u`. Check that the branch follows the remote, and ask the remote directly what it has:

```
$ git branch -vv
* main 81f772e [origin/main] Add coconut cake
$ git ls-remote --heads origin
81f772ee32b93d8fcf80dbb34ab74053b69d97a2 refs/heads/main
```

Recorded in Bash; *ch39-firstrepo/expected-connect-existing.bash.txt*.

The remote's main is at commit 81f772e, the same as yours. Nothing else is needed.

The surprise: "unrelated histories"

Suppose that you had ticked "add a README" when you created the hosted repository. Then it holds a commit of its own, which your project does not have. Watch what happens when you push:

```
$ cd bakery-menu
$ git remote add origin ../hub-with-readme.git
$ git push -u origin main
To ../hub-with-readme.git
 ! [rejected]          main -> main (fetch first)
error: failed to push some refs to '../hub-with-readme.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Recorded in Bash; *ch39-firstrepo/expected-unrelated.bash.txt*.

The push is **rejected**: the remote has work that you do not have. It is the same message as in Chapter 23, and the same advice applies: bring the remote's work in first. But this time:

```
$ git fetch -q origin
$ git pull --no-rebase origin main
From ../hub-with-readme
 * branch          main          -> FETCH_HEAD
fatal: refusing to merge unrelated histories
```

Recorded in Bash; *ch39-firstrepo/expected-unrelated.bash.txt*.

fatal: refusing to merge unrelated histories. The two repositories were started **separately**; they have no commit in common, so Git will not join them unless you say that you mean it. Say so with `--allow-unrelated-histories`:

```
$ git pull --no-rebase --allow-unrelated-histories --no-edit origin main
From ../hub-with-readme
 * branch          main          -> FETCH_HEAD
Merge made by the 'ort' strategy.
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

Recorded in Bash; *ch39-firstrepo/expected-unrelated.bash.txt*.

Git makes a merge commit that joins the two. Look at the graph, and push:

```
$ git log --oneline --graph
* 68c8568 (HEAD -> main) Merge branch 'main' of ../hub-with-readme
|\
| * 36b17f4 (origin/main) Initial commit
* 81f772e Add coconut cake
```

```
* 9f10b43 Raise the price of the white loaf
* 8a52ffe Add the menu
$ git push -u origin main
To ../hub-with-readme.git
   36b17f4..68c8568  main -> main
branch 'main' set up to track 'origin/main'.
```

Recorded in Bash; *ch39-firstrepo/expected-unrelated.bash.txt*.

(On Git 2.55.0 the decoration reads (*origin/main*, *origin/HEAD*): after the *git fetch*, that newer version also records which branch is the remote's default. Git 2.43.0 did not do this for a remote that you added by hand, and it is a good example of why the same commands can print slightly different things.)

The graph has two separate roots, one that the platform made ("Initial commit") and yours, joined by the merge commit. That is fine, and it is the price of starting both ends independently.

How to avoid it. For Route B, create the hosted repository **empty**. If you did tick the README box, this recovery works, but the history keeps its odd shape.

39.5 The daily cycle

Once connected, the cycle is the one from Chapter 23 and Chapter 34:

1. *git fetch* (or *git pull*) to see and bring in what others did.
2. Work on a branch; commit in small steps.
3. *git push* to publish.

git status compares with what you last fetched (Chapter 23), so fetch first when you want to know the truth.

39.6 Looking at the result

After the first push, open the repository's page in a browser and check three things:

- **Is everything there** that should be, and **nothing** that should not be? (Read the file list as a stranger would; Chapter 33.)
- **Is the visibility what you intended?**
- **Is the default branch the one you expect?**

Checked against GitHub's documentation (R238). The quickstart says the contents of the README "are automatically shown on the front page of your repository". What else the repository page shows is covered in Chapter 40.

39.7 Project 3: publish the Sunrise Bakery website

Project 3. Goal: publish your Sunrise Bakery repository (Chapters 19 and 24) to a hosted repository, and prove that it round-trips. Time: about 40 minutes. You need: the sunrise-bakery repository with several commits, a clean `git status`, and either a platform account with a way to sign in (Chapter 38) **or**, if you have none, a bare repository on your own computer (`git init --bare sunrise-bakery.git`) that plays the same role.

1. **Check first.** Run `git status` and `git log --oneline`. Read the file list with `git ls-files` and confirm that it holds nothing private (Chapter 33).
2. **Create an empty hosted repository** named `sunrise-bakery`, with **no** README, `.gitignore` or licence. Choose private for now.
3. **Connect and push.** `git branch -M main`, `git remote add origin <address>`, `git push -u origin main`. Read the output.
4. **Verify the remote.** `git ls-remote --heads origin`: does the hash equal `git rev-parse main`?
5. **Prove the round trip.** In a different folder, `git clone <address> sunrise-check`. Compare the two with `git log --oneline` in each.
6. **Change and sync.** In the clone, edit a file, commit and push. In the original, `git fetch`, look at `git status`, then `git pull`.
7. **Reflect.** Which parts of this were Git, and which were the platform?

Expected result: the hosted (or local bare) repository has your full history; the clone has the same commits; a change made in the clone reaches the original through fetch and pull.

Checkpoint questions: What would have happened if you had ticked "add a README" in step 2? How would you have recovered? Why is private a sensible first choice?

Checkpoint

What You Learned

- Route A: create an empty hosted repository, clone it, work, push. Route B: create an empty hosted repository, add it as origin, push.
- Ticking "add a README" (or similar) makes the platform create a first commit, so the hosted repository is not empty.
- Pushing to a non-empty remote with unrelated history is rejected; `git pull --allow-unrelated-histories` joins the histories with a merge commit.
- `git push -u origin main` sets the upstream; `git ls-remote --heads origin` shows what the remote has.
- Check the repository page after the first push: files, visibility and default branch.

New Vocabulary

No new terms. This chapter uses **remote**, **origin** and **upstream branch** from Chapter 23.

Commands Learned

`git branch -M main`, `git push -u origin main`, `git ls-remote --heads`, `git status -sb`, `git pull --allow-unrelated-histories`.

Common Mistakes

1. **Ticking "add a README" when connecting an existing project.**
2. **Making a repository public without checking what is in the history.**
3. **Copying the platform's three lines without reading them.**
4. **Forgetting `-u` on the first push.**
5. **Assuming that the web page shows your latest work without pushing.**

Practice

Do the exercises in the Exercises section at the end of this chapter, including Project 3 (section 39.7). Every exercise can be done with local bare repositories.

Self-Test

1. What are the two routes to a hosted repository?
2. Why should the hosted repository be empty in Route B?
3. What does refusing to merge unrelated histories mean?
4. What does `git push -u origin main` do?
5. What should you check on the repository page after the first push?

Before Moving On

You are ready for Chapter 40 if you can:

- [] publish an existing repository to an empty remote
- [] clone a remote, commit and push
- [] explain and resolve the unrelated-histories rejection
- [] say what to check after publishing

How this chapter was checked

Claim	Evidence class	Ledger
Clone of an empty remote, first push, status -sb	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0; local bare repository. Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-ls-remote</code>); the ledger row says which.	R239
Connecting an existing repository; <code>ls-remote --heads</code>	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (<code>git-ls-remote</code>); the ledger row says which.	R239

Claim	Evidence class	Ledger
Rejected push and unrelated histories; the merge that joins them	Locally tested (as above). Some concept and option statements for this row were also checked in the Git 2.56.0 manual (git-merge); the ledger row says which.	R240
The website's create form, its options and defaults, and the repository page	Checked against github/docs (commit 2eaab0b); not run on a live account	R237, R238

Where this leads

Chapter 40 walks through the repository page. Chapter 42 improves the repository's front page, and Chapter 45 turns the shared repository into a place where changes are proposed and reviewed.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. **Every exercise can be done with local bare repositories** (`git init --bare <name>.git`) in place of a hosting platform, and you may repeat them on a real platform if you have an account. Use bakery-menu (rebuild it from Chapter 17 if needed).

Level 1 — Guided

1.1 Route A. Create a bare repository `new-repo.git`, clone it, add a `README.md`, commit and push with `-u`. What warning appears when you clone, and why?

1.2 Check the link. Run `git status -sb` and `git log --oneline`. What does `## main...origin/main` tell you?

Level 2 — Partially guided

2.1 Route B. In `bakery-menu`, run `git branch -M main`, add another empty bare repository as origin, and push with `-u`. Use `git ls-remote --heads origin` to compare its hash with `git rev-parse main`.

2.2 Why empty? Explain in two sentences why the hosted repository should be empty in Route B.

Level 3 — Independent

3.1 Cause the rejection. Make a bare repository that already contains a commit (clone it, commit a `README`, push, delete the clone). Then add it as origin of `bakery-menu` and push. Read the message.

3.2 Resolve it. Bring the remote's work in with `git pull --no-rebase --allow-unrelated-histories`. Draw the graph with `git log --oneline --graph` and push.

Level 4 — Professional scenario

4.1 Project 3. Do Project 3 (section 39.7) with a local bare repository, then write three lines saying which steps were Git and which would be platform steps on a hosting service.

Level 5 — Troubleshooting

5.1 A learner ticked "add a README" and then tried to push an existing project. What message will they see, and what are two ways to proceed?

5.2 A learner pushed, but the web page shows an old version. Give two possible reasons.

5.3 A learner made a repository public and then noticed a file with a password in an early commit. What should they do first (Chapter [33](#))?

Chapter 40 — Touring the GitHub Interface [Core]

In this chapter

- how to read a repository's page as a view of Git
- which part of the page shows which part of Git
- the tabs, and which of them belong to the platform and not to Git
- how to find your way when the interface changes

How to read this chapter. A web interface changes often: names, positions and icons move between visits. The chapter was checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026), which describes each feature but not a fixed screen, and no live account was used; so the description of the interface **avoids exact button names and positions**. What is stable is the idea behind each part, which is a piece of Git that you already know. The Git equivalents below were run and recorded.

Before you start. Chapter 39 (a repository on a platform), Chapter 17, Chapter 20, Chapter 29. The recordings ran in Bash and zsh on Git 2.43.0 and were re-run in CI on Git 2.55.0.

40.1 The main idea

A repository's page is a **window onto a Git repository**, with extra services beside it. Almost everything in the window can be reproduced with a command that you already know, and knowing the command tells you what the page is really showing.

Checked in part against GitHub's documentation (R241). The documentation has a section for each feature in the table below: issues, pull requests, GitHub Actions, GitHub Discussions, projects, wikis, code security and repository settings. It also mentions, for example, the **Pulse** view under the **Insights** tab of a repository, and that the contents of the README are "automatically shown on the front page of your repository". The documentation does not fix the order or the labels of the page, so the table is a list of things to look for, not a description of exact wording.

40.2 Git on the page, and its command

You see, on a repository's page	It is showing	The Git command that gives the same information
A list of files and folders with the latest commit message beside each	the tree of one commit (Chapter 30)	<code>git ls-tree HEAD</code>

You see, on a repository's page	It is showing	The Git command that gives the same information
A branch selector	the branches (Chapter 20)	<code>git branch --all</code>
A tag list, or a release list	tags (Chapter 29)	<code>git tag</code>
A count of commits , and a history of the repository or of one file	the log (Chapter 17)	<code>git log, git log -- <file></code>
A comparison of two branches or commits	a diff (Chapter 17)	<code>git diff, git diff <a> </code>
A view of a file with the author of each line	blame (Chapter 24)	<code>git blame</code>
A download of the code as an archive	a snapshot without history	<code>git archive</code>
A clone address	the remote address (Chapter 36)	<code>git clone <address></code>

Try some of these on the sample project. The history of one file, and who made the latest commit:

```
$ cd bakery-menu
$ git log --oneline -- menu.md
81f772e (HEAD -> main) Add coconut cake
9f10b43 Raise the price of the white loaf
8a52ffe Add the menu
$ git log -1 --format='%an, %ad' --date=short
Ada Learner, 2026-01-05
```

Recorded in Bash; *ch40-tour/expected-equivalents.bash.txt*.

`git log -- menu.md` is the command behind a "history" view of a single file: only the commits that changed it. The comparison view corresponds to `git diff`. Comparing the current commit with two commits back, as a summary:

```
$ git diff --stat HEAD~2 HEAD
menu.md | 3 +-
1 file changed, 2 insertions(+), 1 deletion(-)
```

Recorded in Bash; *ch40-tour/expected-equivalents.bash.txt*.

The list of branches, and a tag (a "release" on a platform is built on a tag, Chapter 29):

```
$ git branch --all
* main
$ git tag -a v1.0 -m "First public menu"
$ git tag
v1.0
```

Recorded in Bash; *ch40-tour/expected-equivalents.bash.txt*.

And the "download the code" button gives you the files of one commit **without** the history. `git archive` does the same. The recording lists the archive's content, using `tar -t`:


```
$ git archive --format=tar --prefix=bakery-menu-v1.0/ v1.0 | tar -t
bakery-menu-v1.0/
bakery-menu-v1.0/menu.md
```

Recorded in Bash; *ch40-tour/expected-equivalents.bash.txt*.

The archive holds the files under a folder name, and no `.git` folder: it is a **snapshot**, not a repository. If you download the archive, you cannot run `git log` in it. To get the history, clone (Chapter 39).

40.3 What is not Git

Some tabs and sections belong to the **platform**. They are stored by the service, not in the repository (Chapter 36), and a `git clone` does not bring them:

Typical section	Chapter
Issues	Chapter 44
Pull requests	Chapter 45
Actions	Chapter 54
Projects and Discussions	Chapter 47, Chapter 48
Wiki	Chapter 43
Security	Chapter 62
Insights	Chapter 51
Settings	Chapter 43

Checked in part against GitHub's documentation (R242). That availability varies is confirmed for the parts the documentation covers: repository visibility (public, private, and internal for enterprise organizations), permissions, and the plan (for example, private repositories on the free plan have a limited feature set). Do not assume that a missing tab is a mistake.

40.4 Where things are, when the interface changes

- **Look for the idea, not the label.** If you need "history", look for a clock, a list of commits, or the word commits.
- **Use the search and the keyboard shortcuts** that the platform offers, and its help pages (Chapter 35).
- **Check the date** of any screenshot or tutorial (Chapter 35).
- **Prefer addresses to clicks.** A repository's pages have addresses that you can read and share; the parts of an address name the owner, the repository, and what you are viewing.
- **When in doubt, go back to Git.** A clone and the commands in Parts III and IV always work, whatever the web page looks like.

40.5 A tour to do yourself

With a repository of your own (Chapter 39), or any **public** repository, visit and write one line about each:

1. The **file list**: which is the newest commit, and how do you know?
2. The **branch selector**: how many branches are there?
3. The **history**: who made the first commit?
4. The **history of one file**.
5. A **comparison** between two commits or branches.
6. The **tags or releases**, if any.
7. The **clone address**: can you find both the HTTPS and the SSH address (Chapter 38)?
8. **Which tabs** belong to the platform and not to Git?

For each, name the Git command that gives the same information. If you cannot, look at Chapter 17 again.

Note (R243). These steps were not run on a live interface. If a step does not match what you see, use the "idea, not label" rule, and tell the author.

Checkpoint

What You Learned

- A repository's page is a view of a Git repository, plus platform services.
- Files, branches, tags, history, comparisons, blame and downloads each correspond to a Git command.
- `git archive` produces the files of one commit without history; a download from the page is such a snapshot.
- Issues, pull requests, actions, projects, wiki, security and settings belong to the platform and are not in a clone.
- When the interface changes, look for the idea, not the label, and fall back on Git.

New Vocabulary

No new terms.

Commands Learned

`git ls-tree HEAD`, `git log -- <file>`, `git branch --all`, `git archive --format=tar`.

Common Mistakes

1. **Assuming that a downloaded archive contains the history.**

2. **Assuming that a missing tab means an error.**
3. **Following an old screenshot literally.**
4. **Expecting a clone to include issues and pull requests.**
5. **Not checking what is visible to strangers.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Which Git command gives the same information as the file history on a repository page?
2. What does the "download the code" button give you that a clone does not, and the other way round?
3. Name three sections of a repository page that are not Git.
4. What should you do when a tutorial's screenshot does not match your screen?

Before Moving On

You are ready for Chapter [41](#) if you can:

- [] map each part of the page to the Git idea behind it
- [] separate Git parts from platform parts
- [] find your way when labels change

How this chapter was checked

Claim	Evidence class	Ledge r
Git equivalents: file history, latest commit, diff summary, branches, tag, archive	Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on Git 2.55.0	R243 (Git side)
The layout and names of the repository page, and which sections exist	Feature sections checked against github/docs; exact layout not fixed by the documentation and not seen on a live account	R241, R242
The tour steps	Not run on a live interface.	R243

Where this leads

Chapter [41](#) covers how the platform tells you about activity. Chapter [42](#) shows how to make a repository's front page useful, and Chapter [43](#) covers the Settings, Security and Wiki sections.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your own computer; 4 and 5 need a browser and any repository that you may view (your own, or a public one).

Level 1 — Guided

1.1 The history of one file. In `bakery-menu`, run `git log --oneline -- menu.md`. Which part of a repository page does this correspond to?

1.2 A snapshot. Run `git archive --format=tar --prefix=bakery/ HEAD | tar -t`. Is there a `.git` folder in the list?

Level 2 — Partially guided

2.1 The comparison. Run `git diff --stat HEAD~2 HEAD`. Which view of a platform shows the same information?

2.2 A tag as a release. Create an annotated tag `v1.0` and list the tags. Why is a "release" on a platform built on a tag (Chapter 29)?

Level 3 — Independent

3.1 Make a table. For each part of a repository page (files, branches, tags, history, compare, blame, download, clone address), write the Git command that gives the same information.

3.2 Git or platform? Sort these into "in a clone" and "not in a clone": commits, branches, tags, issues, pull requests, the wiki, the `.gitignore` file, workflow files.

Level 4 — Professional scenario

4.1 The tour. Do the tour in section 40.5 on any repository. For each of the eight items, write one line and the matching Git command. Note anything that did not match this chapter.

Level 5 — Troubleshooting

5.1 A learner downloads the archive of a project and runs `git log`. Git says it is not a repository. Explain.

5.2 A learner follows a tutorial that says "click the third icon at the top", but there is no such icon. What should they do?

5.3 A learner cannot find the Actions tab on a colleague's repository and thinks that it was deleted. Give two other explanations.

Chapter 41 — Notifications, Stars and Watching [Core]

In this chapter

- how the platform decides to tell you about activity
- the difference between **watching**, **starring** and **following**
- how to keep the volume of notifications under control
- how long notifications are kept

How to read this chapter. This chapter is about a service, not about Git, so it has **no recorded commands**. Its statements are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026). No live account was used, so buttons are named by their idea and by the label the documentation gives, and labels can change. Nothing in it changes what is in your Git repository.

Before you start. Chapter [40](#) (the parts of a repository page) and Chapter [37](#) (your account and its email addresses).

41.1 Three different actions

Three actions look alike and do different things.

Action	What it means	Effect
Watch a repository	"Tell me about activity here."	You are subscribed to updates, so you receive notifications.
Star a repository	"Save this, and show appreciation."	The repository is added to your list of stars. It does not , by itself, send you notifications.
Follow a person or an organization	"Show me their public activity."	Their public activity appears on your personal dashboard.

New term: notification. A message from the platform that something happened in a place you are subscribed to: a comment, a mention, a review request, the result of an automatic check. It arrives in an inbox on the website, in a mobile app, by email, or by some mix of these.

Checked against GitHub's documentation (R244). "Notifications are updates that you receive for specific activity that you are subscribed to." The documentation lists what you can subscribe to: a conversation in a specific issue, pull request or gist; all activity in a repository; activity of automatic checks (GitHub Actions); and, if enabled, chosen kinds of events in a repository. Of starring it says: "Starring makes it easy to find a repository or topic again later," and that it "shows appreciation to the repository maintainer"; the stars page lists what you have starred, and adding /stargazers to a repository's address shows who has starred it. Following people, it says, puts "their public activity" on your personal dashboard, and following can also lead the platform to recommend repositories.

41.2 What subscribes you without asking

You do not have to press **Watch** to receive notifications. According to GitHub's documentation you are subscribed to a conversation by default when you:

- are assigned to an issue or pull request;
- open an issue or pull request;
- comment on a thread;
- have your username @mentioned (or a team you belong to is mentioned);
- change the state of a thread, for example by closing an issue or merging a pull request;
- subscribe manually with **Watch** or **Subscribe**.

By default you also watch automatically the repositories that you create under your personal account, and, unless you switch it off in your notification settings, repositories and teams that you join.

New term: subscription. An arrangement that says "send me notifications about this conversation or repository". You can end it with **Unsubscribe**. If you unsubscribe and later somebody mentions you, or a team you are on, you start to receive notifications from that conversation again.

Why this matters. A busy project can produce hundreds of messages a day. Most people who feel overwhelmed have not chosen to watch everything: they were subscribed by taking part.

41.3 Watching, in levels

A repository's **Watch** control is not on or off. The documentation describes these choices:

Level	You receive
Only when you take part or are mentioned	Notifications for conversations you are in, and mentions
All activity	Everything in the repository, including new issues and pull requests
Ignore	Nothing, not even mentions

Level	You receive
Custom	Only the kinds of event you choose (for example issues, pull requests, releases), where the platform offers it

The documentation adds a warning about **Ignore**: "you won't be notified if you're @mentioned", and it does not recommend ignoring a repository for that reason. Choosing **only when participating or mentioned** is the safer way to quieten a repository.

Two limits are stated: you can watch **at most 10,000 repositories**, and a page (github.com/watching) lists everything you watch, with an **Unwatch all** option for one owner's repositories.

A good starting rule. Watch all activity only in the few repositories that you maintain or that your work depends on. Everywhere else, take part or be mentioned.

41.4 The inbox

The **notifications inbox** (at github.com/notifications) is the website's list of your notifications. GitHub's documentation lists these ways to work through it:

- **Done** removes a notification from the inbox; `is:done` finds the ones you finished.
- **Save** flags a notification for later; `is:saved` finds them.
- Mark as read or unread.
- **Unsubscribe** ends the subscription and removes the notification.
- Each notification shows a **reason** label, such as `mention`, `subscribed` or `review requested`, and you can filter by it: `reason:review-requested` shows pull requests waiting for your review.
- You can group by repository or by date and create your own **custom filters**, for example one that shows only the notifications from a project you contribute to in which you are mentioned.

Retention. "Notifications that are not marked as Saved are kept for 3 months. Notifications marked as Saved are kept indefinitely." Email is a second copy: your email client keeps messages for as long as your mail provider does.

41.5 Where notifications go

You choose the places separately for the two big groups, **participating** and **watching**, in your notification settings. The documentation names three places: the inbox on the website, the inbox in the mobile app (which stays in step with the website), and email, which needs a verified email address. If you turn off the web and mobile option for both groups, the inbox stays empty. If you turn off email for both groups, your mail stays quiet.

Depending on the organization that owns a repository, you can also send its notifications to a different address, for example work mail for work repositories, and your organization may require the address to be verified for its domain. Every notification email carries **headers** that name the repository and the reason, which you can use to filter mail in your email program.

CAUTION. A notification email can contain the text of a private conversation, and it goes to whatever address you chose. Use an address you control, and be careful about forwarding rules.

41.6 Keeping the volume under control

1. **Decide what you need to act on**, and set only participating and mentions everywhere else.
2. **Review github.com/watching now and then**, and unwatch what no longer matters.
3. **Use the inbox as a to-do list:** **Done** when handled, **Save** what must wait.
4. **Filter, do not read everything.** Start with `reason:mention` and `reason:review-requested`.
5. **Choose an email address for each purpose**, so that mail rules can sort by it.

Stars, by contrast, cost nothing in attention: a star does not subscribe you. If you want to hear about a project's releases without every conversation, use the custom watch level where it is offered.

Checkpoint

What You Learned

- Watching subscribes you to a repository's activity; starring saves it and shows appreciation; following shows a person's public activity on your dashboard.
- Taking part in a conversation subscribes you to it automatically.
- Watch levels range from "participating and mentions" through "all activity" to "ignore" (which also hides mentions).
- The inbox offers Done, Save, filters and reason labels; unsaved notifications are kept for three months.
- Delivery to the website, the mobile app and email is set separately.

New Vocabulary

Notification, subscription (introduced above).

Commands Learned

None: this chapter is about the platform and not about Git.

Common Mistakes

1. **Thinking that a star is a subscription.**
2. **Watching all activity everywhere**, then ignoring the inbox.
3. **Choosing "Ignore" to quieten a repository**, and missing a mention.
4. **Sending notifications to an address other people can read.**

5. **Treating the inbox as an archive** when unsaved items disappear after three months.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name three things that subscribe you to a conversation without pressing Watch.
2. What is the difference between starring and watching?
3. Why does the documentation not recommend ignoring a repository?
4. How long are notifications kept, and what changes that?
5. Which query shows the pull requests where your review was requested?

Before Moving On

You are ready for Chapter [42](#) if you can:

- [] explain watch, star and follow in one sentence each
- [] say what subscribed you to a conversation
- [] choose a watch level for a repository on purpose

How this chapter was checked

Claim	Evidence class	Ledger
Subscriptions, default subscriptions, inbox actions, retention	Checked against <code>github/docs</code> (commit <code>2eaab0b</code>); no live account	R244, R245
Watch levels, limits, delivery places	Checked against <code>github/docs</code> ; labels may change	R245
Stars, following, stargazers	Checked against <code>github/docs</code>	R246

Where this leads

Chapter [42](#) makes a repository's front page useful to the people who arrive there. Chapter [44](#) and Chapter [45](#) are where most notifications come from.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises need a browser and a GitHub account; if you have none yet, do the paper versions (1.1, 2.1, 3.1, 5.1), which need only the chapter.

Level 1 — Guided

1.1 Three actions. Write one sentence each for watch, star and follow. Which of the three subscribes you to notifications?

1.2 Find your places. On your account, find the notification settings and the page that lists the repositories you watch. Write down the two addresses (the page names, not the buttons).

Level 2 — Partially guided

2.1 Why did I get this? A notification is labelled subscribed, and you never pressed Watch. List two things from section 41.2 that could explain it.

2.2 Set a level. Choose a repository you do not maintain. Set it to notify you only when you take part or are mentioned. Say what you will no longer receive.

Level 3 — Independent

3.1 A filter. Write the inbox query that shows only the notifications where your review was requested.

3.2 Clean-up. Open the page of watched repositories. Unwatch at least three that you do not need. Write down how many you watched before and after.

Level 4 — Professional scenario

4.1 A busy project. You have joined a project that produces about 200 notifications a day. Write a plan of five steps (watch levels, filters, addresses, a routine for Done and Save) that keeps you informed about mentions and review requests only.

Level 5 — Troubleshooting

5.1 A learner set a repository to **Ignore** to stop the noise, and later a teammate mentioned them in a question that went unanswered. Explain what happened and what setting they should have chosen.

5.2 A learner says that notifications from last spring have vanished from the inbox. What is the likely reason, and what should they have done with the ones they wanted to keep?

Part VI — Working with GitHub

Chapter 42 — Professional Repositories and READMEs [Core]

In this chapter

- what a README is for, and where GitHub looks for it
- a README skeleton for each common kind of project
- links inside a repository that keep working in a clone
- the description, website and topics that sit beside a README
- a checklist for a repository that a stranger can understand

How to read this chapter. The Git and file parts were run and recorded. The statements about GitHub are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used. The README skeletons are **this book's suggestions**, not GitHub rules, and each says so.

Before you start. Chapter [39](#) (a repository on the platform) and Chapter [8](#) (the writing format). The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on newer Git versions.

42.1 What a README is for

New term: README. A file in a repository, usually named `README.md`, that introduces the project to the people who arrive: what it is, why it is useful, how to start, where to get help and who looks after it.

A stranger decides in seconds whether a project is worth their time, and the README is the first thing they read. GitHub's documentation lists what READMEs typically contain:

- what the project does
- why the project is useful
- how users can get started
- where users can get help
- who maintains and contributes to the project

Checked against GitHub's documentation (R247). "About READMEs" says a README "is often the first item a visitor will see". If you put it in the hidden `.github` folder, the root, or a `docs` folder, GitHub "will recognize and automatically surface" it; when there is more than one, the one shown is chosen from `.github`, then the root, then `docs`. In the rendered view "any content beyond 500 KiB will be truncated". GitHub also builds a table of contents from the headings of any rendered Markdown file, and offers links to headings. The same page says a README "should only contain information necessary for developers to get started using and contributing to your project" and that longer documentation is "best suited for wikis".

Keep it short at the top. The first screen should answer what is this? in two sentences, and how do I try it? in a few lines. Details go lower, or in other files.

42.2 A first README

The smallest useful README for the Sunrise Bakery project:

```
# Sunrise Bakery menu

The weekly menu of Sunrise Bakery, kept as a plain Markdown file so that
anyone on the team can update it.

## Use it

Open `menu.md`. Prices are in the local currency.

## Change it

Edit `menu.md`, describe the change in one line, and open a pull request.

## Who looks after it

Ada Learner (`@example-owner`).
```

Nothing here needs a tool: it is Markdown (Chapter 8). What makes it good is that each heading answers one of the questions above.

42.3 Skeletons for different kinds of project

These are **suggestions from this book**, not rules of the platform. Take the headings that fit, and delete the rest. The bracketed words are placeholders you replace.

Kind of project	Headings that usually help
Beginner / learning project	What I built · What I learned · How to run it · What I want to improve
Web application	What it does · Screenshot · Run it locally · Configuration (never real secrets) · Tests · Deploy · Licence

Kind of project	Headings that usually help
Mobile application	What it does · Screenshots · Supported systems · Build and run · Permissions it asks for · Privacy · Licence
Open-source project	What it is · Install · Quick example · Documentation · Contributing (link to CONTRIBUTING) · Code of conduct · Security reports · Licence
Library	What it does · Install · Minimal example · Supported versions · API reference link · Changelog · Licence
API service	What it does · Base address · Authentication (how to get access, no keys in the file) · Example request and response · Errors · Rate limits · Support
Command-line tool	What it does · Install · Usage with two or three examples · Options · Exit codes · Licence
Business or internal project	Purpose · Owner and team · How to get access · How to run and deploy · Who to ask · Links to runbooks

A README for a **library** shows the smallest working example near the top, because a reader wants proof that it does what they need:

```
# menu-tools

Small functions for reading a bakery menu file.

## Install

Copy `menu_tools.py` into your project.

## Example

```python
from menu_tools import read_menu

menu = read_menu("menu.md")
print(menu["White loaf"])
```

## Licence

See `LICENSE`.
```

CAUTION. Never put a real password, key or token in a README, even as an "example". Use a clearly fake value, and say so (Chapter 33).

42.4 Links that keep working

A README often points to other files in the repository. GitHub's documentation recommends **relative links**: a link to a file relative to the current file. GitHub "will automatically transform your relative link or image path based on whatever branch you're currently on, so that the link or path always works", links starting with / are relative to the repository root, and ./ and ../

work. The link text "should be on a single line". The documentation adds: "Relative links are easier for users who clone your repository. Absolute links may not work in clones of your repository."

The link is only text. Git does not know it is a link, so **renaming the file breaks it silently**. Here is the whole cycle:

```
$ cd bakery-menu
$ mkdir docs
$ printf '# Sunrise Bakery menu\n\nSee [how to help](docs/CONTRIBUTING.md).\n' > README.md
$ printf 'Be kind.\n' > docs/CONTRIBUTING.md
$ grep -o '](.*)' README.md
](docs/CONTRIBUTING.md)
$ test -f docs/CONTRIBUTING.md && echo "target found"
target found
$ git add README.md docs
$ git commit -q -m "Add a README and a contributing guide"
$ git mv docs/CONTRIBUTING.md docs/HELP.md
$ test -f docs/CONTRIBUTING.md || echo "target missing: the link is now broken"
target missing: the link is now broken
$ git status --short
R docs/CONTRIBUTING.md -> docs/HELP.md
```

Recorded in Bash; *ch42-readme/expected-relative-links.bash.txt*.

The link `docs/CONTRIBUTING.md` was found while the file existed. After `git mv` renamed it to `HELP.md`, the same test reports the target missing, and `git status` shows only a rename: nothing warned that the README now points nowhere. That is why a repository worth its name is checked for broken links before a release (the same idea as this book's own link check).

Checked against GitHub's documentation (R248). The relative-link behaviour above is quoted from "About READMEs" and the shared text it includes. The recorded commands show only the Git side: a link is plain text and a rename does not update it.

42.5 The description, website and topics

Beside the README, the repository page has a short **About** area with a description, an optional website address and **topics**.

New term: topic. A short label that classifies a repository by purpose, subject area, community or language, so that people can find related projects.

Checked against GitHub's documentation (R249). "Classifying your repository with topics" says topics "appear on the main page of a repository", that clicking a topic shows other repositories with it, and that repository admins can add any topics they like. When creating a topic: "use lowercase letters, numbers, and hyphens", "use 50 characters or less", and "add no more than 20 topics". Topic names "are always public, even if you create the topic from within a private repository". On the public site GitHub may suggest topics for public repositories, which admins can accept or reject; private repositories get no suggestions. You edit them from the gear icon beside "About".

Practical advice.

- Write the description as one plain sentence: what it is and who it is for.
- Add three to six topics that a stranger would search for.
- Do not put anything private in a topic or description: both are public for a public repository.

42.6 Files that sit beside a README

A README is one of several files that communicate expectations. GitHub's documentation groups them: "A README, along with a repository license, citation file, contribution guidelines, and a code of conduct, communicates expectations for your project and helps you manage contributions." Later chapters cover each: the licence (Chapter 65), contribution guidelines and code of conduct (Chapter 64) and the security policy (Chapter 62).

42.7 A checklist

1. The first two sentences say what the project is.
2. A reader can run or use it in five minutes by following the README.
3. Every link works, and links to files in the repository are relative.
4. No secret appears anywhere, including the history (Chapter 33).
5. The About description and topics are filled in and contain nothing private.
6. It says who looks after it and where to ask.
7. It names the licence, or says plainly that none has been chosen (Chapter 65).

Checkpoint

What You Learned

- A README introduces a project: what it is, why it helps, how to start, where to get help, who maintains it.
- GitHub shows a README from `.github`, the root or docs (in that order of preference) and truncates beyond 500 KiB.
- Different kinds of project need different headings; the skeletons are suggestions.
- Relative links keep working in clones; a rename silently breaks them, and Git does not warn you.

- Topics are lowercase, at most 50 characters, at most 20 per repository, and always public.

New Vocabulary

README, **topic** (introduced above).

Commands Learned

`grep -o`, `test -f`, `git mv` (used for the link demonstration).

Common Mistakes

1. **A README that only says the project name.**
2. **Absolute links to files in the same repository.**
3. **Renaming a file without fixing the links that point to it.**
4. **Real secrets in "example" configuration.**
5. **Topics or descriptions that reveal something private.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Which folders does GitHub search for a README, and which wins?
2. Why is a relative link better than an absolute one for files in the same repository?
3. What happens to a link when its target file is renamed?
4. Give the limits for topics.
5. Which two sentences should open every README?

Before Moving On

You are ready for Chapter [43](#) if you can:

- [] write a README that answers the five questions
- [] choose the right skeleton for a kind of project
- [] find a broken relative link

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|-----------------|
| A link is plain text; a rename leaves it broken; <code>git mv</code> shows as a rename | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R248 (Git side) |
| README location, order, size limit, contents, relative links | Checked against github/docs (commit 2eaab0b) | R247, R248 |

| Claim | Evidence class | Ledger |
|--------------------------------------|---|--------|
| Topic rules, About area | Checked against github/docs | R249 |
| README skeletons per kind of project | Book's own suggestions , not sourced rules | R247 |

Where this leads

Chapter 43 tours the Settings, Security and Wiki sections. Chapter 44 and Chapter 45 cover how people contribute once they have read the README.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer.

Level 1 — Guided

1.1 Five questions. Write down the five questions a README should answer.

1.2 A first README. In bakery-menu, create README.md with a title, a two-sentence description and headings "Use it" and "Who looks after it". Commit it.

Level 2 — Partially guided

2.1 A relative link. Add docs/CONTRIBUTING.md, and link to it from the README with a relative link. Use `grep -o '](.*)' README.md` and `test -f` to check that the target exists.

2.2 Break it. Rename the target with `git mv`. Run your check again. What does `git status --short` say, and what does it not say?

Level 3 — Independent

3.1 A library README. Write a README for a small library of your own with an install line and a minimal working example near the top.

3.2 A link checker. Write a shell loop that reads every `](...)` target in README.md that does not start with `http` and prints missing: `<target>` for each one that is not a file.

Level 4 — Professional scenario

4.1 A stranger's test. Give your README to someone who has not seen the project. Ask them to say, after one minute, what it is and how to try it. Rewrite the opening until they can.

Level 5 — Troubleshooting

5.1 After a clean-up, every link in a project's README to docs/setup.md leads nowhere, although `git log` shows nothing wrong. Explain, and say how to find every affected link.

5.2 A team added the topic Internal-Billing-Tool-v2 to a public repository. Name two things that are wrong with it.

Chapter 43 — Repository Settings, Security Tab and Wiki Tour [Core]

In this chapter

- what lives in a repository's **Settings**, and which changes are hard to undo
- what the **Security** tab is for, and why its label varies
- the **wiki**, which turns out to be a Git repository
- the **Deployments** view, and why views depend on plan, visibility and role

How to read this chapter. The wiki part was run and recorded with local repositories. Everything about GitHub is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used, and labels can change. The chapter is a **tour**: later chapters cover each feature in depth.

Before you start. Chapter 40 (the parts of a repository page) and Chapter 39. The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on newer Git versions.

43.1 Who sees what

Not everybody sees the same tabs. What appears on a repository depends on four things:

- the **visibility** (public, private, or, for enterprise organizations, internal);
- your **role** on the repository (Chapter 49);
- the **plan** of the owner (some features exist only on paid plans);
- the **settings** of the repository and of its organization.

If a section named in this chapter is missing on your screen, do not assume a mistake. Check these four things first.

43.2 Settings

Settings is where a repository's administrators change how the repository behaves. GitHub's documentation describes many changes there. The ones a beginner should know, with what the documentation says about each:

| Setting | What it does | What to know |
|-------------------|---|--|
| Visibility | Public or private (and internal in enterprise organizations) | Changing it has consequences. Making a repository private detaches public forks into a new network; making it public makes its Actions history and logs visible to everyone. |
| Features | Switches issues, wiki, discussions, projects and other sections on or off | If you switch issues off and on again, "any issues that were previously added will be available". |

| Setting | What it does | What to know |
|-----------------------|--|--|
| Default branch | The branch that the page shows and that new clones check out | Your account also has a setting for the default branch name of new repositories. |
| Danger zone | Change visibility, transfer or delete | Deleting is not always undoable, see below. |

Checked against GitHub's documentation (R250). "Setting repository visibility" lists consequences of a change: for example, making a repository private detaches public forks into a new network, and making one public means "Actions history and logs will be visible to everyone". "Disabling issues" says previously added issues are available again if you re-enable them. The default branch name for new repositories is set in your account settings. "Deleting a repository" says that deleting permanently deletes team permissions and that this "cannot be undone"; deleting a public repository does not delete its forks, but deleting a private one deletes all its forks. "Some deleted repositories can be restored within 90 days", but not one that was part of a fork network that is not empty. A transfer gives the new owner immediate administration, and for a transfer to another personal account the new owner gets a confirmation email; the invitation expires after one day.

CAUTION. Settings changes are made on the platform, and **Git will not record them**: your history does not show that a repository was made public, or which feature was switched off. Treat the danger zone the way you treat `git reset --hard` (Chapter 25): stop, read, and be sure. Before deleting, make sure that at least one clone has all the history you need (Chapter 23).

43.3 The Security tab

A repository page has a tab for security features: where alerts about leaked secrets, vulnerable dependencies and code problems are collected. GitHub's documentation gives the tab a name that **depends on a feature flag**: in its own source the label is written as "Security and quality" when the flag `security-and-quality-tab` is on, and "Security" otherwise. So two people can see different names for the same tab. That is the best example in this book of why the chapters teach ideas and not labels (Chapter 40).

Checked against GitHub's documentation (R251). The documentation source defines the tab's label as "Security and quality" or "Security" depending on a flag. Secret-scanning alerts appear on this tab ("GitHub generates an alert on your repository's ... tab"). What each security feature does, and which plans include it, is covered in Chapter 62; this chapter does not describe them.

43.4 The wiki

Every repository can have a **wiki**: a section for long-form documentation. GitHub's documentation says a README "quickly tells what your project can do", while a wiki is for "additional documentation" such as "how to use it, how you designed it, or its core principles".

The interesting fact for a Git learner is in the documentation of editing: "Wikis are part of Git repositories, so you can make changes locally and push them to your repository using a Git workflow." A wiki is **a second Git repository**, with its own history, that lives beside the code repository. Its address is the repository's address with `.wiki` before `.git`. The documentation gives the pattern `https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.wiki.git`, and says it can be cloned once an initial page exists.

To see this without a platform, the recording below uses a local bare repository as the wiki, in the same way that Chapter 23 used one as a remote:

```
$ git clone -q bakery-menu.wiki.git bakery-wiki
$ cd bakery-wiki
$ ls
Home.md
$ git log --oneline
3ecec76 (HEAD -> main, origin/main, origin/HEAD) Create the Home page
$ printf '# Recipes\n\nHow we bake the white loaf.\n' > Recipes.md
$ git add Recipes.md
$ git commit -q -m "Add the Recipes page"
$ git push -q origin main
$ git log --oneline
34279fc (HEAD -> main, origin/main, origin/HEAD) Add the Recipes page
3ecec76 Create the Home page
$ git -C ../bakery-menu.wiki.git log --oneline main
34279fc (HEAD -> main) Add the Recipes page
3ecec76 Create the Home page
```

Recorded in Bash; *ch43-settings/expected-wiki.bash.txt*.

The page `Home.md` came with the clone. A new page `Recipes.md` was added with an ordinary `git add`, `git commit` and `git push`, and the wiki repository's own log shows both commits. Nothing about the code repository changed: the two histories are separate.

Checked against GitHub's documentation (R252). Points from the wiki pages: every repository has a wiki section; it can be edited on GitHub or locally; "only changes pushed to the default branch will be made live" (branches are allowed); by default only people with write access can edit, though everyone can be allowed in a public repository; a wiki in a public repository is public and one in a private repository is visible only to people with access; "search engines will only index wikis with 500 or more stars that you configure to prevent public editing"; and wikis have "a soft limit of 5,000 total files", beyond which GitHub recommends GitHub Pages. The recorded commands show only the Git side, with a local stand-in for the platform.

A wiki or a docs folder? A wiki has its own history and permissions, so people without write access to the code can be allowed to improve it, and its changes are not reviewed by pull request. A docs folder in the code repository is reviewed like code and versioned with it. Choose on that trade-off. (This is the book's advice, not a rule of the platform.)

43.5 Deployments and environments

If a repository deploys anything (Chapter 58), the platform keeps a **Deployments** page. GitHub's documentation says it shows currently active deployments in each environment, the full history, the commit that triggered each deployment, the connected workflow logs, the deployment address if one exists, and the pull request and branch behind it. You open it from the right-hand sidebar of the repository's front page.

Checked against GitHub's documentation (R253). "Viewing your repository's deployment history" lists what the page shows, and says administrators can pin up to ten environments. Environments on the Free plan can only be configured for public repositories; "organizations with Team and users with Pro can configure environments for private repositories". Creating environments and their rules is Chapter 58's subject.

Checkpoint

What You Learned

- What a repository shows depends on visibility, role, plan and settings.
- Settings changes such as visibility, transfer and deletion happen on the platform and are not recorded in Git; deletion may be restorable for 90 days, but not always.
- The name of the security tab can differ between accounts.
- A wiki is a separate Git repository (<repository>.wiki.git); only changes pushed to its default branch go live.
- The Deployments page shows the history of deployments to environments.

New Vocabulary

Wiki (introduced above).

Commands Learned

`git clone <wiki address>`, `git push` (to a wiki, exactly as to any remote).

Common Mistakes

1. **Changing visibility without reading the consequences.**
2. **Deleting a repository before checking that a clone holds the history.**
3. **Assuming a missing tab is an error.**

4. **Putting private notes in a public repository's wiki.**
5. **Expecting the code repository's history to show settings changes.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name four things that decide which tabs you see.
2. Why is a wiki "a second Git repository"?
3. What is the risk of switching a repository from public to private, and of deleting it?
4. Why might two people see different names for the security tab?

Before Moving On

You are ready for Chapter [44](#) if you can:

- [] say what settings changes Git does not record
- [] clone and push to a wiki-style repository
- [] explain why a missing tab is not necessarily a mistake

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|---|-----------------|
| Clone, edit and push to a wiki-style local repository; separate histories | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R252 (Git side) |
| Settings consequences, deletion, transfer, features | Checked against github/docs (commit 2eaab0b); not run on a live account | R250 |
| Security tab label depends on a flag | Checked in the documentation source | R251 |
| Wiki rules and limits | Checked against github/docs | R252 |
| Deployments page, environments by plan | Checked against github/docs | R253 |

Where this leads

Chapter [44](#) covers the first collaboration feature in depth. Chapter [62](#) returns to the security features, and Chapter [58](#) to environments and deployments.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer.

Level 1 — Guided

1.1 A wiki stand-in. Create a bare repository `demo.wiki.git`, clone it, add `Home.md`, commit and push. Which two histories now exist, and where?

1.2 Four factors. List the four things that decide which tabs you see on a repository.

Level 2 — Partially guided

2.1 Add a page. Clone your stand-in again to a second folder, add `Recipes.md` there, and push. Then pull in the first clone. What did the pull bring?

2.2 A branch on the wiki. Push a branch draft to the stand-in with one new page. According to the chapter, will the page be live on a real wiki? Why or why not?

Level 3 — Independent

3.1 Wiki or docs? For a project with five contributors and outside readers, decide whether the long documentation belongs in a wiki or in a docs folder. Give two reasons from section 43.4.

3.2 The deletion checklist. Write a five-item checklist to complete before deleting a repository.

Level 4 — Professional scenario

4.1 Going public. A team wants to make a private repository public. Using section 43.2, list the consequences to check first, and one action to take before the change.

Level 5 — Troubleshooting

5.1 A colleague says "my security tab is called something different from yours". Give the likely reason, and say whether it is a problem.

5.2 A learner deleted a repository last month and wants it back. What does the documentation say about the time limit, and what other condition can prevent restoration?

Chapter 44 — Issues [Core]

In this chapter

- what an issue is, and what belongs in one
- labels, assignees and milestones
- how to reference issues, and how a pull request closes one
- issue templates and issue forms
- how Git commit messages and issues meet

How to read this chapter. The Git and file parts (an issue form file, a closing keyword in a commit message) were run and recorded. Statements about GitHub are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used, and labels can change.

Before you start. Chapter [39](#) and Chapter [8](#). The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on newer Git versions.

44.1 What an issue is

New term: issue. A numbered item in a repository's tracker: a bug report, a feature idea, a question, or any task that someone wants to write down and discuss. It has a title, a description, a conversation, and a state (open or closed).

GitHub's documentation says issues are for "plan, discuss, and track work", and that they "can track bug reports, new features and ideas, and anything else you need to write down or discuss with your team". An issue lives on the platform: it is **not** in your Git repository, and a normal clone does not bring it (Chapter [36](#)).

An issue has a number that is unique inside its repository (#12) and is the way everything else refers to it.

What makes a good issue. Whoever reads it should be able to act without asking you anything:

1. A title that says the problem, not the feeling: "Rolls price shows 3.00 on menu but 3.20 in shop", not "Menu broken".
 2. What you did, what you expected, what happened instead.
 3. Where: which file, page or version.
 4. Evidence: the exact text of an error, and a small example. Never a secret (Chapter [33](#)).
-

44.2 Organising issues

Checked against GitHub's documentation (R254). "About issues" lists the metadata you can add: issue types, **labels** and **milestones**, plus sub-issues and dependencies for breaking work down. New repositories come with default labels: bug (an unexpected problem or unintended behavior), documentation, duplicate, enhancement (new feature requests), good first issue (a good issue for first-time contributors), help wanted, invalid, question, wontfix and accessibility; you can edit or delete them later. Issues labelled good first issue "are used to populate the repository's contribute page". A **milestone** has a description, a due date, a completion percentage and lists of open and closed issues and pull requests. Issues and pull requests "support up to 10 assignees".

| Tool | What it answers |
|------------------|--|
| Label | What kind of thing is this? (bug, documentation, question) |
| Assignee | Who is doing it? |
| Milestone | When or in which group of work does it belong? |
| Reference | What else is related? |

Use few labels, and use them the same way. A team that invents forty labels ends with none that anyone trusts.

44.3 References and closing keywords

Writing #12 in an issue, a pull request or a comment makes a link to issue 12 in the same repository; the documentation says "mentioning an issue in another issue or pull request will create references between them".

A pull request can also **close** an issue when it is merged.

Checked against GitHub's documentation (R255). "Linking a pull request to an issue" says you can link them with a supported keyword in the pull request description or in a commit message. The keywords are close, closes, closed, fix, fixes, fixed, resolve, resolves and resolved. The syntax is KEYWORD #ISSUE-NUMBER for an issue in the same repository (for example Closes #10), and KEYWORD OWNER/REPOSITORY#ISSUE-NUMBER for another repository; for several issues, "use full syntax for each issue". The keywords "can be followed by colons or in uppercase". When the linked pull request is merged into the **default branch**, the issue is closed automatically. The keywords are "interpreted only when the pull request targets the repository's default branch": for any other branch "these keywords are ignored". Only manually linked pull requests can be manually unlinked; to unlink a keyword link, edit the description.

The keyword is just **text in a message**, so it is Git that stores it and the platform that reads it. Here a template file and a commit message with a keyword are made and looked at:

```

$ cd bakery-menu
$ mkdir -p .github/ISSUE_TEMPLATE
$ printf 'name: Bug report\ndescription: Something on the menu is wrong\nbody:\n - type:
textarea\n  id: what\n  attributes:\n    label: What is wrong?\n  validations:
\n    required: true\n' > .github/ISSUE_TEMPLATE/bug.yml
$ cat .github/ISSUE_TEMPLATE/bug.yml
name: Bug report
description: Something on the menu is wrong
body:
  - type: textarea
    id: what
    attributes:
      label: What is wrong?
    validations:
      required: true
$ git add .github
$ git commit -q -m "Add a bug report form"
$ git ls-files .github
.github/ISSUE_TEMPLATE/bug.yml
$ printf -- '- Rolls (six): 3.20\n' >> menu.md
$ git commit -q -am "Correct the price of the rolls" -m "Fixes #12"
$ git log -1 --format=%B
Correct the price of the rolls

Fixes #12

$ git log --oneline --grep='#12'
8bd8b2b (HEAD -> main) Correct the price of the rolls

```

Recorded in Bash; *ch44-issues/expected-forms-keywords.bash.txt*.

Git stores `Fixes #12` in the commit message like any other words, and `git log --grep` finds it. What the platform does with it is described above; Git does nothing.

CAUTION. A closing keyword acts when the change reaches the default branch. A commit message is part of history and is hard to change once it is shared (Chapter 27), while a pull request description can be edited at any time, and GitHub's documentation says that to unlink a keyword link you edit the description. So put keywords in the pull request description more often than in commit messages.

44.4 Templates and forms

Projects that receive issues from strangers get better issues when they ask for the right information. GitHub offers two ways.

New term: issue form. A template for issues written as a YAML file that defines input fields (text areas, dropdown menus, checkboxes) so that contributors fill in a form instead of free text.

Checked against GitHub's documentation (R256). Issue templates are stored on the repository's **default branch**, in the hidden `.github/ISSUE_TEMPLATE` directory; "if you create a template in another branch, it will not be available". Markdown templates need a `.md` extension; **issue forms** need `.yaml`. A form's top-level keys `name`, `description` and `body` are all required, and `body` is an array of elements with a type such as `textarea`, `dropdown`, `checkboxes`, `input` or `markdown`. When a contributor submits a form, "the form inputs are converted to a standard markdown issue comment". A file `.github/ISSUE_TEMPLATE/config.yml` can set `blank_issues_enabled: false` to remove the blank-issue option, and `contact_links` to send some reports elsewhere. Pull request templates can live in the root, `docs` or `.github`.

The smallest form, from the recording above:

```
name: Bug report
description: Something on the menu is wrong
body:
  - type: textarea
    id: what
    attributes:
      label: What is wrong?
    validations:
      required: true
```

Because the template is a file in `.github/ISSUE_TEMPLATE`, it is **part of your Git repository**: it is committed, reviewed and versioned like code, and it must be on the default branch to take effect. (YAML, the format, is taught in Chapter 53.)

44.5 A workable habit

1. Search for an existing issue before opening one.
2. One problem per issue.
3. Open the issue before the work, so the pull request can close it.
4. When you start, assign yourself; when it is done, close it with the pull request, not by hand.
5. If an issue cannot be acted on, say what is missing and label it, instead of leaving it open forever.

Checkpoint

What You Learned

- An issue is a numbered item on the platform, not in the Git repository.
- Labels say what kind, assignees say who, milestones say when, references say what is related.
- Keywords such as `Fixes #12` in a pull request description or commit message close the issue when the change reaches the default branch.

- Issue templates and forms live in `.github/ISSUE_TEMPLATE` on the default branch, and are files in the repository.

New Vocabulary

Issue, issue form (introduced above).

Commands Learned

`git commit -m ... -m ...` (subject and body), `git log --grep`.

Common Mistakes

1. **Vague titles.**
2. **Closing keywords in commits on the wrong branch**, so nothing closes.
3. **Too many labels.**
4. **Putting secrets in an issue.**
5. **Adding an issue template on a branch other than the default.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Is an issue part of the Git repository? How do you know?
2. Which keyword syntax closes an issue in another repository?
3. Why might Fixes #12 in a pull request into a feature branch not close the issue?
4. Where do issue forms live, and on which branch?
5. Name the four things an issue should say.

Before Moving On

You are ready for Chapter [45](#) if you can:

- [] write an issue that someone else can act on
- [] use a closing keyword correctly
- [] say where an issue form is stored

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|--------------------|
| A form file is committed like any file; a keyword is text in the commit message found by <code>--grep</code> | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R257
(Git side) |

| Claim | Evidence class | Ledger |
|---|--|--------|
| Labels, milestones, assignees, references | Checked against github/docs (commit 2eaab0b) | R254 |
| Closing keywords and their rules | Checked against github/docs | R255 |
| Templates, forms, config.yml | Checked against github/docs; the form file was not run on a live account | R256 |

Where this leads

Chapter 45 shows how a pull request uses these references, and Chapter 46 how it is reviewed. Chapter 47 plans work across many issues.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer.

Level 1 — Guided

1.1 A form file. In bakery-menu, create `.github/ISSUE_TEMPLATE/bug.yml` with the form from section 44.4 and commit it. Run `git ls-files .github`.

1.2 Four things. Write the four things a good issue says.

Level 2 — Partially guided

2.1 A keyword. Make a small change to `menu.md` and commit it with the body `Fixes #12`. Find it again with `git log --grep='#12'`.

2.2 Another repository. Write the closing keyword for issue 100 in the repository `octo-org/octo-repo`.

Level 3 — Independent

3.1 A better title. Rewrite the title "Menu broken" for a case where the roll price differs between two places.

3.2 Three labels. Choose three labels a small project could use, and say what each one means. Explain why three is enough.

Level 4 — Professional scenario

4.1 A template set. Design a bug-report form and a question form for a public project. List the fields for each and which are required.

Level 5 — Troubleshooting

5.1 A pull request says "Fixes #40", is merged, and issue 40 stays open. Give two possible reasons from section 44.3.

5.2 A maintainer added an issue form on a branch called `templates`, but contributors still see no form. Explain.

Chapter 45 — Pull Requests [Core]

In this chapter

- what a pull request is, and what it is not
- base and compare, and why a pull request shows a three-dot diff
- draft pull requests and the parts of a pull request page
- the three ways to merge one, shown as Git commands
- fetching a pull request to your computer
- keeping a pull request up to date, closing it, and reverting it

How to read this chapter. The Git behind pull requests was run and recorded on local repositories that stand in for the platform. Statements about GitHub are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used, and labels can change. A pull request is a platform feature: **Git has no pull request**, only branches, commits and merges.

Before you start. Chapter [20](#), Chapter [21](#), Chapter [23](#) and Chapter [44](#). The recordings ran in Bash and zsh on Git 2.43.0 and were re-run in CI on newer Git versions.

45.1 What a pull request is

New term: pull request. A proposal to merge the changes on one branch into another branch, with a place to discuss and review them first. The name comes from asking the owner of a repository to pull your changes.

GitHub's documentation puts it this way: "A pull request proposes merging code changes from one branch into another", and "pull requests turn a set of code changes into a conversation. Instead of merging work directly, you propose it so that collaborators can weigh in." It lists the benefits: catch bugs and problems early, discuss changes with feedback tied to specific lines, and keep a reviewable history of what changed and why.

The important sentence for a Git learner: a pull request is **not a Git object**. Your repository holds branches and commits. The platform holds the conversation, the reviews and the checks, and links them to two branches. Chapter [36](#) said the same of issues.

The workflow you already know, from Chapter [23](#) and Chapter [34](#), becomes:

1. create a branch and commit on it;
2. push the branch;
3. open a pull request from it into the branch you want to change (usually `main`);
4. discuss and revise: more commits on the same branch update the same pull request;
5. merge, and delete the branch.

Checked against GitHub's documentation (R258). "Creating a pull request" says a pull request is opened by choosing a **base** branch, "the branch where you want to merge your changes", and a **compare** branch, "the topic branch where you made your changes", and that "pull requests can only be opened between two different branches". If you do not have write access, you fork the repository first. "About pull requests" lists what a pull request page gathers: **Conversation** (description, comments, reviews, timeline), **Commits, Checks** (results of automated tests and builds), **Files changed** (the diff reviewers comment on), and a **merge box** that "summarizes whether you can merge your changes or not". The reference page adds a **Findings** tab with automated code-review results.

45.2 Base, compare and the three-dot diff

The **base** is where the change is going; the **compare** (or head) branch is where it comes from. The page shows what the compare branch adds to the base. Git has two ways to compare branches, and they answer different questions. Take a repository in which `main` got a new commit after `add-tea` was branched off:

```
$ cd bakery-menu
$ git log --oneline --graph --all
* cd79e50 (HEAD -> main) Raise the price of the white loaf
| * a13923d (add-tea) Add tea
|/
* 8a52ffe Add the menu
$ git diff --stat main...add-tea
menu.md | 1 +
1 file changed, 1 insertion(+)
$ git diff --stat main..add-tea
menu.md | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
$ git merge-base main add-tea
8a52ffecdf9a4a5f307aa1d31e3429b29ad0d6f9
```

Recorded in Bash; *ch45-pr/expected-compare.bash.txt*.

`git diff main...add-tea` (three dots) compares the compare branch with the **merge base**, the common ancestor: it shows only the one line that the tea branch added. `git diff main..add-tea` (two dots) compares the two branch tips directly: it also shows the price change that `main` made and `add-tea` never saw, as if `add-tea` had undone it.

Checked against GitHub's documentation (R259). "Pull requests on GitHub show a three-dot diff": `git diff A...B` compares "the most recent common commit of both branches (merge base) and the most recent version of the topic branch", while the two-dot `git diff A..B` compares "the most recent state of the base branch ... and the most recent version of the topic branch". The three-dot comparison "focuses on 'what a pull request introduces'" and "keeps showing the changes introduced by the topic branch since the branches diverged". The documentation also warns that "compare pages and pull request pages can calculate changed files from different merge bases", so the same two branches can sometimes show different diffs in the two places, and advises merging the base branch into the topic branch often and merging pull requests soon, so that they stay small.

45.3 Draft pull requests

A **draft** pull request says "not ready yet". GitHub's documentation: "Draft pull requests cannot be merged, and code owners are not automatically requested to review them. Drafts are useful when you want to share work-in-progress without formally requesting reviews." You can convert a pull request to a draft at any time, and mark it **Ready for review** from the merge box when it is ready.

Checked against GitHub's documentation (R260). The quotations in this section are from the pull-request reference and "Changing the stage of a pull request". The names of the buttons are the documentation's and can change.

Use a draft when you want early feedback on a direction, or when checks are still failing and you do not want to ask anybody to review yet.

45.4 Three ways to merge

When a pull request is approved and its checks pass, the platform merges it. GitHub's documentation lists three strategies, and what each does to history:

| Strategy | The documentation's result | Git commands that do the same |
|-------------------------|---|--|
| Merge commit | "Preserves every commit from the pull request branch and adds an explicit merge point." | <code>git merge --no-ff topic</code> |
| Squash and merge | "Combines all commits in the pull request into a single commit on the base branch." | <code>git merge --squash topic</code> , then <code>git commit</code> |
| Rebase and merge | "Adds each commit onto the base branch without a merge commit, for a linear history." | <code>git rebase <base></code> on the topic, then <code>git merge --ff-only topic</code> on the base |

The commands are the ones from Chapter 21 and Chapter 27. Starting from the same base, each strategy is applied to the same two-commit topic (tea, then coffee):

```

$ cd bakery-menu
$ git switch -q strategy-merge
$ git merge --no-ff -m "Merge topic" topic
Merge made by the 'ort' strategy.
 menu.md | 2 ++
 1 file changed, 2 insertions(+)
$ git log --oneline --graph
* 66ffa82 (HEAD -> strategy-merge) Merge topic
|\
| * 85667ec (topic) Add coffee
| * d0ffe5a Add tea
|/
* 7877d40 (strategy-squash, main) Add the menu
$ git switch -q strategy-squash
$ git merge --squash topic
Updating 7877d40..85667ec
Fast-forward
Squash commit -- not updating HEAD
 menu.md | 2 ++
 1 file changed, 2 insertions(+)
$ git commit -q -m "Add tea and coffee"
$ git log --oneline --graph
* 9655a53 (HEAD -> strategy-squash) Add tea and coffee
* 7877d40 (main) Add the menu
$ git switch -q topic
$ git rebase strategy-rebase
Successfully rebased and updated refs/heads/topic.
$ git switch -q strategy-rebase
$ git merge --ff-only topic
Updating ed8302b..14100dc
Fast-forward
 menu.md | 2 ++
 1 file changed, 2 insertions(+)
$ git log --oneline --graph
* 14100dc (HEAD -> strategy-rebase, topic) Add coffee
* 1b66442 Add tea
* ed8302b Raise the price of the white loaf
* 7877d40 (main) Add the menu

```

Recorded in Bash; *ch45-pr/expected-strategies.bash.txt*.

Read the three graphs. The merge commit keeps both commits and adds a merge point where the two lines join. The squash leaves **one** new commit, Add tea and coffee, and the two original commits are not in that branch's history. The rebase puts the two commits, with **new hashes**, after the other change on the base, in a straight line (the hashes differ from before, as Chapter [27](#) explained).

Checked against GitHub's documentation (R261). "Pull request merges" gives the table quoted above and says when to choose each: a merge commit "when your team values complete history or when the individual commits ... are meaningful on their own"; squash "when a pull request represents one logical change, especially if the branch includes many small fixup commits"; rebase "when your team wants a linear history and the pull request commits are already organized clearly". It warns that squash merging "works best for short-lived branches": if you keep working on the same head branch after a squash merge, "later pull requests can include commits that were already squashed into the base branch", which can make conflicts more likely. If the platform cannot safely rebase automatically, you can rebase locally, resolve conflicts and push. A pull request can also be marked merged indirectly, when its commits become reachable from the base by another route. What the platform runs internally is not shown: the table's commands reproduce the results described, not GitHub's implementation.

Which one? There is no best strategy. Use the one your team has chosen; if you choose, prefer a merge commit when the individual commits matter, squash when the branch is one change with noisy commits, and rebase for a straight history. Repository administrators can restrict which strategies are allowed (Chapter 50).

45.5 Keeping it up to date, closing, reverting

- **In sync.** Before merging, update the pull request branch with changes from the base, to catch conflicts and failing tests early. The documentation says you can do it from the page "by merging the base branch into your head branch or by rebasing your changes onto the latest base branch", but only when there are no conflicts; with conflicts you resolve them yourself (Chapter 22).
- **Wrong base.** "If you opened a pull request with the wrong base branch, instead of closing it and opening a new one, you can change the base branch."
- **Closing.** A pull request can be closed without merging. Its branch and its conversation remain.
- **Reverting.** "Reverting a merged pull request creates a new pull request that reverts the original merge commit." If reverting causes conflicts, or the pull request was not merged on the platform, revert individual commits with `git revert` (Chapter 25); reverting a merge commit needs `-m` (Chapter 25 section on reverting a merge).

Checked against GitHub's documentation (R262 part). Quotations in this section are from "Keeping your pull request in sync with the base branch", "Changing the base branch of a pull request" and "Reverting a pull request".

45.6 A pull request on your computer

Reviewers often want to run the change. The platform keeps a reference for every pull request on its server, and GitHub's documentation shows how to fetch it: `git fetch origin pull/ID/head:BRANCH_NAME`, then switch to it. The `refs/pull/` namespace is **read-only**: pushing there produces deny updating a hidden ref.

The recording uses a local bare repository as the stand-in server. It has a branch `add-tea` and a reference `refs/pull/1/head` pointing at the same commit, and it is configured (with `receive.hideRefs`) to refuse pushes to `refs/pull`, as the documentation says the platform does:

```
$ git clone -q hub.git reviewer
$ cd reviewer
$ git ls-remote origin
c4972f0d9a02edb567a40199915178833a1b91b9 HEAD
3b6b90d431743ed80865ba77ac3d3296eb498332 refs/heads/add-tea
c4972f0d9a02edb567a40199915178833a1b91b9 refs/heads/main
3b6b90d431743ed80865ba77ac3d3296eb498332 refs/pull/1/head
$ git fetch -q origin pull/1/head:pr-1
$ git switch -q pr-1
$ git log --oneline
3b6b90d (HEAD -> pr-1, origin/add-tea) Add tea
c4972f0 (origin/main, origin/HEAD, main) Add the menu
$ git push origin HEAD:refs/pull/1/head
To /home/learner/hub.git
! [remote rejected] HEAD -> refs/pull/1/head (deny updating a hidden ref)
error: failed to push some refs to '/home/learner/hub.git'
```

Recorded in Bash; *ch45-pr/expected-pr-ref.bash.txt*.

`git ls-remote` lists the extra reference. The fetch created a local branch `pr-1` with the pull request's commit, the log shows it, and the push to the hidden reference is refused with the message quoted in the documentation. This is only a **stand-in**: the real platform creates and updates `refs/pull/...` itself, and this book did not fetch from GitHub.

Checked against GitHub's documentation (R262). "Checking out pull requests locally" gives the fetch command above, says the `refs/pull/` namespace is read-only and shows the rejection `! [remote rejected] HEAD -> refs/pull/1/head (deny updating a hidden ref)`. The reference page says that when you open a pull request GitHub "creates temporary Git references that point to the pull request's head branch and, when possible, to a simulated merge result".

45.7 A good pull request

1. **Small.** One change that a reviewer can understand in ten minutes.
2. **A title and description that explain why.** Link the issue with a closing keyword (Chapter 44).
3. **Passing checks** before you ask anyone (Chapter 54).
4. **Reviewed by someone else**, and merged by the author or the maintainer according to the team's rules (Chapter 46).

5. **Branch deleted after merging**, so that old branches do not pile up.
-

Checkpoint

What You Learned

- A pull request proposes merging a compare branch into a base branch; it lives on the platform, not in Git.
- The page shows a three-dot diff: what the branch introduced since it diverged.
- A draft cannot be merged and does not request code owners.
- Merge commit, squash and rebase leave different histories: joined, one commit, or a straight line with new hashes.
- `git fetch origin pull/ID/head:NAME` brings a pull request to your computer; `refs/pull/` cannot be pushed to.

New Vocabulary

Pull request, **base branch**, **compare branch** (introduced above).

Commands Learned

`git diff A...B`, `git diff A..B`, `git merge-base`, `git merge --squash`, `git merge --ff-only`, `git ls-remote`, `git fetch origin pull/ID/head:NAME`.

Common Mistakes

1. **Treating a pull request as part of the repository** and expecting a clone to have it.
2. **Very large pull requests** that nobody can review.
3. **Choosing squash for a long-running branch**, then meeting the same conflicts again.
4. **Reading a two-dot diff** as "what this branch changes".
5. **Trying to push to `refs/pull/...`**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Is a pull request stored in your Git repository?
2. What does `git diff main...topic` show that `git diff main..topic` may not?
3. What can a draft pull request not do?
4. How many commits does a squash merge add to the base branch?
5. Which command brings pull request 7 to your computer?

Before Moving On

You are ready for Chapter 46 if you can:

- [] open a pull request from a pushed branch (or explain each step)
- [] say what each merge strategy does to history
- [] explain what the three-dot diff shows

How this chapter was checked

| Claim | Evidence class | Ledge
r |
|--|---|--------------------|
| Three-dot and two-dot diffs; merge base | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R259
(Git side) |
| Merge commit, squash and rebase leave the histories shown | Locally tested (same) with Git commands that reproduce the described results; not GitHub's implementation | R261
(Git side) |
| Fetching a pull request ref and the hidden-ref rejection | Locally tested against a stand-in bare repository; the platform itself was not contacted | R262
(Git side) |
| Pull request page, base/compare, drafts, merge strategies, sync, revert, refs/pull | Checked against github/docs (commit 2eaab0b); not run on a live account | R258-
R262 |

Where this leads

Chapter 46 covers how changes are read and discussed. Chapter 49 covers forks and access, and Chapter 50 how a repository can require reviews and checks before a merge.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 4 need only your computer; 5 needs an account and a repository you may change.

Level 1 — Guided

1.1 Base and compare. In your own words, say what the base branch and the compare branch are.

1.2 Two diffs. In bakery-menu, create a branch add-tea with one new line. Then make one commit on main that changes a different line. Run `git diff --stat main...add-tea` and `git diff --stat main..add-tea`. Which shows only what the branch introduced?

Level 2 — Partially guided

2.1 Three strategies. From one starting commit, make three branches. Merge the same two-commit topic into them with `--no-ff`, with `--squash`, and by rebasing and fast-forwarding. Draw the three `git log --oneline --graph` results.

2.2 New hashes. After the rebase, compare the hashes of the two topic commits with their hashes before. Why did they change?

Level 3 — Independent

3.1 A pull request reference. Make a bare repository as a stand-in server, create a reference `refs/pull/1/head` with `git update-ref`, fetch it into a local branch, and switch to it.

3.2 A hidden reference. Set `receive.hideRefs` to `refs/pull` in the bare repository and try to push to `refs/pull/1/head`. What message do you get?

Level 4 — Professional scenario

4.1 Choose a strategy. A project has ten contributors who each open pull requests with many "fix typo" and "try again" commits. The team wants a clean history. Which strategy would you recommend, and what is the risk if a contributor keeps working on the same branch afterwards?

Level 5 — Troubleshooting

5.1 A reviewer says the pull request page shows changes that you did not make, but `git diff main..topic` on your computer shows them too. Explain, and give the command that matches the page.

5.2 A merged pull request broke the site. You want to undo it. Name two ways from section 45.5 and when to prefer each.

Chapter 46 — Code Review [Core]

In this chapter

- why changes are reviewed, and what a review looks for
- how to comment, suggest a change and submit a decision
- what happens to a suggestion when the author applies it
- how to behave as a reviewer, and how to receive a review

How to read this chapter. The Git side (a co-author trailer, and how Git counts co-authors) was run and recorded. Statements about GitHub are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used. The advice on etiquette is **this book's**, and is labelled as such.

Before you start. Chapter [45](#). The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on newer Git versions.

46.1 Why review

New term: code review. Reading someone else's proposed change, before it is merged, to find mistakes, share knowledge and decide whether it is ready.

GitHub's documentation says reviewing pull requests "is one of the main ways people collaborate", and that when you review a peer's work "you help catch issues early, share knowledge, and decide whether a change is ready to merge".

A review is **not** an exam of the author. It is a second pair of eyes on a change that the author has read so many times that they no longer see it. Reviews also spread knowledge: after a review, at least two people know how that part works.

46.2 What a reviewer looks for

A simple order, from most important to least. This is the book's checklist, not a rule of the platform.

1. **Purpose.** Does the change do what the description and the linked issue say? (The pull request page's sidebar shows linked issues and milestones, and GitHub's documentation suggests reading them to understand the purpose.)
2. **Correctness.** Would it work for the cases that matter, including the unusual ones?
3. **Safety.** Does it add a secret, a risky permission or a suspicious dependency? (Chapter [33](#); GitHub's documentation mentions dependency review, which shows how a pull request changes dependencies, and code scanning alerts on the proposed changes.)
4. **Tests and checks.** Do the automated checks pass, and is the new behaviour covered? (Chapter [54](#).)

5. **Clarity.** Could the next person understand it?
6. **Style.** Last, and mostly left to automatic tools.

Size matters. A review of ten lines is careful; a review of two thousand lines is a skim. Ask for a smaller change when it is too big (Chapter 45).

46.3 How to comment

Checked against GitHub's documentation (R263). "Giving reviews" says anyone with read access can review and comment, and lists three things a reviewer can do: "leave general feedback on the overall pull request", "comment on specific lines to ask questions or explain concerns", and "suggest exact changes that the author can apply with a single click". "Reviewing proposed changes" recommends reviewing "one file at a time": examine each file, leave comments on specific changes, mark the file as **Viewed** to collapse it, and use the **progress bar** in the header to see how many files you have viewed. Comments on the Files changed tab can be single lines, several lines, or whole files, and they support the usual formatting, @mentions, emoji and references. A reply by email is added to the conversation and is not part of a review.

Every comment should be **specific and actionable**:

- Say where (the line) and what (the problem or question).
 - Say why, if it is not obvious.
 - Separate must fix from could improve. A common habit is to write "nit:" before a small preference the author may ignore.
-

46.4 Submitting a decision

When you finish, you submit the review with one of three decisions.

| Decision | Meaning, in the documentation's words |
|------------------------|---|
| Comment | "Leaves general feedback without explicitly approving or requesting changes." |
| Approve | "Signals that the changes are ready to merge." |
| Request changes | "Flags feedback that the author should address before the pull request merges." |

Checked against GitHub's documentation (R264). The three decisions and their wording are from "Giving reviews". Whether an approval is required before merging depends on the repository's rules (Chapter 50).

A decision is a **statement about the change at the moment you review it**. If the author pushes new commits, look again before you approve.

46.5 Suggestions, and what they do to history

A reviewer can propose an exact replacement for a line. The author can apply it with one click, and this makes a real **commit** on the pull request's branch.

Checked against GitHub's documentation (R265). "Incorporating feedback in your pull request" says a person with write access can apply a suggested change directly, or add several suggestions to a batch; "applying one suggested change or a batch of suggested changes creates a single commit on the compare branch of the pull request"; "each person who suggested a change included in the commit will be a co-author of the commit", and "the person who applies the suggested changes will be a co-author and the committer". If a suggestion is out of scope, you can open an issue that tracks it and links back. The page on commits with several authors says the platform reads Co-authored-by: NAME <EMAIL> lines added after a blank line at the end of the commit message, and that for the commit to count as a contribution the address should be one associated with the person's account.

That trailer is only text in the commit message, so **Git** can read it. Here a typo is fixed after a reviewer's suggestion, with the reviewer credited:

```
$ cd bakery-menu
$ grep -n slise menu.md
4:- Coconut cake (slise): 4.00
$ sed -i 's/slise/slice/' menu.md
$ git diff
diff --git a/menu.md b/menu.md
index 462ca3a..bc206a1 100644
--- a/menu.md
+++ b/menu.md
@@ -1,4 +1,4 @@
 # Sunrise Bakery menu

- White loaf: 2.80
-- Coconut cake (slise): 4.00
+- Coconut cake (slice): 4.00
$ git commit -q -am "Fix the typo in the cake line" -m "Co-authored-by: Sam Reviewer
<sam@example.org>"
$ git log -1 --format=%B
Fix the typo in the cake line

Co-authored-by: Sam Reviewer <sam@example.org>

$ git log -1 --format='%(trailers:key=Co-authored-by,valueonly)'
Sam Reviewer <sam@example.org>

$ git shortlog -sne --group=author --group=trailer:co-authored-by HEAD
 2 Ada Learner <ada@example.org>
 1 Sam Reviewer <sam@example.org>
```

Recorded in Bash; *ch46-review/expected-suggestion.bash.txt*.

`git log --format=%B` shows the message with the trailer at the end. `%(trailers:key=Co-authored-by,valueonly)` extracts it. `git shortlog` with `--group=author --group=trailer:co-authored-by` counts Ada twice (she authored the first commit and this one) and the reviewer once. Git does not decide who counts as an author: it counts the words you wrote.

46.6 Being a good reviewer

This section is the book's advice.

1. **Review the change, not the person.** "This loop reads past the end of the list" is about the code; "you always forget the end" is about the author.
 2. **Ask before assuming.** "Was there a reason to use two loops here?" leaves room for a reason you cannot see.
 3. **Explain the reason.** Give one line of why, and a link if there is a guideline.
 4. **Praise what is good.** It is information too: it tells the author what to keep doing.
 5. **Be timely.** A pull request waiting for days blocks the author and grows stale.
 6. **Know when to stop.** Approve when the change is good enough and safe, not when it is what you would have written.
-

46.7 Receiving a review

1. **Thank first, then read twice.** Feedback on your work feels personal. Read all of it before you answer.
 2. **Assume good intent.** Most comments are meant to help.
 3. **Answer every comment:** do it, or say why not.
 4. **Push new commits to the same branch.** The pull request updates itself (Chapter 45).
 5. **Do not argue by silence.** If you disagree, say so and give your reason; if you still disagree, ask a maintainer to decide.
 6. **Keep unrelated requests for another issue,** as the documentation suggests for out-of-scope feedback.
-

Checkpoint

What You Learned

- A review finds mistakes early, spreads knowledge and decides readiness.
- Reviewers comment on lines and files, and finish with Comment, Approve or Request changes.
- A suggestion applied from the page becomes one commit on the pull request branch, with the suggesters credited as co-authors.
- Co-authored-by: is plain text in the message, which Git can read and count.
- Good review is specific, kind and timely; good reception is answering every point.

New Vocabulary

Code review (introduced above).

Commands Learned

```
git log --format='%{trailers:key=Co-authored-by,valueonly}', git shortlog -sne --group=author
--group=trailer:co-authored-by.
```

Common Mistakes

1. **Reviewing a huge change in one pass.**
2. **Vague comments** such as "this is wrong".
3. **Approving without reading.**
4. **Taking feedback as an attack.**
5. **Pushing new commits after an approval and assuming it still stands.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name the three review decisions and what each means.
2. What does applying a suggestion from the page create, and who is credited?
3. Where is a co-author recorded in a commit?
4. Why should you look again after new commits are pushed?
5. Write one comment that is specific and kind.

Before Moving On

You are ready for Chapter [47](#) or Chapter [49](#) if you can:

- [] review a small change with a checklist
- [] write comments that are specific and actionable
- [] receive a review and answer every point

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|-----------------|
| A Co-authored-by trailer is text in the message; % (trailers) and shortlog --group=trailer read it | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R265 (Git side) |
| Review comments, decisions, suggestions, co-authors | Checked against github/docs (commit 2eaab0b); not run on a live account | R263-R265 |
| Reviewer checklist and etiquette | The book's own advice , not sourced rules | R263 |

Where this leads

Chapter 49 covers who may review and merge, forks, and code owners. Chapter 50 covers making reviews mandatory.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer; 4 and 5 are practice with a partner or with your own earlier work.

Level 1 — Guided

1.1 Three decisions. Write the three decisions of a review and one sentence for each.

1.2 A co-author. In `bakery-menu`, fix a typo in a commit whose message ends with `Co-authored-by: Sam Reviewer <sam@example.org>`. Show the trailer with `git log -1 --format='% (trailers:key=Co-authored-by,valueonly)'`.

Level 2 — Partially guided

2.1 Count the credit. Run `git shortlog -sne --group=author --group=trailer:co-authored-by HEAD`. Who is counted, and how many times?

2.2 Rewrite a comment. Turn "this is wrong" into a specific, kind, actionable comment about a line of `menu.md` of your choice.

Level 3 — Independent

3.1 A checklist. Write your own six-item review checklist, in order of importance, for a project you know.

3.2 Must or nit. Take five imaginary comments on a change (for example a missing test, a spelling mistake in a comment, a bug, a variable name, a missing null check). Mark each must fix or nit, and explain.

Level 4 — Professional scenario

4.1 A review round. Ask a friend to review a small change of yours and answer every comment. Afterwards write what you learned about how you write pull request descriptions.

Level 5 — Troubleshooting

5.1 A reviewer approved a pull request on Monday. On Tuesday the author pushed three more commits and merged. What went wrong with the process, and what habit prevents it?

5.2 A learner's suggestion was applied, but the reviewer is not shown as a contributor on the platform. Give two possible reasons from this chapter and Chapter 37.

Chapter 47 — GitHub Projects [Deep]

In this chapter

- what a project is and how it differs from an issue list
- the three layouts: table, board and roadmap
- fields, including iterations
- filters, and how they combine
- built-in automation, and who can see a project

How to read this chapter. This is a **Deep** chapter about a service, so it has **no recorded commands** and can be read later. Everything is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used. Names of buttons and menu items are the documentation's and can change. A project is a platform feature and is **not in your Git repository**.

Before you start. Chapter [44](#) and Chapter [45](#).

47.1 What a project is

New term: project. On GitHub, an adaptable table, board or roadmap built from your issues and pull requests, used to plan and track work across one or many repositories.

GitHub's documentation defines it as "an adaptable table, board, and roadmap that integrates with your issues and pull requests on GitHub to help you plan and track your work effectively at the user or organization level". Items come from issues and pull requests, and information "is synced automatically ... This integration works both ways": if you change an assignee in the project, the issue shows the change too.

Compare with the issue list of one repository (Chapter [44](#)): an issue list is one repository's backlog with labels and milestones; a project can gather items from several repositories, add **custom fields**, and show the same items in several **views**.

Checked against GitHub's documentation (R266). "About Projects" for the definition and the two-way sync above. A project can also hold **draft issues** (items that are not yet issues), which can be converted to issues later.

47.2 Three layouts

| Layout | What it looks like | Good for |
|----------------|---|---|
| Table | A high-density spreadsheet-style list; fields are columns | Sorting, grouping and editing many items at once |
| Board | Columns of cards (a kanban board), for example by status | Seeing the flow of work: Todo, In progress, Done |
| Roadmap | A timeline of items on dates or iterations | Planning releases and seeing what is scheduled when |

The documentation says you can "view your project as a high-density table layout, as a kanban board, or a timeline-style roadmap", and that you can save views and share them with your team. Each view can filter, sort, slice and group.

47.3 Fields

Beyond the built-in data of an issue (assignee, milestone, labels), a project can hold **custom fields**. The documentation lists examples:

- a **date** field for target ship dates;
- a **number** field for the complexity of a task;
- a **single select** field for priority (Low, Medium, High);
- a **text** field for a quick note;
- an **iteration** field to plan work "week-by-week, including support for breaks".

A project can have "up to 50 fields", including the built-in ones.

New term: iteration. A block of time, such as a week or two weeks, in which a team plans to finish a set of items. In GitHub Projects an iteration is a kind of field, so items can be grouped and filtered by iteration.

Iterations "can be set to any length of time, can include breaks, and can be individually edited". You can filter by name, or by @current, @previous and @next. When you first create an iteration field, three iterations are created automatically.

47.4 Filters

You narrow a view with **filters**. The documentation gives these forms:

| You type | You get |
|------------------|----------------------------------|
| assignee:octocat | items assigned to that person |
| label:bug | items with the bug label |
| status:done | items whose Status field is Done |

| You type | You get |
|---|--|
| label:bug status:"In progress" | items matching both (filters combine with AND) |
| label:bug,support | items with either label (commas mean OR within one field) |
| assignee:octocat assignee:stevecat | items assigned to both people |
| assignee:@me status:todo last-updated:5days | your Todo items not updated in five days |
| no:label no:assignee repo:octocat/game | a triage view: items with no label and no assignee |

When you filter a view and then add an item, "the filtered metadata will be applied to new item". If you filter by status:"In progress" and add an item, the new item has that status.

Checked against GitHub's documentation (R267). The qualifiers, the AND and OR rules, the no: filter and the behaviour of new items are from "Filtering projects"; the iteration filters are from "About iteration fields".

47.5 Automation

The documentation says projects include **built-in workflows** that update the **Status** of items on events: for example set Todo when an item is added, close issues when their status changes, or set Done when an issue is closed. Two are on by default: items are set to **Done** when their issue or pull request is closed, and when their pull request is merged. You can also automatically **archive** items that meet criteria, automatically **add** items from a repository that match a filter, and use the API and GitHub Actions for more control (Chapter 54).

47.6 Who can see a project

A project's visibility is separate from its items. GitHub's documentation: public projects can be viewed by "everyone on the internet"; private projects only by users "granted at least read access". **Only the project's visibility is affected:** to see an item, someone must still have access to the repository it belongs to, and items from a private repository appear as hidden to people without access. Project admins and organization owners control visibility.

CAUTION. Do not put secrets or private details in a project's title, README, status updates or custom fields of a public project: everything in it is visible to the world, even if your repositories are private.

47.7 Habits that keep a project useful

The documentation's best practices, in short: use @mentions, assignees and links to communicate; break large issues into smaller ones (sub-issues), which "leads to smaller pull requests, which are easier to review"; use the project description and README, and post status updates ("On track", "At risk"); and create views for different questions (filter by status, group by priority, sort by date, slice by assignee). A project nobody updates is worse than none, so give it an owner.

Checkpoint

What You Learned

- A project is a table, board or roadmap over issues and pull requests, synced both ways.
- Custom fields (date, number, single select, text, iteration) add metadata.
- Filters combine with AND; commas mean OR within one field.
- Built-in workflows update Status; two are on by default.
- Project visibility is separate from repository access.

New Vocabulary

Project, **iteration** (introduced above).

Commands Learned

None: this chapter is about the platform and not about Git.

Common Mistakes

1. **Using a project as a second place to type the same information.**
2. **Too many fields**, so nobody fills them in.
3. **Assuming a public project exposes private repository items.**
4. **Leaving a project without an owner.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name the three layouts and one use for each.
2. What does `label:bug, support show`?
3. Which two built-in workflows are on by default?
4. Who can see the items from a private repository in a public project?

Before Moving On

You are ready for Chapter 48 if you can:

- [] choose a layout for a question
- [] write a filter that combines conditions
- [] say what a project shows and does not show

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|---|--------|
| Definition, layouts, fields, best practices, visibility, built-in workflows | Checked against github/docs (commit 2eaab0b); not run on a live account | R266 |
| Filter syntax and iteration filters | Checked against github/docs | R267 |

Where this leads

Chapter 48 covers open-ended conversation, and Chapter 54 shows how workflows can act on projects.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises are on paper; a real project on your account is optional.

Level 1 — Guided

1.1 Layouts. For each of table, board and roadmap, write one question it answers well.

1.2 Fields. Name the five kinds of custom field and one example use of each.

Level 2 — Partially guided

2.1 Write filters. Write a filter for: (a) items with the label bug and the status In progress; (b) items with the label bug or support; (c) items assigned to you and not updated for five days.

2.2 Triage. Write the filter for a triage view of items without a label and without an assignee in the repository octocat/game.

Level 3 — Independent

3.1 Design a project. For the Sunrise Bakery website, design a project: the statuses, two custom fields, and three views (with layout and filter for each).

Level 4 — Professional scenario

4.1 A public project. A team wants a public roadmap for a product whose code is in private repositories. What can the public see, what can they not see, and what should the team avoid writing?

Level 5 — Troubleshooting

5.1 A new item added while a view is filtered by status: "In progress" appears with the status In progress, and the author did not set it. Explain.

5.2 A closed issue still shows Todo in the project. What built-in workflow should you check?

Chapter 48 — GitHub Discussions [Deep]

In this chapter

- what Discussions are, and how they differ from Issues
- categories and their formats
- moderating: answers, locking, closing, converting
- when to use a discussion, an issue or a pull request

How to read this chapter. This is a **Deep** chapter about a service, so it has **no recorded commands** and can be read later. Everything is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used. A discussion is stored by the platform and is **not** in your Git repository.

Before you start. Chapter [44](#).

48.1 What Discussions are

New term: discussion. An open-ended conversation about a project, kept by the platform in a forum-like section, for questions, ideas, announcements and polls, separate from the tracker of issues.

GitHub's documentation says Discussions let a project's community "engage in conversations about the project's direction and future in an open-ended format", so that maintainers, contributors and visitors "can gather in a central location, without third-party tools". You can share announcements, plan and decide with community input, ask and answer questions ("mark them answered as you respond to them"), and "gauge community opinion using polls".

There are **repository** discussions, for topics specific to one repository, and **organization** discussions, for conversations across repositories. Anyone with access can take part, "however, an administrator must enable Discussions for the repository or organization first". So, like other features in Chapter [43](#), it may simply be absent.

48.2 Discussions, issues or pull requests?

The documentation draws the line like this. Use a discussion for big-picture ideas, brainstorming and developing "a project's specific details before committing it to an issue, which can then be scoped". It suggests discussions when you are in the discovery phase, want feedback from a wider community, want to keep bug fixes, feature requests and general conversations separate, or want to measure interest with polls. **Issues** are for "specific details of a project such as bug reports and planned improvements". **Pull requests** are for comments "directly on proposed changes".

| You have | Use |
|---|-------------------------------|
| A question about how to use the project | A discussion (Q&A category) |
| An idea that is still vague | A discussion (Ideas category) |
| A bug you can describe and reproduce | An issue |
| A specific task someone will do | An issue |
| A change to the files | A pull request |

The habit that ties them together: when a conversation reaches a decision that needs action, **open an issue** and link the discussion.

Checked against GitHub's documentation (R268). Quotations in this section are from "About Discussions" and "Best practices for community conversations on GitHub". The last habit is the documentation's: "Opening an issue to take action based on the conversation, where applicable".

48.3 Categories

Discussions are filed in **categories**, which the documentation says maintainers can customise. Each category has a unique name and emoji pairing and a description, and "each repository or organization can have up to 25 categories". You can group categories into sections. The default categories:

| Category | Purpose | Format |
|---------------|--|-----------------------|
| Announcements | Updates and news from maintainers | Announcement |
| General | Anything relevant to the project | Open-ended discussion |
| Ideas | Ideas to change or improve the project | Open-ended discussion |
| Polls | Polls with multiple options | Polls |
| Q&A | Questions for the community to answer | Question and Answer |
| Show and tell | Creations, experiments, tests | Open-ended discussion |

The **format** decides what a discussion can do: only the Question and Answer format has a "mark as answer" step, and polls have voting.

48.4 Moderation

Checked against GitHub's documentation (R269). With triage permission on the repository, a person can help moderate: mark comments as answers, lock discussions that are no longer useful or damaging to the community, and convert issues to discussions "when an idea is still in the early stages of development". Locking is appropriate "when the entire conversation is not constructive or violates your community's code of conduct" or to make an announcement that should not receive comments. You can also close a discussion, pin discussions "to give topics more visibility", and use repository labels to organise them. Before opening a discussion, check for **contributing guidelines** in the repository.

Moderating is the same craft as everywhere else in this book: be specific, be kind, and prefer removing a conversation from the spotlight (lock, close) to deleting it.

Checkpoint

What You Learned

- Discussions are open-ended conversations kept by the platform; an administrator must enable them.
- Use a discussion for questions and undecided ideas, an issue for specific work, a pull request for a change.
- Categories (up to 25) have formats: announcement, open-ended, poll, question and answer.
- Moderators can mark answers, pin, lock, close and convert.

New Vocabulary

Discussion (introduced above).

Commands Learned

None: this chapter is about the platform and not about Git.

Common Mistakes

1. **Reporting a reproducible bug as a discussion.**
2. **Leaving a decided conversation without an issue for the action.**
3. **Assuming the feature is on.**
4. **Ignoring the contributing guidelines.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Who must switch Discussions on?
2. Which category format allows marking an answer?
3. When does an idea belong in an issue instead?
4. Give one reason to lock a discussion.

Before Moving On

You are ready for Chapter 49 if you can:

- [] choose between discussion, issue and pull request
- [] name the default categories
- [] say what a moderator can do

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|--------|
| What Discussions are, when to use them | Checked against github/docs (commit 2eaab0b); not run on a live account | R268 |
| Categories, formats, moderation | Checked against github/docs | R269 |

Where this leads

Chapter 49 covers who can do what in a repository, and Chapter 64 how open-source projects run their community conversations.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises are on paper.

Level 1 — Guided

1.1 Which one? For each item choose discussion, issue or pull request: (a) "How do I install it on Windows?"; (b) "The menu prints 3.00 but the file says 3.20"; (c) a spelling fix in the README; (d) "Should we support two languages?"

1.2 Categories. Name the six default categories.

Level 2 — Partially guided

2.1 A decision. A discussion in Ideas reaches agreement to add a "kids' menu". What should happen next, according to section 48.2?

2.2 Moderation. Name two moderation actions and one situation for each.

Level 3 — Independent

3.1 Design categories. For a small open-source library, choose four categories, and for each say the format and why.

Level 4 — Professional scenario

4.1 A noisy project. A maintainer is flooded with support questions in issues. Propose a way to use Discussions to reduce the load, and how to tell contributors.

Level 5 — Troubleshooting

5.1 A contributor cannot find the Discussions tab in a repository. Give two possible reasons.

5.2 An announcement discussion keeps receiving off-topic comments. What can a moderator do?

Chapter 49 — Collaboration and Access [Core]

In this chapter

- who can do what in a repository: roles and permissions
- forks and clones: what each is, and how a fork stays in step with its upstream
- code owners (CODEOWNERS)
- community health files that a whole account can share

How to read this chapter. The fork workflow was run and recorded with local bare repositories that stand in for the platform. Statements about GitHub are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used. Access control is a platform feature: **Git itself has no users, roles or permissions**, only whoever can reach a repository's address.

Before you start. Chapter 45 and Chapter 46. The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on newer Git versions.

49.1 Roles

Two questions from Chapter 38 come back: who are you? was authentication; what may you do? is **authorisation**, decided by **permissions**, and a **role** is a set of permissions given to a person or team.

For a repository owned by an **organization**, GitHub's documentation lists five roles, "from least access to most access":

| Role | The documentation recommends it for |
|-----------------|--|
| Read | non-code contributors who want to view or discuss the project |
| Triage | contributors who need to manage issues and pull requests without write access |
| Write | contributors who actively push to the project |
| Maintain | project managers who need to manage the repository without access to sensitive or destructive actions |
| Admin | people who need full access, including sensitive and destructive actions like managing security or deleting a repository |

Organization owners have admin access to every repository of the organization. Enterprise plans can define **custom repository roles** (up to 20 in the documentation for the current version). The documentation's principle: "Choose the role that best fits each person or team's function in your project without giving people more access to the project than they need."

A repository owned by a **personal account** is simpler: it has "a single owner who has full control", and the owner can invite **collaborators**. For finer control the documentation suggests transferring the repository to an organization.

Checked against GitHub's documentation (R270). "Repository roles for an organization" gives the five roles and the quoted principle; "About custom repository roles" says custom roles are limited (up to 20 in the current version); "Permission levels for a personal account repository" describes the single owner and the suggestion to move to an organization. Which exact actions each role may take is a long table in the documentation and is not reproduced here.

Least privilege (Chapter 33) applies: start with the smallest role that lets someone do their job, and remove access when they leave.

49.2 Fork, clone and branch

Three words that beginners mix up:

| | What it is | Where it lives |
|---------------|--|----------------------------|
| Clone | A copy of a repository on your computer | Your disk |
| Branch | A line of work inside one repository | The same repository |
| Fork | A copy of a repository on the platform , owned by you | A second hosted repository |

GitHub's documentation: "A fork is a copy of a repository that you own, connected to the original repository, called the upstream repository." A fork lets you "make changes in your own space without affecting the upstream project, and propose those changes back with a pull request". Use a **branch** "when you have write access and collaborate in a shared repository", and a **fork** "when you need independence from the upstream repository, or when you don't have write access".

New term: fork. A copy of a hosted repository that you own, connected to the original (the upstream), so that you can work without write access to the original and propose your changes back with a pull request.

Two collaboration models follow: in the **shared repository model**, collaborators have push access to one repository and use topic branches (common in teams); in the **fork and pull model**, "anyone can fork" a repository they can read (if the owner allows it) and you "do not need permission from the upstream repository to push to a fork you created" (common in open source).

Checked against GitHub's documentation (R271). "About forks" and the pull-request reference. The reference page adds that a fork and its upstream "share the same Git data": "all content uploaded to a fork is accessible from the upstream and all other forks of that upstream". Do not push anything to a fork that you would not publish to the upstream.

49.3 Keeping a fork in step

A fork is a snapshot from the moment you made it. The upstream keeps moving. To catch up on your computer, you add the upstream repository as a second **remote**, conventionally named `upstream`, then fetch it and merge. The steps below are GitHub's documented commands. The recording uses local bare repositories instead of GitHub: `upstream.git` is the original, `fork.git` is your fork, and `my-clone` is your clone of the fork.

```
$ git clone -q fork.git my-clone
$ cd my-clone
$ git remote -v
origin /home/learner/fork.git (fetch)
origin /home/learner/fork.git (push)
$ git remote add upstream ../upstream.git
$ git remote -v
origin /home/learner/fork.git (fetch)
origin /home/learner/fork.git (push)
upstream ../upstream.git (fetch)
upstream ../upstream.git (push)
$ git switch -q -c add-tea
$ printf -- "- Tea: 1.50\n" >> menu.md
$ git commit -q -am "Add tea"
$ git push -q -u origin add-tea
$ git -C ../upstream.git branch
* main
$ git clone -q ../upstream.git ../maintainer
$ git -C ../maintainer -c user.name=Maintainer -c user.email=maintainer@example.org commit
-q --allow-empty -m "Update the contributing guide"
$ git -C ../maintainer push -q origin main
$ git switch -q main
$ git fetch -q upstream
$ git log --oneline --all --graph
* ce2b049 (upstream/main) Update the contributing guide
| * 9c3dcee (origin/add-tea, add-tea) Add tea
|/
* c4972f0 (HEAD -> main, origin/main, origin/HEAD) Add the menu
$ git merge --ff-only upstream/main
Updating c4972f0..ce2b049
Fast-forward
$ git push -q origin main
$ git log --oneline --graph --all
* ce2b049 (HEAD -> main, upstream/main, origin/main, origin/HEAD) Update the contributing
guide
| * 9c3dcee (origin/add-tea, add-tea) Add tea
|/
* c4972f0 Add the menu
```

Recorded in Bash; *ch49-collab/expected-fork.bash.txt*.

Your Git may show one more label. Git 2.55.0 (re-run in CI) also lists `upstream/HEAD` beside `upstream/main` in the graph, because it records which branch is the upstream's default. The commits are the same.

Follow the story. `git remote -v` first shows only origin, your fork. `git remote add upstream` adds the original. A pushed branch `add-tea` shows in the fork. A maintainer then adds a commit to the upstream. In your clone, `git fetch upstream` brings it as `upstream/main`, and the graph shows the two lines: your `add-tea` branch, and the upstream change. `git merge --ff-only upstream/main` moves your main forward, and `git push origin main` updates your fork. Afterwards `main` and `upstream/main` are the same commit.

`--ff-only` is a safety choice: it refuses to create a merge commit, so if your main had drifted from the upstream you would notice at once (Chapter 21).

Checked against GitHub's documentation (R272). "Configuring a remote repository for a fork" gives `git remote add upstream <URL>`; "Syncing a fork" gives `git fetch upstream`, switching to your local default branch, and `git merge upstream/main`, and also describes an **Update branch** button on the platform (with conflicts, the platform prompts you to create a pull request to resolve them) and a command of the GitHub CLI. The recorded commands show the Git side, with local stand-ins for the platform.

Rule of thumb. Never do your work on the fork's main. Keep main as a mirror of the upstream, and do each change on its own branch (Chapter 20), so that syncing is normally a clean fast-forward.

49.4 Code owners

A repository can say who should review which files, in a file named `CODEOWNERS`. Each line is a pattern followed by owners (`@username` or `@organization/team`). It is a plain file, committed like any other, so it is versioned and reviewed like code.

New term: code owner. A person or team named in a `CODEOWNERS` file as responsible for certain files, who is automatically requested to review pull requests that change them.

Checked against GitHub's documentation (R273). "About code owners": code owners "are automatically requested for review when someone opens a pull request that modifies code that they own", but "are not automatically requested to review draft pull requests"; they are notified when a draft is marked ready. The file is called `CODEOWNERS` and lives in `.github/`, the root or `docs/`; if several exist, GitHub uses the first found in that order. Each file applies to **one branch**, and "to trigger review requests, pull requests use the version of `CODEOWNERS` from the base branch". Owners "must have write permissions for the repository", and a team must be visible and have write permission. The pattern follows most gitignore rules, but three do **not** work: escaping a leading `#`, negating with `!`, and character ranges with `[ ]`. Files must be under 3 MB. Administrators can also require approval from a code owner before merging (Chapter 50).

An example, as text (the syntax is the documentation's):

```
# Everything in the repository is reviewed by the maintainers
*           @example-owner

# The menu file is reviewed by the person who knows the prices
menu.md     @ada
```

49.5 Files that a whole account can share

Files such as `CONTRIBUTING.md`, a code of conduct or issue templates can be shared by all of an account's repositories.

Checked against GitHub's documentation (R273). "Creating a default community health file": put them in a repository named `.github` (it "must be public"), and GitHub will use them for any repository of the account that does not have its own file of that type. For files that can live in several places the order is the `.github` folder, the root, then `docs`. If a repository has its own valid issue templates, none of the default `.github/ISSUE_TEMPLATE` contents are used. Such files "won't appear in the file browser or Git history of the individual repositories, and are not included in their clones, packages, or downloads".

That last sentence has a consequence for a Git user: the shared defaults are **not in your clone**. Someone reading your clone offline does not see them.

Checkpoint

What You Learned

- Organization repositories use five roles: Read, Triage, Write, Maintain, Admin; give the least that works.
- A clone is on your computer, a branch is inside a repository, a fork is a hosted copy you own.
- A fork stays in step through a second remote, upstream: fetch it and fast-forward your main.
- CODEOWNERS requests reviews automatically, from the base branch's copy of the file.
- A `.github` repository can hold defaults for a whole account; they are not in clones.

New Vocabulary

Fork, code owner (introduced above).

Commands Learned

`git remote add upstream <address>`, `git fetch upstream`, `git merge --ff-only upstream/main`.

Common Mistakes

1. **Working on the fork's main**, so it can no longer be fast-forwarded.
2. **Giving admin to everyone** for convenience.
3. **Believing that what you push to a fork is separate from the upstream**: the documentation says a fork and its upstream share Git data.
4. **A CODEOWNERS file on the wrong branch**, so nobody is requested.
5. **Expecting shared default files to appear in a clone**.

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Which role fits a contributor who manages issues but must not push code?
2. What is the difference between a clone and a fork?
3. Which commands add the upstream and bring in its changes?
4. Which copy of CODEOWNERS does a pull request use?
5. Where do shared community health files live?

Before Moving On

You are ready for Chapter 50 if you can:

- [] choose a role for a person by function
- [] set up and use an upstream remote
- [] write a CODEOWNERS line

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|---|-----------------|
| Adding upstream, fetching it, fast-forwarding main, pushing to the fork | Locally tested with local bare repositories as stand-ins: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R272 (Git side) |
| Roles, custom roles, personal-account ownership | Checked against github/docs (commit 2eaab0b); not run on a live account | R270 |
| Forks, shared data, collaboration models | Checked against github/docs | R271 |
| Sync commands and the platform's update button | Checked against github/docs | R272 |
| CODEOWNERS rules and default community health files | Checked against github/docs | R273 |

Where this leads

Chapter 50 covers rules that make reviews and checks mandatory. Chapter 64 shows how open-source projects use forks, and Chapter 68 covers organizations in depth.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer.

Level 1 — Guided

1.1 Roles. List the five roles of an organization repository from least to most access.

1.2 A fork stand-in. Create a bare repository `upstream.git` with one commit, and make `fork.git` with `git clone --bare upstream.git fork.git`. Clone the fork to `my-clone`. Run `git remote -v`.

Level 2 — Partially guided

2.1 Add the upstream. In `my-clone`, add a remote `upstream` pointing to `../upstream.git`. Run `git remote -v` again.

2.2 Catch up. Add a commit to the upstream through a second clone. In `my-clone`, run `git fetch upstream`, then `git merge --ff-only upstream/main`, then `git push origin main`. Show `git log --oneline --graph --all`.

Level 3 — Independent

3.1 A drifted main. Make a commit directly on `my-clone`'s `main`, and add a different commit to the upstream. Try `git merge --ff-only upstream/main`. What happens, and what does it teach?

3.2 CODEOWNERS. Write a `CODEOWNERS` file that makes `@example-owner` the owner of everything and `@ada` the owner of `menu.md`. Where may the file live, and which copy does a pull request use?

Level 4 — Professional scenario

4.1 Roles for a team. A project has a maintainer, three regular developers, a designer who only comments and triages issues, and a visitor from another company who only reads. Assign a role to each and say why.

Level 5 — Troubleshooting

5.1 A maintainer says code owners are never requested on pull requests to the release branch, although they work for `main`. Give a likely reason.

5.2 A team wants shared issue templates for all repositories of the organization, but one repository already has its own `.github/ISSUE_TEMPLATE` folder with a template. What will that repository show?

Chapter 50 — Branch Protection and Rulesets [Deep]

In this chapter

- how Git itself can refuse a dangerous push, on a server you control
- what GitHub's branch protection rules do
- what rulesets add, and how rules from several places layer
- what depends on plan, visibility and role

How to read this chapter. This is a **Deep** chapter and can be read later. The Git server settings were run and recorded with a local bare repository. Statements about GitHub are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used. **Which plans and repository kinds get which protections is time-sensitive and is only partly checked;** look at the current page for your account before you rely on it.

Before you start. Chapter [45](#), Chapter [49](#) and Chapter [27](#) (force pushes).

50.1 The problem

Some actions on shared branches are dangerous: a force push that rewrites history others have built on (Chapter [27](#)), deleting the branch everybody depends on, or merging without review. Good habits help, but habits fail. **Protection** turns the habit into a rule that the server enforces.

Git has a small version of this built in, on the receiving side. It is the ancestor of what a platform offers.

50.2 What Git itself can refuse

A repository that receives pushes (a server repository, such as a bare `hub.git`) can be told to refuse two dangerous kinds of push. Git's documentation of the receive settings says:

- `receive.denyNonFastForwards`: "If set to true, `git-receive-pack` will deny a ref update which is not a fast-forward. Use this to prevent such an update via a push, even if that push is forced."
- `receive.denyDeletes`: "If set to true, `git-receive-pack` will deny a ref update that deletes the ref."

The recording sets both on a server repository, then tries a forced push after amending a commit, and a deletion of `main`:

```
$ git clone -q hub.git work
$ cd work
$ git -C ../hub.git config --get receive.denyNonFastForwards
```

```

true
$ git -C ../hub.git config --get receive.denyDeletes
true
$ git commit -q --amend -m "Add the menu (reworded)"
$ git push --force origin main
remote: error: denying non-fast-forward refs/heads/main (you should pull first)
To /home/learner/hub.git
 ! [remote rejected] main -> main (non-fast-forward)
error: failed to push some refs to '/home/learner/hub.git'
$ git push origin --delete main
remote: error: denying ref deletion for refs/heads/main
To /home/learner/hub.git
 ! [remote rejected] main (deletion prohibited)
error: failed to push some refs to '/home/learner/hub.git'
$ git ls-remote --heads origin
c4972f0d9a02edb567a40199915178833a1b91b9 refs/heads/main

```

Recorded in Bash; *ch50-protect/expected-server-rules.bash.txt*.

The forced push is refused by the server (remote: error: denying non-fast-forward), even though --force was given, and the deletion is refused too. Afterwards git ls-remote shows main unchanged. The word remote: marks messages that come from the other side.

Checked against the documentation (R274). The two settings and their wording are from Git's receive.* configuration documentation (Git 2.56.0). The recorded run shows them working on a local server repository. A platform is not a bare repository with two settings: this section shows the idea that a server can enforce rules whatever the client does.

50.3 Branch protection rules on GitHub

New term: branch protection rule. A setting on a hosted repository that names branches (by pattern) and the requirements or restrictions that apply to changing them.

GitHub's documentation: "You can enforce certain workflows or requirements before a collaborator can push changes to a branch in your repository, including merging a pull request into the branch, by creating a branch protection rule." A rule targets a branch name pattern. Its available settings include:

- **Require pull request reviews before merging:** changes reach the branch "only via a pull request that is approved by the required number of reviewers with write permissions"; if an admin chooses **Request changes**, that person must approve before merge.
- **Require status checks before merging:** required checks must be successful, skipped or neutral. A **strict** setting also requires the branch to be up to date with the base branch; a **loose** one does not.
- **Require conversation resolution, signed commits, linear history, a merge queue, and deployments to succeed.**
- **Lock branch, restrict who can push, allow force pushes, allow deletions.**

"By default, each branch protection rule disables force pushes to the matching branches and prevents the matching branches from being deleted." That is the same pair of protections as the recording above, on the platform.

Two limits matter. By default the restrictions "don't apply to people with admin permissions", unless you choose to apply them to administrators too. And "only a single branch protection rule can apply at a time".

When a push is refused the platform says so, for example: `remote: error: GH006: Protected branch update failed for refs/heads/main. followed by the reason, such as Changes have been requested.`

Checked against GitHub's documentation (R275). "About protected branches" is the source of every quotation and setting in this section. Details of each setting, and its availability for private repositories on each plan, were **not** checked: one item in the documentation limits branch restrictions to public repositories of a free organization and to all repositories of paid organizations, which shows that plan matters.

50.4 Rulesets

New term: ruleset. A named list of rules that a hosting platform enforces on chosen branches or tags, and that can be layered with other rulesets.

Rulesets are the newer, more flexible mechanism. GitHub's documentation: "A ruleset is a named list of rules that applies to a repository or to multiple repositories in an organization" (multi-repository rulesets are for customers on the Team and Enterprise plans). You can have up to 75 rulesets per repository. Differences from branch protection rules, in the documentation's words:

- "Multiple rulesets can apply to the same branch at the same time, while only one branch protection rule applies."
- "You can change a ruleset's enforcement status without deleting the ruleset."
- "Anyone with read access to a repository can view its active rulesets."
- Rulesets can control **commit metadata**, such as messages and author email addresses.
- They can target **branches or tags**, or block pushes to a whole fork network, with `fnmatch` patterns such as `releases/**/*`.

Enforcement statuses. Active enforces at once; Disabled enforces nothing; Evaluate (where offered) "will not be enforced, but you will be able to monitor which actions would or would not violate rules", so you can test a ruleset safely.

Layering. "A ruleset does not have a priority." If several rulesets target the same branch, their rules are **aggregated**, and where the same rule differs, "the most restrictive version of the rule applies"; rulesets also layer with branch protection rules. The documentation's example: one source requires signed commits and three reviews, another requires linear history and two reviews; the result requires signed commits, linear history and **three** reviews.

The available rules include: restrict creations, updates and deletions; require linear history; require a merge queue; require signed commits; require a pull request before merging (with review settings); require status checks; block force pushes; require code-scanning and secret-scanning results; require workflows to pass; restrict commit metadata, file paths, path length, extensions and file size.

Checked against GitHub's documentation (R276). "About rulesets" and "Available rules for rulesets". Bypass lists (roles, teams or apps allowed to bypass a ruleset) exist; who may be added depends on the account type. The documentation also describes converting existing branch protection rules to rulesets.

50.5 Choosing

1. **Protect the default branch first.** Require a pull request, at least one review, and passing checks (Chapter 54).
2. **Block force pushes and deletions** on shared branches; this is the cheapest and most valuable rule.
3. **Use rulesets for anything new** if your plan offers them, because you can test in Evaluate mode and stack rules; keep old branch protection rules working until you convert them.
4. **Keep the bypass list short.** Anyone who can bypass a rule can also make the mistake it prevents.
5. **Write the rules down** in CONTRIBUTING.md (Chapter 64) so contributors are not surprised by a refusal.

CAUTION. A rule that blocks everyone, including you, on the day you need an emergency fix is a real cost. Decide in advance who may bypass, and how that is recorded.

Checkpoint

What You Learned

- A server can refuse dangerous pushes whatever the client does; Git has `receive.denyNonFastForwards` and `receive.denyDeletes`.
- GitHub's branch protection rules default to blocking force pushes and deletions, and add reviews, checks and more; only one rule applies at a time.
- Rulesets stack: several can apply, the most restrictive version of each rule wins, and they can be tested in Evaluate mode.
- What you get depends on plan, visibility and role.

New Vocabulary

Branch protection rule, ruleset (introduced above).

Commands Learned

`git config receive.denyNonFastForwards true`, `git config receive.denyDeletes true` (on a server repository).

Common Mistakes

1. **Assuming `--force` overrides the server.**
2. **Giving many people bypass rights.**
3. **Forgetting that admins are exempt by default.**
4. **Requiring checks that nobody maintains.**
5. **Assuming a protection is available on your plan.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What stops a forced push to a protected branch, whatever the client does?
2. Which two protections does a GitHub branch protection rule enable by default?
3. How do rulesets differ from branch protection rules on overlap?
4. What does Evaluate mode do?

Before Moving On

You are ready for Chapter [51](#) if you can:

- [] explain what a server-side refusal is
- [] list three settings of a protection rule
- [] say how layered rulesets combine

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|---|--------|
| <code>receive.denyNonFastForwards</code> and <code>receive.denyDeletes</code> refuse a forced push and a deletion | Both officially verified (Git 2.56.0 documentation) and locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R274 |
| Branch protection rules, defaults, settings | Checked against <code>github/docs</code> (commit <code>2eaab0b</code>); plan availability not fully checked | R275 |
| Rulesets, layering, enforcement, available rules | Checked against <code>github/docs</code> ; plan availability not fully checked | R276 |

Where this leads

Chapter 58 uses protection with environments and deployments, and Chapter 62 with security features.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer.

Level 1 — Guided

1.1 A server that refuses. Create a bare repository `hub.git` with one commit on `main`, and set `receive.denyNonFastForwards` and `receive.denyDeletes` to `true`. Clone it.

1.2 Try to force. In the clone, amend the commit and run `git push --force origin main`. What does the message say, and which side wrote it?

Level 2 — Partially guided

2.1 Delete. Try `git push origin --delete main`. What is refused?

2.2 Only one. Turn off `receive.denyDeletes` (set it to `false`) and repeat 2.1 on a copy of the branch (push a branch `scratch` first). What changes?

Level 3 — Independent

3.1 Which is which? Match each to a Git server setting or a platform feature: block force pushes; restrict deletions; require a pull request; require passing checks.

3.2 Layering. One ruleset requires two reviews and signed commits on `main`. Another requires three reviews. What applies to `main`?

Level 4 — Professional scenario

4.1 A first policy. Write a five-line protection policy for a team of six that keeps `main` safe, and say who may bypass it.

Level 5 — Troubleshooting

5.1 An administrator force-pushes to a protected branch and it works. Give a likely reason from section 50.3.

5.2 A new ruleset breaks the team's workflow on its first day. Which enforcement status should it have started in?

Chapter 51 — Insights [Deep]

In this chapter

- what the repository's graphs show: pulse, contributors, traffic, commits, code frequency, network
- what each graph counts, and what it does **not** mean
- how to reproduce the counting with Git
- which graphs exist for which repositories

How to read this chapter. This is a **Deep** chapter and can be read later. The Git counting was run and recorded. Statements about GitHub are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used. The dependency graph belongs to Chapter 62, where it is covered with the security features.

Before you start. Chapter 39, Chapter 45 and Chapter 17. The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on newer Git versions.

51.1 What insights are for

Repository **insights** are graphs and lists that summarise activity. If you maintain a project they help you see who is using it and how it is changing. They are **summaries of counts**, and a number is only as meaningful as its rules. This chapter names each graph, says what its documentation says it counts, and shows the same counting in plain Git so that you know what the numbers are made of.

51.2 The graphs

| Graph | What GitHub's documentation says it shows |
|-----------------------|--|
| Pulse | "A list of open and merged pull requests, open and closed issues, and a graph showing the commit activity for the top 15 users who committed to the default branch" in a chosen period |
| Contributors | The top 100 contributors; "merge commits and empty commits aren't counted as contributions for this graph" |
| Traffic | Views, unique visitors, full clones (not fetches), visitors from the past 14 days, referring sites and popular content |
| Commits | All commits in the past year, excluding merge commits: the top graph by week, the bottom by day of the week |
| Code frequency | Content additions and deletions for each week in the repository's history |

| Graph | What GitHub's documentation says it shows |
|---------|--|
| Network | The branch history of the whole repository network, including forks: "up to 100 of the most recently pushed-to branches" |

Some details that change how to read them:

- Traffic is visible to "anyone with push access", updates full clones and visitor information hourly and referring sites and popular content daily, and "all data in the traffic graph uses the UTC+0 timezone".
- "Certain contributor, commit, and code frequency insights are only available for repositories that have fewer than 10,000 commits."
- Your commits count in the contributors graph only if they are merged into the default branch, and you are among the top 100 (Chapter 37 explains that a commit must also match your account by email).
- The graph is **not a measure of skill**. It counts what it counts.

Checked against GitHub's documentation (R277). "About repository graphs", "Viewing a project's contributors", "Using Pulse to view a summary of repository activity", "Analyzing changes to a repository's content", "Understanding connections between repositories" and "Viewing traffic to a repository". The quotations above are from these pages.

Which repositories have which graphs. The documentation says that on the Free plan some graphs (Pulse, Contributors, Traffic, Commits, Code frequency, Network) "are available only in public repositories", while all graphs are available in public and private repositories on paid plans; the traffic page also names Free plans for organizations. Plans and their names change (Chapter 43): check the current page.

51.3 What the counts are made of

Three effects change a count without any change in the work.

Merge commits and empty commits. A merge commit records a merge, and an empty commit changes no file. The contributors graph ignores both. In Git you can count with and without merges. A small repository was built with one person who committed under two different email addresses, plus a merge and an empty commit:

```
$ cd bakery-menu
$ git rev-list --count HEAD
6
$ git shortlog -sne HEAD
  5 Ada Learner <ada@example.org>
  1 Ada L. <ada@old-mail.example>
$ git shortlog -sne --no-merges HEAD
  4 Ada Learner <ada@example.org>
  1 Ada L. <ada@old-mail.example>
$ printf 'Ada Learner <ada@example.org> Ada L. <ada@old-mail.example>\n' > .mailmap
$ git shortlog -sne --no-merges HEAD
  5 Ada Learner <ada@example.org>
```



```
$ git log --shortstat --format='%h %s' --no-merges
da81f52 Trigger the checks again
1fdaf40 Raise the price of the white loaf

1 file changed, 1 insertion(+), 1 deletion(-)
77deeab Add tea

1 file changed, 1 insertion(+)
8b83a9e Add rolls

1 file changed, 1 insertion(+)
a325a15 Add the menu

1 file changed, 3 insertions(+)
```

Recorded in Bash; *ch51-insights/expected-counting.bash.txt*.

Read it in order. `git rev-list --count HEAD` finds six commits. `git shortlog -sne` counts the five by Ada with her main email and one under an older address, **as two people**, and the merge counted for Ada. Adding `--no-merges` drops the merge (five commits by Ada become four). The `git log --shortstat` list shows that the empty commit `Trigger the checks again` has **no changed lines**: it would count in the shortlog but do nothing in the code.

The same person, two identities. Git counts an identity by name and email as written in each commit (Chapter 14). The same person under two addresses appears as two contributors, until a `.mailmap` file (documented in Git's `gitmailmap`) joins them: a line `Proper Name <proper@email> Commit Name <commit@email>` "allows mailmap to replace both the name and the email of a commit matching both the specified commit name and email address". With the mailmap, all five commits count for one Ada.

Lines are not value. The code-frequency graph is additions and deletions per week, as `--shortstat` shows per commit. A deletion of a thousand lines of dead code and an addition of a thousand lines of a generated file both make a tall bar.

Checked against Git documentation and the recorded run (R278). The counting options (`shortlog -sne`, `--no-merges`, `--shortstat`) are shown running on Git 2.43.0 and re-run in CI. The `.mailmap` line format is from Git 2.56.0's `gitmailmap` manual. What the platform counts is as described in R277; the recording reproduces the rules in Git, not GitHub's own computation.

51.4 Using insights well

1. **Ask a question first:** "who reviews our pull requests?", "is anyone using this?", "did the last release change traffic?". Then choose the graph.
2. **Read the period and the rules** before the shape.
3. **Do not use them to rank people.** They count commits, not judgement, review, design, documentation or support.
4. **Do not compare repositories** by these numbers unless the rules are the same.
5. **Check the sample.** A fork's traffic is separate; a private repository on a plan without a graph shows nothing.

Checkpoint

What You Learned

- Insights are summaries of counts, each with its own rules and period.
- The contributors graph ignores merge commits and empty commits and shows the top 100.
- Traffic needs push access, updates on a schedule, and uses UTC.
- The same person under two identities counts twice until a `.mailmap` joins them.
- A graph is not a measure of skill.

New Vocabulary

None: this chapter uses the terms contributor and commit from earlier chapters.

Commands Learned

`git rev-list --count`, `git shortlog -sne`, `git shortlog --no-merges`, `git log --shortstat`, `.mailmap`.

Common Mistakes

1. **Reading a graph as a performance review.**
2. **Forgetting that merge and empty commits are excluded.**
3. **Counting one person twice** because of two email addresses.
4. **Expecting every graph on every plan.**
5. **Ignoring the time period and UTC.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Which commits does the contributors graph ignore?
2. Who can see the traffic graph?
3. Why can one person appear as two contributors?
4. How would you count commits by author without merges?

Before Moving On

You are ready for Chapter [52](#) if you can:

- [] say what a graph counts before reading its shape
- [] reproduce a count with `git shortlog`
- [] explain why the numbers are not a measure of skill

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|---|--------|
| shortlog, --no-merges, --shortstat, .mailmap joining identities | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git; .mailmap format checked in Git 2.56.0's manual | R278 |
| What each graph shows, limits, traffic rules, plan availability | Checked against github/docs (commit 2eaab0b); not run on a live account | R277 |

Where this leads

Chapter 52 begins the automation part of the book. Chapter 62 covers the dependency graph and other security views.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer.

Level 1 — Guided

1.1 Six graphs. Name the six graphs in the table of section 51.2 and say in one line what each shows.

1.2 Count. In bakery-menu, run `git rev-list --count HEAD` and `git shortlog -sne HEAD`.

Level 2 — Partially guided

2.1 Without merges. Make a topic branch, merge it with `--no-ff`, and compare `git shortlog -sne HEAD` with `git shortlog -sne --no-merges HEAD`. What changed?

2.2 An empty commit. Add an empty commit with `git commit --allow-empty -m "..."`. Does `git shortlog` count it? Does `git log --shortstat` show changed lines for it?

Level 3 — Independent

3.1 Two identities. Make one commit with `-c user.name="Ada L." -c user.email="ada@old-mail.example"`. Show that `shortlog -sne` lists two people. Then create a `.mailmap` that joins them and run it again.

3.2 Which graph? Choose the graph that best answers each question: (a) are people cloning the repository? (b) who committed most last year? (c) which branches exist in the forks? (d) what happened last week?

Level 4 — Professional scenario

4.1 A report. A manager asks you to rank the team by the contributors graph. Write a short reply that explains, from this chapter, why you will not, and what you can offer instead.

Level 5 — Troubleshooting

5.1 A contributor's commits are missing from the contributors graph, although they merged pull requests. Give two possible reasons from this chapter and [Chapter 37](#).

5.2 The traffic graph shows a visit on a day when nobody visited in the reader's local time. Explain.

Part VII — Automation and CI/CD

Chapter 52 — CI/CD Concepts [Core]

In this chapter

- what continuous integration and continuous deployment mean
- the stages of a pipeline: build, test, lint, deploy
- how a check script's exit code says pass or fail
- why the same checks should run on a server, not only on your computer

How to read this chapter. The local pipeline was run and recorded. Statements about GitHub Actions are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026); no live account was used. This chapter is about **ideas**, and Chapter 54 shows the platform's way of doing them.

Before you start. Chapter 45 and Chapter 7. The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on newer Git versions.

52.1 The problem that CI solves

Chapter 45 said a pull request is a place for review. Reviewers are people, and people are slow at one job: checking that a change does not break something. A computer is good at exactly that. The idea of **continuous integration** is to let a machine run the same checks on every change, automatically and immediately.

New term: continuous integration (CI). The practice of committing code often to a shared repository and having every change built and tested automatically, so that errors are found early.

GitHub's documentation defines it as "a software practice that requires frequently committing code to a shared repository", and says: "Committing code more often detects errors sooner and reduces the amount of code a developer needs to debug when finding the source of an error. Frequent code updates also make it easier to merge changes from different members of a software development team."

New term: continuous deployment (CD). The practice of using automation to publish and deploy software updates, usually after the code is built and tested automatically.

The documentation: "Continuous deployment (CD) is the practice of using automation to publish and deploy software updates. As part of the typical CD process, the code is automatically built and tested before deployment." CD is "often coupled with" CI.

Checked against GitHub's documentation (R279). "About continuous integration" and "About continuous deployment" are the sources of the quotations above. The documentation notes that tests can include "code linters (which check style formatting), security checks, code coverage, functional tests, and other custom checks", and that you can build and test updates locally before pushing "or you can use a CI server that checks for new code commits in a repository".

52.2 The stages

The stages differ between projects, but four words cover most of them. A sequence of stages is a **pipeline**.

| Stage | Question it answers | Example for the bakery menu |
|---------------|--|---|
| Lint | Is the file written in the agreed style? | No tab characters in menu.md |
| Test | Does it behave correctly? | Every menu item has a price with two decimals |
| Build | Can the deliverable be produced? | Write dist/menu.txt |
| Deploy | Is it published where people use it? | Copy dist/menu.txt to a web server |

New term: pipeline. The ordered stages a change passes through automatically, for example lint, test, build and deploy. A failure in one stage normally stops the later ones.

The order matters: cheap and fast checks first, so that a mistake is reported in seconds and the slow stages do not run for nothing.

52.3 A pipeline in ten lines

A CI service is not magic. At heart it runs a script and looks at whether the script succeeded. Here the bakery project has such a script, `ci.sh`, with three stages. First read it, then run it:

```
$ cd bakery-menu
$ cat ci.sh
#!/bin/sh
echo "lint: no tab characters"
if grep -q "$(printf '\t')" menu.md; then echo "FAIL lint: a tab was found"; exit 1; fi
echo "test: every item has a price"
if grep '^-' menu.md | grep -v -E ': [0-9]+\.[0-9]{2}$'; then echo "FAIL test: a line has
no price"; exit 1; fi
echo "build: write dist/menu.txt"
mkdir -p dist && cp menu.md dist/menu.txt
echo "PASS"
$ sh ci.sh
lint: no tab characters
test: every item has a price
build: write dist/menu.txt
```

```
PASS
$ echo "exit code: $?"
exit code: 0
$ ls dist
menu.txt
```

Recorded in Bash; *ch52-cicd/expected-local-pipeline.bash.txt*.

Every stage prints what it is doing. The script ends with `PASS`, and `echo "exit code: $?"` shows **exit code 0**. In the shell, **exit code 0 means success and anything else means failure**, and that is the signal that CI tools such as GitHub Actions read from each step (Chapter 54) (Chapter 7). A `dist/menu.txt` file was produced by the build stage.

Now a mistake is introduced: a menu line with a price written in words.

```
$ printf -- '- Coffee: two pounds\n' >> menu.md
$ sh ci.sh
lint: no tab characters
test: every item has a price
- Coffee: two pounds
FAIL test: a line has no price
$ echo "exit code: $?"
exit code: 1
```

Recorded in Bash; *ch52-cicd/expected-local-pipeline.bash.txt*.

The lint stage still passes, the test stage prints the offending line and stops with `FAIL test: a line has no price`, and the exit code is **1**. The build stage never runs, so no broken deliverable is produced. This is what a red cross on a pull request means: some stage of such a script returned a non-zero exit code.

The script is a stand-in for a real project's tools (a compiler, a test runner, a linter), but the shape is the same.

52.4 Why run it on a server too

You can run `ci.sh` on your own computer, and you should (Chapter 32 shows a hook that could run it before each push). A CI server adds three things.

1. **A clean, known environment.** "It works on my computer" often means "it depends on something only my computer has". A fresh machine finds that.
2. **Everyone, every time.** Nobody can forget to run it, and it runs on pull requests from people who have never heard of your script.
3. **A visible result.** GitHub's documentation says it "runs your CI tests and provides the results of each test in the pull request, so you can see whether the change in your branch introduces an error", and that when all pass the changes "are ready to be reviewed by a team member or merged".

Two habits keep CI useful. **Keep it fast**, because a check that takes an hour is skipped. **Keep it trustworthy**: a check that fails at random ("flaky") teaches people to ignore red, and then real failures are missed.

CAUTION. Passing checks show that the checks passed, not that the code is right. A test that does not exist cannot fail. And never let a pipeline print or store secrets in its output (Chapter 33, Chapter 60).

52.5 Where GitHub Actions fits

GitHub's documentation says CI with GitHub Actions "offers workflows that can build the code in your repository and run your tests". Workflows "can run on GitHub-hosted virtual machines, or on machines that you host yourself", and can be configured to run when an event occurs (for example, "when new code is pushed to your repository"), "on a set schedule, or when an external event occurs". When you set up CI, GitHub "analyzes the code in your repository and recommends CI workflows based on the language and framework", offering workflow templates. The same tool can run continuous deployment: it "can build the code in your repository and run your tests before deploying", with **environments** that can "require approval for a job to proceed, restrict which branches can trigger a workflow, or limit access to secrets".

Checked against GitHub's documentation (R280). "About continuous integration", "About continuous deployment" and "Understanding GitHub Actions". Chapter 54 defines the terms workflow, event, job, step, runner and action, and Chapter 58 covers environments.

Checkpoint

What You Learned

- CI runs the same checks on every change automatically; CD publishes software automatically after the checks.
- A pipeline is an ordered set of stages; cheap checks come first.
- A check script succeeds with exit code 0 and fails with anything else.
- A server gives a clean environment, coverage of everyone, and a visible result.
- Passing checks are evidence, not proof.

New Vocabulary

Continuous integration (CI), continuous deployment (CD), pipeline (introduced above).

Commands Learned

```
sh script.sh, echo "exit code: $?", grep -E, mkdir -p.
```

Common Mistakes

1. **Believing that green means correct.**
2. **Slow checks** that people bypass.
3. **Flaky checks** that people learn to ignore.
4. **Deploying without a test stage.**
5. **Printing secrets in pipeline output.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What does exit code 0 mean, and what does any other code mean?
2. Why should cheap checks come first?
3. Give two things a CI server provides that your own computer does not.
4. What is the difference between CI and CD?

Before Moving On

You are ready for Chapter 53 if you can:

- [] name the four stages of a pipeline
- [] read a check script and say which stage failed
- [] explain what a red cross on a pull request means

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-------------------|
| A script's exit code decides pass or fail; a failing stage stops the later ones | Locally tested: Bash 5.2 and zsh 5.9 (POSIX sh), Git 2.43.0 environment; CI on newer Git | R280 (local side) |
| Definitions of CI and CD, what CI runs and reports, where Actions fits | Checked against github/docs (commit 2eaab0b); not run on a live account | R279, R280 |
| The stage table and habits | The book's own explanation, not a sourced rule | R279 |

Where this leads

Chapter 53 teaches the file format that workflows use, and Chapter 54 writes a real workflow.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 4 need only your computer.

Level 1 — Guided

1.1 Definitions. In one sentence each, say what continuous integration and continuous deployment are.

1.2 Run a script. In bakery-menu, write a script `ci.sh` that checks that `menu.md` has no tab characters, and prints PASS. Run it and show its exit code with `echo "exit code: $?"`.

Level 2 — Partially guided

2.1 Add a stage. Add a test stage that fails when any line starting with `-` has no price ending in two decimals. Make it fail on purpose, and show that the exit code is not 0.

2.2 Stop early. Add a build stage that writes `dist/menu.txt`. Show that after a failing test stage the build stage did not run and `dist` was not (re)made.

Level 3 — Independent

3.1 A fourth stage. Add a lint rule of your own (for example, no line longer than 80 characters). Where in the order should it go, and why?

3.2 Exit codes in a chain. In your shell, run `false && echo yes` and `true && echo yes`. What does each print, and what does this teach about how a script decides to continue?

Level 4 — Professional scenario

4.1 A pipeline on paper. For a small web project (HTML, CSS and a form), list the stages you would put in a pipeline, in order, with the check for each and what would count as a failure.

Level 5 — Troubleshooting

5.1 A team's checks take 50 minutes and developers have started merging without waiting. Name two changes from this chapter that would help.

5.2 A check fails one run in five with no code change. What is this called, and why is it harmful?

Chapter 53 — YAML for Beginners [Core]

In this chapter

- what YAML is, and why workflows are written in it
- the three building blocks: maps, lists and scalars
- indentation, comments and multi-line text
- the surprises: no, 3.10 and the key on
- how to read a YAML error

How to read this chapter. Every YAML example was parsed by a real parser and the result is shown. The parser is **PyYAML 6.0.1**, which reads some words (no, on) as Booleans, a behaviour that comes from the older YAML **1.1** rules; the chapter says where that matters, and checks the current YAML **1.2.2** specification (from its open repository) for what it says. GitHub reads workflow files with its own tools, so a result here shows how a parser behaves, not how GitHub's does.

Before you start. Chapter [52](#) and Chapter [4](#) (why tabs and spaces are different characters). The recording ran in Bash and zsh with Python 3 and PyYAML, and was re-run in CI.

53.1 What YAML is

New term: YAML. A text format for structured data (settings, lists, nested values) that is meant to be readable by people. Its structure is written with indentation and simple punctuation, not with brackets.

Configuration for automation lives in files: which checks to run, on what event, on which machine. YAML is the format of GitHub Actions workflows (Chapter [54](#)), and it is also used by issue forms (Chapter [44](#)) and many other tools. A mistake in it means a workflow that does not run, so it is worth learning carefully.

The current specification is **YAML 1.2.2** (revision of 1 October 2021), published in the open `yaml/yaml-spec` repository. It says that in block styles "structure is determined by indentation", where indentation is "zero or more space characters at the start of a line", and that "to maintain portability, tab characters must not be used in indentation".

Checked against the YAML specification (R281). The quotations are from YAML 1.2.2, "Indentation Spaces". The specification defines comments as beginning with a hash sign (#), and a Core schema that resolves plain values by pattern: null, true/false (in the spellings true, True, TRUE, false, False, FALSE only), integers, floats, and otherwise strings. The specification recommends the Core schema as the default.

53.2 Three building blocks

A **scalar** is a single value: text, a number, true or false, or null. A **map** (also called a mapping or dictionary) is a set of key: value pairs. A **list** (a sequence) is items introduced by - . Maps and lists nest by **indentation**. All of the following were parsed by the parser; the line after each \$ command is what it produced, with each result written in Python's notation:

```
$ printf 'name: Sunrise Bakery\nopen: true\nprices:\n  loaf: 2.80\n  rolls: 3\n' | python3 y.py
{'name': 'Sunrise Bakery', 'open': True, 'prices': {'loaf': 2.8, 'rolls': 3}}
$ printf 'items:\n  - White loaf\n  - Rolls\n' | python3 y.py
{'items': ['White loaf', 'Rolls']}
```

Recorded in Bash; *ch53-yaml/expected-parse.bash.txt*.

The first document is a map with three keys. The value of open is a real true/false value (True), loaf: 2.80 became the number 2.8, and rolls: 3 the whole number 3. prices is itself a map, nested by two spaces of indentation. The second is a map whose items value is a list of two strings.

Rules that follow from this:

1. **Indent with spaces, never tabs.** Two spaces per level is the usual habit. What matters is that siblings line up exactly.
2. **key: value needs a space after the colon.**
3. **- item needs a space after the dash.**
4. **A comment starts with #** and runs to the end of the line.
5. **Order in a list matters; order of keys in a map should not.** The specification says that "mapping key order" should not be referenced when the data is built, so do not write a file that only works if keys come in a certain order.

53.3 Strings and numbers: the surprises

YAML decides the type of a plain (unquoted) value by looking at it. Usually that is what you want, and sometimes it is a trap. Four cases, parsed:

```
$ printf 'version: 3.10\n' | python3 y.py
{'version': 3.1}
$ printf 'version: "3.10"\n' | python3 y.py
{'version': '3.10'}
$ printf 'answer: no\n' | python3 y.py
{'answer': False}
$ printf 'answer: "no"\n' | python3 y.py
{'answer': 'no'}
```

Recorded in Bash; *ch53-yaml/expected-parse.bash.txt*.

- version: 3.10 became the **number 3.1**: the trailing zero is gone. A version like 3.10 must be written in quotes, "3.10", to stay text. This trap is in YAML 1.1 and 1.2 alike, because 3.10 matches the pattern for a number.

- `answer: no` became **False** in this parser, which follows the older YAML 1.1 habit of reading words such as `yes`, `no`, `on` and `off` as Booleans (the YAML 1.1 working draft for the Boolean type, dated 2005-01-18, lists `y`, `yes`, `n`, `no`, `true`, `false`, `on` and `off` in several capitalisations, and the recording shows the parser doing it). The Core schema of the **1.2.2** specification recognises only `true`, `True`, `TRUE`, `false`, `False` and `FALSE` as Booleans, so `no` is a string there. Different tools follow different versions, so when in doubt, **quote the value**: `"no"`.
- `"no"` in quotes is always the text `no`.

The same rule explains why quoting is the safe habit for anything that must stay text: version numbers, words like `no`, values starting with special characters, and anything that looks like a number but is an identifier (a ZIP code with a leading zero, for instance).

Multi-line text. The specification's block scalars keep line breaks (`|`) or fold them into spaces (`>`). They are used for shell commands that run over several lines (Chapter 54).

53.4 Reading a YAML error

Two of the most common mistakes, parsed:

```
$ printf 'menu:\n - loaf\n - rolls\n' | python3 y.py
ERROR: ParserError
$ printf 'menu:\n\t- loaf\n' | python3 y.py
ERROR: ScannerError
```

Recorded in Bash; *ch53-yaml/expected-parse.bash.txt*.

The first has a list item indented by two spaces and the next by one space: the second `-` does not line up with the first, and the parser stops with a `ParserError`. The second uses a **tab** to indent, which the specification forbids, and the parser stops with a `ScannerError`. The class of the error (and, in a real tool, its line number) tells you where to look.

A checklist when a YAML file "does nothing":

1. Are all tabs gone? (A good editor shows whitespace, Chapter 3.)
2. Do siblings line up exactly, and is each child indented more than its parent?
3. Is there a space after every `:` and `-`?
4. Are values that look like numbers or booleans but should be text in quotes?
5. Is a colon inside a value making the parser think it is a new key? Quote the value.

53.5 A word about `on`

Chapter 54 starts every workflow with a key called `on`. Look at the last recorded line:

```
$ printf 'on: push\n' | python3 y.py
{True: 'push'}
```

Recorded in Bash; *ch53-yaml/expected-parse.bash.txt*.

This parser read the key `on` as the Boolean **True** (the same habit that turned `no` into `false`), and the result is a map whose key is `True`. That is a fact about this parser. GitHub's own documentation writes `on: as an ordinary key in every workflow`, so GitHub's tools do not treat it that way. The lesson is general: **another YAML tool, such as an editor plug-in or a linter, may read `on` differently from GitHub**. If a linter complains about `on`, check which YAML version it follows before changing your workflow.

Checked against the recorded run (R282). The parse results above come from PyYAML 6.0.1 on the runner and locally. What GitHub's parser does with `on` is not tested here; its documentation uses `on: as a plain key` (Chapter 54).

Checkpoint

What You Learned

- YAML has scalars, maps and lists, nested by indentation with spaces, never tabs.
- A plain value's type is guessed: `3.10` becomes `3.1`, and some parsers read `no` as `false`.
- Quote values that must stay text.
- A `ParserError` or `ScannerError` usually means bad alignment or a tab.
- Different YAML tools follow different rules: one parser read the key `on` as a Boolean.

New Vocabulary

YAML (introduced above).

Commands Learned

```
python3 -c 'import yaml'-style parsing (used to check files), printf ... | python3 y.py.
```

Common Mistakes

1. **A tab for indentation.**
2. **An unquoted version number** such as `3.10`.
3. **An unquoted `no`, `yes`, `on` or `off`.**
4. **Children that do not line up with their siblings.**
5. **Trusting one parser's behaviour for all tools.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What are the three building blocks of YAML?

2. Why is `version: 3.10` a problem?
3. Why does the chapter recommend quoting `"no"`?
4. What does a `ScannerError` on a line that starts with a tab tell you?
5. How would you write a two-item list under the key `steps`?

Before Moving On

You are ready for Chapter 54 if you can:

- [] write a small map with a nested list
- [] spot a tab or a misaligned line
- [] decide when to quote a value

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|--------|
| Parse results, error classes, <code>3.10</code> , <code>no</code> , <code>on</code> | Locally tested with PyYAML 6.0.1 and Python 3: Bash 5.2 and zsh 5.9; CI | R282 |
| Indentation rule, no tabs, comments, Core schema and Boolean spellings in YAML 1.2.2; the 1.1 Boolean spellings | Officially verified against the YAML 1.2.2 specification and the YAML 1.1 Boolean type draft (open repository) | R281 |
| What GitHub's parser does with <code>on</code> | Not tested | R282 |

Where this leads

Chapter 54 uses YAML to write real workflows, and Chapter 57 shows how to read the errors GitHub reports.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. You need Python 3 with PyYAML (`python3 -c "import yaml"` should not fail) or an online YAML checker that you trust with non-secret text.

Level 1 — Guided

1.1 A map. Write a YAML map with the keys `name`, `open` (`true`) and `loaf` (`2.80`). Parse it and read the result.

1.2 A list. Write a list of three menu items under the key `items`. Parse it.

Level 2 — Partially guided

2.1 Nested. Write a map `prices` nested inside another map, with two prices. Then write a list inside a map inside a list.

2.2 The trailing zero. Parse `version: 3.10`, then `version: "3.10"`. Explain the difference.

Level 3 — Independent

3.1 The word no. Parse answer: no and answer: "no". What does your parser do? What would you write to be safe with any parser?

3.2 Break it. Make each of these mistakes and read the error class: (a) a tab indent; (b) a sibling one space out of line; (c) a missing space after a colon.

Level 4 — Professional scenario

4.1 Debugging a file. A colleague sends a file that "does nothing". Using the checklist of section 53.4, write the five checks you would make, in order.

Level 5 — Troubleshooting

5.1 A version key python: 3.10 is read as 3.1 and the wrong interpreter is chosen. Explain and fix.

5.2 A linter says the key on is a Boolean. Give two things to check before changing the file.

Chapter 54 — GitHub Actions Fundamentals [Core]

In this chapter

- the six words of GitHub Actions: workflow, event, job, step, runner, action
- where a workflow file lives and how to read one
- on, jobs, runs-on, steps, uses and run
- why each step is its own process
- what a real project's workflow looks like, including this book's own

How to read this chapter. The workflow file and the step behaviour were run and recorded locally. Statements about GitHub Actions are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026). No workflow was run on a live account **by hand**; but this book's repository has its own workflow, which ran on GitHub while the book was written, and section 54.6 reports what it showed.

Before you start. Chapter 53 and Chapter 45. The recording ran in Bash and zsh on Git 2.43.0 and Python 3 with PyYAML, and was re-run in CI.

54.1 The six words

GitHub's documentation defines the pieces of GitHub Actions in one paragraph. Read it slowly, then see the picture.

New term: workflow (GitHub Actions). A configurable automated process, defined in a YAML file in the repository, that runs one or more jobs when triggered by an event.

- A **workflow** is triggered by an **event** in your repository, "such as a pull request being opened or an issue being created".
- The workflow contains one or more **jobs**, which "can run in sequential order or in parallel".
- Each job runs on its own **runner** (a virtual machine, or a container) and has one or more **steps**.
- A step either runs a script that you define, or runs an **action**: "a reusable extension" that does a common task.

New term: event. Something that happens in a repository, such as a push, a pull request or an issue, that can start a workflow. Events can also be a schedule, a call to the platform's programming interface, or a manual start.

New term: job. A set of steps in a workflow that run on the same runner. By default jobs "have no dependencies and run in parallel".

New term: step. One task in a job: either a shell command (run) or an action (uses). Steps run in order.

New term: runner. The machine that runs a job. GitHub provides Linux, Windows and macOS virtual machines, or you can host your own.

New term: action. A pre-defined, reusable set of code that does a specific task inside a workflow, such as pulling your Git repository onto the runner or setting up a language toolchain.

Checked against GitHub's documentation (R283). "Understanding GitHub Actions" is the source of the definitions and quotations. It adds that an event "can also originate from ... a schedule, by posting to a REST API, or manually", that steps "are executed in order and are dependent on each other" and, since they run on the same runner, "you can share data from one step to another", that a **matrix** runs the same job many times with different variables such as operating systems or language versions, and that you can find actions in the Marketplace or write your own.

54.2 A workflow is a file

A workflow is a YAML file (Chapter 53) in the repository. GitHub's documentation is exact about where:

- "Workflow files use YAML syntax, and must have either a .yml or .yaml file extension."
- "You must store workflow files in the .github/workflows directory of your repository."
- The name key gives the workflow's name in the Actions tab; "if you omit name, GitHub displays the workflow file path relative to the root of the repository".

Because it is a file in the repository, a workflow is **versioned, reviewed and merged like code**: you change it on a branch and open a pull request (Chapter 45).

Here is a small workflow for the bakery project. It runs the check script from Chapter 52 on every push and every pull request. After it, a few lines of Python read the file and print its structure:

```
$ cd bakery-menu
$ mkdir -p .github/workflows
$ printf 'name: Check the menu\non: [push, pull_request]\njobs:\n  check:\n    runs-on: ubuntu-latest\n    steps:\n      - name: Get the code\n        uses: actions/
```

```

checkout@v6\n      - name: Run the checks\n      run: sh ci.sh\n' > .github/workflows/
check.yml
$ cat .github/workflows/check.yml
name: Check the menu
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - name: Get the code
        uses: actions/checkout@v6
      - name: Run the checks
        run: sh ci.sh
$ python3 outline.py .github/workflows/check.yml
name: Check the menu
events: ['push', 'pull_request']
job check runs-on ubuntu-latest
  step 1 uses actions/checkout@v6
  step 2 run  sh ci.sh
$ git add .github menu.md
$ git commit -q -m "Add a workflow that runs the checks"
$ git ls-files .github
.github/workflows/check.yml

```

Recorded in Bash; *ch54-actions/expected-workflow-file.bash.txt*.

Read the file from the top.

- name: Check the menu is the name shown on the Actions tab.
- on: [push, pull_request] lists two **events**. The outline tool prints events: ['push', 'pull_request']. (Chapter 53 warned that some YAML tools read a key called on as a Boolean; the outline script handles that, and GitHub's own tools read on as the trigger key.)
- jobs: holds the jobs, each under an identifier of your choice. Here there is one, check.
- runs-on: ubuntu-latest picks the **runner**, a GitHub-hosted Linux machine.
- steps: is a list. Step 1 uses an action, actions/checkout@v6, which puts a copy of your repository on the runner. Step 2 runs a command, sh ci.sh.

The @v6 after the action name selects a **version** of the action. Section 54.5 and Chapter 60 return to why that choice matters.

Checked against GitHub's documentation (R284). The on, jobs, runs-on, steps, uses and run keys are as in the quickstart and the workflow-syntax reference: the quickstart workflow uses on: [push], a job with runs-on: ubuntu-latest, and steps that either run a command or uses the checkout action, whose current documented form is actions/checkout@v6. The syntax reference says a job "contains a sequence of tasks called steps" that "can run commands, run setup tasks, or run an action". What GitHub does with this exact file was **not** run by hand; the outline script only shows that it parses and what it contains.

54.3 Why the first step is almost always `checkout`

A runner starts as a clean machine. Your repository is **not** on it. The step that uses the `checkout` action clones it, and only then can later steps run your scripts, tests or build. Forgetting it is the classic first mistake: the job starts, and `sh ci.sh` fails with "No such file".

The quickstart workflow shows the same order: it prints some information, uses the `checkout` action, then lists the files with `ls`.

54.4 Each step is its own process

GitHub's documentation says: "Each step runs in its own process in the runner environment and has access to the workspace and filesystem. Because steps run in their own process, changes to environment variables are not preserved between steps."

This is ordinary operating-system behaviour, and it can be reproduced with two separate shell processes. The last two lines of the recording do it:

```
$ sh -c 'export MENU_PRICE=2.80; echo "step 1 sets MENU_PRICE=$MENU_PRICE"'
step 1 sets MENU_PRICE=2.80
$ sh -c 'echo "step 2 sees MENU_PRICE=${MENU_PRICE:-nothing}"'
step 2 sees MENU_PRICE=nothing
```

Recorded in Bash; *ch54-actions/expected-workflow-file.bash.txt*.

The first process sets `MENU_PRICE` and prints it. The second, a new process, does not see it: it prints nothing. Files, unlike environment variables, **are** shared, because all steps see the same workspace. So to pass a value between steps you write it to a file, or use the mechanism that Chapter 56 introduces for outputs and environment variables.

Checked against GitHub's documentation (R285). The quotation is from "Workflow syntax: `jobs.<job_id>.steps`". The recording reproduces the operating-system behaviour with `sh -c`; it does not run a GitHub step.

54.5 Reading a workflow you did not write

A method for any workflow file:

1. **Find on.** When does it run?
2. **Find jobs.** How many jobs, and do any depend on others (needs)?
3. **For each job, find runs-on.** Where does it run?
4. **Read the steps in order.** What does each uses, and what does each run?
5. **For every uses, look at what it is and which version it names.** An action is code that runs with access to your repository and possibly your secrets. Treat it like a dependency (Chapter 60).

54.6 A real example: this book's own workflow

The repository of this book has a workflow file, `.github/workflows/validate.yml`, that runs on every push to main, every pull request and on manual start. Its jobs, as they appear as checks on a pull request, are:

| Job (check name) | What it does |
|--|--|
| Structure, references, ledger, secrets | Runs the static checks (<code>tools/check_all.sh</code>) |
| Git command tests (runner's Git version) | Runs the older Git verification scripts on the runner's Git |
| Chapter command transcripts (bash and zsh) | Re-runs every recorded chapter session and compares the output |
| Chapter command transcripts on Git v2.56.0 (built from source) | Builds Git v2.56.0, then re-runs the same sessions |
| PDF edition (build and machine checks) | Builds the PDF and EPUB editions from the manuscript and checks them with machine validators |

Each job uses `ubuntu-latest`, starts with the `checkout` action, installs what it needs with `run` steps, and ends with the command that does the work. When a recording differs on a newer Git, that job turns red and the difference is printed in the job's log, which is exactly how the differences noted in Chapters 20, 27, 28 and 49 were found. This is the CI idea of Chapter 52 applied to a book: the pipeline is the test of the text.

What this shows. A workflow is ordinary configuration in the repository; several jobs run in parallel; a job's failure appears on the pull request. **What it does not show:** how to design a workflow for your project. That is Chapter 56.

Checkpoint

What You Learned

- A workflow is triggered by events, contains jobs, and each job runs steps on a runner; a step runs a command or an action.
- Workflow files are YAML in `.github/workflows` with the extension `.yml` or `.yaml`.
- The `checkout` action puts your repository on the runner; without it, your files are not there.
- Each step is its own process: environment variables do not carry over, but files do.
- Read a workflow by finding `on`, `jobs`, `runs-on` and each step.

New Vocabulary

Workflow (GitHub Actions), event, job, step, runner, action (introduced above). Chapter 34 used workflow for a way of working with Git; here it means the automated process.

Commands Learned

`python3 outline.py <workflow>` (the book's small reader for workflow structure), `sh -c '...'` (to show separate processes).

Common Mistakes

1. **Forgetting the checkout step.**
2. **Expecting an environment variable to carry over to the next step.**
3. **A workflow file outside `.github/workflows`, so it is never found.**
4. **Indenting with a tab** (Chapter 53).
5. **Using an action without knowing what it is or which version.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Where must a workflow file be, and what extensions may it have?
2. What is the difference between a job and a step?
3. What does the checkout action do, and what happens without it?
4. Why is a variable set in one step not visible in the next?
5. Read on: `[push, pull_request]`: when does the workflow run?

Before Moving On

You are ready for Chapter 55 or Chapter 56 if you can:

- [] name the six parts of Actions
- [] read a small workflow file aloud
- [] explain why steps do not share variables

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-------------------------|
| The outline of the workflow file parses; two processes do not share an environment variable | Locally tested: Bash 5.2 and zsh 5.9, Python 3 with PyYAML; CI | R284, R285 (local side) |
| Concepts, file location, name, steps run in their own process, matrix, events | Checked against <code>github/docs</code> (commit <code>2eaab0b</code>); not run by hand on a live account | R283-R285 |
| This book's own workflow and its job names | Observed as pull-request checks while writing this book (GitHub Actions results) | R284 |
| Current form of the checkout action | The <code>github/docs</code> reusable text says <code>actions/checkout@v6</code> ; the tag <code>v6</code> exists in the action's repository | R284 |

Where this leads

Chapter 55 adds expressions and conditions, Chapter 56 builds practical workflows, and Chapter 57 reads their logs.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer (Python 3 with PyYAML); 5 needs a repository where you may add files.

Level 1 — Guided

1.1 Six words. Write one sentence for each of workflow, event, job, step, runner and action.

1.2 A file. In bakery-menu, create `.github/workflows/check.yml` with the workflow of section 54.2 (it may name the checkout action `actions/checkout@v6`). Commit it.

Level 2 — Partially guided

2.1 Read it. Write a small Python script that parses the file with PyYAML and prints the workflow name, its events, and each job with its steps. Remember that PyYAML may read the key on as True.

2.2 Two processes. Run `sh -c 'export A=1; echo $A'` and then `sh -c 'echo ${A:-nothing}'`. Explain what this shows about steps.

Level 3 — Independent

3.1 A second job. Add a job `lint` to the workflow that runs a lint script, with no dependency on `check`. Will the two jobs run in sequence or in parallel, by default?

3.2 A dependency. Read the documentation of `needs` (or ask your teacher) and make `deploy` wait for `check`. Explain what "wait" means.

Level 4 — Professional scenario

4.1 Read a real one. Find a workflow in a public repository you like. Following the method of section 54.5, write down: the events, the jobs, the runner of each job, and every action used with its version. Say which action you trust least and why.

Level 5 — Troubleshooting

5.1 A workflow starts, but its first run step fails with "No such file: ci.sh". The file is in the repository. What is missing?

5.2 A step sets `export VERSION=2` and the next step prints an empty value. Explain, and name two ways to carry a value between steps.

5.3 A learner saved the workflow as `workflows/check.yml` in the repository root and nothing happens. Explain.

Chapter 55 — Expressions, Contexts and Conditionals [Deep]

In this chapter

- what `${{ }}` does
- contexts: the information a workflow can read
- if conditions, comparison rules and status functions
- the Git values behind `github.ref`, `github.ref_name` and `github.sha`

How to read this chapter. This is a **Deep** chapter and can be read later. The Git values behind the `github` context were run and recorded. Everything about expressions is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026). No workflow was run on a live account, so no evaluation result of GitHub's engine is claimed beyond what the documentation says.

Before you start. Chapter [54](#). The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI.

55.1 Expressions

A workflow file is mostly fixed text, but some parts must be worked out at run time: which branch triggered the run, whether a previous step succeeded, which operating system the runner has. An **expression** is a small formula that GitHub evaluates. You write it between `${{` and `}}`.

New term: expression. A small formula, written between `${{` and `}}` in a workflow, that GitHub evaluates while it prepares or runs the workflow, using values, operators, functions and contexts.

Example, from GitHub's documentation: `myString: ${{ 'It's open source!' }}` produces the text `It's open source!`. Inside an expression, a text value uses **single quotes**; the documentation says "wrapping with double quotes will throw an error", and a literal single quote is written as two single quotes.

Checked against GitHub's documentation (R286). "Evaluate expressions in workflows and actions": the data types are boolean (`true`, `false`), null, number ("any number format supported by JSON") and string. In conditionals, "falsy values (`false`, `0`, `-0`, `''`, `null`) are coerced to false and truthy ... to true". The operators are grouping `( )`, index `[ ]`, property `.`, not `!`, the comparisons `<`, `<=`, `>`, `>=`, `==`, `!=`, and `&&` and `||`.

Comparison surprises. The documentation lists rules that beginners do not expect:

- "GitHub ignores case when comparing strings." So `'Main' == 'main'` is true.

- "GitHub performs loose equality comparisons": when the types differ, values are turned into numbers: null becomes 0, true 1 and false 0, and a string is parsed as a number, or becomes NaN (empty string is 0).
- "When NaN is one of the operands of any relational comparison, the result is always false."
- "Objects and arrays are only considered equal when they are the same instance."
- A step output such as `steps.<id>.outputs.<name>` "evaluates as a string": to compare it as a number, convert it with `fromJson()`.

55.2 Contexts

New term: context. A named collection of information that a workflow can read in an expression, such as the `github` context (about the run) or the `runner` context (about the machine).

The documentation's table of contexts:

| Context | It holds |
|-------------------------------|--|
| <code>github</code> | Information about the workflow run |
| <code>env</code> | Variables set in a workflow, job or step |
| <code>vars</code> | Variables set at the repository, organization or environment level |
| <code>job</code> | Information about the current job |
| <code>steps</code> | Information about the steps that have run in the current job |
| <code>runner</code> | Information about the machine running the job |
| <code>secrets</code> | The names and values of secrets available to the run (Chapter 56) |
| <code>strategy, matrix</code> | The matrix (many runs of one job) |
| <code>needs</code> | Outputs of the jobs this job depends on |
| <code>inputs</code> | Inputs of a reusable or manually started workflow |
| <code>jobs</code> | Outputs of jobs, for reusable workflows only |

You read a context with `github.sha` (property syntax) or `github['sha']` (index syntax). Property syntax needs the name to start with a letter or `_`. "If you attempt to dereference a nonexistent property, it will evaluate to an empty string": a misspelt name does not fail, it silently gives nothing.

Contexts are not environment variables. GitHub separates them: default environment variables "exist only on the runner that is executing your job", while most contexts can be used "at any point in your workflow", including before the job is sent to a runner. That is what lets an `if` decide whether a job runs at all.

Checked against GitHub's documentation (R287). "Contexts": the table, the two syntaxes, the empty-string rule, and the contexts-versus-variables distinction. Which contexts are available at which place in a workflow is a further table ("Context availability") and is not reproduced here.

55.3 The Git values inside `github`

Three properties of the `github` context are plain Git ideas. The documentation says:

- `github.ref`: "the fully-formed ref of the branch or tag that triggered the workflow run", in the format `refs/heads/<branch_name>` for branches (and `refs/tags/<tag_name>` for tags); for a workflow triggered by push "this is the branch or tag ref that was pushed".
- `github.ref_name`: "the short ref name of the branch or tag", which "matches the branch or tag name shown on GitHub".
- `github.sha`: "the commit SHA that triggered the workflow", whose value depends on the event.

You can see the same three values in any repository with Git (Chapter 30 explained refs):

```
$ cd bakery-menu
$ git symbolic-ref HEAD
refs/heads/feature-tea
$ git rev-parse --abbrev-ref HEAD
feature-tea
$ git rev-parse HEAD
52240815e6798c7aa39dc658ac7faf9ea4b9b3c2
$ git rev-parse --symbolic-full-name v1.0
refs/tags/v1.0
$ git rev-parse --abbrev-ref v1.0
v1.0
$ git for-each-ref --format='%(refname)'
refs/heads/feature-tea
refs/heads/main
refs/tags/v1.0
```

Recorded in Bash; *ch55-exprs/expected-github-refs.bash.txt*.

`git symbolic-ref HEAD` gives the **full ref** of the current branch, `refs/heads/feature-tea`, the shape of `github.ref`. The abbreviated form, `feature-tea`, is the shape of `github.ref_name`. `git rev-parse HEAD` is the commit SHA. For the tag, the full name is `refs/tags/v1.0` and the short one `v1.0`. `git for-each-ref` lists every ref in the full form.

So a condition like `github.ref == 'refs/heads/main'` (the documentation's own example) asks: "did this run come from the branch `main`?" The full form matters: comparing `github.ref` with `'main'` would never be true.

55.4 Conditions with `if`

An `if` key decides whether a job or a step runs. GitHub's example, in the documentation's contexts page:

```
jobs:
  prod-check:
    if: ${ github.ref == 'refs/heads/main' }
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying to production server on branch $GITHUB_REF"
```

The documentation explains that the `if` "is processed by GitHub Actions, and the job is only sent to the runner if the result is true". Once the job runs, the shell sees the **default environment variable** `$GITHUB_REF`, not the context.

Status functions. In an `if`, four functions look at how the job is going:

| Function | Returns true when |
|--------------------------|---|
| <code>success()</code> | all previous steps have succeeded (this is the default check, applied unless you use another status function) |
| <code>failure()</code> | any previous step of the job failed (for chained jobs, if any ancestor job failed) |
| <code>cancelled()</code> | the workflow was cancelled |
| <code>always()</code> | always, "even when canceled" |

The documentation warns: "Avoid using `always` for any task that could suffer from a critical failure, for example: getting sources, otherwise the workflow may hang until it times out", and recommends `if: ${ !cancelled() }` to run a step whatever its success or failure.

Checked against GitHub's documentation (R287). The status functions, the default `success()` check, and the warning about `always()`, are from "Evaluate expressions in workflows and actions".

55.5 Functions

The documentation lists built-in functions: `contains`, `startsWith`, `endsWith`, `format`, `join`, `toJSON`, `fromJSON`, `hashFiles` and `case`. Two are worth knowing early: `contains('Hello world', 'llo')` tests for a substring (case is ignored), and `fromJSON` turns text into a number or object, which is how you compare a step's output as a number. Chapter 56 uses them.

Checkpoint

What You Learned

- `${{ }}` marks an expression that GitHub evaluates; text inside uses single quotes.
- Comparisons ignore string case and coerce types loosely; a misspelt context property gives an empty string.
- Contexts (`github`, `env`, `steps`, `secrets`, `needs`, `matrix`, and others) are information; default environment variables exist only on the runner.
- `github.ref` is the full ref (`refs/heads/main`), `github.ref_name` the short name, `github.sha` the commit.
- `if` decides whether a job or step runs; `success()`, `failure()`, `cancelled()` and `always()` look at status.

New Vocabulary

Expression, **context** (introduced above).

Commands Learned

`git symbolic-ref HEAD`, `git rev-parse --abbrev-ref`, `git rev-parse --symbolic-full-name`, `git for-each-ref`.

Common Mistakes

1. **Double quotes inside an expression.**
2. **Comparing `github.ref` with a short name.**
3. **A misspelt context property**, which silently gives an empty string.
4. **Using `always()` on a step that fetches the code.**
5. **Comparing a step output as a number without `fromJSON`.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between `github.ref` and `github.ref_name`?
2. Why does `'Main' == 'main'` evaluate to true?
3. What is the default status check of an `if`?
4. Why is a context available before a job reaches a runner, while `$GITHUB_REF` is not?

Before Moving On

You are ready for Chapter 56 if you can:

- `[ ]` read an `if` condition aloud

- [] name the Git command that gives each github value
- [] say what the four status functions mean

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-----------------|
| The Git commands that give the ref forms and the commit SHA | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R287 (Git side) |
| Expression syntax, literals, operators, comparison rules, functions, status functions | Checked against github/docs (commit 2eaab0b); not evaluated by GitHub's engine here | R286 |
| Contexts, syntaxes, contexts vs variables, github.ref, github.sha | Checked against github/docs | R287 |

Where this leads

Chapter 56 uses expressions in practical workflows, and Chapter 57 shows how to find out why an if did not do what you expected.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 and 2 need only your computer.

Level 1 — Guided

1.1 The three values. In bakery-menu, on a branch feature-tea with a tag v1.0, run the Git commands that give the full ref, the short branch name and the commit SHA of HEAD. Which github property is each?

1.2 A tag ref. Run `git rev-parse --symbolic-full-name v1.0`. What form does a tag ref have?

Level 2 — Partially guided

2.1 List all refs. Run `git for-each-ref --format='%(refname)'`. Which lines would a workflow triggered by pushing a tag see in github.ref?

2.2 Read a condition. Say in words when this job runs: `if: ${ github.ref == 'refs/heads/main' }`. Then say what happens if the condition is written `github.ref == 'main'`.

Level 3 — Independent

3.1 Write conditions. Write an if for each: (a) run only when the event is push; (b) run a cleanup step whatever the earlier steps did, unless the run was cancelled; (c) run a step only after a previous failure.

3.2 Types. Say what `'Main' == 'main'`, `1 == '1'` and `null == 0` evaluate to, according to the loose-equality rules in section 55.1, and check them against the documentation.

Level 4 — Professional scenario

4.1 Explain a bug. A step's output count is '10'. The condition `steps.x.outputs.count > 9` behaves unexpectedly. Explain from section 55.1 what to do.

Level 5 — Troubleshooting

5.1 A step that uses `github.actor_name` prints nothing and the workflow does not fail. Explain.

5.2 A team put `if: ${{ always() }}` on the step that checks out the code. Why does the documentation warn against this?

Chapter 56 — Practical Workflows [Core]

In this chapter

- variables and secrets, and passing values between steps
- artifacts and caches, and how they differ
- matrices: one job, many configurations
- manual and scheduled triggers, and how cron fields work
- recipes: tests on pull requests, a format check, a build, issue automation, scheduled maintenance

How to read this chapter. The **cron field rules** and the **matrix expansion** were run and recorded, as tests of what GitHub's documentation says. Everything else is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026). **No workflow in this chapter was run on a live account by hand.** The recipes are built from documented syntax and the book's own suggestions, and each says which. Where a recipe is the book's design and not GitHub's, it is labelled.

Before you start. Chapter [54](#) and Chapter [55](#). The recordings ran in Bash and zsh with Python 3, and were re-run in CI.

56.1 Variables, secrets and passing values

Environment variables. A workflow can define variables with the `env` key, at three levels. GitHub's documentation lists them: for "the entire workflow, by using `env` at the top level", for "the contents of a job", and for "a specific step". "The scope of a custom variable set by this method is limited to the element in which it is defined." For values you want to share across many workflows, you can define a **configuration variable** at the organization, repository or environment level. The documentation warns: "By default, variables render unmasked in your build outputs. If you need greater security for sensitive information, such as passwords, use secrets instead."

Secrets. A **secret** is a named value that you store on the platform (Chapter [38](#) told you why a token never belongs in a file). A workflow reads it through the `secrets` context. GitHub's documentation gives these rules:

- "With the exception of `GITHUB_TOKEN`, secrets are not passed to the runner when a workflow is triggered from a forked repository."
- "Secrets are not automatically passed to reusable workflows", and "are not available to workflows triggered by Dependabot events".
- "Secrets cannot be directly referenced in `if: conditionals`": put the secret in a job-level environment variable and test that.
- "If a secret has not been set, the return value of an expression referencing the secret ... will be an empty string."
- "Avoid passing secrets between processes from the command line, whenever possible", because "command-line processes may be visible to other users"; prefer environment variables or standard input.

- "Mask all sensitive information that is not a GitHub secret by using `::add-mask::VALUE`."

Checked against GitHub's documentation (R288). "Store information in variables", "Using secrets in GitHub Actions" and the reusable text on forked repositories.

Passing a value between steps. Chapter 54 showed that an environment variable does not survive from one step to the next. GitHub's own example writes to a special file: `echo 'NUM_OPEN_ISSUES=$numOpenIssues >> $GITHUB_ENV` in one step, and reads `$NUM_OPEN_ISSUES` in a later step. Files are shared, so this works.

56.2 Artifacts and caches

Two features store files on the platform and are easy to confuse.

| | Artifact | Cache |
|-----------------|---|---|
| What for | Files produced by a job that you want to keep or pass on | Files that are reused between runs and expensive to make |
| Examples | Built programs, test results, logs, coverage reports | Downloaded dependencies |
| Rule | Survives after the run for you to download; can pass files between jobs | A job "should always be able to re-download or regenerate these files if a cache isn't available" |

GitHub's documentation: "An artifact is a file or collection of files produced during a workflow run. Artifacts allow you to persist data after a job has completed, and share that data with another job in the same workflow." The two actions are `upload-artifact` and `download-artifact`.

Why caching exists. "Jobs on GitHub-hosted runners start in a clean runner image and must download dependencies each time, causing increased network utilization, longer runtime, and increased cost."

CAUTION. Caches are a security question. The documentation says: "Caches are shared based on the branch or tag a workflow run uses, not on the identity of the workflow or job", that restored files should be treated "as untrusted input", and that you should "never store secrets or other sensitive data in a cache". "Poisoned caches can lead to code execution in trusted workflows"; GitHub gives workflows that run for low-trust triggers read-only access to caches in the default branch's scope. Chapter 60 returns to this.

Checked against GitHub's documentation (R289). "Store and share data with workflow artifacts", "Dependency caching" and "Manage caches".

56.3 Matrices

To run the same job in several configurations you use a **matrix**. From GitHub's documentation:

```
jobs:
  example_matrix:
    strategy:
      matrix:
        version: [10, 12, 14]
        os: [ubuntu-latest, windows-latest]
```

"A job will run for each possible combination of the variables. In this example, the workflow will run six jobs", created in the order shown in the documentation. A shell loop expands the same matrix, and reproduces the documented count and order:

```
$ for version in 10 12 14; do for os in ubuntu-latest windows-latest; do echo "{version:
{version, os: $os}"; done; done
{version: 10, os: ubuntu-latest}
{version: 10, os: windows-latest}
{version: 12, os: ubuntu-latest}
{version: 12, os: windows-latest}
{version: 14, os: ubuntu-latest}
{version: 14, os: windows-latest}
$ for version in 10 12 14; do for os in ubuntu-latest windows-latest; do echo x; done; done
| wc -l
6
```

Recorded in Bash; *ch56-practice/expected-matrix.bash.txt*.

Six lines, in the documented order: the first variable is the outer loop. This is a **stand-in**, not GitHub's engine: it shows the arithmetic (3 versions times 2 systems is 6 jobs), which is why a matrix with many variables grows fast, and each combination uses a runner and time.

Checked against GitHub's documentation (R290). "Running variations of jobs in a workflow" gives the example, the six combinations and their order. The recording reproduces them with nested shell loops.

56.4 Manual and scheduled triggers

Manual. on: `workflow_dispatch` lets a person start a workflow by hand. GitHub's documentation: "the 'Run workflow' button will be present if the workflow file exists on the default branch", and the workflow can define **inputs** that are read through the `inputs` context. Once a workflow has run at least once you can also start it against any branch or tag through the API or the GitHub CLI.

Scheduled. on: `schedule` runs a workflow at set times, written as **cron** expressions with five fields: minute, hour, day of month, month, day of week. The documentation says:

- "Use POSIX cron syntax", and scheduled workflows run "on the latest commit on the default branch" and (by default) in UTC. The shortest interval is once every 5 minutes.

- The schedule "can be delayed during periods of high loads", including "the start of every hour", so "schedule your workflow to run at a different time of the hour".
- "In a public repository, scheduled workflows are automatically disabled when no repository activity has occurred in 60 days."
- The operators are * (any), , (list), - (range) and / (step); "GitHub Actions does not support the non-standard syntax @yearly, @monthly, @weekly, @daily, @hourly, and @reboot."

The documentation gives examples of what its operators mean. A small script expands the minute and hour fields, and can test them:

```
$ python3 cron.py '15 * * * *'
minutes: [15]
hours: 24 values from 0 to 23
$ python3 cron.py '2,10 4,5 * * *'
minutes: [2, 10]
hours: [4, 5]
$ python3 cron.py '30 4-6 * * *'
minutes: [30]
hours: [4, 5, 6]
$ python3 cron.py '20/15 * * * *'
minutes: [20, 35, 50]
hours: 24 values from 0 to 23
$ python3 cron.py '15 4,5 * * *'
minutes: [15]
hours: [4, 5]
```

Recorded in Bash; *ch56-practice/expected-cron.bash.txt*.

Each result matches the documentation: `15 * * * *` is minute 15 of every hour; `2,10 4,5 * * *` is minutes 2 and 10 of hours 4 and 5; `30 4-6 * * *` is minute 30 of hours 4, 5 and 6; and `20/15 * * * *` gives minutes 20, 35 and 50, the documentation's own words: "every 15 minutes starting from minute 20 through 59". The last line is the documentation's example schedule `15 4,5 * * *`.

The script covers only the minute and hour fields and only these operators; it is a check of the documentation, not a scheduler.

Checked against GitHub's documentation (R291). "Events that trigger workflows": `schedule` and `workflow_dispatch`. Time zones: recent documentation describes an optional `timezone` key for schedules; this book uses UTC and does not depend on it.

56.5 Recipes

Every recipe below has the same shape: an event, a job, a runner, and steps (Chapter 54). They are **the book's designs**, built from documented keys; the two documented examples are marked.

Recipe 1: run the checks on every pull request. Use the check script of Chapter 52:

```

name: Check the menu
on: pull_request
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - run: sh ci.sh

```

Recipe 2: a format check. A separate job for style, so that a formatting mistake is reported in seconds and separately from the tests: the same shape with `run: sh lint.sh`.

Recipe 3: build and keep the result. After the checks, produce the deliverable and keep it as an artifact with the `upload-artifact` action (Section 56.2).

Recipe 4 (documented): comment on a new issue. GitHub's documentation gives this workflow. The GitHub CLI is "preinstalled on all GitHub-hosted runners", and "for each step that uses GitHub CLI, you must set an environment variable called `GH_TOKEN`":

```

name: Comment when opened
on:
  issues:
    types:
      - opened
jobs:
  comment:
    runs-on: ubuntu-latest
    steps:
      - run: gh issue comment $ISSUE --body "Thank you for opening this issue!"
    env:
      GH_TOKEN: ${ secrets.GITHUB_TOKEN }
      ISSUE: ${ github.event.issue.html_url }

```

Whether the default token may comment on issues depends on the permissions given to `GITHUB_TOKEN`, which is Chapter 60's subject.

Recipe 5 (documented): a scheduled report. GitHub's documentation also shows a workflow that runs on: `schedule` with `cron: '20 8 * * *'` ("Daily at 8:20 UTC"), asks the platform how many issues are open with `gh api graphql`, saves the number with `echo 'NUM_OPEN_ISSUES=' ... >> $GITHUB_ENV`, and files an issue with `gh issue create`. That is scheduled maintenance: the same steps at the same time every day, with nobody remembering to do them.

CAUTION. Automation that writes to your repository or its issues can also do damage at scale, and a scheduled workflow keeps running when nobody is watching. Give each workflow the least permission it needs (Chapter 60), and read the log of the first few runs.

Checked against GitHub's documentation (R291). The two documented recipes are from "Using GitHub CLI in workflows" (issue comment and report workflows); the syntax of `run`, `env`, `uses` and `on` is as in Chapter 54.

Checkpoint

What You Learned

- `env` sets variables at workflow, job or step level; secrets come through the secrets context and are not passed to workflows from forks (except `GITHUB_TOKEN`).
- A value moves between steps through a file such as `$GITHUB_ENV`.
- Artifacts keep and pass files produced by a job; caches reuse files that are costly to recreate, and are a security concern.
- A matrix runs one job once per combination: 3 versions and 2 systems make 6 jobs.
- Schedules use five cron fields in UTC, can be delayed, and pause after 60 days without activity in a public repository.

New Vocabulary

Artifact, **matrix**, **cron** (introduced above). Secret was defined in Chapter 33; here it also means a value stored on the platform for workflows.

Commands Learned

`echo 'NAME=value' >> $GITHUB_ENV` (inside a workflow), `gh issue comment`, `gh issue create`.

Common Mistakes

1. **Putting a secret in `env` at the top of a public file** instead of using secrets.
2. **Expecting a secret in a workflow triggered from a fork.**
3. **Scheduling at the start of the hour.**
4. **A matrix that multiplies into dozens of jobs.**
5. **Storing sensitive files in a cache or an artifact.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. How do you pass a value from one step to the next?
2. What is the difference between an artifact and a cache?
3. How many jobs does a matrix with 4 versions and 3 systems create?
4. Which cron expression gives minutes 0, 15, 30 and 45?
5. Why might a scheduled workflow stop running in a public repository?

Before Moving On

You are ready for Chapter 57 if you can:

- [] read a workflow with `env`, secrets, a matrix and a schedule

- [] say what artifacts and caches are for
- [] explain why a workflow should ask for as little as possible

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-------------------------|
| Documented cron examples expand to the documented minutes and hours; a 3 by 2 matrix gives six jobs in the documented order | Locally tested (stand-ins in Python and shell): Bash 5.2 and zsh 5.9; CI | R290, R291 (local side) |
| Variables, secrets, artifacts, caches, matrices, manual and scheduled triggers, the two documented recipes | Checked against github/docs (commit 2eaab0b); not run on a live account | R288-R291 |
| Recipes 1-3 | The book's own designs , built from documented keys, not run | R291 |

Where this leads

Chapter 57 reads logs and fixes failures. Chapter 58 reuses workflows and adds environments, and Chapter 60 covers permissions and untrusted input.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer. Exercise 5 needs a repository where you may add workflows.

Level 1 — Guided

1.1 Expand a matrix. With shell loops, expand a matrix with version: [10, 12, 14] and os: [ubuntu-latest, windows-latest]. How many combinations are there?

1.2 Read a cron. For 2,10 4,5 * * *, say in words when it runs.

Level 2 — Partially guided

2.1 Step values. For */20 * * * * and 20/15 * * * *, list the minutes. Check your answer with a small script.

2.2 Another matrix. How many jobs does a matrix with python: [3.10, 3.11, 3.12], os: [ubuntu-latest, macos-latest] and arch: [x64, arm64] create? Why should you be careful with matrices?

Level 3 — Independent

3.1 A pull request workflow. Write a workflow that runs `sh ci.sh` on every pull request. Then add a second job `lint` that runs in parallel.

3.2 Pass a value. Write two steps: the first stores `PRICE=2.80` in `$GITHUB_ENV`, the second prints it. Explain why the value survives although steps are separate processes.

Level 4 — Professional scenario

4.1 A weekly report. Design a scheduled workflow that files an issue every Monday morning with the number of open pull requests. Choose a cron expression that avoids the start of the hour, and list the permissions it needs.

Level 5 — Troubleshooting

5.1 A workflow works on branches of the repository but the secret is empty when a contributor opens a pull request from a fork. Explain.

5.2 A scheduled workflow in a public repository stopped running after a quiet summer. Why, and how is it reactivated?

5.3 A cache step restores a file that turns out to contain a malicious script. Which documented risk is this, and what does GitHub do to limit it?

Chapter 57 — Debugging and Re-running Workflows [Core]

In this chapter

- a method for finding why a workflow failed, or never started
- reading logs, and turning on debug logging
- the usual causes: triggers, YAML, conditions, permissions
- re-running a workflow, and what a re-run uses
- skipping a run on purpose, and how Git stores that request

How to read this chapter. The Git side (extra Git logging with `GIT_TRACE`, and how a skip request is stored in a commit message) was run and recorded. Statements about GitHub Actions are checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026). No workflow was run on a live account by hand; the logs of this book's own workflow, read while writing it, taught the method in section 57.1.

Before you start. Chapter 56 and Chapter 55. The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI on newer Git versions.

57.1 A method

Most failures fall into one of four groups. Ask the questions in this order, because the earlier ones are cheaper.

1. **Did it start at all?** No run appears: the problem is the trigger (section 57.3).
2. **Did the file load?** A run appears, but with an error about the file: the problem is YAML or syntax (Chapter 53).
3. **Which job or step failed, and what did it print?** Open the log, find the first red step, and read upward from the error: the line that failed is often not the line that caused it.
4. **Does it fail only there?** Try the same command on your own computer, in a clean copy of the repository (Chapter 52 made the script runnable anywhere).

A real example. When this book's workflow first ran the chapter transcripts, one job turned red because a recorded command needed a program (`git filter-repo`) that the runner did not have. The log showed the exact command and a `Python ModuleNotFoundError`. The fix was to install the program in an earlier step of the workflow, and then to make the recording tool, which runs with its own home folder, find it. The lesson: the log usually names the cause. Read the first error, not the last.

57.2 Logs

Checked against GitHub's documentation (R292). "Troubleshooting workflows": "Each workflow run generates activity logs that you can view, search, and download." If the logs "do not provide enough detail to diagnose why a workflow, job, or step is not working as expected, you can enable additional debug logging." The same page also advises enabling the debug or verbose options of the tools you run, and gives Git as its example: "you can use ... `GIT_TRACE=1 GIT_CURL_VERBOSE=1 git ...` for git."

Debug logging. GitHub's documentation on enabling debug logging describes two switches, each set as a **secret or a variable** in the repository (with the same permissions as any secret or variable): `ACTIONS_RUNNER_DEBUG` set to true adds two files to the log archive, the runner process log and the worker process log (in a `runner-diagnostic-logs` folder); `ACTIONS_STEP_DEBUG` set to true "increases the verbosity of a job's logs during and after a job's execution". If both a secret and a variable are set, "the value of the secret takes precedence". Anyone who can run a workflow can also turn on both for a **re-run**. A step can test the `runner.debug` context to run only when debug logging is on.

Git's own trace. When a step fails in a git command, ask Git to explain itself. `GIT_TRACE=1` makes Git print the commands it runs. The recording strips the time stamps and source-line numbers, which vary, and keeps the message:

```
$ cd bakery-menu
$ GIT_TRACE=1 git rev-parse --git-dir 2>&1 | sed -E 's/^[0-9:.]+ +[a-z.]+:[0-9]+ +//'
trace: built-in: git rev-parse --git-dir
.git
```

Recorded in Bash; *ch57-debug/expected-skip-trace.bash.txt*.

trace: built-in: git rev-parse --git-dir says which command ran. On a runner the same variable, set in the step's env, shows every Git command and the programs Git starts. The variable `GIT_CURL_VERBOSE=1` adds the details of web requests, which helps with a failing fetch or push. Remember that a log can print a token: read a trace before you share it (Chapter 33).

Why did the job skip? GitHub's documentation shows how to see how an if was evaluated: open the job, download the log archive, open `JOB-NAME/system.txt`, and look for the Evaluating, Expanded and Result lines. "The Expanded line shows the actual runtime values that were substituted into your if condition, making it clear why the expression evaluated to true or false."

57.3 The workflow never started

The documentation's checklist for triggers, in the words of this book:

- **Is the workflow disabled?** "A disabled workflow does not respond to its triggers."
- **Read the on: field.** Is the event you expect the one listed?
- **Some events run only from the default branch.** "Some triggering events only run from the default branch (i.e. issues, schedule). Workflow file versions that exist outside of the default branch will not trigger on these events."

- **A merge conflict stops pull-request runs.** "Workflows will not run on `pull_request` activity if the pull request has a merge conflict." Resolve the conflict (Chapter 22).
- **Filters.** Branch, tag and path filters can leave no run: "Workflow run creation will be skipped if the filter conditions apply to filter out the workflow." Path filtering "is limited to the first 3,000 files" of a diff in the current documentation; if more files changed, a workflow that depended on later files will not run.
- **A skip request in the commit message.** See section 57.5.
- **A scheduled workflow at an odd time.** Schedules "can be delayed during periods of high loads", especially at the start of an hour (Chapter 56).
- **A cancelled run that will not cancel.** A common cause is `always()`, which returns true even on cancellation; use `!cancelled()` instead.

Checked against GitHub's documentation (R293). All quotations in this section are from "Troubleshooting workflows".

57.4 Re-running

You can re-run all the jobs of a workflow run, or only the failed ones, from the run's page or with the GitHub CLI. Two rules from the documentation matter:

- "Re-runs use the privileges of the actor who initially triggered the workflow, not the privileges of the actor who initiated the re-run. The workflow will also use the same `GITHUB_SHA` (commit SHA) and `GITHUB_REF` (git ref) of the original event that triggered the workflow run."
- "A workflow run can be re-run a maximum of 50 times."

The first rule has a Git meaning: a re-run tests **the same commit**, not the latest one. If you pushed a fix, do not re-run the old run: look for the new run that your push started, or the fix is not being tested. A re-run is right when the failure was in the environment (a network hiccup) and the commit is the one you want to test.

A failure that goes away on a re-run is still a bug. A check that passes on the second try is flaky (Chapter 52). Note it, find the cause, and do not simply re-run until green.

Checked against GitHub's documentation (R294). "Re-running workflows and jobs" (quotations above) and "Enabling debug logging" for the debug options on a re-run.

57.5 Skipping on purpose

Sometimes a change should not start the checks, for example a change to a comment. GitHub's documentation says workflows that would be triggered by push or `pull_request` "won't be triggered" if the commit message (of the push, or the head commit of a pull request) contains any of `[skip ci]`, `[ci skip]`, `[no ci]`, `[skip actions]` or `[actions skip]`. You can instead add a skip-checks: true trailer at the end of the message.

To Git this is just text. Here two commits ask for a skip, one in the subject and one as a trailer, and Git's own trailer tool reads the second:

```
$ printf -- '- Rolls (six): 3.00\n' >> menu.md
$ git commit -q -am "Add rolls [skip ci]"
$ git log -1 --format=%s
Add rolls [skip ci]
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -q -am "Add tea" -m "skip-checks: true"
$ git log -1 --format=%B | git interpret-trailers --parse
skip-checks: true
$ git log --oneline
0de6891 (HEAD -> main) Add tea
785584e Add rolls [skip ci]
5224081 Add the menu
```

Recorded in Bash; *ch57-debug/expected-skip-trace.bash.txt*.

`git interpret-trailers --parse` lists `skip-checks: true` as a recognised trailer. Git did nothing else; it is the platform that reads the message and decides not to start a run.

CAUTION. A skipped check is a check that did not run. The documentation notes that when a workflow is skipped this way, or by path or branch filtering, the checks "will remain in a 'Pending' state", and a pull request that requires those checks can be blocked from merging. Use a skip request only when you are sure, and never on a change that reaches production.

Checkpoint

What You Learned

- Ask in order: did it start, did the file load, which step failed, does it fail elsewhere?
- Logs can be searched and downloaded; `ACTIONS_STEP_DEBUG` and `ACTIONS_RUNNER_DEBUG` add detail; `GIT_TRACE=1` explains a failing Git command.
- A workflow may not start because it is disabled, on the wrong branch or event, blocked by a merge conflict, filtered out, or skipped by the commit message.
- A re-run uses the same commit and the same privileges as the original run, and is limited to 50.
- `[skip ci]` and a `skip-checks: true` trailer are plain text in a commit message that the platform reads.

New Vocabulary

None: this chapter uses the terms workflow, job, step and log from earlier chapters.

Commands Learned

`GIT_TRACE=1 git ..., git interpret-trailers --parse.`

Common Mistakes

1. **Reading the last error instead of the first.**
2. **Re-running an old run after pushing a fix.**
3. **Re-running until green** instead of finding a flaky cause.
4. **Sharing a trace or log that contains a token.**
5. **Skipping the checks on a change that matters.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Which question in the method comes first, and why?
2. Name two switches that add debug detail and what each adds.
3. Give three reasons a workflow may not start.
4. Which commit does a re-run test?
5. How does a skip-checks trailer look to Git?

Before Moving On

You are ready for Chapter 58 if you can:

- [] follow the four-question method
- [] name where to find the reason an if was true or false
- [] say what a re-run uses

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-----------------|
| GIT_TRACE output and <code>git interpret-trailers --parse</code> on a skip-checks trailer | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R294 (Git side) |
| Logs, debug secrets and variables, trigger checklist, re-run rules, skip strings | Checked against <code>github/docs</code> (commit 2eaab0b); not run on a live account | R292-R294 |
| The "real example" of the missing program | Observed in this book's own workflow run while writing it | R292 |

Where this leads

Chapter 58 covers reusable workflows and environments; Chapter 60 covers what workflows should never be allowed to do.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 to 3 need only your computer.

Level 1 — Guided

1.1 The method. Write the four questions of section 57.1 in order.

1.2 Ask Git to explain. Run `GIT_TRACE=1 git rev-parse --git-dir` in a repository. What does the first line say?

Level 2 — Partially guided

2.1 A skip request. Make two commits, one with `[skip ci]` in the subject and one with a `skip-checks: true` trailer. Show the trailer with `git interpret-trailers --parse`.

2.2 Read the log. Find the first red step in a failed run of any repository you can see (or in the sample below) and say which line caused the failure: Run `sh ci.sh / FAIL test: a line has no price / Error: Process completed with exit code 1`.

Level 3 — Independent

3.1 A trigger checklist. A workflow with `on: issues` is in a branch called `new-workflow` and never runs. Give the reason from section 57.3 and the fix.

3.2 Re-run or new run? You pushed a fix commit after a run failed. Should you re-run the failed run? Explain using what a re-run uses.

Level 4 — Professional scenario

4.1 A flaky check. A check fails one time in five. Write the steps you would take, and what you would not do.

Level 5 — Troubleshooting

5.1 A pull request has a merge conflict and its workflows are not running. Explain and fix.

5.2 A job with `if: ${{ always() }}` keeps running after a cancel. What does the documentation suggest instead?

5.3 A required check stays "Pending" forever after a commit message containing `[skip ci]`. Explain, and give two ways to get the pull request mergeable.

Chapter 58 — Reusable Workflows, Environments and Deployments [Deep]

In this chapter

- reusable workflows, and how they differ from composite actions
- environments: approvals, wait timers, branch rules and secrets
- deploying a static website with GitHub Pages and a workflow
- **Project 10:** deploy the Sunrise Bakery website

How to read this chapter. This is a **Deep** chapter and can be read later. The packaging of a site as a Pages artifact (the constraints the documentation sets) was run and recorded locally. Everything else is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026). **No workflow or deployment in this chapter was run on a live account.** Project 10 is written for you to run; its expected result is what the documentation says, not something this book has observed.

Before you start. Chapter [56](#) and Chapter [50](#). The recording ran in Bash and zsh, and was re-run in CI.

58.1 Reusable workflows

When several repositories need the same pipeline, copying the file everywhere means fixing every copy. GitHub's documentation: "Rather than copying and pasting from one workflow to another, you can make workflows reusable."

New term: reusable workflow. A workflow written so that other workflows can call it as a whole, with inputs and secrets passed in. The calling one is the caller; the called one is the called workflow.

A workflow becomes reusable when its `on` includes `workflow_call`:

```
on:
  workflow_call:
    inputs:
      config-path:
        required: true
        type: string
    secrets:
      personal_access_token:
        required: true
```

The documentation says reusable workflows live in `.github/workflows` and that "subdirectories of the workflows directory are not supported". A caller uses the keyword **directly in a job**, "and not from within job steps". The reference forms are:

- `./.github/workflows/{filename}` for a workflow in the same repository (it comes from the same commit as the caller);
- `{owner}/{repo}/.github/workflows/{filename}@{ref}` for another repository, where the `{ref}` can be "a SHA, a release tag, or a branch name". The documentation adds: "Using the commit SHA is the safest option for stability and security."

Three facts to remember. "Secrets are not automatically passed to reusable workflows" (they are passed in `secrets:` or inherited with `secrets: inherit`). If a called workflow uses `actions/checkout`, "the action checks out the contents of the repository that hosts the caller workflow, not the called workflow". And environment secrets cannot be passed from the caller, because `workflow_call` does not support the `environment` keyword.

Reusable workflow or composite action? The documentation compares them: a reusable workflow "allows you to reuse an entire workflow, with multiple jobs and steps"; a composite action "combines multiple steps that you can then run within a job step" and "cannot contain jobs". In the log, a reusable workflow shows every job and step separately; a composite action shows just the one step. If the steps must run on a different type of machine, use a reusable workflow.

Checked against GitHub's documentation (R295). "Reuse workflows" and "Reusing workflow configurations".

58.2 Environments

New term: environment. A named deployment target such as staging or production, with its own protection rules and secrets, that a workflow job can name.

GitHub's documentation: "Environments are used to describe a general deployment target like production, staging, or development." A job can reference **one** environment, and "any protection rules configured for the environment must pass before a job referencing the environment is sent to a runner. The job can access the environment's secrets only after the job is sent to a runner."

The protection rules the documentation lists for an environment:

| Rule | What it does |
|---------------------------|--|
| Required reviewers | Up to 6 people or teams; "only one of the required reviewers needs to approve the job for it to proceed"; an option prevents people from approving runs they triggered (prevent self-review) |
| Wait timer | A number of minutes to wait before the job may proceed |

| Rule | What it does |
|---|--|
| Deployment branches and tags | Which branches and tags may deploy to this environment |
| Custom deployment protection rules | Rules created with GitHub Apps |
| Bypass | Administrators can bypass the rules unless you turn that off |

Environments also hold **environment secrets** and **environment variables**, available "only to workflow jobs that use the environment", and only after the rules pass. A workflow that deploys to an environment appears in the repository's **Deployments** page (Chapter 43).

Checked against GitHub's documentation (R296). "Managing environments for deployment" and "Deployment environments". The documentation notes that on the Free plan environments can only be configured for public repositories; check the page for your plan.

58.3 Deploying a website with GitHub Pages

GitHub Pages publishes static files (HTML, CSS, images) as a website. GitHub's documentation describes building the site with **custom workflows** made of three actions:

1. **actions/configure-pages** gathers information about the site.
2. **actions/upload-pages-artifact** packages the built site. "The Pages artifact should be a compressed gzip archive containing a single tar file. The tar file must be under 10GB in size and should not contain any symbolic or hard links."
3. **actions/deploy-pages** publishes the artifact. The job that uses it must have the permissions `pages: write` and `id-token: write`, must set needs to the build job ("not setting this parameter may result in an independent deployment that continuously searches for an artifact that hasn't been created"), and must name an environment; "the default environment is `github-pages`". The URL of the site comes out of the step as `steps.deployment.outputs.page_url`.

The documentation's deploy job:

```
jobs:
  deploy:
    permissions:
      contents: read
      pages: write
      id-token: write
    runs-on: ubuntu-latest
    needs: jekyll-build
    environment:
      name: github-pages
      url: ${{steps.deployment.outputs.page_url}}
    steps:
```



```
- name: Deploy artifact
  id: deployment
  uses: actions/deploy-pages@v4
```

The packaging rules, tested. The artifact rules are concrete enough to check on your own computer before you push. The recording builds a tiny site in `dist`, checks that there are no symbolic links, packs the folder as one tar file, lists the tar file, and compresses it with `gzip`. Then it adds a symbolic link, to show what the check would catch:

```
$ cd bakery-site
$ find dist -type l | wc -l
0
$ tar -cf artifact.tar -C dist .
$ tar -tf artifact.tar | sort
./
./index.html
./style.css
$ gzip -c artifact.tar > artifact.tar.gz
$ gzip -t artifact.tar.gz && echo "the gzip archive is valid"
the gzip archive is valid
$ ln -s index.html dist/home.html
$ find dist -type l
dist/home.html
```

Recorded in Bash; *ch58-deploy/expected-pages-artifact.bash.txt*.

There are no links in `dist`; the tar file holds `index.html` and `style.css`; the gzip archive is valid. After `ln -s`, `find dist -type l` finds `dist/home.html`, which would break the rule. This is the packaging step only: it does not contact GitHub, and it is not the `upload-pages-artifact` action.

Checked against GitHub's documentation (R297). "Using custom workflows with GitHub Pages" (the three actions, the packaging rules, the permissions, needs, the environment and the URL output; the action versions shown are those in that page: `configure-pages@v5`, `upload-pages-artifact@v4`, `deploy-pages@v4`). The local recording tests the packaging rules only.

58.4 Project 10: deploy the Sunrise Bakery website

Project 10. Goal: publish the Sunrise Bakery site so that a merge to `main` updates the live page, with an approval on the way. Time: about 60 minutes. You need: a GitHub account, a repository with the site in a folder `site/` (Chapter 39), and the ability to change the repository's Pages and environment settings.

1. **Build locally first.** Put `index.html` and `style.css` in `site/`. Run the packaging checks of section 58.3 on `site/` (no symbolic links; one tar file).
2. **Enable Pages with workflows.** In the repository settings, choose GitHub Actions as the source for Pages (the documentation, "Configuring a publishing source", explains the choice). The label may differ; look for the idea.

3. **Write the workflow** `.github/workflows/deploy.yml`: on push to main and workflow_dispatch; a build job that checks out the code, runs `configure-pages` and `upload-pages-artifact` with `path: site`; a deploy job with the permissions, needs: build, the github-pages environment and the deploy-pages step.
4. **Add a protection rule**. In the github-pages environment, add yourself (or a teammate) as a required reviewer, and limit deployment to the main branch.
5. **Push, approve, and look**. Merge the workflow. The deploy job waits for approval; approve it; open the URL from the job.
6. **Break it and read the log**. Introduce a mistake (for example a missing needs) and use Chapter 57 to find it.

Expected result (from the documentation, not observed by this book): the workflow builds the artifact, waits for the required reviewer, deploys, and the site is available at the URL given by `steps.deployment.outputs.page_url`. If a step differs on your account, use the "idea, not label" rule of Chapter 40.

Checkpoint questions: Why must the deploy job list the build job in needs? What does the required reviewer protect against? What would you change to allow a staging environment that deploys without approval?

Checkpoint

What You Learned

- A workflow with `on: workflow_call` can be called by another workflow from a job with `uses`; pin third-party references to a commit SHA.
- Reusable workflows reuse whole jobs; composite actions reuse steps.
- An environment adds approvals, wait timers, branch rules, secrets and a deployment history to a job that names it.
- A Pages deployment uses three actions, the permissions `pages: write` and `id-token: write`, needs on the build job and an environment; the artifact must be one tar file with no links.

New Vocabulary

Reusable workflow, environment (introduced above).

Commands Learned

```
tar -cf, tar -tf, gzip -t, find -type l.
```

Common Mistakes

1. **Calling a third-party workflow by a branch name** that can change under you.
2. **Passing environment secrets from a caller.**
3. **A deploy job without needs.**
4. **Symbolic links in a Pages artifact.**
5. **Giving a production environment no reviewer.**

Practice

Do the exercises in the Exercises section at the end of this chapter. Project 10 is in section 58.4.

Self-Test

1. Which key makes a workflow reusable, and where is it called from?
2. Why is a commit SHA the safest reference for a reusable workflow in another repository?
3. What does a required reviewer on an environment do?
4. Which three actions deploy a Pages site with a custom workflow?

Before Moving On

You are ready for Chapter 59 or Chapter 60 if you can:

- [] write the header of a reusable workflow
- [] say what an environment adds
- [] describe the three Pages actions

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|--|-------------------|
| Packaging rules for a Pages artifact (no links, one tar, gzip) checked on a local folder | Locally tested: Bash 5.2 and zsh 5.9; CI | R297 (local side) |
| Reusable workflows, environments, Pages actions and permissions | Checked against github/docs (commit 2eaab0b); not run on a live account | R295-R297 |
| Project 10 expected result | Documentation's description; not observed | R297 |

Where this leads

Chapter 59 covers where jobs run, Chapter 60 how to secure workflows, and Chapter 69 GitHub Pages in more detail.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1 and 2 need only your computer. Project 10 (section 58.4) needs an account.

Level 1 — Guided

1.1 Package a site. Make a folder `dist` with `index.html` and `style.css`. Check that it has no symbolic links, pack it with `tar -cf artifact.tar -C dist .`, list it, and check a gzip copy with `gzip -t`.

1.2 Header of a reusable workflow. Write the `on:` block of a workflow that can be called, with one required string input `config-path`.

Level 2 — Partially guided

2.1 The link rule. Add a symbolic link to `dist` and run `find dist -type l`. Why must you fix this before a Pages deployment?

2.2 Call it. Write a caller job that calls `./.github/workflows/build.yml` from the same repository, and a second caller job that calls a workflow `example-org/shared/.github/workflows/deploy.yml` pinned to a commit SHA.

Level 3 — Independent

3.1 Environment plan. Design two environments, staging and production, for the bakery site. Say which protection rules each gets and which branches may deploy to each.

3.2 Reusable or composite? For each need, choose reusable workflow or composite action: (a) three jobs on different operating systems; (b) three steps that set up a tool in any job; (c) a shared deployment pipeline for ten repositories.

Level 4 — Professional scenario

4.1 Project 10. Do Project 10 in section 58.4. Write down each step where your screen differed from the description.

Level 5 — Troubleshooting

5.1 A deploy job starts before the site is built and cannot find an artifact. What is missing from the job?

5.2 A called workflow's checkout step shows the wrong files. Explain from the documentation which repository it checked out.

5.3 A team passes an environment secret from a caller workflow and the called workflow gets nothing. Why?

Chapter 59 — Self-Hosted Runners, Concurrency and Limits [Deep]

In this chapter

- GitHub-hosted and self-hosted runners, and what each costs you
- why self-hosted runners are a security decision
- concurrency: stopping runs from colliding, or cancelling outdated ones
- the limits of GitHub Actions (verified facts only)

How to read this chapter. This is a **Deep** chapter and can be read later. Everything is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026), and one limit was checked by arithmetic on your computer. **Limits, prices and plans change often:** the numbers below are those on the page at that commit, and the chapter states only what the page states. No runner was set up and no workflow was run on a live account by hand. This book's own workflow uses the concurrency feature, and section 59.3 says what that showed.

Before you start. Chapter 54 and Chapter 56. The recording ran in Bash and zsh, and was re-run in CI.

59.1 Two kinds of runner

A **runner** is the machine that runs a job (Chapter 54). There are two kinds.

- **GitHub-hosted runners** are machines that GitHub provides. GitHub's documentation says they "start in a clean runner image", which is why a workflow must check out the repository and download its dependencies every time (Chapter 56).
- **Self-hosted runners** are "a system that you deploy and manage to execute jobs from GitHub Actions on GitHub". They "give you more control of hardware, operating system, and software tools", are "free to use with GitHub Actions, though you are responsible for the cost of maintaining your runner machines", can be "physical, virtual, in a container, on-premises, or in a cloud", and "don't need to have a clean instance for every job execution". They "receive automatic updates for the self-hosted runner application only", not for your operating system: "you are responsible for updating the operating system and all other software."

Repository-level runners "are dedicated to a single repository, while organization-level runners can process jobs for multiple repositories in an organization." In a workflow, a job chooses its runner with `runs-on`.

New term: self-hosted runner. A machine that you set up and maintain yourself to run GitHub Actions jobs, instead of a machine that GitHub provides.

Checked against GitHub's documentation (R298). "About self-hosted runners" (the quotations above) and "Understanding GitHub Actions".

When would you want one? When you need hardware or software that GitHub's machines do not offer, when a job must reach a private network, or when you already own the machines.

Note that "free to use" is not the same as "free": the cost moves to you.

59.2 Why self-hosted runners are a security decision

A runner executes whatever the workflow tells it to. On a GitHub-hosted runner that happens on a clean, temporary machine. On a self-hosted runner it happens on **your** machine. GitHub's documentation on secure use says:

- "Self-hosted runners for GitHub do not have guarantees around running in ephemeral clean virtual machines, and can be persistently compromised by untrusted code in a workflow."
- "As a result, self-hosted runners should almost never be used for public repositories on GitHub, because any user can open pull requests against the repository and compromise the environment."
- When a runner is defined at the organization or enterprise level, "GitHub can schedule workflows from multiple repositories onto the same runner. Consequently, a security compromise of these environments can result in a wide impact." Organising runners into separate groups helps to limit the scope.
- Think about what sensitive information is on the machine, such as private SSH keys and API tokens.
- Destroying the runner after each job is only a partial defence: "there is no way to guarantee that a self-hosted runner only runs one job", and some jobs pass secrets as command-line arguments, which another job on the same machine could see. The documentation points to **just-in-time (JIT) runners**, which "perform at most one job before being automatically removed".

Checked against GitHub's documentation (R299). "Secure use reference", section on hardening self-hosted runners. Chapter 60 covers workflow security more widely.

A rule for this book's readers: do not attach a self-hosted runner to a public repository, and treat one attached to a private repository as a machine that trusts every person who can open a pull request there.

59.3 Concurrency

By default GitHub "allows multiple jobs within the same workflow, multiple workflow runs within the same repository, and multiple workflow runs across a repository owner's account to run concurrently". That is usually fine, but not always. The documentation gives two examples of when to limit it: "prevent multiple deployments from running at the same time, or cancel linters checking outdated commits".

New term: concurrency group. A name that GitHub uses to allow only one workflow run or job with that name to be running or pending at a time.

You limit concurrency with the `concurrency` key. Two settings decide what happens to a run that arrives while another in the same group is going: it can wait, or the older one can be cancelled (`cancel-in-progress`). The documentation of its newest version adds that "when you limit concurrency, by default only one run can be pending in a concurrency group; any additional pending runs cancel the previous one", and that queuing must be opted into; that sentence is marked in the documentation source as depending on a feature flag, so check the page for your account.

This book's own workflow. Its file has exactly this at the top:

```
concurrency:
  group: validate-{{ github.ref }}
  cancel-in-progress: true
```

The group name includes the branch (`github.ref`, Chapter 55), so each branch or pull request has its own group, and a new push cancels the run for the older commit. While this book was written, that is what happened when two commits were pushed a minute apart: the first run of the pair ended as cancelled and only the newest commit was tested. That saves time and runner minutes. The price is that you may never see the result of the older commit, which is fine for a check on a branch and would be wrong for a deployment that must finish.

Checked against GitHub's documentation (R300). "Concurrency" and "Control the concurrency of workflows and jobs"; the observation about this book's workflow is from its own run history.

59.4 Limits

GitHub's page on Actions limits says: "You may be rate limited by GitHub Actions when you scale your usage. Some limits can be increased by contacting GitHub Support. Unless otherwise stated, the expected behavior when a limit is reached is that the workflow/job will get cancelled. These limits are subject to change." Some of the limits on the page at the checked commit:

| Limit | Value on that page |
|---|--|
| Workflow run time | 35 days per workflow run, including time spent waiting and on approval |
| Job execution time on GitHub-hosted runners | 6 hours |
| Job execution time on self-hosted runners | 5 days |
| Job queue time on self-hosted runners | 24 hours |

| Limit | Value on that page |
|---|--|
| Job matrix | 256 jobs per workflow run |
| Re-runs | 50 per workflow run |
| Workflow file size | 500 KB per file (a larger file "will not start runs") |
| Concurrent jobs, standard GitHub-hosted runners | 20 on the Free plan and 40 on the Pro plan (of which at most 5 macOS jobs) |
| Dependency cache uploads | 200 per minute per repository |

Testing the matrix limit. A matrix multiplies (Chapter 56), and the limit is easy to pass by accident. Shell arithmetic shows how:

```
$ echo "versions 3 x systems 2 = $((3*2)) jobs"
versions 3 x systems 2 = 6 jobs
$ echo "versions 4 x systems 4 x arch 4 x options 5 = $((4*4*4*5)) jobs"
versions 4 x systems 4 x arch 4 x options 5 = 320 jobs
$ if [ $((4*4*4*5)) -gt 256 ]; then echo "over the documented limit of 256 jobs per workflow run"; fi
over the documented limit of 256 jobs per workflow run
```

Recorded in Bash; *ch59-runners/expected-matrix-limit.bash.txt*.

Three versions on two systems is 6 jobs. A matrix of 4 by 4 by 4 by 5 is 320 jobs, over the documented limit of 256, so a workflow like that would be cancelled or refused. The arithmetic checks the documentation's number; it does not run a matrix.

Checked against GitHub's documentation (R300). "Actions limits", the source of every number in the table. The table is a selection; the page has more (for example runner registration rates and storage limits). Prices and included minutes are **not** in this chapter: they are time-sensitive and depend on your plan, so read the billing page for your account.

Checkpoint

What You Learned

- GitHub-hosted runners are clean machines that GitHub maintains; self-hosted runners are yours to secure, update and pay for.
- Self-hosted runners "should almost never be used for public repositories".
- concurrency with a group name limits simultaneous runs, and cancel-in-progress drops outdated ones.
- Actions has documented limits, including 256 jobs per matrix, 6 hours per hosted job and 35 days per run.
- Numbers on limits and plans change: check the current page.

New Vocabulary

Self-hosted runner, **concurrency group** (introduced above).

Commands Learned

Shell arithmetic `$((a*b))` (to check a matrix size).

Common Mistakes

1. **A self-hosted runner on a public repository.**
2. **Assuming a self-hosted runner is clean for every job.**
3. **cancel-in-progress on a deployment.**
4. **A matrix that multiplies past the limit.**
5. **Copying a limit from an old article.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Who updates the operating system of a self-hosted runner?
2. Why are self-hosted runners risky for public repositories?
3. What does `cancel-in-progress: true` do, and where is it a bad idea?
4. How many jobs may a matrix produce in one run?

Before Moving On

You are ready for Chapter [60](#) if you can:

- [] explain the security difference between the two kinds of runner
- [] write a concurrency block that cancels outdated runs
- [] say where to look up a current limit

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|-------------------|
| The matrix-size arithmetic against the documented 256-job limit | Locally tested (shell arithmetic): Bash 5.2 and zsh 5.9; CI | R300 (local side) |
| Runner types, self-hosted security guidance, concurrency, limits | Checked against <code>github/docs</code> (commit <code>2eaab0b</code>); not run on a live account ; limits change | R298-R300 |
| This book's own concurrency block and its effect | Observed in this repository's workflow runs | R300 |

Where this leads

Chapter 60 covers permissions, third-party actions and untrusted input, and Chapter 61 compares Actions with other CI systems.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises are on paper, with shell arithmetic on your computer.

Level 1 — Guided

1.1 Two kinds. List two differences between a GitHub-hosted runner and a self-hosted runner.

1.2 A matrix size. With shell arithmetic, compute the number of jobs of a matrix with 3 versions, 3 systems and 2 architectures. Is it within the documented limit?

Level 2 — Partially guided

2.1 Over the limit. How many values of a fourth variable would make that matrix exceed 256 jobs?

2.2 Write a block. Write a concurrency block whose group is the workflow name plus the branch, and that cancels in-progress runs.

Level 3 — Independent

3.1 A deployment. Explain why `cancel-in-progress: true` may be wrong for a production deployment, and what you would set instead.

3.2 A risk list. List four risks of a self-hosted runner on a machine that also holds your SSH keys.

Level 4 — Professional scenario

4.1 A request. A colleague wants a self-hosted runner for a public open-source repository "to save minutes". Write a reply from this chapter: what the documentation says, and one alternative.

Level 5 — Troubleshooting

5.1 A run was cancelled and nobody cancelled it. Give two possible reasons from this chapter.

5.2 A workflow file grew until runs stopped starting. Which limit, and what does the documentation suggest?

Chapter 60 — Securing Workflows [Core]

In this chapter

- why a workflow is privileged code, and what that means for secrets and the token
- least privilege for `GITHUB_TOKEN`
- third-party actions and pinning to a commit
- script injection: how a pull request title can run commands, reproduced safely
- untrusted pull requests, and the idea of OpenID Connect

How to read this chapter. The script-injection example from GitHub's documentation was **reproduced with shell scripts** and recorded, without any workflow or GitHub account. Everything else is checked against GitHub's own documentation (the `github/docs` repository at commit `2eaab0b`, 29 September 2026). No workflow was run on a live account by hand. This book's own workflow is used as a small real example of the settings discussed.

Before you start. Chapter 56 and Chapter 33. The recording ran in Bash and zsh on Git 2.43.0 and was re-run in CI.

60.1 A workflow is code that runs with your credentials

A workflow runs commands on a machine, often with access to secrets and to a token that can write to your repository. If someone can make it run **their** commands, they act with your authority. GitHub's documentation, on the risk of a compromised action: it "would have access to all secrets configured on your repository, and may be able to use the `GITHUB_TOKEN` to write to the repository". So the guiding principles are the ones of Chapter 33: **least privilege**, **do not trust input**, and **know what you run**.

60.2 Least privilege for the token

Every workflow run gets a temporary token, `GITHUB_TOKEN`. GitHub's documentation on secure use says:

- "Any user with write access to your repository has read access to all secrets configured in your repository. Therefore, you should ensure that the credentials being used within workflows have the least privileges required."
- "It's good security practice to set the default permission for the `GITHUB_TOKEN` to read access only for repository contents. The permissions can then be increased, as required, for individual jobs within the workflow file."

You set the token's permissions in the workflow with the `permissions` key. It can be at the top, for all jobs; the documentation's example is `permissions: read-all`, and a workflow can list individual scopes. The permissions "are initially set to the default setting for the enterprise, organization, or repository". For workflows from forks, "you can use the `permissions` key to add and remove read permissions ... but typically you can't grant write access".

This book's workflow. Its file starts with:

```
permissions:
  contents: read
```

The book's checks only read the repository, so they ask for nothing more. If a step were compromised, it could not push to the repository with that token.

Checked against GitHub's documentation (R301). "Secure use reference" and the permissions section of "Workflow syntax". The list of available permission scopes is in the documentation and is not reproduced here.

Secrets, again. Chapter 56 listed the rules. Two more from the security page: "because there are multiple ways a secret value can be transformed, automatic redaction is not guaranteed", and if an unredacted secret reaches a log, "you should delete the log and rotate the secret". Do not put a secret inside structured data such as a JSON blob: redaction "largely relies on finding an exact match". Register any value derived from a secret (a Base64 copy, a signed token) as a secret too.

60.3 Third-party actions: pin them

Each uses: line runs someone else's code (Chapter 54). GitHub's documentation lists what to do:

- **Pin actions to a full-length commit SHA.** "Pinning an action to a full-length commit SHA is currently the only way to use an action as an immutable release." The reason: a tag can be moved to different code later, but a commit name cannot, because "they would need to generate a SHA-1 collision for a valid Git object payload" (Chapter 30 explained why the hash identifies the content). "You should verify it is from the action's repository and not a repository fork."
- **Audit the source code** of the action: check "that secrets are not sent to unintended hosts, or are not inadvertently logged".
- **Pin to a tag only if you trust the creator.**

GitHub offers policies to require SHA pinning at the repository and organization level, and Dependabot can keep actions up to date (Chapter 62).

This book's workflow does it. Its uses: lines look like this:

```
- uses: actions/checkout@d23441a48e516b6c34aea4fa41551a30e30af803 # v6
```

The 40-character name is the commit that the v6 tag pointed to when it was written, read from the action's own repository; the comment records which tag that was, so a human can find the next update.

Checked against GitHub's documentation (R302). "Secure use reference", section "Using third-party actions".

60.4 Script injection

A workflow's run: step is a shell script. Before the shell sees it, GitHub replaces every `{{ ... }}` expression with its value, as **text**. If the value comes from a stranger, the stranger's text becomes part of your script.

GitHub's documentation gives the classic example, a step that checks a pull request title:

```
- name: Check PR title
  run: |
    title="{{ github.event.pull_request.title }}"
    if [[ $title =~ ^octocat ]]; then
      echo "PR title starts with 'octocat'"
```

"To inject commands into this workflow, the attacker could create a pull request with a title of `a"; ls $GITHUB_WORKSPACE`." The title closes the quotation, ends the command with a semicolon and adds a command of its own.

Here is the same mechanism with no workflow at all. A shell variable holds a hostile title. Two scripts are written: the first builds itself by pasting the title into its text (what the substitution does), the second receives the title as an **environment variable** and does not paste it:

```
$ title='a"; echo INJECTED-COMMAND-RAN; #'
$ echo "$title"
a"; echo INJECTED-COMMAND-RAN; #
$ printf 'title="%s"\necho "checking: $title"\n' "$title" > vulnerable.sh
$ cat vulnerable.sh
title="a"; echo INJECTED-COMMAND-RAN; #
echo "checking: $title"
$ sh vulnerable.sh
INJECTED-COMMAND-RAN
checking: a
$ printf 'echo "checking: $TITLE"\n' > safe.sh
$ cat safe.sh
echo "checking: $TITLE"
$ TITLE="$title" sh safe.sh
checking: a"; echo INJECTED-COMMAND-RAN; #
```

Recorded in Bash; *ch60-wfsec/expected-injection.bash.txt*.

Read it closely. The hostile title is `a"; echo INJECTED-COMMAND-RAN; #`. Pasted into the script text, it became `title="a"; echo INJECTED-COMMAND-RAN; #`, which is **two commands**, and running the script printed `INJECTED-COMMAND-RAN`: the stranger's command ran. Passed through an environment variable, the same title is only data: the script printed it, whole, and nothing ran. That is the documentation's recommendation: "set the value of the expression to an intermediate environment variable", and use it in the script as `"$TITLE"`:

```
- name: Check PR title
  env:
    TITLE: ${github.event.pull_request.title}
  run: |
    if [[ "$TITLE" =~ ^octocat ]]; then
```

The documentation explains why this works: "the value ... is stored in memory and used as a variable, and doesn't interact with the script generation process". It also names a stronger option: "create a JavaScript action that processes the context value as an argument", since then the value is never used to make a shell script.

Which values are untrusted? The documentation: contexts that "typically end with body, default_branch, email, head_ref, label, message, name, page_name, ref, and title", such as `github.event.issue.title` and `github.event.pull_request.body`. And less obvious ones: "branch names and email addresses, which can be quite flexible in terms of their permitted content. For example, `zzz";echo${IFS}"hello";#` would be a valid branch name". The last line of the recording asks Git whether it agrees, and it does:

```
$ git check-ref-format --branch 'zzz";echo${IFS}"hello";#' && echo "Git accepts that as a
branch name"
zzz";echo${IFS}"hello";#
Git accepts that as a branch name
```

Recorded in Bash; *ch60-wfsec/expected-injection.bash.txt*.

`git check-ref-format --branch` accepts that name (Chapter 20 listed the characters Git forbids, and quotes, semicolons, \$, braces and # are not among them). So a branch name, a commit message and an author name are all text chosen by someone else.

Checked against GitHub's documentation (R303). "Script injections" and "Secure use reference": the example, the recommendation, the list of untrusted contexts and the branch-name example are quoted from them. The recording reproduces the shell mechanism with two small scripts; it is not a GitHub run.

60.5 Untrusted pull requests

Workflows for `pull_request` events from forks run with reduced privileges (no secrets except `GITHUB_TOKEN`, Chapter 56). Two other triggers, `pull_request_target` and `workflow_run`, are different. GitHub's documentation: they are "privileged, which means they share the same cache of the main branch with other privileged workflow triggers, and may have repository write access and access to referenced secrets", and used "with the checkout of an untrusted pull request, expose the repository to security compromises". Its advice: "Avoid using the `pull_request_target` workflow trigger if it's not necessary", and never let such a workflow "explicitly check out untrusted code, including from pull request forks".

The rule for beginners: **never check out and run code from a stranger's pull request in a workflow that has secrets or write access.**

60.6 OpenID Connect, in one paragraph

Deploying to a cloud service needs credentials, which are usually stored as long-lived secrets. GitHub's documentation describes an alternative, **OpenID Connect (OIDC)**: after a trust connection is set up with a cloud provider, "you can configure your workflow to request a short-lived access token directly from the cloud provider". Benefits it lists: "no cloud secrets" to duplicate in GitHub, finer control through the provider's own rules, and credentials that are "only valid for a single job, and then automatically expire". Setting it up depends on the provider and is outside this book; the idea to keep is short-lived credentials beat long-lived secrets.

Checked against GitHub's documentation (R304). "Secure use reference" (untrusted pull requests) and "OpenID Connect" (concept).

60.7 A checklist

1. Set permissions to the least that works, at the top of every workflow.
2. Pin third-party actions to a full commit SHA, from the action's own repository.
3. Never paste an untrusted `${{ }}` value into a `run: script`; use an environment variable.
4. Treat titles, bodies, branch names, commit messages and author names as hostile.
5. Do not use `pull_request_target` or `workflow_run` with code from a fork.
6. Keep secrets out of logs, files and caches; rotate any that leak.
7. Use short-lived credentials where you can.
8. Do not put a self-hosted runner on a public repository (Chapter 59).

Checkpoint

What You Learned

- A workflow runs with credentials; least privilege limits the damage of a mistake.
- `permissions` sets the token's rights; read-only contents is a good default.
- Pin actions to a full commit SHA from the action's repository.
- `${{ }}` is pasted as text into a script; pass untrusted values through an environment variable.
- Branch names, titles and author names can contain shell syntax.
- Do not check out untrusted code in a privileged workflow; prefer short-lived credentials (OIDC).

New Vocabulary

Script injection, **least privilege** (the second from Chapter 33), **OIDC** (introduced above).

Commands Learned

`git check-ref-format --branch`, and passing values as `TITLE="$title" sh safe.sh`.

Common Mistakes

1. **Pasting `${{ github.event... }}` into a `run: script`.**
2. **Pinning to a tag or branch of an unknown action.**
3. **Leaving the token with write access it does not need.**
4. **Using `pull_request_target` with a checkout of the pull request.**
5. **Logging a secret or a value derived from one.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Why is a commit SHA a safer reference than a tag?
2. What does the substitution of `${{ }}` in a `run: script` do to a hostile title?
3. What is the safe way to use an untrusted value in a script?
4. Why is a branch name potentially hostile?
5. Which trigger runs with secrets even for pull requests from forks?

Before Moving On

You are ready for Chapter 62 if you can:

- [] write a permissions block with least privilege
- [] rewrite an injectable step safely
- [] explain why third-party actions are pinned

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-------------------|
| The injection mechanism and the environment-variable defence; Git accepts the documented hostile branch name | Locally tested with shell scripts and Git: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R303 (local side) |
| Token permissions, secrets guidance, pinning, untrusted-input contexts, <code>pull_request_target</code> , OIDC | Checked against <code>github/docs</code> (commit 2eaab0b); not run on a live account | R301-R304 |
| The commit SHA in this book's workflow | Read from the action's repository with <code>git ls-remote</code> | R302 |

Where this leads

Chapter 62 covers GitHub's security features for the whole repository, and Chapter 61 compares Actions with other CI systems.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 1.1 to 2.2 use only your terminal; nothing needs a GitHub account.

Level 1 — Guided

1.1 Reproduce it. In an empty folder, set `title='x'; echo HACKED; #'`. Build `vulnerable.sh` with `printf 'title="%s"\necho "checking: $title"\n' "$title" > vulnerable.sh`, read it with `cat`, and run it with `sh`. What extra line printed?

1.2 Defend it. Write `safe.sh` containing `echo "checking: $TITLE"` and run `TITLE="$title" sh safe.sh`. What is different?

Level 2 — Partially guided

2.1 A hostile branch name. Ask `git check-ref-format --branch` about a name that contains a semicolon and a `$`. Does Git accept it? What does that tell you about branch names in workflows?

2.2 Find the untrusted value. For each context, say whether you would treat it as untrusted: `github.event.issue.title`, `github.sha`, `github.head_ref`, `github.event.pull_request.body`.

Level 3 — Independent

3.1 Rewrite a step. Rewrite this step so that it is not injectable: `run: echo "Hello ${{ github.event.pull_request.title }}"`.

3.2 Pin an action. Explain, in three sentences, why `uses: some/action@v2` is riskier than a full commit SHA, and what you must check before copying a SHA.

Level 4 — Professional scenario

4.1 A review. A colleague proposes a workflow on `pull_request_target` that checks out the pull request branch and runs `npm test` with a deploy secret available. Write the review comment, using the rules of this chapter.

Level 5 — Troubleshooting

5.1 A secret shows in a log as a long unmasked string. Give the order of actions from this chapter.

5.2 A workflow only needs to read the repository but the token has write access. Which key do you add, and where?

Chapter 61 — Actions vs External CI [Deep]

In this chapter

- how any CI system, not only Actions, reports back to a pull request
- checks and commit statuses
- criteria for choosing between GitHub Actions and an external CI system

How to read this chapter. This is a [Deep] chapter about trade-offs; you can skip it on a first read. The statements about GitHub come from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026). **This chapter makes no claims about the features, prices or limits of any other CI product:** they change, and none was checked. The comparison is a list of questions you can ask about any product.

Before you start. Chapter [54](#).

61.1 CI is an idea, not a product

Chapter [52](#) defined continuous integration: every change is built and tested automatically. GitHub Actions is one way to do it. Others are a CI server that you run yourself, or a hosted service from another company. All of them do the same three things: notice a change, run your commands on a machine, and report the result.

The third step is where GitHub matters, because the result must appear on the pull request.

61.2 How a result reaches the pull request

GitHub's documentation on status checks: "Status checks show whether commits meet the conditions set for a repository. They are usually created by external systems, such as continuous integration builds, tests, code scanning, or deployment checks." There are two kinds:

| Kind | Detail | Created by |
|-----------------|---|---------------------------------------|
| Checks | Detailed output, annotations and messages | GitHub Apps, including GitHub Actions |
| Commit statuses | A simpler status for a commit | External services and integrations |

The documentation adds that "GitHub Actions generates checks, not commit statuses, when workflows are run", that both are created through GitHub's API, and that "Anyone with write permissions to a repository can set the state for any status check in the repository". If a status check is **required** for a protected branch (Chapter [50](#)), it must pass before merging; "required status checks can be checks or commit statuses".

The point to remember: **a required check is a name and a result attached to a commit.** GitHub does not care which system produced it, as long as it is reported through the API. That is what makes external CI possible.

The other direction also exists: an external service can start a workflow. The documentation lists the `repository_dispatch` webhook as a way for "an external event" to trigger a workflow, and the `deployment_status` event for "a third party" that provides a deployment status.

Checked against GitHub's documentation (R305). "Status checks" and "Events that trigger workflows".

61.3 Questions to ask when choosing

Use this list on any candidate, GitHub Actions included. The answers change over time, so **look them up at decision time** rather than trusting a book.

| Question | Why it matters |
|---|---|
| Where does the code run: on the vendor's machines or on yours? | Chapter 59 and Chapter 60: self-run machines are yours to secure |
| What does it cost for your amount of use, and for private repositories? | Limits and billing differ by plan and change |
| Which operating systems and processor types can it run? | Your users may need Windows, macOS or Linux tests |
| Where are the secrets kept, and how are they scoped? | A leak reaches everything the pipeline can touch |
| How does the pipeline definition live? | A file in the repository is reviewed like code; a web form is not |
| Can it be reproduced locally? | Chapter 52: the pipeline should be a script you can run yourself |
| How does it report to pull requests, and can the result be required? | See section 61.2 |
| What happens if you leave? | Lock-in: how much of the pipeline is portable |
| Who maintains the reusable pieces, and can you pin them? | Chapter 60 on pinning |

61.4 A portable habit

The most useful habit is independent of the product: **keep the real work in scripts inside the repository and let the CI configuration only call them.** The workflow of this book does that: its checks are the scripts in `tools/` and `verification/`, which a reader can run on their own computer, and the workflow only installs Git and calls them. Moving to another system then means rewriting a short file, not the checks.

Chapter 52 built a local pipeline for the same reason.

61.5 When to keep Actions, when to look elsewhere

There is no universal answer. Actions is convenient when the code is already on GitHub: no second account, no second place for secrets, and results appear as checks on the pull request. Reasons to look elsewhere are usually specific: a required machine type, a policy about where code may run, or an existing pipeline you do not want to rewrite. Decide with the questions in section 61.3 and current documentation. Mixing is common: one system for tests, another for deployment.

Checkpoint

What You Learned

- CI is a practice; any system can do it.
- Results reach a pull request as checks (rich, from apps such as Actions) or commit statuses (simple, from integrations).
- A required status check is a name and result on a commit; it can come from any system.
- Choose by asking questions and reading current documentation.
- Keep real work in scripts so the CI system is replaceable.

New Vocabulary

Commit status and **check** (as GitHub uses them; see the glossary).

Commands Learned

No new commands.

Common Mistakes

1. **Assuming a comparison in a book is current.** Prices and limits change.
2. **Putting the whole pipeline in CI-specific syntax.**
3. **Making a status check required before it reports reliably.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What are the two kinds of status check on GitHub?
2. Which kind does GitHub Actions create?
3. Can an external system's result be required for merging?
4. Give two questions you would ask about any CI product.
5. Why keep the real work in scripts?

Before Moving On

You are ready for Chapter 62 if you can:

- [] distinguish a check from a commit status
- [] list three criteria for choosing a CI system

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|--|--------|
| Checks, commit statuses, required status checks, external triggers | Checked against github/docs (commit 2eaab0b); not run on a live account | R305 |
| The choosing questions and the portable habit | Reasoning and this book's own workflow; no other CI product was examined | R306 |

Where this leads

Part VIII turns to the security features GitHub offers for a whole repository, beginning with Chapter 62.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises are on paper.

Level 1 — Guided

1.1 Two kinds. Name the two kinds of status check on GitHub and who creates each.

1.2 Which one? Which kind does a GitHub Actions workflow create?

Level 2 — Partially guided

2.1 Required. Can a result from a system other than Actions be a required status check? Explain in one sentence.

2.2 Portable. Give one advantage of keeping the real work in scripts in the repository.

Level 3 — Independent

3.1 A checklist. Choose four questions from section 61.3 and answer them for the CI system you use or plan to use, looking up current documentation.

Level 4 — Professional scenario

4.1 A proposal. A team wants to move from an external CI service to Actions. Write five questions to settle before they decide.

Level 5 — Troubleshooting

5.1 A required check named `build` never appears on a pull request from an external system, and the merge is blocked. Give two things to look at.

Part VIII — Security on GitHub

Chapter 62 — GitHub Security Features [Core]

In this chapter

- the dependency graph, Dependabot alerts, security updates and version updates
- secret scanning and push protection
- code scanning and CodeQL (the concept)
- security advisories, SECURITY.md and private vulnerability reporting
- security overview

How to read this chapter. Everything about GitHub here comes from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026). **Nothing was run on a live account:** availability depends on plan, repository visibility and licence, and changes over time, so it is only summarised here, and the settings screens are not described. The one locally tested part is small: a dependabot.yml example from the documentation and a SECURITY.md file, checked as ordinary files in Git.

Before you start. Chapter 33 (secrets and history) and Chapter 60 (least privilege and pinning).

62.1 Why these features exist

Chapter 33 said that Git itself does not protect you: it stores whatever you commit. GitHub adds services around the repository that look for the two most common problems: **a dependency with a known weakness** and **a secret in the code**. This chapter names each feature and says what it does, in the words of the documentation. **Which repositories can use which feature, and at what price, changes:** for anything you plan to rely on, read the current documentation for your account type.

62.2 The dependency graph

Your project uses other people's packages (its **dependencies**). GitHub's documentation: the dependency graph "automatically parses dependencies by analyzing manifests and lock files in your repository". A **manifest** lists what the project needs; a **lock file** records the exact versions chosen. You can also submit dependency data yourself. When a pull request changes dependencies, the graph is used for a **dependency review**, which "indicate[s] whether the dependencies contain vulnerabilities and, if so, the version of the dependency in which the vulnerability was fixed".

For public repositories the graph also lists **dependents**: other public repositories that depend on yours. The supported ecosystems are listed in the documentation.

62.3 Dependabot

Dependabot is the name for three related things.

1. **Dependabot alerts.** GitHub "scans your repository's default branch and sends alerts" when a new vulnerability is added to the GitHub Advisory Database, or when your dependency graph changes. An alert shows the affected file, the vulnerability's details and severity, and a fixed version "when available". Repository administrators and organization owners can enable alerts.
2. **Dependabot security updates.** If enabled, when an alert is raised, Dependabot "automatically tries to fix it": it "raises a pull request to update the dependency to the minimum version that includes the patch and links the pull request to the ... alert". You review and merge it like any pull request (Chapter 45). Alerts close when the related pull request is merged.
3. **Dependabot version updates.** Not about vulnerabilities: it keeps dependencies **up to date**. "You enable ... by checking a `dependabot.yml` configuration file into your repository." When a dependency is outdated it raises a pull request to update the manifest. It also can keep the versions of **actions** in your workflow files up to date, which suits Chapter 60's advice to pin.

The configuration file is ordinary YAML (Chapter 53). The documentation lists the required keys: `version`, `updates`, and for each entry `package-ecosystem`, `directory` (or `directories`) and `schedule.interval` (daily, weekly or monthly). Its minimal example, written and checked locally:

```
$ mkdir -p .github
$ printf 'version: 2\nupdates:\n  - package-ecosystem: "npm"\n    directory: "/" \nschedule:\n  interval: "daily"\n' > .github/dependabot.yml
$ cat .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "daily"
$ python3 check.py .github/dependabot.yml
version: 2
ecosystem: npm | directory: / | interval: daily
```

Recorded in Bash; *ch62-ghsec/expected-config.bash.txt*.

The script only reads the file with PyYAML and prints what it found, so you see that the structure is what the documentation describes. It does **not** contact GitHub, and it does not prove that Dependabot accepts the file: only pushing to GitHub does that. The file lives in the `.github` folder, which is where GitHub looks for such files.

Checked against GitHub's documentation (R307). "About Dependabot alerts", "About Dependabot security updates", "About Dependabot version updates", "About the `dependabot.yml` file", "About the dependency graph". The recording reproduces the documentation's example file as a plain file; no Dependabot ran.

62.4 Secret scanning and push protection

Secret scanning "scans your entire Git history on all branches of your repository for hardcoded credentials, including API keys, passwords, tokens, and other known secret types", and periodically rescans when new secret types are added. It raises an alert on the repository's Security tab. GitHub's advice on an alert: "rotate the affected credential immediately to prevent unauthorized access. While you can also remove secrets from your Git history, this is time-intensive and often unnecessary if you've already revoked the credential." That is exactly the order of Chapter 33: **revoke first**. For many providers GitHub also notifies the provider, "so they can take action, such as revoking the credential". You can add **custom patterns** (regular expressions) for secrets of your own.

Push protection works earlier: it "blocks pushes that contain secrets before they reach your repository". It applies to pushes from the command line, to commits and uploads in the web interface, and to REST API requests. A blocked push comes with a message; you remove the secret and push again. For repositories it "is disabled by default" and can be enabled by a repository administrator, an organization owner or a security manager.

Neither feature is a reason to be careless: they find known patterns. Chapter 33's habits still apply.

Checked against GitHub's documentation (R308). "About secret scanning" and "About push protection".

62.5 Code scanning and CodeQL

Code scanning finds "potential vulnerability or error in your code" and shows alerts in the repository; when you fix the code, the alert closes. You can run scans on a schedule or on events such as a push. It uses **CodeQL**, an analysis product maintained by GitHub, or a third-party tool that outputs **SARIF** (a standard result format). The documentation states that code scanning "uses GitHub Actions, with each workflow run consuming GitHub Actions minutes", and that for private repositories you need a licence. Beginners' takeaway: it is another automated check on your pull requests (Chapter 52).

Checked against GitHub's documentation (R309). "About code scanning".

62.6 Advisories, SECURITY.md and private reporting

You found a hole in your own project, or someone reports one to you. What happens next?

- **SECURITY.md.** A file in the repository that "gives instructions on how to report a security vulnerability in your project". It belongs beside `README.md` (Chapter 42). The recording made a small one: it says to report privately and not in a public issue. **The address in it is an example and does not exist.**
- **Private vulnerability reporting.** Lets a reporter "disclose vulnerability details directly and privately to the repository maintainers by proposing a draft repository advisory".

- **Repository security advisories.** Maintainers can "create a draft security advisory, and use the draft to privately discuss the impact", "privately collaborate to fix the vulnerability in a temporary private fork", and "publish the security advisory to alert your community" when a patch is released.
- **CVE numbers.** A **CVE** is a public identifier for a vulnerability. GitHub "is a CVE Numbering Authority" and can assign one when you publish an advisory.

The steps of the recording, continued:

```
$ printf '# Security policy\n\nReport a vulnerability privately to security@example.org.\nPlease do not open a public issue.\n' > SECURITY.md
$ cat SECURITY.md
# Security policy

Report a vulnerability privately to security@example.org.
Please do not open a public issue.
$ git init -q .
$ git add .github SECURITY.md
$ git ls-files
.github/dependabot.yml
```

Recorded in Bash; *ch62-ghsec/expected-config.bash.txt*.

The practice behind all of this is called **coordinated disclosure**: fix first, then announce.

Checked against GitHub's documentation (R310). "About repository security advisories", "Creating a default community health file" (for SECURITY.md) and the vulnerability-reporting concept pages.

62.7 Security overview

For organizations, **security overview** gives "insights into the overall security landscape of your organization" and helps "identify repositories that require intervention". It is an organization feature and is outside a single-repository beginner's needs; Chapter 68 returns to organizations.

62.8 What to switch on, in order

For a repository you own, and subject to what your plan offers: (1) SECURITY.md; (2) Dependabot alerts; (3) secret scanning and push protection; (4) Dependabot version updates for dependencies and for actions; (5) code scanning if it is available to you. Look at the Security tab first: it shows what is available for **your** repository.

Checkpoint

What You Learned

- The dependency graph lists what you depend on; Dependabot alerts warn about known weaknesses.
- Security updates fix vulnerable dependencies; version updates keep everything current.
- Secret scanning finds committed secrets; push protection blocks them before they arrive. Rotate first.
- Code scanning, with CodeQL or other tools, is an automated check for weaknesses.
- SECURITY.md, private reporting and advisories make reporting and fixing orderly.
- Availability by plan changes: read the current documentation.

New Vocabulary

Dependency graph, Dependabot, push protection, code scanning, CodeQL, security advisory, CVE (see the glossary).

Commands Learned

No new Git commands; `git ls-files` lists tracked files, as in the recording.

Common Mistakes

1. **Believing scanning makes secrets safe.** It finds known patterns.
2. **Cleaning history before rotating the credential.**
3. **Merging every Dependabot pull request without tests.**
4. **Publishing a vulnerability in a public issue.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between Dependabot security updates and version updates?
2. What does push protection do that secret scanning does not?
3. What should you do first when a secret is found?
4. Where does a `dependabot.yml` file live?
5. What is SECURITY.md for?

Before Moving On

You are ready for Chapter 63 if you can:

- [] describe each feature in one sentence
- [] write a minimal `dependabot.yml`

- [] write a `SECURITY.md`

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|--|-------------------|
| The <code>dependabot.yml</code> example is well-formed YAML with the documented keys; a <code>SECURITY.md</code> is an ordinary tracked file | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0, PyYAML 6.0.1; CI on newer Git | R307 (local side) |
| What each feature does; who can enable it; alert, advisory and CVE behaviour | Checked against <code>github/docs</code> (commit <code>2eaab0b</code>); not run on a live account ; availability by plan not established | R307-R310 |

Where this leads

Chapter 63 turns these features into a practice: scopes, signed commits, artifacts and the supply chain.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 and 2.2 use your terminal; none needs a GitHub account.

Level 1 — Guided

1.1 Match. Match each feature to its job: dependency graph, Dependabot alerts, security updates, version updates, secret scanning, push protection, code scanning.

1.2 Order. A secret is found in a commit. Put these in order: rotate the credential, remove it from history, read the alert.

Level 2 — Partially guided

2.1 Write it. In a sandbox folder, create `.github/dependabot.yml` from the documentation's minimal example (npm, directory /, interval daily). Read it back with `cat`.

2.2 A policy. Write a `SECURITY.md` of four lines: how to report, where, what not to do, and what the reporter can expect. Use an example address that does not exist.

Level 3 — Independent

3.1 Explain. In three sentences, explain the difference between Dependabot security updates and Dependabot version updates.

3.2 Actions. Which Dependabot feature helps to keep pinned actions current (Chapter 60)? Say why pinning and updating go together.

Level 4 — Professional scenario

4.1 A small project. You maintain a small open-source project. List, in order, the five things you would switch on this week, and the one you would postpone, with a reason each.

Level 5 — Troubleshooting

5.1 A push is refused with a message that it contains a secret. What are the two safe next steps, and what must you never do?

5.2 Dependabot opened a pull request that fails your tests. What do you do?

Chapter 63 — Repository Security Practice [Core]

In this chapter

- least privilege in practice
- where secrets live: repository, environment and organization
- signed commits on GitHub
- artifacts: checksums and attestations
- the supply chain, and the difference between Git's security and GitHub's

How to read this chapter. GitHub statements come from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026) and were **not run on a live account**. The checksum demonstration is a local recording; it needs sha256sum, which macOS does not provide under that name (it has shasum -a 256), so this book's recording was made on Linux only.

Before you start. Chapter 62 and Chapter 49.

63.1 Least privilege in practice

Chapter 6 introduced least privilege; Chapter 60 applied it to a workflow token. GitHub's documentation on credentials: "When generating credentials, we recommend that you grant the minimum permissions possible. For example, instead of using personal credentials, use deploy keys or a service account. Consider granting read-only permissions if that's all that is needed." When you generate a personal access token, "select the fewest scopes necessary". It also suggests a **GitHub App** instead of a personal token, because an app "uses fine-grained permissions and short lived tokens" and "is not tied to a user, so the workflow will continue to work even if the user who installed the app leaves your organization".

The pattern: **give each person, token and workflow the smallest access that does the job, and no permanent extra**. Chapter 49 covered roles for people; this is the same idea for machines.

63.2 Where a secret lives

Actions secrets can be stored at three levels. From the documentation: "Secrets allow you to store sensitive information in your organization, repository, or repository environments."

| Level | Available to | Notes |
|-------------|--|---|
| Repository | Workflows in that repository | Simple; anyone who can write workflows there can use it in a run |
| Environment | Jobs that target that environment (Chapter 58) | You can require reviewers: "A workflow job cannot access environment secrets until approval is granted" |

| Level | Available to | Notes |
|--------------|--------------------------------------|--|
| Organization | Repositories the organization allows | "you can use a policy to limit access by repository" |

Other rules from the documentation: a workflow "can only read a secret if you explicitly include the secret in a workflow"; secrets are encrypted before they reach GitHub; and log redaction "is not guaranteed" (Chapter 60). Choose the narrowest level that works: an environment secret for a production deployment is stronger than a repository secret, because approval gates it.

Checked against GitHub's documentation (R311). "About secrets" (Actions concepts) and its links.

63.3 Signed commits on GitHub

Chapter 33 signed a commit locally and asked Git to verify it. GitHub can show the result. Its documentation: "If a commit or tag has a GPG, SSH, or S/MIME signature that is cryptographically verifiable, GitHub marks the commit or tag 'Verified' or 'Partially verified.'" The default statuses:

| Status | Meaning |
|------------|---|
| Verified | signed, and the signature verified |
| Unverified | signed, but the signature could not be verified |
| (none) | not signed |

Points the documentation makes:

- "For most individual users, GPG or SSH will be the best choice for signing commits." SSH signatures "are the simplest to generate"; you can upload an existing authentication key to use as a signing key. A GPG key "can expire or be revoked".
- Once verified on GitHub, the record persists: it "can't be edited and will persist ... even if signing keys are rotated, revoked, or if contributors leave the organization".
- **Signing a commit is not the same as signing off** (`git commit -s`, which only adds a line of text; Chapter 64 returns to it).
- **Vigilant mode** (off by default) changes how unsigned commits from you are displayed.

A signature tells you which key made a commit, not that the change is good.

Checked against GitHub's documentation (R312). "About commit signature verification". The SSH-signing behaviour of Git itself was not run (Chapter 33 noted that `ssh-keygen` was unavailable).

63.4 Artifacts: checksums and attestations

An **artifact** is a file your build produces: a package, an archive, a program. People who download it want to know it is the file you published.

A **checksum** is a fingerprint computed from the file's bytes (Chapter 30 used the same idea for Git objects). Publish it beside the file; the downloader computes it again and compares.

```
$ mkdir release && printf 'app v1\n' > release/app.txt
$ tar -cf app.tar release
$ sha256sum app.tar > app.tar.sha256
$ cat app.tar.sha256 | awk '{print length($1), $2}'
64 app.tar
$ sha256sum -c app.tar.sha256
app.tar: OK
$ printf 'tampered\n' >> app.tar
$ sha256sum -c app.tar.sha256 || echo "verification failed: the file changed"
app.tar: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
verification failed: the file changed
```

Recorded in Bash; *ch63-secpractice/expected-checksum.bash.txt*.

The fingerprint printed as 64 characters (SHA-256). After one line was appended, the check said FAILED. Two cautions: a checksum published on the same page as the file proves only that the download was not corrupted, not that the publisher is honest, because an attacker who replaces the file can replace the checksum; and Chapter 66 returns to publishing.

Artifact attestations are GitHub's stronger answer. The documentation: they use **Sigstore**, an open-source signing project, to create a signed statement linking an artifact to "the source code and the build instructions that produced" it; reusable workflows can raise the **SLSA** level. It also warns: "artifact attestations are not a guarantee that an artifact is secure", and "Generating attestations alone doesn't provide any security benefit, the attestations must be verified". It advises signing "binaries people will run, packages people will download", and **not** "frequent builds that are just for automated testing" nor "individual files like source code".

Checked against GitHub's documentation (R313). "Artifact attestations" (concept).
The checksum demonstration is locally tested; attestations were not run.

63.5 The supply chain

Your software is made of your code, your dependencies, your build tools, your workflows' actions, and the machines that run them. An attacker needs only one weak link. The features of Chapter 62 and the habits of Chapter 60 each cover a link:

| Link | Protection |
|---------------------------|---|
| Dependencies | dependency graph, Dependabot, review of updates |
| Actions used by workflows | pin to commit SHAs; keep updated |

| Link | Protection |
|---------|--|
| Secrets | least privilege, scoping, push protection |
| Source | protected branches, reviews, signed commits (Chapters 50 and 46) |
| Outputs | checksums, attestations |

63.6 Git security versus GitHub security

Keep the two apart. **Git** is a local tool: it stores content by hash, can sign commits, and has no notion of who may push; a rewritten history is only detected if someone compares hashes.

GitHub adds accounts, permissions, branch protection, scanning and audit records around it. A protection that GitHub enforces (a required review) does not exist in a plain clone on someone's computer, and a protection Git offers (a signature) means little unless someone verifies it. When you read that "the repository is secure", ask which layer is meant.

Checkpoint

What You Learned

- Give people, tokens and workflows the minimum access.
- Secrets can be scoped to a repository, an environment (with approval) or an organization.
- GitHub marks signed commits Verified or Unverified; a signature says which key, not that the change is good.
- A checksum detects change; attestations link an artifact to its source and build, and must be verified.
- The supply chain has many links; Git and GitHub protect different layers.

New Vocabulary

Checksum, artifact, attestation, supply chain (see the glossary).

Commands Learned

`sha256sum` and `sha256sum -c`, on Linux.

Common Mistakes

1. **Using a personal token where a narrower credential would do.**
2. **Storing a production secret at repository level when an environment gate is possible.**
3. **Treating "Verified" as "reviewed and safe".**
4. **Publishing a checksum only where an attacker could also replace it.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Name the three levels at which Actions secrets can be stored.
2. What does an environment's required reviewer add?
3. What does "Verified" mean on a commit?
4. Why is a checksum next to the file weak evidence?
5. What is the difference between Git's security and GitHub's?

Before Moving On

You are ready for Part IX if you can:

- [] choose the narrowest home for a secret
- [] make and verify a checksum
- [] explain what a signature does and does not show

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|--|-------------------|
| Checksum creation, matching and failure after a change | Locally tested: Bash 5.2 and zsh 5.9, Linux sha256sum; CI | R313 (local side) |
| Credential guidance, secret levels, signature statuses and persistence, attestations | Checked against github/docs (commit 2eaab0b); not run on a live account | R311-R313 |

Where this leads

Part IX turns from protecting a project to sharing it: Chapter [64](#) begins open source.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercise 2.1 needs a Linux terminal (or `shasum -a 256` on macOS).

Level 1 — Guided

1.1 Levels. Name the three levels at which Actions secrets can be stored and one use of each.

1.2 Statuses. What are the default commit signature statuses on GitHub?

Level 2 — Partially guided

2.1 A checksum. Make a small file, compute its SHA-256, save it, verify it, change the file and verify again. What does the second check print?

2.2 Which key? List two advantages of a GitHub App token over a personal access token for a workflow.

Level 3 — Independent

3.1 A weak link. Explain why publishing a checksum next to a download on the same page proves less than publishing an attestation.

3.2 Signed but bad. A commit is marked Verified. Give a reason it can still be a bad change.

Level 4 — Professional scenario

4.1 A production key. A deployment needs a cloud credential. Decide where to store it, who may approve its use, and what else you would do to limit damage. Use the chapter.

Level 5 — Troubleshooting

5.1 A downloaded release fails its checksum. Give three possible causes and the first thing you do.

5.2 A commit you signed shows Unverified. Give two things to check.

Part IX — Open Source and Professional Development

Chapter 64 — How Open Source Works [Core]

In this chapter

- what "open source" means, and what it does not mean
- the roles people play in a project
- the contribution flow, from finding a project to a merged change
- CONTRIBUTING.md, the code of conduct and the community profile
- good first issues, sign-off, and contributing responsibly

How to read this chapter. Statements about GitHub features come from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026); they were **not run on a live account**. The definition of open source and everything about licences is in Chapter 65, and **this book is not legal advice**. The roles and advice in this chapter are the author's description of common practice, not a standard; they are marked as such. One small command demonstration (sign-off) was recorded.

Before you start. Chapter 45 and Chapter 49 (forks, upstream, pull requests).

64.1 What "open source" means

You will meet three separate ideas that people mix up:

1. **Public.** Anyone can see the code (Chapter 39: a public repository).
2. **Source-available.** The code can be read, but the licence may forbid some uses.
3. **Open source.** The code is public **and** the licence grants specific permissions to use, change and share it.

Being visible is not permission. This is the first of the seven distinctions this book keeps returning to: reading is not copying, and neither is modifying or reusing. The permission comes from the licence file, not from the repository being public. Chapter 65 explains licences; **until you have read it, treat any code without a licence as not yours to reuse**.

64.2 Roles in a project

Projects use different words. This is one common description, the author's, not an official list:

| Role | What they usually do |
|-------------|--|
| User | Uses the software; may report problems |
| Contributor | Sends a change, a report, a translation or a documentation fix |
| Maintainer | Reviews and merges changes, triages issues, makes releases |
| Owner | Controls the repository or organization and its settings |

GitHub's permission levels (Chapter 49) are separate: they say what an account **can do**, while a role says what a person **does**. A maintainer with write access is common; a contributor with no access at all, working from a fork, is the normal case for a stranger.

64.3 The contribution flow

The steps below are Chapter 49 and Chapter 45 applied to someone else's project:

1. **Read first.** The README, CONTRIBUTING.md, the code of conduct, the licence.
 2. **Look at the issues.** Is your idea, bug or fix already discussed? Say hello in an existing issue instead of duplicating it.
 3. **Discuss before building anything large.** A quick issue asking "would you accept this?" can save days.
 4. **Fork, clone, branch.** Work on a topic branch, never on main of your fork (Chapter 49).
 5. **Make one small, focused change**, follow the project's style, add or update tests if the project has them.
 6. **Open a pull request** that says what and why, links the issue, and follows the pull request template if there is one (Chapter 45).
 7. **Respond to review** politely and promptly (Chapter 46). A request for changes is normal, not a rejection.
 8. **Accept the decision.** A maintainer may decline a good change because it does not fit the project. You can keep your fork.
-

64.4 The files that tell you the rules

GitHub's documentation describes these files:

- **CONTRIBUTING.md.** "To help your project contributors do good work, you can add a file with contribution guidelines to your project repository's root, docs, or .github folder. When someone opens a pull request or creates an issue, they will see a link to that file." If there are several, the one shown is chosen "in the following order: the .github directory, then the repository's root directory, and finally the docs directory". Filenames "are not case sensitive".
- **Code of conduct.** "A code of conduct defines standards for how to engage in a community." The documentation's advice to maintainers: "Consider carefully whether you are willing and able to enforce it", and, if you use one written by someone else, follow "any attribution guidelines from the source".
- **The community profile.** A checklist that "checks to see if a project includes recommended community health files, such as README, CODE_OF_CONDUCT, LICENSE, or CONTRIBUTING". As a contributor you can use it to see whether the project is set up to welcome you.
- **Templates** for issues and pull requests (Chapters 44 and 45) and SECURITY.md (Chapter 62).
- **The licence** (Chapter 65).

Organizations and personal accounts can also supply **default** community health files for all their repositories, from a special .github repository.

Checked against GitHub's documentation (R314). "Setting guidelines for repository contributors", "Adding a code of conduct to your project", "About community profiles for public repositories", "Creating a default community health file".

64.5 Good first issues

The documentation: "You can apply the good first issue label to issues in your public repository so that people can find them when searching by labels." GitHub "uses an algorithm to determine the most approachable issues in each repository and surface them in various places", and the label "can increase the likelihood that your issues are surfaced". As a newcomer, search for that label, read the issue **and** the surrounding discussion, and say you would like to take it before you start, so that two people do not do the same work.

Checked against GitHub's documentation (R315). "Encouraging helpful contributions to your project with labels". How GitHub's algorithm chooses issues was not examined.

64.6 Sign-off

Some projects ask contributors to **sign off** their commits, meaning to add a line that says the author has the right to submit the change under the project's rules. **It is text, not a cryptographic signature** (Chapter 33 and Chapter 63 covered real signatures). Git can add the line for you:

```
$ git init -q proj && cd proj
$ printf 'hello\n' > a.txt && git add a.txt
$ git commit -q -s -m "Add greeting"
$ git log -1 --format=%B
Add greeting

Signed-off-by: Ada Learner <ada@example.org>

$ git log -1 --format='%(trailers:key=Signed-off-by,valueonly)'
Ada Learner <ada@example.org>
```

Recorded in Bash; *ch64-oss/expected-signoff.bash.txt*.

`git commit -s` added Signed-off-by: with the name and email from your configuration (Chapter 14). The `%(trailers:...)` format picks that line out of the message, and it can be used to see which commits have one:

```
$ git commit -q --allow-empty -m "No sign-off here"
$ git log --format='%s | %(trailers:key=Signed-off-by,valueonly,separator=)'
No sign-off here |
Add greeting | Ada Learner <ada@example.org>
```


Recorded in Bash; *ch64-oss/expected-signoff.bash.txt*.

GitHub's documentation on the web interface: "Commit signoffs enable users to affirm that a commit complies with the rules and licensing governing a repository." A repository or organization can require sign-off for commits made **through the web interface**; the documentation describes that setting only for web-based commits. What a given project means by sign-off is stated in that project's own contribution guide: read it, and do not sign off if you do not have the right to submit the work.

Checked against GitHub's documentation (R316) and locally tested. "Managing the commit signoff policy for your repository". The recording ran on Git 2.43.0 in Bash and zsh and is re-run in CI on newer Git.

64.7 Contributing responsibly

This advice is the author's, and common in open-source projects, not a rule:

- **Respect maintainers' time.** Most are volunteers. Read before asking; make your change easy to review.
- **Keep the change small and single-purpose.**
- **Do not submit what you do not understand or cannot explain**, whoever or whatever wrote it, and do not submit work you have no right to submit (Chapter 65).
- **Never include secrets or personal data** in a pull request (Chapter 33).
- **Be patient and kind.** Follow the code of conduct, in the tone of every comment.
- **Report security problems privately**, using the project's SECURITY.md (Chapter 62), not in a public issue.
- **Say thank you.** It costs nothing.

Checkpoint

What You Learned

- Public does not mean open source; the licence grants the permissions.
- A role (what you do) differs from a permission (what you can do).
- The flow: read, look, discuss, fork and branch, change, open a pull request, respond, accept.
- CONTRIBUTING.md, the code of conduct and the community profile state a project's rules.
- Sign-off is a line of text added with `git commit -s`; read the project's meaning.

New Vocabulary

Open source, maintainer, contributor, code of conduct, sign-off (see the glossary).

Commands Learned

```
git commit -s; git log --format='% (trailers:key=...)'.
```

Common Mistakes

1. **Assuming public code may be reused.**
2. **Starting a large change without asking.**
3. **Working on `main` of a fork.**
4. **Submitting several unrelated changes in one pull request.**
5. **Taking offence at a request for changes.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between public and open source?
2. Name the steps of the contribution flow in order.
3. Where can `CONTRIBUTING.md` live, and which one wins?
4. What does `git commit -s add`?
5. Is sign-off the same as a signed commit?

Before Moving On

You are ready for Chapter 65 if you can:

- [] explain why visible code is not automatically reusable
- [] describe the contribution flow to a friend
- [] find a project's contribution rules

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-------------------|
| <code>git commit -s</code> adds a Signed-off-by trailer; the trailers format reads it | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R316 (local side) |
| Contribution guidelines, code of conduct, community profile, good first issue, commit signoff setting | Checked against github/docs (commit 2eaab0b); not run on a live account | R314-R316 |
| Roles, flow and responsible-contribution advice | The author's description of common practice; not a standard | none |

Where this leads

Chapter 65 explains copyright and licences, the thing that turns public code into code you may use.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercise 2.1 uses your terminal only.

Level 1 — Guided

1.1 Three ideas. Explain the difference between public, source-available and open source in one sentence each.

1.2 Role or permission. For each, say whether it is a role or a permission: maintainer, write access, contributor, admin.

Level 2 — Partially guided

2.1 Sign off. In a new repository, make a commit with `git commit -s` and show its full message with `git log -1 --format=%B`. What line was added?

2.2 Find the rules. Pick a public project and list where you looked to find how to contribute.

Level 3 — Independent

3.1 The flow. Write the eight steps of the contribution flow in your own words, with the Git command or GitHub action for each.

3.2 A small change. Describe a documentation change that would be a good first contribution, and why it is good.

Level 4 — Professional scenario

4.1 A declined pull request. A maintainer closes your pull request with "thanks, but this does not fit the project". Write your reply and say what you do next.

Level 5 — Troubleshooting

5.1 Your pull request has been open for two weeks with no reply. Give three things you can check or do, in order, without being rude.

5.2 A project requires sign-off and your commit lacks it. What command fixes the most recent commit, and what should you check before signing off?

Chapter 65 — Copyright and Licences [Core]

In this chapter

- copyright and a licence: two different things
- why public code is not automatically reusable
- the common software licences, compared by their conditions
- Creative Commons licences for text
- how to state a licence in a repository

This book is not legal advice. It explains how licences are described and what their texts say. Copyright law differs between countries, and a licence's effect on a particular situation is a legal question that this book does not answer. If a decision matters (a company, a product, a dispute), ask a qualified person.

How this chapter was checked. GitHub statements come from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026). Licence summaries come from the choosealicense.com data maintained by GitHub and from the SPDX licence list, version 3.29.0, both read from their public repositories. The Creative Commons wording is from the SPDX copy of the legal code. **The official licence pages (gnu.org, apache.org, mozilla.org, opensource.org, creativecommons.org) could not be read from the test computer**, so the summaries are secondary sources and are marked so. Nothing here was run on a live account.

Before you start. Chapter [64](#).

65.1 Two different things

Copyright is a legal right that a creator has over a creative work, such as a program or a book, so that others may not copy or change it without permission. **A licence** is the permission: a document in which the rights holder says what others may do, under which conditions.

GitHub's documentation puts the consequence plainly: "You're under no obligation to choose a license. However, without a license, the default copyright laws apply, meaning that you retain all rights to your source code and no one may reproduce, distribute, or create derivative works from your work."

So a repository with no licence file is **not** free to reuse. It is only visible.

What GitHub itself allows. The same page adds a note: if you publish source code in a public repository, "according to the Terms of Service, other users of GitHub have the right to view and fork your repository." That is permission to view and fork **on GitHub**. It says nothing about copying the code into your own product.

Checked against GitHub's documentation (R317). "Licensing a repository" and "Adding a license to a repository".

65.2 The seven distinctions

Chapter 64 said that seeing is not permission. Keep these apart every time you read a project:

1. **Ownership:** who owns the work?
2. **Reading:** may I read it?
3. **Downloading:** may I download a copy?
4. **Redistribution:** may I pass it on to others?
5. **Modification:** may I change it?
6. **Commercial use:** may I use it to make money?
7. **Code reuse:** may I put its code into my own project, and who owns my project afterwards?

Each answer is separate: being allowed to read says nothing about copying, and being allowed to copy says nothing about selling. A licence answers these questions in writing. If the answer is not written anywhere, do not assume "yes".

65.3 The common software licences

The licences below are described from the choosealicense.com data (secondary source), which lists for each licence its **permissions**, **conditions** and **limitations**. All of them permit commercial use, modification, distribution and private use; they differ in their conditions. SPDX gives each an identifier.

| Licence | SPDX identifier | Kind | Conditions listed |
|--|-----------------|--------------------|---|
| MIT License | MIT | permissive | include the copyright and licence notice |
| BSD 2-Clause "Simplified" | BSD-2-Clause | permissive | include the copyright and licence notice |
| BSD 3-Clause "New" or "Revised" | BSD-3-Clause | permissive | as BSD 2-Clause, plus no use of the holder's name to promote derived products without permission |
| Apache License 2.0 | Apache-2.0 | permissive | include notices; document changes |
| Mozilla Public License 2.0 | MPL-2.0 | weak copyleft | make source of the licensed files and changes to them available under the same licence; keep notices |
| GNU Lesser General Public License v3.0 | LGPL-3.0-only | copyleft (library) | source of the licensed work and modifications under the same licence or the GNU GPLv3; keep notices; document changes |
| GNU General Public License v3.0 | GPL-3.0-only | strong copyleft | complete source of the work and modifications, including larger works that use it , under the same licence; keep notices; document changes |

Three ideas help to read the table:

- **Permissive** licences ask little: keep the notice. Code under them can end up inside closed products.
- **Copyleft** licences require that what you build from the code is shared under the same licence when you distribute it. "Strong" reaches the larger work; "weak" reaches only the licensed files.
- **Patents.** The choosealicense data lists patent-use among the permissions of Apache-2.0, MPL-2.0, LGPL-3.0 and GPL-3.0, and does not list it for MIT and BSD. What that means in law is a question for a lawyer.

All these licences also list **limitations**: they give the software "as is", with no **warranty**, and no **liability** for the author.

A detail about names. SPDX's list marks the identifier GPL-3.0 as deprecated; the current identifiers are GPL-3.0-only and GPL-3.0-or-later. Use the current one.

What a licence file looks like. GitHub's documentation: "Most people place their license text in a file named LICENSE.txt (or LICENSE.md or LICENSE.rst) in the root of the repository." A detectable licence is shown at the top of the repository page, and "As a best practice, we encourage you to include the license file with your project." The MIT licence's own instructions (from choosealicense) say to replace the year and the name with those of the copyright holders: **use your own name**, and copy the text from an official source, never from memory.

Checked against sources (R318, R319). choosealicense.com data (GitHub-maintained; a secondary source) and the SPDX licence list 3.29.0 (identifiers, deprecation). **Not established:** the exact wording of each official licence page, and how any of them applies to your situation.

65.4 Creative Commons: licences for text and pictures

Software licences are written for code. For books, articles, pictures and teaching materials, **Creative Commons** offers a family of licences built from four **elements**. The SPDX copy of the legal code states: "License Elements means the license attributes listed in the name of a Creative Commons Public License". The elements you will see:

| Element | Short | Meaning in one line |
|---------------|-------|---|
| Attribution | BY | credit the creator as the licence says |
| ShareAlike | SA | share adaptations under the same (or a compatible) licence |
| NonCommercial | NC | no use "primarily intended for or directed towards commercial advantage or monetary compensation" |
| NoDerivatives | ND | no adapted versions (not covered in the text read for this chapter) |

The SPDX identifiers are, for example, CC-BY-4.0, CC-BY-SA-4.0, CC-BY-NC-SA-4.0 and CC0-1.0 (the last one is a dedication with no conditions). The definition of NonCommercial quoted above is from the legal code, which adds that exchanging material "by digital file-sharing or similar means is NonCommercial provided there is no payment of monetary compensation in connection with the exchange." How that applies to your use is a legal question.

Two cautions from the sources:

- The list of licences GitHub can detect includes the Creative Commons family, and the SPDX list does not mark the Creative Commons licences as approved by the Open Source Initiative. Whether a NonCommercial licence counts as "open source" depends on the definition used, and Chapter 64's three-way split (public, source-available, open source) shows why the words matter.
- Creative Commons is said to advise against using its licences for software (the source read for this chapter is only the choosealicense summary, so treat this as unconfirmed). Use a software licence for code and a Creative Commons licence for text.

This book's own licence. A book has text and code samples, which may need different licences, and this book uses two. Its original text is licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International, and its code examples under the MIT License; the copyright page states both, with the address of the official text of the Creative Commons licence. The book is still copyrighted: the licences grant permissions, they do not remove the copyright. They cover only what the rights holder may license, so quotations and other people's material keep their own terms. Use the copyright page and the official licence texts, not this chapter, to know what you may do.

Checked against sources (R320). SPDX's copy of the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International legal code (definitions of License Elements and NonCommercial; ShareAlike condition), and the SPDX list. The official Creative Commons pages were not readable.

65.5 Using other people's code and text

A checklist for someone who wants to reuse something:

1. **Find the licence** (a LICENSE file, the README, the repository page).
2. **No licence? Ask** the owner. Do not copy.
3. **Read the conditions** in the table above and in the file itself.
4. **Comply:** keep the required notices; if the licence is copyleft, understand what you must share.
5. **Record** where the code came from and under what licence, in your repository (a NOTICE or THIRD-PARTY file is a common habit, not a requirement).
6. **Check compatibility** if you combine code under different licences: some combinations are not allowed. This is a legal question; ask.
7. **Do not remove** copyright notices.

A checklist for someone who wants to **share** their own work:

1. Decide what you want others to be able to do, and what you must have the right to license (work made for an employer, or with contributors, may not be yours alone; Chapter 64 on sign-off).
 2. Choose a licence from a maintained list, using current official texts.
 3. Add the licence file at the root, and state the licence in the README.
 4. Ask contributors to agree that their contributions come under the same terms; many projects say so in CONTRIBUTING.md.
-

Checkpoint

What You Learned

- Copyright gives the creator rights; a licence is the written permission. Without a licence, all rights are reserved.
- Public and viewable is not reusable; GitHub's terms allow viewing and forking on GitHub only.
- Permissive licences ask little; copyleft licences require sharing under the same licence; strong and weak copyleft differ in reach.
- SPDX identifiers name licences; prefer the current identifiers.
- Creative Commons licences suit text; they are built from elements such as BY, SA and NC.
- This book is not legal advice; its own licences (CC BY-NC-SA 4.0 for text, MIT for code) are on its copyright page.

New Vocabulary

Copyright, licence, permissive licence, copyleft, SPDX identifier, Creative Commons (see the glossary).

Commands Learned

No new commands.

Common Mistakes

1. **Treating a public repository as free to reuse.**
2. **Copying a licence text from memory or from a random site.**
3. **Putting someone else's name (or none) in the copyright line.**
4. **Using a text licence for software or the reverse.**
5. **Assuming NonCommercial is obvious.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between copyright and a licence?
2. What does GitHub's documentation say applies when a repository has no licence?
3. Name one permissive and one copyleft licence and one difference in their conditions.
4. What does the SA element mean?
5. Why is this chapter not legal advice?

Before Moving On

You are ready for Chapter 66 if you can:

- [] explain why a public repository may not be reused
- [] find a project's licence
- [] tell permissive from copyleft

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|--|--------|
| No licence means default copyright applies; GitHub's terms allow view and fork; licence file placement | Checked against github/docs (commit 2eaab0b); not run on a live account | R317 |
| Permissions, conditions and limitations of the listed licences | Secondary source: choosealicense.com data (GitHub-maintained); official texts not read | R318 |
| SPDX identifiers and the deprecation of GPL-3.0 | SPDX licence list 3.29.0, fetched from its public repository | R319 |
| Creative Commons elements, NonCommercial and ShareAlike wording | SPDX copy of the CC BY-NC-SA 4.0 legal code; official pages not read | R320 |
| Whether Creative Commons advises against software use | Unconfirmed (secondary source only) | R099 |

Where this leads

Chapter 66 turns a state of the project into a named, downloadable version.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises are on paper. **None asks you to give legal advice.**

Level 1 — Guided

1.1 Two words. Explain copyright and licence in one sentence each.

1.2 No licence. A repository has code but no licence file. According to GitHub's documentation, what applies?

Level 2 — Partially guided

2.1 Read a table. Using the table in section 65.3, say which of MIT, Apache-2.0 and GPL-3.0-only requires source of a larger work that uses the code to be shared under the same licence.

2.2 The seven questions. For a project you know, answer the seven distinctions with "stated", "not stated" or "unclear", after reading its licence.

Level 3 — Independent

3.1 Decode. Explain CC-BY-NC-SA-4.0 element by element.

3.2 A name. Why should you use GPL-3.0-only rather than GPL-3.0?

Level 4 — Professional scenario

4.1 A reuse request. A colleague wants to copy a function from a public repository that has no licence into a company product. Write a reply based on this chapter, without claiming legal certainty.

Level 5 — Troubleshooting

5.1 You copied code from a project whose licence requires keeping the copyright notice, and you removed it. What is the first thing you do?

5.2 Two libraries you use have different licences and you are unsure they can be combined. Where do you look, and whom do you ask?

Chapter 66 — Releases [Core]

In this chapter

- what a release is, and how it differs from a tag
- version numbers and Semantic Versioning
- release notes, by hand and generated
- source archives and attached files
- automating a release

How to read this chapter. GitHub statements come from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026); they were **not run on a live account**. The Git side (tags, sorting, notes from history, source archives) and the structure of the release-notes configuration file were recorded on Git 2.43.0 in Bash and zsh, and re-run in CI on newer Git. The rules of Semantic Versioning are from its specification, read from its public repository.

Before you start. Chapter [29](#) (tags) and Chapter [39](#) (repositories on GitHub).

66.1 A tag is a name; a release is an announcement

Chapter [29](#) taught that a tag names one commit. GitHub's documentation: "Releases are deployable software iterations you can package and make available for a wider audience to download and use. Releases are based on Git tags, which mark a specific point in your repository's history. A tag date may be different than a release date since they can be created at different times."

So a **release** is a page on GitHub built around a tag, with a title, notes and downloadable files. Anyone with read access can view and compare releases, but "only people with write permissions to a repository can manage releases". Releases do **not** exist in Git itself: a plain git clone gets the tags but not the release page, the notes or the attached files (Chapter [36](#)).

GitHub automatically provides "links to download a zip file and a tarball containing the contents of the repository at the point of the tag's creation". Your own files (a compiled program, a package) can be attached to a release; the documentation gives limits on the number and size of such **assets**, which change, so read them when you need them.

Checked against GitHub's documentation (R321). "About releases" and "Managing releases in a repository".

66.2 Version numbers

A release needs a name people can compare. The most common scheme is **Semantic Versioning**. Its specification (version 2.0.0) says: "Given a version number MAJOR.MINOR.PATCH, increment the:

1. MAJOR version when you make incompatible API changes
2. MINOR version when you add functionality in a backward compatible manner
3. PATCH version when you make backward compatible bug fixes".

It adds that "Once a versioned package has been released, the contents of that version **MUST** NOT be modified. Any modifications **MUST** be released as a new version", and that "Major version zero (0.y.z) is for initial development. Anything **MAY** change at any time."

Semantic Versioning is about a **public API**, the promises your project makes to the people using it. For a program with no API, or for a book, projects choose other schemes (dates, for example); no scheme is required. Whatever you choose, **write it down and keep to it**.

A Git habit that goes with it: name the tag `v` plus the version (`v1.2.0`). The recording below made four tags. Notice that plain `git tag` lists them alphabetically, which puts `v1.10.0` before `v1.2.0`; the sort option `version:refname` compares the numbers the way a person does:

```
$ git init -q proj && cd proj
$ printf 'app\n' > app.txt && git add app.txt && git commit -q -m "First working version"
$ git tag -a v1.0.0 -m "Version 1.0.0"
$ printf 'fix\n' >> app.txt && git commit -q -am "Fix crash on empty input"
$ git tag -a v1.0.1 -m "Version 1.0.1"
$ printf 'search\n' >> app.txt && git commit -q -am "Add search"
$ git tag -a v1.2.0 -m "Version 1.2.0"
$ printf 'export\n' >> app.txt && git commit -q -am "Add export"
$ git tag -a v1.10.0 -m "Version 1.10.0"
$ git tag
v1.0.0
v1.0.1
v1.10.0
v1.2.0
$ git tag --sort=-version:refname
v1.10.0
v1.2.0
v1.0.1
v1.0.0
```

Recorded in Bash; *ch66-releases/expected-relnotes.bash.txt*.

The `-a` made **annotated** tags (Chapter 29), which carry a message and are the natural choice for releases.

66.3 Release notes

Release notes tell users what changed. Two sources:

- **By hand.** Write a description when you create the release. If you @mention people in it, "the published release will include a **Contributors** section with an avatar list of all the mentioned users".
- **Generated.** "Automatically generated release notes include a list of merged pull requests, a list of contributors to the release, and a link to a full changelog." You can click **Generate release notes** while creating a release, and you can customise the result with a file `.github/release.yml` in which you "specify in YAML the pull request labels and authors you want to exclude" and can "create new categories and list the pull request labels to be included in each of them". The documented options include `changelog.exclude.labels`, `changelog.exclude.authors` and, per category, a required title and required labels, where `*` is "a catch-all for pull requests that didn't match any of the previous categories".

Git alone can give a rough draft, from the history between two tags:

```
$ git log --format='- %s' v1.0.1..v1.10.0
- Add export
- Add search
```

Recorded in Bash; *ch66-releases/expected-relnotes.bash.txt*.

That is the raw material: the subjects of the commits since the last release. It is only as good as your commit messages (Chapter 19).

A minimal configuration file, written and read back with PyYAML (Chapter 53):

```
$ mkdir -p .github
$ printf 'changelog:\n  exclude:\n    labels:\n      - ignore-for-release\n  categories:\n    - title: Breaking changes\n      labels:\n        - breaking-change\n    - title: Other changes\n      labels:\n        - "*" \n' > .github/release.yml
$ python3 ../rel.py .github/release.yml
excluded labels: ['ignore-for-release']
Breaking changes <- ['breaking-change']
Other changes <- ['*']
```

Recorded in Bash; *ch66-releases/expected-relnotes.bash.txt*.

The script only checks that the structure matches the option names in the documentation; it does **not** show what GitHub would generate.

Checked against GitHub's documentation (R322). "Automatically generated release notes" (option names and behaviour). The recording checks the file's structure only.

66.4 Source archives

The zip and tarball that GitHub adds to a release are **source archives**: the files of the tagged commit, without the `.git` folder. Git can make the same kind of archive locally (Chapter 30 listed the files of an archive):

```
$ git describe --tags
v1.10.0
$ git archive --format=tar --prefix=proj-1.10.0/ v1.10.0 | tar -t
proj-1.10.0/
proj-1.10.0/app.txt
$ git archive --format=tar.gz -o ../proj-1.10.0.tar.gz v1.10.0 && gzip -t ../
proj-1.10.0.tar.gz && echo "archive ok"
archive ok
```

Recorded in Bash; *ch66-releases/expected-relnotes.bash.txt*.

`git describe --tags` names the current commit by the nearest tag, here exactly `v1.10.0`. The archive listing shows the files under a folder that carries the project and version (`--prefix`), which is a common habit so that unpacking does not scatter files. The `gzip -t` test only checks that the compressed file is not damaged.

Whether the archives that GitHub makes include Git LFS files is an option that administrators can choose, according to the documentation. What is **not** in a source archive: the history, and any files you attach separately.

Chapter 63 covered checksums for what you publish. For anything people will run, publish a checksum, and consider attestations.

66.5 A release checklist

1. **Everything you want is merged** into the default branch, and CI is green (Chapter 52).
2. **Choose the version** by your scheme.
3. **Create the tag** on the right commit (Chapter 29). A tag on the wrong commit is a release of the wrong code.
4. **Write the notes**: by hand, generated or both. Mention breaking changes first.
5. **Attach your files** and publish a checksum for each.
6. **Mark pre-releases** as such when they are not final. The GitHub command-line tool (Chapter 73) can do this: the documentation's example is `gh release create v1.3.2 --title "v1.3.2 (beta)" --notes "... " --prerelease`.
7. **Check the result** as a user would: download and try it.
8. **If it fixes a security problem**, publish a security advisory as well (Chapter 62); the documentation recommends it.

Editing and immutability. You can edit a release later. The documentation describes an option, **immutable releases**: if enabled, you "cannot add, replace, or delete assets after a release is published", and you "cannot move or delete its tag while the release exists"; titles and notes can still be edited. It recommends creating releases as **drafts** first, attaching all files, and then publishing. Semantic Versioning says the same in spirit: a released version must not change.

66.6 Automating releases

A workflow (Chapter 54) can create a release when a tag is pushed: the trigger is a push of a tag, and a step calls the GitHub command-line tool or the API. The Releases API also exists for scripts. Two cautions:

- The workflow needs permission to write releases, so give it exactly that and no more (Chapter 60). The precise permission name was **not** checked for this chapter; look it up in the current documentation.
- Automation makes mistakes fast: a wrong tag becomes a wrong public release in seconds. Keep a human check, such as a draft release that a person publishes.

Chapter 67 puts the whole lifecycle together in a project of your own.

Checkpoint

What You Learned

- A release is a GitHub page built on a tag; Git itself has tags, not releases.
- Semantic Versioning: MAJOR for incompatible changes, MINOR for compatible features, PATCH for compatible fixes; a released version does not change.
- `git tag --sort=version:refname` sorts versions correctly.
- Release notes can be written by hand or generated, and configured by `.github/release.yml`.
- Source archives contain the files of a tag without history; `git archive` makes one locally.
- Publish checksums for what people run; consider immutable releases and drafts.

New Vocabulary

Release, release notes, Semantic Versioning, asset, source archive, pre-release (see the glossary).

Commands Learned

`git tag --sort=version:refname`, `git describe --tags`, `git archive --prefix`, `git log --format=...tagA..tagB`.

Common Mistakes

1. **Tagging the wrong commit.**
2. **Changing a published release's files.**
3. **Sorting tags alphabetically and thinking v1.10.0 is older than v1.2.0.**
4. **Publishing a program without a checksum.**
5. **Fully automating a release with no human check.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between a tag and a release?
2. What does each part of MAJOR.MINOR.PATCH mean?
3. Why does `git tag list` list `v1.10.0` before `v1.2.0`, and what fixes it?
4. What is in a source archive, and what is not?
5. What does an immutable release forbid?

Before Moving On

You are ready for Chapter 67 if you can:

- [] choose the next version number after a change
- [] draft release notes from history
- [] make a source archive with Git

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|-------------------------|
| Tag sorting, notes from history, <code>git describe</code> , <code>git archive</code> , <code>release.yml</code> structure | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0, PyYAML 6.0.1; CI on newer Git | R322, R323 (local side) |
| Releases are based on tags; permissions; source archives; generated notes; immutability; asset limits exist | Checked against <code>github/docs</code> (commit <code>2eaab0b</code>); not run on a live account | R321, R322 |
| Semantic Versioning rules | Read from the specification in its public repository (version 2.0.0) | R323 |
| The permission name for a release-creating workflow | Not checked | none |

Where this leads

Chapter 67 builds a small open-source library with everything from this Part: layout, documentation, licence, versioning and releases.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 to 2.2 use only your terminal.

Level 1 — Guided

1.1 Tag or release. State two differences between a Git tag and a GitHub release.

1.2 Next version. The current version is 1.4.2. Give the next version after (a) a bug fix, (b) a new backward-compatible feature, (c) a change that breaks existing users.

Level 2 — Partially guided

2.1 Sorting. In a sandbox repository, make annotated tags v1.2.0 and v1.10.0. Run `git tag` and `git tag --sort=-version:refname`. Which output puts the newest first?

2.2 An archive. Make a tar.gz source archive of a tag with a prefix folder, list what is inside and test it with `gzip -t`.

Level 3 — Independent

3.1 A draft of notes. For your sandbox, write release notes from the commit subjects between two tags, using `git log --format`.

3.2 A config file. Write a `.github/release.yml` with one category for the label bug and one catch-all category, and read it back.

Level 4 — Professional scenario

4.1 A wrong tag. You published a release and then notice the tag points to a commit that is missing the last fix. Explain what you would do, given that a released version must not change. Consider what an immutable release would forbid.

Level 5 — Troubleshooting

5.1 After `git clone`, your colleague cannot see the release notes or the attached files. Why, and where are they?

5.2 `git tag` lists v1.10.0 before v1.2.0. Is something broken?

Chapter 67 — Creating an Open-Source Project [Deep]

In this chapter

- **Project 11:** create a small, documented open-source library
- project structure, documentation and community files
- choosing and stating a licence (your decision, not this book's)
- a first release and a maintenance routine

How to read this chapter. This is a **Deep** chapter and can be read later. Everything about Git and the local layout was recorded in Bash and zsh (Git 2.43.0, Python 3) and re-run in CI. Statements about GitHub features refer to earlier chapters, which cite GitHub's documentation. **Nothing on GitHub was run for this chapter.** Chapter 65 said that this book is not legal advice; **the project you create needs a licence chosen by you**, and this chapter deliberately contains no licence text.

Before you start. Chapter 65, Chapter 42 and Chapter 66.

67.1 What "open-source project" asks of you

An open-source project is more than public code (Chapter 64). A visitor should be able to answer five questions in a minute:

1. **What is it?** (the README)
2. **May I use it?** (the licence)
3. **How do I try it?** (install and usage in the README)
4. **How do I help?** (CONTRIBUTING.md, code of conduct)
5. **What if I find a security problem?** (SECURITY.md)

GitHub's community profile checklist (Chapter 64) looks for the same files.

67.2 Project 11: a small library

Project 11. Goal: publish a tiny library with tests, documentation, a licence you chose, community files and a first tagged release. Time: about 90 minutes. You need: Git, Python 3 (any current version; the example uses only its standard library), a GitHub account for the last steps.

The idea is deliberately small so that the project structure is the lesson. This chapter uses wordcount, a function that counts the words in a text. You may replace it with a small idea of your own; keep the same steps.

1. **Create the repository and folders.**

2. **Write the code and two tests**, and run them.
3. **Add a .gitignore**. Generated files must not enter the repository.
4. **Write the README** with a usage example that is correct.
5. **Add the community files**. Then check which are missing.
6. **Choose and add a licence**. See section 67.4.
7. **Commit and tag** the first version.
8. **Push, add a workflow, and create the release** (section 67.5).
9. **Check the result as a stranger would**.

The recording covers steps 1 to 5 and 7:

```
$ git init -q wordcount && cd wordcount
$ mkdir -p src tests .github
$ printf 'def count_words(text):\n    """Return the number of words in text."""\n    return\n    len(text.split())\n' > src/wordcount.py
$ printf 'import unittest\nfrom wordcount import count_words\n\nclass\nCountWordsTest(unittest.TestCase):\n    def test_empty(self):\n\n    self.assertEqual(count_words(""), 0)\n\n    def test_sentence(self):\n\n    self.assertEqual(count_words("one two three"), 3)\n\n\nif __name__ == "__main__":\n    unittest.main()\n' > tests/test_wordcount.py
$ PYTHONPATH=src python3 -m unittest discover -s tests 2>&1 | grep -x OK
OK
```

Recorded in Bash; *ch67-ossproject/expected-library.bash.txt*.

The tests ran and printed OK (only that line is kept, because the full report contains the running time, which changes on every run). The code is tiny on purpose: `text.split()` splits on any run of spaces, so "one two three" has three words.

Now the ignore file, the README and the community files:

```
$ printf '__pycache__/\n' > .gitignore
$ printf '# wordcount\n\nCounts the words in a piece of text.\n\n    from wordcount import\n    count_words\n    count_words("one two three") # 3\n' > README.md
$ printf '# Contributing\n\nOpen an issue first. Run the tests before you send a pull\nrequest.\n' > CONTRIBUTING.md
$ printf '# Security policy\n\nReport vulnerabilities privately to security@example.org.\n'\n' > SECURITY.md
```

Recorded in Bash; *ch67-ossproject/expected-library.bash.txt*.

Next, a small check script in the shell. It only tests whether each file is present, and it reports **honestly** that two are missing:

```
$ for f in README.md LICENSE CONTRIBUTING.md CODE_OF_CONDUCT.md SECURITY.md; do test -f\n"$f" && echo "present: $f" || echo "MISSING: $f"; done
present: README.md
MISSING: LICENSE
present: CONTRIBUTING.md
MISSING: CODE_OF_CONDUCT.md
present: SECURITY.md
```

Recorded in Bash; *ch67-ossproject/expected-library.bash.txt*.

LICENSE and CODE_OF_CONDUCT.md are missing on purpose. **The recording does not create them, because a licence is your decision (Chapter 65) and a code of conduct is a promise you must be able to keep (Chapter 64).** The check is a habit: run something like it before every release.

Finally, the first commit, a tag, and the list of tracked files:

```
$ git add -A && git commit -q -m "Add wordcount library, tests and community files"
$ git tag -a v0.1.0 -m "Version 0.1.0: initial development"
$ git tag -n1
v0.1.0          Version 0.1.0: initial development
$ git ls-files
.gitignore
CONTRIBUTING.md
README.md
SECURITY.md
src/wordcount.py
tests/test_wordcount.py
```

Recorded in Bash; *ch67-ossproject/expected-library.bash.txt*.

Two details deserve attention:

- `git ls-files` shows **no** `__pycache__` folder, because the `.gitignore` line kept it out. Without it, Python's generated files would be committed, and they depend on the Python version (a lesson from an earlier draft of this recording, in which the file name contained `cpython-311`).
- The tag is **annotated** and named `v0.1.0`: by Semantic Versioning (Chapter 66), major version zero "is for initial development" and its API "SHOULD NOT be considered stable". Starting at 0.1.0 is an honest promise.

67.3 Documentation that a stranger can follow

The README example must be **true**. The best habit is to make the README example a test, or to run it by hand before each release. A README that says `count_words("one two three")` returns 3 is a promise, and the recording's second test proves the same sentence works.

Documentation checklist for a first release:

- The **name** and one-sentence purpose.
 - How to **install** (for this tiny library: copy `src/wordcount.py`, or the way your language's package manager works; do not claim an installation method you have not tried).
 - One **usage example**.
 - How to **run the tests**.
 - The **licence** and how to **contribute** (links).
 - The **status** ("early development; the API may change").
-

67.4 The licence: your decision

Chapter 65 explained the options. For this project:

1. **Decide** what you want others to be able to do. If you are unsure, read the chapter again and ask someone qualified: a wrong choice is hard to undo, because people who received your code under a licence keep the rights it gave them.
2. **Get the text from an official source**, such as the licence's own page, or GitHub's licence template tool (Chapter 64, "Adding a license to a repository": create a file named LICENSE and choose a template). Never type a licence from memory.
3. **Put your own name** in the copyright line if the template asks for one, and the correct year.
4. **Say it in the README** in one line, with a link to the file.
5. **Re-run the check script**: it should now print present: LICENSE.

No licence is a licence decision too: GitHub's documentation says that without a licence "the default copyright laws apply". People will not be allowed to reuse your library.

For the code of conduct, GitHub's documentation advises: "Consider carefully whether you are willing and able to enforce it." If you choose one from a template, follow its attribution guidelines.

67.5 First release

After the recording, on your computer and GitHub:

1. Create the repository on GitHub without any files (Chapter 39).
 2. Add the remote and push the branch **and** the tag: `git push -u origin main` and `git push origin v0.1.0`.
 3. Add a workflow that runs the tests on every push and pull request (Chapter 56), and pin any action to a commit (Chapter 60).
 4. Create a release from the tag, with notes (Chapter 66).
 5. Open the repository in a private browsing window, as a stranger would. Can you find the licence? Can you run the example?
-

67.6 Maintenance

A published project is a small commitment:

- **Triage issues** (Chapter 44): labels such as `good first issue` help newcomers.
 - **Review pull requests** with kindness and a clear rule for what you accept (Chapter 46).
 - **Turn on the security features** you can (Chapter 62), and keep dependencies current.
 - **Release regularly**, with a version number that follows your scheme.
 - **Say what you will not do**. A README line such as "no feature requests for X" saves everyone's time.
 - **Be honest about your capacity**. If you can no longer maintain it, say so in the README. GitHub has settings for closing down a repository; read the current documentation before using them.
-

Checkpoint

What You Learned

- A project should answer five questions quickly: what, may I, how, how to help, security.
- A tiny library with tests, a .gitignore, a README and community files is a complete start.
- A check script can tell you which community files are missing.
- The licence is your decision, made from official texts, and is not part of this book.
- A first release is an annotated v0.1.0 tag, honest about being early.

New Vocabulary

No new terms.

Commands Learned

```
python3 -m unittest discover -s tests, test -f, git tag -n1, git ls-files.
```

Common Mistakes

1. **Publishing without a licence and expecting reuse.**
2. **Committing generated files such as `__pycache__`.**
3. **A README example that no longer works.**
4. **Adopting a code of conduct you will not enforce.**
5. **Claiming an installation method you have not tried.**

Practice

Do the exercises in the Exercises section at the end of this chapter. Project 11 is in section 67.2.

Self-Test

1. Which five questions should a project answer quickly?
2. Why does the recording end with a missing LICENSE?
3. Why is `__pycache__` ignored?
4. Why start at version 0.1.0?
5. What must you do before choosing a code of conduct?

Before Moving On

You are ready for Chapter [68](#) if you can:

- [] lay out a small project with tests and community files
- [] explain why the licence is your decision
- [] tag a first release

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|---|-------------------|
| The library, tests, ignore file, community files, check script, tag and tracked-file list | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0, Python 3.11; CI on newer Git and its Python | R324 (local side) |
| Statements about GitHub features | Cited in Chapters 64, 65 and 66; not re-checked or run here | R314-R323 |
| Expected outcome of Project 11 on GitHub | Not observed; written from the earlier chapters | none |

Where this leads

Part X, "Advanced GitHub", starts with Chapter 68: organizations, teams and enterprise concepts.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Project 11 is in section 67.2 of the chapter. **These exercises do not ask you to choose a licence for anyone else.**

Level 1 — Guided

1.1 Five questions. List the five questions a project should answer quickly and the file that answers each.

1.2 Missing files. In the chapter's recording, which two files were reported missing, and why were they left out?

Level 2 — Partially guided

2.1 Run it. Create the wordcount project in a sandbox folder up to the first tag. Show `git tag -n1` and `git ls-files`.

2.2 A third test. Add a test for a text with a leading and a trailing space. Does the function pass? Run the tests and show the result.

Level 3 — Independent

3.1 Your own idea. Replace wordcount with a small idea of your own (for example, a function that reverses the words of a sentence). Follow the same steps and write down what you changed.

3.2 A check script. Write a shell loop that reports whether `.gitignore`, `README.md` and tests exist.

Level 4 — Professional scenario

4.1 A stranger's five minutes. Imagine a stranger opens your repository. Write what they see in the first minute, and improve three things.

Level 5 — Troubleshooting

5.1 Generated `__pycache__` folders appear in `git status` in your project. What do you add, and what do you do if they were already committed?

5.2 A user asks whether they may use your library in a company product, and you have no licence. What do you tell them, and what should you do about it?

Part X — Advanced GitHub

Chapter 68 — Organizations and Enterprise Concepts [Deep]

In this chapter

- the three kinds of account: user, organization, enterprise
- members, owners and teams
- repository roles and base permissions in an organization
- plans, and the difference between Enterprise Cloud and Enterprise Server

How to read this chapter. This is a **Deep** chapter and can be read later; you can use GitHub for years without an organization. Everything here comes from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026) and **was not run on a live account**. Organization settings screens are not described. **Prices, included quantities and plan contents are not stated in this book, because they change;** read the current plans page for your account.

Before you start. Chapter [49](#) (repository roles and collaborators).

68.1 Three kinds of account

GitHub's documentation lists "three types of accounts": **user accounts**, **organization accounts** and **enterprise accounts**.

- **User account.** "Every person who uses GitHub signs in to a user account." It can own repositories, packages and projects, and actions on GitHub are attributed to it. There are two sorts of user account: **personal accounts** (you signed up yourself) and accounts created for you by an enterprise, with some settings managed by the enterprise. Accounts "are intended for humans, but you can create accounts to automate activity", which the documentation calls a **machine user**.
- **Organization account.** "Organizations are shared accounts where a large number of people can collaborate across many projects at once." Like a user account, an organization can own repositories, packages and projects. **"You cannot sign in to an organization."** Each person signs in to their own user account, and the actions they take on organization resources are attributed to that user account. A user can belong to several organizations.
- **Enterprise account.** It "allows central management of multiple organizations" for policy and billing. Unlike organizations, enterprise accounts "cannot directly own resources like repositories, packages, or projects": the organizations inside the enterprise own them.

Chapter [37](#) was about the first kind. This chapter is about the second and the third.

Checked against GitHub's documentation (R325). "Types of GitHub accounts" and "About organizations".

68.2 Why an organization

A personal account is the right place for your own projects. An organization fits when a **group** shares ownership: a team, a company, a club, a community project. Reasons the documentation gives:

- **Roles.** "You can invite people to join your organization, then give these organization members a variety of roles that grant different levels of access to the organization and its data." Only organization owners and security managers can manage the organization's settings; all members can collaborate in repositories and projects.
- **Teams.** "You can create nested teams that reflect your group's structure, with cascading access permissions and mentions."
- **Settings and policy.** For example "restricting the types of repositories that members can create".
- **Security.** "You can enforce security requirements and review the organization's audit log."
- **Continuity.** Repositories owned by an organization do not belong to one person's account. The documentation's first best practice is to **assign multiple owners**, so that the organization does not depend on one person; a repository under one personal account depends on that one person.

Further best practices in the documentation: use teams; keep teams visible "whenever possible and reserving secret teams for sensitive situations".

A note on size. The documentation warns that "If an organization exceeds 100,000 members, some UI experiences and API functionality may be degraded."

68.3 Repository roles in an organization

Chapter 49 introduced repository roles. In an organization, the documentation lists them "from least access to most access":

| Role | Recommended for |
|----------|---|
| Read | non-code contributors who want to view or discuss the project |
| Triage | contributors who need to manage issues and pull requests without write access |
| Write | contributors who actively push to the project |
| Maintain | project managers who need to manage the repository without access to sensitive or destructive actions |
| Admin | people who need full access, including sensitive and destructive actions such as managing security or deleting a repository |

The roles can be given to organization members, to **outside collaborators** (people who are not members but are given access to some repositories) and to teams. The documentation's rule is the one from Chapter 60: choose the role that fits "without giving people more access to the project than they need". Organization owners "have admin access to every repository owned by the organization".

Base permissions. Owners can set a level that applies to **all members** for **all** the organization's repositories. It does not apply to outside collaborators. By default, "members of an organization will have Read permissions to the organization's public repositories". If someone with admin access to a repository grants a member a higher level, "the higher level of access overrides the base permission". Changes affect new and existing members. Internal repositories have "a minimum visibility level of read, even if the base permission has been set to none".

A warning worth repeating. "When someone adds a deploy key to a repository, any user who has the private key can read from or write to the repository (depending on the key settings), even if they're later removed from the organization."

Checked against GitHub's documentation (R326). "Repository roles for an organization" and "Setting base permissions for an organization". Custom repository roles exist for some plans and were not examined.

68.4 Plans, and Enterprise Cloud versus Server

GitHub "offers free and paid plans"; some plans "are available only to personal accounts, while other plans are available only to organization and enterprise accounts". Plans differ in features for private repositories, included Actions minutes and storage, support and controls such as required reviewers and protected branches in private repositories. **This book does not list quantities or prices.** Two facts help to read the plans page:

- **Public repositories** have a full feature set on the free plans; the limits mostly concern **private** repositories.
- The enterprise product "includes two deployment options": **Enterprise Cloud**, "hosted by GitHub in the cloud", and **Enterprise Server**, which is "self-hosted". The documentation you read has versions for each; the documentation's own advice is to select the version that reflects your plan.

Enterprise features named in the documentation include central management of policy and billing for many organizations, enterprise-managed user accounts, and additional controls such as audit-log streaming and an IP allow list. Their details belong to an administrator's course, not to this book.

Checked against GitHub's documentation (R327). "GitHub's plans". Specific figures were deliberately left out.

68.5 Starting an organization

The documentation has a page called "Creating a new organization from scratch"; its steps and screens were **not read** for this book, so follow the current page. Decisions to make first, on paper:

1. **Name and purpose.** The name appears in every URL.

2. **Owners.** At least two, as advised.
3. **Base permission.** Start low (Read or none) and grant more through teams.
4. **Teams.** One per group of people who need the same access.
5. **Repository creation policy.** Who may create which kind of repository?
6. **Two-factor authentication** for members (Chapter 38).
7. **Security defaults** (Chapter 62).

Do the exercises on paper first; creating an organization on a live account is a step outside this chapter (it needs a person signed in to a live account, which the book's author has not yet done).

Checkpoint

What You Learned

- User accounts sign in; organizations cannot be signed in to; enterprise accounts manage many organizations.
- Organizations give roles, teams, policies and continuity.
- Repository roles: Read, Triage, Write, Maintain, Admin; base permissions apply to members, not outside collaborators.
- Assign several owners; use teams; keep access minimal.
- Plans and quantities change; Enterprise Cloud is hosted and Enterprise Server is self-hosted.

New Vocabulary

Organization, enterprise account, machine user, outside collaborator, base permission, team (see the glossary).

Commands Learned

No new commands.

Common Mistakes

1. **Keeping a team's repositories under one person's account.**
2. **A single owner.**
3. **Giving every member Admin.**
4. **Assuming base permissions apply to outside collaborators.**
5. **Trusting a plan list from a book instead of the current page.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Can you sign in to an organization?
2. What can an enterprise account not own directly?
3. Name the five repository roles in order.
4. Who does the base permission not apply to?
5. Why assign more than one owner?

Before Moving On

You are ready for Chapter 69 if you can:

- [] explain the three account types
- [] design a small team structure on paper

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|--|--------|
| Account types; organization features; best practices | Checked against github/docs (commit 2eaab0b); not run on a live account | R325 |
| Repository roles and base permissions | Same | R326 |
| Plans and enterprise deployment options | Same; quantities and prices deliberately not stated | R327 |

Where this leads

Chapter 69 publishes a static website from a repository.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. These exercises are on paper; none needs an organization.

Level 1 — Guided

1.1 Three accounts. Name the three kinds of GitHub account and say which of them can be signed in to.

1.2 Roles in order. List the five repository roles of an organization from least to most access.

Level 2 — Partially guided

2.1 Which role? Choose the least role that fits: (a) a designer who only comments on issues, (b) a developer who pushes code, (c) a lead who manages the repository but must not delete it.

2.2 Base permission. An organization sets the base permission to Read. Does that change an outside collaborator's access? Does it change a member who was granted Write on one repository?

Level 3 — Independent

3.1 A structure. Design the teams for a small company with a website team, a mobile team and a documentation group that everyone can read. Say which repository role each team gets on each repository.

3.2 Owners. Explain in three sentences why an organization needs more than one owner.

Level 4 — Professional scenario

4.1 A move. A team's repositories live under the personal account of its founder. Write a short proposal to move to an organization: the benefits from this chapter and the decisions you would make first (section 68.5).

Level 5 — Troubleshooting

5.1 A member who left still has access through a deploy key. Which warning of the chapter applies, and what should have been done about the key?

5.2 Your colleague reads a plans list in an old book and quotes a price. What do you tell them?

Chapter 69 — GitHub Pages [Deep]

In this chapter

- what GitHub Pages is, and the two kinds of site
- publishing from a branch, or with a workflow
- custom domains, DNS records and HTTPS
- limits and rules of use
- a local pre-publication check for links, mixed content and paths

How to read this chapter. This is a **Deep** chapter and can be read later. GitHub statements come from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026) and were **not run on a live account**: no site was published, and no domain was configured. The pre-publication check is a local recording (Bash and zsh, Git 2.43.0). Limits and rules change; read the current page before you rely on a number.

Before you start. Chapter 56. Chapter 58 built the workflow that deploys a Pages artifact (Project 10); this chapter explains the service itself.

69.1 What Pages is

GitHub's documentation: "GitHub Pages is a static site hosting service that takes HTML, CSS, and JavaScript files straight from a repository on GitHub, optionally runs the files through a build process, and publishes a website."

Static means the files are sent to the browser as they are (Chapter 5): there is no program running on the server that builds each page for each visitor. A site whose pages depend on a database or a login needs a different kind of hosting (Chapter 72).

There are two types of site:

| | User and organization sites | Project sites |
|-----------------|--|---|
| Source files | in a repository named <owner>.github.io, where <owner> is the account name | in a folder within the repository that holds the project's code |
| Limit | one site per account | one site per repository |
| Default address | http(s)://<owner>.github.io | http(s)://<owner>.github.io/<repositoryname> |

Note the last line for project sites: the address has a repository-name folder after the domain. That matters for links, below.

Checked against GitHub's documentation (R328). "What is GitHub Pages?".

69.2 Two ways to publish

In the repository's Pages settings you choose a **publishing source**:

1. **Deploy from a branch.** You choose a branch and, optionally, a folder (the documentation mentions `/` (root) and `/docs`). The files in it are the site. If you choose `/docs` and later remove that folder, the site "won't build" and you will get a build error.
2. **A custom GitHub Actions workflow.** Chapter 58 used it: a workflow builds the site and deploys it as an artifact. Some limits do not apply to workflow builds (see 69.4).

The documentation adds that Pages "will always be deployed with a GitHub Actions workflow run, even if you've configured your GitHub Pages site to be built using a different CI tool", and that external CI tools typically "deploy" by committing the build output to a `gh-pages` branch, "and typically include a `.nojekyll` file". A `.nojekyll` file (an empty file with that name) is the way to tell Pages that the files need no build step. You can look at the workflow run for your site in the repository's workflow runs when something fails (Chapter 57).

Pages also supports building with **Jekyll**, a static site generator, which turns Markdown pages and templates into HTML. It is a subject of its own; this book uses plain HTML.

Checked against GitHub's documentation (R329). "Configuring a publishing source for your GitHub Pages site".

69.3 A check before you publish

Pages serves what you give it. Two problems appear again and again, and both can be found on your computer before publishing.

Problem 1: mixed content. If the site is served over HTTPS but a page still loads an image, style sheet or script through `http://`, the documentation says you are "serving mixed content", which "may make your site less secure and cause trouble loading assets". Its advice: change `http://` to `https://`, and "search your site's source files for `http://`".

Problem 2: absolute paths on a project site. A link that starts with a slash, such as `/style.css`, means "from the root of the domain". On a project site at `<owner>.github.io/<repositoryname>`, the root of the domain is not your site's folder, so `/style.css` asks for a file that is not there. This follows from how addresses work (Chapter 5) and from the project-site address above. Relative links (`style.css`) keep working.

The recording makes a two-line mistake, finds it with `git grep`, fixes it, and adds `.nojekyll`:

```
$ git init -q site && cd site
$ printf '<link rel="stylesheet" href="/style.css">\n\n<a href="about.html">About</a>\n' > index.html
$ printf 'body { margin: 0; }\n' > style.css
$ git add . && git commit -q -m "First version of the site"
$ git grep -n 'http://'
index.html:2:
$ git grep -n -E '(href|src)="/'
index.html:1:<link rel="stylesheet" href="/style.css">
$ printf '<link rel="stylesheet" href="style.css">\n\n<a href="about.html">About</a>\n' > index.html
$ git grep -n 'http://' || echo "no http:// left"
no http:// left
$ git grep -n -E '(href|src)="/" || echo "no root-absolute paths left"
no root-absolute paths left
```

Recorded in Bash; *ch69-pages/expected-check.bash.txt*.

The first search found the `http://` image; the second found the root-absolute stylesheet. After the fix, both searches found nothing (`git grep` exits with a failure code when it finds nothing, so the `||` part prints the message). This check finds only what the patterns match; it does not test the site. Chapter 57 and your browser's address bar are the real test, after publishing.

```
$ : > .nojekyll
$ git add -A && git commit -q -m "Fix mixed content and paths; add .nojekyll"
$ git ls-files
.nojekyll
index.html
style.css
```

Recorded in Bash; *ch69-pages/expected-check.bash.txt*.

69.4 Limits and rules of use

Read the current page ("GitHub Pages limits"). At the time of the commit used for this book, the documentation said:

- Pages "is not intended for or allowed to be used as a free web-hosting service to run your online business, e-commerce site, or any other website that is primarily directed at either facilitating commercial transactions or providing commercial software as a service (SaaS)".
- Use is subject to the Terms of Service.
- Usage limits are listed: one user or organization site per account, a recommended size for source repositories, a maximum size for published sites, a timeout for deployments, "soft" limits on bandwidth and on builds per hour (the build limit "does not apply if you build and publish your site with a custom GitHub Actions workflow"), and rate limiting, with HTTP status 429. **The numbers are not repeated here because they change.**
- **Do not put sensitive data on a Pages site.** The documentation repeats this warning in several places. The documentation describes **privately published** sites, which only people with read access to the repository can open, as an option for organizations on Enterprise Cloud (Chapter 68). Without that option, **treat any published site as public, whatever the visibility of the repository** (Chapter 39).
- A "copy of an existing website as a learning exercise" is allowed only under conditions: you write the code yourself, the site collects no user data, and it carries "a prominent disclaimer" that it is unaffiliated and educational.

Checked against GitHub's documentation (R330). "GitHub Pages limits" and "Securing your GitHub Pages site with HTTPS" (visibility and sensitive-data warnings).

69.5 Custom domains and HTTPS

By default your site lives at `github.io`. You can use your own domain. The documentation lists the types it supports: a `www` subdomain (`www.example.com`), a custom subdomain (`blog.example.com`) and an **apex domain** (`example.com`, with no subdomain). It recommends "always using a `www` subdomain, even if you also use an apex domain", and says `www` subdomains "are the most stable type of custom domain because `www` subdomains are not affected by changes to the IP addresses of GitHub's servers".

At your **DNS provider** (Chapter 5 introduced DNS), you make records:

| Domain type | Record type the documentation names |
|---------------------------------------|-------------------------------------|
| Subdomain (such as <code>www</code>) | CNAME |
| Apex domain | A, ALIAS or ANAME |

By default, a custom domain set for a user or organization site is used for all project sites of the same account (the documentation's example: `www.octocat.com/octo-project`). **The exact records and values are on the managing-a-custom-domain page and change; copy them from there, not from a book.** The documentation also describes **verifying** a domain, to stop others from taking it over. Do that.

HTTPS. "All GitHub Pages sites, including sites that are correctly configured with a custom domain, support HTTPS and HTTPS enforcement." Sites on `github.io` created after 15 June 2016 are served over HTTPS automatically. Admins of the repository can select **Enforce HTTPS**. For a custom domain, GitHub checks your DNS settings and then requests a certificate from Let's Encrypt; this "may take some time", and the documentation gives a troubleshooting step if the certificate is not created. The whole domain name must be shorter than 64 characters for a certificate to be created.

69.6 A publishing checklist

1. Are the files static, and is there nothing secret in them (or in the history that a private repository hides)?
 2. Do the local checks pass: no `http://` references, no root-absolute paths on a project site?
 3. Which publishing source: branch or workflow? Is `.nojekyll` needed?
 4. Are you within the rules of use?
 5. If you use a custom domain: DNS records copied from the current page, domain verified, HTTPS enforced.
 6. Open the published address in a private window and click every link.
-

Checkpoint

What You Learned

- Pages publishes static files from a repository; user/organization sites and project sites differ in address.
- You publish from a branch (and folder) or with a workflow.

- Mixed content and root-absolute paths are the common faults; search for them locally.
- Rules of use forbid running a commercial business on Pages; limits change, so read them.
- Custom domains use CNAME (subdomains) or A/ALIAS/ANAME (apex) records; HTTPS is supported and can be enforced.

New Vocabulary

Static site, GitHub Pages, apex domain, mixed content (see the glossary).

Commands Learned

`git grep -n -E`, and `:` `> .nojekyll` to create an empty file.

Common Mistakes

1. **Using `/style.css` on a project site.**
2. **Loading assets over `http://` from an HTTPS site.**
3. **Publishing secrets or personal data.**
4. **Trusting DNS values from memory.**
5. **Assuming a private repository makes the site private.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is a static site?
2. What is the address of a project site?
3. Which two publishing sources exist?
4. Why does `/style.css` fail on a project site?
5. Which DNS record types does the documentation name for an apex domain?

Before Moving On

You are ready for Chapter [70](#) if you can:

- [] explain user sites versus project sites
- [] find mixed content and absolute paths with `git grep`
- [] list the steps before publishing

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|-------------------------|
| The local checks with <code>git grep</code> and the creation of <code>.nojekyll</code> | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R329, R330 (local side) |

| Claim | Evidence class | Ledger |
|---|--|-----------|
| Site types, publishing sources, limits, rules of use, custom domains, HTTPS | Checked against github/docs (commit 2eaab0b); not run on a live account | R328-R330 |
| Why a root-absolute path fails on a project site | Reasoning from URL rules and the documented project-site address; not observed on GitHub | none |

Where this leads

Chapter 70 moves the development environment itself into the cloud.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 and 2.2 use your terminal; nothing needs a GitHub account.

Level 1 — Guided

1.1 Two sites. State the address of (a) a user site and (b) a project site for the account `ada` and the repository `notes`.

1.2 Two sources. Name the two ways of choosing what Pages publishes.

Level 2 — Partially guided

2.1 Find the faults. In a sandbox repository, create an `index.html` with one `http://` image and one `href="/style.css"` link. Use `git grep` to find both.

2.2 Fix and check. Fix both faults and run the searches again. What does `git grep` print, and what do you add so that the message "none left" appears?

Level 3 — Independent

3.1 A checklist script. Write a small shell script that fails (exit code 1) if `git grep` finds `http://` in your site files.

3.2 Records. For `www.example.com` and for `example.com`, name the DNS record types the chapter mentions. Where do you find the actual values?

Level 4 — Professional scenario

4.1 A shop. A friend wants to sell products on a Pages site. Write a reply that uses the chapter's rules of use, without promising what GitHub will decide.

Level 5 — Troubleshooting

5.1 After publishing a project site, the page shows without styles. Give two causes from the chapter and how you would confirm each.

5.2 "Certificate not yet created" appears after you set a custom domain. What does the chapter say happens, and what is its suggested step?

Chapter 70 — Codespaces and Dev Containers [Deep]

In this chapter

- what a codespace is, and why teams use them
- dev containers: `.devcontainer/devcontainer.json` and a `Dockerfile` (concept only)
- the lifecycle, saving work, and cost
- security of a cloud development environment
- local versus cloud development

How to read this chapter. This is a **Deep** chapter and can be read later. Everything about Codespaces comes from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026) and **was not run on a live account**: no codespace was created. The one recording is local: it lays out dev container configuration files in a repository and shows that they are ordinary tracked files. **Prices, quotas and machine sizes change and are not stated.** Docker is explained at concept level only.

Before you start. Chapter [39](#).

70.1 The problem: "it works on my computer"

Chapter [3](#) and Chapter [13](#) had you install tools on your own computer. On a team, every person's computer differs: another operating system, another version of the language, a missing library. Setting up a new project can take a day.

A **codespace** is one answer. GitHub's documentation: "A codespace is a development environment that's hosted in the cloud. You can customize your project for GitHub Codespaces by committing configuration files to your repository (often known as Configuration-as-Code), which creates a repeatable codespace configuration for all users of your project."

Each codespace "is hosted by GitHub in a Docker container, running on a virtual machine". You choose from several machine types. By default it is created from an Ubuntu Linux image with popular languages and tools, "Regardless of your local operating system, your codespace will run in a Linux environment. Windows and macOS are not supported operating systems for the remote development container." You connect from a browser or in the other ways the documentation lists, and you are "placed within the Docker container".

A **container** is a packaged, isolated environment with its own tools and files that runs on a host machine. **Docker** is the best-known way to build and run containers. For this book, a container is "a ready-made computer-in-a-box that a configuration file describes"; how Docker builds one is out of scope.

Checked against GitHub's documentation (R331). "What are GitHub Codespaces?".

70.2 Dev containers: the configuration in your repository

The configuration lives in the repository. The documentation:

- "Development containers, or dev containers, are Docker containers that are specifically configured to provide a fully featured development environment. Whenever you work in a codespace, you are using a dev container on a virtual machine."
- The files are in a `.devcontainer` directory. The primary file is `devcontainer.json`, "usually located in the `.devcontainer` directory". Alternatively it can be `.devcontainer.json` in the repository root.
- Several configurations are allowed: alternatives "must be located in their own subdirectory at the path `.devcontainer/SUBDIRECTORY/devcontainer.json`", and when someone creates a codespace they choose one. "Settings cannot be imported or inherited between `devcontainer.json` files."
- The file usually refers to a **Dockerfile**, "a text file that contains the instructions needed to create a Docker container image", or uses the `image` property "to refer directly to an existing image". "If neither a Dockerfile nor an image is found then the default container image is used."
- Use it for **customisation**, not **personalisation**: "You should only include things that everyone working on your codebase needs", such as linters; personal preferences like themes belong in your own settings, dotfiles or Settings Sync.

The documentation's Python example uses these keys: `name`, `image`, `features` (extra tools added to the container), `forwardPorts` (ports made available locally), `postCreateCommand` (a command run after the container is created), `customizations` (tool-specific settings such as editor extensions) and `remoteUser`.

A small local model. Two configuration files are written, one at the default place and one for a second choice, and a tiny script reads them as **strict JSON**:

```
$ git init -q proj && cd proj
$ mkdir -p .devcontainer/docs-dev
$ printf '{\n  "name": "Python 3",\n  "image": "mcr.microsoft.com/devcontainers/python:\n0-3.11-bullseye",\n  "postCreateCommand": "pip3 install --user -r requirements.txt"\n}\n' > .devcontainer/devcontainer.json
$ printf '// a comment, as in the documentation examples\n{\n  "name": "Docs",\n  "image": "mcr.microsoft.com/devcontainers/python:0-3.11-bullseye"\n}\n' > .devcontainer/docs-dev/devcontainer.json
$ python3 ../dc.py .devcontainer/devcontainer.json .devcontainer/docs-dev/devcontainer.json
.devcontainer/devcontainer.json -> strict JSON ok; keys: ['image', 'name', 'postCreateCommand']
.devcontainer/docs-dev/devcontainer.json -> not strict JSON: JSONDecodeError
$ git add .devcontainer && git commit -q -m "Add dev container configurations"
```

Recorded in Bash; *ch70-codespaces/expected-devcontainer.bash.txt*.

Two lessons. First, the second file starts with a `//` comment and **strict JSON does not allow comments**: the script reports it. The documentation's own examples are written as `jsonc`, JSON with comments, which dev container tools accept. When you check such files with a strict JSON tool, remove the comments first, or use a tool that understands `jsonc`. Second, the configuration files are **ordinary files**:

```
$ git ls-files
.devcontainer/devcontainer.json
.devcontainer/docs-dev/devcontainer.json
```

Recorded in Bash; *ch70-codespaces/expected-devcontainer.bash.txt*.

They are versioned, reviewed in pull requests and rolled back like any other file (Chapters 19 and 45). Changing them changes the environment of everyone who creates a codespace afterwards.

What was not run. Whether these two files would start a working codespace was not tested. The image name and Python version are copied from the documentation's example; image tags change.

Checked against GitHub's documentation (R332). "Introduction to dev containers" and "Setting up your Python project for GitHub Codespaces". The recording checks file layout and JSON syntax only.

70.3 Lifecycle, saving and cost

The documentation's lifecycle:

- A codespace's life "begins when you create a codespace and ends when you delete it". You can disconnect and reconnect, and you "may stop and restart a codespace without losing changes".
- **Save your work to Git.** "If your codespace is deleted, then your work will be deleted too. To persist your work, you will need to commit your changes and push them to your remote repository." If you create a new codespace each time, push regularly; if you keep one for weeks, pull from the default branch each time you start.
- **Timeouts.** "By default, a codespace will timeout after 30 minutes of inactivity" and stops; the default can be customised. Data is preserved from the last save.
- **Rebuilding** applies changes to the dev container configuration; creating a new codespace is often the alternative.
- **Limits on the number of codespaces** exist; the numbers are in the documentation.

Cost. "All personal GitHub accounts have a monthly quota of free use of GitHub Codespaces included in the Free or Pro plan." If you create a codespace from an organization-owned repository, "use of the codespace will either be charged to the organization (if the organization is configured for this), or to your personal account". Owners of organizations on some plans can pay for members' use and set a spending limit (Chapter 68). **The size of the quota and the prices are not stated here.** Check the billing page before you rely on codespaces for regular work, and stop or delete codespaces you no longer use.

Checked against GitHub's documentation (R333). "Understanding the codespace lifecycle" and "What are GitHub Codespaces?" (billing overview).

70.4 Security

A codespace runs code and holds a token. The documentation's "Security in GitHub Codespaces" says:

- Each codespace has its **own virtual machine** ("Two codespaces are never co-located on the same VM") and **own virtual network**; incoming connections from the internet are blocked and outbound connections are allowed.
- Every time a codespace is created or restarted it is assigned a **new token with an automatic expiry**, scoped to your access: with write access, read/write on the repository; with only read access, clone-only, and a commit or push leads to a fork with read/write access to that fork.
- **Only the creator** can connect to a codespace.
- **Forwarded ports** are private by default (only the creator, after authenticating). They can be made public **to anyone on the internet without authentication**, and organization owners can restrict this. A public port reverts to private when you remove and re-add it or restart the codespace.
- "You should only open and work within repositories you know and trust." Code in a codespace can use its token: the risks of Chapter 60 (untrusted code, secrets) apply.

Treat secrets as in Chapter 63: do not put them in the dev container configuration, which is committed to the repository. GitHub documents a way to declare which secrets a repository recommends; the mechanism was not examined for this book.

70.5 Local or cloud?

| | Local | Codespace |
|-------------------|--------------------------|----------------------------------|
| Set-up | you install and maintain | the configuration file builds it |
| Works offline | yes | no |
| Same for everyone | no | yes, within the configuration |
| Cost | your computer | quota or paid use |
| Speed | your hardware | the chosen machine type |
| Your files | on your disk | in the cloud until pushed |

Neither is "better". A common pattern: local for daily work, a codespace for quick fixes, reviewing a pull request in a running environment, teaching a class, or onboarding a new contributor in minutes.

Checkpoint

What You Learned

- A codespace is a cloud development environment in a container on a virtual machine, always Linux.

- The environment is configuration as code in `.devcontainer/devcontainer.json`; alternatives sit in subfolders.
- Configuration files are ordinary tracked files; comments in `jsonc` are not strict JSON.
- Unpushed work in a deleted codespace is lost; codespaces time out after inactivity and cost quota.
- Security: isolated VM and network, expiring token, private ports by default, trust only repositories you know.

New Vocabulary

Codespace, **dev container**, **container**, **Docker**, **Dockerfile** (see the glossary).

Commands Learned

No new Git commands; `python3 dc.py` was a helper for the recording.

Common Mistakes

1. **Keeping the only copy of work in a codespace.**
2. **Putting personal preferences or secrets in `devcontainer.json`.**
3. **Making a forwarded port public and forgetting it.**
4. **Leaving codespaces running or undeleted.**
5. **Assuming a container works on Windows or macOS natively.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is a codespace made of?
2. Where does `devcontainer.json` live, and how do you offer two configurations?
3. What happens to unpushed work if a codespace is deleted?
4. Who can connect to a codespace?
5. What does it mean that a forwarded port is public?

Before Moving On

You are ready for Chapter [71](#) if you can:

- [] explain a dev container in one paragraph
- [] place configuration files correctly
- [] list three security points

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|--|-------------------|
| Layout of dev container files; strict JSON rejects comments; the files are tracked | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0, Python 3; CI on newer Git | R332 (local side) |
| What codespaces are, lifecycle, cost model, security | Checked against github/docs (commit 2eaab0b); not run on a live account | R331-R333 |
| That the example configuration starts a working codespace | Not tested | none |

Where this leads

Chapter 71 publishes and installs packages from GitHub.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 and 2.2 use your terminal; nothing needs a GitHub account or Docker.

Level 1 — Guided

1.1 Definition. In two sentences, explain what a codespace is and what configuration as code means here.

1.2 Where? Where must `devcontainer.json` be, and where do you put a second configuration?

Level 2 — Partially guided

2.1 Lay it out. In a sandbox repository, create `.devcontainer/devcontainer.json` with `name` and `image` keys, add it and commit it. Show `git ls-files`.

2.2 Comments. Add a `//` comment to the file and check it with `python3 -m json.tool`. What happens? What does the chapter say about it?

Level 3 — Independent

3.1 Customisation or personalisation? Sort into the file or your own settings: a linter everyone must use, your favourite colour theme, the language version, your shell aliases.

3.2 Cost note. Write a short note for your team that says how they should stop wasting codespace quota, using only what the chapter says.

Level 4 — Professional scenario

4.1 Onboarding. Your open-source project needs new contributors to start in minutes. Write the proposal: what goes in `.devcontainer`, what you would test first, and what the security cautions are.

Level 5 — Troubleshooting

5.1 You deleted a codespace and lost an afternoon's work. What did the chapter say you should have done?

5.2 A colleague made a forwarded port public "just for a minute". List the risks and what resets it.

Chapter 71 — GitHub Packages and Containers [Deep]

In this chapter

- what a package and a registry are
- what GitHub Packages offers
- publishing and installing: scopes, names and authentication
- package permissions and visibility
- containers: concept only

How to read this chapter. This is a **Deep** chapter and can be read later. Everything about GitHub Packages comes from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026) and **was not run on a live account**: no package was published or installed, and npm was not run. The local recording writes package metadata and configuration files and shows where a credential must **not** be. **Limits, prices and the list of supported registries change and are not stated in full.** Containers and Docker are explained at concept level only.

Before you start. Chapter 56. Chapter 63 on secrets and artifacts is useful.

71.1 Packages and registries

A **package** is software packaged so that others (or your own projects) can install it with a tool: a library for a language, a program, a container image. The tool is a **package manager** (Chapter 1). The place where packages are stored and downloaded from is a **registry**.

Chapter 67 built a small library that others would copy by hand. A package lets them write one install command instead, and lets the tool pick the right version (Chapter 66 on version numbers).

GitHub's documentation: "GitHub Packages is a platform for hosting and managing packages, including containers and other dependencies. GitHub Packages combines your source code and packages in one place to provide integrated permissions management and billing." It offers registries for "commonly used package managers, such as npm, RubyGems, Apache Maven, Gradle, Docker, and NuGet", and a separate **container registry** "optimized for containers" that supports "Docker and OCI images". The documentation's table lists the package formats and clients: for example package.json and npm for JavaScript, pom.xml and mvn for Maven, and nupkg with the dotnet command-line tool for .NET. Other ecosystems (Python's public index, for example) are not in that list; check the current documentation for what is supported.

You can view a package's README and metadata such as licence, download statistics and version history on GitHub. For organizations there is also a "linked artifacts" view that records metadata about builds without hosting the files.

Checked against GitHub's documentation (R334). "Introduction to GitHub Packages".

71.2 Publishing: names, scopes and credentials

The npm registry is the documentation's main worked example. It says:

- "GitHub Packages only supports scoped npm packages. Scoped packages have names with the format of @NAMESPACE/PACKAGE-NAME."
- "Package names and scopes must only use lowercase letters."
- You can route a scope to GitHub's registry with a project `.npmrc` file (a line of the form `@OWNER:registry=https://npm.pkg.github.com`), or with a `publishConfig` entry in `package.json`. The documentation notes that using an `.npmrc` file "prevents other developers from accidentally publishing the package to npmjs.org instead of GitHub Packages".
- A `repository` field in `package.json` can connect the package to a repository as soon as it is published.
- To **authenticate** you can put a token in your per-user `~/.npmrc` file, or log in with `npm login`. A workflow authenticates differently (section 71.4).

The recording writes those files in a sandbox, using made-up names, and checks the two naming rules with shell commands. The token is a **made-up** value and goes into the **user-level** file, outside the repository:

```
$ git init -q lib && cd lib
$ printf '{\n  "name": "@ada/wordcount",\n  "version": "0.1.0",\n  "repository": "https://\n  github.com/ada/wordcount"\n}\n' > package.json
$ printf '@ada:registry=https://npm.pkg.github.com\n' > .npmrc
$ printf '//npm.pkg.github.com/:_authToken=not-a-real-token\n' > ~/.npmrc
$ name=$(sed -n 's/.*"name": "\(.*\)"/\1/p' package.json); echo "$name"
@ada/wordcount
$ [ "$name" = "$(printf %s "$name" | tr A-Z a-z)" ] && echo "lowercase: ok"
lowercase: ok
$ case "$name" in @*/*) echo "scoped: ok";; *) echo "not scoped";; esac
scoped: ok
```

Recorded in Bash; *ch71-packages/expected-scope.bash.txt*.

Then it commits, lists what Git tracks, and searches the repository for the token line:

```
$ git add -A && git commit -q -m "Add package metadata and scope mapping"
$ git ls-files
.npmrc
package.json
$ git grep -n '_authToken' || echo "no token in the repository"
no token in the repository
$ grep -c '_authToken' ~/.npmrc
1
```

Recorded in Bash; *ch71-packages/expected-scope.bash.txt*.

`git ls-files` shows `.npmrc` (the scope mapping, which contains **no** secret) and `package.json`. The search finds no `_authToken` in the repository, while the user-level file has one. That is the rule of Chapter 33 applied to configuration: **the mapping may be committed; the token may not**. If a token ever lands in a committed file, treat it as leaked: revoke it first.

Which kind of token? The documentation says that to manage a package on some registries you must use a **personal access token (classic)** with the right scope, and that your account needs appropriate permissions; GitHub's advice on tokens is to use the fewest scopes necessary (Chapter 63). The exact scope names are in the documentation; they were not copied here.

Checked against GitHub's documentation (R335). "Working with the npm registry". The recording checks naming rules, file placement and the absence of the token from the repository; it does not contact any registry.

71.3 Permissions and visibility

Two models exist, depending on the registry:

- **Granular permissions:** the package belongs to a personal account or organization, and you set its access and visibility "separately from a repository that is connected (or linked) to a package".
- **Repository-scoped permissions:** the package "inherits the permissions and visibility of the repository in which the package is published". Some registries support only this model.

For linked packages the documentation says: "By default, if you publish a package that is linked to a repository, the package automatically inherits the access permissions (but not the visibility) of the linked repository. For example, a user who has read access to the linked repository will also have read access to the package." This only happens if you link the repository **before** publishing; linking later from the package settings leaves the existing permissions unchanged. When permissions are inherited, you change them in the repository's settings. For packages scoped to a personal account you can assign read, write or admin roles to other users.

Visibility matters twice: a public package can be installed by anyone; a private one only by those with access. Choose it on purpose, and never publish something that includes secrets (Chapter 33).

Checked against GitHub's documentation (R336). "About permissions for GitHub Packages" and "Configuring a package's access control and visibility".

71.4 Publishing from a workflow

A workflow (Chapter 54) can publish a package when you push a tag or create a release (Chapter 66). The documentation says that when a package inherits access permissions from a linked repository, "GitHub Actions workflows in the linked repository also automatically get access to the package". That is why a workflow usually authenticates with its own short-lived token, `GITHUB_TOKEN` (Chapter 60), rather than with a personal token that someone pastes into a secret. Give the workflow only the permission that it needs; the name of the permission that lets a workflow write packages is in the documentation and was not copied here.

A publishing workflow makes an artifact public for many users. Guard it: publish only from a protected branch or a tag, require review for changes to the workflow file (Chapters 50 and 60), and prefer pinned actions.

71.5 Containers, in one page

A **container image** packages a program with everything it needs to run: the system libraries, the language runtime and your code. It is stored in a registry and started as a **container** on any machine that can run containers (Chapter 70 used them for development). **Docker** is the best-known tool.

GitHub's container registry accepts Docker and OCI images; the documentation shows how to link an image to a repository at publication, for example with the `org.opencontainers.image.source` label, so that it inherits the repository's access permissions. The commands to build, tag, log in and push are in the documentation and depend on Docker, which this book did not run.

Whether you need containers at all is a design decision, not a duty. A small library rarely does; a service that must run the same way in many places often does.

71.6 Installing safely

Installing a package runs someone else's code (Chapter 60 and Chapter 63). Before you add a dependency: Who maintains it? Is the licence acceptable (Chapter 65)? How is it versioned? Does it have a security policy (Chapter 62)? Pin versions with a lock file, and let Dependabot propose updates.

Checkpoint

What You Learned

- A registry stores packages; GitHub Packages offers several, and a container registry.
- npm packages on GitHub must be scoped and lowercase; `.npmrc` maps a scope to the registry.
- Commit the mapping, never the token; keep tokens in a user-level file or in a workflow's `GITHUB_TOKEN`.
- Package permissions are either granular or inherited from a linked repository; link before publishing to inherit.
- A container image packages a program with its environment; Docker is only introduced.

New Vocabulary

Package, registry, package manager, scope, container image (see the glossary).

Commands Learned

No new Git commands. The recording used `tr`, `case` and `git grep -n`.

Common Mistakes

1. **Committing a token in `.npmrc`.**
2. **Publishing an unscoped or upper-case package name to GitHub's npm registry.**
3. **Linking a repository after publishing and expecting inherited permissions.**
4. **Publishing from an unprotected workflow.**
5. **Installing a dependency without looking at it.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is a registry?
2. What must the name of an npm package on GitHub look like?
3. Where may a token live, and where not?
4. When does a package inherit a repository's permissions?
5. What does a container image contain?

Before Moving On

You are ready for Chapter [72](#) if you can:

- [] explain package, registry and scope
- [] set up a scope mapping without leaking a token
- [] describe the two permission models

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|---|-------------------|
| Name rules, file placement, absence of the token from the repository | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R335 (local side) |
| What GitHub Packages offers, npm registry rules, permission models | Checked against github/docs (commit 2eaab0b); not run on a live account ; npm and Docker not run | R334-R336 |
| Names of token scopes and workflow permissions for packages; container commands | Not copied or checked | none |

Where this leads

Chapter 72 follows the last step of the chain: getting software running for users.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 and 2.2 use your terminal; nothing needs a GitHub account, a registry or Docker.

Level 1 — Guided

1.1 Vocabulary. Explain package, registry and package manager in one sentence each.

1.2 Names. Which of these npm package names could be published to GitHub's npm registry according to the chapter: `wordcount`, `@Ada/wordcount`, `@ada/wordcount`?

Level 2 — Partially guided

2.1 The mapping. In a sandbox repository, create `package.json` with the name `@ada/wordcount` and an `.npmrc` that maps the scope `@ada` to `https://npm.pkg.github.com`. Commit both.

2.2 The token. Create a user-level file (in your home folder, not in the repository) containing a made-up token line, and prove with `git grep` that the token is not in the repository.

Level 3 — Independent

3.1 A check script. Write a shell script that fails if `_authToken` appears in any tracked file.

3.2 Two models. Describe granular and repository-scoped package permissions and give one advantage of each.

Level 4 — Professional scenario

4.1 A publishing workflow. Write a design (in words, no YAML needed) for publishing a package when a release tag is pushed: the trigger, the credential, who may change the workflow, and what could go wrong.

Level 5 — Troubleshooting

5.1 You linked a repository to a package after publishing it, and the repository's readers still cannot read the package. Why, according to the chapter?

5.2 A token appears in a committed `.npmrc`. List the steps in order.

Chapter 72 — Deployment Pipelines [Deep]

In this chapter

- what deployment means, and the kinds of target
- a pipeline from commit to running software
- release directories and rollback, tried locally
- environments, approvals and deployment secrets
- how GitHub's documentation approaches third-party platforms

How to read this chapter. This is a **Deep** chapter and can be read later. The local part, a release directory scheme with a switch and a rollback, was recorded in Bash and zsh. Statements about GitHub environments come from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026) and **were not run on a live account. No cloud platform, server or container service was used for this chapter**, and their commands are not reproduced: they belong to each provider's documentation and change often.

Before you start. Chapter 58 (environments, Project 10) and Chapter 5 (servers, HTTPS).

72.1 What deployment is

To **deploy** is to put a version of your software somewhere it runs for its users. A **pipeline** (Chapter 52) automates the steps: build, test, package, publish, switch. **Continuous deployment** deploys every change that passes; **continuous delivery** makes it ready to deploy and leaves the final step to a person. Chapter 52 explains the words.

Different software needs different targets:

| Kind | Example | Typical deployment step |
|--|--|--|
| Static website | HTML, CSS, JavaScript | copy files to a host (Chapter 69) |
| Server application (Node.js, Python, others) | a web application | build, install dependencies, restart a process |
| Container | a packaged service | build an image, push it to a registry (Chapter 71), run it somewhere |
| Virtual private server (VPS) | a rented Linux server you administer | copy files or a container, restart services, keep the system updated |
| Managed cloud service | a platform that runs your code for you | hand the package to the platform's tool or API |

Which target is right depends on cost, skills and needs, and this book does not recommend a provider.

72.2 A pipeline from commit to running

Whatever the target, the shape is the same:

1. **Trigger**: a merge to the default branch, a tag, or a manual start (Chapter 58).
2. **Build and test**: the checks of Chapter 52 run again on the exact commit.
3. **Package** into one artifact with a version: an archive, an image, a static folder (Chapter 66).
4. **Approve** (for sensitive environments): a person or a rule allows the deployment.
5. **Deploy**: the artifact goes to the target with credentials that only this step holds.
6. **Check**: a health check confirms the new version works.
7. **Keep the previous version** so that you can go back.

GitHub's documentation on third-party platforms shows example workflows for several: Azure App Service for Node.js, Python, Java, .NET and Docker; Azure Static Web Apps; Amazon Elastic Container Service; Google Kubernetes Engine. Its Node.js example builds, tests and deploys "when there is a push to the main branch", takes the target's name from a variable in the workflow, and reads a **publish profile** from a repository secret (secrets.AZURE_WEBAPP_PUBLISH_PROFILE). Those examples need accounts and resources on those platforms and were not run for this book.

Checked against GitHub's documentation (R337). "Deploying to third-party platforms" (the Node.js to Azure App Service page as a sample) and "Deployments and environments".

72.3 Release directories and rollback

The most useful idea in deployment is small: **never overwrite the running version; put each version in its own folder and switch a pointer**. On a Linux server the pointer can be a **symbolic link**, a file that names another folder. The recording plays this on your own computer, with two versions of the Sunrise Bakery page:

```
$ mkdir -p server/releases
$ mkdir server/releases/v1 server/releases/v2
$ printf 'Sunrise Bakery, version 1\n' > server/releases/v1/index.txt
$ printf 'Sunrise Bakery, version 2\n' > server/releases/v2/index.txt
$ ln -sf releases/v1 server/current
$ cat server/current/index.txt
Sunrise Bakery, version 1
$ readlink server/current
releases/v1
```

Recorded in Bash; *ch72-deploy/expected-release-dirs.bash.txt*.

server/current is the link. It points to releases/v1, so reading current/index.txt shows version 1, and readlink shows where the link points. Deploying version 2 changes only the link, and rolling back changes it back:

```
$ ln -sf releases/v2 server/current
$ cat server/current/index.txt
Sunrise Bakery, version 2
$ ln -sf releases/v1 server/current
$ cat server/current/index.txt
Sunrise Bakery, version 1
$ ls -l server/releases
v1
v2
$ test -f server/current/index.txt && echo "health check: ok"
health check: ok
```

Recorded in Bash; *ch72-deploy/expected-release-dirs.bash.txt*.

Three properties follow:

- **Switching is fast:** one small change, not a copy of many files while visitors are reading them.
- **Rollback is the same operation in reverse**, with the previous version still in place. That is why step 7 of the pipeline keeps it.
- **The health check is a decision point:** a tiny test `-f` here; in real life a request to the running service. If the check fails, switch back automatically.

This is a local illustration; it does not model databases, running processes or caches, which make real rollbacks harder (a database change may not be reversible). The lesson is the shape, not the commands of a particular server.

72.4 Environments, approvals and secrets

GitHub's environments (Chapter 58) attach rules to a deployment target. The documentation: "Each job in a workflow can reference a single environment. Any protection rules configured for the environment must pass before a job referencing the environment is sent to a runner. The job can access the environment's secrets only after the job is sent to a runner." Protection rules can "require a manual approval, delay a job, or restrict the environment to certain branches". Required reviewers: "You can list up to six users or teams as reviewers", and one approval lets the job proceed; you can prevent self-review "so that deployments to protected environments are always reviewed by more than one person". A wait timer delays a job by a set number of minutes. **Availability of some rules depends on plan and repository visibility**; the documentation says, for example, that on some plans required reviewers are only available for public repositories.

Deployment secrets are credentials that let the workflow change your production system. Apply Chapter 63 and Chapter 60:

- store them as **environment secrets**, so approval gates their use;
 - give them the **fewest permissions** the deployment needs (for example, upload to one folder, not administer the account);
 - prefer **short-lived credentials** issued to the job (the OpenID Connect idea of Chapter 60) over long-lived keys;
 - **never print them**, and never put them in the repository or an image;
 - **rotate** them, and revoke first if they leak.
-

72.5 Choosing and staging

Staging is a copy of production where you deploy first. A common design has two environments: staging (deploys automatically after the tests) and production (needs approval). The pipeline deploys the **same artifact** to both, so what you tested is what you release.

Some questions to answer before you build a pipeline:

1. What is the cheapest way to deploy that is safe enough?
 2. How will you notice that it broke (logs, health check, alerts)?
 3. How do you roll back, and have you tried it?
 4. Who may deploy, and how is that recorded (Chapter 50)?
 5. Where is the database, and what does a version change do to it?
-

Checkpoint

What You Learned

- A pipeline turns a commit into running software: trigger, test, package, approve, deploy, check, keep the previous version.
- Targets differ: static site, server application, container, VPS, managed service.
- Release directories plus a switching link give fast deployment and fast rollback.
- Environments add approvals, wait timers and branch limits, and gate environment secrets.
- Deployment secrets: least privilege, short-lived where possible, never printed, rotated.

New Vocabulary

Deployment, continuous deployment, rollback, staging, symbolic link (see the glossary).

Commands Learned

```
ln -sfn, readlink, test -f.
```

Common Mistakes

1. **Overwriting the running version with no way back.**
2. **Deploying a different build from the one you tested.**
3. **Using one long-lived, all-powerful key for deployment.**
4. **Never practising a rollback.**
5. **Assuming a database change can be rolled back like files.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What are the seven steps of the pipeline in section 72.2?
2. Why keep the previous release?
3. What does a symbolic link give a deployment?
4. What can an environment's protection rules do?
5. Why deploy the same artifact to staging and production?

Before Moving On

You are ready for Chapter 73 if you can:

- [] describe a deployment pipeline
- [] perform the switch and rollback locally
- [] name three rules for deployment secrets

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|--------|
| Release directories, the switching link, rollback | Locally tested: Bash 5.2 and zsh 5.9; CI | R338 |
| Environment protection rules, secrets access, sample third-party workflows | Checked against github/docs (commit 2eaab0b); not run on a live account ; no provider used | R337 |
| Which target is best; provider commands | Not addressed; outside this book | none |

Where this leads

Chapter 73 shows GitHub from the command line.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 and 2.2 use your terminal only; no server or account is needed.

Level 1 — Guided

1.1 Steps. List the seven steps of the pipeline in section 72.2 in order.

1.2 Targets. Match each to a deployment step: static website, container, VPS.

Level 2 — Partially guided

2.1 Switch. Create two release folders and a current link. Point the link at the first release, read a file through it, switch to the second and read again.

2.2 Roll back. Roll back to the first release and show that both release folders still exist.

Level 3 — Independent

3.1 A health check. Write a shell command that succeeds only if `current/index.txt` exists and contains the word "Bakery". Show it succeeding.

3.2 Two environments. Design staging and production for the Sunrise Bakery site: what deploys automatically, what needs approval, and why the same artifact goes to both.

Level 4 — Professional scenario

4.1 Secrets review. A colleague stores one all-powerful key as a repository secret and prints it in the log "to debug". Write the review using the deployment-secret rules and [Chapter 60](#).

Level 5 — Troubleshooting

5.1 The new release is live and broken, and the previous release folder was deleted by a clean-up script. What went wrong in the design, and how would you avoid it?

5.2 A database change came with the release. Why might a rollback of files not be enough?

Chapter 73 — GitHub CLI [Deep]

In this chapter

- what gh is, and how it differs from git
- installing it and checking what you installed
- the commands that need no account, run and recorded
- the commands that need one, from GitHub's documentation
- authentication in scripts and workflows, and the CLI's telemetry

How to read this chapter. This is a **Deep** chapter and can be read later. The commands in section 73.3 were run with **GitHub CLI 2.102.0** (released 30 September 2026), downloaded from the official release page and checked against its published checksum, in Bash and zsh, and re-run in CI with the same pinned version. **Commands that talk to GitHub (gh pr list with a login, gh repo create, gh issue create and others) were not run**, because that needs an authenticated account (a step the book's author has not yet done on a live account); they are described from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026). Output wording of a tool changes between versions: expect differences from what you see.

Before you start. Chapter 38 (signing in), Chapter 7 and Chapter 45.

73.1 Two tools with similar names

GitHub's documentation gives the difference directly:

- "The Git command line interface (git) allows you to work with a local or remote Git repository. The remote repository may be hosted on GitHub or it may be hosted by another service."
- "GitHub CLI (gh) is specifically for working with GitHub." It lets you use the command line "to interact with GitHub in all sorts of ways", and "makes it easier for you to create scripts to automate GitHub operations".

The rule of thumb: **git works on your repository and its history; gh works on the things that GitHub adds** (Chapter 36): pull requests, issues, releases, workflow runs, codespaces, repository settings. gh is not a replacement for Git: it uses Git underneath for some tasks, and you still commit and push with git.

| To do this | Use |
|---|------------------------------|
| commit, branch, merge, rebase, push | git |
| open, list, review or merge a pull request | gh pr (or the web interface) |
| create or view an issue | gh issue |
| create a release from a tag (Chapter 66) | gh release |
| list, watch or re-run a workflow run (Chapter 57) | gh run, gh workflow |

| To do this | Use |
|--------------------------------|--------------|
| manage codespaces (Chapter 70) | gh codespace |

73.2 Installing and checking

gh is a separate program. The documentation says how to install it for each system on its pages; this book does not repeat the steps, because they change and depend on your system. Two habits from earlier chapters apply:

- **Get it from an official source** (Chapter 13) and, if you download a file, **check its checksum** (Chapter 63). The version used for this chapter was downloaded from the project's release page on GitHub; the file's SHA-256 matched the value in the release's published checksum list, and the CI job checks it again against a value written in the workflow file.
- **Check what you have:** `gh --version` prints the version and the release page. A very old gh may lack commands described here.

Ask gh for help with gh alone (a list of top-level commands) and add `--help` to any command: `gh issue --help`, `gh issue create --help`. The documentation names the top-level areas `issue`, `pr`, `repo` and others.

73.3 Commands that need no account

Several gh commands only read or write local settings, and the ones that need GitHub tell you so. The first recording makes a shortcut, a setting, and asks about the login state:

```
$ gh alias set prd "pr create --draft"
- Creating alias for prd: pr create --draft
✓ Added alias prd
$ gh alias list | grep prd
prd: pr create --draft
$ gh config set editor "code -w"
$ gh config get editor
code -w
$ gh auth status || echo "not logged in (exit status $?)"
You are not logged into any GitHub hosts. To log in, run: gh auth login
not logged in (exit status 1)
```

Recorded in Bash; *ch73-ghcli/expected-offline.bash.txt*.

- `gh alias set` created an **alias**, a short name for a longer command. The documentation's example is the same: `gh alias set prd "pr create --draft"` and then `gh prd` opens a draft pull request. `gh alias list` shows aliases; here `grep` picks the one we made because the list also contains defaults.
- `gh config set editor ...` stored the editor that gh opens when it needs you to write text (the documentation's example is `gh config set editor "code -w"`, where `-w` makes the command wait until you close the file), and `gh config get editor` read it back. These settings are files in your home folder; Chapter 14 did the same for Git.

- `gh auth status` reported "You are not logged into any GitHub hosts" and **exited with status 1**. In a script, that exit status is how you detect it.

Next, the same tool in a Git repository with no remote asks for something that needs GitHub:

```
$ git init -q project && cd project
$ gh pr list || echo "gh needs authentication (exit status $?)"
To get started with GitHub CLI, please run:  gh auth login
Alternatively, populate the GH_TOKEN environment variable with a GitHub API authentication
token.
gh needs authentication (exit status 4)
```

Recorded in Bash; *ch73-ghcli/expected-offline.bash.txt*.

It did not guess: it said `gh auth login` and offered the other route, an environment variable named `GH_TOKEN`, and exited with status 4. **What `gh pr list` prints when you are logged in was not run.**

73.4 Commands that need an account

These are from GitHub's quickstart for the CLI and were **not run** for this book. When you first use some of them you may be asked to add scopes to your token; the documentation says to follow the on-screen instructions.

| Command | What the documentation says it does |
|--|--|
| <code>gh auth login</code> | signs in (interactive) |
| <code>gh status</code> | shows your current work on GitHub across the repositories you are subscribed to |
| <code>gh repo view OWNER/REPO</code> | shows the description and README; <code>--web</code> opens the browser; inside a clone you can omit the name |
| <code>gh repo clone OWNER/REPO</code> | clones a repository |
| <code>gh repo create</code> | creates a repository, and can push an existing local one |
| <code>gh issue list --repo OWNER/REPO</code> | lists recent open issues |
| <code>gh pr list --repo OWNER/REPO, gh pr list --label NAME</code> | lists open pull requests |
| <code>gh search prs --review-requested=@me --state=open</code> | lists pull requests you were asked to review |
| <code>gh pr create</code> | creates a pull request, following the prompts |
| <code>gh release create TAG</code> | creates a release (Chapter 66) |
| <code>gh run rerun RUN_ID, --failed, --debug, gh run watch</code> | re-run or follow workflow runs (Chapter 57) |
| <code>gh codespace create, gh codespace list</code> | manage codespaces (cs is accepted for codespace) |
| <code>gh auth switch</code> | switch between accounts on the same host |

The documentation adds that `gh` can be extended with **extensions**, installed from other people. An extension is code written by someone else that runs with your credentials: apply Chapter 60's thinking (who wrote it, do you trust it) before installing one.

Checked against GitHub's documentation (R339). "About GitHub CLI" and "GitHub CLI quickstart". The commands in the table were not executed.

73.5 Authentication in scripts and workflows

Interactive login suits a person. For automation, the documentation describes passing a token in an **environment variable**: "Instead of using the `gh auth login` command, pass an access token as an environment variable called `GH_TOKEN`. GitHub recommends that you use the built-in `GITHUB_TOKEN` instead of creating a token." In a workflow you write `GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}` in the step's env (Chapter 54), and give the workflow's token only the permissions the command needs (Chapter 60). If you must create a token, "store your token as a secret".

The recording above showed the same door from the other side: without a login and without `GH_TOKEN`, `gh` refuses.

73.6 Telemetry: what the tool reports

GitHub's documentation has a page on **telemetry**: the CLI collects usage data. It says telemetry "is not collected when the target is GitHub Enterprise Server", that the tool is open source, and that you can see what would be sent. Setting the environment variable `GH_TELEMETRY=log`, or `gh config set telemetry log`, prints "the JSON payload that would normally be sent ... to stderr instead". To opt out, set `GH_TELEMETRY=false` (any falsy value), or `DO_NOT_TRACK=true`, or `gh config set telemetry disabled`; "the environment variables take precedence over the configuration value". Extensions "may collect their own usage data and are not controlled by opting out".

The last recording lines run both modes, with the output cut to two fields so that random identifiers and times do not appear:

```
$ GH_TELEMETRY=log gh config get editor 2>&1 | grep -E '"(type|command)"'
  "type": "command_invocation",
  "command": "gh config get",
$ GH_TELEMETRY=false gh config get editor 2>&1 | grep -c "Telemetry payload" || true
0
```

Recorded in Bash; *ch73-ghcli/expected-offline.bash.txt*.

In log mode the payload named the event type (`command_invocation`) and the command (`gh config get`); with `GH_TELEMETRY=false` no payload was printed (the count is 0). A full payload contains more fields, such as the version and operating system and identifiers; **read your own with `GH_TELEMETRY=log`**. This book does not judge the policy. It tells you that the setting exists, and it is your decision.

Checked against GitHub's documentation (R340) and locally tested. "GitHub CLI telemetry" (behaviour), and the recording (log and disabled modes on version 2.102.0).

73.7 `gh` in your habits

1. **Use it for what GitHub adds:** pull requests, issues, releases, runs.
2. **Keep the browser** for reading long discussions and reviews.
3. **Script carefully:** check the exit status; never print a token; prefer `GITHUB_TOKEN` in workflows.
4. **Put common commands under aliases**, and keep them in your own configuration.
5. **Check `--help`** rather than trusting a book's flags: they change.

Checkpoint

What You Learned

- `git` works on repositories; `gh` works on GitHub's additions and uses your login.
- `gh alias`, `gh config` and `gh auth status` work without an account; other commands ask you to log in or set `GH_TOKEN`.
- Exit status tells scripts what happened: 1 for not logged in, 4 for authentication needed (in this version).
- In workflows pass `GITHUB_TOKEN` as `GH_TOKEN`, with least privilege.
- `gh` has telemetry that can be inspected (`GH_TELEMETRY=log`) and disabled.

New Vocabulary

GitHub CLI, **alias**, **telemetry** (see the glossary).

Commands Learned

`gh alias set`, `gh alias list`, `gh config set`, `gh config get`, `gh auth status`, `gh pr list`, `gh --help`.

Common Mistakes

1. **Thinking `gh` replaces `git`.**
2. **Pasting a token into a command line or a script.**
3. **Installing extensions without looking at them.**
4. **Assuming output and flags are the same in every version.**
5. **Ignoring exit statuses in scripts.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between `git` and `gh`?
2. Which exit status did `gh auth status` return when not logged in?
3. How does a workflow authenticate `gh`?
4. How can you see what `gh` telemetry would send?
5. Why be careful with extensions?

Before Moving On

You are ready for Chapter 74 if you can:

- [] say when to use `gh` rather than `git`
- [] create an alias and read a setting
- [] describe token authentication for scripts

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-------------------------|
| Alias, config, unauthenticated behaviour and exit statuses; telemetry log and disabled modes | Locally tested with <code>gh 2.102.0</code> (checksum-verified download), Bash 5.2, zsh 5.9; CI with the same pinned version | R339, R340 (local side) |
| What <code>gh</code> is; commands that need a login; <code>GH_TOKEN</code> ; extensions; telemetry statements | Checked against <code>github/docs</code> (commit 2eaab0b); commands that need an account not run | R339, R340 |

Where this leads

Chapter 74 goes one level down: the REST API, webhooks and apps, which `gh api` and your scripts use.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 to 2.2 need `gh` installed but no account. **No exercise asks you to paste a token anywhere.**

Level 1 — Guided

1.1 Two tools. Complete: `git` works on ____; `gh` works on _____. Give two examples for each.

1.2 Version. Which command shows the `gh` version, and why does the version matter?

Level 2 — Partially guided

2.1 An alias. Create an alias named `prd` for `pr create --draft`, then list your aliases and find it. Then delete it with `gh alias delete prd`.

2.2 A setting. Set your editor for `gh` to `code -w` (or another editor you have) and read it back. What does `-w` do?

Level 3 — Independent

3.1 A status check. Write a shell snippet that runs `gh auth status` and prints "ready" or "please log in" according to its exit status.

3.2 Telemetry. Show, with `GH_TELEMETRY=log`, the event type of a command of your choice. Then show how you would disable telemetry with an environment variable and with a configuration setting.

Level 4 — Professional scenario

4.1 A workflow step. Describe (words or YAML) a workflow step that runs `gh run list` or another `gh` command using `GITHUB_TOKEN`. Which permission would you restrict, and why?

Level 5 — Troubleshooting

5.1 `gh pr list` in a script prints "To get started with GitHub CLI, please run: `gh auth login`" and stops. Give two ways to give the script credentials, and say which one you prefer for a workflow.

5.2 A colleague's `gh` says a command does not exist. Give two likely reasons.

Chapter 74 — APIs, Webhooks and Apps [Deep]

In this chapter

- what an API is, and GitHub's REST and GraphQL APIs
- requests, pagination, versions and rate limits
- webhooks: GitHub calling you, and verifying the call is genuine
- GitHub Apps versus OAuth apps
- tokens for scripts

How to read this chapter. This is a **Deep** chapter and can be read later. Two kinds of evidence are used, and the text says which. **Recorded and re-run in CI:** the verification of a webhook signature (using the test values published in GitHub's documentation) and the reading of a pagination header, both with no network and no account. **Observed live, once:** a few requests to GitHub's API made on 30 September 2026 from the authoring environment, through a proxy that authenticates the requests on the session's behalf, so **the rate-limit numbers seen there are not typical and are not quoted;** these observations are not re-run in CI. **Everything else** comes from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026) and was not run.

Before you start. Chapter [38](#) (tokens) and Chapter [73](#).

74.1 What an API is

An **API** (application programming interface) is a way for one program to ask another to do something. GitHub's web pages are for people; its **API** is for programs. GitHub's documentation: "You can use GitHub's API to build scripts and applications that automate processes, integrate with GitHub, and extend GitHub. For example, you could use the API to triage issues, build an analytics dashboard, or manage releases."

GitHub offers **two APIs**: "a REST API and a GraphQL API. You can interact with both APIs using GitHub CLI, curl, the official Octokit libraries, and third party libraries. Occasionally, a feature may be supported on one API but not the other."

- **REST** is organised by **resources** with an address for each; you ask with an HTTP request (Chapter [5](#)): for example, a request to get the issues of a repository. Each endpoint returns a fixed structure. The documentation says that REST "returns more data than you requested and returns it in a pre-determined structure".
- **GraphQL** is one address to which you send a **query** naming exactly the fields you want. "The GraphQL API returns exactly the data that you request" and "you can also accomplish the equivalent of multiple REST API request in a single GraphQL request". Node IDs let you move between the two.

"You should use the API that best aligns with your needs and that you are most comfortable using. You don't need to exclusively use one API over the other." This chapter uses REST for its examples.

Checked against GitHub's documentation (R341). "About the REST API" and "Comparing GitHub's REST API and GraphQL API".

74.2 A REST request, as observed

A request has a method (GET reads), an address, and headers; the answer has a status code, headers and a body, normally JSON (Chapter 5 on status codes). The documentation's example asks for a repository's issues:

```
curl --include --request GET \
  --url "https://api.github.com/repos/OWNER/REPO/issues" \
  --header "Accept: application/vnd.github+json"
```

--include shows the response headers, which matter, as the next sections show. **An equivalent read-only request for the commits of this book's repository, asking for one item per page, was made from the authoring environment** (the API was reached through the session's proxy). Observed:

- Status **200** and a JSON list with one item; each item had the keys sha, commit, author, committer, parents, url, html_url and others.
- A **Link** header holding a rel="next" and a rel="last" address, because there were more pages than the one that was returned (section 74.3).
- **X-GitHub-API-Version-Selected: 2022-11-28** although no version was sent (section 74.4).
- Headers whose names start with X-RateLimit- (section 74.5).
- A Cache-Control and an ETag, which a client can use to avoid re-downloading unchanged data.

Anything that could identify the session (token expiry, client identifiers) was in the headers too and is not reproduced. **Never publish raw headers or logs without reading them first:** they can contain identifiers, and in some tools, credentials (Chapter 63).

74.3 Pagination

"When a response from the REST API would include many results, GitHub will paginate the results and return a subset." The documentation's example: a request for a repository's issues "will only return 30 issues ... even though the repository includes over 1600 open issues". You ask for more with the **Link header**, which gives the URLs of the previous, next, first and last pages, and you can choose the page size with **per_page** on endpoints that support it. "If the endpoint does not support pagination, or if all results fit on a single page, the link header will be omitted."

The recording parses the documentation's example header with a short script (link.py) and prints each relation:

```
$ printf '%s\n' '<https://api.github.com/repositories/1300192/issues?page=2>; rel="prev",
<https://api.github.com/repositories/1300192/issues?page=4>; rel="next", <https://
api.github.com/repositories/1300192/issues?page=515>; rel="last", <https://api.github.com/'
```

```

repositories/1300192/issues?page=1>; rel="first" | python3 link.py
first -> https://api.github.com/repositories/1300192/issues?page=1
prev -> https://api.github.com/repositories/1300192/issues?page=2
next -> https://api.github.com/repositories/1300192/issues?page=4
last -> https://api.github.com/repositories/1300192/issues?page=515
$ printf '%s\n' '<https://api.github.com/repositories/1300192/issues?page=2>; rel="next"' |
python3 link.py
first -> (none)
prev -> (none)
next -> https://api.github.com/repositories/1300192/issues?page=2
last -> (none)

```

Recorded in Bash; *ch74-api/expected-verify.bash.txt*.

The second call shows an answer with only a `next` relation: the rest are `(none)`. A script that follows `next` until it is absent has fetched everything, with no need to count pages. **The script reads a header that is pasted in; it makes no request.**

Checked against GitHub's documentation (R342). "Using pagination in the REST API". The live observation in section 74.2 confirmed the header's presence and the `next/last` relations for one endpoint.

74.4 Versions and rate limits

API versions. The documentation says to "use the `X-GitHub-API-Version` header to specify an API version", that requests without it use a default version, and that versions are supported for a period after a newer one is released; asking for an unsupported version "will receive a 410 Gone response". **Observed:** with no header, the answer carried `X-GitHub-API-Version-Selected: 2022-11-28`; with `X-GitHub-API-Version: 2022-11-28` the same; with `X-GitHub-API-Version: 2019-01-01`, a version that (as far as this book knows) never existed, the answer was **400 Bad Request**, not 410. The 410 case (a version that was supported) was not observed. Pin a version in your scripts so that a change of default does not break them silently.

Rate limits. "GitHub limits the number of REST API requests that you can make within a specific amount of time. This limit helps prevent abuse and denial-of-service attacks." Some endpoints, such as search, are stricter, and GraphQL has its own limit. Limits depend on how you authenticate; the numbers are on the documentation page and change, so they are not repeated here. The headers `X-RateLimit-Limit`, `-Remaining`, `-Used`, `-Reset` and `-Resource` tell you where you stand; the documentation says that a rate-limit error is typically **403 or 429**, and that an integration "should wait before making another request". A good script reads these headers, backs off, and caches with ETag.

74.5 Webhooks: GitHub calls you

Polling asks "anything new?" again and again. A **webhook** reverses it: "Webhooks let you subscribe to events happening in a software system and automatically receive a delivery of data to your server whenever those events occur." You give GitHub a URL and choose the events (a push, a pull request opened, a Pages build, a new team member); when one occurs GitHub "will

send an HTTP request with data about the event to the URL that you specified". Uses listed in the documentation: triggering CI on an external server (Chapter 61), notifying a chat service, updating an issue tracker, deploying (Chapter 72), and audit logging.

Webhooks belong to a **repository**, an **organization** or another installation point, and only see what is there. "You cannot create webhooks for individual user accounts." Creating one in a repository needs admin access; in an organization, an owner. A limit exists per event type and changes.

Verify what you receive. Your webhook URL is public: anyone can send a request to it. GitHub's documentation therefore says you should "validate the webhook signature before processing the delivery further". You create a **secret token** (a random, high-entropy string), store it securely ("Never hardcode a token into an application or push a token to any repository"), and GitHub sends, with each delivery, a header X-Hub-Signature-256. It is computed as an **HMAC** (a keyed hash: a fingerprint that only someone with the secret can produce) with SHA-256 over the payload. Points from the documentation: "The hash signature always starts with sha256=", it is a hex digest, handle the payload as UTF-8, and "Never use a plain == operator": use a **constant-time comparison**, so that the time taken does not leak information.

The documentation publishes test values: secret It's a Secret to Everybody and payload Hello, World! must give 757107ea0eb2509fc211221cce984b8a37570b6d7586c22c46f4379c8b043e17. The recording computes it two independent ways (Python's hmac and the openssl command), compares with hmac.compare_digest, and shows a changed payload being rejected:

```
$ python3 sig.py
sha256=757107ea0eb2509fc211221cce984b8a37570b6d7586c22c46f4379c8b043e17
genuine delivery accepted: True
tampered payload accepted: False
$ secret="It's a Secret to Everybody"
$ printf '%s' 'Hello, World!' | openssl dgst -sha256 -hmac "$secret" | awk '{print $NF}'
757107ea0eb2509fc211221cce984b8a37570b6d7586c22c46f4379c8b043e17
```

Recorded in Bash; *ch74-api/expected-verify.bash.txt*.

Both methods printed the documented value; the genuine delivery was accepted and a payload with one changed character was not. **These are public test values; never use a value from a book as your real secret.** Also: a valid signature proves the payload came from someone with the secret and was not altered; it does not prove the event is one you should trust (validate the content too), and an attacker who copies an old signed delivery could replay it (the documentation's best-practice page discusses handling deliveries; this book did not verify its replay advice).

Checked against GitHub's documentation (R343) and locally tested. "About webhooks", "Types of webhooks", "Validating webhook deliveries". No webhook was created and no delivery was received.

74.6 GitHub Apps and OAuth apps

Sometimes your program must act on GitHub for other people or for an organization. GitHub distinguishes:

- **GitHub Apps:** "tools that extend GitHub's functionality. GitHub Apps can do things on GitHub like open issues, comment on pull requests, and manage projects. They can also do things outside of GitHub based on events that happen on GitHub." (For example, post to a chat service when an issue is opened.)
- **OAuth apps:** an older way, where a user authorises an app to act with the user's scopes.

The documentation's advice: "In general, GitHub Apps are preferred over OAuth apps. GitHub Apps use fine-grained permissions, give the user more control over which repositories the app can access, and use short-lived tokens. These properties can harden the security of the app by limiting the damage that could be done if the app's credentials were leaked." This is Chapter 63's least-privilege principle applied to integrations. An app is also not tied to a person, so it keeps working if someone leaves (Chapter 68).

Apps receive **webhooks** for their events, and call the API with tokens that GitHub issues to them. Building one is a software project of its own; the documentation has guides for it.

74.7 Tokens for scripts

1. **Inside a workflow**, use the built-in `GITHUB_TOKEN` (Chapter 60) with the smallest permissions; it is what the documentation recommends before creating any token.
 2. **From your computer**, prefer `gh` (Chapter 73), which keeps your login, over pasting tokens into scripts.
 3. **For an integration**, prefer a GitHub App.
 4. **If you must use a personal access token:** choose the fewest scopes, an expiry, store it in a secret manager or a secret, never in the repository, and revoke it when it is no longer needed (Chapter 33).
 5. **Handle failures:** check the status code, respect rate limits, and log without printing tokens.
-

Checkpoint

What You Learned

- GitHub has a REST API (fixed structures at resource addresses) and a GraphQL API (ask for the fields you want).
- Large results are paginated; follow the `Link` header's next until it is absent.
- Pin an API version; watch rate-limit headers and back off.
- Webhooks push events to your URL; verify the `X-Hub-Signature-256` HMAC with a constant-time comparison.
- Prefer GitHub Apps (fine-grained, short-lived tokens) to OAuth apps; use `GITHUB_TOKEN` in workflows.

New Vocabulary

API, REST, GraphQL, webhook, HMAC, rate limit, GitHub App, OAuth app, pagination (see the glossary).

Commands Learned

`curl --include`, `openssl dgst -sha256 -hmac`, `python3 -c` helpers as in the recordings.

Common Mistakes

1. **Trusting a webhook without verifying its signature.**
2. **Comparing signatures with `==`.**
3. **Ignoring pagination and using only the first page.**
4. **Not pinning an API version.**
5. **Hard-coding a token in a script.**
6. **Publishing raw headers or logs.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the difference between REST and GraphQL in one sentence?
2. What does the `Link` header contain?
3. What does a webhook signature prove?
4. Why is `==` a poor way to compare signatures?
5. Why are GitHub Apps preferred to OAuth apps?

Before Moving On

You are ready for Chapter [75](#) if you can:

- [] follow next links to fetch all pages
- [] verify a webhook signature
- [] choose a token type for a script

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|-------------------------|
| The HMAC signature values, the rejection of a changed payload, and the reading of a <code>Link</code> header | Locally tested: Bash 5.2 and zsh 5.9, Python 3, OpenSSL; CI | R342, R343 (local side) |

| Claim | Evidence class | Ledger |
|---|--|-----------|
| Observations of one paginated endpoint, the default version header, a 400 for a bogus version | Observed live once on 2026-09-30 from the authoring environment, through an authenticating proxy; not re-run in CI; rate-limit values not quoted | R342 |
| REST vs GraphQL, pagination, versions, rate limits, webhooks, apps | Checked against github/docs (commit 2eaab0b); not otherwise run | R341-R343 |
| The 410 response for a formerly supported version; replay protection advice | Not observed / not checked | none |

Where this leads

Chapter 75 covers the remaining platform features: Sponsors, the Marketplace, custom properties and governance.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 and 2.2 need Python 3 or OpenSSL and no account or network. **Use only made-up secrets. Never paste a real token into a terminal, a script or a message.**

Level 1 — Guided

1.1 Two APIs. Explain in one sentence each what REST and GraphQL are and one difference.

1.2 The header. A response contains Link: <https://example.org/items?page=3>; rel="next". What does it tell a script?

Level 2 — Partially guided

2.1 A signature. Using the documentation's test values (secret It's a Secret to Everybody, payload Hello, World!), compute the HMAC-SHA-256 of the payload with `openssl dgst -sha256 -hmac`. Compare it with the value in the chapter.

2.2 Tamper. Change one character of the payload and compute again. Do the two signatures look similar or completely different? What does that tell you?

Level 3 — Independent

3.1 A pager. Write a short script (any language) that, given the text of a Link header, prints the address of the next page or none.

3.2 Constant time. Explain why the documentation says never to use a plain `==` to compare signatures, and name the function you would use in a language you know.

Level 4 — Professional scenario

4.1 Choose. Your team wants a service that comments on pull requests in every repository of the organization. Compare a personal access token, an OAuth app and a GitHub App using the chapter, and recommend one.

Level 5 — Troubleshooting

5.1 Your script gets 30 items when the repository has hundreds. What is wrong and how do you fix it?

5.2 Your webhook handler accepts a request but the signature check fails on every delivery, including genuine ones. List three things to check.

Chapter 75 — Other Platform Features [Deep]

In this chapter

- GitHub Sponsors and the sponsor button (`FUNDING.yml`)
- GitHub Marketplace
- AI assistance in the workflow, described neutrally
- custom properties and governance
- how to keep up with a platform that keeps changing

How to read this chapter. This is a **Deep** chapter and can be read later. It is a tour of features you may meet, not a guide to using them. Statements about GitHub come from GitHub's documentation (github/docs at commit 2eaab0b, 29 September 2026) and **were not run on a live account**; no sponsorship, listing, subscription or setting was created. **Prices, eligibility, supported regions and availability by plan change and are not stated.** The one recording is local: it checks a `FUNDING.yml` file written from the documentation's example. This book takes **no position** on AI tools, sponsorship or any product: it says what the documentation says, and what you must decide for yourself.

Before you start. Chapter [68](#).

75.1 GitHub Sponsors

GitHub's documentation says that anyone can sponsor open-source maintainers on GitHub: "Anyone in any region can sponsor eligible maintainers, but you must reside in a supported region to receive funds." When you become a sponsored developer or sponsored organization, "additional terms" apply, and a matching programme exists with its own conditions, for example that "payments to sponsored organizations and payments from organizations are not eligible". **The details of eligibility, regions, fees and matching are on GitHub's pages and change**; read them before you rely on Sponsors for income or plan a budget around it.

The sponsor button. A repository can show a **Sponsor** button. The documentation: "You can configure your sponsor button by editing a `FUNDING.yml` file in your repository's `.github` folder, on the default branch." Rules stated there: "one username, package name, or project name per external funding platform and up to four custom URLs", and "one organization and up to four sponsored developers" on GitHub Sponsors. Each platform goes on its own line. Anyone with admin permissions can enable the button in the repository settings, and you can also set a default for your organization or account (Chapter [64](#) on default community health files).

Funding links are for supporting open-source projects: the documentation says they are not supported for other purposes such as advertising or for political, community or charity groups.

The recording writes the documentation's example file and reads it with PyYAML (Chapter [53](#)), counting entries per key:


```
$ mkdir -p .github
$ printf 'github: [octocat, surftocat]\npatreon: octocat\ntidelift: npm/octo-
package\ncustom: ["https://www.paypal.me/octocat", octocat.com]\n' > .github/FUNDING.yml
$ cat .github/FUNDING.yml
github: [octocat, surftocat]
patreon: octocat
tidelift: npm/octo-package
custom: ["https://www.paypal.me/octocat", octocat.com]
$ python3 fund.py .github/FUNDING.yml
github 2
patreon 1
tidelift 1
custom 2
problems: none
```

Recorded in Bash; *ch75-platformextras/expected-funding.bash.txt*.

github has two entries, custom has two, and the other keys one each, so the file respects the stated limits. The script does **not** contact GitHub and does not know which platforms GitHub supports: it only counts.

Now two variations:

```
$ printf 'custom: [https://www.paypal.me/octocat, octocat.com]\n' > unquoted.yml
$ python3 fund.py unquoted.yml
custom 2
problems: none
$ printf 'custom: [a.example, b.example, c.example, d.example, e.example]\n' > toomany.yml
$ python3 fund.py toomany.yml
custom 5
problems: ['more than four custom URLs']
```

Recorded in Bash; *ch75-platformextras/expected-funding.bash.txt*.

- The unquoted URL was **accepted** by PyYAML 6.0.1, although the documentation notes that "If a custom URL in an array includes :, you must wrap the URL in quotes". The documentation's rule is stricter than this parser. **Follow the documentation:** the parser that matters is GitHub's, and a rule that one tool tolerates can break in another. (Chapter 53 showed PyYAML reading no as false, another case where one tool's reading is not the whole story.)
- The five custom URLs were flagged by the script, which mirrors the documented limit of four.

Checked against GitHub's documentation (R344) and locally tested. "About GitHub Sponsors" and "Displaying a sponsor button in your repository". The list of supported funding platforms in the documentation is not reproduced here.

75.2 GitHub Marketplace

The **Marketplace** is a directory of tools that extend GitHub's workflows. The documentation: it "connects you to developers who want to extend and improve their GitHub workflows. You can list free and paid tools for developers to use in GitHub." It has two kinds of listing: **GitHub Actions** (Chapter 54) and **apps** (Chapter 74).

Points from the documentation about publishing: "Anyone can share their apps with other users for free on GitHub Marketplace but only apps owned by organizations can sell their app." Paid plans need "publisher verification" of the organization and a financial onboarding process. Free listings need to meet the general requirements for listing.

For a **user** of the Marketplace, the rules of Chapter 60 and Chapter 74 apply: an entry in the Marketplace is not a guarantee of safety. Before you install an app, look at the permissions it asks for (least privilege), who publishes it, and whether the publisher is verified; before you use an action, pin it to a commit and read its source.

Checked against GitHub's documentation (R345). "About GitHub Marketplace for apps".

75.3 AI assistance in the development workflow

GitHub documents an AI assistant, Copilot, as part of the platform. **This section describes what the documentation says and gives no recommendation.** The documentation describes it as "an AI assistant that helps you write, understand, and ship software" that "suggests code as you type, answers questions about a codebase, reviews your changes, and works on tasks you assign it", working "within the development workflow you already use": your code, issues and pull requests, and repository automations. It groups the capabilities into categories, among them **assistive** (it suggests and you "review and apply each suggestion") and **agentic** (you describe a goal and it "can work through multiple steps", with the statement "You remain responsible for reviewing and approving"), plus **customisations** such as instructions and reusable prompts.

What does not change, whatever tool produced a change:

- **A person is accountable** for what is merged. Review (Chapter 46) and branch protection (Chapter 50) apply to every pull request, including one that a tool opened.
- **Licences and provenance still matter** (Chapter 65): do not submit what you do not have the right to submit, and do not submit what you cannot explain (Chapter 64).
- **Secrets stay out of prompts, code and logs** (Chapter 33).
- **Least privilege** applies to any tool's access to your repositories (Chapter 63).
- **Tests and checks** (Chapter 52) are the same for human and machine changes.

Plans, features, limits and policies for these tools differ by account and change often; read the current documentation and your organization's policy.

Checked against GitHub's documentation (R346). "About GitHub Copilot" (first sections only). The claims about Copilot were not checked by using it. This book as a whole was written with a different AI assistant, Claude, working with the author; the Preface and Appendix N say how.

75.4 Custom properties and governance

In an organization with many repositories, you need to organise and control them. **Custom properties** are one tool. The documentation: "With custom properties, you can add metadata to repositories in your organization. You can use those properties to target repositories with rulesets." Organization owners can define the properties (and, where the plan supports it, people with a specific permission). Other points: the visibility of a property matches the repository's visibility, so on a public repository anyone can see its custom properties; a REST API endpoint exists for managing them; and properties can be synced with an external system.

A small example of the idea: define a property such as "team" or "criticality", set it on each repository, then write a ruleset (Chapter 50) that applies to all repositories with `criticality = high`. That gives **one rule for many repositories**.

Governance means the decisions and rules by which a group runs its projects. On GitHub they are made of parts you have already met:

| Governance question | Where it lives |
|---|---|
| Who may do what? | repository roles and base permissions (Chapter 68) |
| Which changes may merge, and after what? | branch protection and rulesets (Chapter 50) |
| Who reviews which files? | code owners (Chapter 50) |
| How do we behave? | code of conduct (Chapter 64) |
| How do we contribute and report problems? | CONTRIBUTING.md, SECURITY.md, templates (Chapters 64, 62) |
| Which licence? | LICENSE (Chapter 65) |
| What is scanned and blocked? | security features (Chapter 62) |
| Who can be reached in an emergency? | ownership with more than one owner (Chapter 68) |

Write the rules down in the repository, so that they are versioned and reviewed like code.

Checked against GitHub's documentation (R347). "Managing custom properties for repositories in your organization". Rulesets that target properties were not examined in detail.

75.5 Keeping up with a platform that changes

This whole Part depends on a platform that changes faster than a book. Habits that help:

1. **Read the source of truth:** GitHub's documentation, with the version for your plan selected (Chapter 35).
 2. **Read the changelog** for features you depend on.
 3. **Test on a repository you can afford to break.**
 4. **Keep notes** of what you observed and when, as this book does in its ledger: a fact without a date is a rumour.
 5. **Prefer principles** (least privilege, verify before trusting, keep secrets out, write it down) to memorised screens.
-

Checkpoint

What You Learned

- Sponsors: anyone can sponsor; receiving funds depends on region and terms; the sponsor button is configured with `.github/FUNDING.yml`, with documented limits.
- A tool can accept what documentation forbids (unquoted URL): follow the documentation.
- The Marketplace lists actions and apps; only organizations can sell apps; a listing is not a safety guarantee.
- AI tools do not change accountability, licensing, secrets or review rules.
- Custom properties add metadata to repositories and can target rulesets; governance is written down in the repository.

New Vocabulary

GitHub Sponsors, custom property, governance (see the glossary).

Commands Learned

No new commands.

Common Mistakes

1. **Relying on Sponsors income without reading the current terms.**
2. **Trusting a Marketplace listing without checking permissions.**
3. **Merging a machine-made change without review.**
4. **Assuming a parser's tolerance equals the platform's.**
5. **Keeping governance in people's heads.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Where does `FUNDING.yml` live?
2. How many custom URLs does the documentation allow?
3. Who can sell an app on the Marketplace?
4. Does an AI tool change who is accountable for a merged change?
5. What can custom properties be used for?

Before Moving On

You are ready for Part XI if you can:

- [] explain what each feature in this chapter is for, without assuming it is right for you
- [] list where a project's governance rules live

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|-------------------|
| Counting entries in <code>FUNDING.yml</code> ; PyYAML accepts an unquoted URL | Locally tested: Bash 5.2 and zsh 5.9, Python 3, PyYAML 6.0.1; CI | R344 (local side) |
| Sponsors, Marketplace, AI assistant description, custom properties | Checked against <code>github/docs</code> (commit <code>2eaab0b</code>); not run on a live account ; eligibility, prices and plans not stated | R344-R347 |
| Governance table | The author's summary of earlier chapters | none |

Where this leads

Part XI, "Master Projects", puts everything together: Chapter 76 has twelve professional scenarios, and Chapter 77 is the capstone.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Exercises 2.1 and 2.2 use your terminal only. **None asks you to sign up for anything or to spend money.**

Level 1 — Guided

1.1 Where? Where does `FUNDING.yml` live, and on which branch must it be?

1.2 Limits. State the two limits on the sponsor button that the chapter mentions.

Level 2 — Partially guided

2.1 Write it. In a sandbox repository, write a `.github/FUNDING.yml` with one GitHub username (a made-up one), one made-up Patreon name and one quoted custom URL, and read it back.

2.2 Count. Parse the file with a YAML parser and count the entries per key.

Level 3 — Independent

3.1 A rule the parser did not enforce. Explain why the chapter tells you to follow the documentation's quoting rule even though the parser accepted an unquoted URL.

3.2 A property plan. Design two custom properties for an organization with 40 repositories and say which ruleset each would help you write.

Level 4 — Professional scenario

4.1 A policy for tools. Write five lines for your team's contribution guide about changes that were made with an AI tool, using the chapter. Do not assume the tool is good or bad.

Level 5 — Troubleshooting

5.1 The Sponsor button does not appear. List three things to check from the chapter.

5.2 An app from the Marketplace asks for permissions that seem too broad. What do you do before installing it?

Part XI — Master Projects

Chapter 76 — Professional Scenarios 1-12 [Core]

In this chapter

- twelve situations you will meet at work, each solved step by step
- the safe way and the dangerous way for each
- a habit: read the state before and after every change

How to read this chapter. Every scenario was **run and recorded** in Bash and zsh on Git 2.43.0 (and re-run in CI on newer Git). The "remote" is a local bare repository standing in for GitHub, and the "colleague" or "maintainer" is a second clone; on GitHub, the steps that a person clicks (a merge, a review) are described in words, from the earlier chapters. **Nothing here ran on GitHub.** Commit names shown in the outputs are stable only because the recordings use fixed dates; on your computer they will differ. All names and prices are made up; the API key in scenario 8 is a made-up value.

Before you start. Chapters [45](#), [60](#), [66](#) and [26](#), and Part III and IV of the book.

How each scenario is laid out. Situation, Goal, the recorded Steps, What happened (read the output line by line), and If it goes wrong.

Scenario 1. Fix a bug on a branch and get it merged

Situation. The menu lists rolls at the wrong price. You have no permission to change main directly; the team merges through pull requests (Chapter [45](#)).

Goal. Make the fix on a topic branch, publish it, and see it merged and cleaned up.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ git switch -q -c fix-roll-price
$ sed -i 's/Rolls: 3.00/Rolls: 3.20/' menu.md
$ git commit -q -am "Correct the price of rolls"
$ git push -q -u origin fix-roll-price
$ git log --format=%s main..fix-roll-price
Correct the price of rolls
$ git clone -q ../hub.git ../maintainer
$ git -C ../maintainer -c user.name=Maintainer -c user.email=maintainer@example.org merge -q --no-ff -m "Merge fix-roll-price" origin/fix-roll-price
$ git -C ../maintainer push -q origin main
$ git fetch -q
$ git switch -q main
$ git merge -q --ff-only origin/main
$ git branch -d fix-roll-price
Deleted branch fix-roll-price (was cd06e61).
$ git push -q origin --delete fix-roll-price
```



```
$ git log --format=%s
Merge fix-roll-price
Correct the price of rolls
Add the menu
```

Recorded in Bash; *ch76-scenarios/expected-sc-bugfix.bash.txt*.

What happened. The first `git log` line proves the branch holds exactly one new commit. The second clone plays the maintainer's part: on GitHub the maintainer would click **Merge**, and this local merge with `--no-ff` records the same kind of merge commit (Chapter 21). Back on your side, `--ff-only` moves `main` forward without inventing a new commit, and `git branch -d` refuses to delete a branch that is not merged, which is why it is the safe deletion.

If it goes wrong. If `git branch -d` refuses, do not use `-D` at once: check that the work really is merged (`git log main..branch`). If the push is rejected, fetch and look before forcing anything (Chapter 23).

Scenario 2. Correct your own last commit before anyone has it

Situation. You committed with a typo in the message and a wrong value, and you have not pushed.

Goal. Rewrite the last commit safely, then push.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ sed -i 's/Rolls: 3.00/Rolls: 3.2/' menu.md
$ git commit -q -am "Corect the price of rolls"
$ git log -1 --format=%s
Corect the price of rolls
$ sed -i 's/Rolls: 3.2$/Rolls: 3.20/' menu.md
$ git commit -q --amend -a -m "Correct the price of rolls"
$ git log --format=%s
Correct the price of rolls
Add the menu
$ git push -q origin main
$ git status -sb
## main...origin/main
```

Recorded in Bash; *ch76-scenarios/expected-sc-amend.bash.txt*.

What happened. `--amend` replaced the last commit with a new one that has the corrected message and content. Because nothing was pushed, nobody else holds the old commit, so rewriting is safe (Chapter 25). After the push the status shows `main...origin/main` with no difference.

If it goes wrong. Never amend a commit that others already have: your rewritten commit and theirs would differ, and a forced push would disturb them (Chapter 27).

Scenario 3. Undo a bad change that is already on the shared branch

Situation. A wrong price was pushed to main and others have already fetched it.

Goal. Undo it in a way that keeps history honest and does not disturb anyone.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ sed -i 's/White loaf: 2.50/White loaf: 25.00/' menu.md
$ git commit -q -am "Change the loaf price"
$ git push -q origin main
$ git revert --no-edit HEAD
[main 2fc2497] Revert "Change the loaf price"
Date: Mon Jan 5 09:13:00 2026 +0000
1 file changed, 1 insertion(+), 1 deletion(-)
$ git push -q origin main
$ git log --format=%s
Revert "Change the loaf price"
Change the loaf price
Add the menu
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.50
- Rolls: 3.00
```

Recorded in Bash; *ch76-scenarios/expected-sc-revert.bash.txt*.

What happened. `git revert` made a **new commit** that undoes the earlier one, so the history reads: the change, then its reversal. The file is back to its old content, and the push is an ordinary one because history only grew (Chapter 25).

If it goes wrong. The dangerous alternative is to reset `main` and force-push, which rewrites what others already have. Use it only on your own private branches.

Scenario 4. Get back a branch you deleted

Situation. You deleted a branch with `-D` and then realised it held your only copy of a commit.

Goal. Find the lost commit through the reflog and give it a branch again.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ git switch -q -c tea-idea
$ printf -- "- Tea: 1.50\n" >> menu.md
$ git commit -q -am "Add tea"
$ git switch -q main
$ git branch -D tea-idea
Deleted branch tea-idea (was 11bdfb7).
```

```
$ git branch
* main
$ sha=$(git reflog --format='%H %gs' | grep 'commit: Add tea' | awk '{print $1}')
$ git branch tea-idea "$sha"
$ git log -1 --format=%s tea-idea
Add tea
```

Recorded in Bash; *ch76-scenarios/expected-sc-recover.bash.txt*.

What happened. `git branch -D` removed the name, but the commit still existed, and the reflog (Chapter 26) recorded it when it was made. The script looked up its full name by the message and created a new branch there. The last line proves the commit is back.

If it goes wrong. The reflog is local and expires (Chapter 26): recover soon. If you never committed the work, the reflog cannot help.

Scenario 5. Two people changed the same line

Situation. Ada and Ben each changed the price of rolls, differently. Ada pushed first.

Goal. Combine the two changes deliberately and publish the result.

Steps.

```
$ git clone -q hub.git ada
$ git clone -q hub.git ben
$ cd ada
$ sed -i 's/Rolls: 3.00/Rolls: 3.20/' menu.md
$ git commit -q -am "Rolls cost 3.20"
$ git push -q origin main
$ cd ../ben
$ sed -i 's/Rolls: 3.00/Rolls: 3.50/' menu.md
$ git commit -q -am "Rolls cost 3.50"
$ git fetch -q
$ git -c advice.mergeConflict=false merge origin/main
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
Automatic merge failed; fix conflicts and then commit the result.
$ cat menu.md
# Sunrise Bakery menu

- White loaf: 2.50
<<<<<< HEAD
- Rolls: 3.50
=====
- Rolls: 3.20
>>>>>> origin/main
$ printf "# Sunrise Bakery menu\n\n- White loaf: 2.50\n- Rolls: 3.30\n" > menu.md
$ git add menu.md
$ git commit -q --no-edit
$ git log --format=%s
Merge remote-tracking branch 'origin/main'
Rolls cost 3.50
```

```
Rolls cost 3.20
Add the menu
$ git push -q origin main
```

Recorded in Bash; *ch76-scenarios/expected-sc-conflict.bash.txt*.

What happened. Ben's merge stopped with a conflict and left markers in the file (Chapter 22). Ben decided on a value that suits both, wrote the file without markers, ran `git add` to say it was resolved, and committed; the merge commit records both parents. `advice.mergeConflict=false` only hides Git's hint text so that the output is the same on every version.

If it goes wrong. The most important step is the decision, not the commands: talk to the other author when the meaning is unclear. If you get lost, `git merge --abort` returns to the state before the merge.

Scenario 6. Prepare a contribution for someone else's project

Situation. You want to add tea to a project's menu. The project's guide asks for a passing check and a sign-off (Chapter 64).

Goal. Branch, change, check, sign off and publish the branch, ready for a pull request.

Steps.

```
$ git clone -q hub.git fork
$ cd fork
$ git switch -q -c add-tea
$ printf -- "- Tea: 1.50\n" >> menu.md
$ sh check.sh
check: ok
$ git commit -q -s -am "Add tea to the menu"
$ git log -1 --format=%B
Add tea to the menu

Signed-off-by: Ada Learner <ada@example.org>

$ git push -q -u origin add-tea
$ git ls-remote --heads origin
b5895b45037f31bbdf8262b45c73140c2d40747d refs/heads/add-tea
75f410ae298df20e7637e6c4692195c081e54673 refs/heads/main
```

Recorded in Bash; *ch76-scenarios/expected-sc-contribute.bash.txt*.

What happened. The check script ran before the commit. `-s` added the Signed-off-by line (text, not a signature; Chapter 64). `git ls-remote` shows the two branches on the remote; the full names are commit hashes, which are the same on every machine because the recordings use fixed dates. On GitHub you would now open a pull request from `add-tea` (Chapter 45).

If it goes wrong. If the project asks for a different flow (for example a fork), follow its guide; the rule is always the same: small change, tests first, sign-off only if you have the right to submit.

Scenario 7. Tidy messy commits before asking for review

Situation. Your branch has three commits called `WIP`, `more` and `fix typo`. Reviewers will read them.

Goal. Turn them into one meaningful commit, without an interactive editor.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ git switch -q -c menu-update
$ printf -- "- Tea: 1.50\n" >> menu.md
$ git commit -q -am "WIP"
$ printf -- "- Coffee: 2.00\n" >> menu.md
$ git commit -q -am "more"
$ printf -- "- Scone: 2.20\n" >> menu.md
$ git commit -q -am "fix typo"
$ git log --format=%s main..HEAD
fix typo
more
WIP
$ git reset -q --soft main
$ git commit -q -m "Add tea, coffee and scones to the menu"
$ git log --format=%s main..HEAD
Add tea, coffee and scones to the menu
```

Recorded in Bash; *ch76-scenarios/expected-sc-squash.bash.txt*.

What happened. `git reset --soft main` moved the branch pointer back to `main` while **keeping all the changes staged**, and one new commit replaced the three. This is the same result as squashing in an interactive rebase (Chapter 27) with fewer moving parts.

If it goes wrong. Do this only on a branch that nobody else has fetched. If you pushed it, a forced push is needed and changes what others see: use `--force-with-lease` (Chapter 27) and tell them.

Scenario 8. A secret was committed by mistake

Situation. A file with an API key (a made-up one here) was committed. Nothing has been pushed.

Goal. Stop tracking the file, keep it out in future, and understand what remains.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ printf 'API_KEY=not-a-real-key\n' > .env
$ git add .env
$ git commit -q -m "Add configuration"
$ git rm -q --cached .env
$ printf '.env\n' > .gitignore
$ git add .gitignore
```

```
$ git commit -q -m "Stop tracking .env"
$ git ls-files
.gitignore
check.sh
menu.md
$ git log --format=%s -S'not-a-real-key'
Stop tracking .env
Add configuration
$ git show --format=%s HEAD~1:.env
API_KEY=not-a-real-key
```

Recorded in Bash; *ch76-scenarios/expected-sc-secret.bash.txt*.

What happened. The commands removed the file from tracking and added it to `.gitignore`. But the search (`-S`) still finds the key in **two commits**, and `git show` prints it from history. Removing a file in a later commit does not remove it from earlier ones (Chapter 33).

If it goes wrong. Revoke the real key first, before any clean-up: assume anyone who could see the repository has seen it. Only then rewrite history if you must (Chapter 33 and Chapter 63), and remember that forks and clones keep the old commits.

Scenario 9. Release a fix while new work is in progress

Situation. Version 1.0.0 is out, `main` already contains work for the next release, and a price bug must be fixed in the released version.

Goal. Fix it from the release tag, tag 1.0.1, and carry the fix to `main`.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ git tag -a v1.0.0 -m "Version 1.0.0"
$ printf "Specials for the next release\n" > specials.md
$ git add specials.md
$ git commit -q -m "Add specials (next release)"
$ git switch -q -c hotfix-1.0 v1.0.0
$ sed -i 's/Rolls: 3.00/Rolls: 3.20/' menu.md
$ git commit -q -am "Fix the price of rolls"
$ git tag -a v1.0.1 -m "Version 1.0.1"
$ git switch -q main
$ git cherry-pick -x hotfix-1.0 > /dev/null
$ git tag --sort=version:refname
v1.0.0
v1.0.1
$ git log --format=%s
Fix the price of rolls
Add specials (next release)
Add the menu
```

Recorded in Bash; *ch76-scenarios/expected-sc-hotfix.bash.txt*.

What happened. The hotfix branch started at the **tag**, so it holds none of the next release's work. After tagging v1.0.1 the script copied the fix onto main with `cherry-pick -x` (Chapters 29 and 27), which records where the commit came from. The final log shows the fix and the specials on main, and the tags are sorted by version (Chapter 66).

If it goes wrong. If the cherry-pick conflicts, resolve it like a merge. Never move a published tag; publish a new patch version.

Scenario 10. Find when a wrong value entered the file

Situation. Rolls cost 30.00 in the menu, and nobody knows since when.

Goal. Find the commit that added that text, and see what it changed.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ sed -i 's/Rolls: 3.00/Rolls: 30.00/' menu.md
$ git commit -q -am "Update rolls"
$ printf -- "- Tea: 1.50\n" >> menu.md
$ git commit -q -am "Add tea"
$ git log --format=%s -S'Rolls: 30.00'
Update rolls
$ git show --stat --format=%s HEAD~1
Update rolls

menu.md | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git diff HEAD~2 HEAD~1
diff --git a/menu.md b/menu.md
index 9a3ace1..1509dce 100644
--- a/menu.md
+++ b/menu.md
@@ -1,4 +1,4 @@
# Sunrise Bakery menu

- White loaf: 2.50
-- Rolls: 3.00
+- Rolls: 30.00
```

Recorded in Bash; *ch76-scenarios/expected-sc-pickaxe.bash.txt*.

What happened. `git log -S'text'` lists the commits that changed how many times the text occurs, so the first result is where it appeared (Chapter 17). `git show --stat` and `git diff` show the change. `bisect` (Chapter 28) is the alternative when you can test each version automatically.

If it goes wrong. The search finds where text was added or removed, not why. Read the commit message and, on GitHub, the linked pull request.

Scenario 11. Interrupted by an urgent fix

Situation. You are halfway through an edit when a colleague asks for a quick fix.

Goal. Put the half-done work aside, fix, and come back.

Steps.

```
$ git clone -q hub.git work
$ cd work
$ printf -- "- Tea: 1.50\n" >> menu.md
$ git stash push -q -m "tea in progress"
$ git status --short
$ git switch -q -c fix-typo
$ sed -i 's/White loaf/White loaves/' menu.md
$ git commit -q -am "Fix the name of the loaf"
$ git switch -q main
$ git stash list | grep -c .
1
$ git stash pop -q
$ git status --short
M menu.md
$ git log --format=%s --all
Fix the name of the loaf
Add the menu
```

Recorded in Bash; *ch76-scenarios/expected-sc-stash.bash.txt*.

What happened. `git stash push` saved the uncommitted change and cleaned the working tree (status printed nothing). After the fix on its own branch, `stash pop` brought the unfinished work back (Chapter 24).

If it goes wrong. A stash is local and easy to forget: give it a message, list it with `git stash list`, and do not leave work there for weeks.

Scenario 12. Review a colleague's change on your own computer

Situation. A colleague pushed `add-tea`. You want to read and try it before approving.

Goal. Fetch the branch, read what changed, run the check, and go back.

Steps.

```
$ git clone -q hub.git author
$ cd author
$ git switch -q -c add-tea
$ printf -- "- Tea: 1.50\n" >> menu.md
$ git commit -q -am "Add tea"
$ git push -q -u origin add-tea
$ cd ..
$ git clone -q hub.git reviewer
$ cd reviewer
$ git fetch -q origin
```



```
$ git log --format='%an: %s' main..origin/add-tea
Ada Learner: Add tea
$ git diff --stat main...origin/add-tea
menu.md | 1 +
1 file changed, 1 insertion(+)
$ git switch -q --detach origin/add-tea
$ sh check.sh
check: ok
$ tail -1 menu.md
- Tea: 1.50
$ git switch -q main
```

Recorded in Bash; *ch76-scenarios/expected-sc-review.bash.txt*.

What happened. Nothing was merged. `fetch` brought the branch, `log` and `diff --stat main...origin/add-tea` showed what it adds relative to `main` (the three dots compare from the common ancestor; Chapter 17). `switch --detach` let you look at the branch without creating a local branch, and the check script ran on that version (Chapter 46).

If it goes wrong. Do not run code from a stranger's branch that you have not read (Chapter 60). Write review comments on the pull request, not only in your head.

Checkpoint

What You Learned

- The safe tool depends on whether others already have the commit: `amend`, `squash` and `reset` for private work; `revert` for shared work.
- The `reflog` and `pickaxe` search find what seems lost or hidden.
- Conflicts need a decision, not just commands; `--abort` is available for as long as the merge or rebase is still in progress.
- A committed secret stays in history: `revoke` it first.
- Hotfixes start from the release tag, and fixes are carried across with `cherry-pick -x`.
- Reading the state (`status`, `log`, `diff`) before and after each step is the habit that prevents most mistakes.

New Vocabulary

No new terms.

Commands Learned

No new commands; this chapter combines the earlier ones.

Common Mistakes

1. **Rewriting shared history** when a `revert` would do.
2. **Skipping the read-before-and-after habit.**
3. **Treating the removal of a secret from the latest commit as enough.**

4. **Moving a published tag** instead of releasing a new version.
5. **Running unread code from someone else's branch.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Which scenarios use a forced push, and why do none of the recorded ones need it?
2. Why is `git revert` right for a shared bad commit?
3. Where does the recovery in scenario 4 get the commit's name from?
4. What does `--soft` keep when you reset?
5. Why start a hotfix from the tag?

Before Moving On

You are ready for Chapter 77 if you can:

- [] choose amend, revert, squash or reset for a given situation
- [] recover a deleted branch
- [] explain what remains after removing a secret in a later commit

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|---|--------|
| All twelve recorded sessions | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git | R348 |
| What a maintainer would do on GitHub (merge, review) | Described from earlier chapters; not run on GitHub | none |

Where this leads

Chapter 77 joins these skills in one long simulation.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Work in a sandbox folder with a local bare repository as the remote, as in the recordings. **Do not practise history rewriting on a repository that matters.**

Level 1 — Guided

1.1 Choose the tool. For each situation choose amend, revert, squash (soft reset) or cherry-pick: (a) a typo in your last unpushed commit message; (b) a bad commit already on the shared main; (c) three messy commits on your private branch; (d) a fix on a hotfix branch that main also needs.

1.2 Read first. Which three commands do the scenarios use to read the state before and after a change?

Level 2 — Partially guided

2.1 Scenario 1. Reproduce scenario 1 in your sandbox: fix on a branch, push, merge from a second clone, fast-forward your main, delete the branch locally and on the remote.

2.2 Scenario 4. Reproduce the deleted-branch recovery. What would change if you waited a very long time before recovering?

Level 3 — Independent

3.1 A new scenario. Write your own thirteenth scenario in the same layout (situation, goal, steps, what happened, if it goes wrong) using only commands from the book, and run it.

3.2 Conflict by design. Create a conflict on purpose, resolve it, and then create it again and abort the merge. Compare the two histories.

Level 4 — Professional scenario

4.1 A leaked key. A colleague says: "I deleted the key file in a new commit, so we are safe." Write your reply, in order of actions, using scenario 8.

Level 5 — Troubleshooting

5.1 `git branch -d` says the branch is not fully merged, but you believe it is. Give two ways to check, and say when `-D` is acceptable.

5.2 After a hotfix cherry-pick, main still shows the old price. List three causes.

Chapter 77 — Capstone Project [Core]

In this chapter

- **Project 12:** a 23-step professional simulation, from an empty repository to a released, audited project
- every step recorded, and what to do on GitHub at each step
- a checklist to assess yourself

How to read this chapter. This is the last project. It joins everything from Parts III to X. **The 23 steps were run and recorded** in Bash and zsh on Git 2.43.0 and are re-run in CI on newer Git. A local bare repository stands in for GitHub, and the "colleagues" are extra clones. **Nothing was run on GitHub:** the right-hand column of each step says what to do on GitHub, from the earlier chapters, and doing it needs your own account (a step the book's author has not yet done on a live account). Names and prices are made up; the token in step 21 is a made-up value. **The project has no licence, because that is the owner's decision** (Chapter 65); when you do this for real, choose one.

Before you start. Chapter 76 and the chapters it links. Set aside two to three hours. Type the commands yourself, in a sandbox folder.

How to use the recording. Read a step, run its commands in your sandbox, compare your output with the recording (commit names will differ), then do the GitHub column if you have an account.

Step 1. Create the shared repository and your clone

Step 1 creates the shared repository and your working copy. Here a bare repository stands in for GitHub.

```
$ echo "== Step 1: Create the shared repository and your clone"
== Step 1: Create the shared repository and your clone
$ git init -q --bare hub.git
$ git -C hub.git symbolic-ref HEAD refs/heads/main
$ git clone -q hub.git bakery 2>/dev/null
$ cd bakery
$ git status -sb
## No commits yet on main...origin/main [gone]
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Create an empty repository (Chapter 39) and clone it; do not let the website add files yet, so that your first push is simple.

Step 2. First files, tests and the ignore file

Files, tests and an ignore file. The tests ran and printed OK; generated files are ignored (Chapter 67).

```
$ echo "== Step 2: First files, tests and the ignore file"
== Step 2: First files, tests and the ignore file
$ mkdir -p src tests
$ printf 'def total(prices):\n    return round(sum(prices), 2)\n' > src/orders.py
$ printf 'import unittest\nfrom orders import total\n\nclass\nTotalTest(unittest.TestCase):\n    def test_total(self):\n\nself.assertEqual(total([2.5, 3.0]), 5.5)\n\n    def test_empty(self):\n\nself.assertEqual(total([]), 0)\n' > tests/test_orders.py
$ printf '__pycache__/\n.env\n' > .gitignore
$ PYTHONPATH=src python3 -m unittest discover -s tests 2>&1 | grep -x OK
OK
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Nothing to do on GitHub yet.

Step 3. Documentation and community files

A README with an example that is true, CONTRIBUTING.md and SECURITY.md (Chapters 42, 64, 62).

```
$ echo "== Step 3: Documentation and community files"
== Step 3: Documentation and community files
$ printf '# Bakery orders\n\nAdds up the prices of an order.\n\n    from orders import\n    total\n    total([2.5, 3.0]) # 5.5\n' > README.md
$ printf '# Contributing\n\nOpen an issue first. Run sh ci.sh before you open a pull\nrequest.\n' > CONTRIBUTING.md
$ printf '# Security policy\n\nReport vulnerabilities privately to security@example.org.\n'\n' > SECURITY.md
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Later, check the repository's community profile.

Step 4. A local pipeline you can run anywhere

A **pipeline you can run anywhere**: *ci.sh* runs the tests and checks that the required files exist. It reports honestly that no LICENSE exists: **the licence is the owner's decision** (Chapter 65), and this book adds none.

```
$ echo "== Step 4: A local pipeline you can run anywhere"
== Step 4: A local pipeline you can run anywhere
$ printf 'PYTHONPATH=src python3 -m unittest discover -s tests >/dev/null 2>&1 && echo\n"tests: ok" || { echo "tests: FAILED"; exit 1; }&\nfor f in README.md CONTRIBUTING.md\nSECURITY.md .gitignore; do test -f "$f" && echo "present: $f" || { echo "MISSING: $f"; exit\n1; }; done&\ntest -f LICENSE && echo "present: LICENSE" || echo "note: no LICENSE yet (the\nowner decides)"&\n' > ci.sh
$ sh ci.sh
tests: ok
```

```
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
present: .gitignore
note: no LICENSE yet (the owner decides)
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. The same script becomes a workflow step (Chapters 52 and 56); pin actions to commits and set minimal permissions (Chapter 60).

Step 5. First commit and push

The first commit and push. `status -sb` shows main and its remote in step.

```
$ echo "== Step 5: First commit and push"
== Step 5: First commit and push
$ git add -A
$ git commit -q -m "Add the order total, tests, documentation and a local pipeline"
$ git push -q -u origin main
$ git status -sb
## main...origin/main
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Look at the commit list and the files.

Step 6. A server rule that protects the shared branch

A **server rule** that refuses history rewrites on the shared repository. It stands in for branch protection.

```
$ echo "== Step 6: A server rule that protects the shared branch"
== Step 6: A server rule that protects the shared branch
$ git -C ../hub.git config receive.denyNonFastForwards true
$ git -C ../hub.git config --get receive.denyNonFastForwards
true
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Configure a branch protection rule or ruleset (Chapter 50): require a pull request, require the checks, block force pushes.

Step 7. A feature branch and a test that fails first

A feature branch, and a **test that fails first**. FAILED here is the point: the test proves the feature is missing.

```
$ echo "== Step 7: A feature branch and a test that fails first"
== Step 7: A feature branch and a test that fails first
$ git switch -q -c feature/discount
```

```
$ printf '\n    def test_discount(self):\n        self.assertEqual(total([10, 10], 0.1),\n18.0)\n' >> tests/test_orders.py
$ PYTHONPATH=src python3 -m unittest discover -s tests 2>&1 | grep -E "^(OK|FAILED)"
FAILED (errors=1)
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Nothing to do on GitHub yet.

Step 8. Make the test pass and commit

The smallest change that makes the test pass, run through the pipeline, committed and published.

```
$ echo "== Step 8: Make the test pass and commit"
== Step 8: Make the test pass and commit
$ printf 'def total(prices, discount=0):\n    return round(sum(prices) * (1 - discount),\n2)\n' > src/orders.py
$ sh ci.sh
tests: ok
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
present: .gitignore
note: no LICENSE yet (the owner decides)
$ git commit -q -am "Support a discount in total()"
$ git push -q -u origin feature/discount
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Open a pull request from the branch (Chapter 45); link the issue that asked for the feature (Chapter 44).

Step 9. A colleague reviews the branch on their computer

A colleague's view: clone, read who changed what, read the diff statistics, run the pipeline on the branch (Chapter 46).

```
$ echo "== Step 9: A colleague reviews the branch on their computer"
== Step 9: A colleague reviews the branch on their computer
$ git clone -q ../hub.git ../reviewer
$ cd ../reviewer
$ git log --format='%an: %s' main..origin/feature/discount
Ada Learner: Support a discount in total()
$ git diff --stat main...origin/feature/discount
src/orders.py      | 4 ++-
tests/test_orders.py | 3 +++
2 files changed, 5 insertions(+), 2 deletions(-)
$ git switch -q --detach origin/feature/discount
$ sh ci.sh
tests: ok
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
```

```
present: .gitignore
note: no LICENSE yet (the owner decides)
$ git switch -q main
$ cd ../bakery
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. The reviewer uses the Files changed tab and writes comments; the checks appear on the pull request.

Step 10. Review feedback: add a follow-up commit

Feedback becomes a follow-up commit on the same branch; the pull request updates by itself.

```
$ echo "== Step 10: Review feedback: add a follow-up commit"
== Step 10: Review feedback: add a follow-up commit
$ printf '\n    def test_no_discount(self):\n        self.assertEqual(total([10, 10], 0),\n20.0)\n' >> tests/test_orders.py
$ git commit -q -am "Test the case without a discount"
$ git push -q origin feature/discount
$ git log --format=%s main..feature/discount
Test the case without a discount
Support a discount in total()
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Reply to each comment and resolve threads when addressed.

Step 11. Meanwhile, main moved: a second developer changes the docs

Meanwhile main moved: a second developer changed the README.

```
$ echo "== Step 11: Meanwhile, main moved: a second developer changes the docs"
== Step 11: Meanwhile, main moved: a second developer changes the docs
$ git clone -q ../hub.git ../ben
$ git -C ../ben switch -q main
$ printf 'See CONTRIBUTING.md to help.\n\n' | cat - ../ben/README.md > ../ben/README.new &&
mv ../ben/README.new ../ben/README.md
$ git -C ../ben commit -q -am "Point the README to the contributing guide"
$ git -C ../ben push -q origin main
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Nothing to do; this is an ordinary event in shared work.

Step 12. Bring main into your branch and check again

You bring main into your branch, run the pipeline again and push. The merge went through without a conflict because the changes touched different places.


```
$ echo "== Step 12: Bring main into your branch and check again"
== Step 12: Bring main into your branch and check again
$ git fetch -q
$ git merge -q --no-edit origin/main
$ sh ci.sh
tests: ok
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
present: .gitignore
note: no LICENSE yet (the owner decides)
$ git push -q origin feature/discount
$ git log --format=%s -4
Merge remote-tracking branch 'origin/main' into feature/discount
Point the README to the contributing guide
Test the case without a discount
Support a discount in total()
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. The pull request shows whether it can be merged; some teams prefer rebase, which Chapter 27 explains.

Step 13. A conflict: both changed the same line

A real conflict: both changed the rounding line. You chose your version (`--ours`), marked it resolved and committed. In real work you would read both sides and talk to the other author (Chapter 22).

```
$ echo "== Step 13: A conflict: both changed the same line"
== Step 13: A conflict: both changed the same line
$ printf 'def total(prices, discount=0):\n    return round(sum(prices) * (1 - discount),\n2) # rounds to cents\n' > ../ben/src/orders.py
$ git -C ../ben commit -q -am "Comment the rounding"
$ git -C ../ben push -q origin main
$ git fetch -q
$ sed -i 's/(1 - discount), 2)/(1 - discount), 2) # cents/' src/orders.py
$ git commit -q -am "Comment the rounding differently"
$ git -c advice.mergeConflict=false merge origin/main
Auto-merging src/orders.py
CONFLICT (content): Merge conflict in src/orders.py
Automatic merge failed; fix conflicts and then commit the result.
$ git checkout -q --ours src/orders.py
$ git add src/orders.py
$ git commit -q --no-edit
$ sh ci.sh
tests: ok
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
present: .gitignore
note: no LICENSE yet (the owner decides)
$ git push -q origin feature/discount
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Use the conflict editor or the local steps above.

Step 14. Maintainer merges the pull request (squash)

The maintainer's **squash merge** puts all the branch's commits into one commit on main and deletes the remote branch (Chapter 45 on strategies).

```
$ echo "== Step 14: Maintainer merges the pull request (squash)"
== Step 14: Maintainer merges the pull request (squash)
$ git clone -q ../hub.git ../maintainer
$ cd ../maintainer
$ git -c user.name=Maintainer -c user.email=maintainer@example.org merge -q --squash
origin/feature/discount
Squash commit -- not updating HEAD
$ git -c user.name=Maintainer -c user.email=maintainer@example.org commit -q -m "Add
discount support (#1)"
$ git push -q origin main
$ git push -q origin --delete feature/discount
$ cd ../bakery
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Choose Squash and merge on the pull request, if the repository allows it.

Step 15. Clean up and update your main

You update your main with `--ff-only`, prune deleted remote branches and delete your local branch. `-D` was needed because a squash merge leaves the branch's own commits unmerged in Git's eyes; check first that the work is in main.

```
$ echo "== Step 15: Clean up and update your main"
== Step 15: Clean up and update your main
$ git switch -q main
$ git fetch -q --prune
$ git merge -q --ff-only origin/main
$ git branch -D feature/discount
Deleted branch feature/discount (was 14445d8).
$ git log --format=%s
Add discount support (#1)
Comment the rounding
Point the README to the contributing guide
Add the order total, tests, documentation and a local pipeline
$ sh ci.sh
tests: ok
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
present: .gitignore
note: no LICENSE yet (the owner decides)
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Use Delete branch if you did not already.

Step 16. Tag the first release

The first release: an **annotated tag** on the merged commit, pushed (Chapter 66).

```
$ echo "== Step 16: Tag the first release"
== Step 16: Tag the first release
$ git tag -a v0.1.0 -m "Version 0.1.0: initial development"
$ git push -q origin v0.1.0
$ git tag -n1
v0.1.0          Version 0.1.0: initial development
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Create a release from the tag with notes.

Step 17. Package the release and publish a checksum

A source archive with a version prefix, and a **checksum** that is verified (Chapter 63). `sha256sum` is the Linux name.

```
$ echo "== Step 17: Package the release and publish a checksum"
== Step 17: Package the release and publish a checksum
$ git archive --format=tar.gz --prefix=bakery-0.1.0/ -o ../bakery-0.1.0.tar.gz v0.1.0
$ tar -tzf ../bakery-0.1.0.tar.gz | head -4
bakery-0.1.0/
bakery-0.1.0/.gitignore
bakery-0.1.0/CONTRIBUTING.md
bakery-0.1.0/README.md
$ (cd .. && sha256sum bakery-0.1.0.tar.gz > bakery-0.1.0.tar.gz.sha256 && sha256sum -c
bakery-0.1.0.tar.gz.sha256)
bakery-0.1.0.tar.gz: OK
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Attach the archive and the checksum to the release.

Step 18. A bug report: discounts above 100 percent

A bug report becomes a **failing test** first.

```
$ echo "== Step 18: A bug report: discounts above 100 percent"
== Step 18: A bug report: discounts above 100 percent
$ printf '\n    def test_bad_discount(self):\n        with self.assertRaises:\n\n            total([10], 1.5)\n' >> tests/test_orders.py
$ PYTHONPATH=src python3 -m unittest discover -s tests 2>&1 | grep -E "(OK|FAILED)"
FAILED (failures=1)
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. The issue links to the fix through the pull request.

Step 19. Fix it and release 0.1.1

The fix, the pipeline, the commit, the tag `v0.1.1`. `git tag --sort=version:refname` lists tags by version.

```
$ echo "== Step 19: Fix it and release 0.1.1"
== Step 19: Fix it and release 0.1.1
$ printf 'def total(prices, discount=0):\n    if not 0 <= discount <= 1:\n        raise\n    ValueError("discount must be between 0 and 1")\n    return round(sum(prices) * (1 -\n    discount), 2)\n' > src/orders.py
$ sh ci.sh
tests: ok
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
present: .gitignore
note: no LICENSE yet (the owner decides)
$ git commit -q -am "Reject discounts outside 0 to 1"
$ git push -q origin main
$ git tag -a v0.1.1 -m "Version 0.1.1"
$ git push -q origin v0.1.1
$ git tag --sort=version:refname
v0.1.0
v0.1.1
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Publish release 0.1.1 with a note about what it fixes.

Step 20. A bad change reaches main, and you revert it

A bad change reaches main; the pipeline fails; you **revert** (Chapter 25), and the pipeline passes again. History shows both the mistake and its correction.

```
$ echo "== Step 20: A bad change reaches main, and you revert it"
== Step 20: A bad change reaches main, and you revert it
$ printf 'def total(prices, discount=0):\n    return round(sum(prices))\n' > src/orders.py
$ git commit -q -am "Round to whole numbers"
$ git push -q origin main
$ sh ci.sh || true
tests: FAILED
$ git revert --no-edit HEAD > /dev/null
$ git push -q origin main
$ sh ci.sh
tests: ok
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
present: .gitignore
note: no LICENSE yet (the owner decides)
$ git log --format=%s -3
Revert "Round to whole numbers"
Round to whole numbers
Reject discounts outside 0 to 1
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. A required check would have blocked the merge in the first place.

Step 21. A secret is committed by mistake

A secret is committed by mistake and caught **before the push**: the commits `Add local settings` and `Stop tracking .env` are still local (ahead 2 at the end). The search shows that the secret remains in history. Because it never left your computer, you can still drop those two commits (Chapter 25 on `reset`); had it been pushed, the first action would be to revoke the key (Chapter 33). Note that `git add -f` was needed because `.env` was ignored: **a forced add is a warning sign**.

```
$ echo "== Step 21: A secret is committed by mistake"
== Step 21: A secret is committed by mistake
$ printf 'TOKEN=not-a-real-token\n' > .env
$ git add -f .env
$ git commit -q -m "Add local settings"
$ git rm -q --cached .env
$ git commit -q -m "Stop tracking .env"
$ git log --format=%s -S'not-a-real-token'
Stop tracking .env
Add local settings
$ git ls-files | grep -c '^.env$' || true
0
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Secret scanning and push protection (Chapter 62) act at this point.

Step 22. Release notes from the history

Release notes from the history between two tags (Chapter 66).

```
$ echo "== Step 22: Release notes from the history"
== Step 22: Release notes from the history
$ git log --format='- %s' v0.1.0..v0.1.1
- Reject discounts outside 0 to 1
```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Generated notes list merged pull requests.

Step 23. Final audit

Final audit: who committed, how many tags the remote holds (`--refs` hides the peeled duplicates), the branch status and the pipeline. Two commits are deliberately not pushed.

```
$ echo "== Step 23: Final audit"
== Step 23: Final audit
$ git shortlog -sn --no-merges main
```

```

      8 Ada Learner
      1 Maintainer
$ git ls-remote --tags --refs origin | wc -l | tr -d ' '
2
$ git status -sb
## main...origin/main [ahead 2]
$ sh ci.sh
tests: ok
present: README.md
present: CONTRIBUTING.md
present: SECURITY.md
present: .gitignore
note: no LICENSE yet (the owner decides)

```

Recorded in Bash; *ch77-capstone/expected-capstone.bash.txt*.

On GitHub. Check Insights and the Security tab (Chapters 51 and 62).

The self-assessment checklist

Tick each item honestly. An item you cannot show is a topic to revisit.

- ☐ I can create a repository, clone it, and explain origin (Chapters 23, 39).
- ☐ I wrote a test that failed first, then made it pass (step 7, 8).
- ☐ I wrote a README whose example is true (step 3).
- ☐ My pipeline runs locally with one command (step 4).
- ☐ I can explain what branch protection prevents (step 6, Chapter 50).
- ☐ I opened, reviewed and merged a change through a pull request, and I know the merge strategies (steps 8 to 15).
- ☐ I resolved a conflict by reading both sides (step 13).
- ☐ I tagged a release and can name what a checksum adds (steps 16 and 17).
- ☐ I turned a bug into a failing test and released a fix (steps 18 and 19).
- ☐ I undid a bad change on the shared branch without rewriting history (step 20).
- ☐ I know what remains after a secret is committed, and what to do first (step 21).
- ☐ I can produce release notes and audit a repository (steps 22 and 23).
- ☐ I have chosen (or decided to postpone) a licence, knowing it is my decision.

What to add next

The simulation stops short of things you should try on GitHub: a real pull request with review comments, a workflow that runs `ci.sh` with pinned actions and minimal permissions, branch protection or a ruleset, Dependabot and secret scanning, a release with generated notes, and a LICENSE of your choice.

Checkpoint

What You Learned

- A professional workflow is a loop: test first, small branch, pipeline, review, merge, tag, release, monitor.
- Every step has a Git command and a GitHub counterpart; you can now do both.
- Mistakes are normal: revert on shared branches, catch secrets before pushing, and read the state before and after.

New Vocabulary

No new terms.

Commands Learned

No new commands.

Common Mistakes

1. **Skipping the failing-test step** and trusting a test that never failed.
2. **Forcing a file into Git** that the ignore file excluded.
3. **Merging without the pipeline.**
4. **Pushing before looking at what you are about to push.**
5. **Assuming a licence is not needed.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. Why does step 7 want the test to fail?
2. What does the server rule of step 6 stop?
3. Why did step 14 leave the branch's own commits unmerged in Git's eyes?
4. Why is the secret of step 21 not yet a disaster?
5. What does --refs do in step 23?

Before Moving On

You are ready for Chapter [78](#) if you can:

- [] repeat the simulation on a different idea
- [] explain each step to a beginner

How this chapter was checked

| Claim | Evidence class | Ledge r |
|-----------------------|---|---------|
| The 23 recorded steps | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0, Python 3; CI on newer Git | R349 |
| The GitHub column | From earlier chapters; not run on GitHub | none |

Where this leads

Part XII is the troubleshooting handbook: Chapter 78 lists errors, their causes and safe fixes.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. The capstone itself (Project 12) is the main exercise; these tasks support it. Work in a sandbox with a local bare repository as the remote. **Do not add a licence text from this book; choosing one is your decision.**

Level 1 — Guided

1.1 Repeat steps 1 to 5. Create the shared repository, your clone, the first files, the pipeline script and the first push. Show `git status -sb`.

1.2 Read the pipeline. Explain what each line of `ci.sh` checks and what its exit status means for a real CI system.

Level 2 — Partially guided

2.1 Steps 7 and 8. Add a different feature (for example a tax rate) test-first: show the failing test, then the passing one.

2.2 Steps 9 and 10. Play the reviewer with a second clone. Write three review comments (in a text file) and answer them with a follow-up commit.

Level 3 — Independent

3.1 A different idea. Repeat all 23 steps with an idea of your own. List which steps you had to change and why.

3.2 The GitHub column. For five steps, perform the "On GitHub" action on a repository you own, and write down every difference between the recording and what you saw.

Level 4 — Professional scenario

4.1 Handover. Write a one-page handover for a new colleague that explains the project's workflow, using the 23 steps as the outline.

Level 5 — Troubleshooting

5.1 In step 14 the squash merge reports a conflict. What does that tell you about the branch, and what do you do?

5.2 Step 21 was pushed by mistake. Write the order of actions, using Chapters [33](#) and [63](#).

Part XII — Troubleshooting and Recovery

Chapter 78 — Troubleshooting Handbook [Core]

In this chapter

- a method for reading any error message
- fifteen common Git errors, each **produced on purpose and recorded**
- for each: meaning, cause, how to diagnose, the safe fix, the dangerous fix, prevention
- errors this book could not reproduce, and where to look

How to read this chapter. Every recording ran in Bash and zsh on Git 2.43.0 and is re-run in CI on newer Git. The recordings show **only the lines that matter**: they keep the fatal:, error: or first line of a message and leave out the "hint:" lines, whose wording changes from version to version (your Git may print more, or different, text; the meaning stays). Paths and names are made up. **Errors that need a network, an account or a password (for example refused SSH keys or HTTP 403) could not be reproduced offline** and are described in the last section without recordings.

Before you start. Chapter [77](#) and Part III.

78.1 A method for reading errors

1. **Read the first line and the last line.** The first says what failed; the last often says what to try.
 2. **Run git status.** It answers half of all questions: which branch, what is staged, what is in conflict.
 3. **Do not run the first fix that you find.** Search results and old answers suggest --force, reset --hard and rm -rf. Read the "dangerous fix" column before you act.
 4. **Look before you change anything:** git log --oneline --graph --all, git diff, git remote -v.
 5. **Make a safety copy when in doubt:** git branch backup costs nothing, and the reflog (Chapter [26](#)) is your second net.
 6. **Change one thing at a time**, and run git status again.
-

78.2 fatal: not a git repository (or any of the parent directories): .git

Message. fatal: not a git repository (or any of the parent directories): .git

Recording.

```
$ echo "== E1: not a git repository"
== E1: not a git repository
$ mkdir empty && cd empty
```

```
$ git status 2>&1 | grep -E '^fatal'
fatal: not a git repository (or any of the parent directories): .git
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. Git looked in this folder and every parent for a `.git` folder and found none.

Cause. You are in the wrong folder, or the folder was never made a repository, or `.git` was deleted.

Diagnose. `pwd` and `ls -a` show where you are; `git rev-parse --show-toplevel` names the repository if there is one.

Safe fix. `cd` into the project folder; or `git init` if this really is a new project; or clone it.

Dangerous fix. Running `git init` in your home folder or another large folder "to make the error go away": it turns an unrelated folder into a repository.

Prevention. Open the terminal in the project folder; show the current branch in your prompt (Chapter 7).

78.3 fatal: pathspec 'x' did not match any files

Message. fatal: pathspec 'x' did not match any files / error: pathspec 'x' did not match any file(s) known to git

Recording.

```
$ echo "== E2: pathspec did not match"
== E2: pathspec did not match
$ git clone -q hub.git w2
$ cd w2
$ git add missing.txt 2>&1 | grep -E '^fatal'
fatal: pathspec 'missing.txt' did not match any files
$ git checkout no-such-branch 2>&1 | grep -E '^error'
error: pathspec 'no-such-branch' did not match any file(s) known to git
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. Git was given a name (a file, or a branch for checkout) that it cannot find.

Cause. A typo, the wrong folder, a file that does not exist yet, or a branch that exists only on the remote and has not been fetched.

Diagnose. `git status`, `ls`, `git branch -a` and `git fetch` show what exists.

Safe fix. Correct the name; `git fetch` and try again for a remote branch.

Dangerous fix. Creating an empty file or branch with the wanted name just to silence the error.

Prevention. Use tab completion; run `git status` before `git add` (Chapter 19).

78.4 Author identity unknown

Message. Author identity unknown

Recording.

```
$ echo "== E3: identity unknown"
== E3: identity unknown
$ git clone -q hub.git w3
$ cd w3
$ printf 'x\n' > a.txt
$ git add a.txt
$ git commit -m "Add a" 2>&1 | grep -E '^Author identity unknown'
Author identity unknown
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. Git will not create a commit without a name and an email address to record.

Cause. You have not set `user.name` and `user.email` on this computer (Chapter 14).

Diagnose. `git config --get user.name` and `--get user.email` print nothing.

Safe fix. Set them once with `git config --global user.name "Your Name"` and `git config --global user.email "you@example.org"`, then repeat the commit.

Dangerous fix. Committing with someone else's identity, or with an address you do not want public: the address is part of every commit forever.

Prevention. Do the first-time configuration as soon as you install Git (Chapter 14).

78.5 On branch main ... nothing to commit

Message. On branch main ... nothing to commit / no changes added to commit

Recording.

```
$ echo "== E4: nothing to commit"
== E4: nothing to commit
$ git config --global user.name "Ada Learner"
$ git config --global user.email "ada@example.org"
$ git clone -q hub.git w4
$ cd w4
$ git commit -m "Nothing" 2>&1 | head -2
On branch main
Your branch is up to date with 'origin/main'.
$ printf 'new\n' > new.txt
$ git commit -m "Nothing" 2>&1 | head -3
On branch main
```

```
Your branch is up to date with 'origin/main'.

$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. There was nothing staged, so no commit was made.

Cause. You forgot `git add`, or the change was already committed, or the new file is untracked (the second try shows it under "Untracked files").

Diagnose. `git status` lists changed, staged and untracked files.

Safe fix. `git add FILE` and commit again; for a tracked file changed in place `git commit -am` also works.

Dangerous fix. Using `--allow-empty` to force a commit that records nothing.

Prevention. Read `git status` before committing.

78.6 error: Your local changes to the following files would be overwritten by checkout

Message. error: Your local changes to the following files would be overwritten by checkout

Recording.

```
$ echo "== E5: checkout would overwrite"
== E5: checkout would overwrite
$ git clone -q hub.git w5
$ cd w5
$ git switch -q -c other
$ printf -- '- Rolls: 3.00\n' >> menu.md
$ git commit -q -am "Rolls"
$ git switch -q main
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git switch other 2>&1 | grep -E '^error'
error: Your local changes to the following files would be overwritten by checkout:
$ git stash -q
$ git switch -q other
$ git stash pop 2>&1 | grep -E '^(CONFLICT|Auto-merging)'
Auto-merging menu.md
CONFLICT (content): Merge conflict in menu.md
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. Switching branches would destroy edits that are not committed.

Cause. You have uncommitted changes in a file that differs between the two branches.

Diagnose. `git status` and `git diff` show your changes.

Safe fix. Commit them, or `git stash` them and bring them back with `git stash pop` (the recording shows a conflict on pop, which you resolve like any conflict), then switch.

Dangerous fix. `git checkout -f` or `git reset --hard`: both throw your changes away without a trace.

Prevention. Commit or stash before switching (Chapter 24).

78.7 ! [rejected] main -> main (fetch first)

Message. ! [rejected] main -> main (fetch first) / error: failed to push some refs

Recording.

```
$ echo "== E6: push rejected"
== E6: push rejected
$ git clone -q hub.git a6
$ git clone -q hub.git b6
$ cd a6
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -q -am "Tea"
$ git push -q origin main
$ cd ../b6
$ printf-- '- Coffee: 2.00\n' >> menu.md
$ git commit -q -am "Coffee"
$ git push origin main 2>&1 | grep -E '^ ! \[rejected\]|^error'
! [rejected]          main -> main (fetch first)
error: failed to push some refs to '/home/learner/hub.git'
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. The remote has commits that you do not have, so Git refuses to overwrite them.

Cause. Someone else pushed first (the recording plays two clones).

Diagnose. `git fetch` then `git log --oneline main..origin/main` shows what you are missing.

Safe fix. `git pull --no-rebase` (a merge) or `git pull --rebase` after reading the difference (Chapters 23 and 27), resolve conflicts, push again.

Dangerous fix. `git push --force`: it overwrites your colleague's commits. If you must, use `--force-with-lease` and only on your own branch.

Prevention. Pull before you start work and before you push; protect shared branches (Chapter 50). The exact wording ("fetch first" or "non-fast-forward") depends on the Git version.

78.8 fatal: refusing to merge unrelated histories

Message. fatal: refusing to merge unrelated histories

Recording.

```
$ echo "== E7: unrelated histories"
== E7: unrelated histories
$ git init -q solo && cd solo
```

```
$ printf 'x\n' > x.txt && git add x.txt && git commit -q -m "Solo"
$ git remote add origin ../hub.git
$ git fetch -q origin
$ git merge origin/main 2>&1 | grep -E '^fatal'
fatal: refusing to merge unrelated histories
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. The two histories have no common ancestor, so Git will not guess how to join them.

Cause. You made a repository with `git init` and added a remote that has its own first commit (or two projects that were created separately).

Diagnose. `git log --oneline --all --graph` shows two separate roots.

Safe fix. If one side is only a scratch start, discard it and clone the remote instead; if both matter, decide which is the base and rebase or merge with `--allow-unrelated-histories` knowingly.

Dangerous fix. Adding `--allow-unrelated-histories` without looking: it can produce a confusing history with duplicate files.

Prevention. Clone an existing repository instead of `init` plus `remote add`; create the remote empty (Chapter 39).

78.9 error: remote origin already exists

Message. error: remote origin already exists / fatal: '../nowhere.git' does not appear to be a git repository

Recording.

```
$ echo "== E8: remote exists / bad remote"
== E8: remote exists / bad remote
$ git clone -q hub.git w8
$ cd w8
$ git remote add origin ../hub.git 2>&1 | grep -E '^error'
error: remote origin already exists.
$ git remote add backup ../nowhere.git
$ git fetch backup 2>&1 | grep -E '^fatal'
fatal: '../nowhere.git' does not appear to be a git repository
fatal: Could not read from remote repository.
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. The first: you tried to add a remote name that is taken. The second: the address of the remote is wrong or unreachable.

Cause. You cloned (which already created origin), or you mistyped a path or URL.

Diagnose. `git remote -v` lists names and addresses.

Safe fix. Use another name, or `git remote set-url origin NEWURL`.

Dangerous fix. Deleting and re-adding remotes at random, which can leave you pushing to the wrong place.

Prevention. Check `git remote -v` before the first push (Chapter 23).

78.10 fatal: a branch named 'idea' already exists

Message. fatal: a branch named 'idea' already exists / error: the branch 'idea' is not fully merged

Recording.

```
$ echo "== E9: branch exists / not merged"
== E9: branch exists / not merged
$ git clone -q hub.git w9
$ cd w9
$ git switch -q -c idea
$ printf 'i\n' > idea.txt && git add idea.txt && git commit -q -m "Idea"
$ git switch -q main
$ git branch idea 2>&1 | grep -E '^fatal'
fatal: a branch named 'idea' already exists
$ git branch -d idea 2>&1 | grep -E '^error' | sed 's/[.]$//'
error: the branch 'idea' is not fully merged
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. The first: the name is taken. The second: `-d` refuses to delete a branch that has commits nobody else has.

Cause. You reused a name; or the branch was never merged (or was squash-merged, which Git cannot see).

Diagnose. `git branch -vv` and `git log main..idea` show the unmerged commits.

Safe fix. Choose another name; merge the branch first, or confirm the work is elsewhere.

Dangerous fix. `git branch -D idea` without looking: the commits are then only reachable through the reflog for a while (Chapter 26).

Prevention. Delete branches only after their pull request is merged (Chapter 20).

78.11 fatal: ambiguous argument 'x': unknown revision or path not in the working tree

Message. fatal: ambiguous argument 'x': unknown revision or path not in the working tree

Recording.

```
$ echo "== E10: bad revision"
== E10: bad revision
$ git clone -q hub.git w10
$ cd w10
$ git log no-such-ref 2>&1 | grep -E '^fatal'
fatal: ambiguous argument 'no-such-ref': unknown revision or path not in the working tree.
$ git show HEAD~5 2>&1 | grep -E '^fatal'
fatal: ambiguous argument 'HEAD~5': unknown revision or path not in the working tree.
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. Git cannot tell what x is: not a branch, tag, commit or file.

Cause. A typo, a branch that has not been fetched, or HEAD~5 in a repository with fewer commits.

Diagnose. `git branch -a`, `git tag`, `git log --oneline` show what is available.

Safe fix. Correct the name; `git fetch`; use `--` before a file name to say that it is a file.

Dangerous fix. Passing a made-up hash, or guessing.

Prevention. Copy names from `git branch` or `git log`.

78.12 fatal: Unable to create '.git/index.lock': File exists

Message. fatal: Unable to create '.git/index.lock': File exists

Recording.

```
$ echo "== E11: index.lock"
== E11: index.lock
$ git clone -q hub.git w11
$ cd w11
$ touch .git/index.lock
$ git add -A 2>&1 | head -1 | cut -c1-60
fatal: Unable to create '/home/learner/w1
$ rm .git/index.lock
$ git status -sb
## main...origin/main
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. Git keeps a lock file while it changes the index; another Git process appears to be running, or crashed and left the file.

Cause. Another Git command (or an editor's Git integration) is running, or a previous command was killed.

Diagnose. Look for running Git processes first; then check whether the lock file is old.

Safe fix. Wait for the other process; if you are sure that none is running, delete `.git/index.lock` (the recording does).

Dangerous fix. Deleting the lock while a Git command is running, which can corrupt the index.

Prevention. Do not run two Git commands at once; let a crashed command finish or restart the computer.

78.13 A file is still tracked after you added it to `.gitignore`

Message. A file is still tracked after you added it to `.gitignore`

Recording.

```
$ echo "== E12: gitignore not working"
== E12: gitignore not working
$ git clone -q hub.git w12
$ cd w12
$ printf 'secret\n' > notes.txt
$ git add notes.txt && git commit -q -m "Add notes"
$ printf 'notes.txt\n' > .gitignore
$ printf 'more\n' >> notes.txt
$ git status --short
M notes.txt
?? .gitignore
$ git rm -q --cached notes.txt
$ git status --short
D notes.txt
?? .gitignore
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. `.gitignore` only affects files that Git does not yet track.

Cause. The file was committed before it was ignored.

Diagnose. `git ls-files FILE` prints the name if it is tracked.

Safe fix. `git rm --cached FILE` stops tracking it and keeps the file on disk; commit.

Dangerous fix. Deleting the file from disk with plain `git rm`, or forcing it in again with `git add -f`.

Prevention. Write `.gitignore` before the first `git add` (Chapter 18). If the file held a secret, see Chapter 33.

78.14 HEAD detached at abc1234

Message. HEAD detached at abc1234

Recording.

```
$ echo "== E13: detached HEAD"
== E13: detached HEAD
$ git clone -q hub.git w13
$ cd w13
$ git switch -q --detach HEAD
$ git branch --show-current | wc -c | tr -d ' '
0
$ git status | head -1
HEAD detached at bbc2826
$ git switch -q -c rescue
$ git branch --show-current
rescue
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. You are looking at a commit, not at a branch, so new commits would belong to no branch.

Cause. You checked out a commit, a tag or a remote branch directly.

Diagnose. `git branch --show-current` prints nothing.

Safe fix. To keep work made here, `git switch -c NAME`; to leave, `git switch main`.

Dangerous fix. Making commits and then switching away without a branch: they become hard to find (Chapter 26).

Prevention. Use `git switch -c NAME` when you want to work; use detached mode to look.

78.15 CONFLICT (content): Merge conflict in menu.md

Message. CONFLICT (content): Merge conflict in menu.md / Automatic merge failed

Recording.

```
$ echo "== E14: merge conflict"
== E14: merge conflict
$ git clone -q hub.git p14
$ git clone -q hub.git q14
$ cd p14
$ sed -i 's/2.50/2.60/' menu.md
$ git commit -q -am "Price up"
$ git push -q origin main
$ cd ../q14
$ sed -i 's/2.50/2.70/' menu.md
$ git commit -q -am "Price different"
$ git -c advice.mergeConflict=false pull -q --no-rebase --no-edit 2>&1 | grep -E
'^((CONFLICT|Automatic))'
CONFLICT (content): Merge conflict in menu.md
Automatic merge failed; fix conflicts and then commit the result.
$ git status --short
UU menu.md
$ git merge --abort
```

```
$ git status --short | wc -l | tr -d ' '
0
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. Two changes touched the same lines and Git needs you to decide.

Cause. Two people changed the same place.

Diagnose. `git status` marks the file `UU`; the file contains `<<<<<<`, `=====` and `>>>>>>`.

Safe fix. Edit the file to the intended result, `git add`, `git commit`; or `git merge --abort` to go back (the recording does this and status is clean).

Dangerous fix. Keeping the markers, or choosing one side blindly with `--ours/--theirs` without reading the other.

Prevention. Small branches, frequent updates from `main`, talking to the other author (Chapter 22).

78.16 fatal: Need to specify how to reconcile divergent branches

Message. fatal: Need to specify how to reconcile divergent branches

Recording.

```
$ echo "== E15: pull with diverged branches"
== E15: pull with diverged branches
$ git clone -q hub.git p15
$ git clone -q hub.git q15
$ cd p15
$ printf -- '- Tea: 1.50\n' >> menu.md
$ git commit -q -am "Tea"
$ git push -q origin main
$ cd ../q15
$ printf -- '- Coffee: 2.00\n' >> menu.md
$ git commit -q -am "Coffee"
$ git pull 2>&1 | grep -E '^fatal'
fatal: Need to specify how to reconcile divergent branches.
$ cd ..
```

Recorded in Bash; *ch78-trouble/expected-handbook.bash.txt*.

Meaning. Your branch and the remote's each have commits the other lacks, and Git will not choose merge or rebase for you.

Cause. You committed locally while others pushed, and no `pull.rebase` or `pull.ff` setting exists.

Diagnose. `git status` says the branches have diverged.

Safe fix. Choose: `git pull --no-rebase` (merge), `git pull --rebase`, or set `git config pull.rebase false` (or `true`) once (Chapter 23).

Dangerous fix. Forcing your version over the remote's.

Prevention. Decide your default once and write it in your configuration.

78.17 Errors that could not be reproduced here

These need a network, an account or a service, so no recording exists. **Treat the causes below as common patterns, not as verified diagnoses.** Read the current GitHub documentation (Chapter 35) for the exact text on your system.

| What you see (approximately) | Usual cause | First safe step |
|--|---|---|
| Permission denied (publickey) | The SSH key is missing, not loaded or not registered with your account (Chapter 38) | <code>ssh -T</code> to the host shows whether the key is accepted; check the key is added to your account |
| Authentication failed over HTTPS | An expired, revoked or wrong token or password (Chapter 38); which credentials a host accepts is in its current documentation | Create a new token with the least scopes, or use a credential helper |
| remote: Repository not found or HTTP 403 or 404 | The address is wrong, the repository is private and you lack access, or you are signed in as another account | <code>git remote -v</code> ; check that you are signed in as the right account |
| A push rejected because it contains a secret | Push protection found a credential (Chapter 62) | Remove the secret, and revoke it if it was ever exposed |
| A push rejected by branch protection | A rule needs a pull request or checks (Chapter 50) | Open a pull request |
| A file too large for the remote | The platform limits file size (see its current documentation) | Remove the file from the commit; consider Git LFS (Chapter 31) |
| fatal: unable to access ... Could not resolve host | No network or a proxy problem | Check the connection; ping or the browser |
| Line-ending warnings | Windows and Linux line endings (Chapters 4 and 32) | Read the warning; set <code>core.autocrlf</code> or <code>.gitattributes</code> deliberately |

Checkpoint

What You Learned

- Read the first and last lines of an error, run `git status`, and do not accept the first fix you find.
- Every error has a safe fix and a dangerous fix; the dangerous one usually destroys work or overwrites others'.
- `-d` is safer than `-D`, `--force-with-lease` is safer than `--force`, and `stash` is safer than `checkout -f`.

- Some errors depend on the network or your account; look them up in the current documentation.

New Vocabulary

No new terms.

Commands Learned

No new commands; this chapter uses many earlier ones.

Common Mistakes

1. **Forcing a push to make an error disappear.**
2. **Running `git init` where a repository already exists (or should).**
3. **Deleting `index.lock` while Git is running.**
4. **Believing that `.gitignore` removes a tracked file.**
5. **Copying a fix from a forum without reading it.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the first command to run when you do not understand a state?
2. Which safe fix corresponds to the error "Your local changes would be overwritten"?
3. Why is `git branch -D` dangerous?
4. What does `git rm --cached` do?
5. Why does a push get rejected with "fetch first"?

Before Moving On

You are ready for Chapter [79](#) if you can:

- [] read an error message with the method of section 78.1
- [] name the dangerous fix for five of the errors

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|--------|
| The fifteen error messages, their triggers and the safe fixes shown | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0; CI on newer Git (hint lines are omitted because they vary) | R350 |
| The table of errors that could not be reproduced | Common patterns, not verified here | none |

Where this leads

Chapter 79 gives step-by-step recoveries for the worst cases: secrets, lost commits, bad merges, failed rebases and wrong branches.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Work in a sandbox with a local bare repository as the remote. **Do not try the dangerous fixes on a repository that matters.**

Level 1 — Guided

1.1 The method. List the six steps of the method in section 78.1.

1.2 Match. Match each message to its meaning: (a) "not a git repository", (b) "Author identity unknown", (c) "refusing to merge unrelated histories", (d) "Unable to create index.lock".

Level 2 — Partially guided

2.1 Produce three. Cause and record, in your sandbox, three errors of your choice from the handbook. Show the message and the safe fix.

2.2 Rejected push. With two clones of one bare repository, produce a rejected push and fix it with a merge. Then produce it again and fix it with a rebase. Compare the histories.

Level 3 — Independent

3.1 A new entry. Find an error message that is not in the handbook (for example from a mistyped command), reproduce it, and write an entry with all seven fields.

3.2 Dangerous or not. For each command say whether it is safe, risky or destructive, and why: `git checkout -f`, `git stash`, `git push --force-with-lease`, `git branch -d`, `git clean -fd`.

Level 4 — Professional scenario

4.1 A colleague's message. A colleague writes: "I got failed to push some refs, so I used --force and now Ana says her work is missing." Write the reply: what happened, how to recover Ana's commits (Chapter 26) and how to prevent a repeat (Chapter 50).

Level 5 — Troubleshooting

5.1 `.gitignore` lists `notes.txt` but `git status` still shows changes to it. Explain and fix.

5.2 `git status` says "HEAD detached at" and you have made two commits. How do you keep them?

Chapter 79 — Recovery Playbooks [Core]

In this chapter

- five step-by-step recoveries: a pushed secret, lost commits, a bad merge, a failed rebase, a commit on the wrong branch
- each recorded, with the reason for every step
- what each recovery cannot undo

How to read this chapter. The five playbooks were **run and recorded** in Bash and zsh on Git 2.43.0 with git-filter-repo 2.47.0, and re-run in CI. A local bare repository stands in for GitHub, and the five playbooks share it, so **later ones start from what earlier ones left behind** (you will see the commit "Stop tracking .env" in the history of later playbooks). The statements about what GitHub keeps after a history rewrite come from GitHub's documentation on removing sensitive data (github/docs at commit 2eaab0b, 29 September 2026) and were **not run on GitHub**. The key in playbook 1 is made up.

Before you start. Chapter 78 (the method) and Chapter 26.

The rules that apply to every playbook.

1. **Stop and read** the state (`git status`, `git log --oneline --graph --all`) before you type a fix.
2. **Make a safety branch** (`git branch backup`) when you are about to do anything that moves history.
3. **Prefer undoing forwards** (`revert`, a new commit) to rewriting when others have the commits.
4. **Tell your team** before you rewrite anything shared.

Playbook 1. A secret was pushed

Situation. A file with a credential was committed and pushed to the shared repository.

The order matters. GitHub's documentation is direct: "if the sensitive data you need to remove is a secret (e.g. password/token/credential), as is often the case, then as a first step you need to revoke and/or rotate that secret. Once the secret is revoked or rotated, it can no longer be used for access, and that may be sufficient to solve your problem. Going through the extra steps to rewrite the history and remove the secret may not be warranted." So:

1. **Revoke** the credential at the service that issued it. This is outside Git and comes first.
2. Stop tracking the file and ignore it (an ordinary commit).
3. Only if you must, **rewrite history** to remove the file everywhere, then push the rewritten history.
4. Deal with the copies you cannot reach.

```
$ echo "== P1: A secret was pushed"
== P1: A secret was pushed
$ git clone -q hub.git work
```

```

$ cd work
$ printf 'API_KEY=not-a-real-key\n' > .env
$ git add .env
$ git commit -q -m "Add configuration"
$ git push -q origin main
$ echo "1. REVOKE the key at the service that issued it (outside Git)"
1. REVOKE the key at the service that issued it (outside Git)
$ git rm -q .env
$ printf '.env\n' > .gitignore
$ git add .gitignore
$ git commit -q -m "Stop tracking .env"
$ git push -q origin main
$ git log --format=%s -S'not-a-real-key'
Stop tracking .env
Add configuration
$ git filter-repo --sensitive-data-removal --invert-paths --path .env --force
NOTICE: Fetching all refs from origin to make sure we rewrite
        all history that may reference the sensitive data, via
        git fetch -q --prune --update-head-ok --refmap "" origin +refs/*:refs/*
Parsed 4 commits
New history written in <n> seconds; now repacking/cleaning...
You rewrote 2 (of 4) commits.

NOTE: First Changed Commit(s) is/are:
      c6f19902e8d163fd177ad0064c2ceca5b2bf778e
NOTE: LFS object orphaning not checked (LFS not in use)

Repacking your repo and cleaning out old unneeded objects
HEAD is now at aa630cc Stop tracking .env
Completely finished after <n> seconds.

NEXT STEPS FOR YOUR SENSITIVE DATA REMOVAL:
* If you are doing your rewrite in multiple steps, ignore these next steps
  until you have completed all your invocations of git-filter-repo.
* See the "Sensitive Data Removal" subsection of the "DISCUSSION" section
  of the manual for more details about any of the steps below.
* Inspect this repository and verify that the sensitive data is indeed
  completely removed from all commits.
* Force push the rewritten history to the server:
    git push --force --mirror origin
* Contact the server admins for additional steps they need to take; the
  First Changed Commit(s) may come in handy here.
* Have other colleagues with a clone either discard their clone and reclone
  OR follow the detailed steps in the manual to repeatedly rebase and
  purge the sensitive data from their copy. Again, the First Changed
  Commit(s) may come in handy.
* See the "Prevent repeats and avoid future sensitive data spills" section
  of the manual.

$ git log --format=%s -S'not-a-real-key'
$ git push -q --force --mirror origin
$ git clone -q ../hub.git ../fresh
$ git -C ../fresh log --all --format=%s -S'not-a-real-key'
$ cd ..

```

Recorded in Bash; *ch79-playbooks/expected-recover.bash.txt*.

What the recording showed. After step 2, the search (-S) still found the key in two commits: removing a file in a new commit does not remove it from history. `git filter-repo` (a separate tool, Chapter 33) rewrote the history without the file; the documented form is `--sensitive-data-removal --invert-paths --path FILE`, which needs a version that has the flag (2.47 or later, per the documentation) and, unlike its default behaviour, keeps the origin remote and fetches all refs first so that the rewrite covers everything. It prints "NEXT STEPS", including the forced push, which is what `git push --force --mirror origin` did. A **fresh clone** of the shared repository afterwards no longer contains the key.

What the rewrite cannot do. GitHub's documentation lists the side effects and limits. Among them: hashes of the affected commits **and all later commits change**, breaking tools that depend on them; open pull requests get confusing; signatures on rewritten commits are lost; branch protection that blocks force pushes has to be turned off temporarily; and "If a fellow developer has a clone from before your rewrite, and after your rewrite simply runs `git pull` followed by `git push`, the sensitive data will return", so colleagues must **discard their clones and clone again** (or follow the manual's cleaning steps). If you only rewrite and force-push, the commits "may still be accessible elsewhere": in clones and forks, "directly via their SHA-1 hashes in cached views on GitHub", and through pull requests that reference them. Fully removing cached views and pull-request references needs a request to GitHub Support, with the number of affected pull requests and the "First Changed Commit(s)" that the tool reports. Forks keep the data unless their owners remove it.

Verdict. The rewrite is a large operation with a small guarantee. Revocation is the real fix. Then add push protection (Chapter 62) to stop it recurring.

Checked against GitHub's documentation (R351) and locally tested. "Removing sensitive data from a repository". The Support process was not exercised.

Playbook 2. Commits seem to be lost after a hard reset

Situation. You ran `git reset --hard HEAD~2` and two commits disappeared from `git log`.

```
$ echo "== P2: Commits seem to be lost after a hard reset"
== P2: Commits seem to be lost after a hard reset
$ git clone -q hub.git p2
$ cd p2
$ printf -- "- Coffee: 2.00\n" >> menu.md
$ git commit -q -am "Add coffee"
$ printf -- "- Scone: 2.20\n" >> menu.md
$ git commit -q -am "Add scone"
$ git reset -q --hard HEAD~2
$ git log --format=%s
Stop tracking .env
Add tea
Add the menu
$ git reflog --format='%gd %gs' -n 4
HEAD@{0} reset: moving to HEAD~2
HEAD@{1} commit: Add scone
HEAD@{2} commit: Add coffee
HEAD@{3} clone: from /home/learner/hub.git
$ git reset -q --hard 'HEAD@{1}'
```

```
$ git log --format=%s
Add scone
Add coffee
Stop tracking .env
Add tea
Add the menu
$ cd ..
```

Recorded in Bash; *ch79-playbooks/expected-recover.bash.txt*.

What happened. `reset --hard` moved the branch back two commits and threw away uncommitted changes, but the two commits were still stored. The **reflog** (Chapter 26) recorded every move of HEAD: `HEAD@{1}` is where you were before the reset. `git reset --hard 'HEAD@{1}'` returned the branch to that place, and the log shows the commits again.

Limits. The reflog is local to your clone and entries expire; commits that were never committed (only edited) are gone. Recover soon, and do not run `git gc --prune=now` in between.

Playbook 3. A bad merge reached the shared branch

Situation. A branch with a wrong price was merged into main and pushed; others have fetched it.

```
$ echo "== P3: A bad merge reached the shared branch"
== P3: A bad merge reached the shared branch
$ git clone -q hub.git p3
$ cd p3
$ git switch -q -c risky
$ printf -- "- Coffee: 20.00\n" >> menu.md
$ git commit -q -am "Add coffee at a wrong price"
$ git switch -q main
$ printf 'Open daily.\n' > hours.md
$ git add hours.md
$ git commit -q -m "Add opening hours"
$ git merge -q --no-ff -m "Merge risky" risky
$ git push -q origin main
$ git log --format=%s
Merge risky
Add opening hours
Add coffee at a wrong price
Stop tracking .env
Add tea
Add the menu
$ git revert -m 1 --no-edit HEAD > /dev/null
$ git push -q origin main
$ git log --format=%s
Revert "Merge risky"
Merge risky
Add opening hours
Add coffee at a wrong price
Stop tracking .env
Add tea
Add the menu
$ cat menu.md
# Sunrise Bakery menu
```

```
- White loaf: 2.50
- Rolls: 3.00
- Tea: 1.50
$ ls
hours.md  menu.md
$ cd ..
```

Recorded in Bash; *ch79-playbooks/expected-recover.bash.txt*.

What happened. `git revert -m 1` created a **new commit that undoes the merge**, and the push was ordinary because history only grew. `-m 1` tells Git to treat the first parent (the main side) as the mainline, so the changes brought in by the branch are removed. The final `menu.md` no longer has the wrong line. **But `hours.md`, which was added on main before the merge, is still there**, as it should be: only what the merge brought in was undone.

Caution about merging again. After reverting a merge, Git considers the branch's commits as already merged, so merging the same branch later will **not** bring its changes back. To reapply them, revert the revert, or make new commits. Git's howto "Revert a faulty merge" (Git 2.56.0) explains why: reverting a merge "undoes the data changes" but "it doesn't undo history", so a later merge of the fixed branch "will not" contain the reverted changes, and the way out is to revert the revert first. This was read, not run here.

Never "fix" this by resetting main and force-pushing when others have the merge.

Playbook 4. A rebase went wrong

Situation. You started `git rebase main` on your branch and it stopped with a conflict.

```
$ echo "== P4: A rebase went wrong"
== P4: A rebase went wrong
$ git clone -q hub.git p4
$ cd p4
$ git switch -q -c feature
$ sed -i 's/Rolls: 3.00/Rolls: 3.20/' menu.md
$ git commit -q -am "Rolls 3.20"
$ git switch -q main
$ sed -i 's/Rolls: 3.00/Rolls: 3.50/' menu.md
$ git commit -q -am "Rolls 3.50"
$ git switch -q feature
$ git -c advice.mergeConflict=false rebase main 2>&1 | grep -E '^CONFLICT'
CONFLICT (content): Merge conflict in menu.md
$ git status -sb | head -1
## HEAD (no branch)
$ git rebase --abort
$ git status -sb | head -1
## feature
$ git log --format=%s
Rolls 3.20
Revert "Merge risky"
Merge risky
Add opening hours
Add coffee at a wrong price
Stop tracking .env
```

```
Add tea
Add the menu
$ cd ..
```

Recorded in Bash; *ch79-playbooks/expected-recover.bash.txt*.

What happened. The first search line shows the conflict. `git status -sb` printed `## HEAD` (no branch): in the middle of a rebase Git checks out the commits one by one, so you are not on your branch. `git rebase --abort` **returned everything to how it was before the rebase**, including the branch name (the next status line) and the branch's own commit at the top of the log.

Your choices in a stopped rebase. Resolve the conflict, `git add` the file and `git rebase --continue`; or skip the commit (`--skip`) if it is no longer needed; or abort. Abort is the safe way out: it restores the branch as it was before the rebase, and it discards the conflict edits you made so far in the stopped rebase. If you have already finished the rebase and regret it, the reflog has the old tip (Playbook 2).

Caution. Rebase rewrites commits. Do it only on commits nobody else has (Chapter 27).

Playbook 5. A commit was made on the wrong branch

Situation. You committed on `main` when you meant to be on a feature branch, and you have not pushed.

```
$ echo "== P5: A commit was made on the wrong branch"
== P5: A commit was made on the wrong branch
$ git clone -q hub.git p5
$ cd p5
$ printf -- "- Coffee: 2.00\n" >> menu.md
$ git commit -q -am "Add coffee"
$ git branch feature/coffee
$ git reset -q --hard origin/main
$ git log --format=%s
Revert "Merge risky"
Merge risky
Add opening hours
Add coffee at a wrong price
Stop tracking .env
Add tea
Add the menu
$ git log --format=%s feature/coffee
Add coffee
Revert "Merge risky"
Merge risky
Add opening hours
Add coffee at a wrong price
Stop tracking .env
Add tea
Add the menu
$ git switch -q feature/coffee
$ git push -q -u origin feature/coffee
$ git status -sb | head -1
## feature/coffee...origin/feature/coffee
$ cd ..
```

Recorded in Bash; *ch79-playbooks/expected-recover.bash.txt*.

What happened. `git branch feature/coffee` created a new branch **at the current commit**, so the commit now has two names. `git reset --hard origin/main` moved main back to the remote's version, which is safe because the commit is kept by the other branch (the two logs show it). Then you switched to the new branch and pushed it. The last line shows the branch and its remote in step.

Never reset before you have given the commit a second name. If you already did, use the reflog (Playbook 2).

Checkpoint

What You Learned

- Revoke a leaked secret first; a history rewrite is optional, large and incomplete.
- The reflog undoes a hard reset if you act soon.
- Revert a merge with `-m 1`; remember that re-merging needs care.
- `git rebase --abort` returns to the state before the rebase.
- Give a wrongly placed commit a second name before moving the branch back.

New Vocabulary

No new terms.

Commands Learned

`git filter-repo --sensitive-data-removal --invert-paths --path`, `git push --force --mirror`, `git revert -m 1`, `git rebase --abort`.

Common Mistakes

1. **Rewriting history before revoking the key.**
2. **Force-pushing a rewritten history without telling the team.**
3. **Resetting main after a bad merge instead of reverting.**
4. **Continuing a rebase you do not understand instead of aborting.**
5. **Resetting a branch before saving the commit that is on it.**

Practice

Do the exercises in the Exercises section at the end of this chapter.

Self-Test

1. What is the first action when a secret is pushed?
2. Where does `HEAD@{1}` point after a reset?
3. Why does `revert` need `-m 1` for a merge commit?

4. What does `git rebase --abort restore`?
5. Why is Playbook 5 safe?

Before Moving On

You are ready for Chapter 80 if you can:

- [] carry out each playbook without the book
- [] name what each recovery cannot undo

How this chapter was checked

| Claim | Evidence class | Ledge
r |
|--|---|---------------|
| The five recorded recoveries | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0, git-filter-repo 2.47.0; CI on newer Git | R351,
R352 |
| What remains after a rewrite on GitHub, and the Support step | Checked against github/docs (commit 2eaab0b); not run on GitHub | R351 |
| Reverting a merge and merging again | Read in Git's howto "Revert a faulty merge" (Git 2.56.0); not run | R352 |

Where this leads

Part XIII, "Mastery", begins with Chapter 80: harder tasks that combine these recoveries.

Exercises

Attempt each exercise before you open the Solutions section at the back of the book. Work in a sandbox with a local bare repository as the remote. **Only practise history rewriting on a repository that you can throw away.**

Level 1 — Guided

1.1 Order. Put the actions of Playbook 1 in the right order: rewrite history, revoke the key, stop tracking the file, tell colleagues to re-clone.

1.2 Name the tool. Which playbook uses the `reflog`, and what does `HEAD@{1}` mean there?

Level 2 — Partially guided

2.1 Playbook 2. Make two commits, reset hard two commits back, and recover them with the `reflog`. Show the log before and after.

2.2 Playbook 5. Commit on `main` by mistake and move the commit to a new branch without losing it. Show both logs.

Level 3 — Independent

3.1 Playbook 3. Create a merge that you regret, push it, and revert it with `-m 1`. Then try to merge the same branch again. What do you observe? Explain it with the chapter.

3.2 Playbook 4. Cause a rebase conflict and abort it. Then cause it again and resolve it with `--continue`. Compare the histories.

Level 4 — Professional scenario

4.1 Incident report. A token was pushed to `main` on Monday and found on Wednesday. Write the incident report: timeline, actions in order, what the rewrite could and could not achieve, what you asked colleagues to do, and prevention.

Level 5 — Troubleshooting

5.1 After a history rewrite a colleague pulls and pushes, and the secret reappears. Explain why, and what they should have done.

5.2 `git reset --hard 'HEAD@{1}'` did not bring your commits back. Give three possible reasons.

Part XIII — Mastery

Chapter 80 — Advanced Challenges [Deep]

In this chapter

- six challenges that combine what you learned: multiple repositories, history surgery, large-repository tricks, workflow design, automated bisecting and rescue of unreachable commits
- for each: a setup you type, a task, and a way to check your result
- **solutions are in a separate file**, the Solutions section at the back of the book; try first

How to read this chapter. This is a **Deep** chapter for people who finished the core path. Every challenge has a **reference solution that was run and recorded** in Bash and zsh (Git 2.43.0, git-filter-repo 2.47.0) and is re-run in CI on newer Git; the recordings are kept with the book's verification files and are not printed here, so that the challenges stay challenges. Setups use local repositories only: nothing needs GitHub. The one paper challenge (workflow design) has no recording. Expect your commit names and some wording to differ from the reference.

Before you start. Chapters [79](#), [31](#), [28](#) and [30](#). Work in an empty sandbox folder. Do not look at the solutions file until you have tried for a while.

Challenge 1. Update a submodule pointer

Idea. A project can contain another repository as a **submodule**; the outer project records which commit of the inner one it uses (Chapter [31](#)).

Setup. Type these lines (they create the inner repository lib, with two versions, and an outer project app):

```
git init -q --bare lib.git
git clone -q lib.git libwork
cd libwork
printf 'version 1\n' > lib.txt
git add lib.txt && git commit -q -m "lib 1" && git push -q origin main
cd ..
git init -q app
cd app
git -c protocol.file.allow=always submodule add -q ../lib.git vendor/lib
git commit -q -m "Add lib as a submodule"
cd ../libwork
printf 'version 2\n' > lib.txt
git commit -q -am "lib 2" && git push -q origin main
cd ../app
```

(protocol.file.allow=always is needed only because this sandbox uses a local file path; Chapter [31](#) explains why it is not a permanent setting.)

Task. Make app use version 2 of lib, and record that decision in a commit of app.

Check. `cat vendor/lib/lib.txt` prints version 2, `git status` in `app` is clean, and `git log` in `app` has a commit that records the change.

Challenge 2. Split a folder into its own repository

Idea. A monorepo (one repository for many things) contains a `site` folder that deserves its own repository, **with its history**.

Setup.

```
git init -q mono && cd mono
mkdir site tools
printf 'a\n' > site/index.html && printf 't\n' > tools/build.sh
git add -A && git commit -q -m "Start"
printf 'b\n' >> site/index.html && git commit -q -am "Change site"
printf 't2\n' >> tools/build.sh && git commit -q -am "Change tools"
printf 'c\n' >> site/index.html && git commit -q -am "Change site again"
cd ..
```

Task. Produce a **new repository** that contains only the contents of `site` at its top level, with only the commits that touched it. Do not modify `mono`.

Check. In the new repository `ls` shows `index.html` and nothing else, and the number of commits (`git rev-list --count HEAD`) is 3, not 4.

Hint. Chapters 33 and 79 used a tool that rewrites history; work on a **clone**.

Challenge 3. Clone only what you need

Idea. A large repository takes long to download. Git can fetch less history and check out fewer files (Chapter 31).

Setup.

```
git init -q big && cd big
mkdir docs src
for i in 1 2 3 4 5 6; do printf "$i\n" > src/f$i.txt; printf "$i\n" > docs/d$i.md; git add
-A; git commit -q -m "Commit $i"; done
cd ..
```

Task. Make a clone of `big` that has **one commit** of history and **only the docs folder** in its working tree.

Check. `git rev-list --count HEAD` prints 1, and `ls` shows only `docs`.

Hint. For a local repository, shallow cloning needs a `file://` address (Chapter 31).

Challenge 4. Design a workflow (on paper)

Situation. A team of six maintains a small web application and a documentation site. They release monthly and sometimes need urgent fixes. New contributors join often, and the application handles customers' data.

Task. Write a one-page **workflow design** with these parts: (1) branch and merge rules; (2) what the automated checks are and when they run; (3) branch protection and review rules; (4) how releases are made and numbered; (5) how a hotfix is made; (6) how secrets and dependencies are handled; (7) what a new contributor reads first. Use only ideas from this book and say which chapter each comes from.

Check. Use the rubric in the solutions file: each of the seven parts must be present, consistent with the others, and justified in a sentence.

Challenge 5. Find the first bad commit automatically

Idea. A file `state.txt` was correct once and is wrong now, somewhere in fourteen commits. Binary search finds the change without reading them all (Chapter 28).

Setup.

```
git init -q hunt && cd hunt
printf 'ok\n' > state.txt && git add state.txt && git commit -q -m "Start"
for i in 1 2 3 4 5 6 7 8; do printf "$i\n" >> notes.txt; git add notes.txt; git commit -q -m "Note $i"; done
printf 'broken\n' > state.txt && git commit -q -am "Change state"
for i in 9 10 11 12; do printf "$i\n" >> notes.txt; git add notes.txt; git commit -q -m "Note $i"; done
cd ..
```

Task. Write a **test script** that succeeds when `state.txt` contains exactly the line `ok`, and use Git to find the first commit for which it fails, **without checking out commits by hand**.

Check. Git names the commit "Change state". Then return the repository to its normal state.

Watch out. A careless test can pass when it should fail: think about what your test matches.

Challenge 6. Rescue a commit that no branch points to

Idea. Work was committed on a branch and the branch was deleted before merging. No branch and no tag points to the commit, and you do not remember its message.

Setup.

```
git init -q lost && cd lost
printf 'x\n' > a.txt && git add a.txt && git commit -q -m "Base"
git switch -q -c temp
printf 'y\n' >> a.txt && git commit -q -am "Precious work"
git switch -q main
git branch -D temp
```

Task. Find the commit **without using the reflog**, look at its message, and give it a branch named `rescued`.

Check. `git log -1 rescued` shows "Precious work".

Hint. Chapter 30 explained that objects exist whether or not a name points to them; one Git command lists objects that nothing reaches.

Scoring yourself

| Challenge | You did it if... |
|-----------|--|
| 1 | the outer project records the new inner commit in its own commit |
| 2 | the new repository has three commits and only the site's files |
| 3 | one commit, only docs |
| 4 | all seven parts present, consistent, justified |
| 5 | Git named the right commit through an automated test |
| 6 | the commit is reachable from <code>rescued</code> |

If you needed the solutions for more than two challenges, revisit Chapters 31, 28, 30 and 79, then try again with a different idea.

Checkpoint

What You Learned

- A submodule records a commit of another repository; updating it is a commit in the outer project.
- History surgery (a subdirectory filter) works on a clone and changes commit names.
- Shallow clones and sparse checkouts cut the cost of a large repository.
- A test that can pass wrongly is worse than none; `git bisect run` automates the search.
- Unreachable objects can still be found until garbage collection removes them.

New Vocabulary

No new terms.

Commands Learned

No new commands.

Common Mistakes

1. **Rewriting history in the original repository** instead of a clone.

2. **Writing a test that matches too loosely** (a word inside another word).
3. **Leaving `git bisect running`** and forgetting to reset.
4. **Assuming a deleted branch's commits are gone.**
5. **Skipping the "paper" design and jumping to commands.**

Practice

The challenges are the practice. The exercises file for this chapter adds three extension tasks: the Exercises section at the end of this chapter.

Self-Test

1. Why do a shallow clone of a local repository need a `file://` address?
2. What does the outer project record about a submodule?
3. Why must the test in challenge 5 match a whole line?
4. What kind of objects does `git fsck --lost-found` report?
5. Why work on a clone in challenge 2?

Before Moving On

You are ready for Chapter [81](#) if you can:

- [] complete at least four challenges without the solutions
- [] explain why each solution works

How this chapter was checked

| Claim | Evidence class | Ledger |
|--|--|--------|
| The reference solutions of challenges 1, 2, 3, 5 and 6 | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0, git-filter-repo 2.47.0; CI on newer Git (recordings kept in <code>verification/ch80-challenges</code> , not printed) | R353 |
| Challenge 4 (paper) | Assessed with a rubric, not run | none |

Where this leads

Chapter [81](#) is the final assessment, in two levels.

Exercises

The six challenges are in the chapter. These tasks extend them. Attempt each before you open the Solutions section at the back of the book.

Level 1 — Guided

1.1 Explain your solution. For challenge 1, write one sentence for each command you used, and say what the outer project actually stores about a submodule.

Level 2 — Partially guided

2.1 Extension 1: two submodules. Extend challenge 1: add a second inner repository theme to app, update both to their newest commits, and make **one** commit in app that records both. Then say what a colleague must run after `git pull` to get the new inner versions.

Level 3 — Independent

3.1 Extension 2: keep two folders. Extend challenge 2: produce a new repository that contains site and tools, each in its own folder, but not anything else you add to mono (create a third folder notes first to test it). Check the commit count.

Level 4 — Professional scenario

4.1 Turn the design into checks. Take your workflow design from challenge 4 and write, for each of its seven parts, one automated check or one written rule that would let a reviewer see whether the team follows it.

Level 5 — Troubleshooting

5.1 Extension 3: a flaky test. Extend challenge 5: make the test script fail **randomly** in one commit (for example, depending on the last digit of a number in the file). Explain why `bisect` can then give a wrong answer, and how you would make the test reliable.

Chapter 81 — Final Mastery Assessment [Core]

In this chapter

- a practical assessment in a sandbox, in two levels: **Proficient** (the Core material) and **Mastery** (adds tasks from the Deep material)
- a checking script that tells you which tasks you completed
- a rubric and an answer key, kept **separate** in the Solutions section at the back of the book

How to read this chapter. You do the tasks on your own computer, in a sandbox that a script creates. **Nothing needs GitHub:** a local bare repository stands in for the remote. The setup and checking scripts are in `companion/assessment/`. The book's AI assistant ran the setup, the checker before any work (0 of 15), a reference solution, and the checker after (15 of 15), in Bash and zsh on Git 2.43.0 and in CI on newer Git (Appendix N says how the book was made); the reference solution is **not** part of the companion files. The checker checks **results**, not the way you reached them, and it can be wrong about unusual but valid solutions: if you think your answer is right and the checker disagrees, read the check in the script and decide for yourself. **This is a self-assessment, not a certificate.**

Before you start. You should have finished Parts I to XII of the book, or be ready to look things up while working. The assessment is **open book**: reading the manual is allowed and expected, as at work. Set aside two hours for the Proficient level and another hour for the Mastery tasks.

81.1 Prepare the sandbox

Run the setup script (sh runs it with the shell; the folder can be changed with `ASSESS_DIR`). Here is what it prints, and what the checker says before you have done anything:

```
$ sh "$STARTER/../assessment/setup.sh"
Assessment sandbox ready in /home/learner/assessment/work
$ sh "$STARTER/../assessment/check.sh"
== Proficient level
FAIL P1 identity set in this repository
FAIL P2 branch feature/tea with a commit adding tea
FAIL P3 branch feature/tea pushed to the remote
FAIL P4 debug.log ignored and not shown by status
FAIL P5 conflict resolved on branch combined
FAIL P6 bad commit reverted on main and pushed
FAIL P7 annotated tag v1.1.0 at the tip of main, pushed
FAIL P8 deleted branch precious recovered
FAIL P9 SECURITY.md and CONTRIBUTING.md on main
FAIL P10 workflow file with permissions, jobs and pinned actions
== Mastery level (extra)
FAIL M1 secret removed from all history, local and remote
FAIL M2 first bad commit found and written down
```

```

FAIL M3 shallow, sparse clone in slim
FAIL M4 release notes for v1.0.0..v1.1.0
FAIL M5 webhook signature computed
Proficient: 0 of 10. Mastery extras: 0 of 5.
Proficient level not yet reached (9 of 10 needed).

```

Recorded in Bash; *ch81-assessment/expected-assess.bash.txt*.

The setup created, under *assessment/* in your home folder:

- *remote.git*, a bare repository that plays the shared remote;
- *work*, your clone, where you do the tasks. It has an untracked *debug.log*, a branch that has been deleted, and a history with hidden problems.

Run the checker as often as you like: `sh companion/assessment/check.sh` (from the book's folder; use the path where you keep the companion files). To start again, run the setup script again: **it deletes and recreates *assessment/***.

81.2 Proficient level: ten tasks

Do all tasks in *assessment/work* unless stated. You need **9 of 10**.

P1. Identity. Set your name and email **for this repository only** (not globally).

P2. A feature branch. Create a branch *feature/tea* from *main*, add the line - Tea: 1.50 to *menu.md*, and commit it with a clear message.

P3. Publish it. Push *feature/tea* to origin.

P4. Ignore the noise. *debug.log* must stop appearing in `git status`, without deleting the file. Make the change with a file that Git tracks.

P5. A conflict. The remote has two branches, *edit-a* and *edit-b*, which changed the same line of *README.md* differently. Create a local branch *combined* that starts at origin/*edit-a* and merges origin/*edit-b*. Resolve the conflict so that the file has **no conflict markers** and still has a line that begins *Opening hours*. Commit the merge.

P6. Undo a bad commit on the shared branch. The last commit on *main*, "Set prices to zero", is wrong and already public. Undo its effect **without rewriting history**, and push. The commit itself must stay in the history.

P7. Release. After your other work on *main* is finished, create an **annotated** tag *v1.1.0* on the tip of *main* and push the tag.

P8. Recover a deleted branch. A branch named *precious* was deleted from *work* before it was merged. Bring it back with the same content, without asking anyone.

P9. Community files. Add *SECURITY.md* and *CONTRIBUTING.md* (each with real content) to *main* and push.

P10. A workflow file. Add *.github/workflows/ci.yml* to *main* (nothing runs it here; it is checked as a file). It must have a trigger, a permissions block, at least one job, and **every uses: line pinned to a full 40-character commit name**. You may reuse the commit name given in Chapter 60.

Order of work. P6, P9 and P10 change main, and P7 tags the tip, so tag last. Push main when you finish.

81.3 Mastery level: five more tasks

The Mastery level needs **all ten Proficient tasks and 4 of these 5**. Work in assessment/work unless stated; the answer files go in assessment/work.

M1. Remove the secret from history. A file `secrets.env` was committed early on. Remove it from **all history** in work **and** from the remote. **Do this last**, after your other work is pushed: rewriting works from a fresh copy of the remote's branches, so a branch that exists only on your computer would be lost. Push every branch you want to keep (combined, precious) and the tag first. (Chapter 79 describes the tool and its side effects, and what it cannot undo.)

M2. Find the culprit. `state.txt` said ok once and says broken now. Find the **first commit** that broke it **with an automated test**, not by reading commits, and write its subject line (only that) into a file `bisect-answer.txt`. Leave the repository on main afterwards.

M3. A small clone. Make a clone of the remote in `assessment/slim` (a sibling of work, not inside it) with **one commit** of history and **without the src folder**, but with docs.

M4. Release notes. Write `RELEASE_NOTES.md` listing, one per line, the subject of every commit between v1.0.0 and v1.1.0.

M5. A signature. A webhook delivery has the payload `Order 42 shipped` and the secret `bakery-test-secret` (both made up). Compute its HMAC-SHA-256 hexadecimal digest and put it alone in a file `signature.txt`. (Chapter 74.)

81.4 How to use your result

| Result | What it means |
|---|--|
| Proficient not reached (fewer than 9 of 10) | Revisit the chapters of the failed tasks (the answer key names them), then run the setup again and retry |
| Proficient reached | You can use Git and GitHub's workflow at a professional level for everyday work |
| Mastery reached | You can also handle advanced repository tasks; keep practising on real projects |

Whatever your result, **the best next step is to use these skills on a real project**, with real reviews and real mistakes. The book's Appendix N lists sources to keep learning.

Checkpoint

What You Learned

- You can be checked on results: a branch, a tag, a file, a history.

- Doing the tasks in an order that does not undo earlier ones (tag last, rewrite history last) is part of the skill.
- The checker cannot judge understanding; the answer key and your own explanations can.

New Vocabulary

No new terms.

Commands Learned

No new commands.

Common Mistakes

1. **Tagging before finishing** main.
2. **Rewriting history before pushing branches you want to keep.**
3. **Setting the identity globally when the task asks for this repository.**
4. **Trusting a test that can pass wrongly (see Chapter 80).**
5. **Reading the answer key before trying.**

Practice

The assessment is the practice. The exercises file adds three reflection tasks: the Exercises section at the end of this chapter.

Self-Test

1. Which tasks change main, and in which order should they be done?
2. Why must M1 be done last?
3. Why does the checker not tell you how you solved a task?
4. What does P6 forbid?
5. How do you start the assessment again?

Before Moving On

You have finished the book if you can:

- [] complete the Proficient level with at most one failed task
- [] explain every solution to a beginner
- [] name the next real project where you will practise

How this chapter was checked

| Claim | Evidence class | Ledger |
|---|--|--------|
| The setup and the checker (0 of 15 before, 15 of 15 after the reference solution) | Locally tested: Bash 5.2 and zsh 5.9, Git 2.43.0, git-filter-repo 2.47.0, OpenSSL, PyYAML; CI on newer Git | R354 |
| That the checker judges every valid solution correctly | Not established ; only one reference solution was tried | none |

Where this leads

This is the last chapter. The appendices give references and checklists; the glossary and the sources log (Appendix N) close the book.

Exercises

The assessment itself is the exercise. Attempt these reflection tasks after your first run and before you open the Solutions section at the back of the book.

Level 1 — Guided

1.1 Predict. Before you run the checker for the first time, write down which tasks you expect to pass. After the run, compare.

Level 2 — Partially guided

2.1 Order. List the ten Proficient tasks in the order in which you would do them, and say which tasks depend on which.

Level 3 — Independent

3.1 Redo with a constraint. Run the setup again and complete the Proficient tasks using only the manual pages (`git help ...`) and no notes. Record which tasks you needed to look up.

Level 4 — Professional scenario

4.1 Explain to a colleague. Choose three tasks and write a short explanation for a new colleague: what the task is for at work, the command you used, and one mistake to avoid.

Level 5 — Troubleshooting

5.1 The checker fails P7, but you are sure you created the tag. List four reasons the check could fail and how you would find out which one applies.

Appendix A — Git Command Reference

This appendix lists the Git commands used in the book, with one line each and the chapter where each is taught. Every command here appeared in a recorded session in the chapter named. **Options change between Git versions:** the book was recorded on Git 2.43.0 and re-run on newer versions in CI; for anything else, use `git help COMMAND`. Commands are grouped by task. "Safe" means the command does not destroy uncommitted work or rewrite shared history; "Careful" means it can.

Set-up and configuration

| Command | What it does | Safety | Chapter |
|---|----------------------------------|--------|--------------------|
| <code>git --version</code> | prints the installed version | safe | 13 |
| <code>git config --global user.name "N" and user.email "E"</code> | sets who you are for commits | safe | 14 |
| <code>git config --list --show-origin</code> | shows every setting and its file | safe | 14 |
| <code>git config --local KEY VALUE</code> | sets a value for one repository | safe | 14 |
| <code>git help COMMAND</code> | opens the manual page | safe | 35 |
| <code>git config alias.NAME "..."</code> | makes a shortcut | safe | 32 |

Starting and copying

| Command | What it does | Safety | Chapter |
|---|--|--------|--------------------|
| <code>git init</code> | makes a repository in the current folder | safe | 16 |
| <code>git init --bare</code> | makes a repository with no working files (a shared "remote") | safe | 23 |
| <code>git clone URL [DIR]</code> | copies a repository, with its history | safe | 23 |
| <code>git clone --depth 1 file:// PATH</code> | copies only the latest commit | safe | 31 |
| <code>git sparse-checkout set DIR</code> | checks out only some folders | safe | 31 |

Looking

| Command | What it does | Safety | Chapter |
|-----------------------------------|-----------------------------------|--------|--------------------|
| <code>git status (-s, -sb)</code> | shows what changed and the branch | safe | 18 |

| Command | What it does | Safety | Chapter |
|--|--|--------|--------------------|
| <code>git log (--oneline, --graph, --all, --format=...)</code> | shows history | safe | 17 |
| <code>git log -S'text'</code> | finds commits that add or remove text | safe | 17 |
| <code>git diff (--staged, A..B, A...B, --stat)</code> | shows differences | safe | 17 |
| <code>git show REV (REV:PATH)</code> | shows one commit or one file at a commit | safe | 17 |
| <code>git blame FILE</code> | shows who last changed each line | safe | 17 |
| <code>git shortlog -sn</code> | counts commits per author | safe | 51 |
| <code>git grep PATTERN</code> | searches tracked files | safe | 17 |
| <code>git describe --tags</code> | names a commit by the nearest tag | safe | 29 |
| <code>git ls-files</code> | lists tracked files | safe | 18 |

Recording changes

| Command | What it does | Safety | Chapter |
|--|---------------------------------------|-----------------------|--------------------|
| <code>git add PATH (-A)</code> | stages changes | safe | 19 |
| <code>git commit -m "msg" (-a, -s, --amend)</code> | records staged changes | amend is careful | 19 |
| <code>git rm PATH (--cached)</code> | removes a file (or stops tracking it) | careful | 18 |
| <code>git mv OLD NEW</code> | renames a tracked file | safe | 18 |
| <code>git restore PATH (--staged)</code> | discards changes or unstages | discarding is careful | 25 |

Branches and merging

| Command | What it does | Safety | Chapter |
|---|------------------------------------|-----------------------|--------------------|
| <code>git branch (-a, -vv, -d, -D)</code> | lists, creates or deletes branches | -D is careful | 20 |
| <code>git switch BRANCH (-c to create, --detach)</code> | changes branch | safe | 20 |
| <code>git merge BRANCH (--no-ff, --ff-only, --squash, --abort)</code> | joins branches | safe until you commit | 21 |
| <code>git checkout --ours FILE</code> | takes your side in a conflict | careful | 22 |

| Command | What it does | Safety | Chapter |
|---|---------------------------------|----------|--------------------|
| <code>git rebase BRANCH (--abort, --continue)</code> | replays commits on another base | rewrites | 27 |
| <code>git cherry-pick -x COMMIT</code> | copies a commit | safe | 27 |
| <code>git stash push -m "msg", git stash pop, git stash list</code> | sets work aside | safe | 24 |
| <code>git worktree add PATH BRANCH</code> | a second working folder | safe | 31 |

Working with remotes

| Command | What it does | Safety | Chapter |
|--|--|--------------------|--------------------|
| <code>git remote -v, add, set-url</code> | manages remotes | safe | 23 |
| <code>git fetch (--prune)</code> | downloads without merging | safe | 23 |
| <code>git pull (--ff-only, --rebase, --no-rebase)</code> | fetch then merge or rebase | can merge | 23 |
| <code>git push (-u, --tags, --delete)</code> | uploads | safe if not forced | 23 |
| <code>git push --force-with-lease</code> | uploads over a moved branch, if it is still what you saw | careful | 27 |
| <code>git push --force --mirror</code> | overwrites every ref on the remote | dangerous | 79 |
| <code>git ls-remote</code> | lists refs on a remote | safe | 23 |

Undoing and recovering

| Command | What it does | Safety | Chapter |
|--|---|---------------------|--------------------|
| <code>git revert COMMIT (-m 1 for merges)</code> | makes a commit that undoes another | safe | 25 |
| <code>git reset --soft, --mixed, --hard</code> | moves the branch, optionally changing the index and files | --hard is dangerous | 25 |
| <code>git reflog</code> | lists where HEAD has been | safe | 26 |
| <code>git bisect start, good, bad, run, reset</code> | finds the commit that broke something | safe | 28 |
| <code>git fsck --lost-found</code> | finds unreachable objects | safe | 30 |
| <code>git clean -n then -fd</code> | removes untracked files | -fd is dangerous | 78 |

Tags, releases and archives

| Command | What it does | Safety | Chapter |
|--|----------------------------------|--------|--------------------|
| <code>git tag -a NAME -m "msg", git tag -nl, --sort=version:refname</code> | annotated tags | safe | 29 |
| <code>git archive --prefix=P/ -o FILE REV</code> | makes an archive without history | safe | 66 |

Inside Git and other tools

| Command | What it does | Safety | Chapter |
|--|--|-----------------------------------|---|
| <code>`git cat-file -t</code>                                    | <code>-p OBJECT, git hash-object`</code> | look at objects | safe |
| <code>git gc, git maintenance</code> | tidy storage | careful with <code>--prune</code> | 30 , 32 |
| <code>`git submodule add</code>                                  | <code>update`</code> | nested repositories | careful |
| <code>git subtree</code> | nested history in a folder | careful | 31 |
| <code>git lfs track</code> | large-file support (separate tool) | safe | 31 |
| <code>git filter-repo</code> | rewrites history (separate tool) | dangerous | 33 , 79 |
| <code>git format-patch, git am, git apply</code> | patches as files | safe (am can conflict) | 28 |
| <code>git check-ref-format --branch NAME</code> | tests a branch name | safe | 20 |
| <code>git interpret-trailers, --format='% (trailers:...)'</code> | reads message trailers | safe | 64 |
| <code>git commit -S, git verify-commit</code> | signed commits | safe | 33 |

Appendix B — GitHub and CLI Reference

This appendix maps GitHub features to the chapters that describe them, and lists the GitHub CLI (gh) commands from Chapter 73. **GitHub changes often:** everything here comes from the documentation as read on 29 September 2026 (github/docs at commit 2eaab0b) and was **not run on a live account** except where a chapter says so. Interface labels are deliberately not listed, because they change; look for the idea, not the label.

B.1 GitHub features and where they are taught

| Feature | What it is for | Chapter |
|--|--------------------------------|----------------------------|
| Account, profile, two-factor authentication | who you are on the platform | 37, 38 |
| Repository (visibility, README, licence, topics) | the project's home | 39, 42 |
| Settings, Security tab, wiki | control and documentation | 43 |
| Issues, labels, milestones, forms | tracking work | 44 |
| Pull requests and merge strategies | proposing and merging changes | 45 |
| Code review, suggestions, approvals | quality control | 46 |
| Projects, Discussions | planning and conversation | 47, 48 |
| Collaborators, forks, teams | access | 49 |
| Branch protection, rulesets, code owners | rules for branches | 50 |
| Insights, graphs | activity data | 51 |
| Actions: workflows, runners, environments | automation | 54, 58, 59 |
| Dependabot, secret scanning, code scanning, advisories | security features | 62 |
| Releases | named versions | 66 |
| Pages | static websites | 69 |
| Codespaces | cloud development environments | 70 |
| Packages | package registries | 71 |
| Organizations, enterprise accounts | groups | 68 |
| REST and GraphQL APIs, webhooks, Apps | programmatic access | 74 |
| Sponsors, Marketplace, custom properties | other features | 75 |

B.2 Special files and folders

| Path | Purpose | Chapter |
|-----------|------------------------------------|--------------------|
| README.md | the front page of a repository | 42 |
| LICENSE | the licence (the owner's decision) | 65 |

| Path | Purpose | Chapter |
|---------------------------------|---------------------------------------|---|
| CONTRIBUTING.md | how to contribute | 64 |
| CODE_OF_CONDUCT.md | expected behaviour | 64 |
| SECURITY.md | how to report vulnerabilities | 62 |
| .github/workflows/*.yml | workflows | 54 |
| .github/dependabot.yml | dependency update settings | 62 |
| .github/FUNDING.yml | sponsor button | 75 |
| .github/release.yml | release-note categories | 66 |
| .devcontainer/devcontainer.json | cloud development environment | 70 |
| .gitignore, .gitattributes | what Git ignores; how it treats files | 18 , 32 |

B.3 GitHub CLI commands

Run with `gh`; add `--help` to any command. The commands below were described in [Chapter 73](#); only those marked "run" were executed for the book (with `gh 2.102.0`).

| Command | What it does | Status |
|---|-------------------------------|---|
| <code>gh --version</code> | prints the version | run |
| <code>gh alias set NAME "...", gh alias list, gh alias delete NAME</code> | shortcuts | run (set, list) |
| <code>gh config set KEY VALUE, gh config get KEY</code> | settings | run |
| <code>gh auth status, gh auth login, gh auth switch</code> | sign in and out | status run without login |
| <code>gh status</code> | your work across repositories | documented, not run |
| <code>gh repo view, clone, create</code> | repositories | documented, not run |
| <code>gh issue list, create</code> | issues | documented, not run |
| <code>gh pr list, create, checkout</code> | pull requests | list run only to show the login message |
| <code>gh search prs --review-requested=@me</code> | search | documented, not run |
| <code>gh release create TAG</code> | releases | documented, not run |
| <code>gh run rerun, gh run watch</code> | workflow runs | documented, not run |
| <code>gh codespace create, list</code> | codespaces | documented, not run |
| <code>gh api PATH</code> | a request to the API | mentioned only |

Environment variables: `GH_TOKEN` (a token instead of a login), `GH_TELEMETRY` (log, or a falsy value to disable), `DO_NOT_TRACK`.

B.4 API details worth remembering

- Link header: pagination; follow `rel="next"` until it is absent ([74](#)).
- `X-GitHub-API-Version`: pin a version.
- `X-Hub-Signature-256`: webhook signature, `sha256=` plus an HMAC hex digest; compare in constant time.
- Rate-limit headers start with `X-RateLimit-`.

Appendix C — Common Git Errors

A quick index to Chapter 78 and Chapter 79. For each message: what it means in a phrase, the **safe** first step, and the chapter that explains it. Messages are shortened; your Git may add "hint:" lines or change the wording.

| Message (shortened) | Meaning | Safe first step | Where |
|---|---|--|--------|
| fatal: not a git repository | no repository in this folder or its parents | check the folder with <code>pwd</code> ; <code>cd</code> to the project | 78 |
| fatal: pathspec 'x' did not match | Git cannot find that file or branch | <code>git status</code> , <code>git branch -a</code> | 78 |
| Author identity unknown | no name and email configured | <code>git config --global user.name</code> and <code>user.email</code> | 14, 78 |
| nothing to commit/no changes added to commit | nothing staged | <code>git status</code> , then <code>git add</code> | 78 |
| Your local changes ... would be overwritten by checkout | uncommitted edits would be lost | <code>commit</code> or <code>git stash</code> | 78, 24 |
| ! [rejected] ... (fetch first) / non-fast-forward | the remote has commits you lack | <code>git fetch</code> , read, merge or rebase | 78, 23 |
| fatal: refusing to merge unrelated histories | no common ancestor | clone instead of init; decide the base | 78 |
| error: remote origin already exists | that remote name is taken | <code>git remote -v</code> ; use another name or <code>set-url</code> | 78 |
| does not appear to be a git repository | wrong or unreachable address | <code>git remote -v</code> ; check the address | 78 |
| fatal: a branch named 'x' already exists | the name is taken | choose another name | 78 |
| error: the branch 'x' is not fully merged | unmerged commits would be lost | check <code>git log main..x</code> before <code>-D</code> | 78 |
| fatal: ambiguous argument 'x': | Git cannot tell what x is | <code>git branch -a</code> , <code>git log --oneline</code> | 78 |
| Unable to create '.git/index.lock': File exists | another Git process, or a leftover lock | make sure nothing is running, then remove the file | 78 |
| .gitignore seems ignored | the file is already tracked | <code>git rm --cached FILE</code> | 78, 18 |
| HEAD detached at ... | you are on a commit, not a branch | <code>git switch -c NAME</code> to keep work | 78 |
| CONFLICT (content): Merge conflict in ... | both sides changed the same lines | edit, <code>git add</code> , <code>commit</code> ; or <code>git merge --abort</code> | 78, 22 |

| Message (shortened) | Meaning | Safe first step | Where |
|---|--|-------------------------------|---|
| Need to specify how to reconcile divergent branches | local and remote each have new commits | choose merge or rebase | 78 , 23 |
| Permission denied (publickey) | SSH key missing or not registered | check the key on your account | 38 (not reproduced) |
| push rejected for a secret | push protection found a credential | remove it; revoke if exposed | 62 (not reproduced) |

The dangerous "fixes" to avoid (see each entry for the safer alternative): `git push --force`, `git reset --hard`, `git checkout -f`, `git branch -D`, `git clean -fd`, deleting `.git`, and `--allow-unrelated-histories` used blindly.

Appendix D — GitHub Actions YAML Reference

A compact reference to the keys and expressions used in Chapters 54 to 60. It is **not** the full syntax: for that, read GitHub's workflow syntax reference (the book checked the pages listed in the ledger against github/docs at commit 2eaab0b). Only structure and file layout were tested locally; **no workflow was run on GitHub for this book**.

D.1 File and top-level keys

| Key | Meaning | Chapter |
|------------------------------------|---|---------|
| file in .github/workflows/NAME.yml | one workflow per file | 54 |
| name | display name | 54 |
| on | the events that start the workflow | 54, 56 |
| permissions | rights of GITHUB_TOKEN (set the least that works) | 60 |
| env | environment variables for the workflow, a job or a step | 54 |
| concurrency | one run per group; cancel-in-progress | 59 |
| jobs | a map of job identifiers to jobs | 54 |

YAML reminders (Chapter 53): indentation with spaces, never tabs; quote strings that look like other types ("3.10", "no"); the key on may be read as a boolean by some parsers, which is why some tools show true.

D.2 Events (on)

| Event | When | Chapter |
|-----------------------------------|--|---------|
| push, pull_request | a push, a pull request | 54 |
| workflow_dispatch (with inputs) | started by hand | 58 |
| schedule with cron | at set times (UTC, five fields) | 56 |
| workflow_call | called by another workflow (reusable) | 58 |
| pull_request_target, workflow_run | privileged triggers: do not check out untrusted code | 60 |

D.3 Jobs and steps

| Key | Meaning | Chapter |
|---------|--|---------|
| runs-on | the runner (for example ubuntu-latest) | 54, 59 |
| needs | jobs that must finish first | 54 |
| if | condition for a job or step | 55 |

| Key | Meaning | Chapter |
|-----------------------------------|--|------------------------|
| strategy.matrix | one job per combination | 56, 59 |
| timeout-minutes | stop a job that runs too long | 59 |
| environment | the deployment environment (approvals, secrets) | 58 |
| steps | list of steps | 54 |
| step name | label | 54 |
| step uses | run an action (pin to a full commit name) | 54, 60 |
| step with | inputs for the action | 54 |
| step run | run a shell command | 54 |
| step env | variables for that step (use for untrusted values) | 60 |
| step id and steps.ID.outputs.NAME | pass values between steps | 55 |

D.4 Expressions and contexts

Written `${{ ... }}`. Common contexts: `github` (for example `github.ref`, `github.ref_name`, `github.sha`, `github.event_name`), `env`, `vars`, `secrets`, `steps`, `job`, `runner`, `matrix`, `needs`, `inputs`. Functions: `contains`, `startsWith`, `format`, `join`, and status checks `success()`, `failure()`, `always()`, `cancelled()` (Chapter [55](#)).

Danger: `${{ }}` is pasted as text into `run: scripts`. Pass untrusted values (titles, branch names, bodies) through `env` and read them as `"$NAME"` (Chapter [60](#)).

D.5 Actions used in the book's examples

| Action | Use | Chapter |
|--|----------------------|------------------------|
| actions/checkout | get the code | 54 |
| actions/upload-artifact, actions/download-artifact | keep and pass files | 54 |
| actions/configure-pages, actions/upload-pages-artifact, actions/deploy-pages | publish a Pages site | 58, 69 |

D.6 A minimal, safe skeleton

```
name: ci
on: [pull_request]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
```



```
steps:
  - uses: actions/checkout@<full 40-character commit name>
  - run: sh ci.sh
```

Replace the placeholder with a real commit name from the action's own repository (Chapter 60). This book's own workflow does exactly that.

D.7 Limits and costs

Numbers such as maximum matrix size, run time and storage are in GitHub's limits page and change; Chapter 59 gives the values as read on 29 September 2026 and says so.

Appendix E — Git Terminology Map

The 180 terms first explained in Parts I to IV (computers, version control, Git itself), with a one-line meaning and the chapter to read. Terms about GitHub are in Appendix F. The full definitions are in the Glossary.

| Term | Meaning in one line | First explained |
|-----------------------|--|----------------------------|
| .gitignore | A text file listing patterns of files Git should ignore. | Chapter 18 |
| Absolute path | A path that starts at the root and means the same wherever you are. | Chapter 2 |
| Account | A record a service keeps to represent you. | Chapter 6 |
| Administrator | An account or permission level allowed to change the computer's system settings. | Chapter 1 |
| Alias | Your own short name for a Git command. | Chapter 32 |
| Annotated tag | A tag with a message and tagger. | Chapter 24 |
| Application (program) | Software that does a specific job for a person. | Chapter 1 |
| Argument | The thing a command acts on, such as a file name. | Chapter 7 |
| Atomic commit | A commit containing one logical change. | Chapter 19 |
| Authentication | Proving who you are. | Chapter 6 |
| Authorization | Deciding what you are allowed to do once your identity is known. | Chapter 6 |
| Backup | A copy of your files kept elsewhere so they can be restored after loss. | Chapter 9 |
| Bare repository | A repository with no working files, used as a shared hub. | Chapter 23 |
| Bash | A widely used shell; the one this book uses for its examples. | Chapter 7 |
| Binary file | A file that is not plain text, such as a photo or a word-processor document. | Chapter 3 |
| Bisect | Find the commit that broke something by halving. | Chapter 28 |
| Bitbucket | Another hosting platform for Git repositories, from a different company. | Chapter 12 |
| Blob | The stored contents of one file. | Chapter 30 |
| Branch | A movable name that points to a commit. | Chapter 15 |

| Term | Meaning in one line | First explained |
|---------------------------------------|--|-----------------|
| Browser | An application for viewing web pages. | Chapter 5 |
| Byte | A unit for measuring stored information; enough for one English letter. | Chapter 1 |
| cd | Moves you into another folder. | Chapter 7 |
| Centralised version control | One server holds the only complete history; users hold working copies. | Chapter 11 |
| Character | A single unit of text: a letter, digit, symbol, emoji or space. | Chapter 4 |
| Cherry-pick | Copy one commit onto another branch. | Chapter 28 |
| Client | The computer or program that asks for something. | Chapter 5 |
| Clone | A complete copy of a repository, with all its history, made on another computer. | Chapter 11 |
| Code editor | A text editor with extra help for writing structured text such as software. | Chapter 3 |
| Command | One instruction typed to the shell. | Chapter 7 |
| Commit | A recorded snapshot with author, time, message and a link to the previous commit; also the action of making one. | Chapter 15 |
| CommonMark | A precise written specification of the core Markdown syntax. | Chapter 8 |
| Computer | A machine that follows instructions to take in, change, keep and show information. | Chapter 1 |
| Configuration (setting) | A named value that changes how Git behaves. | Chapter 14 |
| Conflict markers | The lines Git writes into a conflicted file. | Chapter 22 |
| Credential helper | A program that remembers passwords for Git. | Chapter 32 |
| CSS | A text format that describes how web pages look. | Chapter 3 |
| Current directory (working directory) | The folder that counts as 'here' when you give a relative path. | Chapter 2 |
| Dangling object | A stored object nothing points to. | Chapter 30 |
| Detached HEAD | HEAD points directly at a commit instead of a branch. | Chapter 20 |
| Diff | A report of the differences between two versions, line by line. | Chapter 17 |
| Distributed version control | Every user holds a complete copy of the repository and its history. | Chapter 11 |

| Term | Meaning in one line | First explained |
|--------------------------|--|----------------------------|
| Diverged | Each branch has commits the other lacks. | Chapter 21 |
| DNS (domain name system) | The internet's phone book: it turns names into numbers. | Chapter 5 |
| Domain name | A human-friendly name, such as example.org, for a host on the internet. | Chapter 5 |
| Empty commit | A commit with a message but no file change. | Chapter 19 |
| Encoding | The rule that turns characters into bytes and back. | Chapter 4 |
| Environment variable | A variable that programs started from the shell can read. | Chapter 7 |
| Extension | The letters after the last dot in a file name; a label for the kind of contents. | Chapter 2 |
| Fast-forward | A merge that only moves a branch pointer forward. | Chapter 21 |
| Fetch | Download new commits without changing your branches. | Chapter 23 |
| File | A named collection of information kept in storage. | Chapter 2 |
| Folder (directory) | A container that holds files and other folders. | Chapter 2 |
| Garbage collection | Git's housekeeping of stored objects. | Chapter 30 |
| Git | A free, open-source version-control system that runs on your computer. | Chapter 12 |
| git add | Stages files so they are part of the next commit. | Chapter 16 |
| Git Bash | A Bash environment for Windows that is installed with Git for Windows; it is not Git itself. | Chapter 7 |
| git branch | Lists, creates, renames and deletes branches. | Chapter 20 |
| git commit | Records the staged snapshot with a message. | Chapter 16 |
| git commit --amend | Replaces the last commit with a new one, to fix its message or add forgotten changes. | Chapter 19 |
| git config | The command that reads and changes Git's settings. | Chapter 13 |
| git init | Creates a new repository in the current folder. | Chapter 16 |
| Git LFS | A way to keep large files outside the history. | Chapter 31 |

| Term | Meaning in one line | First explained |
|--------------------------|---|-----------------|
| git log | Shows the history of commits, newest first. | Chapter 17 |
| git mv | Renames or moves a file and stages the change. | Chapter 18 |
| git reset | Move the current branch to another commit. | Chapter 25 |
| git restore | Bring a file back from the index or an older commit. | Chapter 25 |
| git revert | Undo a commit by adding a new one. | Chapter 25 |
| git rm | Deletes a file and stages the deletion. | Chapter 18 |
| git show | Displays one commit: its message and changes. | Chapter 17 |
| git status | Shows which files are untracked, modified or staged, and the current branch. | Chapter 16 |
| git switch | Changes the current branch. | Chapter 20 |
| GitHub | A widely used online platform for hosting Git repositories and collaborating on software. | Chapter 12 |
| GitLab | Another hosting platform for Git repositories. | Chapter 12 |
| Global setting | A Git setting that applies to your user account in every repository. | Chapter 13 |
| Hardware | The physical parts of a computer. | Chapter 1 |
| Hash (commit identifier) | The long code that uniquely identifies a commit. | Chapter 15 |
| HEAD | The pointer that says which commit you are currently on. | Chapter 15 |
| Hidden file | A file that ordinary listings do not show unless asked. | Chapter 2 |
| History | The recorded sequence of versions of a project, with reasons and authors. | Chapter 9 |
| Home folder | The folder that belongs to your user account. | Chapter 2 |
| Hook | A script Git runs at a fixed moment. | Chapter 32 |
| Hosting platform | An online service that stores shared repositories and adds collaboration features. | Chapter 12 |
| HTML | A text format that marks up web pages so a browser knows how to show them. | Chapter 3 |

| Term | Meaning in one line | First explained |
|-----------------------|--|-----------------|
| HTTP | The rules web clients and servers follow to exchange requests and replies. | Chapter 5 |
| HTTPS | HTTP with encryption and a check of the server's identity. | Chapter 5 |
| Hunk | One contiguous chunk of changes in a diff. | Chapter 17 |
| Ignored file | A file Git has been told to leave alone. | Chapter 18 |
| Index (staging area) | The holding area that lists exactly what the next commit will contain. | Chapter 15 |
| Installer | A file or program that puts an application on your computer. | Chapter 1 |
| Internet | The worldwide system of connected networks. | Chapter 5 |
| IP address | A number that identifies a computer on a network. | Chapter 5 |
| Key pair | A shareable public key and a private key that never leaves your computer. | Chapter 6 |
| Least privilege | Giving only the access that is needed, and no more. | Chapter 6 |
| Lightweight tag | A tag that is just a name. | Chapter 24 |
| Line ending (newline) | The invisible character or characters that end a line of text. | Chapter 4 |
| Local | On the computer you are using. | Chapter 5 |
| Local repository | The complete history, kept in the hidden .git folder on your computer. | Chapter 15 |
| Lock | A marker that stops others from changing a file until you release it. | Chapter 10 |
| Markdown | A way of writing formatted text using plain-text marks. | Chapter 8 |
| Memory (RAM) | Fast, temporary working space that is emptied when the power goes off. | Chapter 1 |
| Merge | To combine two sets of changes into one. | Chapter 10 |
| Merge commit | A commit that joins two lines of work. | Chapter 21 |
| Merge conflict | Git cannot combine two changes by itself. | Chapter 22 |
| Metadata | Information about a file, such as its size, dates, owner and permissions. | Chapter 2 |
| mkdir | Creates a new folder. | Chapter 7 |
| Mojibake | Garbled text caused by reading with the wrong encoding. | Chapter 4 |

| Term | Meaning in one line | First explained |
|-----------------------------|--|-----------------|
| Network | Computers connected so that they can exchange information. | Chapter 5 |
| Object | Any item in Git's database. | Chapter 30 |
| Operating system (OS) | The main software that manages the computer, its files and the other programs. | Chapter 1 |
| Option (flag) | A word that changes how a command behaves. | Chapter 7 |
| Override | To replace a setting's value for one command or one shell. | Chapter 14 |
| Package manager | A program that installs and updates applications from a trusted collection. | Chapter 1 |
| Packfile | One file holding many objects compactly. | Chapter 30 |
| Password | A secret string you present to prove you own an account. | Chapter 6 |
| Password manager | An application that stores all your passwords securely and fills them in. | Chapter 6 |
| Patch | A text file that describes a change. | Chapter 28 |
| Path | A written description of where a file or folder is in the tree. | Chapter 2 |
| PATH (environment variable) | The list of folders the shell searches for programs. | Chapter 7 |
| Permission | A rule about who may read, change or run a file. | Chapter 2 |
| Phishing | Tricking someone into giving away a secret by pretending to be trusted. | Chapter 6 |
| Plain text | Information stored as ordinary characters with no fonts, colours or layout. | Chapter 3 |
| Processor (CPU) | The part that carries out instructions. | Chapter 1 |
| Prompt | The text the shell shows when it is waiting for you to type. | Chapter 7 |
| Pull | Fetch, then merge into your current branch. | Chapter 23 |
| pwd | Prints the folder you are currently in. | Chapter 7 |
| Rebase | Replay your commits on top of another commit. | Chapter 27 |
| Ref | A name that points to an object. | Chapter 30 |
| Reflog | Git's private diary of where HEAD has been. | Chapter 26 |
| Relative path | A path that starts from where you are now. | Chapter 2 |

| Term | Meaning in one line | First explained |
|----------------------------|---|-----------------|
| Remote | On a different computer, reached over a network. | Chapter 5 |
| Remote-tracking branch | Your note of where a remote branch was when you last looked. | Chapter 23 |
| Renderer | A program that turns marked-up text into a formatted result. | Chapter 8 |
| Repository | A project together with the complete record of its history. | Chapter 11 |
| rerere | Git remembers how you resolved a conflict. | Chapter 32 |
| Revision | Any way of naming a commit: a hash, a branch name, HEAD, HEAD~2. | Chapter 17 |
| Root | The top folder of a tree; everything else is inside it. | Chapter 2 |
| Scope | The level at which a setting is stored: system, global, local or worktree. | Chapter 14 |
| Secret | A value that gives access if someone else has it. | Chapter 33 |
| Semantic versioning | A convention for MAJOR.MINOR.PATCH release numbers. | Chapter 29 |
| Server | A computer or program that answers requests. | Chapter 5 |
| Shallow clone | A clone with only the recent history. | Chapter 31 |
| Shell | The program that reads your commands and has them carried out. | Chapter 7 |
| Signed commit | A commit that carries a verifiable signature. | Chapter 33 |
| Single point of failure | A part whose failure stops everything or loses something irreplaceable. | Chapter 11 |
| Snapshot | A complete picture of all the files of a project at one moment. | Chapter 15 |
| Software | Instructions that tell hardware what to do. | Chapter 1 |
| Software project | The folder of files that makes up one piece of work. | Chapter 3 |
| Source code (source files) | The human-written files from which a project is made. | Chapter 3 |
| Squash | Turn many commits into one. | Chapter 21 |
| SSH (Secure Shell) | A secure way to connect to another computer, which can use a key pair to prove your identity. | Chapter 6 |
| Stage | To add a file's current contents to the index so the next commit includes them. | Chapter 15 |

| Term | Meaning in one line | First explained |
|---------------------------------|--|----------------------------|
| Stash | A shelf for unfinished changes. | Chapter 24 |
| Storage | Slower, permanent space that keeps information when the power is off. | Chapter 1 |
| Submodule | A repository pinned inside another repository. | Chapter 31 |
| Synchronised (cloud) folder | A folder whose contents are kept the same across computers or an online service. | Chapter 9 |
| Synopsis | The short line showing how to write a command. | Chapter 35 |
| Syntax highlighting | Colouring parts of text according to their role so the structure is easy to see. | Chapter 3 |
| Tag | A fixed name for one commit. | Chapter 24 |
| Tag object | The object an annotated tag creates. | Chapter 30 |
| Terminal | A window in which you type commands and read text replies. | Chapter 7 |
| Text editor | A program for creating and changing plain text files. | Chapter 3 |
| Token | A limited stand-in for a password; still a secret. | Chapter 6 |
| touch | Creates an empty file, or updates a file's last-changed time. | Chapter 7 |
| Tracked file | A file Git knows about, because it is in the last commit or in the index. | Chapter 15 |
| Tree | A stored list of a folder's contents. | Chapter 30 |
| Two-factor authentication (2FA) | Proving who you are with two proofs from different families. | Chapter 6 |
| Unicode | A universal list that gives every character a number. | Chapter 4 |
| Upstream branch | The remote branch a local branch follows. | Chapter 23 |
| URL | The written address of a page or resource on the internet. | Chapter 5 |
| Username | The name that identifies your account. | Chapter 6 |
| UTF-8 | The most widely used way to store Unicode text as bytes. | Chapter 4 |
| Variable | A named piece of text the shell remembers. | Chapter 7 |
| Version | One particular state of a file or project at a point in time. | Chapter 9 |
| Version control | A system that records a project's history so changes can be seen, reversed and combined. | Chapter 9 |

| Term | Meaning in one line | First explained |
|----------------------|--|----------------------------|
| Web (World Wide Web) | Linked pages and applications you view with a browser. | Chapter 5 |
| Word processor | A program for writing formatted documents, saved in a packaged format. | Chapter 3 |
| Workflow | The agreed way a team uses Git. | Chapter 34 |
| Working tree | The folder of files you see and edit. | Chapter 15 |
| Worktree | An extra working folder for the same repository. | Chapter 31 |
| ZIP archive | One file that holds other files in compressed form. | Chapter 3 |

Appendix F — GitHub Terminology Map

The 107 terms first explained from Part V onwards (GitHub and what is built on it). **GitHub's names and features change**; where a term depends on the platform, the chapter says which documentation it was checked against. The full definitions are in the Glossary.

| Term | Meaning in one line | First explained |
|------------------------|---|----------------------------|
| Access token | A password-like secret for programs. | Chapter 38 |
| Action | A reusable piece of code that a step can use. | Chapter 54 |
| Apex domain | A domain name with nothing in front of it, like example.com. | Chapter 69 |
| API | A way for one program to ask another to do something. | Chapter 74 |
| Artifact | A file a workflow produces and keeps. | Chapter 56 |
| Asset | A file attached to a release. | Chapter 66 |
| Attestation | A signed statement about where and how a file was built. | Chapter 63 |
| Base branch | The branch a pull request wants to change. | Chapter 45 |
| Base permission | The lowest level of access every member of an organization has to its repositories. | Chapter 68 |
| Branch protection rule | A setting that protects chosen branches from risky changes. | Chapter 50 |
| Check | A detailed result of an automated job, shown on a pull request. | Chapter 61 |
| Checksum | A short fingerprint of a file that changes if the file changes. | Chapter 63 |
| Code of conduct | A written set of rules for how people behave in a project. | Chapter 64 |
| Code owner | A person or team responsible for reviewing changes to certain files. | Chapter 49 |
| Code review | Reading someone else's proposed change before it is merged. | Chapter 46 |
| Code scanning | Automatic analysis of your code for weaknesses. | Chapter 62 |
| CodeQL | GitHub's code analysis engine. | Chapter 62 |
| Codespace | A development environment that runs in the cloud and that you use through a browser or an editor. | Chapter 70 |
| Commit status | A simple pass/fail label that a service attaches to a commit. | Chapter 61 |
| Compare branch | The branch that holds the changes in a pull request. | Chapter 45 |
| Concurrency group | A name that lets only one run at a time use a shared resource. | Chapter 59 |
| Container | An isolated, packaged environment that runs on a host computer. | Chapter 70 |
| Container image | A packaged program together with everything it needs to run. | Chapter 71 |
| Context | A collection of information that a workflow can read. | Chapter 55 |

| Term | Meaning in one line | First explained |
|-----------------------------|--|----------------------------|
| Continuous deployment | Deploying every change that passes the checks, automatically. | Chapter 72 |
| Continuous deployment (CD) | Publishing software automatically after it passes its checks. | Chapter 52 |
| Continuous integration (CI) | Checking every change automatically as soon as it is committed. | Chapter 52 |
| Contributor | A person who sends a change or report to a project. | Chapter 64 |
| Copyleft | A licence rule that what you build from the code must be shared under the same licence. | Chapter 65 |
| Copyright | The legal right of a creator over their work. | Chapter 65 |
| Creative Commons | A family of licences for text, pictures and other non-software work. | Chapter 65 |
| Cron | A compact way to write repeating times. | Chapter 56 |
| Custom property | A label with a value that you attach to repositories in an organization. | Chapter 75 |
| CVE | A public identification number for a known vulnerability. | Chapter 62 |
| Dependabot | GitHub's helper that warns about vulnerable dependencies and opens pull requests to update them. | Chapter 62 |
| Dependency graph | A list of the packages your project depends on. | Chapter 62 |
| Deployment | Putting a version of your software where its users can use it. | Chapter 72 |
| Dev container | A container set up with the tools a project needs. | Chapter 70 |
| Discussion | An open-ended conversation about a project, kept apart from the list of issues. | Chapter 48 |
| Docker | A popular tool for building and running containers. | Chapter 70 |
| Dockerfile | A text file with instructions for building a container image. | Chapter 70 |
| Enterprise account | An account that manages policy and billing for many organizations. | Chapter 68 |
| Environment | A named deployment target with its own rules and secrets. | Chapter 58 |
| Event | Something that happens in a repository and can start a workflow. | Chapter 54 |
| Expression | A small formula in a workflow that GitHub works out while running. | Chapter 55 |
| Fork | Your own hosted copy of someone else's repository. | Chapter 49 |
| GitHub App | An integration with its own limited permissions that acts on GitHub. | Chapter 74 |
| GitHub CLI | A command-line program, gh, for working with GitHub. | Chapter 73 |
| GitHub Pages | GitHub's service for publishing a static website from a repository. | Chapter 69 |
| GitHub Sponsors | GitHub's way of supporting open-source maintainers with money. | Chapter 75 |
| Governance | The written rules by which a group runs a project. | Chapter 75 |

| Term | Meaning in one line | First explained |
|----------------------|--|----------------------------|
| GraphQL | An API where you send a query that names exactly the fields you want. | Chapter 74 |
| HMAC | A fingerprint that only someone with a secret key can produce. | Chapter 74 |
| Issue | A numbered item where a problem, idea or task is written down and discussed. | Chapter 44 |
| Issue form | A form, defined in a file, that guides people to write a useful issue. | Chapter 44 |
| Iteration | A block of time in which a team plans to finish some work. | Chapter 47 |
| Job | A group of steps that run together on one machine. | Chapter 54 |
| Licence | A written permission that says what others may do with a work. | Chapter 65 |
| Lock file | A file that records the exact versions chosen. | Chapter 62 |
| Machine user | A user account made for automation instead of a person. | Chapter 68 |
| Maintainer | A person who looks after a project and decides what is merged. | Chapter 64 |
| Manifest | A file that lists the packages a project needs. | Chapter 62 |
| Matrix | A way to run the same job in several configurations. | Chapter 56 |
| Mixed content | An HTTPS page that loads some of its parts over plain HTTP. | Chapter 69 |
| Notification | A message from a platform that something happened where you are subscribed. | Chapter 41 |
| OAuth app | An integration that acts with the permissions a user gives it. | Chapter 74 |
| OIDC | A way for a workflow to get a short-lived cloud credential instead of storing a long-lived secret. | Chapter 60 |
| Open source | Software whose code is public and whose licence lets others use, change and share it. | Chapter 64 |
| Organization | A shared GitHub account that a group of people use to own repositories together. | Chapter 68 |
| Outside collaborator | Someone who is not a member of an organization but has access to some of its repositories. | Chapter 68 |
| Package | Software prepared so that a tool can install it. | Chapter 71 |
| Pagination | Splitting a long list of results into pages. | Chapter 74 |
| Permissive licence | A licence that asks only for credit and notices. | Chapter 65 |
| Pipeline | The ordered steps a change goes through automatically. | Chapter 52 |
| Pre-release | A release marked as not yet final. | Chapter 66 |
| Project | A table, board or roadmap for planning work across issues and pull requests. | Chapter 47 |
| Pull request | A proposal to merge one branch into another, with a place to discuss it first. | Chapter 45 |

| Term | Meaning in one line | First explained |
|--------------------|--|----------------------------|
| Push protection | A block that stops a push containing a secret. | Chapter 62 |
| Rate limit | A cap on how many requests you may make in a period. | Chapter 74 |
| README | A file that introduces a project to the people who arrive at it. | Chapter 42 |
| Registry | A place that stores packages for download. | Chapter 71 |
| Release | A named, downloadable version of a project on GitHub. | Chapter 66 |
| Release notes | A description of what changed in a release. | Chapter 66 |
| REST | A style of API where each kind of thing has an address and you ask with HTTP requests. | Chapter 74 |
| Reusable workflow | A workflow that other workflows can call as a whole. | Chapter 58 |
| Rollback | Going back to the previous working version. | Chapter 72 |
| Ruleset | A named list of rules that a platform enforces on branches or tags. | Chapter 50 |
| Runner | The machine that runs a job. | Chapter 54 |
| Script injection | Someone else's text becoming part of your script and being run. | Chapter 60 |
| Security advisory | A published notice about a vulnerability and its fix. | Chapter 62 |
| Self-hosted runner | A machine you run yourself to do the jobs of a workflow. | Chapter 59 |
| Sign-off | A line in a commit message saying the author has the right to submit the change. | Chapter 64 |
| Source archive | A zip or tar file of the files at a tag, without history. | Chapter 66 |
| SPDX identifier | A short standard name for a licence. | Chapter 65 |
| Staging | A practice copy of production where you deploy first. | Chapter 72 |
| Static site | A website made of files that are sent to the browser exactly as they are. | Chapter 69 |
| Step | One task inside a job. | Chapter 54 |
| Subscription | An arrangement to be told about activity in one conversation or repository. | Chapter 41 |
| Supply chain | Everything your software is made from and built with. | Chapter 63 |
| Symbolic link | A file that points to another file or folder. | Chapter 72 |
| Team | A named group of people in an organization that shares access and mentions. | Chapter 68 |
| Telemetry | Usage information a program sends back to its maker. | Chapter 73 |
| Topic | A short label that helps people find a repository. | Chapter 42 |
| Webhook | A message GitHub sends to your web address when something happens. | Chapter 74 |

| Term | Meaning in one line | First explained |
|---------------------------|---|----------------------------|
| Wiki | A section of a repository for long documentation, kept as its own Git repository. | Chapter 43 |
| Workflow (GitHub Actions) | An automated process defined in a file in the repository. | Chapter 54 |
| YAML | A readable text format for settings and lists, written with indentation. | Chapter 53 |

Appendix G — Security Checklist

A checklist that gathers the security advice of the book. Tick what you have done; a box you cannot tick is a task. **GitHub features vary by plan and repository visibility**: use the Security tab of your repository to see what is available to you (Chapter 62). Items marked Git work anywhere; items marked GitHub were taken from GitHub's documentation and were not run on a live account.

G.1 You and your account

- [] Two-factor authentication is on, and recovery codes are stored safely (GitHub; Chapter 38).
- [] You sign in with a password manager and a password used nowhere else (Chapter 6).
- [] Your SSH key has a passphrase and you know where the private key is (Chapter 38).
- [] Personal access tokens have the fewest scopes, an expiry, and a note saying what they are for; old ones are revoked (Chapter 63).
- [] You would recognise a phishing message: unexpected urgency, a link to a look-alike address (Chapter 6).

G.2 Secrets

- [] No password, key or token is in the repository, in its history, in an issue or in a log (Chapter 33).
- [] `.env` and other local settings files are in `.gitignore` **before** the first `git add` (Chapter 18).
- [] A forced add (`git add -f`) of an ignored file is treated as a warning sign (Chapter 77).
- [] Secret scanning and push protection are on where available (GitHub; Chapter 62).
- [] If a secret leaks: **revoke first**, then clean up (Chapter 79).

G.3 Repository and access

- [] `SECURITY.md` says how to report a vulnerability privately (Chapter 62).
- [] Roles are the least that work; the base permission is low; at least two owners in an organization (Chapter 68).
- [] The default branch is protected: pull requests, required checks, no force pushes (GitHub; Chapter 50).
- [] Deploy keys and outside collaborators are reviewed regularly (Chapter 68).
- [] Commits are signed where your team requires it, and you know a signature shows a key, not a good change (Chapter 63).

G.4 Dependencies and supply chain

- [] Dependabot alerts and updates are on (GitHub; Chapter 62).
- [] Every dependency was chosen on purpose: maintainer, licence, activity (Chapter 71).
- [] Lock files are committed (Chapter 62).
- [] Artifacts people run come with a checksum, and where possible an attestation (Chapter 63).

G.5 Workflows

- [] permissions is set to the least that works, at the top of each workflow (Chapter 60).
- [] Third-party actions are pinned to a full commit name from the action's own repository (Chapter 60).
- [] No untrusted value (title, branch name, body) is pasted into a run: script; use env (Chapter 60).
- [] pull_request_target and workflow_run never check out code from a fork (Chapter 60).
- [] Self-hosted runners are not used for public repositories (Chapter 59).
- [] Deployment credentials are environment secrets, least-privilege and, where possible, short-lived (Chapter 72).

G.6 Your computer

- [] You read a script before you run it, especially one that deletes things (Chapter 81).
- [] Downloads are from official sources and checked against a published checksum (Chapter 13).
- [] Extensions and integrations (editor, gh, apps) are treated as third-party code (Chapters 73 and 74).

Appendix H — Professional Repository Checklist

Use this list to review a repository before you share it, or to review someone else's. It combines Chapters [42](#), [43](#), [64](#), [65](#) and [66](#).

H.1 First impression

- [] The repository has a clear name and a one-sentence description.
- [] The README says what it is, how to try it, how to get help and how to contribute, and its examples are true (Chapter [42](#)).
- [] Topics or labels help people find it.
- [] There is a picture or a short demonstration only if it adds value, with a text alternative.

H.2 Files

- [] LICENSE exists and is **a deliberate choice of the owner** (Chapter [65](#)); the README says which licence applies.
- [] CONTRIBUTING.md, CODE_OF_CONDUCT.md (only if you can enforce it) and SECURITY.md exist (Chapters [64](#) and [62](#)).
- [] .gitignore matches the tools in use; no generated or secret files are tracked (Chapter [18](#)).
- [] Issue and pull request templates guide reporters (Chapters [44](#) and [45](#)).
- [] Nothing large or binary is committed without a reason; large files use the right tool (Chapter [31](#)).

H.3 History and branches

- [] Commit messages explain why, in the imperative, and each commit is one logical change (Chapter [19](#)).
- [] The default branch is protected and every change arrives by pull request (Chapter [50](#)).
- [] Finished branches are deleted; long-lived branches have a reason.

H.4 Automation

- [] A pipeline runs the tests on each pull request, and the same script runs locally (Chapters [52](#) and [77](#)).
- [] Workflows have least-privilege permissions and pinned actions (Chapter [60](#)).
- [] Dependency updates are automated and reviewed (Chapter [62](#)).

H.5 Releases

- [] Versions follow a stated scheme (Chapter [66](#)).
- [] Each release is an annotated tag with notes; artifacts have checksums.
- [] A hotfix path exists: branch from the tag, fix, new patch version (Chapter [76](#)).

H.6 People

- ☐ Issues get an answer, even if the answer is "not now".
- ☐ good first issue labels are kept honest (Chapter [64](#)).
- ☐ There are at least two people who can administer the repository or organization (Chapter [68](#)).

Appendix I — Open-Source Project Checklist

For someone who is about to publish a project for others to use. Chapter 67 builds one; Chapters 64, 65 and 66 explain the parts. **This book gives no legal advice; the licence is your decision.**

I.1 Before you publish

- [] I own the work, or have the right to publish it (employer, contributors, copied code) (Chapter 65).
- [] I have removed secrets and personal data, including from history (Chapters 33 and 79).
- [] I decided what others may do with the code, and chose a licence from an official text, with my own name and the correct year.
- [] The licences of everything I include are compatible with mine (a question for a qualified person if unsure).
- [] I read the rules of any platform I publish on (for example rules of use for Pages, Chapter 69).

I.2 What visitors need

- [] What it is, how to install and use it, one true example, how to run the tests.
- [] Status: "early development" or "stable", and the versioning scheme.
- [] How to report a bug and how to ask for help.
- [] How to contribute; whether sign-off is required (Chapter 64).
- [] How to report a security problem privately (Chapter 62).

I.3 Health of the project

- [] Tests exist and run on each pull request.
- [] Releases have notes, checksums and tags; old versions stay available (Chapter 66).
- [] I have said what I will not do, and how much time I have.
- [] I can hand over or close the project honestly if I can no longer maintain it (Chapter 67).

I.4 Contributing to someone else's project

- [] I read the README, CONTRIBUTING.md, the code of conduct and the licence first.
- [] I asked before starting anything large.
- [] My change is small, tested and explained; one purpose per pull request.
- [] I did not include anything I have no right to submit, and no secrets.
- [] I respond politely to review, and accept the maintainers' decision.

Appendix J — Deployment Checklist

Companion to Chapter 72. A deployment is a change to a system that people use; treat it with more care than a commit.

J.1 Before

- [] The exact commit to deploy has passed the pipeline (tests and checks), and it is the same artifact that was tried on staging.
- [] The artifact is versioned and reproducible (a tag; Chapter 66); it has a checksum.
- [] I know how to **roll back**, and I have practised it (Chapter 72).
- [] Database or data changes are planned to work with both the old and the new version.
- [] Deployment secrets are environment secrets with least privilege, short-lived where possible (Chapter 60).
- [] Approval is required for production (an environment protection rule; Chapter 58).
- [] Someone is available to watch and to react.

J.2 During

- [] Deploy to a new folder or instance; switch only when it is ready (release directories; Chapter 72).
- [] Run a health check right after the switch.
- [] Watch logs and errors for the agreed time.

J.3 After

- [] If the check fails: roll back first, investigate second.
- [] Record what was deployed, when, and by whom (the release, the run, the approval).
- [] Keep the previous release for the agreed time, then clean up.
- [] Write down anything that surprised you.

J.4 Static sites (Chapter 69)

- [] No `http://` references; no root-absolute paths on a project site; `.nojekyll` if the files need no build.
- [] Nothing sensitive in the site or in what it links to.
- [] Custom domain records copied from the current documentation; HTTPS enforced.

Appendix K — Git Recovery Decision Tree

Start at the top and answer each question honestly. Every leaf names the chapter that has the recorded steps. **Before any move that changes history, make a safety branch: `git branch backup`.** Read the state first: `git status`, `git log --oneline --graph --all`.

```

Something went wrong. What is it?
|
+-- A command printed an error message
|   -> look it up in Appendix C, then Chapter 78<!--ref:trouble-->
|
+-- I lost work
|   |
|   +-- I never committed it (only edited)
|       -> it may still be in the editor's history or a stash: `git stash list`;
otherwise it cannot be recovered by Git
|   |
|   +-- I committed it, then moved or deleted the branch or ran reset --hard
|       -> `git reflog`, find the commit, `git branch rescue COMMIT` (Chapter 26<!--
ref:reflog-->, Chapter 79<!--ref:playbooks-->)
|   |
|   +-- I deleted a branch and the reflog does not show it
|       -> `git fsck --lost-found` lists unreachable commits (Chapter 30<!--
ref:objects-->, Chapter 80<!--ref:challenges-->)
|
+-- I made a mistake in a commit
|   |
|   +-- not pushed yet
|       +-- last commit only -> `git commit --amend` (Chapter 25<!--ref:undo-->)
|       +-- several messy commits -> `git reset --soft BASE` and commit again
(Chapter 76<!--ref:scenarios-->)
|   |
|   +-- already pushed, others may have it
|       -> `git revert COMMIT` (for a merge: `-m 1`), then push (Chapter 25<!--
ref:undo-->, Chapter 79<!--ref:playbooks-->)
|
+-- I am in the middle of something
|   +-- merge with conflicts -> resolve, `git add`, `git commit`; or `git merge --
abort` (Chapter 22<!--ref:conflicts-->)
|   +-- rebase stopped -> resolve and `--continue`, or `git rebase --abort`
(Chapter 79<!--ref:playbooks-->)
|   +-- cherry-pick or am stopped -> `--abort` restores the state before (Chapter
28<!--ref:tools-->)
|   +-- detached HEAD -> `git switch -c NAME` to keep commits, or `git switch
main` (Chapter 78<!--ref:trouble-->)
|
+-- I committed on the wrong branch
|   -> `git branch NEW`, then `git reset --hard origin/BRANCH` on the wrong one
(Chapter 79<!--ref:playbooks-->)
|
+-- A push was rejected
|   -> `git fetch`, read `git log main..origin/main`, merge or rebase, push (Chapter
23<!--ref:remotes-->, Chapter 78<!--ref:trouble-->)
|       never `--force` on a shared branch; use `--force-with-lease` only on your own
|
+-- A secret was committed

```

```

|
+-- not pushed -> drop the commits or rewrite locally; still revoke if it was ever
shared (Chapter 33<!--ref:gitsec-->)
+-- pushed -> REVOKE the secret first; then decide whether a history rewrite is worth
it (Chapter 79<!--ref:playbooks-->)

```

K.1 Rules that hold at every leaf

1. Revoke a leaked secret before anything else.
2. Prefer undoing forwards (revert, a new commit) to rewriting when others have the commits.
3. Tell your team before you rewrite shared history.
4. After any recovery, run `git status` and `git log --oneline --graph --all` to confirm the result.

Appendix L — Learning Roadmap After the Book

The book gives you the foundations and a way to check what you read. The best next step is to **use these skills on real work**. This roadmap suggests an order; it recommends no product, course or vendor.

L.1 The first month

1. Repeat the capstone (Chapter 77) with an idea of your own, on GitHub this time, doing the "On GitHub" column of every step.
2. Contribute one small documentation fix to a project you use, following its CONTRIBUTING.md (Chapter 64).
3. Set up the security features on a repository you own (Appendix G).
4. Take the assessment again without notes (Chapter 81).

L.2 Months two to six

- **Read Git's own documentation** regularly, starting with the tutorial and the manual pages of commands you use (Chapter 35).
- **Read GitHub's documentation** for features your team uses, with the version for your plan selected.
- **Automate something real** with a workflow: tests, a linter, a release (Chapters 54 to 60).
- **Review other people's pull requests**: it teaches more than writing your own (Chapter 46).
- **Practise recovery** on a throw-away repository until it is calm (Chapter 79).

L.3 Going deeper (choose by need)

| If you need... | Study |
|-------------------------------|---|
| to understand Git's internals | Chapter 30 again, then the "Git Internals" material in Git's own documentation |
| to maintain a large project | branching strategies, release management, governance (Chapters 50, 75) |
| to run infrastructure | the documentation of the platform you use; container basics (Chapters 71 and 72) |
| to work on security | supply-chain security, secret management, code review for security (Chapters 62 and 63) |
| to teach others | pair with a beginner; use this book's exercises as a starting point |

L.4 Habits that last

1. **Verify before you trust**: read the source, check the date, run the command.
2. **Write things down** in the repository, so that they are versioned and reviewed.

3. **Keep changes small.**
4. **Read the state before and after each step.**
5. **Be kind in reviews and in issues.**

L.5 Keeping this book's facts current

GitHub changes faster than a book. Appendix N lists what was checked and what is open; the ledger names the source of each claim, so you can compare it with the current documentation. Where they differ, trust the current documentation.

Appendix M — Git Bash / PowerShell / Command Prompt Equivalence Guide

Git commands are the same in every shell. What differs is the surrounding shell: file commands, variables, quoting, paths and line endings. This appendix extends the table of Chapter 7.

Partly checked [R127]. The PowerShell cmdlet names, aliases and the `-Force` and `-Recurse` parameters shown here were checked against Microsoft's PowerShell 7.5 reference (the `MicrosoftDocs/PowerShell-Docs` repository); an earlier draft wrongly listed `mkdir` as an alias of `New-Item`, and the reference lists only `ni`. **Nothing was run on Windows**, and the Command Prompt column was not checked against Microsoft's documentation. Check each entry on your own machine. The Bash and zsh column was run throughout the book.

M.1 File and folder commands

| Task | Bash / zsh / Git Bash | PowerShell | Command Prompt |
|----------------------------------|--|---|---|
| Where am I? | <code>pwd</code> | <code>Get-Location</code> (alias <code>pwd</code>) | <code>cd</code> |
| List files | <code>ls</code> (<code>ls -a</code> shows hidden) | <code>Get-ChildItem</code> (alias <code>ls</code> ; <code>-Force</code> shows hidden) | <code>dir</code> (<code>dir /a</code>) |
| Change folder | <code>cd folder</code> | <code>Set-Location folder</code> (alias <code>cd</code>) | <code>cd folder</code> |
| Make a folder | <code>mkdir folder</code> | <code>New-Item -ItemType Directory folder</code> (alias <code>ni</code>) | <code>mkdir folder</code> |
| Show a file | <code>cat file</code> | <code>Get-Content file</code> (alias <code>cat</code>) | <code>type file</code> |
| Copy | <code>cp a b</code> | <code>Copy-Item a b</code> | <code>copy a b</code> |
| Move or rename | <code>mv a b</code> | <code>Move-Item a b</code> | <code>move a b</code> or <code>ren a b</code> |
| Delete a file | <code>rm file</code> | <code>Remove-Item file</code> | <code>del file</code> |
| Delete a folder and its contents | <code>rm -r folder</code> | <code>Remove-Item -Recurse folder</code> | <code>rmdir /s folder</code> |
| Search text in files | <code>grep TEXT file</code> | <code>Select-String TEXT file</code> | <code>findstr TEXT file</code> |

Deleting is permanent in all three. Check the path before you press Enter (Chapter 7).

M.2 Variables, quoting and paths

| Topic | Bash / zsh / Git Bash | PowerShell | Command Prompt |
|--------------------------------|-------------------------|---|-------------------------|
| Show a variable | echo "\$HOME" | \$env:USERPROFILE | echo %USERPROFILE% |
| Set a variable for one command | NAME=value
command | \$env:NAME = "value" first | set NAME=value
first |
| Quote text with spaces | 'single' or
"double" | 'single' or "double" | "double" |
| Path separator | / | \ (and / often works) | \ |
| Home folder | ~ | ~ | %USERPROFILE% |
| Chain commands | a && b | a; b (newer versions also
accept &&) | a & b or a && b |

Git itself accepts / in paths on every system. The book's Git commands use /.

M.3 Things that differ for Git

- **Line endings.** Windows tools often use CRLF, and Linux and macOS use LF; Git can convert (core.autocrlf, .gitattributes). Chapters 4 and 32 explain and record the effect.
- **Quotes in commit messages.** Use double quotes on Windows shells; single quotes do not group text in Command Prompt.
- **Executable scripts.** The recordings use sh SCRIPT to run scripts; on Windows use Git Bash for these.
- **Symbolic links.** Their creation on Windows may need special permission; the book's deployment recording (Chapter 72) uses Linux.
- **sha256sum** (Chapter 63) is Linux; macOS has shasum -a 256; PowerShell has Get-FileHash.

M.4 Which shell should I use?

The book uses one path: **Bash or zsh syntax**. On Windows, the simplest way to follow it is **Git Bash**, which is installed with Git for Windows. Git is not Git Bash: Git is the tool; Git Bash is a shell (Chapter 7).

Appendix N — Sources and Verification Log

This book separates what was **checked** from what was **assumed**. Every claim that could be wrong was entered in a research ledger with an evidence class. This appendix summarises the ledger and lists what remains open. The full ledger is the file `research/research-ledger.csv` in the book's companion repository (<https://github.com/Sharifwa123/Books>, folder `git-and-github-mastery`), and the list of fetched sources with their SHA-256 values is `research/sources-manifest.csv` in the same place.

N.1 Evidence classes

| Evidence class | Row
s | Meaning |
|---|----------|---|
| Both officially verified and locally tested | 127 | compared with an official source and run in the authoring environment |
| Officially verified (docs only) | 113 | compared with an official source; not run |
| Time-sensitive (unverified) | 56 | about a platform that changes; not yet checked |
| Locally tested | 34 | run in the authoring environment; no official source consulted |
| Needs re-verification | 23 | checked once; must be checked again before publication |
| Not applicable | 3 | a general concept with no product claim |

Total rows: 356.

N.2 What was tested, and where

- **Git:** every command shown was run in Bash and zsh on Git 2.43.0 (the Linux package in the authoring environment, a sandbox used by the author and the AI assistant) and re-run in continuous integration on the runner's Git (2.55.0) and on Git 2.56.0 built from source. Where newer versions print different text, alternate recordings are kept and noted.
- **GitHub:** statements were compared with the `github/docs` repository at commit `2eaab0b` (29 September 2026) and the `github/site-policy` repository, both read from their public sources. **No statement about GitHub was checked on a live account;** the book says so wherever it matters (a signed-in account of the plan).
- **Licences:** licence summaries come from GitHub's `choosealicense.com` data and the SPDX licence list (secondary sources); the official pages could not be read. The book's own licences (Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International for the text, the MIT License for code samples) are stated on its copyright page; the official Creative Commons pages could not be read from the testing computer, and the complete legal code is not reproduced in the book.
- **Quotations:** short quotations from GitHub's documentation are used with attribution; that documentation is published under the Creative Commons Attribution 4.0 licence (as stated in its repository), and the quoted passages stay under it, not under this book's licence. Quotations from other sources stay under their own terms.

- **Other tools:** the GitHub CLI 2.102.0 (checksum-verified download), git-filter-repo 2.47.0, PyYAML 6.0.1, OpenSSL.

N.3 Where the sources came from

| Host | Files fetched |
|---------------------------|---------------|
| raw.githubusercontent.com | 138 |

Files fetched with the book's tool are hashed in the manifest, so a reader can check that a source has not changed since it was read.

N.4 What is still open (79 ledger rows)

These rows were planned during research and could not be verified from an official source at the time (the official hosts were blocked, or the claim needs a live account). Many are covered in part by later, chapter-level rows; they remain open until each claim is checked. Treat each as an **individual dependency**, not as a reason to distrust the rest.

| Row | Topic | Claim to verify |
|-------|-----------------------------------|--|
| R0 32 | Git versions | current stable Git version at edition date; planned breaking changes (e.g. future default branch / hash changes) - confi |
| R0 34 | Git vs GitHub definition | GitHub's own description of itself relative to Git |
| R0 35 | Account creation & username rules | steps, username constraints, email handling, plan defaults |
| R0 36 | 2FA / passkeys / recovery | supported methods, mandatory-2FA policy, recovery codes, passkey behaviour |
| R0 37 | Repository visibility | public/private/internal definitions; who can see; plan dependencies |
| R0 38 | Repository creation options | README, .gitignore templates, licence templates, default branch name setting |
| R0 39 | Profiles & profile README | special repo naming rule, pinned repos, contribution graph counting rules |
| R0 40 | README rendering & Markdown (GFM) | supported syntax, README locations/precedence, alerts, Mermaid support |
| R0 41 | Issues | labels, assignees, milestones, closing keywords, issue forms/templates, sub-issues, issue types |
| R0 42 | Discussions | enablement, categories, Q&A/answers, plan/visibility conditions |
| R0 43 | Projects | current Projects product (views, fields, iterations, automations, roadmap), limits |
| R0 44 | Wikis | availability by plan/visibility |

| Row | Topic | Claim to verify |
|------------|--|---|
| R0 45 | Pull requests | draft PRs, reviews, suggested changes, merge methods, auto-merge, merge queue availability |
| R0 46 | Branch protection vs rulesets | capabilities, differences, plan/visibility availability, bypass |
| R0 47 | CODEOWNERS | file locations, syntax, review enforcement |
| R0 48 | Releases & tags | release creation, notes generation, assets, immutability options, source archives |
| R0 49 | Notifications, stars, watching | current behaviour and settings |
| R0 50 | Insights | Pulse, Contributors, Traffic, Commits, Code frequency, Dependency graph, Network, Forks - definitions and retention |
| R0 51 | Actions overview | workflow syntax basics, events, jobs, steps, contexts/expressions |
| R0 52 | Actions runners | GitHub-hosted labels/images, self-hosted security notes, larger runners |
| R0 53 | Actions limits & billing | included minutes, storage, concurrency limits by plan/visibility, pricing |
| R0 54 | Actions secrets & variables | repository/environment/organization scope, naming, availability to forks |
| R0 55 | Actions environments | required reviewers, protection rules, secrets scoping, plan limits |
| R0 56 | Actions permissions & GITHUB_TOKEN | default permissions, permissions key, fork PR restrictions |
| R0 57 | Actions artifacts & caching | retention, size limits, actions/cache & actions/upload-artifact current major versions |
| R0 58 | Actions matrix, concurrency, timeouts | syntax and limits |
| R0 59 | Reusable workflows & composite actions | workflow_call syntax, limits, nesting |
| R0 60 | Actions security hardening | pinning to SHA, script injection, pull_request_target risks, OIDC |
| R0 61 | Actions workflow events | schedule, workflow_dispatch, pull_request, push, release, issues triggers - exact semantics |
| R0 62 | Dependabot | alerts, security updates, version updates config file, availability |
| R0 63 | Dependency graph & dependency review | availability, supported ecosystems |

| Row | Topic | Claim to verify |
|------------|---|---|
| R0 64 | Secret scanning & push protection | availability by visibility/plan, default behaviour for public repos, supported patterns |
| R0 65 | Code scanning & CodeQL | availability by visibility/plan, supported languages, default setup |
| R0 66 | Security advisories & private vulnerability reporting | enablement and workflow |
| R0 67 | SECURITY.md & community health files | recognised files and locations, .github repo defaults |
| R0 68 | Organizations, teams, roles | role names, base permissions, repository roles, custom roles availability |
| R0 69 | Forks & upstream sync | fork behaviour, sync fork button, private-fork rules |
| R0 70 | Packages | supported registries, auth, permissions, visibility, current limits/pricing |
| R0 71 | Container registry | ghcr.io usage and auth |
| R0 72 | GitHub Pages | publishing sources, Actions-based deploy, custom domain, HTTPS, limits, plan/visibility |
| R0 73 | Codespaces | quotas, pricing, machine types, dev container config, security, availability by plan |
| R0 74 | Dev containers | devcontainer.json spec status and GitHub support |
| R0 75 | GitHub CLI | install, gh auth login, gh repo/issue/pr/workflow/release command list and flags |
| R0 76 | REST API | versioning header, auth, rate limits, pagination |
| R0 77 | GraphQL API | endpoint, auth, rate limit model |
| R0 78 | Webhooks | events, secrets, signature verification |
| R0 79 | GitHub Apps vs OAuth apps | differences and recommendations |
| R0 80 | Personal access tokens | fine-grained vs classic status, scopes, expiry policy |
| R0 81 | SSH keys on GitHub | supported key types, host fingerprints, setup steps, SSH over HTTPS port |
| R0 82 | HTTPS auth for Git | password auth removed - token/credential manager requirement |
| R0 83 | Commit signature verification | 'Verified' badge rules, SSH/GPG/S/MIME support, vigilant mode |

| Row | Topic | Claim to verify |
|------------|--|--|
| R0 84 | GitHub Enterprise | Cloud vs Server vs data residency naming and concepts |
| R0 85 | Sponsors / Marketplace | current availability and terms (neutral description only) |
| R0 86 | Copilot-related repository features | only neutral description of repo-level features - verify names |
| R0 87 | Plans and pricing | Free/Team/Enterprise feature matrix - only cite if verified; otherwise omit |
| R0 88 | UI navigation labels | names/locations of tabs, buttons, settings pages used in tutorials |
| R0 93 | Apache-2.0, GPL family, LGPL, BSD, MPL | Key conditions summarised accurately |
| R1 02 | GitHub terms vs repository licence | What rights GitHub's own Terms give other users of a public repo independent of the repo's licence (view/fork) |
| R1 03 | Git for Windows / Git Bash | What Git for Windows installs, what Git Bash is (a shell/ environment, not Git), installer options incl. line-ending and |
| R1 04 | PowerShell and Command Prompt equivalents | Correct cmdlets/commands for Appendix M (pwd/ls/cd/mkdir/ cp/mv/rm equivalents; aliases) |
| R1 05 | macOS default shell | Current default shell on macOS (zsh?) and Terminal app behaviour |
| R1 06 | Installing Git on Linux/macOS | Package-manager install commands per platform and current versions |
| R1 07 | Newlines and encodings facts | LF/CRLF conventions per OS; UTF-8 statements used in the text chapter |
| R1 08 | Current Git release and supported versions | Latest stable version and release date at publication; deprecations / planned breaking changes |
| R1 09 | Git secret-removal tools | Current recommended tool(s) for purging secrets from history and their install/usage syntax (git-filter-repo, BFG) and G |
| R1 11 | Operating-system families | Who makes Windows/macOS; Linux distributions; Unix-like relationship (Ch 1) |
| R1 13 | Path separators and drive letters | Windows uses drive letters and backslash in native tools; macOS/Linux use a single tree and slash (Ch 2); Bash, Git Bash |
| R1 14 | File-name case sensitivity | Linux file systems here are case-sensitive (tested); Windows and macOS defaults are usually case-insensitive but case-pr |
| R1 15 | Hidden files | Unix-like: leading dot hides a file from plain ls (tested); Windows uses a hidden attribute (Ch 2) |
| R1 22 | Reserved example domains | example.org/example.com are reserved for documentation (Ch 5) |

| Row | Topic | Claim to verify |
|-------|--|--|
| R1 23 | HTTPS meaning | Encryption in transit and server identity; not a guarantee of trustworthiness (Ch 5) |
| R1 24 | Authentication vs authorization; 2FA; keys | General security concepts used in Ch 6 |
| R1 28 | macOS Terminal default shell | macOS Terminal uses zsh by default (Ch 7) |
| R1 34 | Backups, synchronised folders and tracked changes: general behaviour | General descriptions of what backups, synchronised/cloud folders and word-processor tracked changes do and do not record |
| R1 36 | History of version-control systems | Eras (manual copies, local file-versioning tools, centralised systems, distributed systems) and example systems (Ch 10) |
| R1 38 | Why Git became widely used | Causes given in Ch 10: speed, distributed design, cheap branching, integrity, hosting platforms |
| R1 39 | Centralised versus distributed version control | Definitions, trade-offs, and classification of example systems (Subversion-style centralised; Git and Mercurial distribu |
| R1 40 | Git, GitHub, GitLab, Bitbucket: what each is | Git is a version-control tool; GitHub, GitLab and Bitbucket are hosting platforms built around Git repositories with col |
| R2 17 | Supply-chain guidance and incidents | general practice about hooks, submodules and dependencies; no incident examples verified (Ch 33) |

N.5 AI assistance

This book was written with an AI assistant, Claude (made by Anthropic, used through Claude Code), working with the author. The assistant drafted the text, exercises, solutions, glossary, appendices and code; ran the commands and recorded their output in a sandbox (the "authoring environment"); fetched and read the sources and kept the ledger summarised here; read the whole book for errors; built the PDF and EPUB editions and the checks; drew the draft cover; and drafted the author and licensing information. The author set the goals and rules, chose the licences and imprint name, approved the plan and is responsible for the book. The recorded output is real output from that sandbox; nothing was run by hand on a live GitHub account, on Windows or on macOS. The repository lists Claude as a contributor and co-author of its commits. The full statement, area by area, is the file `AI-ASSISTANCE.md` in the companion repository.

N.6 How to use this log

1. Before relying on a fact about GitHub, find its chapter's ledger row and read its status and date.
2. Prefer GitHub's current documentation for your plan over this book for anything time-sensitive.
3. If you find an error, the ledger tells you where the claim came from, so it can be corrected at the source.

Solutions to the Exercises

Attempt each exercise before you read its answer. Chapters are listed in book order.

Chapter 1 — What a Computer Is

Read these only after attempting the exercises.

Level 1

1.1 Answers depend on your computer. A correct answer names the operating system family (Windows, macOS or Linux) and a version.

1.2 Sample answers for a laptop: input device: keyboard or touchpad; output device: screen or speakers; storage: the internal solid-state drive. Any real example of each kind is correct. A touch screen is both an input and an output device.

Level 2

2.1 (a) storage: it is kept although the device may have been switched off. (b) memory: it has not been saved, so it exists only in the fast temporary workspace. (c) memory: a running program is loaded into memory while it runs. (Its installed files, before running, are in storage.)

2.2 Top to bottom: you; applications (for example a browser, an editor); operating system (for example Windows, macOS or Linux); hardware (processor, memory, storage, screen, network adapter). Requests flow down and results flow back up.

Level 3

3.1 A good explanation sounds like: "While you type, the computer keeps your words on a fast, temporary desk. If the power goes, the desk is cleared. Saving copies the words into a cupboard that keeps things when the power is off." Any explanation that keeps the temporary/permanent distinction is correct.

Level 4

4.1 Suggested steps: (1) do not install yet; (2) ask where the advertisement led and who makes the program; (3) look for the maker's own website through a separate search, not the advertisement; (4) check whether the task can be done with a tool your organization already approves or a web version; (5) if it is approved, install it from the official source; (6) refuse the administrator prompt if it appears without a clear reason. Two questions for the colleague: "Who told you about it, and can you show me the address you downloaded it from?" and "What exactly do you need it to do?"

Level 5

5.1 (a) The computer is not broken. The essay was unsaved, so it existed only in memory, which is emptied when the power is lost. The photos were saved in storage, which keeps information without power. (b) Save early and often; use a program's automatic-save or auto-recovery feature if it has one; keep a copy on a second device or in an online service. (Chapter 9 explains why even these habits are not the same thing as version control.)

Chapter 2 — Files, Folders and Paths

Read these only after attempting the exercises.

Level 1

1.1 and **1.2** depend on your computer. Check your tree against the diagram in section 2.2 of the chapter.

Level 2

2.1 Root (/), then home, learner, sharif-git-lab, sunrise-bakery, css; the file is style.css.

2.2 (a) style.css (b) ../menu.html (c) ../images/logo.svg

Level 3

3.1 A good answer uses lowercase names with hyphens and no spaces, for example: school-project/report.docx, school-project/photos/photo-1.jpg, photo-2.jpg, photo-3.jpg, school-project/data/results.xlsx. The two-sentence explanation should say that related things share a folder and that names describe contents without words like "final".

Level 4

4.1 One proposal: final-report/ containing chapter-one.docx, photos/img-0042.jpg. Changes: spaces and capitals removed; "FINAL", "NEW" and "v2" removed because they say nothing lasting (Chapter 9 explains why); the extension kept; the photo moved into a folder that says what it is. Names without spaces need no special quoting when they are typed in a command (Chapter 7).

Level 5

5.1 Renaming changes only the label (the extension). The contents are still SVG text, so a program that trusts the .png label tries to read it as a PNG image and fails. To obtain a real PNG, use a program that converts the picture (export or save-as PNG), which rewrites the contents.

5.2 One person's file system treats names that differ only in capitals as the same, so it can hold only one of the two files (or one overwrote the other). Rule: never create names that differ only by capitalisation; use lowercase names.

Chapter 3 — Editors and Project Folders

Level 1

1.1, **1.2** depend on your computer and editor. If line numbers are missing, search the editor's settings for "line numbers".

Level 2

2.1

| File | Text or binary | Group |
|-------------------------------------|----------------|-----------------------|
| index.html, menu.html, contact.html | text | source files |
| css/style.css | text | source file |
| images/logo.svg | text | source file and asset |
| README.md | text | documentation |

2.2 The page shows, but the logo is missing (a broken-image symbol or its alternative text). `index.html` refers to `images/logo.svg`, a relative path, which is resolved from the page's own folder. The `images` folder is no longer there, so the path leads nowhere.

Level 3

3.1 A good note says: a `.docx` file is a packaged container, not lines of text; Git can store each version but shows only that the file changed; a plain text format shows the exact lines added and removed; so keep files you want to compare closely as plain text.

Level 4

4.1 A workable recommendation: keep the schedule as a plain text or Markdown table in a repository, edited by a small number of people, with changes reviewed through pull requests (Chapter 45); note that an occasional printed or formatted copy can still be exported. One thing better in a word processor: layout and printing for people who need a formatted handout. If six non-technical people must edit at the same time, a shared online document may serve them better than Git; that is a legitimate alternative.

Level 5

5.1 Most likely the word processor added formatting or curly quotation marks, or saved in its own packaged format instead of plain text. Confirm by opening the file in a text editor: unfamiliar characters or a garbled start (such as PK) point to this. Fix: restore the original file and edit it only in a text or code editor.

5.2 (1) Did you save? Look for an unsaved marker in the editor tab. (2) Did you edit the same file the browser is showing (check the folder and file name; you might have edited a copy)? (3) Did you reload, and is the browser showing a stored older copy? Try a forced reload or reopen the file.

Chapter 4 — Text, Encodings and Newlines

Level 1

1.1 and **1.2** depend on your editor. A well-configured project shows UTF-8 and either LF or CRLF (whichever the project chose).

Level 2

2.1 naïve has five characters. Four are basic English characters (one byte each) and `i` takes two bytes, so the word is 6 bytes, plus 1 byte for the line break: **7 bytes**.

2.2 The bytes are `H`, `i`, carriage return, line feed: the text `Hi` followed by a **CRLF** line ending.

Level 3

3.1 A good answer: Git compares the bytes of a file, not what a person sees. Line endings, tabs versus spaces, trailing spaces, a missing final line break and encodings are all bytes that do not show on screen. Changing any of them changes the file, so Git reports it as changed.

Level 4

4.1 Example: (1) UTF-8 everywhere; (2) one line-ending convention for the repository (LF is a common choice), enforced by a setting in the repository rather than by each person's habits (Chapter 14 and Chapter 32); (3) show whitespace on; (4) files end with a line break; (5) no trailing spaces. Checks: a shared configuration file, a review checklist and automated checks in pull requests (Chapter 54).

Level 5

5.1 The page's bytes are UTF-8 (é is c3 a9), but the page is being read as if it were an older single-byte encoding, so each byte shows as its own character (Ã and ©). Evidence: the file's own bytes show c3 a9; the browser or page declares no encoding or the wrong one. Fix: save as UTF-8 and declare UTF-8 in the page (the starter pages contain a line that does this).

5.2 Editing on Windows changed the line endings to CRLF. The first line of the script names its interpreter, and the invisible carriage return became part of the name, so the system could not find it. Prevention: configure the editor to save that file with LF endings, and set a repository rule (Chapter 32) so that scripts always keep LF.

Chapter 5 — The Internet, the Web and Servers**Level 1**

1.1 A typical answer: `https://en.example.org/menu/cakes` → scheme `https`, host `en.example.org`, path `/menu/cakes`.

1.2 The server produced the message; it responded that the page was not found (status 404). The browser, the client, displayed it.

Level 2

2.1 (a) client; (b) server; (c) client; (d) server (the laptop answers your requests while you are the client).

2.2 Local: the photo on your phone, `http://localhost:8765/` (it names your own computer). Remote: the document in an online office suite, the page on `example.org`.

Level 3

3.1 Any accurate analogy is acceptable: for example "the internet is the postal network, and the web is one kind of mail service that runs on it; email is another".

Level 4

4.1 A good reply: the padlock means the connection is encrypted and that the server proved it owns the name in the address; it does not mean the owner is honest or that the site is safe. Check the exact spelling of the address and whether you reached the site through a link you trust; do not enter passwords unless you expected to log in.

Level 5

5.1 Check the spelling of the domain name first, then whether DNS can resolve it (does the name exist?), then whether the server is reachable. "Cannot be found" before any page appears usually means the name could not be turned into an address, or the server could not be reached, not that a page is missing.

5.2 404 means the server could not find that resource. Check whether `images/logo.svg` exists at that path relative to the page, and that the folder name and capitalisation match exactly (Chapter 2).

Chapter 6 — Accounts, Passwords and Trust**Level 1**

1.1 depends on your account. If 2FA is off, consider switching it on and saving recovery codes the same day.

1.2 Length usually matters more than character-class rules: a long passphrase has many more possible combinations than a short password with a capital and a number, and it is easier to remember. It should be unique to that service.

Level 2

2.1 (a) authentication; check the password (and the account name). (b) authorization; ask an owner or administrator to grant the permission. (c) authentication (the second factor); use the phone or a recovery code. (d) authorization (visibility): only people who have been given access can see it.

2.2 Any consistent analogy: for example at a library, showing your card is authentication and the card's borrowing rights (children's section only) are authorization.

Level 3

3.1 A good answer: "You give everyone an open padlock (which only you can open). To prove it is you, someone hands you a puzzle that needs the matching key to solve. You solve it at home with the key and send back only the answer. The key never travels."

Level 4

4.1 One plan: each person has their own account (no shared password); permissions are per person or per team, and contractors get access only to the projects they work on; a token or key is issued for each tool and each has the minimum permissions; access is reviewed regularly; when a contractor leaves, remove their access and revoke their tokens and keys the same day.

Level 5

5.1 Authorization. The colleague's identity was accepted. Someone who owns or administers the resource must grant the needed permission.

5.2 Suggested order: (1) use any other signed-in device or a backup second factor if one exists; (2) look for the service's account-recovery process and follow it exactly (expect proof of identity and possibly a waiting period); (3) once in, generate new recovery codes and store them safely; (4) add a second factor, such as a security key. Next time: save recovery codes and register a backup factor on the day 2FA is enabled.

Chapter 7 — The Terminal Without Fear

Level 1

1.1 `pwd` shows your home folder. Then: `mkdir sharif-git-lab, cd sharif-git-lab, mkdir -p sunrise-bakery/css sunrise-bakery/images, ls sunrise-bakery`.

1.2 `cd sunrise-bakery, echo "Sunrise Bakery" > README.md, echo "Fresh bread every morning." >> README.md, cat README.md`.

Level 2

2.1 `cp README.md copy.md (ls shows both), mv copy.md notes.txt (ls shows notes.txt and no copy.md), rm notes.txt (gone; no message)`.

2.2 `touch .draft; ls` omits it; `ls -a` lists `.draft`. Git's data lives in a hidden `.git` folder, invisible in ordinary listings for the same reason.

Level 3

3.1 `mkdir experiments, cd experiments, touch a.txt b.txt c.md, ls *.txt (prints a.txt b.txt), then rm a.txt b.txt c.md, cd .., rmdir experiments`.

Level 4

4.1 Do not run it. Find out what each part means by reading the documentation for `rm`; run `ls` (not `rm`) with the same wildcards to see exactly what they would match; check `pwd`; ask the colleague why; and run it only on a copy or in a practice folder. The general rule: a delete command with wildcards must be previewed with a harmless command first.

Level 5

5.1 (a) The space split the name into two arguments; quote the name (`cd "Sunrise Bakery"`) or, better, avoid spaces in names. (b) Names are case-sensitive: `Sunrise-Bakery` is not `sunrise-bakery`; check with `ls` and match the case. (c) The program's folder is not on `PATH`, or the terminal was opened before the installation; open a new terminal and check `PATH` (Chapter 13). (d) A single `>` replaced the file; use `>>` to append.

5.2 A variable set with `PATH="..."` lives only in that shell session; a new terminal starts with the default environment. To make it permanent you would put the setting in a startup file that the shell reads when it starts (the file's name depends on the shell), or install programs into a folder that is already on `PATH`. Chapter 13 shows how Git's installer handles this.

Chapter 8 — Markdown in Five Pages

Level 1

1.1

```
# Study notes

Today I learned what Markdown is.

- headings
- lists
- links
```

1.2 `[our menu](menu.html)` and `(images/logo.svg)`.

Level 2

2.1

```
Item	Price
Loaf	2.50
Rolls	3.00
Cake slice	2.00
```

Without the dashed row, the lines render as plain text, not as a table.

2.2 Two lines with no blank line: one paragraph (the lines join). With a blank line between: two paragraphs.

Level 3

Any README meeting the list is correct. Check: a `#` title, a paragraph, a `##` section, 1. 2. 3. steps, and a fenced code block.

Level 4

4.1 A six-row Markdown table with a header row and the dashed second row is a correct answer. Better for tracking: Git shows exactly which rows changed. Still better in a `.docx`: a formatted, printable rota with colours and layout for people who will not use Git.

Level 5

5.1 Add a blank line between `Ingredients:` and the first dash. It is a good habit because it works in every renderer and costs nothing.

5.2 (1) The dashed second row is missing or malformed. (2) The renderer does not support tables (they are an extension), or there is no blank line before the table.

5.3 A path that depended on where the project sat (for example an absolute path such as `/home/me/project/menu.html`). Use a relative path from the file that contains the link (Chapter 2).

Chapter 9 — The Problem of Changing Files

Level 1

1.1 Five files: `menu.txt`, `menu_v2.txt`, `menu_final.txt`, `menu_final_v2.txt`, `menu_final_REAL.txt`.

1.2 2c2 means line 2 of the first file was changed into line 2 of the second; < marks the first file's version of the line and > the second's. 2a3 means a line was added after line 2 of the first file to make the second; > shows the added line.

Level 2

2.1 Bob's copy did not contain Alice's change, and a plain file copy replaces the destination entirely. Neither file knows about the other's changes, so nothing could be flagged.

2.2 (a) which is current (1); (b) why it changed (3); (c) who did what (6); (d) can I go back (4).

Level 3

3.1 Possible rules: one person edits at a time, announced in a shared message; always copy the shared file down before editing and only then copy it back; add a date and initials to every file name; keep a log of changes. Failures: someone forgets to announce, or copies an old file back; the log is not updated; the naming becomes inconsistent.

Level 4

4.1 Two people edit different copies at nearly the same time, or someone saves an old copy over the new one; the folder synchronises whichever save arrived last, so the earlier change is replaced (or a second "conflicted" copy appears and is ignored). A synchronised folder records the latest state and, in some services, a time-based history; it does not record reasons or combine changes intelligently. Recommendation: move the price list to a text file under version control with named, explained versions, or agree an editing rota in the meantime.

Level 5

5.1 By hand: compare the three files (with `diff`), then create a new file containing the cake line and the new bread price, and copy it to the shared location. A tool should compare all three versions automatically, identify that the two changes are on different lines, and combine them without human error, reporting a conflict only when the same line was changed differently in both.

5.2 A backup keeps moments, not meaningful named versions; it records no reasons; it does not combine two people's work; and it may keep only a few recent copies.

Chapter 10 — A Short History of Version Control

Level 1

1.1 (1) dated copies: people and folders; (2) locking tools on one computer: history per file, one editor at a time; (3) central-server systems: one shared official history and whole-project tracking; (4) distributed systems: every user has the complete history.

Level 2

2.1

| Stage | Solved | Left |
|----------------|--|---|
| Dated copies | Some history | Confusion, no reasons, collisions |
| Locking tools | Collisions (one editor at a time) | People wait; single computer; per-file |
| Central server | One shared history; whole projects; parallel work with merging | Needs the network; single point of failure; awkward branching |
| Distributed | Offline work; every copy a backup; cheap experiments | Larger copies; more concepts; agreement needed on the official copy |

Level 3

A good paragraph says: the complete local history lets you read and record work without a network, makes every copy a backup, and lets you experiment privately; it also changes how teams share (by exchanging history), which shapes everything in Git.

Level 4

4.1 A good reply grants that popularity brings tools, teaching and hiring advantages, then asks: how many people, from where, with what network; do you need private experiments and parallel work; what large binary files exist; what must be controlled centrally; which platform will host it; and what will training cost.

Level 5

5.1 Examples: What is the date and what evidence supports "won"? Which other factors (platforms, network effects, free licensing) are being ignored? Who is the author and what source is cited? Is the claim about the tool or about a platform?

Chapter 11 — Centralised vs Distributed

Level 1

1.1 Centralised: a central box holding the full history; three people each with a working copy only. Distributed: each person's computer holds a full repository; optionally a shared repository on a server, also full.

Level 2

2.1 (a) distributed; (b) centralised (by design; in a distributed team by agreement); (c) distributed; (d) centralised.

Level 3

3.1 "Teams usually choose one shared repository as the meeting place, by convention. It is a copy like any other, and each person still has the whole history."

Level 4

4.1 Distributed. Points: works offline; every copy is a backup; private experiments; fast. The group must agree which shared repository is official and how changes reach it (a workflow, Chapter 34).

Level 5

5.1 The local repository holds the complete history, not only the latest files; the copy on a platform is another complete copy.

5.2 The half who say work can continue. Everyone can still read history, record work, branch and merge locally. What cannot happen until the server returns is exchanging history through it (sharing and receiving others' work), although people could exchange directly or use another shared repository.

Chapter 12 — Git, GitHub, GitLab, Bitbucket**Level 1**

1.1 Git is the version-control system; GitHub is an online platform built around Git repositories and collaboration on software. Camera/photo-site: Git takes and keeps the photos on your device; GitHub is the site that stores copies so others can see and discuss them.

Level 2

2.1 Git: recording a version; comparing two versions; creating a branch. Platform: opening an issue; reviewing a proposed change; a profile page.

Level 3

3.1 A good reply: Git is a separate program that works on your own computer, with no account; you can learn all the recording, comparing, branching and merging locally (Part III does exactly that); GitHub comes later, in Part V, and you can create the account then.

Level 4

4.1 The commands, the concepts and the way of working stay the same. The decision is about the platform: where it runs, its review and automation features, access control, cost and support.

Level 5

5.1 A platform holds a copy of the history; every contributor's clone holds a complete copy (Chapter 11). Check whether a local clone exists and, if so, copy the history back to a new shared repository.

5.2 Pull requests belong to the platform, so Git's own documentation does not describe them (in the sense used here). The platform's documentation is the place to look (Chapter 45).

Chapter 13 — Installing Git

Level 1

1.1 and **1.2** depend on your computer. On Linux the path is typically `/usr/bin/git`; on other systems it differs.

1.3 Your three settings appear as `user.name=...`, `user.email=...`, `init.defaultbranch=main`.

Level 2

2.1 `-m`, `--message <message>`: add a message from the command line instead of opening an editor.

2.2 (a) `git help merge`: the full manual. (b) `git merge -h`: a summary that needs no manual pages. (c) the online documentation, because it can be switched between versions (Chapter 35).

Level 3

3.1 Git records the name and email in every commit so that each change can be attributed; without them Git refuses to commit or guesses from the computer's name. `--global` means the setting applies to your user account in every repository, not only one.

Level 4

4.1 A good checklist: (1) download Git only from the project's official page or the company's approved software source; (2) accept the administrator prompt only when you started the installer; (3) read each installer screen; (4) open a new terminal; (5) `git --version` prints a version; (6) run the three `git config --global` commands for name, email and default branch; (7) if git is not found, close and reopen the terminal, then check the `PATH`; (8) do not download installers from links in messages.

Level 5

5.1 A terminal reads the `PATH` when it starts. A terminal that was open during the installation still has the old `PATH`, so it cannot find git. Close it and open a new one.

5.2 Another, older Git is earlier on the `PATH` and is found first. `command -v git` (or `where git` on Windows) shows which program runs; remove or reorder the older one, or run the new one by its full path.

Chapter 14 — Configuration Scopes and Environments

Level 1

1.1 Something like `global file:/home/you/.gitconfig you@example.org`.

1.2 `git config user.email` prints `lab@example.org`; `git config --get-all user.email` prints your global address first, then `lab@example.org`.

1.3 The first prints `temp@example.invalid`; the second prints the stored value. `-c` changes nothing on disk.

Level 2

2.1 After `git config --local --unset user.email`, the global value is effective again. When no value exists at any scope, the command prints nothing and its exit code is 1.

2.2 With the global file replaced by nothing, there is no `user.email` unless a local one exists, so the command prints nothing (exit code 1) if the repository has none.

Level 3

A working configuration: `git config --global includeIf.gitdir:~/work/.path ~/.gitconfig-work` with `~/gitconfig-work` containing a `[user]` section with the work email. In a repository under `~/work/`, `git config --show-origin user.email` names `.gitconfig-work`; elsewhere it names `.gitconfig`.

Level 4

4.1 Compare the settings that the two machines actually use: on each, `git config --list --show-origin --show-scope` (or `--get-all` for a name), and look at `commit.gpgsign`, `gpg.format`, `user.signingkey`. Change the setting at the narrowest scope that affects only the build server: its system file or the local `.git/config` of the build repository, not your global file.

Level 5

5.1 (1) `git config --show-origin --get commit.gpgsign` to see whether signing is on and where it is set; (2) `git config --list --show-origin --show-scope | grep -i -E "gpg|sign"` for related settings; (3) check whether a system or included file sets it. Safest way to commit meanwhile: `git -c commit.gpgsign=false commit -m "Update"`, which changes nothing on disk.

5.2 The `pull.*` settings (`pull.rebase`, `pull.ff`); compare with `git config --show-origin --get-all pull.rebase` and `pull.ff` on both computers.

5.3 Any of `commit.gpgsign`, `user.name/user.email`, `core.editor`, `init.defaultBranch`, `core.autocrlf`. Make the script explicit: pass `-c` options for what it needs, set `GIT_CONFIG_GLOBAL=/dev/null` and `GIT_CONFIG_NOSYSTEM=1` so that no inherited settings apply, and supply the identity itself.

Chapter 15 — Git's Core Model

Level 1

1.1 (a) untracked (Untracked files); (b) staged (Changes to be committed: new file); (c) clean (nothing to commit, working tree clean); (d) modified (Changes not staged for commit).

1.2 `.git/HEAD` contains `ref: refs/heads/main` (or your branch name). The branch file contains the 40-character hash of the latest commit.

Level 2

2.1 Untracked: working tree only. Staged: index (and working tree). Committed: repository (working tree and index match it). Modified after a commit: working tree only.

2.2 `a.txt` is staged (Changes to be committed); `b.txt` is untracked. After `git commit`, `a.txt` is committed and `b.txt` is still untracked.

Level 3

Any analogy that keeps the three steps apart is right. Example: "Saving is writing on a page; staging is putting the page in the folder that will be posted; committing is posting it, with a label."

Level 4

4.1 Saving only changes the working tree. Git records changes only when you stage them into the index (`git add`) and commit them into the repository (`git commit`). Until then the change is not part of any snapshot and could be lost. They should run `git add` for the files they want and then `git commit -m "..."`.

Level 5

5.1 The file was modified but not staged, so the index (which defines the commit) held nothing new. Fix: `git add menu.html` and commit again, or use `git commit -am "Update menu"` for tracked files (Chapter 19).

5.2 They deleted the repository (the history). The project files (the working tree) still exist. If a copy of the repository exists elsewhere (a clone or a remote), they can restore it from there; otherwise the history is gone and they can start again with `git init`.

Chapter 16 — Your First Repository

Level 1

1.1 Your `git log --oneline` shows three lines, newest first, each a short hash and your message.

1.2 Branch (main) received the commit; (root-commit) marks the first commit, which has no parent; the short hash names the commit; the summary says how many files and lines changed.

Level 2

2.1 `git status` lists README.md, contact.html, css/, images/, index.html, menu.html. `git add .` then `git commit -m "Add the Sunrise Bakery starter site"`.

2.2 Commit before staging: nothing added to commit but untracked files present; fix: `git add`.

Nothing to commit: nothing to commit, working tree clean; fix: none needed (no change).

Modification not staged: no changes added to commit; fix: `git add <file>` or `git commit -am`.

Level 3

Edit and save index.html, `git status` (shows modified), `git add index.html`, `git commit -m "Update the home page heading"`, `git log --oneline`.

Level 4

4.1 Stage the source files. Do not stage passwords.txt (a secret: anything committed stays in history, Chapter 33). The large photographs need a decision: they are binary and heavy; leave them out until you understand ignoring (Chapter 18) and large files (Chapter 31).

Level 5

5.1 The file was modified but not staged, so the index holds nothing new. Fix: `git add <file>` and commit again, or `git commit -am "Update"` (works for files Git already tracks).

5.2 Git now treats the whole home folder as a project. Since the repository is new and holds no commits, remove only the hidden `.git` folder in the home folder (check `pwd` and `ls -a` first); this deletes the (empty) history, not your files. Never delete anything else.

5.3 The messages do not say what changed, so history cannot be read or searched. In future write a short sentence that says what the change does (Chapter 19).

Chapter 17 — Seeing Changes and History

Level 1

1.1 Your `git log --oneline` should list three commits, newest first.

1.2 `git log`: full hash, author, date and message. `--oneline`: one line per commit. `--stat`: adds the files changed and how big. `-p -1`: adds the actual changes of the newest commit.

Level 2

2.1 Unstaged: `git diff` shows the change; `git diff --staged` shows nothing; `git diff HEAD` shows the change. After staging: `git diff` shows nothing; `git diff --staged` shows the change; `git diff HEAD` shows the change. The picture: `git diff` compares working tree and index; `--staged` compares index and last commit; `HEAD` compares working tree and last commit.

2.2 `git rev-parse HEAD~1`, `git rev-parse HEAD^` and `git rev-parse <short-hash>` print the same 40-character hash.

Level 3

Example: `git diff <first-hash> HEAD` shows the total difference; `git log --oneline <first-hash>..HEAD` lists the commits in between; `git show <hash>` shows each one.

Level 4

4.1 Find Monday's last commit with `git log --oneline` (and, if needed, the date shown by `git log`), then `git diff --stat <that-commit> HEAD` for a summary and `git diff <that-commit> HEAD` for the detail; `git log --oneline <that-commit>..HEAD` lists the commits since.

Level 5

5.1 After `git add`, the change is in the index and the working tree equals the index, so `git diff` (which compares them) is empty. `git diff --staged` shows the change.

5.2 Git is showing the output through a pager, waiting for a key. Press `q` to quit (the space bar shows the next page).

5.3 The first character of a diff line is the marker: `-` means removed, `+` means added, a space means unchanged. The rest is the line's own text: here the line is a Markdown list item that starts with `-` White loaf, so the line begins `-` (marker) then `-` White loaf (the text).

Chapter 18 — Tracking, Ignoring, Renaming, Deleting

Level 1

1.1 `.gitignore` containing `*.log, *.tmp, .env`. `git status --short` shows `?? .gitignore`; with `--ignored` it also shows `!! .env, !! build.log, !! notes.tmp`.

1.2 `git mv menu.html carte.html, git rm contact.html, git commit -m "Rename the menu page and remove the contact page"`. The stat shows a rename with 0 lines changed and a 27-line deletion.

Level 2

2.1 With `todo.txt` (no leading slash) the rule matches `todo.txt` in any folder, so `docs/todo.txt` becomes ignored too.

2.2 `git add scratch.txt, git commit, printf 'scratch.txt\n' > .gitignore; git status --short` shows only `?? .gitignore` (still tracked). Then `git rm --cached scratch.txt` and `commit`; the file stays on disk and is now ignored.

2.3 `git config --global core.excludesFile ~/.gitignore_global`, write the patterns into that file, create `.DS_Store` in a new repository, and `git check-ignore -v .DS_Store` shows the global file as the source.

Level 3

3.1 Example: `assets/raw/, dist/, *.log, .env`. An exception like `!assets/raw/logo-final.psd` does **not** work while the folder `assets/raw/` itself is ignored, because Git does not look inside an ignored folder (recorded in section 18.4.3). Fix: ignore the contents instead of the folder (`assets/raw/*` then `!assets/raw/logo-final.psd`). Test with `git check-ignore -v`.

Level 4

4.1 Add `node_modules/` to `.gitignore`; remove it from tracking with `git rm -r --cached node_modules` and `commit`; tell the colleague that dependencies are downloaded again from the project's list, not committed; before their next commit ask them to run `git status` and `git diff --staged`. Note that the files remain in earlier history (size), which Chapter 31 addresses.

Level 5

5.1 `secrets.txt` was already tracked, so ignoring has no effect. Proof: `git ls-files secrets.txt` prints the name. Fix: `git rm --cached secrets.txt`, `commit`; and if it holds a real secret, follow the incident steps (Chapter 33).

5.2 The key is still in the earlier commit and readable with `git show <commit>:.env` or `git log -p`, and in every clone. First: treat the key as stolen and **revoke or replace it** immediately. Second: add `.env` to `.gitignore` (and check no other copy exists) before considering any cleaning of history (Chapter 33).

5.3 (1) The pattern is in a different `.gitignore` (a subfolder or another repository); (2) it has a typo or trailing space; (3) a later `!` exception un-ignored the file; (4) the path does not match (for example `app.log` is inside a folder and the rule is anchored elsewhere); (5) the name is different in case.

Chapter 19 — Writing Good Commits

Level 1

1.1 Before: ... stuff; after: ... Add the lemon tart to the menu; the hash is different because the message is part of what the hash is calculated from.

1.2 `git show --stat --oneline HEAD` lists both `menu.md` and `notes.txt` under one commit.

Level 2

2.1 `git add menu.md && git commit -m "Raise the price of rolls"`, then `git add hours.md && git commit -m "Add the opening hours"`. The stat shows one file per commit.

2.2 `git commit --allow-empty -m "Subject" -m "Body paragraph"` (or with a real change). A blank line separates subject from body.

2.3 `git commit --author="Grace Baker <grace@example.net>" -m "..."`; the fuller format shows Author: and Commit: as different people.

Level 3

3.1 Sample rewrites: Fix the broken link to the menu page; Add a draft of the weekend menu; Raise the price of rolls to 3.20; Correct the address on the contact page; Add the coconut cake to the menu (and a separate commit for the CSS change, since the original mixes two things).

Level 4

4.1 A good proposal: subject in the imperative, about 50 characters, body explains why; one logical change per commit; review with `git status` and `git diff --staged`; amend only unshared commits; do not commit secrets or generated files. Git rules: the structure (first line/body separation matters to tools), that amend creates a new commit. Conventions: length, tense, footer format, the definition of "logical change".

Level 5

5.1 Amend created a new commit with a different hash. The colleague still has the old one, so the two histories now diverge from the same starting point. Advice: do not rewrite shared commits; if it has happened, tell the colleague and agree which version is kept (Chapter 23 and Chapter 27).

5.2 `git reflog` lists the old commit, and `git show <old-hash>` shows it. Chapter 26 shows how to recover it.

5.3 `-a` stages every change to every tracked file, including the unrelated one. Avoid it by staging only what belongs (`git add <file>`) and checking `git status` and `git diff --staged` before committing.

Chapter 20 — Branching

Level 1

1.1 The graph shows `add-tea` one commit ahead of `main`, with `HEAD -> add-tea`.

1.2 On main there is no tea line; on add-tea there is. Git rewrites the working tree to match the branch you switch to.

Level 2

2.1 `git switch -c fix-typo`, `git branch -m fix-typo tidy-menu`, `git switch main`, `git branch -d tidy-menu`. It succeeded because tidy-menu had no commits of its own.

2.2 It refuses because add-tea has a commit that no other branch contains; deleting it would leave that commit unreachable.

2.3 feature/add-tea works; bad name fails with fatal: 'bad name' is not a valid branch name.

Level 3

`git switch -c add-hours-page`, create and commit hours.html, `git switch main`, `ls` (no hours.html), `git switch add-hours-page`, `ls` (present).

Level 4

4.1 `git branch -a` and `git log --oneline --all --graph` to see all branches and commits; `git reflog` to see where HEAD has been. Likely explanations: the commits are on another branch (switch to it), or they were made in a detached HEAD and are on no branch (create a branch at the commit's hash before doing anything else).

Level 5

5.1 Safe: commit the changes (they become part of the current branch); or `git stash` them and `git stash pop` after switching. Unsafe: discarding them with a forced checkout or `git restore .`, which permanently loses the uncommitted edit.

5.2 The commits exist but belong to no branch, so they are unreachable except through the reflog. The learner should have created a branch (`git switch -c rescue`) before switching away. Recover with `git reflog`, find the hash, and `git branch rescue <hash>` (Chapter 26).

5.3 Git printed Deleted branch experiment (was <hash>). That hash is the tip of the deleted branch, and `git branch experiment <hash>` recreates it, as long as the commit has not been cleaned up (Chapter 26).

Chapter 21 — Merging

Level 1

1.1 The output shows Updating <old>..<new> and Fast-forward; the history stays a straight line.

1.2 main moved forward to the tea commit; add-tea did not move. Both now point to the same commit.

Level 2

2.1 `git switch -c add-tea`, edit and commit, `git switch main`, add hours.md and commit, then `git merge --no-edit add-tea`. `git log --merges` lists one commit, Merge branch 'add-tea'.

2.2 Two parents: the previous tip of main and the tip of add-tea. Note that `git show --stat` on a merge commit lists the changes brought in from the other branch.

Level 3

3.1 With `--no-ff` the graph joins two lines even though a fast-forward was possible, whereas 1.2 showed a straight line.

3.2 `git merge --squash <branch>`, `git status --short` shows the new file staged, `git commit -m "..."`. The squash commit has one parent only, so nothing in the history refers to the branch.

Level 4

4.1 Squash merge: one commit per feature, straight history. Lost: the individual commits and the record that a branch existed. Commands: `git switch main`, `git merge --squash feature`, `git commit -m "Add feature"`.

Level 5

5.1 The work is in `main`, but Git cannot see the branch's commits there because the squash created a new commit. Check with `git diff main feature` (no output means the content matches) or `git log --oneline main`; then `git branch -D feature`.

5.2 The merge went the wrong way: `git merge main` on `add-tea` brought `main` into `add-tea`, which changed `add-tea` and left `main` alone. Switch to `main` and run `git merge add-tea`.

Chapter 22 — Merge Conflicts

Level 1

1.1 Git prints `Auto-merging menu.md`, `CONFLICT (content): Merge conflict in menu.md` and `Automatic merge failed; fix conflicts and then commit the result`.

1.2 You have unmerged paths and both modified: `menu.md`. The second command prints `menu.md`.

Level 2

2.1 `HEAD` (between `<<<<<<` and `=====`) holds 2.90; the `raise-bread` side holds 3.00.

2.2 Replace the whole conflicted block with `- White loaf: 2.95`, run `git add menu.md`, then `git commit -m "Merge raise-bread, settling on 2.95"`. The graph shows a merge commit with two parents.

Level 3

3.1 After `--abort`, `git status` says the working tree is clean, `menu.md` shows 2.90, and the graph shows two diverged branches.

3.2 With `diff3`, a `|||||` line begins a section that holds the ancestor's line, `White loaf: 2.80`.

Level 4

4.1 After resolving and staging one file, `git status` still lists the other under `Unmerged paths`. Only when every file is resolved and staged does Git say `All conflicts fixed but you are still merging`; then `git commit` concludes.

Level 5

5.1 The marker lines were staged and committed because Git does not check for them. Edit the file to remove the markers and make a new commit, for example `git commit -am "Remove leftover conflict markers"`. Do not rewrite shared history.

5.2 Either no merge was in progress (it was already committed or aborted), or the conflict came from another operation such as a rebase or cherry-pick, which has its own abort command.

Chapter 23 — Remotes**Level 1**

1.1 warning: You appear to have cloned an empty repository. The hub has no commits yet.

1.2 Before the push, `git branch -vv` shows `[origin/main: gone]` because the remote branch does not exist yet. After `push -u`, it shows `[origin/main]`, and `git status` says Your branch is up to date with 'origin/main'.

Level 2

2.1 Bob sees Alice's commit with `HEAD -> main, origin/main, origin/HEAD`, and the same remote address as Alice.

2.2 Before the fetch: up to date (Git has not asked the remote). After the fetch: behind 'origin/main' by 1 commit, and can be fast-forwarded. After the pull: up to date, with a fast-forward merge.

Level 3

3.1 `! [rejected] main -> main (fetch first)`, followed by error: failed to push some refs.

3.2 `git fetch`, `git pull --no-rebase --no-edit` (or `git pull` after choosing a strategy), then `git push`. The graph shows a merge commit with two parents.

Level 4

4.1 `git init --bare backup.git`, `git remote add backup ../backup.git`, `git push backup main`, `git branch -r` lists `backup/main`, `git remote rename backup mirror`, `git remote remove mirror`. Removing the remote also removes its remote-tracking branches.

Level 5

5.1 `git status` compares with the last fetched state and does not contact the remote. Run `git fetch`, then `git status`; it will report behind.

5.2 The local and remote branches have diverged and Git will not choose for you. Three ways: `git pull --no-rebase` (merge), `git pull --rebase` (rebase, Chapter 27), or `git pull --ff-only` (refuses unless a fast-forward is possible). A setting such as `pull.rebase false` makes the choice permanent.

5.3 A forced push would make the remote branch match yours exactly, discarding the colleague's commit from the shared branch. The safe alternative is to fetch, integrate with a merge, and push normally.

Chapter 24 — Stash and Small Tools

Level 1

1.1 After `git stash`, the status is empty (for tracked files) and the list shows `stash@{0}: WIP on main: ....` `git stash pop` restores the edit and drops the entry.

1.2 The log shows `tag: v1.0, tag: v0.1` beside the commit.

Level 2

2.1 With plain `git stash`, `ideas.txt` stays because it is untracked. With `git stash -u` it is shelved too, and the tree is completely clean.

2.2 `git clean -n` lists only files; `git clean -n -d` also lists untracked folders.

Level 3

3.1 `git grep -n "Rolls"` gives `menu.md:4:...`; `git grep "loaf" HEAD~2` shows the older price with the `HEAD~2:` prefix.

3.2 The hash on line 3 names the commit that last changed that line; `git show <hash>` shows that change.

Level 4

4.1 `git stash -u`, `git switch other-branch`, make the fix, `git switch main`, `git stash pop`.

Level 5

5.1 `git stash list` shows entries; `git stash show -p stash@{0}` shows the content. (The show form was not recorded for this book; check it with your own Git before relying on it.)

5.2 No. Untracked files were never committed, so Git holds no copy. This is why `-n` comes first.

5.3 No. Deleting a tag removes only the name; the commit remains and can be found through the branch history, and the tag can be recreated on it.

Chapter 25 — Undoing Things Safely

Level 1

1.1 `git status --short` is empty and `cat menu.md` shows the committed content. The edit is gone for good.

1.2 After `git add`, the status shows `M` (first column). After `restore --staged`, it shows `M` (second column): the edit is still in the file, no longer staged.

Level 2

2.1 The file shows the older, shorter content; the status shows `M`. `git restore menu.md` returns it to the latest commit.

2.2 After `--soft`, the change is staged (`M`) and the commit is gone from the log. `git commit -m "..."` creates the new commit.

Level 3

3.1 `--soft`: log loses the commit, change staged. Default: log loses the commit, change in the file but unstaged. `--hard`: log loses the commit and the change is gone from the file.

3.2 `git reflog` lists the moves, with the lost commit at `HEAD@{1}`; `git reset --hard HEAD@{1}` restores both the commit and the file content.

Level 4

4.1 `git revert --no-edit <commit>` adds a new commit that cancels the pie line. `reset` would remove the commit from the branch, but the colleague already has it, so the histories would diverge and force a messy repair (Chapter 23).

Level 5

5.1 No. The edit was never committed, so Git holds no copy, and the reflog records only moves of HEAD. They should have committed the work to a branch or run `git stash` first.

5.2 Git cannot recover it: `restore` overwrote the file and the edit was never committed. Look outside Git (an editor's undo history or backups).

5.3 `git reflog`, to find the commit HEAD pointed to before the reset; then `git reset --hard <that commit>` (or `HEAD@{1}`).

Chapter 26 — Recovery with the Reflog

Level 1

1.1 `HEAD@{0}` is the newest entry: the current position of HEAD. The text describes the event, for example commit: Add coconut cake.

1.2 `git branch -D experiment` prints Deleted branch experiment (was <hash>).

Level 2

2.1 `git branch experiment-restored <hash>`; `git log --oneline experiment-restored` shows the commit.

2.2 `git reflog` shows a commit: line for the experiment commit; `git branch experiment-restored HEAD@{1}` (or the appropriate `HEAD@{n}`).

Level 3

3.1 Git says it is leaving one commit behind, connected to none of your branches, and prints `git branch <new-branch-name> <hash>`.

3.2 Find the commit: line in the reflog, note the hash, and run `git branch rescued <hash>`.

Level 4

4.1 `git reflog` to find the commit HEAD pointed to before the reset (often `HEAD@{1}`), then `git reset --hard HEAD@{1}` (or the hash). Check `git status` first, because `--hard` discards uncommitted changes.

Level 5

5.1 The reflog is local and is not copied by cloning. A fresh clone starts with just a `clone:` entry, and the colleague's branch was never in it.

5.2 Git cannot help: the reflog records commits and moves, not uncommitted edits. Try the editor's undo history or a backup.

5.3 `git switch -c <new-name>` from the detached state (or `git branch <new-name>` and then switch); the new branch points at the current commit and keeps all five.

Chapter 27 — Rebase**Level 1**

1.1 The graph shows two lines splitting from the same commit: `main` with "Add opening hours" and `add-tea` with "Add tea".

1.2 The graph becomes a straight line: "Add tea" now sits on top of "Add opening hours".

Level 2

2.1 The commit's parent changed, and a commit's hash covers its parent, so the replayed commit is a new commit with a new hash.

2.2 A fast-forward: `main` had not moved, so Git only moves the pointer.

Level 3

3.1 After `git rebase --abort`, `git status` says the working tree is clean, and the graph shows the original fork.

3.2 Edit the file to remove the markers, `git add menu.md`, `git rebase --continue`, and read the graph: a straight line with your resolution.

Level 4

4.1 In the list, leave line 1 as pick and change lines 2 and 3 to squash. After the rebase, `git log --oneline` shows one commit, whose message contains all three original messages.

Level 5

5.1 The colleague still has the old commits, and the learner's branch now has different commits with the same content; the two histories diverge and merging them produces duplicates and conflicts. The learner should have used `git merge` or `git revert` for shared work.

5.2 In the reflog (Chapter 26): find the entry from before rebase (`start`), and reset the branch to it.

5.3 They did not stage the resolved file: run `git add menu.md` first, then `git rebase --continue`.

Chapter 28 — Cherry-pick, Bisect and Patches

Level 1

1.1 `git cherry-pick drinks~1` on main; the graph shows a new "Add tea" on main, and the original still on drinks.

1.2 The copy has a different parent, and a commit's hash includes its parent, so it is a new commit.

Level 2

2.1 main now has copies of both commits; drinks still has the originals. The two are not linked.

2.2 The file starts with `From <hash>`, the author, the date, and `Subject: [PATCH] Add tea`.

Level 3

3.1 `git apply --check` prints nothing when the patch would apply; `git am` creates the commit with the original author and message.

3.2 Each step checks out a commit halfway through the remaining range. After the last answer, Git names the first bad commit. `git bisect reset` returns to the branch.

Level 4

4.1 About three steps for five commits; about ten steps for 1000 ($\log_2(1000)$ is roughly 10).

Level 5

5.1 The commit depended on an earlier commit that was not copied. Cherry-pick only copies the change of the one commit; pick the earlier one first, or use a merge.

5.2 `git bisect reset`.

5.3 No. The check found a problem, such as a mismatch with the current files, and applying anyway could fail halfway or change the wrong thing. Fix the cause first.

Chapter 29 — Tags and Versioning

Level 1

1.1 `git tag -a v0.1.0 <hash> -m "First draft"`; `git show v0.1.0` prints the tag's details, then the commit.

1.2 `git tag -a v1.0.0 -m "First public menu"`; `git tag` lists both.

Level 2

2.1 Plain order puts `v1.0.0-rc.1` after `v1.0.0`; `--sort=version:refname` alone does the same; with `-c versionsort.suffix=-rc` the release candidate comes first.

2.2 `git describe` prints `v1.0.0` on the tagged commit; the second prints `v0.1.0`.

Level 3

3.1 `git ls-remote --tags origin` lists only `v1.0.0` (two lines: the tag object and the peeled commit `^{}`).

3.2 After `--tags`, every local tag is there. An annotated tag shows twice (the tag object and the `^{}` commit); a lightweight tag shows once.

Level 4

4.1 Do not move or re-use the tag; others have already fetched it, and their copies would not change. Create a new correct tag, for example `git tag -a v1.0.1 <right-commit> -m "..."`, push it, and tell the team.

Level 5

5.1 `git tag -d` deletes only the local tag. Remove it on the remote with `git push origin --delete <tag>`.

5.2 Delete the unwanted tags on the remote (`git push origin --delete <tag>` for each) if nobody has used them, but people may already have fetched them. The lesson is to push named tags rather than `--tags`, and to check `git tag` first.

5.3 `git tag` sorts alphabetically, so `1.10` comes before `1.2`. Use `git tag --sort=version:refname`.

Chapter 30 — Inside Git: The Object Model**Level 1**

1.1 blob, 17, and the text `White loaf: 2.80`.

1.2 The file is `.git/objects/28/adfbf...`: the folder is the first two characters of the hash, the file name is the rest.

Level 2

2.1 No: identical content has the same hash, so it is the same object.

2.2 `printf 'blob 17\0White loaf: 2.80\n' | shasum` prints the same hash as `git hash-object`.

Level 3

3.1 The commit shows tree `<hash>`; `git cat-file -p <hash>` lists the entries (`menu.md` as a blob, `drinks` as a tree); `git cat-file -p <blob-hash>` prints the file.

3.2 `.git/HEAD` says `ref: refs/heads/<branch>`; the branch file has 41 characters: 40 for the hash and a newline.

Level 4

4.1 The lightweight tag is a ref straight to the commit, so `git cat-file -t` prints `commit`; the annotated tag is its own object and prints `tag`. `<tag>^{commit}` gives the commit hash for both.

Level 5

5.1 Deleting a branch removes only the ref (the small file). The commits stay in the database until they are unreachable and garbage collection removes them. Check with `git reflog` or `git fsck --no-reflogs`.

5.2 `count` counts loose objects. After `gc`, the objects are in a pack, so `in-pack` rose instead. Nothing was lost.

5.3 `git rev-parse main` (or `git rev-parse HEAD`). Files in `.git` may change format: refs can be stored in a packed list instead of as separate files.

Chapter 31 — Big and Complex Repositories**Level 1**

1.1 Two lines: the original folder on `main` and the new folder on `hotfix`, both at the same commit.

1.2 The graph shows the new commit on `hotfix`, ahead of `main`: one shared history.

Level 2

2.1 Yes: removing a worktree removes the folder, not the branch.

2.2 One commit, marked grafted.

Level 3

3.1 After `--unshallow` all commits appear; `--is-shallow-repository` changes from `true` to `false`.

3.2 After `set docs`, `ls` shows `docs` and the top-level files but not `src`; after `disable`, `src` is back.

Level 4

4.1 Three lines: `version`, `oid sha256:...` and `size`. The real file is stored by LFS outside the normal history.

4.2 Git prints `fatal: transport 'file' not allowed`. The override is for one command in a controlled demonstration; a permanent setting would remove a protection against malicious repositories reading local files.

Level 5

5.1 Run `git worktree prune`, which removes records of worktree folders that no longer exist. (This command was not run for the book; check `git worktree --help`.) Next time, use `git worktree remove`.

5.2 The history was cut off at the shallow boundary; `blame` cannot see beyond it. Run `git fetch --unshallow`.

5.3 The video was already committed, and it stays in the history; LFS only handles files committed after tracking begins.

Chapter 32 — Customising Git

Level 1

1.1 In the user (global) configuration, `~/.gitconfig`, when defined with `--global`.

1.2 `'st'` is aliased to `'status --short'`.

Level 2

2.1 For example `git config --global alias.hello '!echo hello'; git hello` prints `hello`.

2.2 It prints one line per alias, `alias.<name> <definition>`.

Level 3

3.1 The hook exits with a non-zero status (1); no commit is made and the change stays in the working folder, as `M` in `git status --short`.

3.2 `git commit --no-verify` skips the hook. A clone does not receive it: hooks are not copied.

Level 4

4.1 `i/lf` is the line ending in the index (what is stored), `w/crlf` is the line ending in the working file, and `attr/text eol=lf` is the attribute rule in force.

4.2 Git prints `Resolved '<file>'` using previous resolution. The status stays `UU` because Git leaves it to you to check the result, stage it and commit.

Level 5

5.1 No. A local hook can be skipped with `--no-verify` and is not copied to other clones. The rule must also be checked on the server (Chapter 54).

5.2 The token is stored in plain text in a file; anyone who can read the file can use the token. Revoke the token, switch to the operating system's protected helper, and delete the file (Chapter 33).

5.3 Hooks run code on your computer with your permissions, so an untrusted hook could do anything. Read the script before using it, or remove it.

Chapter 33 — Git Security

Level 1

1.1 `git show HEAD~1:.env` prints `API_KEY=not-a-real-key`. The deleting commit did not remove it from the earlier commit.

1.2 It lists the commit that added the key and the commit that removed it, because `-S` finds commits that change the number of occurrences of the text.

Level 2

2.1 `git status --short` does not list `.env`; `git check-ignore -v .env` prints the file, line and pattern of the rule that ignores it.

2.2 A plain `git add` refuses with a hint; `git add -f` stages the file anyway. `.gitignore` guards against accidents only; it does not stop anyone who forces.

Level 3

3.1 Both commits that touched `.env` were removed, and the commits that came after them (if any) got new hashes; earlier commits keep their hashes.

3.2 Copies that other people already have (clones, forks, downloads), copies held by a hosting platform (caches, pull requests), and the fact that the secret itself is still valid.

Level 4

4.1 1) Revoke the token. 2) Check the service's logs for misuse. 3) Tell the team (following your organization's rules). 4) Rewrite history if needed, agreed with everyone who shares the repository. 5) Add a prevention. Revoking comes first because it is the only step that makes the leaked value useless.

4.2 `gpg --batch --passphrase '' --quick-gen-key "Name <email>" default default never, git config user.signingkey <email>, git commit -S, and git log --format='%G? %s'` shows G for the signed commit and N for the unsigned one.

Level 5

5.1 Deleting a file adds a commit; the secret stays in the earlier commit and on the remote. First revoke or rotate the secret; then consider a rewrite.

5.2 Their local branch still has the old commits, and the remote now has different ones with the same content, so Git sees diverged histories. They must not simply merge or force-push the old history back; they should agree with the team how to reset their branch to the new history.

5.3 A G shows that the commit was signed with a key that Git could check, and that the commit was not changed afterwards. It does not show that the person is trustworthy or that the code is safe.

Chapter 34 — Git Workflows

Level 1

1.1 `git switch -c add-tea`, edit and commit, `git push -u origin add-tea`.

1.2 `origin/add-tea` is new; Bob's own branches did not change, because `fetch` only updates remote-tracking branches.

Level 2

2.1 That the change does what the message says, that it touches only what it should, that nothing sensitive is included (Chapter 33), and that it fits the project's style.

2.2 The graph has a merge commit joining two lines: the branch and `main`.

Level 3

3.1 `git push origin --delete add-tea` on the remote; `fetch --prune` removes `origin/add-tea`; `[origin/add-tea: gone]` says that the branch which the local one followed no longer exists on the remote.

3.2 `git branch -d` succeeds because the branch's commits are contained in `main` (Chapter 21).

Level 4

4.1 An example: every change on a short branch named for its purpose; open a review request when ready; at least one teammate reviews; merge into `main` with `--no-ff` only when tests pass; never commit straight to `main`; delete merged branches; tag `vYYYY.MM.DD` (or a version number) on `main` for each weekly release; hotfixes follow the same rules and are tagged too. Answers vary, as long as they are short and everyone could state them.

Level 5

5.1 The branch drifted while `main` moved, so many files changed on both sides. Keep branches short, merge or rebase `main` into the branch often, and split large changes into smaller ones.

5.2 Protect `main` so that changes arrive only through reviewed merges, and run automated tests before merging (Chapter 54).

5.3 Say that the team has no named workflow, and describe what it does in a few lines. Then write those lines down (for example in `CONTRIBUTING.md`) so that the next new person can read them.

Chapter 35 — Reading Official Documentation**Level 1**

1.1 `git version 2.x.y` (your own number). You can also see it in an installer, a package manager listing, or a "Programs" page, depending on your system.

1.2 For example `git tag [-a | -s | -u <key-id>] [-f] [-m <msg> | -F <file>] [-e] <tagname> [<commit> | <object>]` (your version may show a slightly different line).

Level 2

2.1 Optional: `-a/-s/-u`, `-f`, `-m/-F`, `-e`, and the final commit or object. Required: `<tagname>`. Placeholders: `<key-id>`, `<msg>`, `<file>`, `<tagname>`, `<commit>`, `<object>`.

2.2 ... means that the item may be repeated: `git tag -d v0.1 v0.2` deletes both.

Level 3

3.1 Answers vary. For example, `git switch -c <branch>` creates and switches; `git restore --staged <file>` unstages.

3.2 It depends on the system. On a system without manual pages, Git says so; then rely on `-h` and on the platform's or Git's online documentation for your version.

Level 4

4.1 Check: the date of the post; the Git version it describes; whether the command changes or deletes anything; who wrote it and how they know. Then run it in a scratch copy of the repository first, never in the real one.

Level 5

5.1 The error message itself: `git: 'comit' is not a git command`, with The most similar command is `commit`.

5.2 A different Git version (messages and advice change), a different configuration (Chapter 14), or a different operating system or shell (Chapter 7).

5.3 Ask whether it works on any Git repository without an account (then it is Git), or whether it exists only on the website or through the platform's own tool (then it is the platform). Chapter 12 gave the distinction.

Chapter 36 — What GitHub Is (and Is Not)**Level 1**

1.1 For example: "Git is a program on your computer that records the history of your files." "GitHub is an online service that hosts Git repositories and adds features for working together."

1.2 Two lines, one (`fetch`) and one (`push`), both showing the address that was added.

Level 2

2.1 Scheme `https`, host `github.com`, owner `example-owner`, repository name `sunrise-bakery` (with `.git` at the end).

2.2 Nothing has been pushed or fetched, so no remote-tracking branch exists yet, and `main` follows none (Chapter 23). `git remote add` only records the address.

Level 3

3.1 Git: `commit`, `branch`, `tag`, `.gitignore`. Platform features: pull request, issue, release page (a release page is built on a Git tag, but the page is the platform's).

3.2 A clone copies the repository's files, commits, branches and tags. It does not copy issues, pull requests or other things that the platform stores itself.

Level 4

4.1 For example: "GitHub hosts Git repositories; it does not replace Git. Every command in the book, such as `commit`, `branch`, `merge` and `undo`, ran on a computer with no account. GitHub adds a shared place and features for collaborating, but you still use Git to make and combine changes."

Level 5

5.1 Issues are stored by the platform, not in the Git repository, so a clone does not include them.

5.2 The hosted copy may not be the only place that is missing files (a mistake pushed there is copied too); the account or repository can be removed or become unavailable; the service's terms and features can change. A backup plan keeps independent copies.

5.3 The interface changes over time, and the tutorial may be old or written for a different plan. Check the tutorial's date and compare with GitHub's current documentation.

Chapter 37 — Account and Profile

Level 1

1.1 Answers vary. A good answer chooses an address that will last and can be read, and a username that is neutral and professional.

1.2 Both. Git writes the name and the email into every commit as text.

Level 2

2.1 Usually fine, if you choose: favourite language, your employer (if allowed), a general location. Usually not: your phone number and precise home details. A photo is your choice; consider who can see it.

2.2 <https://github.com/ada-bakes/sunrise-bakery> (or the same with `.git` at the end when used as a Git address).

Level 3

3.1 Use the address that you are willing to publish. The risk of the other is that it appears in every public commit and stays in history (Chapter 30). The platform may offer a special no-reply address (unverified, Chapter 38).

3.2 Answers vary: pin complete, well-described work that shows different skills.

Level 4

4.1 For example: separate accounts (or clearly separated identities) as the employer allows; separate email addresses; set `user.email` per repository with `git config` inside work repositories (Chapter 14 showed the scopes); never mix credentials; follow the employer's rules.

Level 5

5.1 Commits are permanent and identified by their contents, including the email (Chapter 30), so changing it means rewriting history and force-pushing, and copies elsewhere remain (Chapter 33). Set `user.email` before the first public commit.

5.2 The email in the commits does not match a confirmed email address on the account, so the platform cannot link them (unverified).

5.3 The graph counts platform activity by its own rules and says nothing about quality; the learner can point to what the repositories contain and their descriptions instead.

Chapter 38 — Authentication in Depth

Level 1

1.1 The first `remote -v` shows the `https://...` address twice (fetch and push); after `set-url` it shows `git@github.com:example-owner/sunrise-bakery.git` twice.

1.2 `demo_key` (private) and `demo_key.pub` (public).

Level 2

2.1 The private key has `-rw-----` (only you can read and write it). If others could read it, they could impersonate you, so SSH refuses to use it.

2.2 One line: the key type (`ssh-ed25519`), the key data, and the label. Only the `.pub` file may be shared.

Level 3

3.1 The signed commit shows `G` (good, signer in the allowed-signers file); the unsigned one shows `N`.

3.2 It prints that the signature is good and names the signer's principal.

Level 4

4.1 1) Remove the matching public key from every service that trusts it. 2) Create a new key pair (with a passphrase) and register the new public key. 3) Check the services' access logs for uses that were not theirs. 4) Tell the people who need to know. 5) Ask the chat's administrator to delete the message (a deleted message may already have been copied).

4.2 Examples: give each token the fewest permissions; give it an expiry; one token per purpose; store tokens in a credential manager or a secret store, never in the repository; revoke tokens that are unused or exposed; review the list of tokens regularly.

Level 5

5.1 An SSH key proves possession of a file on one computer. If the account's password is stolen or phished, someone can sign in on the website and change keys or settings. 2FA protects the account itself.

5.2 Restrict the permissions (for example `chmod 600 <keyfile>`). SSH refuses keys that others can read, because those keys are no longer secret.

5.3 Revoke or rotate the token at its source first. Deleting the file leaves the token in history (Chapter 33).

Chapter 39 — Your First GitHub Repository

Level 1

1.1 warning: You appear to have cloned an empty repository. The remote has no commits yet.

1.2 `main` follows `origin/main`, and neither is ahead of the other: they are in sync.

Level 2

2.1 `ls-remote` shows the hash of the remote's main; it equals `git rev-parse main` when the push succeeded.

2.2 An empty remote has no commit that yours lacks, so the push is a plain fast-forward from nothing. A remote with its own first commit has a history that yours does not contain.

Level 3

3.1 `! [rejected] main -> main (fetch first)`, with advice to pull first.

3.2 The first pull says `fatal: refusing to merge unrelated histories`; with `--allow-unrelated-histories` Git creates a merge commit. The graph has two roots joined by the merge commit, and the push then succeeds.

Level 4

4.1 Git steps: `branch -M`, `remote add`, `push -u`, `clone`, `fetch`, `pull`. Platform steps: creating the empty repository, choosing visibility, signing in, and looking at the web page.

Level 5

5.1 `rejected ... (fetch first)`, then refusing to merge unrelated histories on a plain pull. Either pull with `--allow-unrelated-histories` and push, or (if nothing valuable is on the platform yet) start again with an empty hosted repository.

5.2 The push may have failed (read its output), or the web page shows a different branch, or the page is cached; check `git ls-remote` and the branch selector.

5.3 Revoke or change the password first; the file stays in history even if it is removed later (Chapter 33). Then consider making the repository private and rewriting history.

Chapter 40 — Touring the GitHub Interface**Level 1**

1.1 It corresponds to the "history" of a single file on the page.

1.2 No. The archive contains only the files of the commit, under the folder name given by `--prefix`.

Level 2

2.1 The comparison (compare) view.

2.2 A release page presents a specific point in history, and a tag is the fixed name for a point in history; the platform builds the page on top of the tag (details in Chapter 66).

Level 3

3.1 Files: `git ls-tree HEAD`; branches: `git branch --all`; tags: `git tag`; history: `git log`; compare: `git diff <a> <b>`; blame: `git blame <file>`; download: `git archive`; clone address: `git remote -v` or `git config --get remote.origin.url`.

3.2 In a clone: commits, branches, tags, .gitignore, workflow files (they are files in the repository). Not in a clone: issues, pull requests, the wiki (kept separately by the platform).

Level 4

4.1 Answers vary. The point is to name, for each item, the Git command from section 40.2, and to record differences between the real page and this chapter.

Level 5

5.1 The archive is a snapshot of the files without the .git folder, so it is not a repository. Clone the repository to get the history.

5.2 The interface has probably changed, or the tutorial is old. Look for the idea (for example, the word or icon for "history"), check the tutorial's date, and consult the platform's current documentation.

5.3 The feature may be switched off for that repository, the colleague's role or plan may not include it, or the repository may not use it. It is not necessarily deleted.

Chapter 41 — Notifications, Stars and Watching

Level 1

1.1 Watch: subscribe to a repository's activity. Star: save the repository and show appreciation. Follow: see a person's public activity on your dashboard. Only watching subscribes you to notifications.

1.2 Answers vary with the labels on your screen. The documentation names the inbox at github.com/notifications and the list of watched repositories at github.com/watching.

Level 2

2.1 For example: you were assigned to the issue or pull request, you opened it, you commented on it, you were mentioned (or a team you belong to was), or you changed its state.

2.2 You will no longer receive notifications about activity that does not involve you: new issues and pull requests that you did not take part in, and similar general activity. You still receive conversations you are in and mentions.

Level 3

3.1 reason:review-requested.

3.2 Answers vary. The point is to record the numbers.

Level 4

4.1 A good plan: (1) watch all activity only where you maintain or depend on the code; (2) elsewhere choose participating and mentions; (3) start each day from reason:mention and reason:review-requested; (4) mark handled notifications Done and Save those that must wait; (5) use an address you control, and review the watched list monthly.

Level 5

5.1 With **Ignore**, GitHub sends nothing, and the documentation says that includes mentions ("you won't be notified if you're @mentioned"). They should have chosen the level that notifies only when they take part or are mentioned.

5.2 Notifications that are not marked Saved are kept for three months. Notifications they wanted to keep should have been marked Saved (kept indefinitely), or kept in email.

Chapter 42 — Professional Repositories and READMEs

Level 1

1.1 What the project does; why it is useful; how to get started; where to get help; who maintains it.

1.2 Answers vary. Check that the first two sentences say what the project is.

Level 2

2.1 `grep -o ](docs/CONTRIBUTING.md), and test -f docs/CONTRIBUTING.md && echo found` prints found.

2.2 `git status --short` shows a rename (`R docs/CONTRIBUTING.md -> docs/HELP.md`). It does not say that a link in the README now points to a file that does not exist: a link is plain text and Git does not track it.

Level 3

3.1 Answers vary. The install line and a short example should appear before any long explanation.

3.2 For example:

```
grep -o '](\([^)]*\))' README.md | sed 's/^\(//; s/)$//' | while read -r t; do
  case "$t" in http*) continue ;; esac
  [ -f "$t" ] || echo "missing: $t"
done
```

Level 4

4.1 Answers vary. A common fix is to move the "what it is" sentence to the very top and the install steps directly under it.

Level 5

5.1 The file was renamed or moved, and links are only text, so nothing in Git updates them. Search the repository for the old path (`git grep "docs/setup.md"`) and fix each result.

5.2 Topics use lowercase letters, numbers and hyphens, so the capital letters are wrong; and topic names are always public, so an internal tool's name should not be published as one. (Also, at 24 characters it is within the 50-character limit, so length is not the problem.)

Chapter 43 — Repository Settings, Security Tab and Wiki Tour

Level 1

1.1 The stand-in repository `demo.wiki.git` has its own history (the commit that created `Home.md`), and your clone has a copy of it. A real wiki's history is separate from the code repository's.

1.2 Visibility, your role, the plan, and the settings of the repository and its organization.

Level 2

2.1 The pull brings the commit that added `Recipes.md`; it is an ordinary fetch and merge.

2.2 No: the documentation says only changes pushed to the wiki's default branch are made live.

Level 3

3.1 For example: outside readers benefit from a wiki that people without write access can be allowed to edit; but changes to a `docs` folder are reviewed by pull request and versioned with the code. Any two sound reasons from the trade-off are accepted.

3.2 For example: confirm a clone has all branches and tags; export what is not in Git (issues, wiki); tell the collaborators; check for forks that will be deleted; confirm you can restore within 90 days if needed.

Level 4

4.1 Check for secrets in the whole history (Chapter 33); know that Actions history and logs become visible to everyone; know that private forks are detached; then remove or rotate any secret before the change.

Level 5

5.1 The documentation source names the tab "Security and quality" or "Security" depending on a feature flag. It is not a problem: it is the same tab.

5.2 Some deleted repositories can be restored within 90 days. A repository that was part of a fork network that is not empty cannot be restored unless every other repository in the network is deleted or detached.

Chapter 44 — Issues

Level 1

1.1 `git ls-files .github` prints `.github/ISSUE_TEMPLATE/bug.yml`.

1.2 A title that states the problem; what you did, expected and saw; where; and evidence (with no secrets).

Level 2

2.1 `git commit -am "Change" -m "Fixes #12"` stores the words in the message; `git log --oneline --grep='#12'` lists the commit.

2.2 Fixes octo-org/octo-repo#100.

Level 3

3.1 For example: "Rolls price is 3.00 in menu.md but 3.20 in the shop page".

3.2 For example bug, documentation, question. Few labels stay meaningful, and everybody uses them the same way.

Level 4

4.1 Answers vary. A bug form might have: what happened (required textarea), what you expected (required textarea), version (input), and a checkbox confirming that no secrets are included. A question form might have just the question and what you have already tried.

Level 5

5.1 The pull request may not have targeted the default branch (keywords are ignored for other branches); or the keyword was not in a supported form or was not followed by a valid issue reference.

5.2 Issue templates must be on the default branch; a template on another branch is not available. Merge it into the default branch.

Chapter 45 — Pull Requests

Level 1

1.1 The base branch is where the change is going (usually main); the compare branch is the topic branch that holds the commits.

1.2 The three-dot form (main...add-tea) shows only the line the branch added. The two-dot form also shows the change made on main, reversed.

Level 2

2.1 Merge: two topic commits plus a merge commit, joined. Squash: one new commit, the topic commits absent. Rebase: two commits in a straight line after the other change on the base.

2.2 A commit's hash covers its parent, so putting the commits on a new base gives them new hashes (Chapter 27).

Level 3

3.1 `git init --bare hub.git`; push a branch; `git -C hub.git update-ref refs/pull/1/head <commit>`; then in a clone `git fetch origin pull/1/head:pr-1` and `git switch pr-1`.

3.2 `git -C hub.git config receive.hideRefs refs/pull`, then `git push origin HEAD:refs/pull/1/head` prints `! [remote rejected] HEAD -> refs/pull/1/head (deny updating a hidden ref)`.

Level 4

4.1 Squash and merge: many noisy commits become one. The risk (from the documentation) is that if the contributor keeps working on the same head branch, later pull requests can include commits already squashed into the base, causing repeated conflicts; start the next change from a fresh branch off the updated base.

Level 5

5.1 A two-dot diff compares the two tips, so it includes changes made on the base after the branch diverged. The page shows the three-dot comparison: `git diff main...topic`.

5.2 Revert the pull request (the platform creates a new pull request that reverts the merge commit), or revert commits yourself with `git revert` (with `-m 1` for a merge commit). Prefer the command line when the platform's revert conflicts or the pull request was not merged on the platform.

Chapter 46 — Code Review**Level 1**

1.1 Comment: general feedback without approving or requesting changes. Approve: the changes are ready to merge. Request changes: feedback the author should address before merging.

1.2 The command prints Sam Reviewer <sam@example.org>.

Level 2

2.1 Ada is counted for each commit she authored, and the reviewer once for the commit with the trailer (in the recording: 2 and 1).

2.2 For example: "Line 4: 'slise' looks like a typo for 'slice'. Could you check the spelling?"

Level 3

3.1 Answers vary. A sensible order: purpose, correctness, safety, tests and checks, clarity, style.

3.2 Missing test, bug and missing null check: must fix. Spelling in a comment and variable name: nits, unless the name is misleading.

Level 4

4.1 Answers vary.

Level 5

5.1 The approval was about the earlier version of the change; new commits were merged without a fresh look. Look again after every push before approving, and repositories can require re-approval after new commits (Chapter 50).

5.2 The email in the Co-authored-by line may not be associated with the reviewer's account (Chapter 37 explains email matching), or it may be a different address from the one the account confirmed.

Chapter 47 — GitHub Projects

Level 1

1.1 Table: which items are unassigned? Board: what is in progress? Roadmap: what is planned for next month?

1.2 Date (target ship date), number (complexity), single select (priority), text (a note), iteration (a weekly block).

Level 2

2.1 (a) `label:bug status:"In progress"`; (b) `label:bug,support`; (c) `assignee:@me last-updated:5days` (with a status if wanted).

2.2 `no:label no:assignee repo:octocat/game`.

Level 3

3.1 Answers vary. For example statuses Todo, In progress, Done; fields Priority (single select) and Target date; views: Backlog (table, status:todo), Board (board, no filter), Roadmap (roadmap, grouped by iteration).

Level 4

4.1 The public can see the project, its fields, README and status updates. Items from private repositories appear hidden to people without access to those repositories. Avoid secrets, private customer names and unannounced details in titles, fields and updates.

Level 5

5.1 When you filter a view and add an item, the filtered metadata is applied to the new item.

5.2 The built-in workflow that sets Done when an issue or pull request is closed; check that it is enabled.

Chapter 48 — GitHub Discussions

Level 1

1.1 (a) discussion (Q&A); (b) issue; (c) pull request; (d) discussion.

1.2 Announcements, General, Ideas, Polls, Q&A, Show and tell.

Level 2

2.1 Open an issue for the action and link the discussion.

2.2 For example: mark an answer in a Q&A discussion once a comment solves it; lock a conversation that violates the code of conduct.

Level 3

3.1 Answers vary. For example: Announcements (announcement), Q&A (question and answer), Ideas (open-ended), Show and tell (open-ended).

Level 4

4.1 Create a Q&A category, point the contributing guidelines and the issue chooser to it for usage questions, and keep issues for reproducible bugs and planned work. Convert misfiled issues to discussions.

Level 5

5.1 An administrator has not enabled Discussions for the repository, or the person does not have access to the repository.

5.2 Lock the discussion (it is a documented use of locking for announcements), and point people to another category for comments.

Chapter 49 — Collaboration and Access**Level 1**

1.1 Read, Triage, Write, Maintain, Admin.

1.2 origin points to fork.git for fetch and push.

Level 2

2.1 `git remote -v` now lists origin and upstream, each with fetch and push.

2.2 The graph shows upstream/main one commit ahead of your main, then after the fast-forward and push all of main, origin/main and upstream/main on the same commit.

Level 3

3.1 `git merge --ff-only upstream/main` refuses (it cannot fast-forward because the histories diverged). It teaches why the fork's main should stay a mirror of the upstream, and why changes go on branches.

3.2 * @example-owner and menu.md @ada (the documentation says "the last matching pattern takes the most precedence"). The file may live in .github/, the root or docs/; a pull request uses the copy on its **base branch**.

Level 4

4.1 Maintainer: Maintain (or Admin if they must manage security). Developers: Write. Designer: Triage. Visitor: Read. Least privilege.

Level 5

5.1 The CODEOWNERS file is missing from the release branch; a pull request uses the version from its base branch, and each file applies to one branch.

5.2 That repository keeps using its own templates: if a repository defines valid issue templates in its own .github/ISSUE_TEMPLATE, none of the default folder's contents are used.

Chapter 50 — Branch Protection and Rulesets

Level 1

1.1 `git -C hub.git config receive.denyNonFastForwards true` and `git -C hub.git config receive.denyDeletes true`.

1.2 The server prints `remote: error: denying non-fast-forward refs/heads/main` (you should pull first) and the push shows `[remote rejected] main -> main (non-fast-forward)`. The `remote: line` was written by the server side.

Level 2

2.1 The deletion: denying ref deletion for `refs/heads/main` and (deletion prohibited).

2.2 With `receive.denyDeletes false` the deletion of `scratch` succeeds; the non-fast-forward rule is separate and still applies.

Level 3

3.1 Block force pushes: `receive.denyNonFastForwards`; restrict deletions: `receive.denyDeletes`; require a pull request and require passing checks: platform features (branch protection or rulesets), not Git server settings.

3.2 Rules are aggregated and the most restrictive version applies: signed commits and three reviews.

Level 4

4.1 For example: require a pull request with one review; require passing checks; block force pushes and deletions; require conversation resolution; only repository administrators may bypass, and each bypass is noted in the pull request.

Level 5

5.1 By default the restrictions of a branch protection rule do not apply to people with admin permissions, unless the rule is set to apply to administrators too.

5.2 Evaluate (where offered): it monitors which actions would violate the rules without enforcing them.

Chapter 51 — Insights

Level 1

1.1 Pulse: recent activity summary; Contributors: top 100 contributors; Traffic: views, visitors, clones, referrers; Commits: commits over the past year; Code frequency: additions and deletions per week; Network: branch history including forks.

1.2 The recording in section 51.3 shows the form of the output.

Level 2

2.1 The merge commit is counted by the first command and not by the second.

2.2 `git shortlog` counts it; `git log --shortstat` shows no changed-lines line for it.

Level 3

3.1 Two lines appear until `.mailmap` contains `Ada Learner <ada@example.org> Ada L. <ada@old-mail.example>`; then they merge into one.

3.2 (a) Traffic (full clones); (b) Contributors or Commits; (c) Network; (d) Pulse (choose the period).

Level 4

4.1 The graph counts commits by identity, ignores merge and empty commits, shows only the top 100, and cannot see review, design, documentation or support. Offer instead: review counts, time to first response, or a written summary of each person's work agreed with them.

Level 5

5.1 The commits may not be merged into the default branch; the commit email may not match an account (Chapter 37); or the person is not among the top 100, or is counted under another identity.

5.2 All data in the traffic graph uses UTC+0, regardless of location, so days are cut at UTC midnight.

Chapter 52 — CI/CD Concepts

Level 1

1.1 CI: committing often to a shared repository and having every change built and tested automatically. CD: using automation to publish and deploy software updates, usually after building and testing.

1.2 For example: `if grep -q "$(printf '\t')" menu.md; then echo FAIL; exit 1; fi; echo PASS.` On a clean file the exit code is 0.

Level 2

2.1 A line such as `- Coffee: two pounds` is printed and the script stops with exit code 1.

2.2 The `exit 1` in the failing stage ends the script, so later stages do not run.

Level 3

3.1 Cheap, fast checks (such as a line-length lint) go first, so mistakes are reported quickly.

3.2 `false && echo yes` prints nothing (the first command failed, so the second is skipped); `true && echo yes` prints yes. A shell continues a chain only after success, which is the same idea as stopping a pipeline at the first failed stage.

Level 4

4.1 For example: lint (HTML validator, no errors), test (links resolve, form has required fields), build (produce the site folder), deploy (publish the folder), each failing on a non-zero exit code.

Level 5

5.1 Make the checks faster (run cheap checks first, cut what is not needed) and keep them trustworthy; a fast, reliable pipeline is waited for.

5.2 A flaky check. It teaches people to ignore red, so real failures are missed.

Chapter 53 — YAML for Beginners**Level 1**

1.1 `name: Sunrise Bakery, open: true, loaf: 2.80` parse to `{'name': 'Sunrise Bakery', 'open': True, 'loaf': 2.8}`.

1.2 `items:` followed by three lines each starting with two spaces, a dash and a space parse to a map with a list.

Level 2

2.1 A map nested with two-space indentation; a list inside a map inside a list needs each level indented more than its parent.

2.2 `3.10` is a number, so it becomes `3.1`; quoted, it stays the text `3.10`.

Level 3

3.1 PyYAML gives `False` for unquoted `no` and `'no'` for the quoted form. Write `"no"` in quotes to be safe.

3.2 (a) `ScannerError`; (b) `ParserError`; (c) a missing space after the colon makes the whole line a plain string, not a key: value pair, so the result is a text value or an error depending on the rest of the file.

Level 4

4.1 Tabs gone; siblings aligned and children indented more than parents; space after every colon and dash; ambiguous values quoted; colons inside values quoted.

Level 5

5.1 `3.10` matches the number pattern and becomes `3.1`. Write `python: "3.10"`.

5.2 Check which YAML version the linter follows, and how GitHub's documentation writes the key (`on:` as an ordinary key). Do not change a working workflow because of a linter that follows different rules.

Chapter 54 — GitHub Actions Fundamentals**Level 1**

1.1 Workflow: an automated process defined in a YAML file. Event: something that starts it. Job: steps that run on one runner. Step: a command or an action. Runner: the machine that runs a job. Action: a reusable piece of code used by a step.

1.2 The file must be in `.github/workflows/`.

Level 2

2.1 For example: `d = yaml.safe_load(open(path)); on = d.get("on", d.get(True))` then loop over `d["jobs"].items()`.

2.2 The second process prints nothing: environment variables do not carry over between processes, so they do not carry over between steps.

Level 3

3.1 In parallel: by default jobs have no dependencies and run in parallel.

3.2 Adding needs: check to the deploy job makes it wait until check has finished (and, by default, succeeded) before starting.

Level 4

4.1 Answers vary. Look for actions with unpinned or floating versions and unfamiliar publishers.

Level 5

5.1 The checkout step: a runner starts clean, so the repository is not there until an action such as `actions/checkout` puts it there.

5.2 Steps are separate processes. Carry a value in a file in the workspace, or with the mechanism for step outputs and environment files (Chapter 56).

5.3 Workflow files must be in `.github/workflows` (with a leading dot) in the repository; `workflows/check.yml` at the root is not discovered.

Chapter 55 — Expressions, Contexts and Conditionals

Level 1

1.1 `git symbolic-ref HEAD` gives `refs/heads/feature-tea (github.ref)`; `git rev-parse --abbrev-ref HEAD` gives `feature-tea (github.ref_name)`; `git rev-parse HEAD` gives the commit SHA (`github.sha`).

1.2 `refs/tags/v1.0`.

Level 2

2.1 The line `refs/tags/v1.0`, for a tag.

2.2 It runs only when the run came from the branch `main`. With `'main'`, the condition would never be true because `github.ref` is the fully-formed ref `refs/heads/main`.

Level 3

3.1 (a) `if: ${{ github.event_name == 'push' }}`; (b) `if: ${{ !cancelled() }}`; (c) `if: ${{ failure() }}`.

3.2 `'Main' == 'main'` is true (string case is ignored); `1 == '1'` is true (the string is parsed as the number 1); `null == 0` is true (null is coerced to 0).

Level 4

4.1 A step output is a string, so convert it: `fromJson(steps.x.outputs.count) > 9`.

Level 5

5.1 Dereferencing a nonexistent property evaluates to an empty string; it does not fail. The real property is `github.actor`.

5.2 If the checkout fails critically, an `always()` step can keep going and the workflow may hang until it times out; use `!cancelled()` instead where suitable.

Chapter 56 — Practical Workflows**Level 1**

1.1 Six: the loop over `version` is outer and the loop over `os` inner, giving the order in the documentation.

1.2 At minutes 2 and 10 of hours 4 and 5, every day (UTC).

Level 2

2.1 `*/20` gives 0, 20, 40; `20/15` gives 20, 35, 50.

2.2 3 times 2 times 2 is 12 jobs. Each combination uses a runner and time, so matrices grow quickly.

Level 3

3.1 on: `pull_request` and two jobs with no needs; jobs run in parallel by default.

3.2 The first step runs `echo 'PRICE=2.80' >> $GITHUB_ENV`. Steps are separate processes, so environment variables are not preserved, but the file that `$GITHUB_ENV` names is shared and GitHub reads it for later steps.

Level 4

4.1 For example `17 6 * * 1` (Monday at 06:17 UTC). It needs to read pull requests and create an issue; give the token only those permissions (Chapter 60).

Level 5

5.1 Except for `GITHUB_TOKEN`, secrets are not passed to the runner when a workflow is triggered from a forked repository.

5.2 In a public repository, scheduled workflows are automatically disabled when there has been no repository activity for 60 days; a user with write permission who changes the cron schedule in a commit reactivates it.

5.3 Cache poisoning: caches are shared by branch or tag and restored as-is, so treat restored files as untrusted; GitHub gives workflows that run for low-trust triggers read-only access to caches in the default branch's scope.

Chapter 57 — Debugging and Re-running Workflows

Level 1

1.1 Did it start at all? Did the file load? Which job or step failed, and what did it print? Does it fail only there?

1.2 `trace: built-in: git rev-parse --git-dir` (after removing the time stamp and source location).

Level 2

2.1 `git commit -am "Change [skip ci]"` and `git commit -am "Change" -m "skip-checks: true"; git log -1 --format=%B | git interpret-trailers --parse` prints `skip-checks: true`.

2.2 The cause is `FAIL test: a line has no price`; the last line only says the process exited with code 1.

Level 3

3.1 `issues` runs only from the default branch, so a workflow file that exists only on another branch will not trigger; merge it into the default branch.

3.2 No. A re-run tests the same commit and the same ref as the original run, so it would not test your fix. Look at the new run started by your push.

Level 4

4.1 Note the failures, read the logs of failing and passing runs, look for timing, network or ordering causes, fix or isolate the cause, and do not simply re-run until green.

Level 5

5.1 Workflows do not run on `pull_request` activity when the pull request has a merge conflict; resolve the conflict.

5.2 Use `!cancelled()`, the inverse of `cancelled()`, which does not return true on cancellation.

5.3 A skipped workflow leaves its checks pending, and a pull request that requires them cannot merge. Push a commit without the skip string so the checks run, or change the rules that require them (with the right permission).

Chapter 58 — Reusable Workflows, Environments and Deployments

Level 1

1.1 As in the recording: `find dist -type l | wc -l` prints 0; `tar -tf artifact.tar` lists `./`, `./index.html` and `./style.css`; `gzip -t` succeeds silently.

1.2 `on: then workflow_call: then inputs: then config-path: with required: true and type: string.`

Level 2

2.1 The documentation says the tar file "should not contain any symbolic or hard links", so the artifact would break the rule.

2.2 uses: `./.github/workflows/build.yml` in one job; uses: `example-org/shared/.github/workflows/deploy.yml@<full commit SHA>` in another.

Level 3

3.1 For example: staging with no reviewers, deploying from any branch; production with one required reviewer, prevent self-review, and only main allowed.

3.2 (a) Reusable workflow (different machines); (b) composite action (steps); (c) reusable workflow (central maintenance of a whole pipeline).

Level 4

4.1 Answers vary.

Level 5

5.1 needs: `<build job>`.

5.2 The repository that hosts the caller workflow: "the action checks out the contents of the repository that hosts the caller workflow, not the called workflow".

5.3 Environment secrets cannot be passed from the caller, because `workflow_call` does not support the `environment` keyword; use the environment inside the called job or pass an ordinary secret.

Chapter 59 — Self-Hosted Runners, Concurrency and Limits**Level 1**

1.1 Hosted: GitHub provides and maintains it, and it starts clean. Self-hosted: you deploy, update and secure it, and it need not be clean for each job.

1.2 `echo $((3*3*2))` is 18, well within 256.

Level 2

2.1 18 times n exceeds 256 when n is 15 or more (18 times 14 is 252; 18 times 15 is 270).

2.2 concurrency: then group: `${{ github.workflow }}-${{ github.ref }}` and `cancel-in-progress: true`.

Level 3

3.1 Cancelling a deployment part-way can leave production half-updated. Prefer letting the running deployment finish and queueing or dropping the newer one, with `cancel-in-progress: false`.

3.2 Any four of: untrusted code could read the keys; the machine is not clean between jobs; a compromised job can persist; secrets passed on command lines can be seen by other jobs; the runner may serve several repositories.

Level 4

4.1 The documentation says self-hosted runners "should almost never be used for public repositories" because any user can open a pull request and compromise the environment. Use GitHub-hosted runners, and reduce minutes by caching and by cancelling outdated runs.

Level 5

5.1 A newer push in the same concurrency group with `cancel-in-progress`; or a documented limit (for example the 6-hour job limit).

5.2 The 500 KB workflow file limit; move shared logic into a reusable workflow or a composite action.

Chapter 60 — Securing Workflows**Level 1**

1.1 HACKED printed before the "checking" line: the pasted title closed the quotation and added a command.

1.2 The script printed the whole title as text and ran nothing, because the value travelled in an environment variable instead of being pasted into the script text.

Level 2

2.1 Yes: quotes, semicolons and `$` are allowed by Git's rules for names. A branch name, like a commit message, is text chosen by someone else and must not be pasted into a script.

2.2 Untrusted: `github.event.issue.title`, `github.head_ref`, `github.event.pull_request.body`. `github.sha` is a hash made by Git, so it has a fixed safe shape.

Level 3

3.1 Use `env:` with `TITLE: ${github.event.pull_request.title}` and run: `echo "Hello $TITLE"`.

3.2 A tag can be moved to different code later, while a commit name identifies its content and cannot be moved without a hash collision. Before copying a SHA, check that it comes from the action's own repository and not from a fork. Also read the action's code, or trust its creator.

Level 4

4.1 "This trigger runs with secrets and possibly write access, and checking out the pull request runs a stranger's code there. GitHub's documentation warns against this combination. Use the `pull_request` trigger without secrets, or split the work so untrusted code never runs in the privileged workflow."

Level 5

5.1 Delete the log, rotate the secret, then find how it reached the log (for example a derived value that was not registered as a secret).

5.2 Add permissions: `contents: read` at the top level of the workflow file.

Chapter 61 — Actions vs External CI

Level 1

1.1 Checks (detailed; created by GitHub Apps including Actions) and commit statuses (simple; created by external services and integrations).

1.2 Checks.

Level 2

2.1 Yes: GitHub's documentation says required status checks can be checks or commit statuses, so a result from any system that reports it through the API can be required.

2.2 The checks can be run on your own computer and the CI system can be replaced by rewriting a short configuration file.

Level 3

3.1 Answers depend on your system. A good answer states where the code runs, the cost model, the operating systems available and how secrets are scoped, each with a link to current documentation.

Level 4

4.1 For example: Which operating systems do we need? What will it cost for our usage? Where will secrets live? Which of our pipeline steps are scripts and which are service-specific? How do we report results to pull requests and which checks are required?

Level 5

5.1 Check that the external system reports a status with exactly the required name, and that it reports for the commit of the pull request's head. Also check that the integration has the permission to set statuses.

Chapter 62 — GitHub Security Features

Level 1

1.1 Dependency graph: lists what you depend on. Alerts: warn of known vulnerabilities in them. Security updates: open a pull request with the fix. Version updates: keep dependencies current. Secret scanning: finds committed secrets. Push protection: blocks a push that contains a secret. Code scanning: finds weaknesses in your own code.

1.2 Read the alert; rotate the credential; then, only if needed, remove it from history.

Level 2

2.1 `mkdir -p .github`, then write the five-line example from the chapter and `cat .github/dependabot.yml`.

2.2 For example: "Report vulnerabilities privately to `security@example.org`. Do not open a public issue. Include the version and steps to reproduce. We will reply within a reasonable time." The address is an example.

Level 3

3.1 Security updates react to a vulnerability alert and open a pull request that moves to the minimum fixed version. Version updates work on a schedule and keep dependencies up to date whether or not there is a vulnerability. Both open pull requests that you review.

3.2 Version updates, which can update action references in workflow files. Pinning fixes a version; updating moves it forward in a reviewed pull request.

Level 4

4.1 A sensible answer: SECURITY.md, Dependabot alerts, secret scanning, push protection, version updates for dependencies and actions; postpone code scanning if it is not available for your repository type or if you have no time to triage alerts. Justify each in a sentence. Check current availability first.

Level 5

5.1 Remove the secret from the commit and push again, and rotate the credential if it was ever exposed. Never bypass the protection or push the secret anyway.

5.2 Read the failure, the changelog in the pull request and the release notes. Fix the code, or postpone the update with a note; do not merge a failing pull request.

Chapter 63 — Repository Security Practice**Level 1**

1.1 Repository (a secret one project's workflows use), environment (a production credential gated by required reviewers) and organization (a shared secret limited by a policy to chosen repositories).

1.2 Verified, Unverified, and no status for an unsigned commit.

Level 2

2.1 For example `printf 'a\n' > f, sha256sum f > f.sha256, sha256sum -c f.sha256` prints `f: OK`; after `printf 'b\n' >> f` the check prints `f: FAILED`.

2.2 It has fine-grained permissions and short-lived tokens, and it is not tied to a person who may leave.

Level 3

3.1 An attacker who can replace the file can replace the checksum on the same page. An attestation is a signed statement linking the artifact to its source and build, which a consumer can verify against a policy.

3.2 A signature shows which key made the commit, not that the change is correct or safe; a compromised or careless key holder still produces Verified commits.

Level 4

4.1 An environment secret, with required reviewers for the production environment, a credential with the fewest permissions the deployment needs, and where possible a short-lived credential instead (Chapter 60).

Level 5

5.1 A corrupted download, a changed or replaced file, or a wrong checksum file. First download again, then compare from a second source before running the file.

5.2 Whether the key used is registered with your account, and whether the commit author's email matches an identity for that key.

Chapter 64 — How Open Source Works**Level 1**

1.1 Public: anyone can see the code. Source-available: the code can be read but the licence may forbid some uses. Open source: the code is public and the licence grants permissions to use, change and share it.

1.2 Maintainer and contributor are roles; write access and admin are permissions.

Level 2

2.1 Signed-off-by: Ada Learner <ada@example.org>, with the name and email from your configuration.

2.2 Typical answers: the README, CONTRIBUTING.md, the code of conduct, the community profile, the issues list.

Level 3

3.1 Read the rules; look at the issues; discuss a large idea; fork, clone and branch (`git switch -c`); make a small change and test; open a pull request; respond to review with further commits; accept the decision.

3.2 For example fixing an unclear step in the README: it is small, easy to review and helps every later reader.

Level 4

4.1 Thank the maintainer, ask nothing further about the decision, and move on. You may keep your fork for your own use if the licence allows it, or look for another project.

Level 5

5.1 Check that the project's guide sets no waiting rule and that checks passed; wait a reasonable time; add one polite comment asking whether anything is needed from you; do not repeat it daily.

5.2 `git commit --amend -s --no-edit`, and check that you have the right to submit the work under the project's rules. If the branch was already pushed, you will have to push the amended commit, which changes history (Chapter 27).

Chapter 65 — Copyright and Licences

Level 1

1.1 Copyright is the legal right of a creator over a work; a licence is the written permission that says what others may do with it, and under which conditions.

1.2 GitHub's documentation says the default copyright laws apply, so the owner retains all rights and no one may reproduce, distribute or create derivative works from the work.

Level 2

2.1 GPL-3.0-only: strong copyleft, including larger works that use it. MIT and Apache-2.0 are permissive and ask for notices (Apache-2.0 also asks to document changes).

2.2 Answers depend on the project. A good answer quotes the licence file for each "stated" and says "not stated" where the licence is silent.

Level 3

3.1 Attribution (credit the creator), NonCommercial (no use primarily intended for commercial advantage or monetary compensation), ShareAlike (adaptations under the same or a compatible licence), version 4.0.

3.2 SPDX marks GPL-3.0 as deprecated; the current identifiers are GPL-3.0-only and GPL-3.0-or-later.

Level 4

4.1 For example: "Without a licence, the default copyright law applies, which means we have no permission to copy it. I suggest we ask the owner for a licence or written permission, or write our own version, and ask the company's legal contact if unsure. Being public is not permission."

Level 5

5.1 Restore the notice, and check the rest of the licence's conditions. If you already distributed the code, ask the responsible person in your organization.

5.2 Read both licence files and the compatibility statements of their projects, then ask a qualified person. Compatibility is a legal question.

Chapter 66 — Releases

Level 1

1.1 A tag is a name for a commit and exists in Git, so a clone gets it. A release is a GitHub page built on a tag, with notes and attached files; it does not exist in a plain clone. Also: a tag date may differ from the release date.

1.2 (a) 1.4.3, (b) 1.5.0, (c) 2.0.0, following Semantic Versioning.

Level 2

2.1 The second output, `git tag --sort=-version:refname`, puts the newest first (v1.10.0, then v1.2.0). Plain `git tag` sorts alphabetically, so in an oldest-first reading it wrongly lists v1.10.0 before v1.2.0.

2.2 `git archive --format=tar.gz --prefix=proj-1.0.0/ -o ../proj-1.0.0.tar.gz v1.0.0`, then `tar -tzf ../proj-1.0.0.tar.gz` and `gzip -t ../proj-1.0.0.tar.gz`.

Level 3

3.1 `git log --format='- %s' v1.0.0..v1.1.0`, then tidy the lines into groups.

3.2 For example: `changelog:`, then `categories:`, an item with `title: Bug fixes` and `labels: [bug]`, and a second item with `title: Other changes` and `labels: ["*"]`; read back with a YAML parser or `cat`.

Level 4

4.1 Do not move the tag of a published release: people may already have used it. Publish a new patch version that contains the fix, and say in its notes what it corrects. With immutable releases enabled, the documentation says you cannot move or delete the tag while the release exists. Preparing releases as drafts first helps to catch such a mistake before publication.

Level 5

5.1 Releases are a GitHub feature, not part of Git; a clone contains tags but not the release page, notes or attached files. They are on the repository's Releases page (or through the Releases API).

5.2 No. `git tag` sorts alphabetically; use `--sort=version:refname` to sort by version number.

Chapter 67 — Creating an Open-Source Project

Level 1

1.1 What is it? README. May I use it? Licence. How do I try it? README (install and usage). How do I help? CONTRIBUTING.md and the code of conduct. What about a security problem? SECURITY.md.

1.2 LICENSE and CODE_OF_CONDUCT.md. The licence is the author's decision, made from an official text; a code of conduct is a promise the maintainer must be able to keep.

Level 2

2.1 After the steps of the recording, `git tag -n1` shows v0.1.0 with its message and `git ls-files` lists .gitignore, CONTRIBUTING.md, README.md, SECURITY.md, src/wordcount.py and tests/test_wordcount.py.

2.2 `count_words(" one two ")` returns 2, because `split()` ignores leading and trailing white space, so the test passes. A test such as `self.assertEqual(count_words(" one two "), 2)` is correct.

Level 3

3.1 Answers depend on the idea. A good answer shows the new function and tests, a README example that was run, and the same community files.

3.2 `for f in .gitignore README.md tests; do test -e "$f" && echo "present: $f" || echo "MISSING: $f"; done`

Level 4

4.1 Typical improvements: a one-sentence purpose at the top of the README, a working usage example, a visible licence, and a link to CONTRIBUTING.md.

Level 5

5.1 Add `__pycache__` to `.gitignore`. If they are already committed, remove them from the index with `git rm -r --cached` and commit (Chapter 18 on removing tracked files without deleting them).

5.2 Tell them that without a licence the default copyright applies and you cannot give permission by chat, and that you will decide on a licence. Then choose one from an official text (Chapter 65), add it, and reply with a link.

Chapter 68 — Organizations and Enterprise Concepts**Level 1**

1.1 User accounts, organization accounts and enterprise accounts. Only a user account can be signed in to; you cannot sign in to an organization.

1.2 Read, Triage, Write, Maintain, Admin.

Level 2

2.1 (a) Read, (b) Write, (c) Maintain.

2.2 No change for an outside collaborator: base permissions do not apply to outside collaborators. No downgrade for the member either: a higher level granted on a repository overrides the base permission.

Level 3

3.1 For example: a website team with Write on the website repository, a mobile team with Write on the mobile repository, a documentation team with Write on the documentation repository, and Read for everyone on the documentation repository, possibly through the base permission or through a team that includes all members.

3.2 A single owner is a single point of failure: if that person is away or leaves, nobody can manage the organization's settings. The documentation recommends assigning multiple owners so that ownership continues.

Level 4

4.1 Benefits: shared ownership that does not depend on one person, roles and teams, policies, an audit log, several owners. Decisions first: the name, at least two owners, the base permission, the teams, the repository-creation policy, two-factor authentication and security defaults.

Level 5

5.1 The warning that anyone who has a deploy key's private key can read or write the repository even after being removed from the organization. The key should have been deleted or replaced when the person left.

5.2 Plans, prices and quantities change. Read the current plans page for the account type; do not quote a book.

Chapter 69 — GitHub Pages**Level 1**

1.1 (a) `https://ada.github.io` (the repository is named `ada.github.io`); (b) `https://ada.github.io/notes`.

1.2 Deploy from a branch (with an optional folder), or a custom GitHub Actions workflow.

Level 2

2.1 `git grep -n 'http://'` finds the image line; `git grep -n -E '(href|src)="/'` finds the stylesheet line.

2.2 Nothing is printed, and `git grep` exits with a failure status. Add `|| echo "none left"` to print a message.

Level 3

3.1 For example: `if git grep -q 'http://' -- '*.html'; then echo "mixed content found"; exit 1; fi`.

3.2 `www.example.com`: CNAME. `example.com`: A, ALIAS or ANAME. The actual values are on GitHub's current page for managing a custom domain, not in this book.

Level 4

4.1 For example: "GitHub's documentation says Pages is not intended for or allowed to be used as a free web-hosting service for an online business or an e-commerce site, so a shop is not a good fit. You could use Pages for a catalogue and link to a proper shop service. Check the current rules page, because they can change."

Level 5

5.1 A root-absolute path such as `/style.css` on a project site (confirm by looking at the requested address in the browser's developer tools or by searching with `git grep`), or a stylesheet loaded over `http://` from an HTTPS site (mixed content; confirm by searching for `http://`).

5.2 GitHub checks the DNS settings, then requests a certificate from Let's Encrypt, which may take some time. If it does not finish several minutes after saving, remove the custom domain, type it again and save, which restarts the process.

Chapter 70 — Codespaces and Dev Containers

Level 1

1.1 A codespace is a development environment hosted in the cloud, in a container on a virtual machine. Configuration as code means the environment is described by files committed to the repository, so everyone who creates a codespace gets the same setup.

1.2 In `.devcontainer/devcontainer.json` (or `.devcontainer.json` in the root). A second configuration goes in its own subfolder: `.devcontainer/SUBDIRECTORY/devcontainer.json`.

Level 2

2.1 For example `git add .devcontainer && git commit -m "Add dev container"`, then `git ls-files` lists `.devcontainer/devcontainer.json`.

2.2 `python3 -m json.tool` reports an error, because strict JSON does not allow comments. The chapter says the documentation's examples are `jsonc` (JSON with comments) and dev container tools accept them, so remove the comments to check with a strict tool or use a `jsonc`-aware tool.

Level 3

3.1 In the file: the linter and the language version (everyone needs them). In your own settings, dotfiles or Settings Sync: the colour theme and the shell aliases.

3.2 For example: "Stop codespaces you are not using, delete the ones you no longer need, push your work before deleting, and remember that a codespace times out after 30 minutes of inactivity by default, but that it still exists until it is deleted. Check the billing page for the quota."

Level 4

4.1 A sensible proposal: a `.devcontainer/devcontainer.json` using an image (or a Dockerfile), a `postCreateCommand` that installs dependencies, and only shared tools. Test first by creating a codespace from a clean branch. Cautions: no secrets in the file, only trusted repositories, forwarded ports private, and a note that the configuration is reviewed like code.

Level 5

5.1 Commit and push your work to the remote repository (or publish it to a new one) before deleting; work in a deleted codespace is deleted too.

5.2 Anyone on the internet can reach a public forwarded port without authentication. It reverts to private when the port is removed and re-added, or when the codespace restarts; organization owners can also restrict public ports.

Chapter 71 — GitHub Packages and Containers

Level 1

1.1 A package is software packaged for installation by a tool; a registry is the place where packages are stored and downloaded from; a package manager is the tool that installs them.

1.2 Only @ada/wordcount: GitHub's npm registry supports only scoped names, and names and scopes must be lowercase.

Level 2

2.1 package.json with "name": "@ada/wordcount" and .npmrc with the line @ada:registry=https://npm.pkg.github.com, then git add package.json .npmrc and git commit.

2.2 For example printf '//npm.pkg.github.com/:_authToken=not-a-real-token\n' > ~/.npmrc, and then git grep -n '_authToken' || echo "no token in the repository" prints the message.

Level 3

3.1 For example: if git grep -q '_authToken'; then echo "token found"; exit 1; fi.

3.2 Granular: the package belongs to an account or organization and has its own access and visibility, so you can publish without a repository. Repository-scoped: the package inherits the repository's permissions and visibility, so there is one place to manage access.

Level 4

4.1 A sensible design: trigger on a tag push (or a published release) on the default branch; authenticate with the workflow's short-lived GITHUB_TOKEN with only the permission needed to write packages; require review and protect the branch so only reviewed changes alter the workflow; pin any third-party action to a commit; risks include publishing from the wrong tag, a compromised action, and publishing secrets inside the package.

Level 5

5.1 A package inherits a repository's permissions automatically only if the repository is linked before publishing; linking later from the package settings leaves the existing permissions unchanged.

5.2 Revoke the token first; create a new one with the fewest scopes; remove it from the file and from history if you wish (Chapter 33), move the token to a user-level file or a workflow secret; check where else it was used.

Chapter 72 — Deployment Pipelines

Level 1

1.1 Trigger; build and test; package; approve; deploy; check; keep the previous version.

1.2 Static website: copy files to a host. Container: build an image, push it to a registry and run it. VPS: copy files or a container and restart services on the server you administer.

Level 2

2.1 `mkdir -p server/releases/v1 server/releases/v2`, write a file in each, `ln -sf releases/v1 server/current`, `cat server/current/index.txt`, then `ln -sf releases/v2 server/current` and `cat` again.

2.2 `ln -sf releases/v1 server/current`, then `ls -l server/releases` shows v1 and v2.

Level 3

3.1 `grep -q Bakery server/current/index.txt && echo "health check: ok"`.

3.2 Staging deploys automatically after tests pass; production needs an approval from a required reviewer, and may be limited to the default branch. The same artifact goes to both, so what was tested on staging is exactly what is released.

Level 4

4.1 For example: "This key can do everything, so a leak is a disaster; use the fewest permissions the deployment needs and, where possible, a short-lived credential. Store it as an environment secret so an approval gates its use. Never print it: log redaction is not guaranteed, and if it appears in a log we must delete the log and rotate the key."

Level 5

5.1 The design did not keep the previous version. Keep at least the previous release folder (and practise the rollback), and make clean-up scripts keep the release that is live and the one before it.

5.2 Files can be switched back, but a database that has already been changed (columns removed, data converted) may not accept the old version's code. Plan database changes to work with both versions, or make them reversible.

Chapter 73 — GitHub CLI

Level 1

1.1 `git` works on a repository and its history (examples: `commit`, `merge`, `push`). `gh` works on things GitHub adds (examples: pull requests, issues, releases, workflow runs).

1.2 `gh --version`. Commands and output change between versions, and an old `gh` may lack commands.

Level 2

2.1 `gh alias set prd "pr create --draft", gh alias list | grep prd, gh alias delete prd.`

2.2 `gh config set editor "code -w"` and `gh config get editor`. The `-w` flag makes the editor command wait until you close the file, so `gh` can read what you wrote.

Level 3

3.1 For example: `if gh auth status >/dev/null 2>&1; then echo ready; else echo "please log in"; fi`.

3.2 `GH_TELEMETRY=log gh config get editor 2>&1 | grep '"type"'` shows `command_invocation`. Disable with `export GH_TELEMETRY=false` (or `DO_NOT_TRACK=true`), or with `gh config set telemetry disabled`. The environment variable takes precedence over the configuration value.

Level 4

4.1 For example: a step with `env: GH_TOKEN: ${ secrets.GITHUB_TOKEN }` and `run: gh run list`. Restrict the workflow's token to the read permission that the command needs, because a compromised step can use whatever the token allows (Chapter 60).

Level 5

5.1 Run `gh auth login` (interactive, for a person), or set the `GH_TOKEN` environment variable. In a workflow prefer `GH_TOKEN` set from the built-in `GITHUB_TOKEN`, with least privilege.

5.2 The command is newer than the installed version, or it is provided by an extension that is not installed. Check `gh --version` and `gh --help`.

Chapter 74 — APIs, Webhooks and Apps

Level 1

1.1 REST exposes resources at addresses and returns a fixed structure for each request; GraphQL takes a query that names exactly the fields you want, at a single address, so one query can replace several REST requests.

1.2 The response is paginated and there is a next page at that address; the script should request it, and continue until no next relation remains.

Level 2

2.1 `secret="It's a Secret to Everybody"; printf '%s' 'Hello, World!' | openssl dgst -sha256 -hmac "$secret"` prints a value ending with `757107ea0eb2509fc211221cce984b8a37570b6d7586c22c46f4379c8b043e17`.

2.2 They look completely unrelated: a hash changes entirely when the input changes by one character. That is why a matching signature shows that the payload was not altered.

Level 3

3.1 For example, in Python: `import re, sys; print(dict((r, u) for u, r in re.findall(r'<([>]+)>; rel="(\\w+) "', sys.stdin.read()))).get("next", "none")`.

3.2 A plain comparison can stop at the first difference, so the time it takes may leak how much of the guess was right. Use a constant-time comparison such as Python's `hmac.compare_digest`, Ruby's `Rack::Utils.secure_compare` or Node's `crypto.timingSafeEqual`.

Level 4

4.1 A personal access token is tied to a person and often broad; an OAuth app acts with a user's scopes; a GitHub App has fine-grained permissions, can be limited to chosen repositories, uses short-lived tokens and is not tied to a person. Recommend a GitHub App with only the permission to comment on pull requests, installed on the organization's repositories.

Level 5

5.1 The endpoint paginates (30 items is a default page). Request a larger `per_page` if the endpoint supports it and follow the `next` link until it is absent.

5.2 Check that you use the raw request body (not a re-serialised copy), that you handle it as UTF-8, that the secret in your server equals the one in the webhook's settings, that you compute HMAC-SHA-256 and prefix the hex digest with `sha256=`, and that you compare against the `X-Hub-Signature-256` header.

Chapter 75 — Other Platform Features**Level 1**

1.1 In the `.github` folder of the repository, on the default branch.

1.2 One username, package name or project name per external funding platform with up to four custom URLs; and one organization plus up to four sponsored developers on GitHub Sponsors.

Level 2

2.1 For example: `github: [ada-example], patreon: ada-example and custom: ["https://example.org/support"]`, written with `printf` into `.github/FUNDING.yml`, then `cat`.

2.2 With PyYAML: `python3 -c "import yaml,sys; d=yaml.safe_load(open('.github/FUNDING.yml')); print({k:(len(v) if isinstance(v,list) else 1) for k,v in d.items()})"`.

Level 3

3.1 GitHub's own parser and rules are what matter, and a different tool may reject what one tool accepts. A tolerant parser does not prove the file is valid for GitHub.

3.2 For example: `team` (a name), useful to write rulesets that require review by that team's owners; and `criticality` (low, medium, high), useful to write a stricter ruleset for high-criticality repositories. Properties are metadata that rulesets can target.

Level 4

4.1 For example: (1) A person is accountable for every change, whatever produced it. (2) Tell the reviewer when a tool produced a significant part. (3) Never put secrets or private data in prompts. (4) Do not submit what you cannot explain or have no right to submit. (5) The same tests and reviews apply to all changes.

Level 5

5.1 That the file is named `FUNDING.yml` and is in the `.github` folder on the default branch; that the sponsor button is enabled in the repository settings by someone with admin permissions; that the entries follow the documented syntax and limits.

5.2 Do not install it yet. Read what it asks for, who publishes it and whether the publisher is verified, and ask whether a narrower option exists (least privilege).

Chapter 76 — Professional Scenarios 1-12

Level 1

1.1 (a) amend; (b) revert; (c) squash with `git reset --soft main` and one new commit; (d) cherry-pick (with `-x`).

1.2 `git status`, `git log` (with a format such as `--format=%s`) and `git diff`.

Level 2

2.1 As in the recording: `git switch -c`, edit, `git commit -am`, `git push -u origin BRANCH`, then in a second clone `git merge --no-ff origin/BRANCH` and push; in yours `git fetch`, `git switch main`, `git merge --ff-only origin/main`, `git branch -d BRANCH`, `git push origin --delete BRANCH`.

2.2 The relog entry expires after some time (Chapter 26), and unreferenced commits may be removed by garbage collection, so the recovery would no longer be possible.

Level 3

3.1 Answers vary. A good scenario states who else has the commits, chooses a tool accordingly, and shows the output before and after.

3.2 After the resolution the history has a merge commit with two parents; after the abort the history and working tree are as before the merge attempt.

Level 4

4.1 Revoke or rotate the key first, because anyone who saw the repository may have it; then remove the file from tracking and add it to `.gitignore`; explain that the key is still in earlier commits and in clones and forks; rewrite history only if needed, with the team informed; check where else the key was used.

Level 5

5.1 Check with `git log main..branch` (no output means everything is in main) or `git branch --merged main`. `-D` is acceptable only when you have confirmed that you do not need the unmerged commits, or that they exist elsewhere.

5.2 The cherry-pick was made on another branch; it conflicted and was left unfinished; the wrong commit was picked; or the local `main` was not pushed or updated.

Chapter 77 — Capstone Project

Level 1

1.1 After the steps, `git status -sb` shows `## main...origin/main` with nothing ahead or behind.

1.2 The first line runs the tests and stops with status 1 on failure; the loop checks that the required files exist and stops with status 1 if one is missing; the last line only reports a missing LICENSE. A CI system treats a non-zero exit status as a failed check.

Level 2

2.1 Write the failing test and show FAILED; write the minimum code; show OK (or the pipeline's tests: ok).

2.2 For example: "Please name the constant." "Add a test for zero." "Explain the rounding in the README." Each answered by a commit with a clear message.

Level 3

3.1 Answers vary. Typical changes: file names, test commands for another language, the check list in `ci.sh`.

3.2 Answers vary. Typical differences: the layout of pages, wording of buttons, options offered by a repository's merge settings.

Level 4

4.1 The handover should follow the steps: test first, branch, pipeline, review, merge strategy, tag and release, revert rather than rewrite, secrets and what to do first, audit.

Level 5

5.1 The branch and `main` changed the same lines and the branch was not up to date. Bring `main` into the branch (or rebase), resolve the conflict by reading both sides, run the pipeline, push, then merge again.

5.2 Revoke or rotate the key first; then stop tracking the file and add it to `.gitignore`; remove it from history only if needed and after telling the team; check where else the key was used and for signs of misuse; add push protection so that it does not recur.

Chapter 78 — Troubleshooting Handbook**Level 1**

1.1 Read the first and last lines; run `git status`; do not run the first fix you find; look before you change anything; make a safety copy when in doubt; change one thing at a time.

1.2 (a) Git found no repository in this folder or its parents. (b) Git has no name and email to record. (c) The two histories share no commit. (d) Another Git process is running or left a lock file.

Level 2

2.1 Any three from the handbook; for each show the message, and the safe fix (for example `git config --global user.name ...` for the identity error).

2.2 After a merge fix the history has a merge commit; after a rebase fix it is linear. Both leave the remote with all commits; the rebase rewrites only your unpushed commits.

Level 3

3.1 Answers vary; a good entry has message, meaning, cause, diagnosis, safe fix, dangerous fix, prevention, and a recording.

3.2 checkout -f: destructive (throws away uncommitted changes). stash: safe. push --force-with-lease: risky but safer than --force; only on your own branch. branch -d: safe (refuses if unmerged). clean -fd: destructive (deletes untracked files and folders); use -n first to preview.

Level 4

4.1 "--force overwrites the remote branch with your version, so commits that only the remote had are no longer on the branch. Ana's commits still exist in her clone and possibly in the remote's reflog; she can push them back from her clone, or you can recover them from the reflog. In future, fetch and merge or rebase instead of forcing, and protect main so that force pushes are blocked."

Level 5

5.1 .gitignore does not affect files that are already tracked. Run `git rm --cached notes.txt` and commit; the file stays on disk.

5.2 Create a branch at your current position: `git switch -c keep-my-work`. The commits are then on that branch.

Chapter 79 — Recovery Playbooks

Level 1

1.1 Revoke the key; stop tracking the file; rewrite history (only if needed); tell colleagues to discard their clones and clone again.

1.2 Playbook 2. `HEAD@{1}` is where HEAD pointed before the most recent move, here the tip before the reset.

Level 2

2.1 After `git reset --hard HEAD~2` the log lacks the two commits; `git reflog` lists them; `git reset --hard 'HEAD@{1}'` restores them.

2.2 `git branch NEW`, then `git reset --hard origin/main`; the log of main lacks the commit and the log of NEW has it.

Level 3

3.1 The revert removes the branch's changes, but a second merge of the same branch brings none of them back, because Git regards the branch's earlier commits as merged. The way out is to revert the revert first (Git's howto "Revert a faulty merge").

3.2 After the abort the branch is as before; after `--continue` the branch's commit sits on top of main with a linear history.

Level 4

4.1 A good report has: timeline (pushed Monday, found Wednesday); revoke and rotate first, and check the service's logs for use in that period; stop tracking and ignore; decide whether to rewrite (large effort, incomplete: clones, forks, cached views, pull requests); if so, coordinate, force-push, ask everyone to re-clone, contact Support for cached views; prevention with push protection and secret scanning.

Level 5

5.1 Their old clone still has the old commits. `git pull` merged the old history with the new, and `git push` sent the old commits back. They should have discarded the clone and cloned again.

5.2 The reflog entry index was not the one you thought (use `git reflog` and pick the right entry); the commits were never committed (only edited); the reflog entries expired or were pruned; you are in another clone.

Chapter 80 — Advanced Challenges

Reference solutions were run and recorded (`verification/ch80-challenges`). Commit names differ on your computer.

Challenge 1

```
cd app
git -c protocol.file.allow=always submodule update --remote vendor/lib
git status --short      # shows " M vendor/lib": the pointer changed
git commit -am "Update lib to version 2"
```

`cat vendor/lib/lib.txt` prints version 2. The outer project stores the inner commit's name, so updating means committing the new pointer.

Challenge 2

```
git clone mono site-only
cd site-only
git filter-repo --subdirectory-filter site
git log --format=%s      # Change site again / Change site / Start
ls                       # index.html
git rev-list --count HEAD # 3
```

Work on a clone because the tool rewrites history and removes the origin remote. The commit "Change tools" touched no site file and disappears.

Challenge 3

```
git clone --depth 1 --no-checkout file://$PWD/big slim
cd slim
git sparse-checkout set docs
git checkout main
git rev-list --count HEAD # 1
ls                       # docs
```

The `file://` address makes Git use the real transport, which honours `--depth`; a plain path is copied directly.

Challenge 4 — rubric

Award one point per part that is present, consistent with the others and justified in a sentence; six or seven points is a good design.

1. **Branches and merges:** short-lived topic branches from main, pull requests, one merge strategy stated (Chapters 45 and 21).
2. **Automated checks:** tests and linters on every pull request, plus security checks; local script runnable by contributors (Chapters 52, 54, 77).
3. **Protection and review:** required checks and at least one review, no force pushes to main, code owners for sensitive folders (Chapters 46 and 50).
4. **Releases:** annotated tags, semantic versioning, notes, checksums (Chapters 29 and 66).
5. **Hotfix:** branch from the release tag, fix, new patch tag, carry the fix to main with cherry-pick -x (Chapter 76).
6. **Secrets and dependencies:** push protection, secret scanning, dependency alerts, least-privilege tokens, pinned actions (Chapters 60, 62, 63).
7. **Newcomers:** README, CONTRIBUTING.md, code of conduct, good first issue labels (Chapters 42 and 64).

Because the application handles customers' data, a strong answer stresses secrets handling and review.

Challenge 5

```
printf 'grep -qx ok state.txt\n' > ../test.sh
git bisect start HEAD HEAD~12
git bisect run sh ../test.sh
git bisect reset
```

Git reports "Change state" as the first bad commit. The test uses -x (whole-line match): with grep -q ok, the word broken also contains ok and the test would wrongly pass everywhere.

Challenge 6

```
git fsck --lost-found          # dangling commit <name>
git log -1 <name>              # Precious work
git branch rescued <name>
```

fsck lists objects that nothing reaches, without needing the reflog. Garbage collection would eventually remove them, so rescue soon.

Exercise 1.1

The update command fetches the newest inner commit and moves the inner working tree to it; git status shows the pointer as modified; the commit in the outer project records the new inner commit name. The outer project stores only the name of one commit of the inner repository (and its path), not the inner files.

Exercise 2.1

Add the second submodule the same way, run git submodule update --remote for both, and commit once with git commit -am. A colleague runs git submodule update --init after git pull so that the inner working trees match the recorded commits.

Exercise 3.1

`git filter-repo --path site --path tools` on a fresh clone keeps only those folders in place. The commits that touched only notes disappear, so the count is lower than in `mono`.

Exercise 5.1

Bisect assumes that a commit is either good or bad deterministically. A test that fails randomly can mark a good commit bad, so bisect converges on a wrong commit. Make the test deterministic, or run it several times and treat any failure as bad, and check the result by testing the neighbouring commits.

Exercise 4.1

For example: (1) branch rule: protection requires pull requests; (2) checks: the workflow file runs `ci.sh` on each pull request; (3) review: a rule requires one approval and code-owner review for sensitive folders; (4) releases: a check that the tag matches the version file; (5) hotfix: a documented branch name pattern and a rule that hotfixes are cherry-picked to `main`; (6) secrets: push protection enabled and a scheduled dependency update; (7) newcomers: a check that README, contributing guide and code of conduct exist (as in the capstone's pipeline).

Chapter 81 — Final Mastery Assessment

Do not read this before you have tried the assessment.

Rubric

| Level | Requirement |
|------------|--|
| Proficient | at least 9 of the 10 tasks P1 to P10 pass |
| Mastery | all ten Proficient tasks and at least 4 of the 5 tasks M1 to M5 pass |

Beyond the checker, a reviewer can ask you to explain: why P6 is a revert and not a reset; what the tag in P7 names; why M1 needs a forced push; why the test in M2 matches a whole line.

Answer key

Commit names in your repository differ from anything printed here.

| Task | One way to do it | Chapters to revisit |
|------|--|---------------------|
| P1 | <code>git config user.name "Your Name"</code> and <code>git config user.email "you@example.org"</code> (no <code>--global</code>) | 14 |
| P2 | <code>git switch -c feature/tea</code> , append the line, <code>git commit -am "Add tea"</code> | 20, 19 |
| P3 | <code>git push -u origin feature/tea</code> | 23 |
| P4 | write <code>*.log</code> in <code>.gitignore</code> , <code>git add .gitignore</code> , commit | 18 |
| P5 | <code>git fetch</code> , <code>git switch -c combined origin/edit-a</code> , <code>git merge origin/edit-b</code> , edit README.md, <code>git add</code> , <code>git commit</code> | 22 |
| P6 | <code>git revert COMMIT</code> (find it with <code>git log --grep</code>), then <code>git push origin main</code> | 25, 76 |

| Task | One way to do it | Chapters to revisit |
|------|---|---------------------|
| P7 | <code>git tag -a v1.1.0 -m "Version 1.1.0" on the final tip of main, then <code>git push origin v1.1.0</code></code> | 29, 66 |
| P8 | <code>git reflog</code> , find "Precious work", <code>git branch precious NAME</code> | 26, 79 |
| P9 | write both files, <code>git add</code> , <code>commit</code> , <code>push</code> | 64, 62 |
| P10 | a file with <code>on:</code> , <code>permissions:</code> , <code>jobs:</code> and <code>uses:</code> <code>actions/checkout@<40 hex characters></code> | 54, 60 |
| M1 | <code>push combined</code> , <code>precious</code> and the tag; then <code>git filter-repo --sensitive-data-removal --invert-paths --path secrets.env --force</code> and <code>git push --force --mirror origin</code> | 33, 79 |
| M2 | <code>git bisect start HEAD FIRST-COMMIT</code> , then <code>git bisect run sh test.sh with grep -qx ok state.txt</code> ; write the subject "Change state" into <code>bisect-answer.txt</code> ; <code>git bisect reset</code> | 28, 80 |
| M3 | <code>git clone --depth 1 --no-checkout file:///PATH/remote.git slim</code> , then <code>git sparse-checkout set docs</code> and <code>git checkout main</code> in <code>slim</code> | 31 |
| M4 | <code>git log --format='- %s' v1.0.0..v1.1.0 > RELEASE_NOTES.md</code> | 66 |
| M5 | <code>printf '%s' 'Order 42 shipped' openssl dgst -sha256 -hmac 'bakery-test-secret'</code> , keep only the hexadecimal digest | 74 |

Note on M3. The `sparse-checkout set` command works in "cone" mode by default, which always keeps the files at the top level of the repository; that is why the check tests for the absence of the `src` folder and not of top-level files.

Reflection answers.

1.1 Compare your prediction with the result; large gaps show topics that you overestimate.

2.1 One order: P1, P2, P3, P4, P5, P8, P6, P9, P10, then `push main`, then P7. P7 depends on all changes to `main`; M1 depends on everything being pushed.

3.1 Record the pages you needed; they are your revision list.

4.1 A good explanation states purpose, command and pitfall; for example P6: purpose "undo a public mistake without disturbing others", command `git revert`, pitfall "resetting and force-pushing rewrites what colleagues already have".

5.1 The tag is lightweight and not annotated; it points to a commit that is no longer the tip of `main`; it was not pushed; or the name differs (for example `v1.1` or `v1.1.0`). Use `git cat-file -t v1.1.0`, `git rev-parse v1.1.0^{commit}` `main` and `git ls-remote --tags origin`.

Glossary

Every term that the book defines, in alphabetical order. Each entry gives a simple definition, a technical one, an example, related terms and the chapter where the term is first explained.

▪

.gitignore. A text file listing patterns of files Git should ignore.

Technically: A file whose lines are patterns (with comments and negations) applied to paths under its directory; normally tracked. Example: *.log

Related: ignored file; core.excludesFile. First explained in Chapter [18](#).

A

Absolute path. A path that starts at the root and means the same wherever you are.

Technically: A path that begins at the file-system root (or a drive letter on Windows) and fully specifies a location. Example: /home/learner/sharif-git-lab/sunrise-bakery/menu.html

Related: relative path; root. First explained in Chapter [2](#).

Access token. A password-like secret for programs.

Technically: A secret string that a program presents instead of a password, which can be limited in permissions and lifetime and revoked separately. Example: a personal access token

Related: secret; authentication. First explained in Chapter [38](#).

Account. A record a service keeps to represent you.

Technically: A stored identity with credentials, settings and permissions maintained by a service. Example: Your email account.

Related: username; authentication. First explained in Chapter [6](#).

Action. A reusable piece of code that a step can use.

Technically: A predefined, reusable unit of code used in a workflow step to perform a task such as checking out a repository or setting up a toolchain. Example: actions/checkout@v6

Related: step; workflow. First explained in Chapter [54](#).

Administrator. An account or permission level allowed to change the computer's system settings.

Technically: A privileged user role that may modify system-wide configuration and install software for all users. Example: The prompt that asks whether to allow an installer to make changes.

Related: installer; permission. First explained in Chapter [1](#).

Alias. Your own short name for a Git command.

Technically: A name defined in configuration (alias.<name>) as a shortcut for another Git command, or for a shell command when it starts with !. Example: `git config --global alias.st "status --short"`

Related: configuration. First explained in Chapter [32](#).

Annotated tag. A tag with a message and tagger.

Technically: A tag stored as its own object with tagger, date and message; the usual choice for releases. Example: `git tag -a v1.0 -m "First public menu"`

Related: tag. First explained in Chapter [24](#).

Apex domain. A domain name with nothing in front of it, like example.com.

Technically: A domain without a subdomain, also called a bare or root domain. Example: example.com

Related: DNS; custom domain. First explained in Chapter [69](#).

API. A way for one program to ask another to do something.

Technically: An application programming interface: a defined set of requests and responses that programs use to communicate. Example: A script asking GitHub for a repository's issues.

Related: REST; GraphQL. First explained in Chapter [74](#).

Application (program). Software that does a specific job for a person.

Technically: A program designed to perform a task for a user, running on top of an operating system. Example: A text editor, a browser, Git.

Related: software; operating system; installer. First explained in Chapter [1](#).

Argument. The thing a command acts on, such as a file name.

Technically: A value passed to a command on the command line. Example: notes.txt in `ls -l notes.txt`

Related: command; option. First explained in Chapter [7](#).

Artifact. A file a workflow produces and keeps.

Technically: A file or collection of files produced during a workflow run and stored so that it persists after the job and can be shared with other jobs. Example: A built program or a test report.

Related: cache; workflow (GitHub Actions). First explained in Chapter [56](#).

Asset. A file attached to a release.

Technically: A file uploaded to a GitHub release, in addition to the automatic source archives. Example: A compiled program.

Related: release; artifact. First explained in Chapter [66](#).

Atomic commit. A commit containing one logical change.

Technically: A commit that is complete and self-contained, with a single purpose, that can be reviewed or reverted independently. Example: Raise the price of rolls

Related: commit. First explained in Chapter 19.

Attestation. A signed statement about where and how a file was built.

Technically: A signed record, for example made with Sigstore, linking an artifact to its source and build instructions. Example: A GitHub artifact attestation.

Related: artifact; signed commit. First explained in Chapter 63.

Authentication. Proving who you are.

Technically: The process of verifying the identity claimed by a user or program. Example: Entering a password and a code.

Related: authorization; account. First explained in Chapter 6.

Authorization. Deciding what you are allowed to do once your identity is known.

Technically: The process of determining whether an authenticated identity may perform a requested action. Example: You can read a project but not delete it.

Related: authentication; least privilege. First explained in Chapter 6.

B

Backup. A copy of your files kept elsewhere so they can be restored after loss.

Technically: A duplicate of data stored separately from the original to allow recovery after loss or damage. Example: A nightly copy of a laptop to an external drive.

Related: version control; synchronised folder. First explained in Chapter 9.

Bare repository. A repository with no working files, used as a shared hub.

Technically: A repository holding only Git's database, with no checked-out working tree; the form used for shared remotes. Example: `git init --bare shared/bakery.git`

Related: remote; clone. First explained in Chapter 23.

Base branch. The branch a pull request wants to change.

Technically: The branch into which the changes of a pull request are to be merged, usually the default branch. Example: `main`

Related: pull request; compare branch. First explained in Chapter 45.

Base permission. The lowest level of access every member of an organization has to its repositories.

Technically: An organization-wide default repository permission for members; it does not apply to outside collaborators. Example: `Read`.

Related: organization; repository role. First explained in Chapter 68.

Bash. A widely used shell; the one this book uses for its examples.

Technically: The Bourne-Again Shell, a Unix command interpreter. Example: The shell in Git Bash.

Related: shell; Git Bash; zsh. First explained in Chapter 7.

Binary file. A file that is not plain text, such as a photo or a word-processor document.

Technically: A file whose bytes are not a sequence of characters intended to be read as text. Example: logo.png, report.docx

Related: plain text; ZIP archive. First explained in Chapter 3.

Bisect. Find the commit that broke something by halving.

Technically: A binary search through history that repeatedly checks out the middle of the remaining range to find the first bad commit. Example: `git bisect run ./test.sh`

Related: commit. First explained in Chapter 28.

Bitbucket. Another hosting platform for Git repositories, from a different company.

Technically: A hosting platform for Git repositories with collaboration features. Example: A team's repository hosted outside GitHub.

Related: GitHub; hosting platform. First explained in Chapter 12.

Blob. The stored contents of one file.

Technically: An object holding the contents of a single file, with no name, folder or date. Example: 28adfbf...

Related: tree; object. First explained in Chapter 30.

Branch. A movable name that points to a commit.

Technically: A reference under refs/heads that holds the identifier of a commit and moves forward with new commits. Example: main

Related: HEAD; commit. First explained in Chapter 15.

Branch protection rule. A setting that protects chosen branches from risky changes.

Technically: A repository setting on a hosting platform that matches branch names by pattern and applies restrictions or requirements, such as required reviews and blocked force pushes. Example: A rule on main that requires one review.

Related: ruleset; pull request. First explained in Chapter 50.

Browser. An application for viewing web pages.

Technically: A client application that requests, interprets and displays web resources. Example: The program in which you open index.html.

Related: web; client. First explained in Chapter 5.

Byte. A unit for measuring stored information; enough for one English letter.

Technically: A unit of digital information made of eight bits. Example: The word 'Hi' takes two bytes in simple text.

Related: storage; file. First explained in Chapter 1.

C

cd. Moves you into another folder.

Technically: Change directory: sets the shell's current working directory. Example: `cd ..`

Related: `pwd`; path. First explained in Chapter 7.

Centralised version control. One server holds the only complete history; users hold working copies.

Technically: A version-control architecture with a single central repository that clients connect to. Example: A shared server everyone must reach to record work.

Related: distributed version control; single point of failure. First explained in Chapter 11.

Character. A single unit of text: a letter, digit, symbol, emoji or space.

Technically: An abstract unit of written text identified by a Unicode code point. Example: a, é, 7, the space

Related: encoding; Unicode. First explained in Chapter 4.

Check. A detailed result of an automated job, shown on a pull request.

Technically: A check run created by a GitHub App, including GitHub Actions, with output, annotations and messages. Example: The Checks tab of a pull request.

Related: commit status; workflow. First explained in Chapter 61.

Checksum. A short fingerprint of a file that changes if the file changes.

Technically: A value computed from a file's bytes with a hash function such as SHA-256, used to detect change. Example: A 64-character SHA-256 value next to a download.

Related: hash; artifact. First explained in Chapter 63.

Cherry-pick. Copy one commit onto another branch.

Technically: Apply the change introduced by an existing commit as a new commit on the current branch. Example: `git cherry-pick drinks~1`

Related: rebase; patch. First explained in Chapter 28.

Client. The computer or program that asks for something.

Technically: The party in a client-server exchange that initiates requests. Example: Your browser.

Related: server; HTTP. First explained in Chapter 5.

Clone. A complete copy of a repository, with all its history, made on another computer.

Technically: A full copy of a repository including history, produced by cloning. Example: Your copy of a project from a shared server.

Related: repository; remote. First explained in Chapter 11.

Code editor. A text editor with extra help for writing structured text such as software.

Technically: A text editor providing features such as syntax highlighting, line numbers, multi-file search and folder navigation. Example: An editor showing your project's files in a side panel.

Related: text editor; syntax highlighting. First explained in Chapter 3.

Code of conduct. A written set of rules for how people behave in a project.

Technically: A document that defines standards of engagement in a community and procedures for problems. Example: A CODE_OF_CONDUCT file.

Related: CONTRIBUTING; community profile. First explained in Chapter 64.

Code owner. A person or team responsible for reviewing changes to certain files.

Technically: A user or team named in a CODEOWNERS file who is automatically requested to review pull requests that change matching files. Example: @ada owns menu.md.

Related: review; pull request. First explained in Chapter 49.

Code review. Reading someone else's proposed change before it is merged.

Technically: The practice of examining a proposed change to detect defects, share knowledge and decide whether it is ready to be integrated. Example: A colleague comments on three lines of your pull request.

Related: pull request; approval. First explained in Chapter 46.

Code scanning. Automatic analysis of your code for weaknesses.

Technically: A GitHub feature that runs analysis tools such as CodeQL and shows alerts. Example: An alert on a pull request.

Related: CodeQL; check. First explained in Chapter 62.

CodeQL. GitHub's code analysis engine.

Technically: The analysis product maintained by GitHub that code scanning can use. Example: A CodeQL alert for a possible injection.

Related: code scanning. First explained in Chapter 62.

Codespace. A development environment that runs in the cloud and that you use through a browser or an editor.

Technically: A GitHub-hosted development environment in a Docker container on a virtual machine, created from a repository. Example: A codespace for the wordcount repository.

Related: dev container; container. First explained in Chapter 70.

Command. One instruction typed to the shell.

Technically: A line of input consisting of a program name followed by options and arguments, executed by the shell. Example: `ls -l notes.txt`

Related: option; argument; prompt. First explained in Chapter 7.

Commit. A recorded snapshot with author, time, message and a link to the previous commit; also the action of making one.

Technically: A Git object recording a tree, parent commits, author and committer identities, timestamps and a message. Example: `git commit -m "Add README"`

Related: snapshot; hash; branch. First explained in Chapter 15.

Commit status. A simple pass/fail label that a service attaches to a commit.

Technically: A state (such as success or failure) with a context name that an external service sets on a commit through GitHub's API. Example: An external build reporting success on the latest commit of a pull request.

Related: check; status check. First explained in Chapter 61.

CommonMark. A precise written specification of the core Markdown syntax.

Technically: A standardised specification of Markdown syntax intended to make different implementations behave consistently. Example: The syntax taught in the Markdown chapter.

Related: Markdown. First explained in Chapter 8.

Compare branch. The branch that holds the changes in a pull request.

Technically: The topic (head) branch of a pull request, whose commits are proposed for merging into the base branch. Example: `add-tea`

Related: pull request; base branch. First explained in Chapter 45.

Computer. A machine that follows instructions to take in, change, keep and show information.

Technically: A programmable electronic device that executes stored instructions to process data received as input, retain results in storage and present output. Example: A laptop, a phone, a cash machine.

Related: hardware; software; processor. First explained in Chapter 1.

Concurrency group. A name that lets only one run at a time use a shared resource.

Technically: A label under which the platform allows at most one workflow run or job to be running, with later runs waiting or earlier ones cancelled. Example: `validate-refs/heads/main`

Related: workflow (GitHub Actions). First explained in Chapter 59.

Configuration (setting). A named value that changes how Git behaves.

Technically: A key of the form `section.name` with a value, read from Git's configuration files, environment and command line. Example: `user.email`, `init.defaultBranch`

Related: scope; git config. First explained in Chapter 14.

Conflict markers. The lines Git writes into a conflicted file.

Technically: The <<<<<<, ===== and >>>>>> lines that delimit the two versions; all must be removed before committing. Example: <<<<<< HEAD

Related: merge conflict. First explained in Chapter [22](#).

Container. An isolated, packaged environment that runs on a host computer.

Technically: An isolated runtime environment with its own filesystem and tools, started from an image. Example: A Python environment in a container.

Related: Docker; dev container. First explained in Chapter [70](#).

Container image. A packaged program together with everything it needs to run.

Technically: A stored, layered file system and configuration from which containers are started. Example: An image in the container registry.

Related: container; Docker. First explained in Chapter [71](#).

Context. A collection of information that a workflow can read.

Technically: A named object, such as github, env, steps or runner, whose properties can be read in workflow expressions. Example: github.sha

Related: expression. First explained in Chapter [55](#).

Continuous deployment. Deploying every change that passes the checks, automatically.

Technically: A practice in which each change that passes automated checks is deployed to production without a manual step. Example: A merge to main that goes live after the tests pass.

Related: continuous integration; deployment. First explained in Chapter [72](#).

Continuous deployment (CD). Publishing software automatically after it passes its checks.

Technically: The practice of using automation to publish and deploy software updates, typically after the code has been built and tested automatically. Example: A merge to main publishes the website.

Related: continuous integration; pipeline. First explained in Chapter [52](#).

Continuous integration (CI). Checking every change automatically as soon as it is committed.

Technically: The practice of frequently committing code to a shared repository and automatically building and testing each change to detect errors early. Example: A server runs the tests on every pull request.

Related: continuous deployment; pipeline. First explained in Chapter [52](#).

Contributor. A person who sends a change or report to a project.

Technically: Someone who provides code, documentation, reports or other work to a project. Example: A stranger who fixes a typo through a pull request.

Related: maintainer; fork. First explained in Chapter [64](#).

Copyleft. A licence rule that what you build from the code must be shared under the same licence.

Technically: A licence condition requiring that distributed derivative works be licensed under the same or a compatible licence, with reach that varies (weak or strong). Example: The GNU General Public License.

Related: permissive licence. First explained in Chapter [65](#).

Copyright. The legal right of a creator over their work.

Technically: A legal right, differing by country, that lets the creator control copying, distribution and adaptation of a work. Example: The author of a program holds the copyright unless they transfer it.

Related: licence. First explained in Chapter [65](#).

Creative Commons. A family of licences for text, pictures and other non-software work.

Technically: Public licences built from elements such as Attribution, ShareAlike and NonCommercial. Example: CC-BY-4.0

Related: licence; SPDX identifier. First explained in Chapter [65](#).

Credential helper. A program that remembers passwords for Git.

Technically: A program that Git asks to store, return or delete the username and password or token for a remote address. Example: credential.helper

Related: remote. First explained in Chapter [32](#).

Cron. A compact way to write repeating times.

Technically: A five-field notation (minute, hour, day of month, month, day of week) with the operators *, comma, hyphen and slash that describes when a scheduled task runs. Example: 20 8 * * * means 08:20 every day.

Related: schedule; workflow (GitHub Actions). First explained in Chapter [56](#).

CSS. A text format that describes how web pages look.

Technically: Cascading Style Sheets: a language for describing the presentation of documents written in a markup language. Example: css/style.css

Related: HTML. First explained in Chapter [3](#).

Current directory (working directory). The folder that counts as 'here' when you give a relative path.

Technically: The directory against which a process resolves relative paths. Example: sunrise-bakery after you have moved into it.

Related: relative path; path. First explained in Chapter [2](#).

Custom property. A label with a value that you attach to repositories in an organization.

Technically: Organization-defined metadata on repositories that can be used, for example, to target rulesets. Example: criticality = high

Related: ruleset; organization. First explained in Chapter 75.

CVE. A public identification number for a known vulnerability.

Technically: Common Vulnerabilities and Exposures identifier assigned by a numbering authority. Example: An advisory that carries a CVE number.

Related: security advisory. First explained in Chapter 62.

D

Dangling object. A stored object nothing points to.

Technically: An object in the database that no ref leads to; removed by garbage collection eventually. Example: dangling commit ee9eec6

Related: reflog; garbage collection. First explained in Chapter 30.

Dependabot. GitHub's helper that warns about vulnerable dependencies and opens pull requests to update them.

Technically: A set of features: alerts, security updates and version updates. Example: A pull request that raises a package to a fixed version.

Related: dependency graph; pull request. First explained in Chapter 62.

Dependency graph. A list of the packages your project depends on.

Technically: GitHub's map of a repository's dependencies built from manifests and lock files. Example: Packages listed in a manifest file.

Related: Dependabot; manifest. First explained in Chapter 62.

Deployment. Putting a version of your software where its users can use it.

Technically: The process of releasing a build to an environment where it runs, such as a server or a hosting platform. Example: Copying a new site version to a web host.

Related: pipeline; environment. First explained in Chapter 72.

Detached HEAD. HEAD points directly at a commit instead of a branch.

Technically: A state in which HEAD contains a commit identifier rather than a symbolic reference to a branch. Example: HEAD detached at 9f10b43

Related: HEAD; branch. First explained in Chapter 20.

Dev container. A container set up with the tools a project needs.

Technically: A Docker container configured for development, described by devcontainer.json in a .devcontainer folder. Example: .devcontainer/devcontainer.json

Related: codespace; Dockerfile. First explained in Chapter 70.

Diff. A report of the differences between two versions, line by line.

Technically: A description of changes between two file or tree states, shown as removed (-) and added (+) lines in hunks. Example: `git diff`

Related: `hunk`; `git show`. First explained in Chapter 17.

Discussion. An open-ended conversation about a project, kept apart from the list of issues.

Technically: A hosting-platform forum thread, filed in a category, used for questions, ideas, announcements and polls that are not yet tracked work. Example: A Q&A thread asking how to install the tool.

Related: `issue`; `category`. First explained in Chapter 48.

Distributed version control. Every user holds a complete copy of the repository and its history.

Technically: A version-control architecture in which each participant has a full repository, and repositories exchange history peer to peer. Example: `Git`

Related: `centralised version control`; `clone`. First explained in Chapter 11.

Diverged. Each branch has commits the other lacks.

Technically: Two branches have diverged when neither tip is an ancestor of the other, so a fast-forward is impossible. Example: Hours added on `main` while `tea` is added on `add-tea`.

Related: `merge`; `fast-forward`. First explained in Chapter 21.

DNS (domain name system). The internet's phone book: it turns names into numbers.

Technically: A distributed naming system that resolves domain names to IP addresses and other records. Example: Turning `example.org` into an address.

Related: `domain name`; `IP address`. First explained in Chapter 5.

Docker. A popular tool for building and running containers.

Technically: A platform for building container images and running containers. Example: A `Dockerfile` builds an image.

Related: `container`; `Dockerfile`. First explained in Chapter 70.

Dockerfile. A text file with instructions for building a container image.

Technically: A text file of instructions such as `FROM`, `COPY` and `RUN` that create a Docker image. Example: `FROM` an existing image, then `RUN` an install command.

Related: `Docker`; `dev container`. First explained in Chapter 70.

Domain name. A human-friendly name, such as `example.org`, for a host on the internet.

Technically: A hierarchical name registered in the domain name system that resolves to network addresses. Example: `example.org`

Related: `DNS`; `URL`. First explained in Chapter 5.

E

Empty commit. A commit with a message but no file change.

Technically: A commit whose tree is identical to its parent's, created with `--allow-empty`.

Example: Mark the start of the spring menu

Related: commit. First explained in Chapter [19](#).

Encoding. The rule that turns characters into bytes and back.

Technically: A mapping between sequences of characters and sequences of bytes used to store or transmit text. Example: UTF-8

Related: Unicode; UTF-8; mojibake. First explained in Chapter [4](#).

Enterprise account. An account that manages policy and billing for many organizations.

Technically: An account type that centrally manages multiple organizations; it does not directly own repositories. Example: A large company with many organizations.

Related: organization. First explained in Chapter [68](#).

Environment. A named deployment target with its own rules and secrets.

Technically: A named target such as staging or production, configured with protection rules, secrets and variables, that a workflow job can reference. Example: github-pages

Related: deployment; reusable workflow. First explained in Chapter [58](#).

Environment variable. A variable that programs started from the shell can read.

Technically: A variable exported into the environment inherited by child processes. Example: HOME, PATH

Related: variable; PATH. First explained in Chapter [7](#).

Event. Something that happens in a repository and can start a workflow.

Technically: An activity in a repository, such as a push, pull request or issue, or a schedule or manual request, that triggers a workflow run. Example: push

Related: workflow. First explained in Chapter [54](#).

Expression. A small formula in a workflow that GitHub works out while running.

Technically: A construct written between `${{` and `}}` in a workflow file that GitHub evaluates using literals, operators, functions and contexts. Example: `${{ github.ref == 'refs/heads/main' }}`

Related: context; workflow (GitHub Actions). First explained in Chapter [55](#).

Extension. The letters after the last dot in a file name; a label for the kind of contents.

Technically: A suffix of a file name, conventionally indicating the file's format; it does not change the contents. Example: .html in menu.html

Related: file. First explained in Chapter [2](#).

F

Fast-forward. A merge that only moves a branch pointer forward.

Technically: A merge in which the receiving branch has not diverged, so Git moves its pointer to the source commit and creates no merge commit. Example: main moves to add-tea's commit.

Related: merge; merge commit. First explained in Chapter 21.

Fetch. Download new commits without changing your branches.

Technically: Retrieve objects and update remote-tracking branches, leaving local branches and the working tree untouched. Example: `git fetch`

Related: pull; remote-tracking branch. First explained in Chapter 23.

File. A named collection of information kept in storage.

Technically: A named sequence of bytes stored in a file system, with associated metadata.
Example: `menu.html`

Related: folder; extension; metadata. First explained in Chapter 2.

Folder (directory). A container that holds files and other folders.

Technically: A file-system object that contains references to files and other directories.
Example: `sunrise-bakery`

Related: file; path; root. First explained in Chapter 2.

Fork. Your own hosted copy of someone else's repository.

Technically: A copy of a hosted repository owned by another account, connected to the original (the upstream), used to propose changes back through pull requests without write access to the original. Example: Your copy of an open-source project on the platform.

Related: clone; upstream; pull request. First explained in Chapter 49.

G

Garbage collection. Git's housekeeping of stored objects.

Technically: `git gc` packs loose objects into packfiles and removes unreachable objects after a safety period. Example: `git gc`

Related: packfile; dangling object. First explained in Chapter 30.

Git. A free, open-source version-control system that runs on your computer.

Technically: A distributed version-control system that records the history of a project in a repository. Example: The program you use to record versions of `sunrise-bakery`.

Related: GitHub; repository. First explained in Chapter 12.

git add. Stages files so they are part of the next commit.

Technically: Updates the index with the current content of the given paths. Example: `git add notes.md`

Related: stage; index. First explained in Chapter 16.

Git Bash. A Bash environment for Windows that is installed with Git for Windows; it is not Git itself.

Technically: A terminal providing a Bash shell and Unix-style utilities on Windows, distributed with Git for Windows. Example: The window you open to type Git commands on Windows.

Related: Bash; Git; terminal. First explained in Chapter 7.

git branch. Lists, creates, renames and deletes branches.

Technically: Manages branch references under refs/heads. Example: `git branch add-tea`

Related: branch; git switch. First explained in Chapter 20.

git commit. Records the staged snapshot with a message.

Technically: Creates a commit object from the index and advances the current branch.

Example: `git commit -m "Add notes"`

Related: commit; stage. First explained in Chapter 16.

git commit --amend. Replaces the last commit with a new one, to fix its message or add forgotten changes.

Technically: Creates a new commit from the index and message that replaces the tip commit; the replaced commit is left unreferenced. Example: `git commit --amend --no-edit`

Related: commit; reflog. First explained in Chapter 19.

git config. The command that reads and changes Git's settings.

Technically: Reads, writes and lists Git configuration values across system, global, local and worktree scopes. Example: `git config user.email`

Related: configuration; scope. First explained in Chapter 13.

git init. Creates a new repository in the current folder.

Technically: Initialises an empty Git repository by creating the `.git` directory. Example: `git init`

Related: repository; git status. First explained in Chapter 16.

Git LFS. A way to keep large files outside the history.

Technically: An extension that stores large file contents on a separate server and leaves small pointer files in the repository. Example: `git lfs track "*.png"`

Related: object. First explained in Chapter 31.

git log. Shows the history of commits, newest first.

Technically: Lists commits reachable from the current revision in reverse chronological order, with options for format and filtering. Example: `git log --oneline`

Related: commit; revision. First explained in Chapter [17](#).

git mv. Renames or moves a file and stages the change.

Technically: Moves a file in the working tree and index in one operation. Example: `git mv menu.html carte.html`

Related: `git rm`; rename. First explained in Chapter [18](#).

git reset. Move the current branch to another commit.

Technically: Moves the current branch to a commit; `--soft` keeps index and working tree, the default resets the index, `--hard` resets both. Example: `git reset --soft HEAD~1`

Related: `git revert`; `reflog`. First explained in Chapter [25](#).

git restore. Bring a file back from the index or an older commit.

Technically: Restores content into the working tree or, with `--staged`, into the index, from the index or a named commit. Example: `git restore menu.md`

Related: `git reset`; index. First explained in Chapter [25](#).

git revert. Undo a commit by adding a new one.

Technically: Creates a new commit that applies the inverse of an earlier commit, leaving history intact. Example: `git revert --no-edit HEAD`

Related: `git reset`. First explained in Chapter [25](#).

git rm. Deletes a file and stages the deletion.

Technically: Removes a path from the working tree and the index; with `--cached` only from the index. Example: `git rm contact.html`

Related: `git mv`; `git restore`. First explained in Chapter [18](#).

git show. Displays one commit: its message and changes.

Technically: Shows a commit's metadata and diff (or other objects). Example: `git show HEAD~1`

Related: `git log`; diff. First explained in Chapter [17](#).

git status. Shows which files are untracked, modified or staged, and the current branch.

Technically: Reports the state of the working tree and index relative to HEAD; read-only. Example: `git status`

Related: working tree; index. First explained in Chapter [16](#).

git switch. Changes the current branch.

Technically: Moves HEAD to another branch and updates the index and working tree to match. Example: `git switch add-tea`

Related: branch; HEAD. First explained in Chapter [20](#).

GitHub. A widely used online platform for hosting Git repositories and collaborating on software.

Technically: A hosting platform built around Git repositories with collaboration and automation features. Example: github.com

Related: Git; hosting platform. First explained in Chapter [12](#).

GitHub App. An integration with its own limited permissions that acts on GitHub.

Technically: An integration that uses fine-grained permissions and short-lived tokens and is not tied to a user account. Example: An app that comments on pull requests.

Related: OAuth app; token. First explained in Chapter [74](#).

GitHub CLI. A command-line program, `gh`, for working with GitHub.

Technically: GitHub's official command-line tool for pull requests, issues, releases, workflow runs, codespaces and more. Example: `gh pr create`

Related: Git; command line. First explained in Chapter [73](#).

GitHub Pages. GitHub's service for publishing a static website from a repository.

Technically: A static site hosting service that takes files from a repository, optionally builds them, and publishes a website. Example: <https://ada.github.io/notes>

Related: static site; workflow. First explained in Chapter [69](#).

GitHub Sponsors. GitHub's way of supporting open-source maintainers with money.

Technically: A GitHub program through which people and organizations sponsor maintainers, with additional terms and regional eligibility for those who receive funds. Example: A sponsor button on a repository.

Related: open source; `FUNDING.yml`. First explained in Chapter [75](#).

GitLab. Another hosting platform for Git repositories.

Technically: A hosting platform for Git repositories with collaboration and automation features. Example: A team's self-hosted code server.

Related: GitHub; hosting platform. First explained in Chapter [12](#).

Global setting. A Git setting that applies to your user account in every repository.

Technically: A configuration value stored in the user's global file (`~/.gitconfig`) and applied to all repositories of that user. Example: `user.name` set with `--global`

Related: scope; configuration. First explained in Chapter [13](#).

Governance. The written rules by which a group runs a project.

Technically: The decisions, roles and rules of a project or organization, expressed on GitHub through permissions, rulesets, code owners and community files. Example: A `CONTRIBUTING` file and a ruleset.

Related: organization; code of conduct. First explained in Chapter [75](#).

GraphQL. An API where you send a query that names exactly the fields you want.

Technically: A query language for APIs served at one endpoint that returns the requested structure. Example: A query that asks for the logins of your followers.

Related: API; REST. First explained in Chapter [74](#).

H

Hardware. The physical parts of a computer.

Technically: The tangible components of a computing system, such as the processor, memory, storage devices, display and network adapter. Example: A solid-state drive.

Related: software; processor; memory; storage. First explained in Chapter [1](#).

Hash (commit identifier). The long code that uniquely identifies a commit.

Technically: A cryptographic hash of an object's content, used as its identifier; usually abbreviated. Example: c3c1c38

Related: commit; branch. First explained in Chapter [15](#).

HEAD. The pointer that says which commit you are currently on.

Technically: A reference, normally symbolic, to the current branch or commit; the parent of the next commit. Example: HEAD -> main

Related: branch; commit. First explained in Chapter [15](#).

Hidden file. A file that ordinary listings do not show unless asked.

Technically: A file whose name (on Unix-like systems, a leading dot) or attribute marks it to be omitted from default listings. Example: .git, .gitignore

Related: file; .git. First explained in Chapter [2](#).

History. The recorded sequence of versions of a project, with reasons and authors.

Technically: The ordered record of a project's versions together with metadata such as author, date and description. Example: Every recorded change to the bakery menu.

Related: version; version control. First explained in Chapter [9](#).

HMAC. A fingerprint that only someone with a secret key can produce.

Technically: A keyed hash message authentication code, here HMAC-SHA-256, used to check that a message is genuine and unaltered. Example: The X-Hub-Signature-256 header of a webhook delivery.

Related: webhook; checksum. First explained in Chapter [74](#).

Home folder. The folder that belongs to your user account.

Technically: The directory assigned to a user account for that user's files and settings; often written ~ in a shell. Example: /home/learner

Related: path; folder. First explained in Chapter 2.

Hook. A script Git runs at a fixed moment.

Technically: A script in `.git/hooks` that Git runs at a specific moment, such as before a commit; a failing exit status stops the operation. Example: `.git/hooks/pre-commit`

Related: configuration. First explained in Chapter 32.

Hosting platform. An online service that stores shared repositories and adds collaboration features.

Technically: A web service providing remote repository storage and collaboration tools such as issue tracking and review. Example: GitHub, GitLab, Bitbucket

Related: Git; remote. First explained in Chapter 12.

HTML. A text format that marks up web pages so a browser knows how to show them.

Technically: HyperText Markup Language: a markup language using tags to describe the structure of a web page. Example: `<h1>Sunrise Bakery</h1>`

Related: CSS; web page. First explained in Chapter 3.

HTTP. The rules web clients and servers follow to exchange requests and replies.

Technically: HyperText Transfer Protocol: an application-layer protocol of requests and responses carrying methods, status codes, headers and bodies. Example: Status 200 OK.

Related: HTTPS; server; client. First explained in Chapter 5.

HTTPS. HTTP with encryption and a check of the server's identity.

Technically: HTTP carried over an encrypted, authenticated TLS connection. Example: `https://example.org`

Related: HTTP; authentication. First explained in Chapter 5.

Hunk. One contiguous chunk of changes in a diff.

Technically: A group of adjacent changed lines with context in a unified diff, introduced by an `@@` header. Example: `@@ -1,4 +1,4 @@`

Related: diff. First explained in Chapter 17.

I

Ignored file. A file Git has been told to leave alone.

Technically: A file matching an ignore pattern that Git omits from untracked listings and from git add of directories. Example: `build.log`

Related: `.gitignore`; untracked file. First explained in Chapter 18.

Index (staging area). The holding area that lists exactly what the next commit will contain.

Technically: Git's staging area (index): a file in `.git` recording the content that will form the next commit. Example: Files after `git add`.

Related: stage; commit. First explained in Chapter [15](#).

Installer. A file or program that puts an application on your computer.

Technically: A program or package that copies an application's files to the correct locations and registers it with the operating system. Example: A Git setup file downloaded from the maker's site.

Related: package manager; administrator. First explained in Chapter [1](#).

Internet. The worldwide system of connected networks.

Technically: The global interconnection of networks that use common protocols to exchange data. Example: The connection between your laptop and a server in another country.

Related: web; network. First explained in Chapter [5](#).

IP address. A number that identifies a computer on a network.

Technically: A numeric identifier assigned to a network interface, used to route packets. Example: 127.0.0.1 means this computer.

Related: DNS; network. First explained in Chapter [5](#).

Issue. A numbered item where a problem, idea or task is written down and discussed.

Technically: A tracked item in a hosted repository with a title, description, discussion and open or closed state, identified by a number unique within the repository. Example: Issue #12: rolls price is wrong.

Related: pull request; label; milestone. First explained in Chapter [44](#).

Issue form. A form, defined in a file, that guides people to write a useful issue.

Technically: A YAML template stored under `.github/ISSUE_TEMPLATE` that defines input fields whose values are converted into a Markdown issue. Example: `.github/ISSUE_TEMPLATE/bug.yml`

Related: issue; YAML. First explained in Chapter [44](#).

Iteration. A block of time in which a team plans to finish some work.

Technically: A repeating, fixed-length period of planned work; in GitHub Projects, a field type that groups items by such periods. Example: A two-week iteration.

Related: project. First explained in Chapter [47](#).

J

Job. A group of steps that run together on one machine.

Technically: A set of steps in a workflow executed on the same runner; jobs run in parallel unless one declares a dependency on another. Example: check

Related: workflow; step; runner. First explained in Chapter [54](#).

K

Key pair. A shareable public key and a private key that never leaves your computer.

Technically: A matched public/private cryptographic key pair in which data signed or locked with one key is verified or unlocked with the other. Example: An SSH key pair.

Related: SSH; authentication. First explained in Chapter [6](#).

L

Least privilege. Giving only the access that is needed, and no more.

Technically: A design principle that grants a subject the minimum permissions required for its task. Example: A token that can read one project only.

Related: authorization; token. First explained in Chapter [6](#).

Licence. A written permission that says what others may do with a work.

Technically: A document in which the rights holder grants permissions, subject to conditions and with stated limitations. Example: A LICENSE file in the root of a repository.

Related: copyright; open source. First explained in Chapter [65](#).

Lightweight tag. A tag that is just a name.

Technically: A tag that is only a reference to a commit, with no tagger, date or message of its own. Example: `git tag v0.1`

Related: tag; annotated tag. First explained in Chapter [24](#).

Line ending (newline). The invisible character or characters that end a line of text.

Technically: A control character sequence marking a line break: LF on Unix-like systems, CRLF traditionally on Windows. Example: `\n` (LF) or `\r\n` (CRLF)

Related: plain text; whitespace. First explained in Chapter [4](#).

Local. On the computer you are using.

Technically: Residing on, or executing on, the machine at hand rather than reached over a network. Example: A file in your `sharif-git-lab` folder.

Related: remote. First explained in Chapter [5](#).

Local repository. The complete history, kept in the hidden `.git` folder on your computer.

Technically: The `.git` directory holding a project's objects, references and configuration. Example: `sunrise-bakery/.git`

Related: repository; working tree. First explained in Chapter [15](#).

Lock. A marker that stops others from changing a file until you release it.

Technically: A mechanism preventing concurrent modification of a resource by others.
Example: Checking out a file exclusively.

Related: merge. First explained in Chapter 10.

Lock file. A file that records the exact versions chosen.

Technically: A file generated by a package manager pinning resolved versions. Example: A file that fixes each package to one version.

Related: manifest. First explained in Chapter 62.

M

Machine user. A user account made for automation instead of a person.

Technically: A user account created to automate activity, such as continuous integration.
Example: An account a build system uses.

Related: user account; bot. First explained in Chapter 68.

Maintainer. A person who looks after a project and decides what is merged.

Technically: A person who reviews and merges changes, triages issues and manages releases of a project. Example: The person who approves a pull request.

Related: contributor; pull request. First explained in Chapter 64.

Manifest. A file that lists the packages a project needs.

Technically: A dependency definition file read by package managers and by GitHub's dependency graph. Example: A file that names packages and version ranges.

Related: lock file; dependency graph. First explained in Chapter 62.

Markdown. A way of writing formatted text using plain-text marks.

Technically: A lightweight markup language whose plain-text syntax is converted to structured documents such as HTML. Example: # Heading, emphasis

Related: renderer; CommonMark; plain text. First explained in Chapter 8.

Matrix. A way to run the same job in several configurations.

Technically: A job strategy that defines variables with lists of values and runs the job once for each combination. Example: Three versions on two systems gives six jobs.

Related: job; workflow (GitHub Actions). First explained in Chapter 56.

Memory (RAM). Fast, temporary working space that is emptied when the power goes off.

Technically: Random-access memory: volatile, fast storage used for the data and instructions a computer is currently using. Example: The words you have typed but not yet saved.

Related: storage; processor. First explained in Chapter 1.

Merge. To combine two sets of changes into one.

Technically: The operation of integrating changes from two lines of work into a single version.
Example: Joining Alice's cake addition and Bob's price change.

Related: branch; conflict. First explained in Chapter 10.

Merge commit. A commit that joins two lines of work.

Technically: A commit with two or more parents, recording where lines of work were combined.
Example: Merge branch 'add-tea'.

Related: merge; diverged. First explained in Chapter 21.

Merge conflict. Git cannot combine two changes by itself.

Technically: A situation in which both sides changed the same part of a file differently, so Git stops the merge and asks the user to decide. Example: Two branches set the white loaf price differently.

Related: merge; conflict markers. First explained in Chapter 22.

Metadata. Information about a file, such as its size, dates, owner and permissions.

Technically: Attributes a file system records about a file separately from the file's contents.
Example: The size and last-changed date shown in a detailed listing.

Related: file; permission. First explained in Chapter 2.

Mixed content. An HTTPS page that loads some of its parts over plain HTTP.

Technically: Assets such as images, style sheets or scripts referenced with http:// from a page served over HTTPS. Example: An image with a src that begins http://

Related: HTTPS; static site. First explained in Chapter 69.

mkdir. Creates a new folder.

Technically: Make directory: creates directories; -p also creates missing parent directories.
Example: mkdir -p sunrise-bakery/css

Related: cd; rmdir. First explained in Chapter 7.

Mojibake. Garbled text caused by reading with the wrong encoding.

Technically: Incorrect characters displayed when bytes are decoded using an encoding different from the one used to encode them. Example: cafÃ© instead of café

Related: encoding; UTF-8. First explained in Chapter 4.

N

Network. Computers connected so that they can exchange information.

Technically: A set of computing devices linked by communication channels. Example: Your home Wi-Fi network.

Related: internet. First explained in Chapter 5.

Notification. A message from a platform that something happened where you are subscribed.

Technically: An update delivered to a notifications inbox, a mobile app or email because of activity in a conversation or repository to which a user is subscribed. Example: A message that someone commented on your pull request.

Related: subscription; watching; inbox. First explained in Chapter [41](#).

O

OAuth app. An integration that acts with the permissions a user gives it.

Technically: An app that a user authorises to act on GitHub within granted scopes; generally less preferred than a GitHub App. Example: A third-party tool you sign in to with GitHub.

Related: GitHub App; token. First explained in Chapter [74](#).

Object. Any item in Git's database.

Technically: One of the four kinds of stored item in Git's object database: blob, tree, commit or tag, each named by a hash. Example: a commit

Related: blob; tree. First explained in Chapter [30](#).

OIDC. A way for a workflow to get a short-lived cloud credential instead of storing a long-lived secret.

Technically: OpenID Connect: the workflow requests a short-lived access token from a cloud provider that trusts GitHub. Example: A deployment job that receives a token valid for one job.

Related: secret; token. First explained in Chapter [60](#).

Open source. Software whose code is public and whose licence lets others use, change and share it.

Technically: Software distributed under a licence that grants permissions to use, study, modify and redistribute the source. Example: A library with a licence file that permits reuse.

Related: licence; public repository. First explained in Chapter [64](#).

Operating system (OS). The main software that manages the computer, its files and the other programs.

Technically: System software that manages hardware resources, provides services such as file storage and process scheduling, and hosts applications. Example: Windows, macOS, Linux.

Related: application; hardware; distribution. First explained in Chapter [1](#).

Option (flag). A word that changes how a command behaves.

Technically: A command-line parameter, usually prefixed with one or two dashes, that modifies a command's behaviour. Example: `-l` in `ls -l`

Related: command; argument. First explained in Chapter [7](#).

Organization. A shared GitHub account that a group of people use to own repositories together.

Technically: An account type that can own repositories, packages and projects; people access it through their own user accounts and hold roles in it. Example: A company account with several teams.

Related: team; enterprise account. First explained in Chapter [68](#).

Outside collaborator. Someone who is not a member of an organization but has access to some of its repositories.

Technically: A person given access to specific repositories of an organization without being a member. Example: A contractor.

Related: organization; base permission. First explained in Chapter [68](#).

Override. To replace a setting's value for one command or one shell.

Technically: To supply a value that takes precedence over stored configuration, for example with `git -c` or an environment variable. Example: `git -c commit.gpgsign=false commit`

Related: scope; configuration. First explained in Chapter [14](#).

P

Package. Software prepared so that a tool can install it.

Technically: A versioned, packaged unit of software distributed through a registry and installed by a package manager. Example: `@ada/wordcount` version 0.1.0

Related: registry; package manager. First explained in Chapter [71](#).

Package manager. A program that installs and updates applications from a trusted collection.

Technically: A tool that resolves, retrieves, installs, updates and removes software packages from configured repositories. Example: `apt` on Ubuntu.

Related: installer; application. First explained in Chapter [1](#).

Packfile. One file holding many objects compactly.

Technically: A file that stores many objects together, using differences between similar objects; created by `git gc` and by network transfers. Example: `.git/objects/pack`

Related: garbage collection; object. First explained in Chapter [30](#).

Pagination. Splitting a long list of results into pages.

Technically: Returning results in subsets with links (the `Link` header) to further pages. Example: 30 issues per page by default.

Related: API; rate limit. First explained in Chapter [74](#).

Password. A secret string you present to prove you own an account.

Technically: A memorised secret used as a credential in authentication. Example: A long passphrase of unrelated words.

Related: password manager; two-factor authentication. First explained in Chapter [6](#).

Password manager. An application that stores all your passwords securely and fills them in.

Technically: Software that keeps credentials in an encrypted vault protected by a master secret.
Example: An app protected by one strong master password.

Related: password. First explained in Chapter 6.

Patch. A text file that describes a change.

Technically: A text file describing changes to files so that they can be applied elsewhere, for example with git apply or git am. Example: 0001-Add-tea.patch

Related: diff; cherry-pick. First explained in Chapter 28.

Path. A written description of where a file or folder is in the tree.

Technically: A sequence of directory names, separated by a separator character, identifying a location in a file system. Example: css/style.css

Related: absolute path; relative path. First explained in Chapter 2.

PATH (environment variable). The list of folders the shell searches for programs.

Technically: An environment variable containing an ordered list of directories searched to resolve command names. Not to be confused with a file path. Example: A tool in a folder not on PATH gives 'command not found'.

Related: environment variable; command; path. First explained in Chapter 7.

Permission. A rule about who may read, change or run a file.

Technically: An access-control attribute governing read, write and execute rights of users on a file or directory. Example: -rw-r--r-- on a text file

Related: administrator; metadata. First explained in Chapter 2.

Permissive licence. A licence that asks only for credit and notices.

Technically: A licence whose main condition is preserving copyright and licence notices; derived works may use other terms. Example: MIT and BSD licences.

Related: copyleft; licence. First explained in Chapter 65.

Phishing. Tricking someone into giving away a secret by pretending to be trusted.

Technically: A social-engineering attack that impersonates a trusted party to obtain credentials or induce harmful actions. Example: An urgent email with a link to a lookalike site.

Related: password; account. First explained in Chapter 6.

Pipeline. The ordered steps a change goes through automatically.

Technically: An ordered sequence of automated stages, such as lint, test, build and deploy, in which a failure normally stops later stages. Example: Lint, then test, then build.

Related: continuous integration; exit code. First explained in Chapter 52.

Plain text. Information stored as ordinary characters with no fonts, colours or layout.

Technically: A sequence of encoded characters with no formatting metadata, stored as bytes according to a character encoding. Example: A .txt or .md file.

Related: text editor; binary file. First explained in Chapter 3.

Pre-release. A release marked as not yet final.

Technically: A release flagged as a pre-release, for example a beta. Example: v1.3.2 (beta)

Related: release. First explained in Chapter 66.

Processor (CPU). The part that carries out instructions.

Technically: The central processing unit: the hardware component that executes program instructions. Example: The chip that runs your browser's instructions.

Related: memory; hardware. First explained in Chapter 1.

Project. A table, board or roadmap for planning work across issues and pull requests.

Technically: A hosting-platform planning tool that presents issues, pull requests and draft items in table, board and roadmap layouts with custom fields, filters and automation. Example: A board with the columns Todo, In progress and Done.

Related: issue; pull request; iteration. First explained in Chapter 47.

Prompt. The text the shell shows when it is waiting for you to type.

Technically: A string printed by the shell to indicate readiness for input. Example: \$

Related: shell; command. First explained in Chapter 7.

Pull. Fetch, then merge into your current branch.

Technically: git fetch followed by integrating the upstream branch into the current branch, by default with a merge. Example: git pull

Related: fetch; merge. First explained in Chapter 23.

Pull request. A proposal to merge one branch into another, with a place to discuss it first.

Technically: A hosting-platform feature that records a request to merge a compare branch into a base branch and collects discussion, reviews and automated check results; it is not a Git object. Example: A pull request from add-tea into main.

Related: branch; merge; review. First explained in Chapter 45.

Push protection. A block that stops a push containing a secret.

Technically: A secret scanning feature that rejects pushes with detected secrets before they reach the repository. Example: A refused push with a message naming the secret type.

Related: secret scanning; secret. First explained in Chapter 62.

pwd. Prints the folder you are currently in.

Technically: Print working directory: outputs the absolute path of the current directory. Example: /home/learner

Related: current directory; cd. First explained in Chapter 7.

R

Rate limit. A cap on how many requests you may make in a period.

Technically: A limit on API requests within a time window, reported in response headers.

Example: The X-RateLimit-Remaining header.

Related: API. First explained in Chapter 74.

README. A file that introduces a project to the people who arrive at it.

Technically: A documentation file, conventionally named README.md, placed at the root (or in a .github or docs folder) of a repository and displayed on its front page by hosting platforms.

Example: README.md

Related: repository; markdown; topic. First explained in Chapter 42.

Rebase. Replay your commits on top of another commit.

Technically: Reapply the commits of a branch, one by one, on a new base, producing new commits with new hashes. Example: git rebase main

Related: merge; squash. First explained in Chapter 27.

Ref. A name that points to an object.

Technically: A named pointer, stored as a small file under .git/refs, holding a hash or another ref's name; branches and tags are refs. Example: refs/heads/main

Related: branch; tag. First explained in Chapter 30.

Reflog. Git's private diary of where HEAD has been.

Technically: A local, temporary log of the positions of HEAD and each branch, used to recover commits that are no longer reachable. Example: git reflog

Related: HEAD; git reset. First explained in Chapter 26.

Registry. A place that stores packages for download.

Technically: A service that hosts packages and serves them to package managers. Example: GitHub's npm registry.

Related: package; package manager. First explained in Chapter 71.

Relative path. A path that starts from where you are now.

Technically: A path interpreted relative to the current working directory. Example: ../images/logo.svg

Related: absolute path; current directory. First explained in Chapter 2.

Release. A named, downloadable version of a project on GitHub.

Technically: A GitHub page built on a Git tag, with a title, notes and attached files. Example: Release v1.2.0 with a zip file and a notes page.

Related: tag; release notes. First explained in Chapter [66](#).

Release notes. A description of what changed in a release.

Technically: Text attached to a release, written by hand or generated from merged pull requests. Example: A list of merged pull requests and contributors.

Related: release; changelog. First explained in Chapter [66](#).

Remote. On a different computer, reached over a network.

Technically: Residing on another machine, accessed through a network connection. Example: A page on example.org; later, a repository on GitHub.

Related: local; server. First explained in Chapter [5](#).

Remote-tracking branch. Your note of where a remote branch was when you last looked.

Technically: A read-only local reference such as origin/main that Git updates on fetch, pull and push. Example: origin/main

Related: remote; upstream branch. First explained in Chapter [23](#).

Renderer. A program that turns marked-up text into a formatted result.

Technically: Software that parses a markup language and produces a presentation, such as HTML, from it. Example: The preview in your editor.

Related: Markdown. First explained in Chapter [8](#).

Repository. A project together with the complete record of its history.

Technically: A store containing a project's files and the full history of changes to them; in Git, the project folder plus its .git directory. Example: The sunrise-bakery folder after Git is started in it.

Related: clone; version control. First explained in Chapter [11](#).

rerere. Git remembers how you resolved a conflict.

Technically: A feature (reuse recorded resolution) that records a conflict resolution and applies it when the same conflict recurs. Example: git config rerere.enabled true

Related: merge conflict. First explained in Chapter [32](#).

REST. A style of API where each kind of thing has an address and you ask with HTTP requests.

Technically: An API style organised by resources addressed with URLs and HTTP methods, returning structured responses. Example: GET /repos/OWNER/REPO/issues

Related: API; GraphQL. First explained in Chapter [74](#).

Reusable workflow. A workflow that other workflows can call as a whole.

Technically: A workflow whose on key includes `workflow_call`, so that a job in another workflow can run it using the `uses` keyword, passing inputs and secrets. Example: A shared deploy workflow.

Related: workflow (GitHub Actions); environment. First explained in Chapter 58.

Revision. Any way of naming a commit: a hash, a branch name, HEAD, HEAD~2.

Technically: A specifier that resolves to a commit (or other object), as defined by Git's revision syntax. Example: HEAD~1

Related: hash; HEAD. First explained in Chapter 17.

Rollback. Going back to the previous working version.

Technically: Restoring the previously deployed version after a failed or faulty deployment. Example: Pointing the current link back at the old release folder.

Related: deployment; release. First explained in Chapter 72.

Root. The top folder of a tree; everything else is inside it.

Technically: The topmost directory of a file-system hierarchy, written `/` on Unix-like systems. Example: `/`

Related: path; folder. First explained in Chapter 2.

Ruleset. A named list of rules that a platform enforces on branches or tags.

Technically: A set of rules applied by a hosting platform to selected branches or tags, layered with other rulesets so that the most restrictive version of each rule applies. Example: A ruleset that blocks force pushes to release branches.

Related: branch protection rule. First explained in Chapter 50.

Runner. The machine that runs a job.

Technically: A virtual machine or container, hosted by the platform or by the user, that executes the jobs of a workflow. Example: `ubuntu-latest`

Related: job; workflow. First explained in Chapter 54.

S

Scope. The level at which a setting is stored: system, global, local or worktree.

Technically: The configuration file level (system, global, local, worktree) at which a value is stored, determining its reach and precedence. Example: A local value beats a global one in that repository.

Related: configuration; override. First explained in Chapter 14.

Script injection. Someone else's text becoming part of your script and being run.

Technically: An attack in which untrusted input is substituted into the text of a script so that it is interpreted as commands. Example: A pull request title that closes a quotation and adds a command.

Related: least privilege; workflow. First explained in [Chapter 60](#).

Secret. A value that gives access if someone else has it.

Technically: A password, token, API key, private key or file containing them, which gives access to a system when held by someone else. Example: `API_KEY=...`

Related: credential helper. First explained in [Chapter 33](#).

Security advisory. A published notice about a vulnerability and its fix.

Technically: A report describing a vulnerability, affected versions and fixed versions, drafted privately and then published. Example: An advisory with a CVE number.

Related: CVE; SECURITY.md. First explained in [Chapter 62](#).

Self-hosted runner. A machine you run yourself to do the jobs of a workflow.

Technically: A machine deployed and maintained by the user that executes GitHub Actions jobs instead of a machine provided by the platform. Example: A server in your own office.

Related: runner; security. First explained in [Chapter 59](#).

Semantic versioning. A convention for MAJOR.MINOR.PATCH release numbers.

Technically: A versioning convention in which MAJOR changes for incompatible changes, MINOR for compatible additions and PATCH for compatible fixes; a hyphenated label marks a pre-release. Example: `v1.4.2`

Related: tag. First explained in [Chapter 29](#).

Server. A computer or program that answers requests.

Technically: A process (or the computer running it) that listens for and responds to requests from clients. Example: The program that sends you a web page.

Related: client; remote. First explained in [Chapter 5](#).

Shallow clone. A clone with only the recent history.

Technically: A clone limited to a given number of recent commits, with older history cut off (grafted) until fetched. Example: `git clone --depth 1`

Related: clone. First explained in [Chapter 31](#).

Shell. The program that reads your commands and has them carried out.

Technically: A command interpreter that reads input, parses commands and asks the operating system to execute them. Example: Bash, zsh, PowerShell.

Related: terminal; command. First explained in [Chapter 7](#).

Sign-off. A line in a commit message saying the author has the right to submit the change.

Technically: A Signed-off-by trailer added with `git commit -s`; it is text, not a cryptographic signature. Example: Signed-off-by: Ada Learner <ada@example.org>

Related: signed commit; trailer. First explained in [Chapter 64](#).

Signed commit. A commit that carries a verifiable signature.

Technically: A commit carrying a cryptographic signature made with the author's private key, which others can check against the public key. Example: `git commit -S`

Related: commit. First explained in Chapter 33.

Single point of failure. A part whose failure stops everything or loses something irreplaceable.

Technically: A component in a system whose failure causes the entire system to fail. Example: A single central server with no backup.

Related: centralised version control. First explained in Chapter 11.

Snapshot. A complete picture of all the files of a project at one moment.

Technically: The state of the tracked files at a point in time, as recorded by a commit's tree. Example: The whole bakery site as it was when you committed.

Related: commit; working tree. First explained in Chapter 15.

Software. Instructions that tell hardware what to do.

Technically: Programs and data that direct the operation of hardware. Example: A web browser.

Related: application; operating system; hardware. First explained in Chapter 1.

Software project. The folder of files that makes up one piece of work.

Technically: A set of files, typically under one directory, that together form a program, website, document set or tool. Example: sunrise-bakery

Related: source code; folder. First explained in Chapter 3.

Source archive. A zip or tar file of the files at a tag, without history.

Technically: An archive of a tagged commit's tree, made by GitHub for each release or locally with `git archive`. Example: `proj-1.0.0.tar.gz`

Related: tag; release. First explained in Chapter 66.

Source code (source files). The human-written files from which a project is made.

Technically: Human-readable files that specify a program, website or document set and from which deliverables are produced. Example: `index.html`, `style.css`

Related: software project. First explained in Chapter 3.

SPDX identifier. A short standard name for a licence.

Technically: A short identifier from the SPDX licence list, such as MIT or Apache-2.0. Example: CC-BY-NC-SA-4.0

Related: licence. First explained in Chapter 65.

Squash. Turn many commits into one.

Technically: Combine the changes of several commits into a single new commit, leaving no link to the originals. Example: `git merge --squash add-juice`.

Related: merge commit. First explained in Chapter [21](#).

SSH (Secure Shell). A secure way to connect to another computer, which can use a key pair to prove your identity.

Technically: A cryptographic network protocol for secure remote login and command execution, supporting public-key authentication. Example: Connecting to a server without sending a password.

Related: key pair; remote. First explained in Chapter [6](#).

Stage. To add a file's current contents to the index so the next commit includes them.

Technically: To record a file's current state in the index (`git add`). Example: `git add menu.md`

Related: index; commit. First explained in Chapter [15](#).

Staging. A practice copy of production where you deploy first.

Technically: An environment that mirrors production and is used to check a release before it goes live. Example: A staging site that the team can test.

Related: environment; deployment. First explained in Chapter [72](#).

Stash. A shelf for unfinished changes.

Technically: A stack of saved working-tree and index states that lets you return to a clean tree and reapply the changes later. Example: `git stash`

Related: working tree. First explained in Chapter [24](#).

Static site. A website made of files that are sent to the browser exactly as they are.

Technically: A site consisting of HTML, CSS and JavaScript files served without server-side code generating each page. Example: A bakery page with a few HTML files.

Related: GitHub Pages; server. First explained in Chapter [69](#).

Step. One task inside a job.

Technically: A single command (run) or action (uses) executed in order within a job, in its own process. Example: `run: sh ci.sh`

Related: job; action. First explained in Chapter [54](#).

Storage. Slower, permanent space that keeps information when the power is off.

Technically: Non-volatile data storage, such as a solid-state drive or hard disk, that retains data without power. Example: A saved document.

Related: memory; file. First explained in Chapter [1](#).

Submodule. A repository pinned inside another repository.

Technically: A repository at a fixed path in another repository; the outer one records one commit of the inner one (mode 160000), described in `.gitmodules`. Example: `vendor/lib`

Related: repository. First explained in Chapter 31.

Subscription. An arrangement to be told about activity in one conversation or repository.

Technically: A recorded preference that causes a platform to send notifications about a specific conversation or repository to a user until the user unsubscribes. Example: Commenting on an issue subscribes you to it.

Related: notification; watching. First explained in Chapter 41.

Supply chain. Everything your software is made from and built with.

Technically: The code, dependencies, tools, build systems and people involved in producing software. Example: A dependency that an attacker changes.

Related: dependency graph; Dependabot. First explained in Chapter 63.

Symbolic link. A file that points to another file or folder.

Technically: A file-system entry that names another path; reading through it reads the target. Example: `current` pointing to `releases/v1`

Related: path; deployment. First explained in Chapter 72.

Synchronised (cloud) folder. A folder whose contents are kept the same across computers or an online service.

Technically: A directory whose contents are replicated automatically between devices or a service. Example: A folder that appears on your laptop and phone.

Related: backup; version control. First explained in Chapter 9.

Synopsis. The short line showing how to write a command.

Technically: The compact notation at the start of a command's usage or manual page that shows its options and arguments, with brackets for optional parts and angle brackets for placeholders. Example: `git tag -d <tagname>...`

Related: command; option. First explained in Chapter 35.

Syntax highlighting. Colouring parts of text according to their role so the structure is easy to see.

Technically: Display of tokens of structured text in different colours or styles according to their syntactic category. Example: Tags in HTML shown in one colour and words in another.

Related: code editor. First explained in Chapter 3.

T

Tag. A fixed name for one commit.

Technically: A reference that names a commit and, unlike a branch, does not move when new commits are made. Example: `v1.0`

Related: branch; annotated tag. First explained in Chapter [24](#).

Tag object. The object an annotated tag creates.

Technically: An object naming another object with a type, tag name, tagger and message.

Example: `git cat-file -p v1.0`

Related: annotated tag; ref. First explained in Chapter [30](#).

Team. A named group of people in an organization that shares access and mentions.

Technically: A group of organization members used to grant repository access and to mention people together; teams can be nested. Example: A documentation team.

Related: organization. First explained in Chapter [68](#).

Telemetry. Usage information a program sends back to its maker.

Technically: Data about how software is used, collected and sent by the software, which may be inspected or disabled where the software allows. Example: `GH_TELEMETRY=log` prints what `gh` would send.

Related: GitHub CLI; privacy. First explained in Chapter [73](#).

Terminal. A window in which you type commands and read text replies.

Technically: A text-based interface (terminal emulator) through which a user interacts with a shell. Example: The Terminal application or Git Bash window.

Related: shell; prompt; command. First explained in Chapter [7](#).

Text editor. A program for creating and changing plain text files.

Technically: An application that edits plain text without adding formatting information.

Example: Any editor that saves only characters.

Related: code editor; word processor; plain text. First explained in Chapter [3](#).

Token. A limited stand-in for a password; still a secret.

Technically: A credential string that grants specified, often time-limited, permissions to its holder. Example: A personal access token used by a tool.

Related: password; least privilege. First explained in Chapter [6](#).

Topic. A short label that helps people find a repository.

Technically: A lowercase label attached to a hosted repository to classify its purpose, subject area, community or language, and to enable search and browsing by that label. Example: `static-site`

Related: README; repository. First explained in Chapter [42](#).

touch. Creates an empty file, or updates a file's last-changed time.

Technically: Creates an empty file if it does not exist, or updates its timestamps. Example: `touch .hidden-note`

Related: file. First explained in Chapter 7.

Tracked file. A file Git knows about, because it is in the last commit or in the index.

Technically: A file present in the index or in HEAD's tree. Example: README.md after the first commit.

Related: untracked file; staged. First explained in Chapter 15.

Tree. A stored list of a folder's contents.

Technically: An object listing the entries of one folder, each with a mode, type, hash and name. Example: drinks/

Related: blob; commit. First explained in Chapter 30.

Two-factor authentication (2FA). Proving who you are with two proofs from different families.

Technically: Authentication requiring two distinct factors: something known, something held, or something inherent. Example: Password plus a code from a phone app.

Related: authentication; password. First explained in Chapter 6.

U

Unicode. A universal list that gives every character a number.

Technically: A standard that assigns a unique code point to each character of the world's writing systems and many symbols. Example: U+00E9 is é

Related: UTF-8; encoding. First explained in Chapter 4.

Upstream branch. The remote branch a local branch follows.

Technically: The branch named by branch.<name>.remote and branch.<name>.merge, used by plain push, pull and status. Example: main follows origin/main

Related: remote-tracking branch. First explained in Chapter 23.

URL. The written address of a page or resource on the internet.

Technically: Uniform Resource Locator: a string that identifies a resource by scheme, host, optional port, path, optional query and optional fragment. Example: https://example.org/menu/cakes?sort=price#top

Related: domain name; HTTP. First explained in Chapter 5.

Username. The name that identifies your account.

Technically: A unique identifier for an account within a service. Example: sharif-example

Related: account. First explained in Chapter 6.

UTF-8. The most widely used way to store Unicode text as bytes.

Technically: A variable-width encoding of Unicode that uses one to four bytes per character and is backward compatible with ASCII for basic English characters. Example: é is stored as the two bytes c3 a9.

Related: Unicode; encoding. First explained in Chapter 4.

V

Variable. A named piece of text the shell remembers.

Technically: A named storage location in a shell holding a string value that can be referenced in later commands. Example: name="Sunrise Bakery"

Related: environment variable. First explained in Chapter 7.

Version. One particular state of a file or project at a point in time.

Technically: A distinct recorded state of a file or project. Example: The menu as it was on Monday.

Related: history; version control. First explained in Chapter 9.

Version control. A system that records a project's history so changes can be seen, reversed and combined.

Technically: A system that records changes to files over time, supporting retrieval of earlier versions, comparison, and collaborative work. Example: Git

Related: history; repository. First explained in Chapter 9.

W

Web (World Wide Web). Linked pages and applications you view with a browser.

Technically: A system of interlinked resources, identified by URLs and transferred using HTTP, accessed with browsers over the internet. Example: The Sunrise Bakery menu page.

Related: internet; browser; URL. First explained in Chapter 5.

Webhook. A message GitHub sends to your web address when something happens.

Technically: An HTTP request with event data that a service sends to a URL you registered when an event you subscribed to occurs. Example: A push triggering a request to your server.

Related: API; HMAC. First explained in Chapter 74.

Wiki. A section of a repository for long documentation, kept as its own Git repository.

Technically: A documentation area attached to a hosted repository, stored as a separate Git repository (addressed with a .wiki.git suffix on GitHub) with its own history. Example: bakery-menu.wiki.git

Related: README; repository. First explained in Chapter 43.

Word processor. A program for writing formatted documents, saved in a packaged format.

Technically: An application that edits formatted documents, storing styles and layout along with the text, normally in a binary or packaged format. Example: A program that produces .docx files.

Related: text editor; binary file. First explained in Chapter 3.

Workflow. The agreed way a team uses Git.

Technically: The agreed way a person or team uses branches, commits, reviews, merges and releases to get changes into a project. Example: feature-branch workflow

Related: branch; merge. First explained in Chapter 34.

Workflow (GitHub Actions). An automated process defined in a file in the repository.

Technically: A configurable automated process defined by a YAML file in .github/workflows that runs one or more jobs when triggered by an event. Example: .github/workflows/check.yml

Related: event; job; step; runner. First explained in Chapter 54.

Working tree. The folder of files you see and edit.

Technically: The checked-out files of a repository on disk, including changes not yet staged or committed. Example: The sunrise-bakery folder.

Related: index; repository. First explained in Chapter 15.

Worktree. An extra working folder for the same repository.

Technically: An additional working directory linked to one repository, with its own checked-out branch and shared history. Example: git worktree add ../hotfix -b hotfix

Related: branch; working tree. First explained in Chapter 31.

Y

YAML. A readable text format for settings and lists, written with indentation.

Technically: A human-readable data-serialisation format that represents scalars, sequences and mappings using indentation (spaces only) and simple punctuation; used for configuration such as GitHub Actions workflows. Example: name: Sunrise Bakery

Related: workflow; indentation. First explained in Chapter 53.

Z

ZIP archive. One file that holds other files in compressed form.

Technically: A container format that stores multiple files, optionally compressed, in a single file. Example: Many word-processor documents are ZIP archives internally.

Related: binary file. First explained in Chapter 3.

Index of terms

Each entry lists the chapters that mention the term, with page numbers. Command names are indexed in Appendix A.

.gitignore 26, 193, 265, 367, 392, 414, 423, 602, 664, 676
Absolute path 26, 76, 616
Access token 405, 555, 575, 629, 641, 648
Account 15, 26, 57, 67, 76, 115, 121, 128, 136, 253
Action 15, 26, 47, 76, 115, 157, 392, 429, 443, 477
Administrator 15, 26, 136, 145, 477, 489, 568, 610
Alias 76, 145, 356, 616, 641
Annotated tag 265, 323, 331, 423, 676, 721
Apex domain 616
API 193, 323, 367, 392, 436, 471, 514, 528, 549, 562
Application (program) 15
Argument 76, 180, 219, 555, 691
Artifact 528, 542, 575, 616, 629, 635
Asset 37, 595
Atomic commit 208
Attestation 575
Authentication 67, 399, 405, 436, 482, 575, 610, 622, 629, 641
Authorization 67
Backup 67, 107, 121, 193, 253, 376, 392, 691, 705
Bare repository 128, 253, 323, 343, 376, 414, 443, 456, 482, 489
Base branch 456, 482, 489
Base permission 610
Bash 15, 26, 47, 76, 93, 107, 136, 145, 157, 168
Binary file 37, 47
Bisect 297, 311, 343, 664, 715, 721
Bitbucket 128
Blob 276, 331, 343, 555
Branch 37, 128, 136, 145, 157, 168, 180, 193, 208, 219
Branch protection rule 489, 676
Browser 15, 37, 57, 128, 157, 384, 405, 414, 423, 429
Byte 15, 47, 331
cd 76, 107, 145, 157, 168, 180, 193, 208, 219, 232
Centralised version control 121
Character 37, 47, 67, 180, 193, 208, 219, 399, 482, 555
Check 15, 26, 37, 47, 57, 67, 76, 93, 115, 121
Checksum 331, 575, 595, 641, 676
Cherry-pick 297, 311, 376, 664
Client 57, 121, 429, 489, 648
Clone 121, 193, 253, 289, 343, 356, 367, 376, 392, 414
Code editor 37, 47
Code of conduct 436, 477, 482, 582, 602, 656
Code owner 482
Code review 465
Code scanning 465, 562, 568

CodeQL 568
Codespace 622, 641
Command 15, 26, 37, 47, 76, 93, 107, 128, 136, 145
Commit 67, 115, 128, 136, 145, 157, 168, 180, 193, 208
Commit status 562
CommonMark 93
Compare branch 456, 465
Computer 15, 26, 37, 47, 57, 67, 76, 93, 107, 115
Concurrency group 549
Configuration (setting) 145
Conflict markers 243, 721
Container 26, 37, 514, 549, 622, 629, 635
Container image 622, 629
Context 180, 521, 528, 536, 555
Continuous deployment 502, 635
Continuous deployment (CD) 502
Continuous integration (CI) 502
Contributor 193, 449, 456, 465, 477, 482, 495, 528, 582, 622
Copyleft 588
Copyright 582, 588, 602
Creative Commons 588
Credential helper 136, 145, 356, 405, 691
Cron 528
CSS 26, 37, 76, 168, 193, 208, 502, 542, 616, 635
Current directory (working directory) 26
Custom property 656
CVE 343, 568
Dangling object 331
Dependabot 528, 555, 568, 575, 629, 676
Dependency graph 495, 568, 575
Deployment 128, 443, 502, 542, 549, 562, 575, 610, 635
Detached HEAD 219, 289, 297, 691
Dev container 622
Diff 107, 180, 193, 208, 232, 243, 253, 265, 276, 297
Discussion 128, 208, 477, 582, 705
Distributed version control 121
Diverged 232, 243, 253, 297, 456, 691
DNS (domain name system) 57
Docker 622, 629, 635
Dockerfile 622
Domain name 57, 616
Empty commit 168, 208, 276, 495
Encoding 37, 47
Enterprise account 610
Environment 15, 26, 37, 47, 76, 93, 128, 136, 145, 180
Environment variable 76, 145, 180, 514, 521, 528, 555, 641
Event 289, 429, 502, 508, 514, 521, 528, 536, 555, 562
Expression 521, 528, 536, 555
Extension 26, 37, 93, 157, 343, 449, 514, 641, 715
Fast-forward 232, 243, 253, 297, 376, 456, 482, 489, 664, 691
Fetch 57, 128, 157, 253, 276, 297, 343, 376, 392, 405

File [15](#), [26](#), [37](#), [47](#), [57](#), [67](#), [76](#), [93](#), [107](#), [115](#)
Folder (directory) [26](#)
Fork [297](#), [376](#), [399](#), [443](#), [456](#), [482](#), [489](#), [495](#), [528](#), [555](#)
Garbage collection [289](#), [331](#), [715](#)
Git [15](#), [26](#), [37](#), [47](#), [57](#), [67](#), [76](#), [93](#), [107](#), [115](#)
git add [145](#), [157](#), [168](#), [180](#), [193](#), [208](#), [219](#), [232](#), [243](#), [253](#)
Git Bash [15](#), [26](#), [76](#), [136](#), [145](#)
git branch [168](#), [219](#), [232](#), [253](#), [289](#), [343](#), [376](#), [392](#), [414](#), [423](#)
git commit [136](#), [145](#), [157](#), [168](#), [180](#), [193](#), [208](#), [219](#), [232](#), [243](#)
git commit --amend [208](#), [289](#)
git config [136](#), [145](#), [157](#), [168](#), [193](#), [219](#), [232](#), [243](#), [253](#), [297](#)
git init [128](#), [145](#), [157](#), [168](#), [180](#), [193](#), [253](#), [331](#), [343](#), [414](#)
Git LFS [343](#), [595](#), [691](#)
git log [145](#), [157](#), [168](#), [180](#), [193](#), [208](#), [219](#), [232](#), [243](#), [253](#)
git mv [193](#), [436](#)
git reset [243](#), [276](#), [289](#), [331](#), [356](#), [443](#), [664](#), [691](#), [705](#)
git restore [157](#), [168](#), [193](#), [219](#), [265](#), [276](#), [297](#), [311](#), [384](#)
git revert [276](#), [297](#), [367](#), [456](#), [664](#), [676](#), [705](#)
git rm [157](#), [168](#), [193](#), [343](#), [367](#), [664](#), [676](#), [691](#), [705](#)
git show [37](#), [180](#), [193](#), [208](#), [232](#), [265](#), [276](#), [297](#), [323](#), [331](#)
git status [93](#), [136](#), [145](#), [157](#), [168](#), [180](#), [193](#), [208](#), [219](#), [232](#)
git switch [157](#), [219](#), [232](#), [243](#), [276](#), [289](#), [297](#), [311](#), [331](#), [356](#)
GitHub [37](#), [57](#), [67](#), [93](#), [121](#), [128](#), [136](#), [157](#), [193](#), [253](#)
GitHub App [575](#), [648](#)
GitHub CLI [405](#), [482](#), [528](#), [536](#), [641](#), [648](#)
GitHub Pages [57](#), [443](#), [542](#), [616](#)
GitHub Sponsors [656](#)
GitLab [128](#)
Global setting [136](#)
Governance [648](#), [656](#)
GraphQL [528](#), [648](#)
Hardware [15](#), [57](#), [405](#), [549](#), [622](#)
Hash (commit identifier) [157](#)
HEAD [67](#), [136](#), [145](#), [157](#), [168](#), [180](#), [193](#), [208](#), [219](#), [232](#)
Hidden file [26](#)
History [15](#), [26](#), [37](#), [47](#), [57](#), [107](#), [115](#), [121](#), [128](#), [136](#)
HMAC [648](#), [721](#)
Home folder [26](#), [76](#), [145](#), [168](#), [356](#), [536](#), [629](#), [641](#), [691](#), [721](#)
Hook [356](#), [367](#), [502](#)
Hosting platform [115](#), [128](#), [232](#), [243](#), [253](#), [323](#), [343](#), [356](#), [367](#), [376](#)
HTML [26](#), [37](#), [47](#), [57](#), [93](#), [157](#), [168](#), [180](#), [193](#), [219](#)
HTTP [57](#), [93](#), [343](#), [436](#), [616](#), [648](#), [691](#)
HTTPS [37](#), [57](#), [67](#), [93](#), [343](#), [356](#), [392](#), [405](#), [423](#), [443](#)
Hunk [180](#), [208](#)
Ignored file [193](#)
Index (staging area) [157](#)
Installer [15](#), [76](#), [136](#), [356](#)
Internet [37](#), [47](#), [57](#), [67](#), [107](#), [121](#), [128](#), [136](#), [145](#), [253](#)
IP address [57](#)
Issue [128](#), [367](#), [392](#), [399](#), [429](#), [449](#), [456](#), [465](#), [471](#), [477](#)
Issue form [449](#)

Iteration 471

Job 15, 57, 67, 115, 145, 243, 311, 376, 392, 399

Key pair 67, 392, 405

Least privilege 67, 482, 555, 568, 575, 635, 641, 656

Licence 93, 392, 414, 436, 568, 582, 588, 595, 602, 629

Lightweight tag 265, 323, 331

Line ending (newline) 47

Local 37, 57, 121, 128, 145, 157, 193, 219, 253, 276

Local repository 157, 343, 414, 443, 715

Lock 115, 219, 367, 405, 477, 489, 562, 568, 629, 691

Lock file 568, 629, 691

Machine user 610

Maintainer 208, 429, 449, 456, 465, 477, 482, 582, 664, 676

Manifest 136, 392, 568

Markdown 26, 37, 47, 76, 93, 436, 449, 616

Matrix 514, 521, 528, 549

Memory (RAM) 15

Merge 115, 128, 136, 145, 168, 180, 208, 219, 232, 243

Merge commit 232, 243, 253, 276, 297, 343, 414, 456, 482, 495

Merge conflict 243, 297, 311, 356, 414, 536, 664, 676, 691, 705

Metadata 26, 449, 471, 489, 629, 656

Mixed content 616

mkdir 76, 107, 145, 157, 168, 180, 193, 253, 265, 331

Mojibake 47

Network 15, 26, 57, 67, 107, 115, 121, 128, 253, 331

Notification 429

OAuth app 648

Object 67, 157, 265, 276, 323, 331, 343, 384, 456, 521

OIDC 555

Open source 376, 384, 482, 521, 575, 582, 588, 641

Operating system (OS) 15

Option (flag) 76

Organization 121, 128, 136, 193, 367, 376, 392, 399, 405, 414

Outside collaborator 610

Override 136, 145, 253, 367

Package 15, 37, 76, 136, 193, 367, 542, 568, 575, 595

Package manager 15, 136, 602, 629

Packfile 331

Pagination 648

Password 67, 136, 180, 193, 356, 367, 399, 405, 414, 436

Password manager 67, 405

Patch 180, 297, 311, 323, 568, 595, 664

Path 15, 26, 37, 57, 76, 93, 115, 136, 145, 157

PATH (environment variable) 76

Permission 15, 26, 67, 76, 136, 145, 208, 465, 477, 482

Permissive licence 588

Phishing 67

Pipeline 502, 514, 542, 562, 635, 676

Plain text 26, 37, 47, 67, 76, 93, 107, 193, 356, 405

Pre-release 323, 595

Processor (CPU) 15

Project [26](#), [37](#), [47](#), [57](#), [67](#), [76](#), [93](#), [107](#), [115](#), [121](#)
Prompt [76](#), [136](#), [145](#), [208](#), [405](#), [691](#)
Pull [93](#), [128](#), [145](#), [157](#), [253](#), [297](#), [343](#), [367](#), [376](#), [384](#)
Pull request [128](#), [376](#), [392](#), [399](#), [429](#), [436](#), [443](#), [449](#), [456](#), [465](#)
Push protection [367](#), [568](#), [575](#), [676](#), [691](#), [705](#)
pwd [76](#), [168](#), [343](#), [691](#)
Rate limit [648](#)
README [37](#), [76](#), [93](#), [145](#), [157](#), [168](#), [193](#), [392](#), [399](#), [414](#)
Rebase [115](#), [145](#), [208](#), [232](#), [243](#), [253](#), [276](#), [289](#), [297](#), [311](#)
Ref [15](#), [26](#), [37](#), [47](#), [57](#), [67](#), [76](#), [93](#), [107](#), [115](#)
Reflog [208](#), [219](#), [265](#), [276](#), [289](#), [297](#), [311](#), [331](#), [367](#), [384](#)
Registry [629](#), [635](#)
Relative path [26](#), [37](#), [76](#)
Release [47](#), [67](#), [115](#), [136](#), [180](#), [193](#), [219](#), [265](#), [323](#), [343](#)
Release notes [219](#), [343](#), [384](#), [595](#), [676](#), [721](#)
Remote [57](#), [121](#), [145](#), [157](#), [253](#), [265](#), [297](#), [323](#), [331](#), [343](#)
Remote-tracking branch [157](#), [253](#), [297](#), [664](#), [676](#)
Renderer [93](#)
Repository [15](#), [37](#), [67](#), [76](#), [93](#), [115](#), [121](#), [128](#), [136](#), [145](#)
rerere [356](#)
REST [26](#), [37](#), [76](#), [107](#), [157](#), [168](#), [180](#), [208](#), [219](#), [243](#)
Reusable workflow [542](#)
Revision [107](#), [115](#), [136](#), [157](#), [180](#), [367](#), [508](#), [691](#)
Rollback [635](#)
Root [26](#), [145](#), [157](#), [168](#), [180](#), [193](#), [253](#), [331](#), [399](#), [414](#)
Ruleset [489](#), [656](#), [676](#)
Runner [47](#), [76](#), [128](#), [136](#), [157](#), [180](#), [193](#), [208](#), [219](#), [343](#)
Scope [145](#), [465](#), [528](#), [549](#), [622](#), [629](#)
Script injection [555](#)
Secret [47](#), [67](#), [157](#), [193](#), [265](#), [276](#), [289](#), [331](#), [367](#), [405](#)
Security advisory [568](#), [595](#)
Self-hosted runner [549](#), [555](#)
Semantic versioning [323](#), [595](#), [602](#)
Server [37](#), [47](#), [57](#), [67](#), [115](#), [121](#), [128](#), [145](#), [323](#), [343](#)
Shallow clone [343](#), [715](#)
Shell [15](#), [47](#), [67](#), [76](#), [136](#), [145](#), [219](#), [356](#), [405](#), [436](#)
Sign-off [582](#), [588](#), [664](#)
Signed commit [367](#), [405](#), [582](#)
Single point of failure [121](#)
Snapshot [115](#), [121](#), [157](#), [168](#), [208](#), [331](#), [423](#), [482](#)
Software [15](#), [37](#), [57](#), [107](#), [115](#), [128](#), [136](#), [392](#), [414](#), [502](#)
Software project [37](#), [648](#)
Source archive [595](#), [676](#)
Source code (source files) [37](#)
SPDX identifier [588](#)
Squash [232](#), [297](#), [343](#), [456](#), [664](#), [676](#), [691](#)
SSH (Secure Shell) [67](#)
Stage [15](#), [115](#), [157](#), [168](#), [180](#), [193](#), [208](#), [243](#), [297](#), [331](#)
Staging [157](#), [168](#), [193](#), [208](#), [243](#), [276](#), [542](#), [635](#)
Stash [180](#), [219](#), [232](#), [243](#), [253](#), [265](#), [276](#), [289](#), [323](#), [343](#)
Static site [616](#), [635](#)

Step [15](#), [26](#), [67](#), [76](#), [107](#), [115](#), [145](#), [157](#), [168](#), [180](#)
Storage [15](#), [26](#), [37](#), [265](#), [331](#), [343](#), [549](#), [610](#)
Submodule [343](#), [715](#)
Subscription [429](#), [656](#)
Supply chain [568](#), [575](#)
Symbolic link [542](#), [635](#)
Synchronised (cloud) folder [107](#)
Synopsis [384](#)
Syntax highlighting [37](#)
Tag [93](#), [115](#), [157](#), [265](#), [323](#), [331](#), [367](#), [376](#), [384](#), [392](#)
Tag object [323](#), [331](#)
Team [47](#), [67](#), [115](#), [121](#), [128](#), [145](#), [168](#), [193](#), [208](#), [219](#)
Telemetry [641](#)
Terminal [15](#), [26](#), [37](#), [47](#), [57](#), [67](#), [76](#), [93](#), [107](#), [128](#)
Text editor [37](#), [47](#), [93](#), [136](#), [168](#)
Token [67](#), [193](#), [356](#), [367](#), [384](#), [405](#), [436](#), [528](#), [536](#), [542](#)
Topic [219](#), [297](#), [376](#), [429](#), [436](#), [456](#), [477](#), [482](#), [495](#), [582](#)
touch [15](#), [26](#), [76](#), [107](#), [115](#), [297](#), [367](#), [562](#), [691](#)
Tracked file [157](#), [193](#), [208](#), [265](#), [568](#), [629](#), [691](#)
Tree [15](#), [26](#), [76](#), [145](#), [157](#), [168](#), [180](#), [193](#), [219](#), [243](#)
Two-factor authentication (2FA) [67](#)
Unicode [47](#)
Upstream branch [253](#), [414](#)
URL [57](#), [93](#), [145](#), [253](#), [343](#), [356](#), [392](#), [405](#), [482](#), [528](#)
Username [67](#), [356](#), [399](#), [405](#), [429](#), [443](#), [482](#), [656](#)
UTF-8 [37](#), [47](#), [343](#), [648](#)
Variable [76](#), [145](#), [180](#), [297](#), [465](#), [514](#), [521](#), [528](#), [536](#), [549](#)
Version [15](#), [26](#), [37](#), [47](#), [57](#), [76](#), [93](#), [107](#), [115](#), [121](#)
Version control [93](#), [107](#), [115](#), [121](#), [128](#), [157](#), [168](#)
Web (World Wide Web) [57](#)
Webhook [562](#), [648](#), [721](#)
Wiki [423](#), [436](#), [443](#)
Word processor [37](#), [93](#)
Workflow [128](#), [168](#), [376](#), [423](#), [443](#), [456](#), [471](#), [482](#), [489](#), [502](#)
Workflow (GitHub Actions) [514](#)
Working tree [145](#), [157](#), [168](#), [180](#), [193](#), [219](#), [243](#), [253](#), [265](#), [276](#)
Worktree [145](#), [219](#), [343](#)
YAML [449](#), [502](#), [508](#), [514](#), [521](#), [528](#), [536](#), [542](#), [549](#), [555](#)
ZIP archive [37](#), [47](#)

Index of commands

Commands that appear in code in the chapters, with the chapters where they appear (page numbers). Appendix A explains each command.

- gh alias** [641](#)
- gh api** [528](#), [641](#)
- gh auth** [405](#), [641](#)
- gh codespace** [641](#)
- gh config** [641](#)
- gh issue** [528](#), [641](#)
- gh pr** [641](#)
- gh release** [595](#), [641](#)
- gh repo** [641](#)
- gh run** [641](#)
- gh search** [641](#)
- gh status** [641](#)
- gh workflow** [641](#)
- git add** [145](#), [157](#), [168](#), [180](#), [193](#), [208](#), [219](#), [232](#)
- git am** [311](#)
- git apply** [311](#)
- git archive** [331](#), [423](#), [595](#), [676](#)
- git bisect** [311](#), [715](#)
- git blame** [180](#), [265](#), [343](#), [423](#)
- git branch** [168](#), [219](#), [232](#), [253](#), [289](#), [343](#), [376](#), [392](#)
- git cat-file** [157](#), [331](#), [343](#)
- git check-attr** [356](#)
- git check-ignore** [193](#), [367](#)
- git check-ref-format** [219](#), [555](#)
- git checkout** [219](#), [676](#), [691](#)
- git cherry-pick** [311](#), [664](#)
- git clean** [265](#), [691](#)
- git clone** [253](#), [343](#), [356](#), [392](#), [414](#), [423](#), [443](#), [456](#)
- git commit** [136](#), [145](#), [157](#), [168](#), [180](#), [193](#), [208](#), [219](#)
- git config** [136](#), [145](#), [157](#), [168](#), [193](#), [219](#), [232](#), [243](#)
- git count-objects** [331](#)
- git credential** [356](#)
- git describe** [323](#), [595](#)
- git diff** [180](#), [208](#), [232](#), [243](#), [253](#), [276](#), [297](#), [311](#)
- git fetch** [157](#), [253](#), [297](#), [343](#), [376](#), [392](#), [414](#), [456](#)
- git filter-branch** [367](#)
- git filter-repo** [367](#), [536](#), [705](#)
- git for-each-ref** [521](#)
- git format-patch** [311](#)
- git fsck** [276](#), [331](#), [715](#)
- git gc** [289](#), [331](#), [705](#)
- git grep** [265](#), [616](#), [629](#)
- git hash-object** [331](#)
- git help** [136](#), [219](#), [356](#), [384](#), [721](#)

git init 128, 145, 157, 168, 180, 193, 253, 331
git interpret-trailers 536
git lfs 343
git log 145, 157, 168, 180, 193, 208, 219, 232
git ls-files 157, 168, 193, 331, 343, 356, 414, 449
git ls-remote 323, 331, 414, 456, 489, 555, 664, 676
git ls-tree 331, 343, 423
git maintenance 356
git merge 136, 232, 243, 276, 297, 311, 356, 376
git merge-base 456
git mv 193, 436
git pull 145, 157, 253, 297, 376, 392, 414, 691
git push 157, 253, 297, 323, 367, 376, 392, 414
git rebase 289, 297, 311, 456, 705
git reflog 208, 276, 289, 664, 705
git remote 253, 323, 392, 405, 414, 482, 691
git rerere 356
git reset 243, 276, 289, 331, 356, 443, 664, 691
git restore 157, 168, 193, 219, 265, 276, 297, 311
git rev-list 495, 715
git rev-parse 180, 331, 343, 414, 521, 536, 691
git revert 276, 297, 367, 456, 664, 676, 705
git rm 157, 168, 193, 343, 367, 664, 676, 691
git send-email 311
git shortlog 465, 495, 676
git show 180, 193, 208, 232, 265, 276, 297, 323
git show-ref 331
git sparse-checkout 343
git stash 219, 265, 276, 289, 664, 691
git status 93, 136, 145, 157, 168, 180, 193, 208
git submodule 343, 715
git subtree 343
git switch 157, 219, 232, 243, 276, 289, 297, 311
git symbolic-ref 521, 676
git tag 265, 323, 331, 384, 423, 595, 602, 664
git update-ref 456
git verify-commit 405
git version 136, 384
git worktree 343

Author and Publisher

The author

Sharif Issah Tingane is the founder of SHARIF TECHNOLOGIES, the name under which Sharif builds and publishes software and books from Wenchi, Ghana. The SHARIF TECHNOLOGIES website lists custom web and mobile software, cybersecurity, networking and ICT training.

Public projects include SAIBA, an AI assistant for business customer support; Sharif NOVA, an open-source programming language and toolchain, still at an early version number; and CodeCast, an Android app that its repository describes as turning a codebase into a video tutorial. Sharif's public ORCID record lists interests in software development, artificial intelligence, cybersecurity, networking and developer tools, and an education entry in General Science at Techiman Senior High School, Ghana.

This book follows one habit: every command was run and recorded, each claim is tied to its source, and what could not be checked is written down. It was written with the help of an AI assistant, Claude, as the copyright page explains.

The imprint

SHARIF TECHNOLOGIES is the name under which Sharif Issah Tingane writes and publishes software and books. This book is published under the SHARIF TECHNOLOGIES imprint. The name is not licensed by the book's licences.

Knowledge Is Power

Website: www.shariftechnologies.online

AI assistance

This book was written with the help of an AI assistant, Claude, made by Anthropic, working under the author's direction. Appendix N and the Preface say what the assistant did.

Final competency checklist

You have finished the book when you can tick every line. The assessment in Chapter 81 checks many of them for you.

- [] I can explain what a repository, a commit, a branch and a remote are.
- [] I can make a change on a branch, publish it and have it reviewed and merged.
- [] I can resolve a conflict by reading both sides.
- [] I can undo a mistake safely, and tell when a fix would be dangerous.
- [] I can recover lost commits with the reflog.
- [] I can keep secrets out of a repository, and I know what to do first if one leaks.
- [] I can write a small workflow that runs tests, with least privilege and pinned actions.
- [] I can tag and describe a release.
- [] I know the difference between a licence and copyright, and I know that choosing a licence is my decision.

- [] I read the current documentation before I rely on a fact about a platform.